Do notebook à nuvem

DevOps e AWS para devs full stack

Claude

Do notebook à nuvem: DevOps e AWS para devs full stack

Texto escrito por Claude, um modelo de IA da Anthropic.

Preços, limites e versões de serviços da AWS mudam com frequência. Confira a documentação oficial antes de tomar decisões com base neles.

Edição gerada em 06/11/2026.

Sumário

Parte I: Fundamentos de DevOps para desenvolvedores

1 **Do notebook para o mundo**

O link que não abriu
Localhost: o endereço que nunca sai de casa
Endereços IP: o CEP de cada máquina
Portas: o apartamento dentro do prédio
TCP e UDP: como a conversa acontece
Roteamento: de salto em salto até o destino
IP público, IP privado e o truque do NAT
CIDR e subnets: fatiando endereços
Na prática: seguindo os pacotes do Rangô
Armadilhas e casos reais
Resumo
Próxima parada: um nome e um cadeado
Referências

2 **DNS, HTTPS e TLS**

Quatro números e um aviso vermelho
DNS: a agenda de contatos da internet
HTTP por baixo dos panos
TLS: o cadeado
CDN e cache na borda
Na prática: um nome e um cadeado para o Rangô local
Armadilhas e casos reais
Resumo
Próxima parada: o servidor que ninguém configurou
Referências

3 **Servidores, containers e Kubernetes**

Sexta-feira, 19h, primeiro deploy
O que é um servidor, afinal
O mínimo de Linux para rodar uma aplicação
Proxy reverso: quem atende a porta 443
Voltando ao "na minha máquina funciona"
Containers
Na prática: o Rangô num container
Quando um servidor vira dez: orquestração
Kubernetes em quatro conceitos
Armadilhas e casos reais
Resumo

Ponte: o dia em que o Rangô apareceu na TV
Referências

4 **Arquitetura que escala**

Quinta-feira, 21h14
Primeiro, as contas
Para cima ou para os lados
O balanceador de carga
O segredo que o código guardava
Cache: a resposta mais barata é a que você não calcula
Na prática: três APIs, um Nginx e um Redis
Monolito, microsserviços e serverless
Os gargalos que aparecem depois
Armadilhas e casos reais
Resumo
Ponte: o botão que trava tudo
Referências

5 **Filas, eventos e sistemas distribuídos**

Vinte e cinco segundos para dizer "recebido"
O que a API precisa esperar?
Filas: a comanda presa no trilho
Pub/sub: um evento, vários interessados
Pelo menos uma vez
Idempotência: repetir sem estragar
Tentar de novo, mas com calma
A mensagem envenenada
Consistência eventual
As oito falácias
Na prática: o pedido sai da requisição
Armadilhas e casos reais
Resumo
Ponte: três da manhã
Referências

6 **Produção de verdade**

Sábado, 3h40
Backup só conta depois do primeiro restore
RPO e RTO: quanto perder e quanto esperar
Réplicas: primária, leitura e failover
Alta disponibilidade e recuperação de desastres
Segredos fora do código
Na prática: backup, desastre e restore testado
Infraestrutura como código
CI/CD: a esteira entre o commit e a produção

Estratégias de deploy: como trocar o motor com o carro andando
Observabilidade: saber antes do cliente
Incidentes e postmortems
Armadilhas e casos reais
Resumo
Ponte: hora de alugar a infraestrutura
Referências

Parte II: AWS na prática

7 Bem-vindo à AWS

Onde o Rangô vai morar
O que a nuvem vende
O mapa da Parte II
Quem cuida de quê
Regiões e zonas de disponibilidade
Na prática: a conta do Rangô
Console, CLI e CloudShell
Armadilhas e casos reais
Resumo
Ponte: a sugestão perigosa
Referências

8 IAM: quem pode fazer o quê

Dez segundos no GitHub
Toda chamada é uma pergunta
O JSON por dentro
Usuários, grupos e roles
Menor privilégio sem paralisia
Por que nunca usar o root
IAM Identity Center: um login para gente
Na prática: o Rangô sem nenhuma chave permanente
Armadilhas e casos reais
Resumo
Para onde vamos
Referências

9 EC2: seu primeiro servidor

O notebook que precisa dormir
O que a AWS aluga quando você pede um servidor
Na prática: o Rangô no EC2
Armadilhas e casos reais
Resumo
O próximo passo, e o perigo que vem com ele
Referências

10 **VPC: a rede na AWS**
A porta 5432 aberta para o mundo
Uma rede só sua dentro da AWS
Na prática: a rede do Rangô
Armadilhas e casos reais
Resumo
As fotos que não cabem no disco
Referências

11 **S3: armazenamento de objetos**
Trezentas e quarenta fotos num disco de 10 GiB
Um dicionário gigante acessado por HTTP
Classes de armazenamento: a despensa e o depósito
Lifecycle: regras que trabalham enquanto você dorme
Versionamento: o git log do bucket
Quem pode ler o bucket
Presigned URLs: um ingresso com hora marcada
Na prática: as fotos do Rangô no S3
Armadilhas e casos reais
Resumo
O banco que ninguém quer acordar para cuidar
Referências

12 **RDS: banco de dados gerenciado**
Duas e quarenta e sete da manhã
O que o RDS assume por você
Cache gerenciado: ElastiCache
Na prática: o Rangô ganha um banco de verdade
Armadilhas e casos reais
Resumo
Nenhum dado mora mais na API
Referências

13 **Load Balancer e Auto Scaling**
O plano das dez instâncias
As peças da AWS
O que ainda guarda estado
Na prática: o Rangô atrás de um balanceador
Armadilhas e casos reais
Resumo
Um endereço que ninguém vai digitar
Referências

14 **Route 53, CloudFront e certificados**
Um endereço para a TV

Os três serviços
Na prática: rango.com.br no ar
Armadilhas e casos reais
Resumo
Arqueologia
Referências

15 **Infraestrutura como código com CDK**
Homologação para quinta-feira
A infraestrutura como uma migration
Por que não escrever o YAML direto
Na prática: o Rangô inteiro em Python
Armadilhas e casos reais
Resumo
O que ainda está escrito num arquivo
Referências

16 **Segredos e criptografia**
Onde mora a senha do banco
Configuração, segredo e chave
KMS: quem guarda a chave
Na prática: o Rangô sem senha em arquivo
Armadilhas e casos reais
Resumo
Uma imagem que envelhece
Referências

17 **Containers com ECR, ECS e Fargate**
Uma linha, uma hora e quarenta
Três serviços para uma imagem
Na prática: o Rangô em containers
E o Kubernetes?
Armadilhas e casos reais
Resumo
Sexta-feira, 18h
Referências

18 **CI/CD na AWS**
Sexta-feira, 18h07
O pipeline completo
Na prática
E o CodePipeline?
Armadilhas e casos reais
Resumo
Três vezes por dia

Referências

19 **Serverless com Lambda e API Gateway**

Seis minutos de trabalho, 24 horas de conta
Código que só existe quando é chamado
A porta HTTP da função
Na prática
Quando serverless vale a pena, e quando não
Armadilhas e casos reais
Resumo
Entregadores a cada cinco segundos
Referências

20 **DynamoDB**

Quarenta coordenadas por segundo
Um dicionário do tamanho que você precisar
Começar pelas perguntas
Na prática
Quando não usar o DynamoDB
Armadilhas e casos reais
Resumo
O pedido que ninguém enxerga inteiro
Referências

21 **SQS, SNS, EventBridge e Step Functions**

Terça, 20h40, sem gás na cozinha
Quatro serviços, quatro trabalhos
Na prática
Idempotência, peça por peça
Armadilhas e casos reais
Resumo
Dez lugares para procurar
Referências

22 **Observabilidade com CloudWatch**

Sexta, 21h10, "cadê o meu pedido?"
Um serviço, várias perguntas
Na prática
Armadilhas e casos reais
Resumo
Quem apagou a fila?
Referências

23 **Segurança em produção**

Segunda-feira, 16 de novembro, 8h40
CloudTrail: o registro de quem chamou a API

O robô no login: AWS WAF
Organizações com várias contas
SCPs: o teto das permissões
Os logs num lugar que ninguém apaga
GuardDuty: quem olha os logs às 3h
Config e Security Hub: o que está configurado errado agora
AWS Backup: o backup que o administrador não apaga
Quando acontecer de verdade
Armadilhas e casos reais
Resumo
A fatura de dezembro
Referências

24 **Custos e arquitetura**

Segunda-feira, 4 de janeiro
Como a AWS cobra
Cost Explorer: a fatura aberta em fatias
Tags de custo: quem gastou o quê
Avisos que chegam antes da fatura
Os vilões clássicos
Pagar menos pelo que fica ligado
O Rangô contra o Well-Architected
Para onde ir daqui
Armadilhas e casos reais
Resumo
O fim do livro, o começo da operação
Referências

PARTE I

Fundamentos de DevOps para desenvolvedores

CAPÍTULO 1

Do notebook para o mundo

Este livro foi escrito para quem já constrói aplicações completas, do banco de dados ao botão na tela, mas sempre teve alguém cuidando de onde elas rodam. Esse alguém costuma ser um colega de DevOps que ninguém nunca viu almoçar, ou uma plataforma mágica com um botão roxo escrito "Deploy". Você sabe escrever uma API, sabe montar um front em React, sabe depurar uma query lenta. O que acontece entre o `git push` e o usuário abrindo o app no celular continua meio nebuloso. Talvez você tenha uma teoria, algo como "o código sobe para a nuvem". A nuvem, como diz a camiseta, é só o computador de outra pessoa. Este livro ensina a alugar o computador da Amazon sem passar vergonha e sem vender um rim para pagar a fatura. É esse trecho que vamos percorrer juntos.

O livro tem duas partes. Na Parte I, que começa agora, você vai ver os conceitos de infraestrutura e DevOps sem nenhum provedor de nuvem no meio: redes, DNS, servidores, containers, escala, filas, backups, deploy. A ideia é montar o vocabulário e entender o porquê de cada peça antes que alguém cobre por hora pelo seu aprendizado. Na Parte II, a partir do capítulo 7, cada um desses conceitos volta, agora implementado na AWS e com profundidade.

Para não ficar só na teoria, vamos acompanhar um único projeto do começo ao fim: o **Rangô**, um app de pedidos de comida. O cliente escolhe um restaurante, monta o pedido, paga e acompanha a entrega. Hoje ele vive inteiro no seu notebook, junto com quarenta abas do Chrome e uma pasta chamada `projetos-novos-FINAL-2`. No último capítulo, ele estará em produção na AWS, aguentando tráfego, com backups, monitoramento e uma fatura que não assusta ninguém. Sim, isso existe. Eu também não acreditava.

Conhecendo o Rangô

O Rangô tem um backend em **FastAPI** e um frontend em React, criado com Vite. Os dados ficam num PostgreSQL rodando localmente.

Se você vem do Flask, o FastAPI vai parecer familiar logo de cara, como reencontrar um amigo de faculdade que agora faz CrossFit: mesmo rosto, mais disciplina. As rotas também são funções decoradas com `@app.get` ou `@app.post`. A diferença principal é que ele usa as anotações de tipo do Python para validar a entrada e gerar a documentação da

API automaticamente, e aceita funções `async` sem cerimônia. Quem roda a aplicação é o **Uvicorn**, um servidor ASGI (o equivalente assíncrono do WSGI que o Gunicorn usa com Flask e Django). O nome Gunicorn vem de *Green Unicorn*, o unicórnio verde. A comunidade Python tem uma relação com cavalos de chifre que ninguém explica e todo mundo respeita.

Por enquanto, o backend do Rangô é bem pequeno. Ele tem uma rota de saúde e uma lista fixa de restaurantes, guardada em memória enquanto o banco não entra em cena. Dois restaurantes. É menos do que a praça de alimentação de um shopping do interior, mas é um começo:

```python
# backend/app/main.py
import os

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="Rangô")

origens = os.getenv("CORS_ORIGINS", "http://localhost:5173").split(",")
app.add_middleware(
    CORSMiddleware,
    allow_origins=origens,
    allow_methods=["GET"],
    allow_headers=["*"],
)

RESTAURANTES = [
    {"id": 1, "nome": "Cantina da Nona", "cozinha": "italiana"},
    {"id": 2, "nome": "Sabor do Sertão", "cozinha": "nordestina"},
]


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/restaurantes")
def listar_restaurantes():
    return RESTAURANTES
```

O frontend busca essa lista e mostra na tela:

```tsx
// frontend/src/App.tsx
import { useEffect, useState } from "react";

type Restaurante = { id: number; nome: string; cozinha: string };

const API_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

export default function App() {
  const [restaurantes, setRestaurantes] = useState<Restaurante[]>([]);

  useEffect(() => {
```

```
    fetch(`${API_URL}/restaurantes`)
      .then((resposta) => resposta.json())
      .then(setRestaurantes);
  }, []);

  return (
    <main>
      <h1>Rangô</h1>
      <ul>
        {restaurantes.map((r) => (
          <li key={r.id}>
            {r.nome} ({r.cozinha})
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Para rodar, o projeto pede Python 3.12 ou mais recente e Node.js 20 ou mais recente. Ele sobe em dois terminais:

```
# Terminal 1: backend
cd backend
python -m venv .venv
source .venv/bin/activate        # no Windows: .venv\Scripts\activate
pip install "fastapi[standard]"
uvicorn app.main:app --reload --port 8000
```

```
# Terminal 2: frontend
cd frontend
npm install
npm run dev
```

Com os dois de pé, o navegador abre `http://localhost:5173` e os dois restaurantes aparecem. Funciona na sua máquina. Essa frase, aliás, já encerrou mais reuniões técnicas do que qualquer argumento racional, e boa parte da indústria de DevOps existe para que ela pare de ser dita. Os problemas começam quando outra pessoa tenta abrir.

O link que não abriu

A sua sócia no Rangô mora em Recife. Ela cuida da parte comercial e quer ver o app antes de mostrar para os primeiros restaurantes parceiros. Você, animado como quem acha que já terminou, manda uma mensagem: "Está rodando, abre aí: http://localhost:5173".

Dois minutos depois, chega a resposta com um print. O navegador dela mostra "Não é possível acessar esse site. A conexão com localhost foi recusada." Logo abaixo, ela acrescenta: "É assim mesmo?". Não é.

Você pensa um pouco e lembra que `localhost` é a sua máquina. Abre o terminal, descobre que o seu notebook tem o endereço `192.168.0.12` e manda de novo, agora com a confiança de quem nunca estudou redes: "Tenta http://192.168.0.12:5173". Desta vez o navegador dela fica girando por quase meio minuto até desistir com um erro de tempo esgotado. A sua sócia começa a se perguntar se escolheu bem o sócio.

Repare que os dois erros são diferentes. No primeiro, alguém respondeu "não" na hora. No segundo, ninguém respondeu nada, como mensagem de WhatsApp que fica com um tique cinza só, para sempre. Essa diferença, que parece detalhe, diz muito sobre o que está acontecendo, e você vai aprender a ler esse tipo de sinal ao longo deste capítulo.

A pergunta que fica é simples de fazer e trabalhosa de responder: o que precisa acontecer para que uma requisição saia do celular da sua sócia, em Recife, atravesse o país e chegue ao processo do Uvicorn rodando no seu notebook? Para responder, vamos seguir o caminho dessa requisição peça por peça.

Localhost: o endereço que nunca sai de casa

Quando você digita `localhost`, o sistema operacional traduz esse nome para `127.0.0.1` (ou `::1`, a versão em IPv6). Esse endereço pertence a uma interface de rede especial chamada **loopback**. Ela existe só em software, e tudo o que entra nela volta para a própria máquina. Nenhum pacote enviado para `127.0.0.1` passa pelo Wi-Fi ou pelo cabo. Existe uma camiseta clássica de evento de tecnologia com a frase "There's no place like 127.0.0.1". Se você riu, acaba de perder pontos em qualquer festa.

Por isso o primeiro link falhou de forma tão rápida. O navegador da sua sócia tentou se conectar ao `localhost` dela, ou seja, ao celular dela. Lá não havia nada escutando na porta 5173, e o próprio sistema do celular respondeu na hora: conexão recusada.

Toda máquina tem o seu próprio `localhost`. Mandar esse endereço para alguém é como dizer "passa aqui em casa" sem dizer onde fica a casa. Ou como o seu tio, que manda no grupo da família um print da tela do Wi-Fi achando que está compartilhando a internet.

Endereços IP: o CEP de cada máquina

Para uma máquina encontrar outra, cada uma precisa de um endereço. Na internet, esse endereço é o **endereço IP** (de *Internet Protocol*). Na versão mais usada, o **IPv4**, ele tem 32 bits, escritos como quatro números de 0 a 255 separados por pontos: `192.168.0.12`, `200.147.67.142`, `8.8.8.8`.

Um detalhe que costuma surpreender: o IP pertence a uma **interface de rede**, e uma máquina pode ter várias. O seu notebook tem pelo menos duas: a loopback, com `127.0.0.1`, e a placa de Wi-Fi, com algo como `192.168.0.12`. Se você conectar um cabo de rede, ganha uma terceira, com outro IP.

Com 32 bits, cabem cerca de 4,3 bilhões de endereços IPv4. Parecia muito nos anos 1980, a mesma década que achou que ombreira era uma boa ideia. Hoje existem mais celulares do que isso, e os blocos livres de IPv4 já se esgotaram nos registros regionais que distribuem esses endereços. A resposta de longo prazo é o **IPv6**, com 128 bits e endereços como `2804:14c:65:8a00::1`. Ele é o protocolo do futuro desde os anos 1990 e, pelo visto, gosta do cargo. Já carrega boa parte do tráfego no Brasil, mas o IPv4 continua presente em quase tudo, e foram as gambiarras para fazer o IPv4 durar que criaram metade dos conceitos deste capítulo. A internet é, em grande parte, um puxadinho muito bem documentado em RFCs. Vamos chegar nelas.

Portas: o apartamento dentro do prédio

O IP leva a requisição até a máquina certa. Só que no seu notebook rodam dezenas de programas que usam a rede ao mesmo tempo: o Uvicorn, o Vite, o PostgreSQL, o navegador, o Slack (que, sozinho, usa mais memória do que o Rangô inteiro vai usar em produção). Quando um pacote chega, como o sistema operacional sabe para qual deles entregar?

É para isso que existem as **portas**. Uma porta é um número de 0 a 65535 que identifica um processo escutando conexões naquela máquina. Se o IP é o endereço do prédio, a porta é o número do apartamento.

Essa analogia pode ir um passo além, com algo que você já conhece. Numa URL como `http://192.168.0.12:8000/restaurantes`, cada pedaço resolve um nível:

- o IP (`192.168.0.12`) escolhe a máquina;
- a porta (`8000`) escolhe o processo dentro dela, no caso o Uvicorn;
- o caminho (`/restaurantes`) escolhe a função dentro do processo, a que tem o decorator `@app.get("/restaurantes")`.

O roteamento do FastAPI faz, dentro da aplicação, o que o IP e a porta fazem na rede.

Algumas portas viraram convenção. HTTP usa a 80, HTTPS usa a 443, SSH usa a 22 e o PostgreSQL usa a 5432. Quando você digita `https://google.com` sem porta, o navegador assume a 443. As portas abaixo de 1024 são chamadas de **privilegiadas**: em Linux, um processo comum precisa de permissão especial para escutar nelas. São a área VIP da

máquina, com cordinha de veludo e segurança na porta. É por isso que, no desenvolvimento, usamos 8000 e 5173 em vez de 80. No capítulo 3, você vai ver como um servidor em produção resolve isso com um proxy reverso.

Uma porta só pode ter um dono por vez, para um mesmo endereço. Se você tentar subir um segundo Uvicorn na 8000 enquanto o primeiro está rodando, recebe o famoso `[Errno 48] Address already in use` (no Linux, o número do erro é 98). A mensagem quer dizer exatamente isso: o apartamento já tem morador. Em nove de cada dez casos, o morador é um Uvicorn que você mesmo esqueceu rodando num terminal escondido atrás de outras quinze janelas, três dias atrás.

Escutar em qual endereço?

Lembra que o notebook tem várias interfaces, cada uma com um IP? Quando um processo começa a escutar numa porta, ele escolhe também em qual endereço vai escutar. Esse passo se chama *bind*.

O Uvicorn, por padrão, faz bind em `127.0.0.1`. Ele aceita conexões que chegam pela loopback e ignora as que chegam pelo Wi-Fi. É uma escolha segura: você não quer que qualquer pessoa no Wi-Fi da cafeteria acesse a sua API de desenvolvimento, muito menos o rapaz da mesa ao lado, de capuz e notebook coberto de adesivos de caveira. O Vite faz o mesmo.

Para aceitar conexões de outras máquinas, você precisa fazer bind em `0.0.0.0`. Nesse contexto, `0.0.0.0` significa "todas as interfaces desta máquina". Guarde essa distinção, porque ela vai aparecer de novo na seção de armadilhas, em uma história nada agradável e com pedido de resgate.

TCP e UDP: como a conversa acontece

Saber o IP e a porta permite que um pacote chegue ao destino. Mas uma requisição HTTP com uma resposta de alguns megabytes vira centenas de pacotes. Eles podem se perder no caminho, chegar fora de ordem ou chegar duplicados, como convidados de casamento. Alguém precisa organizar essa bagunça, e é aí que entram os protocolos de transporte.

O **TCP** (*Transmission Control Protocol*) cria uma conexão confiável entre os dois lados. Antes de qualquer dado, os dois lados fazem o *three-way handshake*, um aperto de mão em três passos. É o protocolo mais cerimonioso da internet. A piada clássica resume bem: "Oi, quer ouvir uma piada de TCP?" "Oi, quero ouvir uma piada de TCP." "Certo, vou contar uma piada de TCP." No diagrama, fica assim:

```
Cliente                          Servidor
   |  ---- SYN ------------------>  |   "quero conversar"
```

```
    |  <--- SYN-ACK ---------------  |   "ok, eu também"
    |  ---- ACK ------------------>  |   "combinado"
    |                                |
    |  ==== dados (HTTP) =========>  |
```

Depois disso, o TCP numera cada pedaço de dado, confirma o que recebeu, reenvia o que se perdeu e entrega tudo na ordem certa para a aplicação. O Uvicorn nunca vê um pacote perdido. Ele recebe a requisição inteira, como se a rede fosse perfeita, no mesmo estado de ignorância feliz em que um gerente recebe um relatório consolidado. HTTP/1.1 e HTTP/2, que carregam praticamente todas as APIs que você já escreveu, rodam sobre TCP.

O handshake explica os dois erros que a sua sócia viu. Quando o navegador manda um SYN para uma máquina que existe, mas não tem ninguém escutando naquela porta, o sistema operacional dessa máquina responde com um pacote **RST** (*reset*). O navegador traduz isso como "conexão recusada", e acontece em milissegundos. É o "não" educado e imediato. Quando o SYN nunca chega a lugar nenhum, ou chega e é descartado em silêncio, o navegador fica esperando um SYN-ACK que nunca vem, como quem espera o "a gente se fala" do recrutador. Depois de algumas tentativas, desiste com um *timeout*.

Essa diferença vale ouro na hora de investigar problemas de rede:

Sintoma	O que provavelmente aconteceu
Conexão recusada, na hora	A máquina foi encontrada, mas nada escuta naquela porta (ou o processo fez bind só em `127.0.0.1`)
Timeout, depois de vários segundos	O pacote não chegou: endereço inalcançável, firewall descartando ou rota inexistente

O **UDP** (*User Datagram Protocol*) segue o caminho oposto. Ele não faz handshake, não confirma recebimento e não reenvia nada. Manda o pacote e pronto. Eu até contaria uma piada de UDP, mas não teria certeza se você ia receber. Parece pior, mas é a escolha certa quando rapidez importa mais do que garantia: numa chamada de vídeo, um quadro perdido deve ser simplesmente ignorado, porque reenviá-lo meio segundo depois só atrapalharia. Ninguém quer ver a cara congelada do chefe duas vezes. O DNS, que você vai conhecer no próximo capítulo, usa UDP na maior parte das consultas. E o HTTP/3 roda sobre o QUIC, um protocolo construído em cima do UDP que reimplementa a confiabilidade do seu próprio jeito. Basicamente, alguém olhou para o TCP, disse "eu faço melhor" e, para surpresa geral, fez.

Roteamento: de salto em salto até o destino

Agora você sabe como dois processos conversam quando um sabe o IP do outro. Mas entre o celular em Recife e o seu notebook existem dezenas de equipamentos. Quem decide o caminho?

Ninguém decide o caminho inteiro. A internet funciona de salto em salto. Cada equipamento no meio do caminho, chamado **roteador**, olha o IP de destino do pacote, consulta a sua própria **tabela de rotas** e decide só qual é o próximo salto. O roteador seguinte faz o mesmo, e assim por diante. É parecido com pedir informação na rua: ninguém te explica o trajeto completo, cada pessoa só aponta para a próxima esquina e diz "é logo ali", o que no Brasil pode significar qualquer distância entre duzentos metros e quatro quilômetros. A diferença é que os roteadores, ao contrário do seu Zé da banca de jornal, costumam acertar.

O seu notebook também tem uma tabela de rotas, e ela costuma ser bem curta. Em geral diz duas coisas: "endereços da rede local, entregue direto" e "todo o resto, mande para o roteador de casa". Esse "todo o resto" é a **rota padrão**, e o roteador para onde ela aponta é o **gateway padrão**. É a filosofia de vida do estagiário: o que eu não sei, mando para alguém acima de mim.

Quando várias rotas servem para um mesmo destino, vence a mais específica. Esse critério se chama *longest prefix match*. Se você já escreveu rotas no FastAPI, conhece a ideia: uma rota `/restaurantes/destaques` precisa ser tratada antes de `/restaurantes/{id}`, senão a palavra "destaques" seria interpretada como um id. Na tabela de rotas, a rota padrão é escrita como `0.0.0.0/0`, que quer dizer "qualquer endereço". Ela é a menos específica de todas, uma espécie de rota curinga, usada só quando nenhuma outra serve. Em breve você vai entender exatamente o que esse `/0` significa.

IP público, IP privado e o truque do NAT

Volte ao segundo link, o `http://192.168.0.12:5173`. Por que ele deu timeout?

Porque `192.168.0.12` é um **IP privado**. Para adiar o esgotamento do IPv4, a RFC 1918 reservou três faixas de endereços para uso interno, que qualquer rede pode usar à vontade e que nunca são roteadas na internet pública:

Faixa	Notação CIDR	Uso típico
10.0.0.0 a 10.255.255.255	`10.0.0.0/8`	Empresas e redes na nuvem
172.16.0.0 a 172.31.255.255	`172.16.0.0/12`	Empresas, redes do Docker

Faixa	Notação CIDR	Uso típico
192.168.0.0 a 192.168.255.255	`192.168.0.0/16`	Redes domésticas

Neste exato momento, milhões de casas usam o endereço `192.168.0.12`. Mandar esse endereço para alguém é como mandar uma carta para "Rua Principal, número 12", sem cidade nem estado. Quando o celular da sua sócia tentou alcançá-lo, os roteadores entre ela e a internet não tinham rota para esse endereço fora da rede local dela. O pacote foi descartado em silêncio e o navegador esperou até desistir.

Um **IP público**, por outro lado, é único no mundo inteiro e roteável na internet. O seu roteador de casa recebe um do provedor. Todas as máquinas da sua casa saem para a internet usando esse mesmo IP público, inclusive a geladeira inteligente que você nunca entendeu por que precisa de Wi-Fi. Como isso funciona?

NAT: a portaria do condomínio

O truque se chama **NAT** (*Network Address Translation*). Quando o seu notebook faz uma requisição para fora, o roteador troca o IP privado de origem (`192.168.0.12`) pelo IP público dele e anota essa troca numa tabela. Quando a resposta volta, ele consulta a tabela, descobre que aquela conexão pertence ao notebook e entrega o pacote para `192.168.0.12`. Ele trabalha como o porteiro que anota tudo no caderninho, com a vantagem de nunca cochilar na cadeira às três da tarde.

Se você quiser pensar em código, a tabela do NAT é um dicionário Python mais ou menos assim:

```
# Visão simplificada da tabela de NAT do roteador
tabela_nat = {
    # (IP público, porta pública): (IP privado, porta privada)
    ("203.0.113.45", 51001): ("192.168.0.12", 62344),  # notebook
    ("203.0.113.45", 51002): ("192.168.0.23", 49210),  # celular
}
```

Os endereços `203.0.113.x` usados aqui fazem parte de uma faixa reservada para documentação (RFC 5737) e não pertencem a ninguém. Vamos usar essa faixa e a `198.51.100.x` em todos os exemplos de IP público do livro.

Agora pense no que acontece quando chega um pacote de fora que não responde a nenhuma requisição anterior. A sua sócia tenta se conectar ao IP público da sua casa, na porta 5173. O roteador procura na tabela, não encontra nenhuma entrada para aquela porta e descarta o pacote. Por isso o NAT, além de economizar endereços, funciona como uma barreira involuntária: conexões de entrada não solicitadas simplesmente não entram.

Existe um jeito de furar essa barreira, o *port forwarding*: você configura no roteador uma regra fixa do tipo "tudo que chegar na porta 5173 vai para `192.168.0.12`". Funciona, mas deixa a sua máquina exposta para a internet inteira, e não só para a sua sócia. É como pedir ao porteiro que deixe subir qualquer pessoa que diga "vim pela 5173". Logo você vai ver por que isso é uma péssima ideia.

CGNAT: quando nem o roteador tem IP público

Em muitas conexões residenciais e quase todas as conexões móveis no Brasil, existe ainda uma segunda camada. O provedor coloca vários clientes atrás de um único IP público, fazendo NAT do lado dele. Isso se chama **CGNAT** (*Carrier-Grade NAT*). É um NAT dentro de outro NAT, uma espécie de *A Origem* das redes, só que sem o Leonardo DiCaprio e com muito mais ligações para o suporte. O roteador da sua casa recebe um endereço da faixa `100.64.0.0/10`, reservada para esse fim, e não um IP público de verdade.

Com CGNAT, nem o port forwarding resolve, porque você não controla o NAT do provedor. Um jeito fácil de desconfiar disso é comparar o IP que o seu roteador mostra no painel de administração com o IP que um serviço externo enxerga. Se forem diferentes, há um NAT a mais no caminho.

CIDR e subnets: fatiando endereços

Você já esbarrou em notações como `192.168.0.0/16` e `0.0.0.0/0`. Chegou a hora de entender o que essa barra quer dizer, porque ela vai acompanhar você por todo o livro e, na Parte II, vai aparecer em praticamente toda configuração de rede. Calma, a matemática aqui é só potência de 2, a única que programador ainda lembra com algum carinho.

Um IPv4 tem 32 bits. A notação **CIDR** (*Classless Inter-Domain Routing*) descreve um bloco de endereços dizendo quantos bits do começo são fixos. O número depois da barra é essa quantidade. Os bits restantes podem variar, e cada combinação é um endereço do bloco.

Em `192.168.0.0/24`, os primeiros 24 bits (os três primeiros números, `192.168.0`) são fixos. Sobram 8 bits livres, que geram $2^8 = 256$ endereços, de `192.168.0.0` até `192.168.0.255`. A regra geral é que um bloco `/n` tem 2^(32−n) endereços. Quanto maior o número depois da barra, menor o bloco, o que é contraintuitivo exatamente na medida certa para cair em entrevista de emprego:

CIDR	Bits livres	Endereços	Exemplo
`/32`	0	1	Um único host
`/28`	4	16	Uma subnet bem pequena
`/24`	8	256	Uma rede doméstica típica
`/16`	16	65.536	Uma rede corporativa ou de nuvem
`/0`	32	Todos	A rota padrão, "qualquer lugar"

Agora o `0.0.0.0/0` da tabela de rotas fica claro: zero bits fixos, qualquer endereço serve. O *longest prefix match* também. Uma rota para `192.168.0.0/24` fixa 24 bits, então é mais específica que `0.0.0.0/0`, que não fixa nenhum.

Na prática, nem todos os endereços de um bloco podem ser usados por máquinas. Em redes tradicionais, o primeiro endereço identifica a própria rede e o último é o de *broadcast*. A AWS vai além e reserva cinco endereços em cada subnet, então um bloco `/24` lá oferece 251 endereços utilizáveis. É um dos raros momentos em que a AWS fica com algo seu sem mandar a conta depois. Você vai encontrar isso de novo no capítulo 10.

Subnet: quem conversa direto com quem

Uma **subnet** é um bloco de endereços cujas máquinas se falam diretamente, sem passar por um roteador. A rede da sua casa é uma subnet, quase sempre `192.168.0.0/24` ou `192.168.1.0/24`. Quando o notebook quer falar com o celular, que está em `192.168.0.23`, ele consulta a tabela de rotas, vê que o destino está na mesma subnet e entrega direto. Para qualquer outro destino, manda para o gateway.

Subnets servem para organizar e para isolar. Numa empresa, você pode colocar os servidores numa subnet, os notebooks dos funcionários em outra e as impressoras numa terceira, controlando o que passa de uma para outra. Isolar as impressoras é sempre sábio: elas são os únicos equipamentos de TI com comprovada vontade própria. Na nuvem, a separação mais comum é entre **subnets públicas**, que têm rota para a internet, e **subnets privadas**, que não têm. O servidor web fica na pública. O banco de dados fica na privada, onde ninguém de fora consegue chegar. Guarde essa imagem, porque o Rangô vai precisar dela.

Na prática: seguindo os pacotes do Rangô

Chega de teoria. Vamos acompanhar o que o terminal mostra num notebook comum com o Rangô rodando e, no fim, colocar o Rangô no celular. As saídas estão todas aqui, então dá para entender tudo só lendo, sem óculos escuros de hacker de filme dos anos 1990. Os

comandos abaixo funcionam em macOS e Linux. Quando o Windows tiver um equivalente diferente, ele aparece no comentário, que é o lugar onde a comunidade Unix tradicionalmente acomoda o Windows.

Os endereços do notebook

Primeiro, o IP privado do notebook na rede local:

```
# macOS (en0 costuma ser o Wi-Fi)
ipconfig getifaddr en0

# Linux
ip -4 addr show

# Windows: rode "ipconfig" e procure "Endereço IPv4"
```

No notebook do exemplo, a resposta é `192.168.0.12`. No Linux, a lista mostra também a interface `lo`, com `127.0.0.1`: é a loopback.

Agora, o IP público com que o mundo enxerga você:

```
curl https://checkip.amazonaws.com
```

```
203.0.113.45
```

São dois endereços diferentes. O primeiro só existe dentro da sua casa. O segundo pertence ao roteador, ou ao provedor, se houver CGNAT. Se o painel do roteador mostrar um IP de WAN começando com `100.`, entre 100.64 e 100.127, a conexão está atrás de CGNAT.

Quem está escutando

Com o backend do Rangô rodando do jeito padrão, dá para perguntar ao sistema quem está escutando na porta 8000:

```
lsof -nP -iTCP:8000 -sTCP:LISTEN

# Linux, alternativa: ss -ltnp | grep 8000
# Windows: netstat -ano | findstr :8000
```

```
COMMAND   PID  USER   FD   TYPE  DEVICE  SIZE/OFF NODE NAME
Python  41233  voce    3u  IPv4  0x9a1c       0t0  TCP 127.0.0.1:8000 (LISTEN)
```

A coluna `NAME` mostra o bind: `127.0.0.1:8000`. O Uvicorn só aceita conexões pela loopback. O `nc` (netcat), que tenta abrir uma conexão TCP e informa o resultado, confirma:

```
nc -zv 127.0.0.1 8000
nc -zv 127.0.0.1 8001
```

```
Connection to 127.0.0.1 port 8000 [tcp/*] succeeded!
nc: connectx to 127.0.0.1 port 8001 (tcp) failed: Connection refused
```

O texto exato muda um pouco entre sistemas, mas o resultado é o mesmo: a 8000 completou o handshake, e a 8001 respondeu com RST, porque ninguém mora lá.

Agora vem o teste mais revelador: uma conexão à porta 8000 pelo IP do Wi-Fi, e não pelo da loopback:

```
curl http://192.168.0.12:8000/health
```

```
curl: (7) Failed to connect to 192.168.0.12 port 8000 after 2 ms: Couldn't connect to
server
```

Conexão recusada, na própria máquina. O seu notebook acabou de dar um fora em si mesmo. O Uvicorn está rodando, mas fez bind só em `127.0.0.1`, então ignora o que chega pela interface do Wi-Fi. Para comparar, veja como é um timeout de verdade, usando um IP de documentação que não leva a lugar nenhum:

```
curl --connect-timeout 5 http://203.0.113.10:8000/health
```

```
curl: (28) Failed to connect to 203.0.113.10 port 8000 after 5002 ms: Timeout was
reached
```

Dois milissegundos contra cinco segundos. Agora você reconhece a assinatura de cada erro.

A tabela de rotas

```
# Linux
ip route

# macOS: netstat -rn -f inet
# Windows: route print
```

```
default via 192.168.0.1 dev wlan0 proto dhcp metric 600
192.168.0.0/24 dev wlan0 proto kernel scope link src 192.168.0.12 metric 600
```

São as duas regras que você já conhece. A segunda diz que a subnet `192.168.0.0/24` é entregue direto pela interface `wlan0`. A primeira é a rota padrão: todo o resto vai para `192.168.0.1`, o roteador de casa. No macOS, a rota padrão aparece como `default` na coluna `Destination`.

Os saltos até o destino

O `traceroute` mostra cada roteador no caminho até um destino. Ele envia pacotes com um limite de saltos crescente e anota quem devolve o aviso de "limite esgotado" em cada etapa. É um truque engenhoso: você descobre o caminho mandando pacotes que morrem de propósito, um pouco mais longe a cada vez. Em qualquer outro contexto, isso seria crueldade.

```
traceroute checkip.amazonaws.com
# Windows: tracert checkip.amazonaws.com
```

```
 1  192.168.0.1      2.1 ms
 2  100.64.0.1       8.4 ms
 3  198.51.100.17   11.9 ms
 4  198.51.100.94   14.2 ms
 5  * * *
 6  203.0.113.201  142.7 ms
```

A saída acima foi simplificada, e os endereços dos saltos externos foram trocados por IPs de documentação. Mesmo assim, ela mostra uma história típica. O primeiro salto é o seu roteador. O segundo, com `100.64.0.1`, denuncia o CGNAT do provedor. Os asteriscos indicam um roteador que não responde a esse tipo de sonda, o que é comum e não significa que o caminho esteja quebrado. E o salto de 142 ms mostra o pacote atravessando um oceano: `checkip.amazonaws.com` costuma responder de servidores nos Estados Unidos. Guarde esse número, porque a distância até a região escolhida na AWS vai importar bastante na Parte II.

Contas de CIDR com Python

O módulo `ipaddress`, da biblioteca padrão, é ótimo para conferir contas de CIDR sem papel e caneta:

```
>>> import ipaddress
>>> casa = ipaddress.ip_network("192.168.0.0/24")
>>> casa.num_addresses
256
>>> ipaddress.ip_address("192.168.0.23") in casa
True
>>> ipaddress.ip_address("192.168.1.23") in casa
False
>>> ipaddress.ip_address("192.168.0.12").is_private
True
>>> ipaddress.ip_network("10.0.0.0/16").num_addresses
65536
>>> list(casa.subnets(new_prefix=26))
[IPv4Network('192.168.0.0/26'), IPv4Network('192.168.0.64/26'),
 IPv4Network('192.168.0.128/26'), IPv4Network('192.168.0.192/26')]
```

A última linha divide o `/24` da sua casa em quatro subnets de 64 endereços. É exatamente esse tipo de divisão que você vai fazer quando desenhar a rede do Rangô na AWS.

O Rangô no celular

Com tudo isso, já dá para fazer o Rangô funcionar pelo menos num celular conectado ao mesmo Wi-Fi do notebook. Os dois estão na mesma subnet, então não há NAT nem roteador no meio. Só falta o bind.

O backend precisa subir escutando em todas as interfaces e liberando a origem do front acessado pelo IP:

```
cd backend
CORS_ORIGINS="http://localhost:5173,http://192.168.0.12:5173" \
  uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

No front, o Vite tem a opção `--host` para o mesmo fim:

```
cd frontend
npm run dev -- --host
```

```
  VITE v6.x  ready in 312 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://192.168.0.12:5173/
```

Na primeira vez, o macOS ou o Windows podem perguntar se devem permitir conexões de entrada para o Python ou o Node. A resposta certa é permitir só para redes privadas.

No celular, o endereço `http://192.168.0.12:5173` carrega a página, o título "Rangô" aparece... e a lista de restaurantes fica vazia. Tão vazia quanto a geladeira de quem passou o fim de semana caçando um bug.

Pare um instante e tente descobrir o motivo antes de ler a próxima linha. Eu espero. Na verdade não espero, isto é um livro, mas faça de conta.

O código React roda no navegador do celular. Quando ele executa `fetch("http://localhost:8000/restaurantes")`, o `localhost` é o próprio celular, onde não existe API nenhuma. É o mesmo erro do link que você mandou para a sua sócia, só que escondido dentro do código, onde ninguém pode tirar print para rir de você. O front precisa saber o endereço real da API. A correção é um arquivo `frontend/.env.local`:

```
# frontend/.env.local
VITE_API_URL=http://192.168.0.12:8000
```

Depois de reiniciar o `npm run dev -- --host` (o Vite só lê as variáveis de ambiente na inicialização) e recarregar a página no celular, os dois restaurantes aparecem.

Esse bug vai voltar, em outras formas, durante todo o livro, como vilão de filme de terror que sempre aparece na continuação. Qualquer endereço fixo no código é uma aposta sobre onde a aplicação vai rodar, e essa aposta quase sempre perde quando o código muda de máquina. Por isso o endereço da API e as origens do CORS vieram de variáveis de ambiente desde o começo.

E a sua sócia em Recife? Continua sem acesso. O seu notebook tem um IP privado, atrás de um NAT, talvez atrás de dois. Para ela alcançar o Rangô, ele precisa estar numa máquina com IP público, ligada o tempo todo, e não num notebook que você fecha para ir almoçar. Um app de comida que sai do ar na hora do almoço é uma ironia que nenhum investidor vai achar engraçada.

Armadilhas e casos reais

O `0.0.0.0` no lugar errado

Na sua rede de casa, fazer bind em `0.0.0.0` expõe o serviço para quem estiver no mesmo Wi-Fi, e o NAT do roteador barra o resto. No pior caso, quem vê a sua API é o seu cunhado. Num servidor com IP público, não existe essa proteção. Escutar em `0.0.0.0` significa aceitar conexões da internet inteira, a menos que um firewall diga o contrário.

> **Na vida real:** no começo de janeiro de 2017, grupos de atacantes passaram a varrer a internet atrás de servidores MongoDB acessíveis publicamente e sem senha de administrador. Ferramentas de busca como o Shodan listavam dezenas de milhares deles. Os atacantes copiavam os dados, apagavam tudo e deixavam uma nota pedindo resgate em bitcoin. Em poucos dias, mais de 27 mil bancos tinham sido sequestrados. Em muitos casos, a causa era a mesma: o banco escutando em todas as interfaces de um servidor com IP público, sem autenticação e sem firewall na frente. Foi provavelmente a campanha de conscientização sobre senhas de banco de dados mais eficiente da história, patrocinada, sem querer, pelo crime organizado.

O PostgreSQL do Rangô, instalado localmente, escuta só em `localhost` por padrão, e é bom que continue assim. A pergunta "quem consegue chegar nesta porta?" deve ser feita para cada serviço, e a resposta para o banco de dados deveria ser "só a aplicação". Se a resposta for "não sei", você acaba de encontrar o seu próximo incidente.

O Docker que fura o firewall

> **Armadilha:** em Linux, publicar uma porta com `docker run -p 5432:5432 postgres` faz bind em `0.0.0.0` por padrão, e o Docker, prestativo como estagiário no primeiro dia, cria as próprias regras de firewall no `iptables`. Na prática, isso pode passar por cima de regras do `ufw` que você achava que estavam bloqueando a porta. Se o container só precisa ser acessado pela própria máquina, publique com `-p 127.0.0.1:5432:5432`.

Você vai conhecer containers a fundo no capítulo 3, mas essa armadilha vale ser conhecida desde já, porque muita gente sobe um Postgres em container no primeiro servidor alugado.

Port forwarding como atalho

Liberar uma porta no roteador de casa para mostrar o app para alguém parece inofensivo. O problema é que, em poucos minutos, robôs que varrem a internet sem parar vão encontrar essa porta aberta e começar a testar o que há do outro lado. Eles não dormem, não tiram férias e não participam de reunião de alinhamento, por isso são tão produtivos. Um servidor de desenvolvimento com `--reload`, rotas de debug e sem autenticação é exatamente o que eles procuram. E, se você estiver atrás de CGNAT, o atalho nem funciona, o que pode ser uma sorte. O provedor salvou você de você mesmo. Mande flores.

Faixas privadas que colidem

> **Armadilha:** duas redes privadas podem usar a mesma faixa sem problema algum, até o dia em que precisam se conectar. É como dois vizinhos com o mesmo nome no mesmo prédio: tudo tranquilo até chegar a primeira encomenda. Se a VPN da empresa usa `192.168.0.0/24` e a sua casa também, o notebook não sabe para qual lado mandar os pacotes para `192.168.0.10`. A tabela de rotas tem duas respostas para a mesma pergunta, e algum serviço some. Na nuvem, escolher faixas que não se sobreponham às redes da empresa e de outros ambientes economiza muita dor de cabeça futura. Esse assunto volta no capítulo 10.

IPv6 e a falsa sensação de segurança

Com IPv6 há endereços de sobra, então cada dispositivo pode receber um endereço público e o NAT deixa de ser necessário. Isso também quer dizer que a barreira involuntária do NAT desaparece. Os roteadores domésticos modernos costumam bloquear conexões de entrada em IPv6 por padrão, mas o ponto importante é outro:

quem protege um serviço é um firewall configurado de propósito, e não um efeito colateral da falta de endereços. Contar com o NAT como segurança é como contar com o trânsito para impedir que o ladrão chegue à sua casa. Às vezes funciona, mas ninguém deveria chamar isso de plano.

Resumo

- `localhost` (`127.0.0.1`) aponta para a loopback, uma interface que nunca sai da máquina. Cada máquina tem o seu.
- Um endereço IP identifica uma interface de rede. O IPv4 tem 32 bits e está esgotado. O IPv6 tem 128 bits.
- A porta escolhe o processo dentro da máquina. IP, porta e caminho da URL resolvem máquina, processo e função.
- Fazer bind em `127.0.0.1` aceita só conexões locais. Em `0.0.0.0`, aceita por todas as interfaces.
- O TCP abre conexões com o handshake SYN, SYN-ACK, ACK e garante entrega em ordem. O UDP manda e esquece.
- Conexão recusada significa que a máquina respondeu, mas nada escuta na porta. Timeout significa que o pacote não chegou ou foi descartado.
- O roteamento acontece salto a salto. Vence a rota mais específica, e `0.0.0.0/0` é a rota padrão.
- IPs privados (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) não circulam na internet. O NAT traduz para um IP público e barra conexões de entrada não solicitadas. O CGNAT adiciona mais uma camada no provedor.
- No CIDR, o número depois da barra diz quantos bits são fixos. Um `/24` tem 256 endereços, e um `/16` tem 65.536.
- Uma subnet agrupa máquinas que se falam direto. Subnets públicas e privadas separam o que a internet pode alcançar.
- O Rangô agora roda no celular, na mesma rede, com o endereço da API vindo de `VITE_API_URL` e as origens do CORS vindo de `CORS_ORIGINS`.

Próxima parada: um nome e um cadeado

Digamos que, num passe de mágica, o Rangô já estivesse rodando numa máquina com IP público, `203.0.113.45`. A sua sócia conseguiria acessar. Mas ela teria de digitar `http://203.0.113.45:5173` no navegador, e nenhum cliente faz pedido de comida

digitando quatro números separados por pontos. Com fome, a pessoa mal consegue digitar o próprio CEP.

O que ela quer, e o que os clientes esperam, é digitar `rango.com.br` e ver um cadeado ao lado do endereço. Isso levanta duas perguntas novas. Como um nome vira um IP, se nenhum computador do caminho sabe o que é "rangô"? E, quando o navegador mostra o cadeado, quem garante que aquele site é mesmo o seu, e não o de alguém que interceptou a conexão no Wi-Fi da praça de alimentação? No próximo capítulo, vamos abrir o DNS e o TLS por dentro para responder a essas duas perguntas.

Referências

- RFC 791, *Internet Protocol* (IPv4): https://www.rfc-editor.org/rfc/rfc791
- RFC 9293, *Transmission Control Protocol* (TCP): https://www.rfc-editor.org/rfc/rfc9293
- RFC 768, *User Datagram Protocol* (UDP): https://www.rfc-editor.org/rfc/rfc768
- RFC 1918, *Address Allocation for Private Internets*: https://www.rfc-editor.org/rfc/rfc1918
- RFC 4632, *Classless Inter-domain Routing* (CIDR): https://www.rfc-editor.org/rfc/rfc4632
- RFC 6598, faixa `100.64.0.0/10` para CGNAT: https://www.rfc-editor.org/rfc/rfc6598
- RFC 5737, endereços IPv4 reservados para documentação: https://www.rfc-editor.org/rfc/rfc5737
- Documentação do FastAPI: https://fastapi.tiangolo.com/
- Configurações do Uvicorn (`--host` , `--port`): https://www.uvicorn.org/settings/
- Variáveis de ambiente no Vite: https://vite.dev/guide/env-and-mode
- Módulo `ipaddress` do Python: https://docs.python.org/3/library/ipaddress.html
- Ataques de resgate a bancos MongoDB, janeiro de 2017 (The Register): https://www.theregister.com/2017/01/09/mongodb/
- Mesma onda de ataques (SiliconANGLE): https://siliconangle.com/2017/01/10/27000-mongodb-installs-hijacked-held-ransom/

CAPÍTULO 2

DNS, HTTPS e TLS

No capítulo anterior, você seguiu uma requisição do celular até o processo do Uvicorn: endereço IP para achar a máquina, porta para achar o processo, TCP para manter a conversa em ordem, roteamento salto a salto e o NAT barrando quem chega de fora sem ser chamado. No fim, o Rangô rodava no seu notebook e abria no seu celular, pela rede de casa, e a sua sócia em Recife continuava sem acesso. Ficamos com uma máquina imaginária de IP público, `203.0.113.45`, e com duas perguntas: como um nome vira esse número, e quem garante que o site com cadeado é mesmo o seu. Máquina imaginária, diga-se, é a melhor que existe: nunca cai, nunca pede atualização de kernel e nunca acorda ninguém de madrugada.

Quatro números e um aviso vermelho

De volta à máquina mágica com IP público. Você manda o link para a sua sócia, `http://203.0.113.45:5173`, e funciona. Ela mostra para o dono da Cantina da Nona, que gosta e pergunta qual endereço divulgar no cardápio. Não dá para imprimir aquilo num folheto. Quer dizer, até dá, mas o folheto ficaria com cara de código de barras de boleto.

E há um problema menos visível: na barra de endereços aparece "Não seguro". O Chrome marca assim qualquer página em HTTP puro desde 2018. O dono da cantina não sabe o que isso quer dizer, mas não vai digitar o número do cartão numa página que o próprio navegador chama de insegura. Para quem vende comida, a palavra "inseguro" perto do cardápio lembra muito uma visita da vigilância sanitária.

E o aviso tem fundamento. Uma requisição HTTP viaja em texto puro. No Wi-Fi aberto de uma praça de alimentação, qualquer pessoa na mesma rede, com um programa gratuito de captura de pacotes, lê o endereço de entrega, o token de sessão e a senha do login. Se a senha for `123456`, ela nem precisava ter se dado ao trabalho. E quem controla o roteador do shopping pode trocar o JavaScript do Rangô por outro, que o navegador vai executar sem reclamar.

Então são dois problemas, e eles se resolvem com duas peças separadas. O DNS transforma `rango.com.br` em `203.0.113.45`. O TLS, a camada que transforma HTTP em HTTPS, cifra a conversa e prova que quem responde em `rango.com.br` é mesmo o dono

desse nome. Vamos ver uma de cada vez.

DNS: a agenda de contatos da internet

O *DNS* (*Domain Name System*) é o sistema que traduz nomes em endereços. A analogia mais próxima é a agenda do celular: você procura "Mãe" e o telefone disca o número. (Se na sua agenda ela aparece como "Mãe Número Novo 2", você já entende por que nomes precisam de manutenção.) Só que essa agenda tem bilhões de entradas, muda o tempo todo e não pode ficar num lugar só, porque ninguém aguentaria responder às consultas do planeta inteiro.

A solução foi distribuir a agenda numa hierarquia. O nome se lê de trás para a frente: `rango.com.br.` (sim, com um ponto no final, que normalmente fica oculto). O ponto final é a **raiz**. É o caractere mais importante da internet e quase ninguém sabe que ele existe, um pouco como o DBA da empresa. Abaixo dela vem `br`, o domínio de topo do Brasil. Abaixo de `br` vem `com.br`, e abaixo dele vem `rango`. Cada nível sabe apenas quem cuida do nível de baixo. Os servidores da raiz não fazem ideia de qual é o IP do Rangô, mas sabem quem responde por `br`. Os servidores de `br`, mantidos pelo Registro.br, sabem quem responde por `rango.com.br`. E esse último servidor, escolhido por você, sabe o IP.

Essa ideia de "eu não sei, mas sei quem sabe" se chama **delegação**, e é ela que permite que a agenda cresça sem um dono central. É o mesmo princípio de uma central de atendimento telefônico, com a diferença de que aqui a transferência funciona e ninguém precisa ouvir música de espera.

Quem faz as perguntas: o resolver

O navegador da sua sócia não sai perguntando para a raiz por conta própria. Ele pede ajuda ao sistema operacional, que repassa a pergunta para um **resolver recursivo**. Normalmente é um servidor do provedor de internet, configurado automaticamente quando o celular entra na rede. Também pode ser um resolver público, como o `1.1.1.1` da Cloudflare ou o `8.8.8.8` do Google, endereços tão fáceis de decorar que parecem senha de Wi-Fi de pousada.

O resolver faz o trabalho pesado. Ao receber a pergunta "qual é o IP de `rango.com.br`?", ele segue a cadeia:

1. Pergunta a um servidor raiz. A resposta é: "não sei, mas `br` está com estes servidores aqui", seguida de uma lista como `a.dns.br`, `b.dns.br` e outros.
2. Pergunta a um servidor de `br`. A resposta é: "não sei o IP, mas `rango.com.br` está delegado para `ns1.provedor-dns.example` e `ns2.provedor-dns.example`".
3. Pergunta a um desses servidores. Esse sabe: "`rango.com.br` é `203.0.113.45`".

4. Devolve a resposta ao celular e guarda uma cópia.

O servidor do passo 3 é o **servidor autoritativo** do domínio: ele não repassa nada, responde com autoridade sobre a zona que lhe pertence. A **zona** é o conjunto de registros de um domínio, mais ou menos como uma tabela no banco: cada linha diz que tal nome, de tal tipo, tem tal valor. Quem edita essa tabela é você, pelo painel do provedor de DNS que escolheu.

A raiz tem 13 nomes de servidores, de `a.root-servers.net` a `m.root-servers.net`, e cada um responde de muitos lugares do mundo ao mesmo tempo com *anycast*: o mesmo IP anunciado em vários pontos, e o roteamento leva cada pacote ao mais próximo. São mais de mil instâncias no total. É a única franquia com mais de mil unidades em que todas atendem pelo mesmo endereço e ninguém reclama do atendimento.

Quanto ao transporte, a consulta típica é um pacote UDP para a porta 53 e outro de volta, sem handshake. Respostas grandes demais são repetidas por TCP, na mesma porta.

TTL: o prazo de validade da resposta

Se todo acesso disparasse a cadeia inteira, a raiz e os servidores de `br` estariam soterrados. O que salva o sistema é o cache. Cada resposta vem com um **TTL** (*Time To Live*), em segundos, que diz por quanto tempo ela pode ser reaproveitada. Com TTL de 3600, o resolver do provedor da sua sócia responde com a cópia guardada durante uma hora, sem consultar ninguém. Pense no `max_age` de um decorator de cache em Python. E há várias camadas de cache no caminho (navegador, sistema operacional, resolver), cada uma contando o próprio relógio.

Isso explica a famosa "propagação de DNS". Quando você muda o IP de um domínio, nada se propaga: a mudança vale na hora no servidor autoritativo. As cópias antigas, espalhadas por resolvers do mundo inteiro, só expiram quando o TTL delas vence. Mesmo assim, "é a propagação do DNS" segue firme como a segunda desculpa mais usada da TI, logo atrás de "já tentou reiniciar?".

Os tipos de registro que você vai usar

Cada linha da zona tem um tipo. Existem dezenas, mas o Rangô vai precisar de poucos, como num cardápio de quarenta páginas em que todo mundo acaba pedindo o mesmo prato:

Tipo	O que diz	Exemplo no Rangô
`A`	Nome aponta para um IPv4	`rango.com.br` → `203.0.113.45`
`AAAA`	Nome aponta para um IPv6	`rango.com.br` → `2001:db8::45`

Tipo	O que diz	Exemplo no Rangô
`CNAME`	Nome é um apelido de outro nome	`www.rango.com.br` → `rango.com.br`
`MX`	Servidores que recebem e-mail do domínio	`rango.com.br` → `10 mx.provedor-email.example`
`TXT`	Texto livre, usado para verificações	prova de posse do domínio para uma autoridade certificadora
`NS`	Quais servidores são autoritativos	`rango.com.br` → `ns1.provedor-dns.example`

O `CNAME` merece um parágrafo. Ele diz "para saber o valor deste nome, vá perguntar por aquele outro". É como um alias de import em Python: `www.rango.com.br` vira só outro nome para `rango.com.br`. Ele tem uma restrição que pega muita gente. Um nome com `CNAME` não pode ter nenhum outro registro, e o domínio raiz (o `rango.com.br` sem prefixo, chamado de *apex*) sempre precisa ter registros `NS` e `SOA`. Resultado: você não pode colocar um `CNAME` no apex. Essa regra já arrancou mais suspiros de desenvolvedor do que qualquer outra do DNS, e o DNS, imperturbável, mantém a regra desde os anos 1980. Os provedores de DNS criaram soluções próprias para contornar isso, e a AWS tem a dela, que você vai conhecer no capítulo 14.

O `TXT` é o canal pelo qual você prova a terceiros que controla um domínio: eles pedem um valor aleatório num registro `TXT`, e se ele aparecer na consulta, você manda na zona.

Registrar o nome e apontar para quem responde

Para que `rango.com.br` exista, alguém precisa registrá-lo no Registro.br, por algumas dezenas de reais por ano. É a assinatura mais barata da sua vida, mais barata que qualquer streaming, e mesmo assim todo ano alguém esquece de renovar. No registro, você informa quais servidores autoritativos respondem pelo domínio, e o Registro.br publica isso como registros `NS` na zona de `br`: é a delegação do passo 2. O registrador e o provedor de DNS podem ser empresas diferentes. Na Parte II, a zona do Rangô vai morar no Amazon Route 53.

HTTP por baixo dos panos

Com o IP em mãos, o navegador abre uma conexão TCP com `203.0.113.45` na porta 80 e manda a requisição. Você usa HTTP todos os dias, mas talvez nunca tenha visto como ele é no fio. É texto, linha por linha, tão simples que alguns veteranos ainda digitam requisições à mão num terminal, só para lembrar aos mais novos que um dia foi assim:

```
GET /restaurantes HTTP/1.1
Host: rango.com.br
User-Agent: Mozilla/5.0 ...
Accept: application/json
```

A linha em branco marca o fim dos cabeçalhos. Repare no `Host`. O IP já levou o pacote até a máquina, então por que repetir o nome? Porque uma máquina com um único IP pode servir dezenas de sites, e o servidor web lê o `Host` para decidir qual aplicação responde. Isso se chama *virtual hosting*, e o TLS vai precisar de algo parecido. (O `User-Agent` com `Mozilla/5.0`, aliás, aparece até em navegadores que não têm nada a ver com a Mozilla. Começou nos anos 1990, quando cada navegador fingia ser o Netscape para receber a versão boa das páginas, e ninguém nunca teve coragem de parar.)

O HTTP/2 e o HTTP/3 mudaram o transporte (várias requisições numa conexão, formato binário e, no HTTP/3, QUIC sobre UDP), mas rotas, verbos, cabeçalhos e status continuam iguais para o seu código FastAPI. Na prática, os navegadores só usam essas versões sobre TLS.

TLS: o cadeado

O *TLS* (*Transport Layer Security*) é uma camada que fica entre o TCP e o HTTP. O HTTP continua igual, mas, em vez de ir em texto puro, vai cifrado. HTTPS é simplesmente HTTP dentro de TLS, por padrão na porta 443. O nome antigo, SSL, ainda aparece em todo lugar (`--ssl-keyfile`, "certificado SSL"), mas as versões de SSL estão aposentadas há anos por falhas de segurança. O SSL virou o "xerox" da segurança: a tecnologia saiu de cena e o nome ficou. Hoje se usa TLS 1.2 e, cada vez mais, TLS 1.3.

O TLS garante sigilo (quem captura os pacotes vê bytes embaralhados), integridade (um byte alterado no caminho derruba a conexão) e autenticação (o navegador verifica que fala com o dono de `rango.com.br`, e não com o roteador do shopping). A autenticação é a parte difícil, e é nela que os certificados entram.

O problema de combinar uma senha em público

Para cifrar a conversa, o navegador e o servidor precisam de uma chave em comum. Só que eles nunca se viram antes, e todo pacote entre os dois passa por máquinas desconhecidas. Como combinar um segredo gritando numa sala cheia?

A criptografia resolve isso com uma troca de chaves (no TLS moderno, uma variante chamada ECDHE). Cada lado gera um par de números, troca a parte pública e combina com a própria parte privada. Os dois chegam ao mesmo segredo, e quem só viu as partes públicas não consegue calculá-lo. É matemática, e funciona. Como toda boa matemática, ninguém na sala precisa entendê-la para se beneficiar dela.

Só que há um furo. Se o roteador do shopping estiver no meio, ele pode fazer a troca de chaves com o navegador fingindo ser o Rangô, e outra troca com o Rangô fingindo ser o navegador. Os dois lados acham que têm uma conexão segura, e o intermediário lê tudo. É o ataque conhecido como *man-in-the-middle*, uma espécie de telefone sem fio em que o garoto do meio presta atenção em tudo. A troca de chaves, sozinha, garante que você está falando em segredo com alguém. Não garante quem é esse alguém.

Certificados: o documento de identidade do site

O **certificado** fecha o furo. Ele é um arquivo que diz, em essência: "a chave pública X pertence a quem controla `rango.com.br`, válido de tal data a tal data", com a assinatura digital de uma **autoridade certificadora** (a *CA*, de *Certificate Authority*). Durante o handshake, o servidor envia o certificado e prova que tem a chave privada correspondente, assinando os dados daquela conexão específica. O intermediário do shopping pode copiar o certificado do Rangô, que é público, mas não tem a chave privada, então não consegue produzir essa assinatura.

E por que o navegador confia na assinatura da CA? Porque o sistema operacional e o navegador vêm de fábrica com uma lista de CAs confiáveis, o chamado **repositório de raízes** (*root store*). São algumas dezenas de organizações auditadas, e a chave pública de cada uma está gravada no seu computador. A analogia mais próxima é o cartório: você confia na firma reconhecida porque confia no cartório, e confia no cartório porque o Estado o credenciou. Para um leitor brasileiro, é a analogia mais intuitiva do livro: nada nos tranquiliza tanto quanto um carimbo.

Na prática, a raiz fica guardada offline e assina certificados **intermediários**, que assinam os dos sites. O resultado é uma **cadeia**: `rango.com.br` assinado pelo intermediário, assinado pela raiz que está no repositório do navegador. O servidor envia o próprio certificado e os intermediários; a raiz o navegador já tem.

Antes de mostrar o cadeado, o navegador confere se a cadeia termina numa raiz confiável, se nada está vencido e se o nome digitado aparece no campo *Subject Alternative Name* (SAN) do certificado. Um certificado para `rango.com.br` não vale para `www.rango.com.br`, a menos que os dois estejam no SAN ou que se use um curinga, `*.rango.com.br`. E o curinga cobre um nível só: vale para `api.rango.com.br`, mas não para `rango.com.br` nem para `v2.api.rango.com.br`. Curinga de certificado é bem menos generoso que curinga de baralho.

Como a CA sabe que o domínio é seu

A CA não pede documento com foto. Nem comprovante de residência, nem as três últimas contas de luz. Para os certificados comuns, chamados de *Domain Validated*, ela só verifica se você controla o domínio. Os desafios mais usados são dois. No desafio por

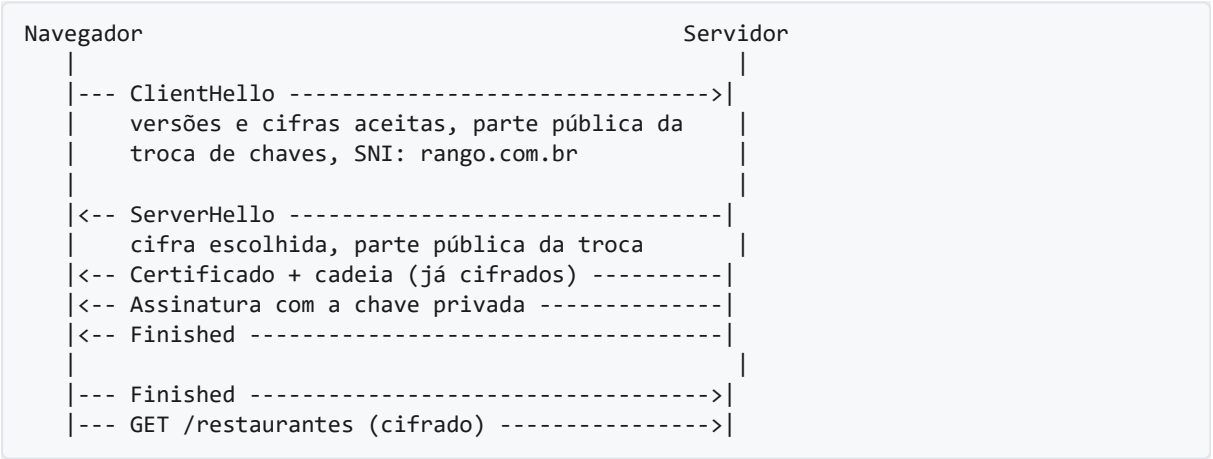
HTTP, a CA pede que você publique um arquivo com um conteúdo aleatório num caminho específico do site, e depois tenta baixá-lo. No desafio por DNS, ela pede um registro `TXT` com um valor aleatório. Se você consegue fazer qualquer um dos dois, controla o domínio.

Até meados da década passada, certificados custavam dinheiro e davam trabalho para renovar, o que mantinha boa parte da web em HTTP. Isso mudou com o **Let's Encrypt**, uma CA sem fins lucrativos que emite certificados gratuitos, de forma automatizada, pelo protocolo ACME. Ferramentas como o Certbot pedem o certificado, respondem ao desafio e renovam sozinhas. Na AWS, o AWS Certificate Manager faz o mesmo papel para os serviços dela, também sem custo, e você vai usá-lo no capítulo 14. Sim, existe algo gratuito na AWS, e ele aparece logo no capítulo 2. Aproveite a sensação.

A validade dos certificados vem encolhendo. Os do Let's Encrypt valem 90 dias. Em 2025, o CA/Browser Forum, que reúne CAs e fabricantes de navegadores, aprovou uma redução gradual do prazo máximo: 200 dias a partir de março de 2026, 100 dias a partir de março de 2027 e 47 dias a partir de março de 2029. Renovar à mão, com lembrete no calendário, deixa de ser viável. Com 47 dias de validade, quem renova certificado à mão está, na prática, num relacionamento sério com ele.

O handshake, passo a passo

Juntando as peças, um handshake de TLS 1.3 fica assim:



Repare no campo **SNI** (*Server Name Indication*) do primeiro pacote. Lembra do `Host` no HTTP? O problema é o mesmo: um IP com vários sites. Só que, no TLS, o servidor precisa escolher qual certificado enviar antes de qualquer HTTP ser trocado. Então o navegador já diz no `ClientHello` qual nome quer acessar. Esse campo ainda viaja visível na maioria das conexões, e por isso quem observa a rede consegue saber qual site você acessou, mesmo sem ver o conteúdo. O envelope é lacrado, mas o nome do destinatário continua escrito do lado de fora.

No TLS 1.3, o handshake custa uma ida e volta antes da primeira requisição (no TLS 1.2 eram duas). Some o TCP, o DNS e a própria requisição: com 140 ms de ida e volta até um servidor nos Estados Unidos, a primeira visita gasta perto de meio segundo antes do primeiro byte do Rangô. Meio segundo parece pouco, mas é tempo de sobra para o usuário concluir que o app travou e apertar o botão mais três vezes.

CDN e cache na borda

Resta a distância. Os arquivos do React e as fotos dos pratos são iguais para todo mundo, mas cada cliente em Manaus ou Porto Alegre busca tudo no mesmo servidor, pagando todas as idas e voltas até lá. É como se toda pizza do país saísse de um único forno em São Paulo.

Uma **CDN** (*Content Delivery Network*) é uma rede de servidores espalhados pelo mundo, os **pontos de presença** ou servidores de **borda** (*edge*), que guardam cópias do seu conteúdo perto dos usuários. O DNS da CDN, ou o anycast, manda cada cliente para a borda mais próxima. Se a borda já tem o arquivo, responde na hora, e isso é um *cache hit*. Se não tem, busca no seu servidor (que, nesse contexto, se chama **origem**), entrega ao cliente e guarda uma cópia: um *cache miss*.

O handshake TLS também passa a acontecer na borda, a poucos milissegundos do cliente, e a borda reaproveita conexões já abertas com a origem. Mesmo para a API, que não dá para cachear, isso encurta a primeira resposta.

Quem decide o que pode ficar em cache, e por quanto tempo, é você, com o cabeçalho `Cache-Control` da resposta. A analogia com o TTL do DNS é direta. O build do Vite ajuda muito aqui: ele gera arquivos com um hash no nome, como `assets/index-4f9a1c2b.js`. Se o conteúdo muda, o nome muda. Por isso esses arquivos podem ser cacheados por um ano:

```
Cache-Control: public, max-age=31536000, immutable
```

O `index.html` é o oposto. Ele tem sempre o mesmo nome e aponta para os arquivos com hash da versão atual. Se ficar em cache por um dia, os clientes vão continuar carregando a versão antiga do front durante um dia inteiro depois do deploy. Para ele, o cabeçalho certo é:

```
Cache-Control: no-cache
```

Apesar do nome, `no-cache` permite guardar uma cópia, desde que o cache confirme com a origem, a cada uso, se ela ainda vale. Quem proíbe guardar é o `no-store`, que é o que você quer para respostas com dados pessoais. Os nomes do HTTP têm essa tradição: é o

mesmo protocolo que, desde os anos 1990, escreve o cabeçalho `Referer` com um "r" a menos, porque o erro de grafia entrou na especificação e ficou.

Na AWS, a CDN se chama Amazon CloudFront, e o front do Rangô vai morar nela no capítulo 14.

Na prática: um nome e um cadeado para o Rangô local

Sem máquina com IP público, dá para montar quase tudo localmente: um nome apontando para o notebook e HTTPS com certificado confiável. Os pré-requisitos são o Rangô do capítulo anterior, o `curl`, o `openssl` e o `dig` (no Debian e no Ubuntu, pacote `dnsutils`). No Windows, os comandos rodam dentro do WSL, a maneira elegante que a Microsoft encontrou de dizer "tudo bem, vocês venceram" aos usuários de Linux.

Consultas de DNS com dig

O ponto de partida é uma consulta de DNS. O `dig`, aliás, é sigla de *Domain Information Groper*, algo como "apalpador de informações de domínio", um nome que nenhum departamento de marketing aprovaria hoje. Os exemplos abaixo usam `rango.com.br` como se ele fosse nosso, com IPs de documentação. Com qualquer domínio real, como `registro.br`, a saída tem o mesmo formato, só que com números reais:

```
dig rango.com.br A
```

```
;; QUESTION SECTION:
;rango.com.br.                  IN      A

;; ANSWER SECTION:
rango.com.br.           3600    IN      A       203.0.113.45

;; Query time: 38 msec
;; SERVER: 192.168.0.1#53(192.168.0.1) (UDP)
```

A saída foi encurtada. O `3600` é o TTL restante, em segundos. A linha `SERVER` mostra qual resolver respondeu (no caso, o roteador de casa, que repassa para o provedor) e confirma o UDP. O mesmo comando, executado de novo logo em seguida, mostra:

```
rango.com.br.           3421    IN      A       203.0.113.45

;; Query time: 1 msec
```

O TTL diminuiu e o tempo caiu para 1 ms. A resposta veio do cache. Com a opção `+trace`, o próprio `dig` faz o papel do resolver, começando pela raiz:

```
dig +trace rango.com.br
```

```
.                       518400  IN  NS  a.root-servers.net.
.                       518400  IN  NS  b.root-servers.net.
;; Received 239 bytes from 192.168.0.1#53(192.168.0.1) in 12 ms

br.                     172800  IN  NS  a.dns.br.
br.                     172800  IN  NS  b.dns.br.
;; Received 701 bytes from 198.41.0.4#53(a.root-servers.net) in 24 ms

rango.com.br.           3600    IN  NS  ns1.provedor-dns.example.
rango.com.br.           3600    IN  NS  ns2.provedor-dns.example.
;; Received 612 bytes from 200.219.148.10#53(a.dns.br) in 31 ms

rango.com.br.           3600    IN  A   203.0.113.45
;; Received 57 bytes from 198.51.100.53#53(ns1.provedor-dns.example) in 45 ms
```

Os quatro blocos são os passos da cadeia: raiz, delegação para `br`, delegação para os servidores do Rangô e a resposta com autoridade. Para perguntar direto ao autoritativo, pulando os caches, existe o `@` (é assim que você descobre se o problema está na zona ou no cache de alguém):

```
dig @ns1.provedor-dns.example rango.com.br A
```

Um nome para o notebook

Antes de consultar qualquer resolver, o sistema operacional olha um arquivo local, o `hosts`. Ele funciona como uma zona de DNS particular, que vale só para a sua máquina. No exemplo, ele ganha dois nomes para o Rangô: `rango.test` para o front e `api.rango.test` para a API. O domínio de topo `.test` é reservado justamente para testes (RFC 6761) e nunca vai existir na internet, então não há risco de colidir com um site real. É, provavelmente, o único domínio do mundo que nenhum especulador conseguiu comprar.

No macOS e no Linux, o arquivo é o `/etc/hosts`, editado com `sudo`. No Windows, ele fica em `C:\Windows\System32\drivers\etc\hosts`, e quem edita é o Bloco de Notas aberto como administrador, naquele que costuma ser o momento mais hacker da vida de muita gente. As linhas acrescentadas são:

```
127.0.0.1   rango.test
127.0.0.1   api.rango.test
```

O `dig` consulta o DNS diretamente e ignora o arquivo `hosts`, por isso ele não serve para conferir essa mudança. Quem confere é o Python, que pergunta ao sistema operacional como qualquer aplicação faria:

```
python3 -c "import socket; print(socket.gethostbyname('api.rango.test'))"
```

```
127.0.0.1
```

O HTTP em texto puro

Com o backend rodando como no capítulo 1, ainda sem TLS, o `curl -v` mostra a conversa inteira:

```
cd backend
uvicorn app.main:app --reload --port 8000
```

```
curl -v http://api.rango.test:8000/health
```

```
* Connected to api.rango.test (127.0.0.1) port 8000
> GET /health HTTP/1.1
> Host: api.rango.test:8000
> User-Agent: curl/8.7.1
> Accept: */*
>
< HTTP/1.1 200 OK
< content-type: application/json
< content-length: 15
<
{"status":"ok"}
```

As linhas com `>` são o que saiu, e as com `<` são o que voltou. O `Host` está lá, com o nome usado na URL. Se alguém capturasse esses pacotes, veria exatamente este texto. Neste caso, o espião levaria para casa um `{"status":"ok"}`, o segredo industrial menos valioso já interceptado.

Primeira tentativa: um certificado que ninguém assinou

A primeira tentativa de HTTPS é a ingênua, com um certificado **autoassinado**, gerado pela própria pessoa. Na raiz do projeto, os comandos criam uma pasta `certs`:

```
mkdir certs && cd certs
openssl req -x509 -newkey rsa:2048 -nodes -days 7 \
  -keyout autoassinado-key.pem -out autoassinado.pem \
  -subj "/CN=api.rango.test" \
  -addext "subjectAltName=DNS:api.rango.test"
cd ..
```

O comando gera uma chave privada e um certificado que diz "esta chave pertence a `api.rango.test`", assinado pela própria chave. A sintaxe do `openssl`, você deve ter reparado, parece ter sido projetada por alguém que recebia por opção de linha de comando. Com esse certificado, o Uvicorn sobe com TLS:

```
cd backend
uvicorn app.main:app --reload --port 8000 \
  --ssl-keyfile ../certs/autoassinado-key.pem \
  --ssl-certfile ../certs/autoassinado.pem
```

E a chamada de teste:

```
curl https://api.rango.test:8000/health
```

```
curl: (60) SSL certificate problem: self-signed certificate
```

No navegador, a mesma URL mostra uma página inteira de alerta, com o código `NET::ERR_CERT_AUTHORITY_INVALID`. A conexão é cifrada, mas nada prova quem está do outro lado: é um documento de identidade que você mesmo carimbou. Equivale a chegar na portaria de uma empresa com um crachá feito em casa e plastificado na papelaria da esquina. É esse alerta que o usuário veria se um intermediário tentasse se passar pelo Rangô.

O certificado autoassinado não volta a ser usado, então sai de cena:

```
rm certs/autoassinado.pem certs/autoassinado-key.pem
```

Sua própria autoridade certificadora com mkcert

Para desenvolvimento, a saída é criar uma CA que só o seu computador conhece e instalá-la no repositório de raízes da sua máquina. Ser dono de uma autoridade certificadora parece título de nobreza, mas o reino tem um único súdito: o seu notebook. O `mkcert`, uma ferramenta de código aberto, faz isso em dois comandos. Ele se instala pelo gerenciador de pacotes (`brew install mkcert` no macOS, `choco install mkcert` ou `scoop install mkcert` no Windows; no Linux, as instruções estão no repositório oficial, nas referências). Depois da instalação:

```
mkcert -install
```

O comando cria uma CA nova e a adiciona ao repositório de raízes do sistema (e do Firefox, se ele estiver instalado). O passo seguinte emite um certificado para os dois nomes do Rangô:

```
cd certs
mkcert rango.test api.rango.test
cd ..
```

O resultado são dois arquivos, `rango.test+1.pem` (o certificado) e `rango.test+1-key.pem` (a chave privada): um único certificado com os dois nomes no SAN. Antes de seguir, essa pasta precisa ficar fora do repositório. Chave privada não entra no git, nem a de desenvolvimento, nem aquela que "é só para testar". O GitHub está cheio de coisas que eram só para testar. Uma linha resolve:

```
echo "certs/" >> .gitignore
```

HTTPS no backend e no front

A API sobe com o certificado novo e com o CORS liberando a origem nova do front. Vale lembrar que a origem inclui o esquema: `https://rango.test:5173` e `http://rango.test:5173` são origens diferentes para o navegador.

```
cd backend
CORS_ORIGINS="https://rango.test:5173" \
  uvicorn app.main:app --reload --port 8000 \
  --ssl-keyfile ../certs/rango.test+1-key.pem \
  --ssl-certfile ../certs/rango.test+1.pem
```

```
curl https://api.rango.test:8000/health
```

```
{"status":"ok"}
```

Sem erro. O `curl` confiou no certificado porque a CA do `mkcert` está no repositório do sistema. Se a sua versão do `curl` usar outro repositório e ainda reclamar, a solução é apontar a CA explicitamente com `--cacert "$(mkcert -CAROOT)/rootCA.pem"`.

No front, o Vite passa a servir HTTPS com o mesmo certificado. A configuração fica em `frontend/vite.config.ts`:

```
// frontend/vite.config.ts
import fs from "node:fs";
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    host: "127.0.0.1",
    allowedHosts: ["rango.test"],
    https: {
      key: fs.readFileSync("../certs/rango.test+1-key.pem"),
      cert: fs.readFileSync("../certs/rango.test+1.pem"),
    },
  },
});
```

O `host: "127.0.0.1"` fixa a escuta no IPv4 para onde o arquivo `hosts` aponta; sem ele, o Node às vezes escuta só no IPv6 `::1`. O `allowedHosts` existe porque, desde o início de 2025, o Vite recusa requisições com um cabeçalho `Host` desconhecido, como proteção contra um ataque chamado *DNS rebinding*. Por padrão, ele aceita só `localhost` e IPs.

O `frontend/.env.local` passa a apontar para a API pelo nome, com HTTPS:

```
# frontend/.env.local
VITE_API_URL=https://api.rango.test:8000
```

Com o front rodando via `npm run dev`, o endereço `https://rango.test:5173` mostra o cadeado, e a lista com a Cantina da Nona e o Sabor do Sertão também. É um cadeado legítimo, com reconhecimento garantido num raio de um metro e meio do teclado. Se `VITE_API_URL` ficasse em HTTP, o navegador bloquearia as chamadas por *mixed content*: uma página HTTPS não pode buscar dados por HTTP.

O celular da rede de casa deixou de funcionar nessa configuração, porque não tem as entradas do `hosts` nem a CA do `mkcert`. O nome público e o certificado de uma CA reconhecida chegam na Parte II.

O certificado aberto pelo openssl

O `openssl s_client` faz o handshake e mostra o que veio do servidor:

```
openssl s_client -connect api.rango.test:8000 -servername api.rango.test </dev/null
2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

```
subject=O=mkcert development certificate, OU=voce@notebook
issuer=O=mkcert development CA, OU=voce@notebook, CN=mkcert voce@notebook
notBefore=Oct 15 13:02:11 2026 GMT
notAfter=Jan 15 13:02:11 2029 GMT
X509v3 Subject Alternative Name:
    DNS:rango.test, DNS:api.rango.test
```

Os valores exatos variam com o usuário e a data. O `-servername` é o SNI: é ele que diz ao servidor qual nome você quer. O `issuer` é a CA do `mkcert`, e o SAN tem os dois nomes. Com um site real no lugar do host e a porta 443, o mesmo comando mostra no `issuer` o nome de um intermediário de uma CA pública.

Armadilhas e casos reais

O TTL longo na véspera da migração

Situação hipotética, mas comum: o registro `A` do Rangô tem TTL de 86400 (um dia). Na sexta-feira à noite, você troca o IP e desliga a máquina antiga. (O primeiro erro já foi mexer em produção numa sexta-feira à noite, mas esse nem é culpa do DNS.) Nas próximas 24 horas, parte dos clientes continua chegando no IP antigo, onde não existe mais nada. O suporte recebe reclamações que você não consegue reproduzir, porque o seu resolver já pegou o valor novo.

> **Dica:** Um ou dois dias antes de uma migração, baixe o TTL do registro para 60 ou 300 segundos e espere o TTL antigo vencer. Faça a troca. Quando tudo estiver estável, suba o TTL de novo. E mantenha a máquina antiga respondendo por um

> tempo depois da troca.

Existe também o cache negativo: se você consultar `api.rango.com.br` antes de criar o registro, a resposta "esse nome não existe" também fica em cache, por um tempo definido no registro `SOA` da zona.

Quando o DNS some, tudo some

Em 4 de outubro de 2021, Facebook, Instagram e WhatsApp ficaram fora do ar por cerca de seis horas. Segundo o relato de engenharia da própria empresa, um comando numa manutenção de rotina derrubou o backbone que liga os data centers, e os servidores DNS, projetados para isso, pararam de anunciar suas rotas para a internet. Com isso, os servidores autoritativos de `facebook.com` ficaram inalcançáveis, e os resolvers do mundo inteiro passaram a não conseguir traduzir o nome. As aplicações estavam de pé, mas ninguém conseguia encontrá-las, e a falha ainda dificultou o acesso das equipes às ferramentas internas que usariam para corrigi-la.

Há um haicai famoso entre administradores de sistemas que resume dias assim: "Não pode ser o DNS. / Não tem como ser o DNS. / Foi o DNS."

A raiz que venceu e levou aparelhos antigos junto

Certificados raiz também vencem, como iogurte, só que sem o cheiro para avisar. Em 30 de setembro de 2021, expirou a raiz DST Root CA X3, que tinha assinado de forma cruzada a raiz do Let's Encrypt nos primeiros anos da CA. Sistemas atualizados já confiavam na raiz própria do Let's Encrypt. Clientes antigos (OpenSSL velho em servidores, aparelhos embarcados, celulares desatualizados) começaram a falhar com erro de certificado, sem nenhuma mudança no código.

A lição para o Rangô: o repositório de raízes de quem chama a sua API importa tanto quanto o certificado que você instala. No Python, `requests` e `httpx` usam por padrão o repositório do pacote `certifi`, separado do sistema operacional. Por isso um script Python pode rejeitar o certificado do `mkcert` mesmo com o navegador mostrando o cadeado.

O certificado que venceu num domingo

Certificado vencido é uma das causas mais comuns de indisponibilidade, quase sempre com a mesma história: alguém emitiu o certificado à mão, ele valia um ano, a pessoa saiu da empresa e o lembrete foi junto. Um lembrete no calendário pessoal de quem já pediu demissão é a bomba-relógio mais discreta já inventada.

> **Armadilha:** A renovação automática também falha, por exemplo quando alguém muda um redirecionamento e o desafio por HTTP quebra. Monitore o vencimento do certificado que o servidor está entregando, e não só o do arquivo no disco.

HSTS: fácil de ligar, difícil de desligar

Depois de redirecionar a porta 80 para a 443, o próximo passo é o cabeçalho `Strict-Transport-Security` (HSTS), que manda o navegador nunca mais acessar o domínio por HTTP. Ele protege bem, mas é difícil de desfazer: com um `max-age` de um ano, se o HTTPS quebrar, os navegadores que já o viram vão se recusar a abrir o site durante esse ano. Funciona como uma tatuagem: fácil de fazer numa sexta à noite, caro e demorado de remover. Comece com um valor curto e aumente aos poucos.

O arquivo da CA do mkcert

O `mkcert -install` deixa na pasta indicada por `mkcert -CAROOT` o arquivo `rootCA-key.pem`, a chave privada da sua CA local. Com ele, qualquer pessoa emite certificados para qualquer site, inclusive o do seu banco, que o seu navegador vai aceitar. Nunca o copie nem o coloque num repositório.

Resumo

- O DNS traduz nomes em IPs por uma hierarquia de delegações: raiz, domínio de topo, domínio. Cada nível só sabe quem cuida do nível de baixo.
- O resolver recursivo faz a cadeia de perguntas em nome do cliente. O servidor autoritativo responde com os registros da zona.
- O TTL define quanto tempo cada resposta fica em cache. "Propagação" é só o cache antigo expirando. Baixe o TTL antes de migrações.
- Os registros que importam agora são `A`, `AAAA`, `CNAME`, `MX`, `TXT` e `NS`. `CNAME` não pode ficar no apex.
- HTTP é texto. O cabeçalho `Host` permite vários sites num mesmo IP. HTTP/2 e HTTP/3 mudam o transporte, e não a semântica.
- TLS garante sigilo, integridade e autenticação. A troca de chaves cria o segredo, e o certificado prova quem está do outro lado.
- O certificado liga uma chave pública a nomes (SAN), assinado por uma CA. O navegador valida a cadeia até uma raiz do repositório local, a validade e o nome. O SNI diz qual certificado o servidor deve enviar.
- CAs validam o domínio por desafio HTTP ou DNS. Let's Encrypt e ACM emitem certificados gratuitos. Prazos máximos caem para 47 dias até 2029, e renovação

manual deixa de ser viável.

- Uma CDN guarda conteúdo na borda, perto do usuário, e termina o TLS ali. `Cache-Control` controla o cache: arquivos com hash por um ano, `index.html` com `no-cache`.
- O Rangô agora roda localmente em `https://rango.test:5173`, com a API em `https://api.rango.test:8000`, nomes no arquivo `hosts` e certificado do `mkcert`.

Próxima parada: o servidor que ninguém configurou

Agora você sabe o que precisa existir para a sua sócia digitar `rango.com.br` e ver o cadeado: um registro `A` apontando para um IP público e um certificado válido servido nesse IP. Falta o próprio servidor. E, ao pensar nele, começam a aparecer perguntas que o seu notebook escondia. Quem vai rodar o Uvicorn quando a máquina reiniciar às 4h da manhã? (Servidores, assim como bebês, têm um talento especial para escolher o pior horário.) Como servir o front e a API na porta 443, que só o administrador pode abrir, sem rodar a aplicação inteira com privilégios de administrador? Onde fica o certificado, e quem renova?

E tem a pergunta mais traiçoeira. Você instala Python, as dependências e o Postgres no servidor, sobe o Rangô, e a primeira requisição devolve um erro 500 que nunca apareceu no seu notebook: uma biblioteca em outra versão, uma variável de ambiente esquecida. Na sua máquina funciona. No servidor, não. No próximo capítulo, vamos ver o que é um servidor por dentro, o que um proxy reverso faz na frente da aplicação e por que os containers viraram a resposta padrão para esse "na minha máquina funciona".

Referências

- RFC 1034, *Domain Names: Concepts and Facilities*: https://www.rfc-editor.org/rfc/rfc1034
- RFC 1035, *Domain Names: Implementation and Specification*: https://www.rfc-editor.org/rfc/rfc1035
- RFC 2308, cache negativo de consultas DNS: https://www.rfc-editor.org/rfc/rfc2308
- RFC 6761, nomes de domínio de uso especial (`.test`): https://www.rfc-editor.org/rfc/rfc6761
- Servidores raiz do DNS: https://root-servers.org/
- Registro.br: https://registro.br/
- RFC 9110, *HTTP Semantics*: https://www.rfc-editor.org/rfc/rfc9110
- RFC 9111, *HTTP Caching*: https://www.rfc-editor.org/rfc/rfc9111

- RFC 8446, *The Transport Layer Security (TLS) Protocol Version 1.3*: https://www.rfc-editor.org/rfc/rfc8446
- RFC 8555, protocolo ACME: https://www.rfc-editor.org/rfc/rfc8555
- RFC 6797, HTTP Strict Transport Security: https://www.rfc-editor.org/rfc/rfc6797
- Let's Encrypt, como funciona: https://letsencrypt.org/how-it-works/
- Let's Encrypt, expiração da raiz DST Root CA X3: https://letsencrypt.org/docs/dst-root-ca-x3-expiration-september-2021/
- CA/Browser Forum, Baseline Requirements: https://cabforum.org/working-groups/server/baseline-requirements/
- Facebook Engineering, detalhes da indisponibilidade de 4 de outubro de 2021: https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
- Cloudflare, análise da mesma indisponibilidade: https://blog.cloudflare.com/october-2021-facebook-outage/
- mkcert (repositório oficial): https://github.com/FiloSottile/mkcert
- Uvicorn, opções de TLS (`--ssl-keyfile`, `--ssl-certfile`): https://www.uvicorn.org/settings/
- Vite, opções do servidor (`server.https`, `server.allowedHosts`): https://vite.dev/config/server-options
- MDN, `Cache-Control`: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cache-Control

CAPÍTULO 3

Servidores, containers e Kubernetes

No capítulo anterior, você viu como o DNS transforma `rango.com.br` num IP e como o TLS prova que o navegador está falando com o servidor certo. O Rangô ganhou nomes locais (`rango.test` e `api.rango.test`) e um certificado do `mkcert`. Faltou o servidor: uma máquina que fique ligada, rode o Uvicorn sozinha e sirva o Rangô na porta 443.

Sexta-feira, 19h, primeiro deploy

Você decide que chegou a hora. Sexta-feira às 19h é, reconhecidamente, o horário oficial das más decisões em TI, mas ninguém tinha avisado você. Aluga um servidor Linux num provedor qualquer (a AWS fica para a Parte II), recebe um IP público, `203.0.113.45`, e um usuário com acesso por SSH. Entra na máquina, clona o repositório e repete o que sempre fez no notebook:

```
git clone https://github.com/seu-usuario/rango.git
cd rango/backend
python3 -m venv .venv
source .venv/bin/activate
pip install "fastapi[standard]"
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

O Uvicorn sobe sem reclamar. Você abre `http://203.0.113.45:8000/health` no navegador e recebe `{"status": "ok"}`. Ótimo. Abre `/restaurantes` e recebe isto:

```
Internal Server Error
```

Três palavras capazes de estragar um fim de semana inteiro. No notebook, essa mesma rota funciona há uma semana. Na semana passada, a sua sócia pediu que a taxa de entrega aparecesse já formatada em reais, e você resolveu do jeito mais rápido (que, como todo mundo aprende cedo ou tarde, costuma ser o que mais demora), usando o módulo `locale` da biblioteca padrão:

```
# backend/app/main.py
import locale
import os

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
```

```
app = FastAPI(title="Rangô")

origens = os.getenv("CORS_ORIGINS", "http://localhost:5173").split(",")
app.add_middleware(
    CORSMiddleware,
    allow_origins=origens,
    allow_methods=["GET"],
    allow_headers=["*"],
)

RESTAURANTES = [
    {"id": 1, "nome": "Cantina da Nona", "cozinha": "italiana", "taxa_entrega": 6.9},
    {"id": 2, "nome": "Sabor do Sertão", "cozinha": "nordestina", "taxa_entrega": 4.5},
]


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/restaurantes")
def listar_restaurantes():
    locale.setlocale(locale.LC_MONETARY, "pt_BR.UTF-8")
    return [
        {**r, "taxa_entrega_fmt": locale.currency(r["taxa_entrega"], grouping=True)}
        for r in RESTAURANTES
    ]
```

No terminal do servidor, o Uvicorn mostra o motivo:

```
  File "/home/deploy/rango/backend/app/main.py", line 31, in listar_restaurantes
    locale.setlocale(locale.LC_MONETARY, "pt_BR.UTF-8")
  ...
locale.Error: unsupported locale setting
```

O seu notebook tem o locale `pt_BR.UTF-8` instalado porque o sistema dele está em português. O servidor veio com uma instalação mínima, em inglês, e ninguém instalou esse locale. O código Python é idêntico nas duas máquinas. O que mudou foi tudo ao redor dele.

E há mais. O `pip install` baixou versões mais novas das bibliotecas, porque você nunca fixou versões. A variável `CORS_ORIGINS` não existe no servidor. E, quando você fecha o terminal para ir jantar, o Uvicorn morre junto com a sessão SSH. Solidário, ele também encerra o expediente.

"Na minha máquina funciona" tem um sentido bem literal: a aplicação depende de muita coisa que não está no repositório. Versão do Python, bibliotecas do sistema, locale, variáveis de ambiente, até o processo que mantém o programa rodando. Vamos primeiro entender o que existe num servidor e, depois, empacotar o ambiente junto com o código.

O que é um servidor, afinal

Servidor é um computador cuja função é atender requisições de outros computadores. Fisicamente, lembra o seu notebook. A diferença está no uso: ele fica num data center, ligado o tempo todo, com energia redundante e um link de rede rápido, e ninguém se senta na frente dele. Em compensação, ninguém derruba café no teclado dele.

Ter um servidor já significou comprar uma máquina física e instalá-la num rack, de preferência descobrindo, na hora de parafusar, que o trilho que veio na caixa era de outro modelo. Isso ainda existe (o chamado *bare metal*), mas hoje a maior parte das aplicações roda em **máquinas virtuais**.

Máquinas virtuais

Uma máquina virtual (VM) é um computador simulado em software, rodando dentro de um computador físico. Um programa chamado **hypervisor** divide os recursos da máquina física entre várias VMs: cada uma recebe uma fatia de CPU, de memória e de disco, e enxerga um hardware virtual como se fosse só dela. Cada VM roda o seu próprio sistema operacional completo, com o seu próprio kernel.

Se você já usou o VirtualBox, já usou uma VM, talvez para instalar um Linux "só para ver como é" e nunca mais abrir. Os provedores de nuvem fazem o mesmo em escala industrial: um servidor físico com dezenas de núcleos é fatiado em VMs alugadas para clientes diferentes. Quando você "aluga um servidor" de 2 vCPUs e 4 GB, recebe uma dessas fatias. Ela nasce em minutos, pode ser destruída quando não servir mais, e o hypervisor impede que um cliente enxergue a memória do outro. É um prédio de apartamentos em que as paredes, felizmente, são bem mais grossas que as do seu. No capítulo 9, você vai criar exatamente uma VM dessas na AWS, chamada instância EC2.

O mínimo de Linux para rodar uma aplicação

Quase todo servidor web roda Linux. Você não precisa virar administrador de sistemas, nem decorar as opções do `tar`, façanha que, segundo a lenda, ninguém alcançou até hoje. Mas precisa conhecer algumas peças para entender o que deu errado na sexta-feira à noite.

Distribuições, pacotes e onde as coisas ficam

O Linux propriamente dito é só o kernel: a parte que conversa com o hardware, gerencia memória, processos e rede. (Neste exato momento, algum leitor já começou a escrever um e-mail para explicar que o certo é dizer GNU/Linux.) O que você instala é uma **distribuição**, que junta o kernel com ferramentas, bibliotecas e um gerenciador de

pacotes. Ubuntu e Debian usam `apt`. Amazon Linux e Red Hat usam `dnf`. O papel do gerenciador de pacotes é parecido com o do `pip`, só que para o sistema inteiro: `sudo apt install nginx` instala o Nginx e tudo de que ele depende.

Isso já explica parte do problema. O Ubuntu 24.04 traz o Python 3.12; o Ubuntu 22.04 traz o 3.10. Locales e bibliotecas em C usadas por pacotes Python também variam de uma instalação para outra.

Os diretórios seguem uma convenção: configuração em `/etc`, logs em `/var/log`, programas instalados à mão em `/opt` ou `/srv`, pastas dos usuários em `/home`. E o `/tmp`, para arquivos temporários, que é para onde vai tudo o que alguém jurou que apagaria depois.

Usuários e permissões

No Linux, todo arquivo e todo processo pertencem a um usuário. O usuário `root` pode tudo: apagar qualquer arquivo, abrir qualquer porta, matar qualquer processo. O comando `sudo` executa um comando com os poderes do `root`.

Lembra das portas privilegiadas do capítulo 1, abaixo de 1024? No Linux, só o `root` (ou um processo com uma permissão específica, a `CAP_NET_BIND_SERVICE`) pode escutar nelas. As portas 80 e 443 estão nessa faixa. A saída preguiçosa seria rodar o Uvicorn como `root`. É uma saída bem popular: a frase "roda com sudo que funciona" já resolveu e causou mais problemas do que qualquer outra na história do Linux. Só que aí qualquer falha na sua aplicação, qualquer dependência comprometida, roda com permissão para fazer qualquer coisa na máquina. A regra é a mesma do código: cada parte com o mínimo de permissão de que precisa. A aplicação roda com um usuário próprio, sem privilégios, numa porta alta como a 8000. Daqui a pouco você vai ver quem fica na porta 443.

Processos e sinais

Cada programa em execução é um **processo**, identificado por um número, o PID. O comando `ps aux` lista todos eles. Um processo pode receber **sinais**, que são avisos do sistema operacional. Os que importam para você:

Sinal	Quando chega	O que costuma acontecer
`SIGTERM`	Alguém pediu para o processo encerrar (`kill PID`)	O processo termina o que está fazendo e sai. O Uvicorn conclui as requisições em andamento antes de sair.
`SIGKILL`	Encerramento forçado (`kill -9 PID`)	O kernel mata o processo na hora, sem chance de limpeza.

Sinal	Quando chega	O que costuma acontecer
`SIGHUP`	O terminal que controlava o processo foi fechado	Por padrão, o processo morre.
`SIGINT`	Você apertou Ctrl+C	Parecido com o `SIGTERM`.

O `SIGTERM` é um pedido educado para sair; o `SIGKILL` é a versão digital de puxar a cadeira de alguém. O `SIGHUP` explica por que o Uvicorn morreu quando você fechou o SSH: o processo estava preso ao seu terminal. O nome vem de *hang up*, "desligar o telefone", do tempo em que terminais se conectavam por linha telefônica e a ligação caindo levava os programas junto. Dá para contornar com `nohup` ou `tmux`, mas, se o processo cair às 3h ou a máquina reiniciar, ninguém sobe o Uvicorn de novo.

Serviços com systemd

Quem resolve isso é o gerenciador de serviços. Na maioria das distribuições atuais, ele se chama **systemd**: o primeiro processo a subir quando a máquina liga (o PID 1), que inicia, para, reinicia e registra os logs dos outros serviços. Ele também é, ao lado da eterna briga entre Vim e Emacs, um dos assuntos que mais rendem discussões acaloradas em fóruns de Linux.

O serviço é descrito num arquivo de unidade. Para o Rangô, ficaria assim:

```
# /etc/systemd/system/rango-api.service
[Unit]
Description=API do Rangô (Uvicorn)
After=network.target

[Service]
User=rango
Group=rango
WorkingDirectory=/srv/rango/backend
EnvironmentFile=/etc/rango/api.env
ExecStart=/srv/rango/backend/.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
--workers 2
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

O serviço roda com o usuário `rango`, sem privilégios. As variáveis de ambiente vêm de `/etc/rango/api.env`, fora do repositório. O Uvicorn escuta só em `127.0.0.1`, então ninguém de fora acessa a porta 8000. E `Restart=on-failure` sobe o processo de novo se ele cair. Persistente assim, só corretor de imóveis com o seu número de telefone.

A ativação fica assim:

```
sudo systemctl daemon-reload
sudo systemctl enable --now rango-api
systemctl status rango-api
journalctl -u rango-api -f
```

O `enable` faz o serviço subir junto com a máquina; o `--now` já o inicia; o `journalctl` mostra os logs. Repare que o arquivo é declarativo: você diz o que quer que esteja rodando, e o systemd mantém assim. Guarde essa ideia, porque ela volta bem maior no fim do capítulo.

Proxy reverso: quem atende a porta 443

Sobrou uma pergunta da ponte do capítulo 2: se o Uvicorn roda sem privilégios na porta 8000, quem atende na 443, com o certificado?

A resposta padrão é um **proxy reverso**. É um servidor que fica na frente da aplicação, recebe as conexões da internet e repassa cada requisição para o processo certo lá atrás. O mais usado é o **Nginx**, que se pronuncia "engine-x", embora muita gente diga "en-ginx" com uma convicção admirável. Na mesma máquina, o desenho fica assim:

```
Internet
   │  HTTPS (porta 443)
   ▼
┌──────────────────────────────┐
│ Nginx                        │
│  - termina o TLS             │
│  - rango.com.br     → arquivos do React (dist/)
│  - api.rango.com.br → 127.0.0.1:8000
└──────────────────────────────┘
   │  HTTP local
   ▼
Uvicorn (usuário rango, porta 8000)
```

Em termos de FastAPI, o proxy reverso lembra um middleware que roda antes da sua aplicação, só que num processo separado. Ele termina o TLS, então o certificado fica com o Nginx e o Uvicorn recebe HTTP simples pela interface local. Roteia pelo cabeçalho `Host` do capítulo 2: `rango.com.br` serve os arquivos estáticos do build do React direto do disco, `api.rango.com.br` vai para o Uvicorn. E protege a aplicação de clientes lentos: um celular com sinal ruim (aquela barrinha solitária dentro do elevador) pode levar segundos para enviar a requisição, e o Nginx recebe tudo antes de repassar, sem prender um worker do Uvicorn. É a recepcionista que ouve a história inteira do cliente antes de incomodar o médico.

E a porta privilegiada? O processo principal do Nginx começa como `root`, abre as portas 80 e 443 e cria processos de trabalho que rodam como um usuário sem privilégios (`www-data`, no Ubuntu). Quem processa as requisições nunca tem poderes de administrador. O

dono abre a loja de manhã e vai embora; quem atende no balcão não tem a chave do cofre.

A configuração para o Rangô num servidor de verdade seria algo assim:

```
# /etc/nginx/sites-available/rango
server {
    listen 80;
    server_name rango.com.br api.rango.com.br;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name rango.com.br;
    ssl_certificate     /etc/letsencrypt/live/rango.com.br/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/rango.com.br/privkey.pem;

    root /srv/rango/frontend/dist;
    location / {
        try_files $uri /index.html;
    }
}

server {
    listen 443 ssl;
    server_name api.rango.com.br;
    ssl_certificate     /etc/letsencrypt/live/rango.com.br/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/rango.com.br/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

O primeiro bloco redireciona HTTP para HTTPS. O `try_files` faz rotas desconhecidas caírem no `index.html`, que é o que uma SPA precisa. Os cabeçalhos `X-Forwarded-*` existem porque, para o Uvicorn, toda requisição vem de `127.0.0.1`; sem eles, a aplicação perderia o IP real do cliente e não saberia que a conexão original era HTTPS. O Uvicorn só confia nesses cabeçalhos quando vêm de um endereço autorizado (por padrão, `127.0.0.1`), para que um cliente qualquer não forje o próprio IP.

E o certificado? O Certbot obtém um certificado do Let's Encrypt pelo desafio HTTP do capítulo 2 e agenda a renovação automática. As perguntas da ponte anterior estão respondidas: o systemd sobe o Uvicorn depois de um reboot, o Nginx atende a 443 sem dar privilégios à aplicação, e o Certbot renova o certificado.

Falta a pergunta mais difícil.

Voltando ao "na minha máquina funciona"

O systemd e o Nginx resolvem como a aplicação roda, e não o que ela encontra quando roda. O erro de locale continua lá. Você poderia instalar o locale no servidor e anotar isso num README, aquele arquivo que todo mundo escreve com carinho e ninguém lê. Poderia fixar as bibliotecas com `pip freeze`, o que ajuda. Mas o Python, as bibliotecas do sistema e os locales continuam dependendo de quem configurou cada máquina: o seu Mac, o Windows do colega novo, o servidor do CI, o de produção. Com o tempo, elas se afastam umas das outras.

A saída é entregar o código já dentro do ambiente, em vez de entregar o código e uma lista de instruções. Um pacote com o Python na versão certa, as bibliotecas do sistema, as dependências fixadas e a sua aplicação. Esse pacote é uma **imagem de container**. É a diferença entre mandar a receita do bolo e mandar o bolo.

Containers

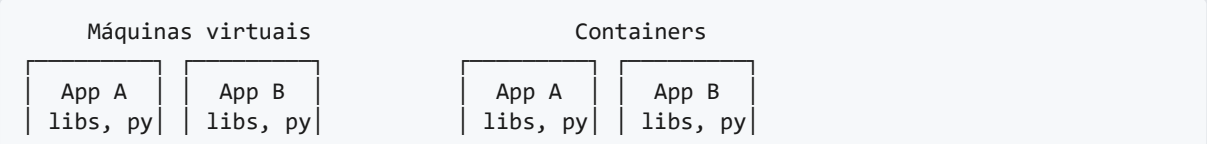
Um **container** é um processo comum do Linux que roda isolado do resto da máquina. Ele enxerga o próprio sistema de arquivos, a própria árvore de processos, a própria interface de rede e o próprio `localhost`. Do lado de dentro, parece que ele está sozinho num computador. Do lado de fora, é só mais um processo na lista do `ps`. É um processo com síndrome de protagonista.

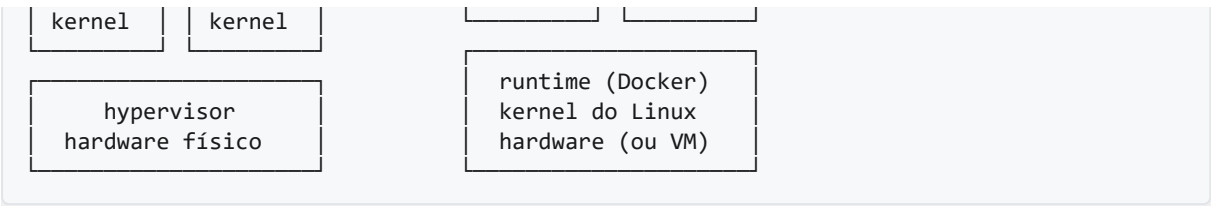
Esse isolamento vem de dois recursos do kernel do Linux:

- **Namespaces** limitam o que o processo enxerga. Com um namespace de processos, o container acha que o seu programa é o PID 1 e não vê os outros processos da máquina. Com um namespace de rede, ele tem as próprias interfaces e as próprias portas. Com um namespace de montagem, ele vê o próprio sistema de arquivos, e não o da máquina.
- **cgroups** (*control groups*) limitam o quanto o processo consome: no máximo 512 MB de memória, no máximo meio núcleo de CPU, e assim por diante.

Container e máquina virtual

A diferença para uma VM está no kernel. Cada VM roda um sistema operacional completo, com o seu próprio kernel. Todos os containers de uma máquina compartilham o kernel dela:





Sem kernel próprio, um container sobe em fração de segundo e ocupa megabytes, enquanto uma VM leva de segundos a minutos e ocupa gigabytes. O preço é um isolamento mais fraco: uma falha grave no kernel afeta todos os containers da máquina. VMs são casas geminadas, cada uma com a sua fundação; containers são quartos numa república, dividindo a mesma cozinha e o mesmo encanamento. Na nuvem, o normal é usar os dois juntos, com containers dentro de VMs.

E no seu Mac ou no Windows, que não usam o kernel do Linux? O Docker Desktop cria uma VM Linux pequena e roda os containers dentro dela. Você não vê essa VM, mas ela está lá, e a ventoinha do notebook faz questão de lembrar.

Imagem, container e registry

Três palavras aparecem o tempo todo, e vale separar bem cada uma.

Uma **imagem** é o pacote: um sistema de arquivos com tudo de que a aplicação precisa, mais alguns metadados (qual comando rodar, qual usuário usar, quais variáveis definir). Ela é somente leitura. Um **container** é uma imagem em execução. A relação é a mesma entre classe e instância em Python: a partir de uma imagem, você cria quantos containers quiser, e cada um tem o seu próprio estado.

A imagem é montada em **camadas**. Cada instrução do arquivo de construção gera uma camada com as mudanças em relação à anterior, mais ou menos como cada commit no git guarda uma diferença em relação ao commit pai. Camadas iguais são reaproveitadas entre imagens e entre builds, o que deixa a construção rápida quando só o seu código muda.

Um **registry** é um repositório de imagens, como o PyPI é para pacotes Python. O Docker Hub é o mais conhecido; dele vem a imagem oficial `python`. Você constrói a imagem uma vez, envia com `docker push`, e qualquer servidor baixa a mesma imagem com `docker pull`. É o "write once, run anywhere" que o Java prometeu nos anos 1990, agora entregue por uma baleia. Na AWS, o registry se chama Amazon ECR, e ele aparece no capítulo 17.

O formato das imagens é padronizado pela OCI (*Open Container Initiative*), então Docker, Podman e containerd rodam as mesmas imagens. Padrão aberto é isso: as empresas brigam por todo o resto, mas concordam no formato da caixa.

Na prática: o Rangô num container

Os pré-requisitos são o Docker instalado (Docker Desktop no macOS e no Windows, ou Docker Engine no Linux) com o plugin Compose, que traz o comando `docker compose`. Os comandos `docker version` e `docker compose version` confirmam a instalação.

Fixando as dependências

O primeiro passo congela as versões que funcionam no notebook. É o tipo de congelamento que dá certo, ao contrário do congelamento de preços do Plano Cruzado:

```
cd backend
source .venv/bin/activate
pip freeze > requirements.txt
```

O `requirements.txt` agora lista cada pacote com a versão exata, incluindo as dependências das dependências. Ele vai para o repositório.

O Dockerfile

A receita da imagem fica num arquivo chamado `Dockerfile`, na pasta `backend`:

```
# backend/Dockerfile
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

RUN useradd --create-home rango
USER rango

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Linha a linha:

1. `FROM python:3.12-slim` parte da imagem oficial do Python 3.12 sobre um Debian enxuto. A versão do Python deixa de depender da máquina.
2. As variáveis de ambiente evitam arquivos `.pyc` dentro da imagem e fazem os logs saírem na hora, sem buffer.
3. `COPY requirements.txt` vem antes de `COPY app` de propósito. As dependências mudam pouco, e o código muda o tempo todo. Com essa ordem, a camada do `pip`

`install` fica em cache e só é refeita quando o `requirements.txt` muda. Um build que levaria um minuto passa a levar poucos segundos.

4. `useradd` e `USER rango` fazem a aplicação rodar sem ser `root` dentro do container, pelo mesmo motivo do systemd.
5. `CMD` define o comando de inicialização. O `--host 0.0.0.0` é obrigatório aqui, e o capítulo 1 explica por quê: o container tem o próprio `localhost`. Se o Uvicorn escutar em `127.0.0.1`, só o próprio container consegue falar com ele.

O projeto ganha também um `backend/.dockerignore`, que funciona como um `.gitignore` para o build:

```
.venv
__pycache__
.env
```

Sem ele, o Docker enviaria a sua `.venv` inteira para o build (e toda `.venv` que se preze pesa mais do que o projeto), e um `.env` com segredos poderia acabar dentro da imagem.

Construindo e rodando

```
docker build -t rango-api:0.1.0 .
docker run --rm -p 127.0.0.1:8000:8000 \
  -e CORS_ORIGINS="https://rango.test:5173" \
  rango-api:0.1.0
```

O `-t` dá nome e versão à imagem. No `docker run`, o `-p 127.0.0.1:8000:8000` liga a porta 8000 do seu notebook à porta 8000 do container, aceitando conexões só da sua máquina (lembra da armadilha do capítulo 1, do Docker furando o firewall? Este é o jeito de evitá-la). O `-e` passa a variável de ambiente, e o `--rm` apaga o container quando ele parar. Sem ele, containers parados vão se acumulando em silêncio, como potes de plástico sem tampa no armário da cozinha.

Em outro terminal, duas chamadas:

```
curl http://localhost:8000/health
curl http://localhost:8000/restaurantes
```

O `/health` responde `{"status":"ok"}`. O `/restaurantes` devolve `Internal Server Error`. No terminal do container:

```
  File "/app/app/main.py", line 31, in listar_restaurantes
    locale.setlocale(locale.LC_MONETARY, "pt_BR.UTF-8")
  ...
locale.Error: unsupported locale setting
```

Este é o momento mais importante do capítulo. Se você sentiu vontade de comemorar um erro 500, seja bem-vindo ao DevOps. O erro que só acontecia no servidor agora acontece no seu notebook. A imagem `python:3.12-slim` não traz o locale `pt_BR`, assim como o servidor de sexta-feira. Só que agora o ambiente é o mesmo em todo lugar, então o bug aparece onde você desenvolve, e não em produção às 19h.

Corrigindo o bug de verdade

Daria para instalar o locale no Dockerfile. Só que o `setlocale` tem outro problema: ele altera uma configuração global do processo, compartilhada por todas as threads, e o FastAPI roda rotas síncronas em várias threads ao mesmo tempo. Formatar moeda não precisa de nada disso:

```
# backend/app/main.py (trecho alterado; remova também o "import locale")
def formatar_reais(valor: float) -> str:
    texto = f"{valor:,.2f}"  # 1,234.50
    return "R$ " + texto.replace(",", "_").replace(".", ",").replace("_", ".")


@app.get("/restaurantes")
def listar_restaurantes():
    return [
        {**r, "taxa_entrega_fmt": formatar_reais(r["taxa_entrega"])}
        for r in RESTAURANTES
    ]
```

É a velha dança de trocar vírgula por ponto e ponto por vírgula, a mesma que todo programador brasileiro já fez pelo menos uma vez, normalmente depois de uma planilha importada com valores multiplicados por mil. A imagem é reconstruída com uma versão nova e roda de novo:

```
docker build -t rango-api:0.1.1 .
docker run --rm -p 127.0.0.1:8000:8000 \
  -e CORS_ORIGINS="https://rango.test:5173" \
  rango-api:0.1.1
```

Repare na saída do build: as camadas até o `pip install` aparecem como `CACHED`. Só a cópia do código foi refeita. E agora:

```
curl http://localhost:8000/restaurantes
```

```
[{"id":1,"nome":"Cantina da
Nona","cozinha":"italiana","taxa_entrega":6.9,"taxa_entrega_fmt":"R$ 6,90"},
{"id":2,"nome":"Sabor do
Sertão","cozinha":"nordestina","taxa_entrega":4.5,"taxa_entrega_fmt":"R$ 4,50"}]
```

API e proxy reverso juntos com Docker Compose

Rodar um container com um `docker run` comprido é aceitável. Rodar dois, com rede entre eles, pede o **Docker Compose**: um arquivo YAML que descreve vários containers e como eles se ligam. YAML é aquele formato em que um espaço a mais na indentação muda o significado do arquivo e ninguém na sala consegue enxergar a diferença. Vamos colocar um Nginx na frente da API, do mesmo jeito que faríamos num servidor, usando o certificado do `mkcert` gerado no capítulo 2.

A configuração do Nginx fica em `nginx/rango.conf`, na raiz do projeto:

```
# nginx/rango.conf
server {
    listen 443 ssl;
    server_name api.rango.test;
    ssl_certificate     /etc/nginx/certs/rango.test+1.pem;
    ssl_certificate_key /etc/nginx/certs/rango.test+1-key.pem;

    location / {
        proxy_pass http://api:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

E o `compose.yaml`, também na raiz:

```
# compose.yaml
services:
  api:
    build: ./backend
    image: rango-api:0.1.1
    environment:
      CORS_ORIGINS: "https://rango.test:5173"
    restart: unless-stopped

  proxy:
    image: nginx:1.27-alpine
    ports:
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/rango.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on:
      - api
    restart: unless-stopped
```

O serviço `api` não publica nenhuma porta: só o Nginx fala com o Uvicorn. O Compose cria uma rede interna em que cada serviço é encontrado pelo nome, por isso o `proxy_pass` aponta para `http://api:8000`. Os certificados entram como volume somente leitura (`:ro`), fora da imagem. E `restart: unless-stopped` faz o papel do `Restart=on-failure` do systemd.

Se o Uvicorn da API ainda estiver rodando com `--ssl-keyfile` do capítulo anterior, ou o container do teste anterior estiver ativo, os dois precisam ser parados antes. Depois:

```
docker compose up -d
docker compose ps
curl --cacert "$(mkcert -CAROOT)/rootCA.pem" https://api.rango.test:8443/restaurantes
```

O `--cacert` diz ao `curl` para confiar na CA local do `mkcert`, já que nem todo `curl` consulta o mesmo repositório de certificados que o navegador. A resposta é a mesma lista de antes, agora passando pelo Nginx com TLS. Os logs dos dois serviços juntos aparecem com `docker compose logs -f`.

Falta avisar o front. Em `frontend/.env.local`:

```
VITE_API_URL=https://api.rango.test:8443
```

Depois de reiniciar o `npm run dev`, o endereço `https://rango.test:5173` mostra o React buscando os restaurantes pelo Nginx, com o cadeado no lugar. O Uvicorn deixou de cuidar do TLS e, como todo funcionário que perde uma responsabilidade, fingiu que não se importou: o certificado agora mora no proxy, como vai morar em produção.

Quando um servidor vira dez: orquestração

Com o Compose, o Rangô roda igual em qualquer máquina que tenha Docker. Agora pense um pouco mais adiante. O Rangô cresce, e um servidor não basta. Você passa a ter dez servidores e trinta containers da API. Surgem perguntas novas:

- Em qual servidor cada container deve rodar, considerando a CPU e a memória livres de cada um?
- Se um servidor desligar, quem recria os containers dele em outro lugar?
- Como trocar a versão 0.1.1 pela 0.1.2 sem tirar o Rangô do ar?
- Como o Nginx sabe os endereços dos containers da API, se eles mudam a cada recriação?

Responder isso com scripts e SSH é possível, e é um caminho garantido para incidentes. Quem já fez isso costuma contar a história em voz baixa, olhando para o chão. Um **orquestrador** de containers é o sistema que assume essas decisões. Você não diz a ele "rode este container naquele servidor"; você diz "quero três cópias da API na versão 0.1.1, cada uma com 128 MB de memória", e ele trabalha continuamente para que a realidade fique igual ao que você pediu.

Se essa ideia lembrou o React, a comparação é boa. No React, você não manipula o DOM: você declara como a tela deve ficar a partir do estado, e o React calcula as diferenças e aplica as mudanças. Um orquestrador faz isso com infraestrutura. Você declara o **estado desejado**, ele observa o **estado atual** e age para aproximar um do outro. Esse ciclo se chama **reconciliação**, e ele nunca para: se um container morrer às 3h, o estado atual fica com duas cópias, o desejado continua pedindo três, e o orquestrador cria outra.

O orquestrador mais usado no mercado é o **Kubernetes**, abreviado como K8s (o 8 conta as oito letras entre o K e o s, porque programador abrevia até quando a economia é de sete caracteres). O nome vem do grego e quer dizer timoneiro, quem pilota o navio, o que explica o leme no logotipo. Ele nasceu no Google, inspirado no sistema interno que a empresa usava para rodar os próprios serviços, e hoje é mantido pela Cloud Native Computing Foundation.

Kubernetes em quatro conceitos

O Kubernetes tem dezenas de tipos de objeto. Para entender como ele funciona, quatro bastam por enquanto.

Um **cluster** é o conjunto de máquinas gerenciadas pelo Kubernetes. As máquinas que rodam os seus containers são os **nós** (*nodes*). Um grupo separado de componentes, a **control plane**, guarda o estado desejado, decide onde cada coisa roda e executa os ciclos de reconciliação. Você fala com a control plane pela linha de comando, o `kubectl`, enviando arquivos YAML. A pronúncia de `kubectl` ("kube-control", "kube-c-t-l", "kube-cuddle") é tema de um debate que a comunidade nunca encerrou, e provavelmente nunca vai encerrar.

Um **pod** é a menor unidade que o Kubernetes executa. Em geral, um pod contém um container, mas pode conter alguns que precisam ficar juntos e compartilhar rede e disco. O nome vem de *pod of whales*, um grupo de baleias, numa homenagem à baleia do logotipo do Docker. Cada pod recebe um IP próprio dentro do cluster. Pods são descartáveis: o Kubernetes os mata e os recria o tempo todo, em outros nós e com outros IPs. Você quase nunca cria um pod diretamente.

Um **deployment** é o que você cria no lugar. Ele declara qual imagem rodar, quantas réplicas manter e como trocar de versão. Quando você muda a imagem de `0.1.1` para `0.1.2`, o deployment faz um *rolling update*: sobe pods novos aos poucos e só derruba os antigos quando os novos estão prontos. É trocar o pneu com o carro andando, com a diferença de que aqui funciona. O capítulo 6 volta a essa e a outras estratégias de deploy.

Um **service** resolve o problema dos IPs que mudam. Ele dá um nome e um IP estáveis para um grupo de pods, escolhido por rótulos (*labels*), e distribui as requisições entre eles. O Nginx, ou qualquer outro pod, chama `http://rango-api` e o service entrega para um dos pods vivos naquele momento. Funciona como uma interface estável na frente de implementações que vêm e vão. É o telefone fixo da pizzaria: os entregadores mudam todo dia, mas o número para pedir continua o mesmo.

Existem muitos outros objetos (Ingress para tráfego externo, ConfigMap e Secret para configuração, por exemplo), mas com esses quatro você já consegue ler a maior parte dos arquivos de um projeto real.

Na prática: o Rangô num cluster local

Para experimentar sem gastar nada, existe o **kind** (*Kubernetes in Docker*), que cria um cluster inteiro dentro de containers no seu notebook. É container rodando Kubernetes que roda containers, e se isso deu um nó na sua cabeça, é porque você entendeu. Os pré-requisitos são Docker, `kind` e `kubectl` instalados (as páginas oficiais têm instaladores para cada sistema operacional).

O primeiro passo cria o cluster e carrega nele a imagem construída antes, já que o cluster não enxerga as imagens locais do seu Docker:

```
kind create cluster --name rango
kind load docker-image rango-api:0.1.1 --name rango
```

O estado desejado fica descrito em `k8s/api.yaml`, na raiz do projeto:

```yaml
# k8s/api.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rango-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: rango-api
  template:
    metadata:
      labels:
        app: rango-api
    spec:
      containers:
        - name: api
          image: rango-api:0.1.1
          ports:
            - containerPort: 8000
          env:
            - name: CORS_ORIGINS
              value: "https://rango.test:5173"
          readinessProbe:
```

```
            httpGet:
              path: /health
              port: 8000
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: rango-api
spec:
  selector:
    app: rango-api
  ports:
    - port: 80
      targetPort: 8000
```

O deployment pede três réplicas. O `readinessProbe` só libera tráfego para um pod depois que `/health` responder, um primeiro *health check* que o capítulo 4 aprofunda. O bloco `resources` reserva um décimo de núcleo (`100m`) e 128 MB por pod e mata o que passar de 256 MB. O service seleciona os pods com o rótulo `app: rango-api` e liga a porta 80 à 8000 deles.

A aplicação do arquivo e a listagem dos pods:

```
kubectl apply -f k8s/api.yaml
kubectl get pods
```

```
NAME                          READY   STATUS    RESTARTS   AGE
rango-api-6b8f7d9c5f-2xkqp    1/1     Running   0          15s
rango-api-6b8f7d9c5f-h7vzn    1/1     Running   0          15s
rango-api-6b8f7d9c5f-tq4lm    1/1     Running   0          15s
```

Agora a parte divertida: apagar um pod, como se o servidor dele tivesse pegado fogo, e listar de novo:

```
kubectl delete pod rango-api-6b8f7d9c5f-2xkqp
kubectl get pods
```

Em poucos segundos, aparece um pod com outro nome no lugar do apagado. Você não pediu isso. O deployment continua declarando três réplicas, o estado atual tinha duas, e a reconciliação resolveu a diferença. É a hidra da mitologia grega com outra contabilidade: na lenda, cada cabeça cortada virava duas; o Kubernetes repõe exatamente uma. (Numa execução real, o nome é outro; o sufixo é aleatório.)

O acesso ao service a partir do notebook passa por um túnel temporário:

```
kubectl port-forward service/rango-api 8000:80
```

E, em outro terminal, `curl http://localhost:8000/restaurantes` devolve a lista de sempre, atendida por algum dos três pods.

No fim, o cluster é apagado. Ele consome memória do notebook, mas nada fora dele:

```
kind delete cluster --name rango
```

Você precisa de Kubernetes?

Para o Rangô de hoje, não. Uma API, um front e um banco cabem com folga num servidor com Compose, ou nos serviços gerenciados da Parte II. Kubernetes cobra caro em complexidade: control plane, atualizações a cada poucos meses, redes, permissões e muito YAML. Tanto YAML que o mercado inventou, meio de brincadeira, o cargo de "engenheiro de YAML". Ele compensa com muitos serviços, várias equipes e gente dedicada a operar a plataforma. Adotar Kubernetes para um app com dois restaurantes é como contratar um maestro para tocar campainha. Mesmo assim, estado desejado, réplicas, health checks e nomes estáveis aparecem em toda plataforma de containers, inclusive nas da AWS. No capítulo 17, o Rangô vai para o Amazon ECS com Fargate, e você vai ver também quando vale escolher o Amazon EKS, a versão gerenciada de Kubernetes da AWS.

Armadilhas e casos reais

> **Armadilha:** usar a tag `latest`. `latest` é só um nome, sem nenhuma garantia de ser a versão mais nova, e muda de conteúdo a cada push. É o "confia em mim" das tags. Dois servidores que baixaram `rango-api:latest` em dias diferentes podem estar rodando códigos diferentes, e o rollback vira adivinhação. Use versões explícitas (`0.1.1`) ou o hash do commit como tag.
>
> **Armadilha:** segredos dentro da imagem. Se o Dockerfile fizer `COPY .env .` e depois `RUN rm .env`, o arquivo continua na camada anterior, e qualquer pessoa com acesso à imagem consegue extraí-lo. Apagar numa camada o que foi copiado na anterior é como apagar uma mensagem no grupo depois que todo mundo já leu. Valores passados com `ENV` ou `ARG` aparecem em `docker history`. Segredos entram em tempo de execução, por variável de ambiente ou por um gerenciador de segredos, assunto do capítulo 16.

Na vida real: em abril de 2021, a Codecov, ferramenta de cobertura de testes usada em pipelines de CI de milhares de empresas, revelou que um erro no processo de criação das suas imagens Docker permitiu que um atacante extraísse uma credencial. Com ela, o atacante alterou o script de upload que os clientes baixavam nos seus pipelines, e o script adulterado passou a enviar as variáveis de ambiente desses pipelines (com tokens e chaves) para um servidor externo. O acesso começou no fim de janeiro e só foi descoberto em abril, quando um cliente notou que o hash do script baixado não batia com o publicado no GitHub.

Armadilha: gravar dados dentro do container. O sistema de arquivos de um container é descartável: quando ele é recriado, tudo o que foi escrito ali some. Uploads e arquivos de banco de dados precisam de um volume ou, melhor, de um serviço externo. As fotos dos pratos do Rangô vão parar no Amazon S3 no capítulo 11.

Armadilha: construir no Mac com Apple Silicon e rodar num servidor Intel. A imagem é construída para a arquitetura da máquina que fez o build (`arm64`), e o servidor (`amd64`) devolve `exec format error` ao tentar rodá-la. Use `docker build --platform linux/amd64` ou construa no CI, na arquitetura do destino.

Dica: prefira a forma de lista no `CMD` (`CMD ["uvicorn", ...]`) à forma de texto (`CMD uvicorn ...`). Na forma de texto, o Docker roda o comando através de `/bin/sh -c`, e o `SIGTERM` enviado pelo `docker stop` pode não chegar ao Uvicorn. Depois de 10 segundos, o Docker desiste e manda `SIGKILL`, cortando requisições no meio.

Dica: a imagem `python:3.12` completa passa de 1 GB; a `python:3.12-slim` fica perto de um décimo disso (valores aproximados). Imagens menores baixam mais rápido e trazem menos programas com falhas conhecidas. Evite o Alpine para Python: com a biblioteca C dele (musl), muitos pacotes compilam do zero.

Na vida real: em fevereiro de 2018, a empresa de segurança RedLock relatou que atacantes encontraram um console administrativo do Kubernetes da Tesla acessível pela internet sem senha. Num dos pods havia credenciais da conta de nuvem da empresa, e os atacantes usaram o cluster para minerar criptomoeda, escondendo o tráfego para dificultar a detecção. Alguém minerou criptomoeda, e quem pagou pela capacidade de processamento foi a Tesla. Um painel de administração exposto é uma porta aberta para toda a infraestrutura que ele controla.

Resumo

- Servidor quase sempre é uma VM: um hypervisor divide uma máquina física em várias, cada uma com o próprio kernel.
- No Linux, a aplicação roda com um usuário sem privilégios, numa porta alta. `SIGTERM` pede para encerrar, `SIGKILL` força, `SIGHUP` chega quando o terminal fecha.
- O systemd mantém serviços rodando depois de falhas e reboots, e guarda os logs (`journalctl`).
- O proxy reverso (Nginx) atende as portas 80 e 443, termina o TLS, roteia pelo cabeçalho `Host`, serve arquivos estáticos e repassa a API para o Uvicorn em `127.0.0.1:8000`.
- "Na minha máquina funciona" vem do ambiente ao redor do código. Containers empacotam o ambiente junto com a aplicação.
- Container é um processo isolado por namespaces e limitado por cgroups, compartilhando o kernel da máquina. Imagem é o pacote em camadas; container é a imagem em execução; registry guarda e distribui imagens.
- O Rangô ganhou `requirements.txt`, `backend/Dockerfile`, `backend/.dockerignore`, `compose.yaml` e `nginx/rango.conf`. A API roda como `rango-api:0.1.1` atrás de um Nginx com o certificado do `mkcert` em `https://api.rango.test:8443`.
- Orquestradores trabalham com estado desejado e reconciliação. No Kubernetes, pods rodam containers, deployments mantêm réplicas e trocam versões, e services dão um nome estável a um grupo de pods.
- Kubernetes compensa em sistemas grandes, com equipe para operá-lo. Para o Rangô de hoje, sobra.

Ponte: o dia em que o Rangô apareceu na TV

O Rangô agora tem um pacote que roda igual em qualquer lugar e um proxy que cuida do TLS. Num servidor com 2 vCPUs, ele aguenta tranquilo as poucas centenas de pedidos por dia da fase atual.

Até a terça-feira em que a sua sócia liga, eufórica: um programa de culinária da TV vai mostrar a Cantina da Nona na quinta, e o apresentador vai dizer o nome do Rangô ao vivo. Ela já está escolhendo a roupa para assistir; você já está procurando o antiácido. Você faz as contas. Se uma fração pequena dos espectadores abrir o app nos minutos seguintes, o tráfego multiplica por cinquenta. As três réplicas que você viu no kind rodavam todas no seu notebook. No servidor, o processador vai a 100%, as requisições começam a esperar na fila do Nginx, e o tempo de resposta passa de 80 milissegundos para 12 segundos. Um servidor só não aguenta. Compra-se um maior ou vários

pequenos? Como a sua sócia, digitando `rango.com.br`, chega ao servidor que está menos ocupado? E o que acontece com quem estava no meio de um pedido quando o servidor caiu? No próximo capítulo, você vai ver como uma arquitetura escala de verdade.

Referências

- Docker, documentação oficial: https://docs.docker.com/
- Referência do Dockerfile: https://docs.docker.com/reference/dockerfile/
- Docker Compose: https://docs.docker.com/compose/
- Imagem oficial do Python no Docker Hub: https://hub.docker.com/_/python
- Open Container Initiative: https://opencontainers.org/
- systemd, unidades de serviço: https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html
- Nginx, módulo de proxy: https://nginx.org/en/docs/http/ngx_http_proxy_module.html
- Uvicorn, deploy e proxies: https://www.uvicorn.org/deployment/
- Kubernetes, pods: https://kubernetes.io/docs/concepts/workloads/pods/
- Kubernetes, deployments: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Kubernetes, services: https://kubernetes.io/docs/concepts/services-networking/service/
- kind: https://kind.sigs.k8s.io/
- Codecov, comunicado sobre o incidente de 2021: https://about.codecov.io/security-update/
- Fortune, sobre o caso da Tesla (fev/2018): https://fortune.com/2018/02/20/tesla-hack-amazon-cloud-cryptocurrency-mining/

CAPÍTULO 4

Arquitetura que escala

No capítulo anterior, o Rangô deixou de depender da sua máquina. A API virou a imagem `rango-api:0.1.1`, roda atrás de um Nginx que cuida do TLS, e o `compose.yaml` sobe os dois juntos em qualquer lugar que tenha Docker. Você também viu um orquestrador manter três réplicas vivas num cluster local. No fim, ficou uma ameaça marcada para quinta-feira: um programa de culinária vai falar do Rangô ao vivo, e um servidor só não vai dar conta. A notícia boa é que o Rangô vai aparecer na TV. A notícia ruim é a mesma.

Quinta-feira, 21h14

Eis como seria a noite se nada mudasse.

O Rangô roda num servidor com 2 vCPUs e 4 GB de memória. Num dia comum, o pico do almoço chega a umas 5 requisições por segundo, e cada uma leva uns 80 milissegundos. A CPU passa o dia entre 10% e 15%. Ninguém pensa em infraestrutura. Infraestrutura é como encanamento: só vira assunto quando alguém precisa chamar o encanador.

Às 21h14, o apresentador prova a lasanha da Cantina da Nona e diz: "e dá para pedir pelo Rangô". Em dois minutos, o tráfego sobe de 5 para 250 requisições por segundo. A CPU vai a 100%, as requisições se acumulam na fila do Nginx, e o tempo de resposta passa de 80 milissegundos para 12 segundos. Às 21h19, o processo fica sem memória, o kernel o mata, e o `restart: unless-stopped` o recria só para ele morrer de novo um minuto depois. Quem estava montando um pedido perde o carrinho. Quem clicou em "Confirmar" não sabe se o pedido foi feito. Nas redes sociais, alguém já publicou o print da tela de erro com a legenda "a lasanha derrubou o Rangô".

O capítulo anterior terminou com três perguntas: compra-se um servidor maior ou vários pequenos? Como a requisição chega ao servidor menos ocupado? E o que acontece com quem estava no meio de um pedido quando um servidor cai? Este capítulo responde as três, e no caminho você vai descobrir uma decisão escondida no código do Rangô que impede qualquer resposta de funcionar.

Primeiro, as contas

Se cada requisição consome uns 80 milissegundos de processador, um processo Python atende por volta de 12 requisições por segundo (1 dividido por 0,08). A aproximação é grosseira, mas dá a ordem de grandeza: para 250 requisições por segundo, você precisaria de uns 21 processos trabalhando ao mesmo tempo, mais uma margem. E repare no efeito da lentidão: com 12 segundos por requisição, a mesma taxa de chegada significa 3.000 requisições acumuladas esperando. A lentidão aumenta a fila, e a fila aumenta a lentidão. Quem já enfrentou um caixa de supermercado na véspera de Natal conhece esse sistema por dentro.

Agora, as duas formas de conseguir esses 21 processos.

Para cima ou para os lados

Escala vertical é trocar o servidor por um maior: de 2 para 16 vCPUs, de 4 para 64 GB de memória. **Escala horizontal** é colocar mais servidores iguais lado a lado e dividir o trabalho entre eles. Em inglês, você vai ouvir *scale up* para a primeira e *scale out* para a segunda.

A vertical é tentadora porque dispensa mudanças no código: você desliga o servidor, escolhe um tamanho maior e liga de novo. Para muitas aplicações, isso resolve por anos. É como comprar uma panela maior quando a família cresce: funciona muito bem até o dia em que não existe panela maior à venda. Mas existe um teto (o maior servidor à venda), a troca quase sempre exige desligar a máquina, e, o mais grave, continua existindo um único servidor. Se ele cair, o Rangô inteiro cai junto. Isso tem nome, **ponto único de falha** (*single point of failure*), e você vai ouvir essa expressão muitas vezes neste livro. Em empresas pequenas, o ponto único de falha costuma ter nome, sobrenome e férias marcadas.

E há uma pegadinha específica para quem programa em Python.

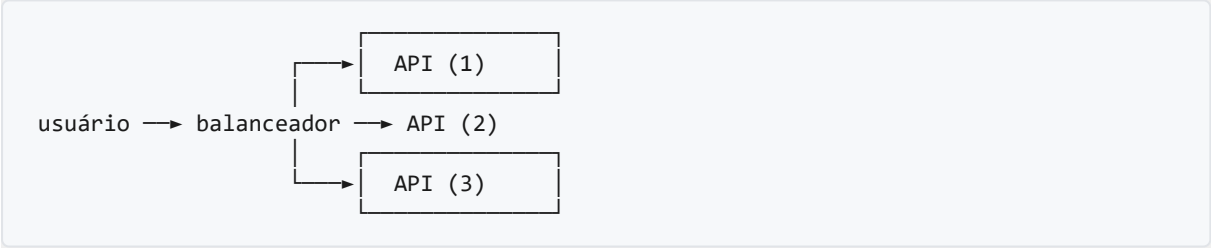
> **Armadilha:** o `CMD` do Dockerfile do Rangô roda o Uvicorn com um único processo. Por causa do GIL, um processo Python executa código Python em um núcleo de cada vez. Se você trocar o servidor de 2 para 16 vCPUs sem mudar mais nada, o Rangô continua usando praticamente um núcleo, e os outros 15 ficam olhando, como quinze amigos assistindo um só carregar a geladeira na mudança. Para aproveitar a máquina maior, seria preciso rodar vários processos, com `--workers` no Uvicorn ou vários containers. (O Python 3.13 trouxe uma versão experimental sem GIL, mas ela ainda está longe de ser o padrão em produção.)

A escala horizontal resolve esses limites: sempre dá para adicionar mais uma máquina, sem desligar as outras, e se uma cair as demais continuam atendendo. Em troca, alguém precisa distribuir as requisições, e a aplicação precisa funcionar quando requisições do mesmo usuário caem em servidores diferentes. A primeira exigência é um componente novo. A segunda é uma mudança no seu código, e é a que pega a maioria das equipes desprevenida. Geralmente numa quinta-feira à noite, com as câmeras ligadas.

O balanceador de carga

Com três servidores rodando a API, `rango.com.br` precisa apontar para algum lugar. Você poderia colocar os três IPs no DNS, e alguns sistemas fazem isso, mas você lembra do capítulo 2: resolvers guardam a resposta em cache pelo tempo do TTL, e um servidor que caiu continua recebendo usuários até o cache expirar. O DNS não sabe se o servidor está vivo. Ele é ótimo com nomes e péssimo com obituários.

A solução comum é colocar um componente na frente de todos: o **balanceador de carga** (*load balancer*). O usuário fala com um único endereço, o do balanceador, e ele repassa cada requisição para um dos servidores de trás. Os servidores que recebem o tráfego costumam ser chamados de *backends* ou alvos.



Se isso lembrou o proxy reverso do capítulo 3, a lembrança está certa: com mais de um backend, o Nginx passa a ser também um balanceador. Ganhou uma função nova sem aumento de salário, como tanta gente por aí.

Camada 4 e camada 7

Você vai ver balanceadores classificados por "camada", uma referência ao modelo de camadas de rede, o modelo OSI, com as suas sete camadas, que todo estudante decora para a prova e esquece na semana seguinte. Pouca gente sabe explicar de cabeça o que fazem as camadas 5 e 6, e a internet funciona mesmo assim. Na prática, a diferença é o que o balanceador enxerga.

Um balanceador de **camada 4** trabalha no nível do TCP (ou UDP). Ele vê IPs e portas e encaminha a conexão inteira para um backend, sem ler o HTTP. Por isso é muito rápido e serve para qualquer protocolo, inclusive conexões com bancos de dados.

Um balanceador de **camada 7** entende HTTP. Ele termina o TLS, lê a requisição e decide pelo conteúdo: `/api/*` para um grupo de servidores, `/admin/*` para outro, `Host: parceiros.rango.com.br` para um terceiro. Também adiciona cabeçalhos como o `X-Forwarded-For` e responde redirecionamentos. Custa um pouco mais de processamento, e compensa na maioria das aplicações web. O de camada 4 é o carteiro que entrega o envelope sem abrir; o de camada 7 abre, lê e decide para quem repassar, como uma secretária muito eficiente e um pouco indiscreta.

O Nginx com `proxy_pass`, do jeito que o Rangô usa, é um balanceador de camada 7. Na AWS, as duas variedades existem como serviços separados, e o capítulo 13 coloca o Rangô atrás do de camada 7.

Como escolher o servidor

A forma mais simples de distribuir é o **round robin**: a primeira requisição vai para o servidor 1, a segunda para o 2, a terceira para o 3, a quarta de volta para o 1. Funciona bem quando as requisições custam mais ou menos a mesma coisa. É o garçom da pizzaria de rodízio, que passa por todas as mesas na ordem, sem perguntar quem ainda está com fome.

Quando algumas requisições são muito mais pesadas (um relatório de 3 segundos ao lado de listagens de 20 milissegundos), o round robin pode empilhar trabalho num servidor enquanto outro está livre. O algoritmo de **menos conexões** (*least connections*) manda cada requisição para o servidor com menos requisições em andamento, e responde a pergunta sobre o servidor menos ocupado. É o garçom atento, que leva a próxima pizza para a mesa com o prato vazio.

Existe uma terceira família, que escolhe o servidor por um *hash* de alguma coisa da requisição, como o IP do cliente. O mesmo cliente cai sempre no mesmo servidor. Guarde essa ideia, porque ela vai voltar daqui a pouco como uma tentação perigosa.

Health checks: quem está vivo?

Distribuir requisições para um servidor morto é pior que não ter balanceador. Seria como continuar mandando comandas para um cozinheiro que já foi para casa. O balanceador precisa saber quem está em condições de receber tráfego, e ele descobre isso com **health checks**.

No **health check ativo**, o balanceador consulta cada backend periodicamente, por exemplo `GET /health` a cada 10 segundos; quem falha duas vezes seguidas sai da lista e volta quando se recupera. É a mãe que liga de dez em dez segundos para saber se você

chegou bem, só que com critério para desistir. No **health check passivo**, ele observa o tráfego real: se as requisições para um backend falham com erro de conexão ou *timeout*, ele o deixa de lado por um tempo.

O endpoint `/health` que o Rangô tem desde o capítulo 1 existe exatamente para isso. E o `readinessProbe` do Kubernetes, configurado no capítulo 3, é um health check ativo feito pelo orquestrador.

> **Armadilha:** um health check "profundo", que testa o banco, o Redis e a API de pagamento antes de responder 200, parece mais responsável. Pense no que acontece quando o Redis fica lento por um minuto. Todas as instâncias da API respondem 500 no `/health` ao mesmo tempo, o balanceador tira todas de circulação, e uma lentidão parcial vira o Rangô inteiro fora do ar. É o detector de fumaça que, ao sentir cheiro de torrada, desliga a energia do prédio inteiro. O health check usado pelo balanceador deve responder à pergunta "este processo consegue atender requisições?". Problemas de dependências compartilhadas são assunto de alarme e monitoramento, que o capítulo 6 apresenta.

O segredo que o código guardava

Com balanceador e health checks, bastaria subir três cópias da API atrás do Nginx. Antes de fazer isso, pense no carrinho de compras.

A forma mais natural de implementar um carrinho num protótipo é um dicionário em memória: `CARRINHOS["ana"] = {"Lasanha": 2}`. Com um processo só, funciona. Com três atrás de um round robin, a Ana adiciona a lasanha na instância 1, e o React pede o carrinho à instância 2, que nunca ouviu falar da Ana. O carrinho aparece vazio. Ela adiciona de novo, cai na instância 3, e agora existem três carrinhos diferentes da Ana. É o sonho de qualquer gerente comercial e o pesadelo de qualquer cliente.

Esse é o problema do **estado local**. Uma aplicação é **stateful** quando guarda, no próprio processo ou no próprio disco, informação de que requisições futuras dependem. É **stateless** quando cada requisição traz ou busca tudo o que precisa, e qualquer instância pode atendê-la igualmente bem. Para escalar horizontalmente, a aplicação precisa ser stateless. Stateless, aliás, é o estado de espírito ideal para uma segunda-feira de manhã: não lembrar de nada e atender quem chegar.

Pense numa função pura em Python, que não depende de nada guardado entre uma chamada e outra. Numa instância stateless, o estado existe (o carrinho precisa ficar em algum lugar), mas fica fora dela, num lugar que todas compartilham. O processo pode morrer ou ser multiplicado por dez sem que ninguém perca nada.

Além do carrinho, o estado costuma se esconder em outros lugares:

- **Sessões de login** em memória ou no disco local: o usuário loga na instância 1, a próxima requisição cai na 2, e ele está deslogado.
- **Uploads** salvos no disco do servidor. O capítulo 11 leva as fotos do Rangô para o Amazon S3 por isso.
- **Caches em memória** do processo, como um `functools.lru_cache` sobre dados que mudam: cada instância fica com uma versão.
- **Tarefas agendadas** dentro do processo da API: o e-mail "você esqueceu algo no carrinho" sai uma vez por instância.

Três e-mails idênticos lembrando a mesma lasanha é o tipo de insistência que faz qualquer cliente desinstalar o app.

> **Armadilha:** resolver o carrinho com **sessões aderentes** (*sticky sessions*), aquela escolha de servidor por hash que apareceu há pouco. O balanceador manda a Ana sempre para a instância 1, e o carrinho volta a funcionar. Mas quando a instância 1 cair, ou for substituída num deploy, os carrinhos de todo mundo "grudado" nela somem. E, se a aderência for por IP, os usuários atrás do mesmo CGNAT que você viu no capítulo 1 caem todos no mesmo servidor, disputando a mesma instância como passageiros disputam a única tomada livre do aeroporto.

Para sessões de login, há duas saídas comuns. Uma é guardar a sessão num armazenamento compartilhado. A outra é usar um token assinado, como um JWT, que o próprio cliente envia em cada requisição: qualquer instância verifica a assinatura e sabe quem é o usuário sem consultar nada. É a pulseira de festa: o segurança não consulta a lista, só olha o pulso. Para o carrinho, o Rangô vai usar um armazenamento compartilhado e rápido: o Redis.

Cache: a resposta mais barata é a que você não calcula

O Redis aparece em quase todo sistema que escala também por outro motivo: cache. Um **cache** é uma cópia de um dado guardada num lugar mais rápido ou mais próximo que a origem. Você já viu o cache do navegador e o da CDN no capítulo 2. Do lado do servidor, a aplicação pode guardar resultados prontos num armazenamento em memória compartilhado.

O **Redis** é um banco de dados chave-valor que mantém os dados em memória e responde em menos de um milissegundo dentro da mesma rede. Pense num dicionário Python acessível pela rede por todas as instâncias, com estruturas como hashes e

contadores e um prazo de validade em cada chave. Em 2024, o Redis mudou de licença, e parte da comunidade criou um fork compatível, o Valkey (nada mobiliza tanto uma comunidade de código aberto quanto uma mudança de licença); o capítulo 12 volta a isso com o cache gerenciado da AWS.

O padrão cache-aside

A forma mais comum de usar um cache na aplicação se chama **cache-aside**. A lógica cabe em três passos:

1. Procure o dado no cache.
2. Se estiver lá (um *hit*), devolva.
3. Se não estiver (um *miss*), busque na origem, grave no cache com um prazo de validade e devolva.

É o mesmo raciocínio do `@lru_cache`, só que compartilhado entre processos e com expiração. A lista de restaurantes do Rangô é uma candidata perfeita: todo usuário que abre o app a pede, ela muda poucas vezes por dia, e uma versão com um minuto de atraso não prejudica ninguém. Na noite da TV, com 250 requisições por segundo, um cache de 60 segundos transforma 15 mil consultas por minuto ao banco em uma. Se o banco de dados pudesse, mandaria uma cesta de café da manhã.

O difícil: quando o dado muda

Phil Karlton, engenheiro da Netscape, costumava dizer que só existem duas coisas difíceis em computação: invalidar cache e dar nome às coisas. A versão estendida da piada acrescenta uma terceira: erros de diferença de um. Quando a Cantina da Nona muda a taxa de entrega de R$ 6,90 para R$ 7,90, o cache continua servindo o valor antigo até expirar. Você tem duas ferramentas. A primeira é o TTL: aceitar que o dado pode ficar velho por no máximo aquele tempo. A segunda é a invalidação explícita: no mesmo código que atualiza a taxa no banco, apagar a chave do cache para que a próxima leitura busque o valor novo. Na prática, as duas andam juntas, porque a invalidação explícita às vezes falha (o processo cai entre gravar no banco e apagar a chave), e o TTL limita o estrago.

O que pode ser cacheado é uma decisão de negócio. A lista com um minuto de atraso é aceitável. O valor cobrado ao confirmar o pedido vem da origem, sempre, ou você terá um problema com o Procon ou com a Cantina da Nona.

> **Armadilha:** o **estouro de cache** (*cache stampede*). Às 21h15 da noite da TV, a chave `restaurantes` expira, centenas de requisições fazem miss no mesmo instante, e todas vão ao banco executar a mesma consulta, como clientes na porta de uma loja

> na abertura da Black Friday. As defesas usuais são deixar uma única requisição recalcular enquanto as outras recebem o valor antigo e acrescentar uma variação aleatória ao TTL, para que as chaves não expirem todas no mesmo segundo.

Na prática: três APIs, um Nginx e um Redis

Hora de transformar o Rangô. O plano tem duas etapas. Primeiro, a API se multiplica, o balanceamento funciona e o carrinho quebra. Depois, o carrinho é corrigido com Redis e a lista de restaurantes ganha cache. Raro é o plano de projeto que inclui "quebrar o carrinho" como etapa, mas este inclui de propósito.

Os pré-requisitos são o projeto do capítulo 3 com Docker e Docker Compose, o certificado do `mkcert` em `certs/` e a entrada `api.rango.test` no arquivo hosts.

Etapa 1: mostrando quem atendeu

Para enxergar o balanceamento, cada instância vai se identificar num cabeçalho da resposta. Dentro de um container, o nome do host é o ID curto do container, e isso basta. É o equivalente a pendurar um crachá em cada garçom. Em `backend/app/main.py`, entram os imports que faltam no topo e o middleware logo depois da criação do `app`:

```python
# backend/app/main.py (trechos novos)
import socket
from collections import defaultdict

from pydantic import BaseModel

INSTANCIA = socket.gethostname()


@app.middleware("http")
async def marcar_instancia(request, call_next):
    resposta = await call_next(request)
    resposta.headers["X-Rango-Instancia"] = INSTANCIA
    return resposta
```

E agora o carrinho, implementado do jeito ingênuo, em memória. O nome do usuário vai no path para simplificar o exemplo; numa aplicação real, ele viria do token de autenticação.

```python
# backend/app/main.py (continuação)
CARRINHOS: dict[str, dict[str, int]] = defaultdict(dict)


class Item(BaseModel):
    prato: str
    quantidade: int = 1
```

```python
@app.post("/carrinho/{usuario}/itens")
def adicionar_item(usuario: str, item: Item):
    carrinho = CARRINHOS[usuario]
    carrinho[item.prato] = carrinho.get(item.prato, 0) + item.quantidade
    return carrinho


@app.get("/carrinho/{usuario}")
def ver_carrinho(usuario: str):
    return CARRINHOS[usuario]
```

No Nginx, o destino único dá lugar a um grupo de servidores, em `nginx/rango.conf`:

```
# nginx/rango.conf
upstream rango_api {
    server api:8000 max_fails=2 fail_timeout=10s;
}

server {
    listen 443 ssl;
    server_name api.rango.test;
    ssl_certificate     /etc/nginx/certs/rango.test+1.pem;
    ssl_certificate_key /etc/nginx/certs/rango.test+1-key.pem;

    location / {
        proxy_pass http://rango_api;
        proxy_connect_timeout 2s;
        proxy_next_upstream error timeout;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

O bloco `upstream` define o grupo. Há uma única linha `server`, mas, com o serviço escalado, o DNS interno do Compose devolve o IP de todas as réplicas para o nome `api`, e o Nginx usa todos em round robin. `max_fails=2 fail_timeout=10s` é o health check passivo: duas falhas em 10 segundos tiram o backend de circulação por 10 segundos. O `proxy_connect_timeout 2s` evita esperar os 60 segundos padrão por um backend mudo, e `proxy_next_upstream` passa a requisição adiante quando a conexão falha.

No `compose.yaml`, só a tag da imagem muda, para `rango-api:0.2.0`. Repare que o serviço `api` não publica portas nem tem `container_name`, e é isso que permite criar várias réplicas dele. O comando sobe três:

```
docker compose up -d --build --scale api=3
docker compose ps
```

```
NAME            IMAGE               SERVICE   STATUS          PORTS
rango-api-1     rango-api:0.2.0     api       Up 8 seconds
rango-api-2     rango-api:0.2.0     api       Up 8 seconds
```

```
rango-api-3     rango-api:0.2.0     api       Up 8 seconds
rango-proxy-1   nginx:1.27-alpine   proxy     Up 7 seconds    127.0.0.1:8443->443/tcp
```

O prefixo dos nomes vem da pasta do projeto, então pode ser diferente em outra máquina. Seis requisições seguidas mostram o cabeçalho:

```
CA="$(mkcert -CAROOT)/rootCA.pem"
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -D - --cacert "$CA" https://api.rango.test:8443/health \
    | grep -i x-rango-instancia
done
```

```
x-rango-instancia: 4be1f07c2a93
x-rango-instancia: 9d02c6e8b17f
x-rango-instancia: e57a3b90d4c1
x-rango-instancia: 4be1f07c2a93
x-rango-instancia: 9d02c6e8b17f
x-rango-instancia: e57a3b90d4c1
```

Os IDs mudam de uma máquina para outra, mas o padrão é este: um, dois, três, de volta ao um. Round robin funcionando.

O carrinho quebra, como previsto

O teste adiciona duas lasanhas ao carrinho da Ana e depois consulta o carrinho três vezes:

```
curl -s --cacert "$CA" -X POST https://api.rango.test:8443/carrinho/ana/itens \
  -H "Content-Type: application/json" \
  -d '{"prato": "Lasanha", "quantidade": 2}'
echo
for i in 1 2 3; do
  curl -s --cacert "$CA" https://api.rango.test:8443/carrinho/ana
  echo
done
```

```
{"Lasanha":2}
{}
{}
{"Lasanha":2}
```

A primeira linha é a resposta do `POST`. Das três consultas, só a que caiu na mesma instância do `POST` encontra a lasanha; a posição dela varia, mas duas das três vêm vazias. A Ana ganhou um carrinho de Schrödinger: a lasanha está e não está lá, dependendo de quem abre a caixa.

Derrubando uma instância

Antes da correção, vale ver o health check passivo trabalhar: uma das réplicas é parada, e o laço do cabeçalho se repete:

```
docker stop "$(docker compose ps -q api | head -n 1)"
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -D - --cacert "$CA" https://api.rango.test:8443/health \
    | grep -i x-rango-instancia
done
```

Todas as respostas vêm das duas instâncias restantes, e nenhuma falha chega ao `curl`. O cliente nem fica sabendo que um terço da equipe faltou ao trabalho. Uma requisição pode demorar até 2 segundos: é o Nginx esbarrando no `proxy_connect_timeout`, marcando a falha e repassando a requisição para a próxima instância.

Agora o detalhe que responde a terceira pergunta do capítulo anterior. Por padrão, o Nginx só repassa para outro backend requisições idempotentes, como `GET`. Um `POST /pedidos` que falhou no meio do caminho fica sem reenvio, porque o Nginx não sabe se a instância chegou a gravar o pedido antes de cair. Com o estado fora da instância, o carrinho sobrevive. Mas a requisição em andamento naquele instante falha, e o cliente precisa tentar de novo sem duplicar nada. Isso se chama idempotência, um dos assuntos do capítulo 5. A palavra é difícil de pronunciar e fácil de esquecer, o contrário do problema que ela resolve.

Depois, a instância volta e o proxy é reiniciado:

```
docker compose up -d --scale api=3
docker compose restart proxy
```

> **Armadilha:** o Nginx resolve o nome `api` uma única vez, quando inicia. Se você muda o número de réplicas, ou se uma réplica é recriada com outro IP, ele continua usando a lista antiga até ser reiniciado. Balanceadores gerenciados mantêm a lista de alvos atualizada sozinhos, como você verá no capítulo 13.

Etapa 2: o estado vai para o Redis

O Redis entra no `compose.yaml`, e a API recebe o endereço dele:

```
# compose.yaml
services:
  api:
    build: ./backend
    image: rango-api:0.3.0
    environment:
      CORS_ORIGINS: "https://rango.test:5173"
      REDIS_URL: "redis://redis:6379/0"
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7.4-alpine
```

```
    restart: unless-stopped

  proxy:
    image: nginx:1.27-alpine
    ports:
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/rango.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on:
      - api
    restart: unless-stopped
```

O Redis não publica porta nenhuma: só a API, dentro da rede do Compose, fala com ele. Lembra dos bancos MongoDB sequestrados do capítulo 1? Redis exposto na internet sem senha teve o mesmo destino muitas vezes.

O cliente Python é instalado e entra no `requirements.txt`, com o ambiente virtual do backend ativado:

```
cd backend
pip install redis
pip freeze > requirements.txt
cd ..
```

No `main.py`, o dicionário `CARRINHOS` e o import de `defaultdict` saem, a classe `Item` fica, e as duas rotas do carrinho mudam:

```
# backend/app/main.py (carrinho com Redis)
import redis

cache = redis.Redis.from_url(os.environ["REDIS_URL"], decode_responses=True)


def ler_carrinho(chave: str) -> dict[str, int]:
    return {prato: int(qtd) for prato, qtd in cache.hgetall(chave).items()}


@app.post("/carrinho/{usuario}/itens")
def adicionar_item(usuario: str, item: Item):
    chave = f"carrinho:{usuario}"
    cache.hincrby(chave, item.prato, item.quantidade)
    cache.expire(chave, 3600)
    return ler_carrinho(chave)


@app.get("/carrinho/{usuario}")
def ver_carrinho(usuario: str):
    return ler_carrinho(f"carrinho:{usuario}")
```

Cada carrinho vira um *hash* do Redis, um pequeno dicionário guardado sob a chave `carrinho:ana`. O `HINCRBY` soma a quantidade de forma atômica: se duas instâncias adicionarem lasanha ao mesmo carrinho no mesmo instante, nenhuma das somas se

perde. O `EXPIRE` faz o carrinho abandonado sumir depois de uma hora, sem nenhuma rotina de limpeza. Quem dera a geladeira do escritório tivesse um `EXPIRE`.

Agora a lista de restaurantes com cache-aside. Para sentir a diferença, a busca vai simular uma consulta lenta ao Postgres com um `time.sleep`. A rota `/restaurantes` passa a ser esta:

```
# backend/app/main.py (restaurantes com cache-aside)
import json
import random
import time


def buscar_restaurantes() -> list[dict]:
    time.sleep(0.3)  # simula uma consulta de 300 ms ao Postgres
    return [
        {**r, "taxa_entrega_fmt": formatar_reais(r["taxa_entrega"])}
        for r in RESTAURANTES
    ]


@app.get("/restaurantes")
def listar_restaurantes():
    em_cache = cache.get("restaurantes")
    if em_cache is not None:
        return json.loads(em_cache)
    restaurantes = buscar_restaurantes()
    cache.set("restaurantes", json.dumps(restaurantes), ex=60 + random.randint(0, 10))
    return restaurantes
```

O `ex` define o TTL em segundos, e o `random.randint(0, 10)` é a variação aleatória contra o estouro de cache. Os imports podem ficar todos juntos no topo do arquivo, como de costume; aqui eles aparecem perto do código que os usa para facilitar a leitura.

Depois, tudo é reconstruído e sobe de novo:

```
docker compose up -d --build --scale api=3
docker compose restart proxy
```

O mesmo teste do carrinho agora mostra:

```
{"Lasanha":2}
{"Lasanha":2}
{"Lasanha":2}
{"Lasanha":2}
```

As três consultas, cada uma numa instância diferente, encontram a mesma lasanha. O próprio Redis confirma o dado na fonte:

```
docker compose exec redis redis-cli HGETALL carrinho:ana
```

```
1) "Lasanha"
2) "2"
```

Para medir o efeito do cache, o teste dispara 100 requisições, 10 de cada vez, e cronometra. O cache é esvaziado antes da primeira rodada para começar do zero:

```
docker compose exec redis redis-cli DEL restaurantes
time (seq 100 | xargs -P 10 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
  --cacert "$CA" https://api.rango.test:8443/restaurantes | sort | uniq -c)
```

```
    100 200

real    0m0.71s
```

Para comparar, a linha `em_cache = cache.get("restaurantes")` é trocada temporariamente por `em_cache = None`, a imagem é reconstruída e o teste roda de novo. Sem o cache, cada requisição paga os 300 milissegundos da consulta simulada, e a mesma rodada leva perto de 3 segundos. Os números exatos variam com a máquina; a proporção é o que importa. Uma única linha com `get` acaba de valer mais do que três servidores extras. A troca é desfeita antes de seguir.

No fim, `docker compose down` desliga tudo. Os dados do Redis somem junto, o que para um carrinho de teste está ótimo.

Monolito, microsserviços e serverless

Repare no que o Rangô é agora: um único código, empacotado numa única imagem, rodando em três cópias idênticas. Isso é um **monolito**, e ele acabou de escalar horizontalmente sem dificuldade. Monolito virou quase palavrão em palestra de conferência, mas ele não assistiu às palestras e escala muito bem, obrigado. Essa observação desmonta um argumento comum a favor de microsserviços.

Na arquitetura de **microsserviços**, a aplicação é dividida em serviços pequenos e independentes, cada um com o seu código, o seu deploy e, idealmente, o seu banco de dados. O Rangô poderia virar um serviço de catálogo, um de pedidos, um de pagamentos e um de entregas, conversando entre si por HTTP ou por mensagens.

A vantagem principal aparece em organizações grandes: equipes diferentes fazem deploy sem coordenar entre si, e uma falha no serviço de recomendações não derruba o checkout. Os custos costumam ser subestimados. Uma chamada de função vira uma chamada de rede, que pode ficar lenta, falhar ou chegar duas vezes. Uma transação que garantia "pedido criado e estoque baixado juntos" vira uma coordenação entre dois serviços, sem `BEGIN` e `COMMIT` para proteger. E cada serviço precisa do seu pipeline, do

seu monitoramento e de alguém de plantão. Muita empresa descobre, tarde demais, que trocou um monolito por um monolito distribuído: todos os defeitos do primeiro, agora com latência de rede. Para uma equipe de três pessoas, microsserviços costumam trazer o custo sem o benefício.

> **Na vida real:** em 2023, a equipe do Prime Video que monitora a qualidade dos streams publicou que migrou um serviço de uma arquitetura distribuída, com AWS Step Functions orquestrando funções AWS Lambda, para um processo único em containers no Amazon ECS. Segundo a equipe, o custo de infraestrutura caiu mais de 90%. O caso gerou discussão por vir de dentro da própria Amazon. Foi como o dono da churrascaria recomendar a salada.

O caminho mais seguro para o Rangô é um **monolito modular**: um único deploy, com o código organizado em módulos de fronteiras claras (`catalogo`, `pedidos`, `pagamentos`), sem um acessar as tabelas do outro. Se um dia uma parte precisar ser extraída, a fronteira já existe. É a casa construída com a laje pronta para o segundo andar: ninguém precisa demolir a sala para subir a escada.

Serverless como ideia

Existe uma terceira forma de pensar a execução do código. No modelo **serverless**, você entrega uma função ao provedor de nuvem e ele cuida de todo o resto: em quais máquinas rodar, quantas cópias manter, como escalar. O nome engana, porque os servidores continuam existindo; você só deixa de vê-los e de administrá-los. Serverless tem servidores do mesmo jeito que wireless tem fios: eles existem, só não ficam do seu lado.

A função roda quando um evento chega (uma requisição HTTP, um arquivo enviado, um horário agendado), e você paga pelo tempo de execução. Sem tráfego, a conta fica perto de zero (uma frase rara num livro sobre nuvem, aproveite); com mil requisições simultâneas, o provedor sobe mil cópias. Por isso a função precisa ser stateless. As contrapartidas: a primeira requisição depois de um período parado espera a cópia ser criada (o *cold start*), as execuções têm duração máxima, e com tráfego alto e constante pagar por execução pode sair mais caro que manter servidores. O *cold start*, aliás, é o equivalente computacional de quem acorda, passa o café e só então atende o telefone. O capítulo 19 aprofunda o modelo com AWS Lambda.

Os gargalos que aparecem depois

Escalar a API resolve o primeiro gargalo e revela o próximo, como no jogo de fliperama de acertar a toupeira: você acerta uma, e outra aparece do lado. Alguns aparecem em quase todo sistema.

O banco de dados

Todas as réplicas da API falam com o mesmo Postgres. Cada processo Python costuma manter um *pool* de conexões abertas com o banco para não pagar o custo de abrir uma a cada consulta. O SQLAlchemy, por padrão, mantém até 5 conexões por pool e permite mais 10 temporárias em momentos de pico, ou seja, até 15 por processo. O Postgres, por padrão, aceita 100 conexões no total.

Faça a conta da noite da TV. Com 21 processos da API, são até 315 conexões tentando entrar num banco que aceita 100. A partir da centésima, o Postgres recusa com uma mensagem que você não quer ler às 21h20:

```
FATAL:  sorry, too many clients already
```

O Postgres é educado: pede desculpas antes de estragar a sua noite.

As saídas são reduzir o pool de cada processo, colocar um intermediário que reaproveita conexões entre processos (o PgBouncer é o mais conhecido no mundo Postgres) e, para a carga de leitura, usar réplicas do banco, que o capítulo 6 apresenta e o capítulo 12 coloca em prática. O cache que você acabou de implementar também ajuda muito aqui, porque cada consulta evitada é uma conexão a menos ocupada.

O balanceador como ponto único de falha

Você tirou o ponto único de falha da API e o colocou no Nginx, como quem arruma o quarto empurrando a bagunça para dentro do armário. Se o servidor dele cair, as três instâncias ficam vivas e inalcançáveis. Os balanceadores gerenciados da nuvem já rodam distribuídos em várias máquinas e data centers, e esse é um dos argumentos mais concretos para usar o da AWS em vez de manter um Nginx próprio na borda.

O tempo para escalar

Escala horizontal automática, o *auto scaling*, acompanha uma métrica (CPU acima de 60% por três minutos, por exemplo) e adiciona instâncias quando ela passa do limite. Só que uma instância nova leva alguns minutos para ligar, baixar a imagem, iniciar e passar no health check. Na noite da TV, o tráfego subiu cinquenta vezes em dois minutos. Para eventos com hora marcada, a saída é escalar antes: às 20h30, o Rangô já precisa estar

com a capacidade da noite. Ninguém começa a cozinhar o almoço de domingo quando a família já está sentada à mesa. O auto scaling fica para os picos que ninguém avisou e para devolver as máquinas depois. O capítulo 13 configura as duas coisas na AWS.

> **Na vida real:** no lançamento do Pokémon GO, em julho de 2016, a Niantic e o Google planejaram a infraestrutura com uma estimativa de pior caso de cerca de 5 vezes o tráfego de referência. O tráfego real chegou a 50 vezes, dez vezes o pior caso. Ninguém tinha previsto quantos adultos sairiam à rua de madrugada caçando monstrinhos. Os primeiros dias tiveram instabilidade, e o post do Google sobre o episódio lista entre as lições provisionar capacidade com mais folga.

Chamadas lentas a terceiros

Há um último gargalo que nenhuma quantidade de réplicas resolve. Se uma rota espera um serviço externo (o gateway de pagamento, o provedor de e-mail), cada requisição prende um processo durante toda a espera. Se o provedor de e-mail passar de 1 para 10 segundos, cada processo atende dez vezes menos requisições. O sistema inteiro passa a andar na velocidade do fornecedor mais lento, como comboio de caminhões na serra. Guarde isso para a ponte.

Armadilhas e casos reais

Algumas armadilhas apareceram ao longo do capítulo. Estas são as que faltam:

> **Armadilha:** escalar horizontalmente uma aplicação com tarefas agendadas dentro do processo. Se a API do Rangô tem um agendador que fecha o caixa do dia à meia-noite, três réplicas fecham o caixa três vezes, e fechar o caixa três vezes não deixa ninguém três vezes mais rico. Tarefas agendadas precisam rodar num processo separado, com uma única cópia, ou usar um mecanismo de trava que garanta uma execução só. Os capítulos 19 e 21 mostram como a AWS resolve agendamentos fora da API.
>
> **Armadilha:** tratar o Redis como banco de dados definitivo. Ele guarda tudo em memória, e uma reinicialização sem persistência configurada apaga todos os carrinhos. Para carrinhos com prazo de uma hora, a perda é tolerável. Para pedidos confirmados, o lugar é o Postgres. Memória RAM é ótima para lembrar e péssima para guardar.

> **Dica:** antes de um evento previsto, faça um teste de carga com tráfego parecido com o esperado, num ambiente igual ao de produção. É no teste que você descobre o `too many clients already`, e não às 21h14 de quinta-feira. Ensaio geral existe no teatro por um bom motivo. Ferramentas como k6 e Locust (esta escrita em Python) simulam milhares de usuários seguindo roteiros realistas, como abrir o app, ver a lista e montar um carrinho.

Resumo

- Escala vertical troca o servidor por um maior e mantém o ponto único de falha. Escala horizontal adiciona servidores iguais e exige balanceador e aplicação stateless.
- Em Python, mais vCPUs só ajudam com mais processos.
- Balanceadores de camada 4 encaminham conexões TCP; os de camada 7 entendem HTTP. Round robin distribui em sequência; menos conexões escolhe o servidor mais livre.
- Health checks ativos consultam os backends; passivos observam o tráfego. O do balanceador testa o processo, sem depender de recursos compartilhados.
- Estado guardado na instância impede a escala horizontal. Sticky sessions só escondem o problema.
- Cache-aside com TTL e variação aleatória; invalidar é a parte difícil.
- O Rangô roda como `rango-api:0.3.0` em três réplicas atrás de um `upstream` no Nginx, com carrinho e cache no Redis.
- Monolito escala horizontalmente. Microsserviços servem a equipes grandes. Serverless entrega a escala ao provedor.
- Próximos gargalos: conexões com o banco, o balanceador, o tempo para escalar e a espera por serviços externos.

Ponte: o botão que trava tudo

Quinta-feira, 21h14, de novo, agora com o Rangô preparado. Seis instâncias da API sobem às 20h30, o cache segura a lista de restaurantes, os carrinhos sobrevivem a qualquer instância que caia. O tráfego chega, a CPU fica em 45%, e a sua sócia manda uma figurinha comemorando.

Às 21h22, os gráficos mudam. A listagem continua rápida, mas o botão "Confirmar pedido" começa a girar sem parar. A rota `POST /pedidos` faz tudo em sequência: cobra o cartão no gateway de pagamento (2,5 segundos), avisa o restaurante pela API dele (2

segundos) e envia o e-mail de confirmação (1 segundo). Naquela noite, o provedor de e-mail, também sobrecarregado, passa a levar 20 segundos para responder. Cada pedido prende um processo da API por quase meio minuto. Em pouco tempo, todos os processos das seis instâncias estão parados esperando o e-mail, e até a listagem, que não tem nada a ver com isso, deixa de responder. Pior: alguns clientes, cansados de esperar, clicam de novo em "Confirmar", e o cartão é cobrado duas vezes. Poucas coisas destroem a reputação de um app de comida tão rápido quanto cobrar duas lasanhas e entregar uma.

Por que um e-mail atrasado consegue derrubar a confirmação do pedido? A API precisa mesmo esperar o restaurante e o e-mail para dizer ao cliente que o pedido foi recebido? E como garantir que um clique repetido não vire uma cobrança dupla? No próximo capítulo, você vai ver como filas e eventos desacoplam as partes de um sistema, e o que muda quando as coisas passam a acontecer fora de ordem.

Referências

- Nginx, balanceamento de carga HTTP: https://nginx.org/en/docs/http/load_balancing.html
- Nginx, módulo `upstream` (`max_fails`, `fail_timeout`, `least_conn`): https://nginx.org/en/docs/http/ngx_http_upstream_module.html
- Nginx, módulo `proxy` (`proxy_next_upstream`, `proxy_connect_timeout`): https://nginx.org/en/docs/http/ngx_http_proxy_module.html
- Docker Compose, `docker compose up` e a opção `--scale`: https://docs.docker.com/reference/cli/docker/compose/up/
- Redis, documentação oficial: https://redis.io/docs/latest/
- redis-py, cliente Python: https://redis.readthedocs.io/
- Valkey: https://valkey.io/
- The Twelve-Factor App, processos stateless: https://12factor.net/processes
- PostgreSQL, `max_connections`: https://www.postgresql.org/docs/current/runtime-config-connection.html
- SQLAlchemy, pool de conexões: https://docs.sqlalchemy.org/en/20/core/pooling.html
- Martin Fowler, "Microservices": https://martinfowler.com/articles/microservices.html
- Martin Fowler, "MonolithFirst": https://martinfowler.com/bliki/MonolithFirst.html
- Google Cloud, "Bringing Pokémon GO to life on Google Cloud" (2016): https://cloud.google.com/blog/products/containers-kubernetes/bringing-pokemon-go-to-life-on-google-cloud

- InfoQ, sobre a migração do monitoramento do Prime Video para um monolito (2023): https://www.infoq.com/news/2023/05/prime-ec2-ecs-saves-costs/

CAPÍTULO 5

Filas, eventos e sistemas distribuídos

No capítulo anterior, o Rangô aprendeu a crescer para os lados, como todo mundo depois das festas de fim de ano. A API ficou stateless, os carrinhos foram para o Redis, a lista de restaurantes ganhou cache, e o Nginx passou a distribuir as requisições entre várias instâncias, desviando das que caem. Ficou um aviso no fim: nenhuma quantidade de réplicas resolve uma rota que passa a maior parte do tempo esperando outro sistema responder. E ficou uma promessa: explicar por que um `POST /pedidos` que falha no meio não pode simplesmente ser repetido. Quem já apertou F5 numa tela de pagamento conhece o frio na barriga que essa frase provoca.

Vinte e cinco segundos para dizer "recebido"

Vamos voltar àquela quinta-feira, às 21h22. Seis instâncias, CPU em 45%, cache funcionando. E o botão "Confirmar pedido" girando sem parar. O círculo que gira é o jeito mais educado que a computação inventou para dizer "não faço a menor ideia de quando isso termina".

A conta do que a rota `POST /pedidos` fazia é simples. Cobrar o cartão no gateway de pagamento levava 2,5 segundos. Avisar o restaurante pela API dele, mais 2 segundos. Enviar o e-mail de confirmação, mais 1 segundo. Em uma noite normal, o cliente esperava 5,5 segundos olhando o botão girar, o que já é ruim. Naquela noite, o provedor de e-mail passou a responder em 20 segundos, e cada confirmação passou a levar 24,5 segundos.

Lembra da conta de capacidade do capítulo 4? Um processo que atende uma requisição em 80 ms aguenta cerca de 12 requisições por segundo. Uma rota síncrona do FastAPI (declarada com `def`) roda num pool de 40 threads por processo, e cada pedido ocupa uma thread durante 24,5 segundos. Com seis instâncias, bastaram pouco mais de 240 pedidos simultâneos para ocupar todas as threads. A partir daí, a listagem de restaurantes, que responde em milissegundos e não tem nada a ver com e-mail, ficou na fila atrás dos pedidos. O Rangô inteiro parou por causa de um serviço que só manda uma mensagem de "obrigado pela compra". Foi o agradecimento mais caro da história da empresa.

E veio o segundo problema. Depois de 15 segundos olhando o botão girar, alguns clientes clicaram de novo. A segunda requisição foi para outra instância, que cobrou o cartão outra vez. Na sexta de manhã, a sua sócia tinha 37 pedidos de reembolso no e-mail. Nenhum daqueles clientes tinha pedido duas lasanhas. O extrato do cartão discordava.

Nenhuma das duas falhas tem a ver com capacidade de processamento. A rota amarra o tempo de resposta ao pior tempo de três sistemas que você não controla, e ela não sabe distinguir um pedido novo de um pedido repetido.

O que a API precisa esperar?

Pense em como funciona um restaurante de verdade. O garçom anota o pedido, leva a comanda até a cozinha, prende no trilho e volta para o salão. Ele não fica parado ao lado do fogão esperando a lasanha ficar pronta para só então atender a próxima mesa. Se ficasse, o restaurante atenderia uma mesa por hora e teria o garçom mais bem informado sobre lasanhas do país. A cozinha trabalha no ritmo dela, e o garçom continua atendendo.

Essa é a diferença entre comunicação **síncrona** e **assíncrona**. Na síncrona, quem chama espera a resposta para continuar, como uma chamada de função comum em Python. Na assíncrona, quem chama entrega o trabalho para alguém e segue em frente; o resultado chega depois, por outro caminho.

Cuidado com uma confusão comum aqui. O `async def` do FastAPI resolve outro problema. Ele permite que um processo atenda outras requisições enquanto espera a rede, mas o cliente daquela requisição continua esperando os 24,5 segundos. Ele só espera de um jeito mais elegante, como quem aguarda o ônibus lendo um livro: o ônibus não chega mais cedo por isso. O que buscamos é tirar o trabalho de dentro da requisição.

A pergunta certa para cada etapa do pedido é: o cliente precisa do resultado disto para receber a resposta?

- **Pagamento:** sim, em boa parte dos casos. Se o cartão for recusado, o cliente precisa saber agora, enquanto ainda está na tela, para tentar outro cartão.
- **Avisar o restaurante:** não. O cliente precisa saber que o pedido foi registrado. Se o restaurante vai receber o aviso em 200 milissegundos ou em 4 segundos, tanto faz para quem está na tela.
- **E-mail de confirmação:** não. Um e-mail que chega 30 segundos depois ninguém percebe. Um botão que gira 30 segundos todo mundo percebe.

Com essa divisão, a rota cobra o cartão, registra o pedido, deixa um recado dizendo "o pedido 8f3a... foi criado" e responde. O resto acontece depois. O tempo de resposta cai de 24,5 para cerca de 2,5 segundos, e o atraso do provedor de e-mail deixa de afetar a API. O provedor continua lento, só que agora lento no tempo dele, sem plateia.

> **Dica:** o pagamento também pode sair da requisição, com a tela mostrando "aguardando pagamento" até a confirmação chegar. Fica mais complexo, e o capítulo 21 mostra como orquestrar isso na AWS.

Filas: a comanda presa no trilho

O "recado" precisa ficar em algum lugar. Esse lugar é uma **fila de mensagens**: um serviço que recebe mensagens de quem produz trabalho e as guarda até que alguém as retire para processar.

O vocabulário é curto. Quem coloca a mensagem na fila é o **produtor** (a API do Rangô). Quem retira e processa é o **consumidor**, também chamado de *worker* (um processo separado que manda e-mails). O serviço que guarda as mensagens no meio é o *broker*. Os mais conhecidos são RabbitMQ, Apache Kafka e o próprio Redis, que você já tem rodando. O Kafka ganhou o nome do escritor Franz Kafka porque, segundo Jay Kreps, um dos criadores, era um sistema otimizado para escrita e ele gostava do autor. Quem já configurou um cluster de Kafka desconfia que a homenagem foi ao clima kafkiano da configuração. Na AWS, o equivalente gerenciado é o Amazon SQS, assunto do capítulo 21.

A fila traz três benefícios que valem a pena separar.

O primeiro é o **desacoplamento no tempo**. O produtor e o consumidor não precisam estar no ar ao mesmo tempo. Se o worker de e-mail cair por 10 minutos, a API continua aceitando pedidos; as mensagens se acumulam e são processadas quando ele voltar. Na versão síncrona, 10 minutos de e-mail fora do ar eram 10 minutos sem pedidos.

O segundo é a **absorção de picos**. Se a TV gera 300 pedidos por minuto e o provedor de e-mail aguenta 100, a fila cresce durante o pico e esvazia depois, como uma caixa d'água. Os e-mails só chegam alguns minutos mais tarde, e ninguém na história abriu um e-mail de confirmação de pedido com pressa.

O terceiro é a **escala independente**. Se a fila de e-mails não para de crescer, você sobe mais workers de e-mail, sem tocar na API. Vários workers lendo a mesma fila disputam as mensagens, e cada mensagem vai para um só deles. Esse arranjo tem nome:

competing consumers, consumidores concorrentes. Funciona como os cozinheiros pegando a próxima comanda do trilho.

Em troca, você ganha uma peça a mais para operar. A métrica que importa passa a ser a idade da mensagem mais antiga: 5 mil mensagens processadas em um minuto indicam saúde, 20 mensagens paradas há duas horas indicam um worker morto. Em sistemas distribuídos, ninguém morre de forma dramática: o processo só para de responder, como um colega em reunião com a câmera desligada.

Pub/sub: um evento, vários interessados

Até aqui, a API manda um recado para o worker de e-mail. Mas o restaurante também precisa saber do pedido. Daqui a alguns meses, o time de dados vai querer contar pedidos por bairro (o time de dados sempre quer contar alguma coisa por bairro), e alguém vai querer dar pontos de fidelidade. A API vai ter que conhecer cada uma dessas filas e escrever em todas?

Existe uma forma mais elegante de pensar nisso. Em vez de mandar **comandos** ("envie este e-mail", "avise este restaurante"), a API publica um **evento**: um fato que aconteceu, com nome no passado. `pedido.criado`. Ela não sabe quem vai se interessar, e nem precisa saber. É a fofoca na sua forma mais pura: alguém conta o fato, e quem quiser que faça alguma coisa com ele.

Esse modelo se chama **publish/subscribe**, ou *pub/sub*. O publicador envia o evento para um tópico, e cada assinante recebe a sua própria cópia. É o mesmo princípio dos eventos do navegador que você usa no React: o botão dispara `click`, e quem registrou um listener reage. O botão não conhece os listeners.

A diferença para a fila simples está em quem recebe cada mensagem:

	Fila	Pub/sub
Quem recebe cada mensagem	Um único consumidor	Todos os assinantes, cada um com sua cópia
Uso típico	Distribuir trabalho	Avisar que algo aconteceu
Analogia	Comanda no trilho da cozinha	Aviso no grupo da família

Na prática, os dois se combinam. O evento `pedido.criado` é entregue a dois grupos de assinantes, "restaurante" e "e-mail", e dentro de cada grupo pode haver vários workers disputando as mensagens. Esse arranjo de um evento para vários destinos é chamado de

fan-out. Na AWS, ele aparece como SNS publicando em várias filas SQS, ou como regras do Amazon EventBridge, ambos no capítulo 21. Na parte prática deste capítulo, o Rangô monta exatamente isso com Redis Streams.

Um evento bem desenhado carrega dados suficientes para os assinantes trabalharem sem voltar à API, incluindo o ID do pedido, que vai ser importante daqui a pouco.

Pelo menos uma vez

Agora vem a parte em que sistemas distribuídos começam a ficar estranhos. Leslie Lamport já tinha avisado, numa definição famosa: um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode deixar o seu próprio computador inutilizável.

Um worker retira a mensagem da fila, envia o e-mail e avisa o broker: "pronto, pode apagar". Esse aviso se chama **confirmação**, ou *ack* (de *acknowledgement*). Enquanto o ack não chega, o broker mantém a mensagem guardada. Se o worker morrer antes de confirmar, o broker entrega a mensagem de novo, para ele mesmo quando voltar ou para outro worker.

Agora acompanhe o pior momento possível. O worker envia o e-mail com sucesso. Um milissegundo depois, antes de mandar o ack, o container é reiniciado por um deploy. Para o broker, a mensagem nunca foi processada. Ele a entrega de novo, e o cliente recebe dois e-mails. Ele não reclama; no máximo, conclui que o Rangô gosta muito dele.

Você poderia inverter a ordem: confirmar primeiro, processar depois. Aí o problema se inverte. Se o worker morrer entre o ack e o envio, a mensagem some e o e-mail nunca sai.

Não existe uma terceira ordem. Entre o efeito no mundo externo e o aviso ao broker sempre há um intervalo, e uma falha pode cair nele. Você precisa escolher qual erro prefere:

- **No máximo uma vez** (*at-most-once*): confirma antes de processar. Nunca duplica, às vezes perde.
- **Pelo menos uma vez** (*at-least-once*): confirma depois de processar. Nunca perde, às vezes duplica.

Para pedidos de comida, perder o aviso ao restaurante significa um cliente com fome esperando uma lasanha que ninguém vai preparar. Quase todo sistema escolhe **pelo menos uma vez**, e quase todos os brokers trabalham assim por padrão, incluindo o SQS.

Existe uma piada clássica entre quem trabalha com isso: "Os dois problemas mais difíceis em sistemas distribuídos são: 2. entregar a mensagem exatamente uma vez; 1. garantir a ordem das mensagens; 2. entregar a mensagem exatamente uma vez." Quem ri de nervoso já passou por isso.

> **Armadilha:** algumas ferramentas anunciam entrega "exatamente uma vez" (*exactly-once*). Leia as letras miúdas. Em geral, a garantia vale dentro do próprio sistema (o Kafka, por exemplo, consegue isso entre tópicos dele mesmo) ou dentro de uma janela de deduplicação limitada. No instante em que o worker manda um e-mail ou chama a API de um restaurante, a garantia acaba. O efeito "exatamente uma vez" no mundo real é algo que o seu código constrói.

E como o seu código constrói isso? Com idempotência.

Idempotência: repetir sem estragar

Uma operação é **idempotente** quando executá-la duas vezes tem o mesmo efeito que executá-la uma vez. `pedido.status = "pago"` é idempotente: rodar de novo não muda nada. Já `saldo -= 58.90` tira mais dinheiro a cada execução. É a operação favorita do banco e a menos favorita do cliente.

Você já conhece isso do HTTP. `GET`, `PUT` e `DELETE` são idempotentes pela especificação; `POST` não. Foi por isso que, no capítulo 4, o Nginx reenviou as requisições `GET` para outra instância quando uma caiu, mas deixou o `POST /pedidos` falhar. Ele não tinha como saber se a instância chegou a cobrar o cartão antes de cair.

A solução clássica para tornar um `POST` seguro de repetir é a **chave de idempotência** (*idempotency key*). O cliente gera um identificador único para aquela intenção ("este pedido, desta tela") e o envia num cabeçalho. O servidor guarda a chave junto com a resposta. Se a mesma chave chegar de novo, ele devolve a resposta guardada, sem cobrar outra vez.

O detalhe que faz isso funcionar está em **quem gera a chave**. Se o servidor gerasse, cada clique viraria uma chave nova e nada mudaria. Quem gera é o React, uma única vez quando a tela de confirmação abre. O cliente pode clicar dez vezes (e vai, porque todo usuário acredita que clicar com mais força faz a internet andar mais rápido), a rede pode reenviar, o usuário pode atualizar a página no meio: todas as tentativas carregam a mesma chave.

> **Na vida real:** a API da Stripe, uma das mais usadas para pagamentos, aceita o cabeçalho `Idempotency-Key` em requisições `POST` exatamente para isso. Se uma requisição de cobrança falha por timeout, o cliente reenvia com a mesma chave, e a Stripe devolve o resultado da primeira tentativa em vez de cobrar duas vezes. Uma empresa que vive de cobrar levou muito a sério a ideia de não cobrar demais. O gateway do Rangô, nos exemplos deste capítulo, segue o mesmo modelo.

Do lado dos workers, vale o mesmo princípio. O evento chega com o ID do pedido. Antes de enviar o e-mail, o worker verifica se já existe um registro "e-mail do pedido 8f3a... enviado". Se existir, ele confirma a mensagem e segue. Assim, uma entrega repetida vira uma operação inofensiva.

Repare que a verificação reduz a janela de duplicação, sem fechá-la por completo. Se o worker enviar o e-mail e morrer antes de gravar a marca, a próxima entrega envia de novo. Para e-mail, um duplicado raro é aceitável. Para dinheiro, você empurra a idempotência até o sistema que move o dinheiro, passando a chave para o gateway, como a Stripe permite. A regra prática: a proteção mais forte fica o mais perto possível do efeito.

Tentar de novo, mas com calma

Falhas passageiras são o normal em sistemas distribuídos. O provedor de e-mail devolve `503` por um segundo, uma conexão cai, um DNS demora. Tentar de novo resolve a maioria delas. Só que existe um jeito errado de tentar de novo, e ele já derrubou sistemas grandes.

O provedor de e-mail daquela quinta-feira falhava porque recebia mais do que aguentava. Se cada worker tentar de novo imediatamente, sem parar, a carga sobre quem já está caindo se multiplica, e ele não se recupera. Esse efeito se chama **tempestade de retentativas** (*retry storm*). É a estratégia de quem aperta F5 sem parar no site de venda de ingressos de um show disputado, só que automatizada e multiplicada por mil workers.

A primeira correção é o **backoff exponencial**: esperar cada vez mais entre as tentativas. Meio segundo, depois 1, depois 2, depois 4, com um teto (30 segundos, por exemplo). O sistema que está caindo ganha fôlego.

A segunda correção é menos óbvia. Se mil workers falharem no mesmo instante e todos usarem backoff exponencial, todos esperam exatamente 1 segundo e voltam juntos, como uma multidão empurrando a porta no mesmo segundo. A solução é o **jitter**: sortear o tempo de espera dentro do intervalo. Em vez de esperar 4 segundos, cada worker espera

um valor aleatório entre 0 e 4. As tentativas se espalham. É o único lugar da computação em que um pouco de bagunça de propósito organiza tudo. É a mesma ideia do TTL com jitter que você usou no cache do capítulo 4.

```
espera = random.uniform(0, min(teto, base * 2 ** tentativa))
```

Retentativa também precisa de limite. Sem ele, uma mensagem que nunca vai dar certo fica girando para sempre. E retentativa só é segura em operação idempotente: repetir uma cobrança sem chave de idempotência pode resolver um timeout e criar uma cobrança dupla.

> **Na vida real:** em 7 de dezembro de 2021, a região `us-east-1` da AWS teve uma interrupção de várias horas. Segundo o relatório publicado pela própria AWS, uma atividade automática de escala de um serviço provocou um comportamento inesperado em um grande número de clientes da rede interna, que geraram um pico de conexões e congestionaram os dispositivos entre a rede interna e a rede principal. O congestionamento aumentou a latência, o que provocou ainda mais tentativas de conexão e retentativas. Os clientes tinham lógica de backoff testada, mas um defeito latente impediu que ela funcionasse naquele evento. Até a AWS, que publicou o texto mais citado sobre backoff com jitter, foi atingida por retentativas que não recuaram.

A mensagem envenenada

Algumas mensagens nunca vão dar certo. O e-mail do cliente é `ana@@exemplo.com`, e o provedor recusa. Alguém digitou com pressa, ou o arroba gaguejou. O evento foi publicado com um campo faltando por causa de um bug. Nenhuma quantidade de retentativas resolve isso.

Essa é a **mensagem envenenada** (*poison message*). O nome é dramático, mas justo: basta uma para estragar o almoço de todos os workers. Sem tratamento, ela é reentregue para sempre, consome o tempo dos workers e, em alguns brokers, bloqueia as mensagens que vêm atrás dela.

A saída é a **fila de mensagens mortas**, ou *dead letter queue* (DLQ). Depois de um número máximo de tentativas, a mensagem é movida para uma fila separada, junto com a causa do erro, e sai do caminho. A fila principal volta a andar. A DLQ funciona como a caixa de "pedidos com problema" no balcão: ninguém joga fora, alguém olha com calma depois, corrige a causa e reprocessa.

Uma DLQ só tem valor se alguém olhar para ela. Uma DLQ sem alarme é uma lixeira com nome bonito. O alarme que dispara quando ela recebe a primeira mensagem é assunto do capítulo 6 e, na AWS, do capítulo 22.

Consistência eventual

Com o pedido assíncrono, aparece uma situação nova. O cliente recebe "pedido recebido" às 21h22min03s. O painel do restaurante mostra o pedido às 21h22min05s. Durante 2 segundos, o sistema discorda de si mesmo: para a API, o pedido existe; para o restaurante, ainda não.

É o "já estou saindo de casa" da computação: uma afirmação que vai ser verdade em algum momento, não necessariamente agora.

Isso se chama **consistência eventual**: se nada mais mudar, todas as partes acabam concordando, mas não instantaneamente. É o oposto do que você está acostumado com uma transação no Postgres, em que o `COMMIT` torna tudo visível de uma vez.

O cuidado está na interface. Se a tela seguinte busca "meus pedidos" num lugar atualizado por um worker, o pedido recém-criado pode não aparecer, e o cliente confirma de novo. Mostre o pedido a partir da resposta da própria API e deixe a atualização chegar depois.

A ordem também deixa de ser garantida: com vários workers, `pedido.cancelado` pode ser processado antes de `pedido.criado`, e o restaurante recebe o cancelamento de uma lasanha que nem sabia que existia. Eventos com número de versão permitem ao consumidor ignorar uma informação mais antiga do que a que já tem.

Por fim, a **escrita dupla**: a API grava o pedido e depois publica o evento. Se ela cair entre as duas operações, ninguém avisa o restaurante. O padrão **transactional outbox** resolve isso gravando o pedido e o evento numa tabela `outbox`, na mesma transação do banco, e deixando um processo separado publicar o que está nela. Quando os pedidos do Rangô forem para o Postgres, ele merece atenção.

As oito falácias

Em 1994, Peter Deutsch, então na Sun Microsystems, listou suposições que todo desenvolvedor faz ao escrever o primeiro sistema distribuído, e que estão todas erradas. A lista original tinha sete; James Gosling acrescentou a oitava depois. Combina que a falácia da rede homogênea tenha vindo do criador do Java, a linguagem do slogan

"escreva uma vez, rode em qualquer lugar". Ele conhecia de perto o trabalho que dava cumprir essa promessa. Elas ficaram conhecidas como **as falácias da computação distribuída**:

1. A rede é confiável.
2. A latência é zero.
3. A largura de banda é infinita.
4. A rede é segura.
5. A topologia não muda.
6. Existe um único administrador.
7. O custo de transporte é zero.
8. A rede é homogênea.

A lista tem mais de trinta anos e continua atual, o que diz pouco sobre a lista e bastante sobre nós.

A rota síncrona do Rangô acreditava nas duas primeiras: que o provedor de e-mail responderia sempre e rápido. O gateway, o provedor e a API do restaurante pertencem a empresas diferentes, cada uma com seus deploys de sexta-feira (falácia 6). E a falácia 7 vai aparecer na sua fatura no capítulo 24, como tráfego de saída.

Na prática: o pedido sai da requisição

O exemplo reproduz a quinta-feira, separa o pedido em uma parte síncrona e uma assíncrona com Redis Streams, adiciona idempotência e mostra uma mensagem sendo reentregue e outra indo para a DLQ. As saídas aparecem todas, então dá para acompanhar a quinta-feira inteira sem precisar sofrer uma.

Pré-requisitos: o projeto como terminou o capítulo 4 (`compose.yaml` com `api` , `redis` e `proxy` , certificados do mkcert em `certs/`), Docker com Compose e o `uuidgen` , que vem no macOS e na maioria das distribuições Linux. Os comandos usam a variável `CA` do capítulo anterior:

```
CA="$(mkcert -CAROOT)/rootCA.pem"
```

Etapa 1: serviços externos de mentira

O Rangô ainda não tem gateway de pagamento nem provedor de e-mail de verdade. Para os testes, entra em cena um módulo que simula os três serviços externos, com atraso e taxa de falha controlados por variáveis de ambiente. Eles têm uma vantagem enorme sobre os de verdade: falham exatamente quando alguém pede.

```python
# backend/app/externos.py
import os
import random
import time


class ServicoIndisponivel(Exception):
    pass


def _simular(nome: str, atraso_padrao: float) -> None:
    time.sleep(float(os.environ.get(f"ATRASO_{nome}", atraso_padrao)))
    if random.random() < float(os.environ.get(f"FALHA_{nome}", "0")):
        raise ServicoIndisponivel(f"{nome.lower()} respondeu 503")


def cobrar_cartao(pedido_id: str, total: float, chave: str) -> str:
    _simular("PAGAMENTO", 2.5)
    return f"cobranca-{chave[:8]}"


def avisar_restaurante(pedido: dict) -> None:
    _simular("RESTAURANTE", 2.0)
    print(f"restaurante avisado: pedido {pedido['id']}", flush=True)


def enviar_email(pedido: dict) -> None:
    _simular("EMAIL", 1.0)
    print(f"e-mail enviado: pedido {pedido['id']} para {pedido['usuario']}", flush=True)
```

Agora a versão síncrona da rota, igual à que travou na quinta-feira. No `main.py`, entra este trecho:

```python
# backend/app/main.py (versão síncrona, para reproduzir o problema)
import uuid

from app.externos import avisar_restaurante, cobrar_cartao, enviar_email


class NovoPedido(BaseModel):
    usuario: str
    restaurante: str
    total: float


@app.post("/pedidos")
def criar_pedido(novo: NovoPedido):
    pedido = {"id": str(uuid.uuid4()), **novo.model_dump()}
    cobrar_cartao(pedido["id"], novo.total, pedido["id"])
    avisar_restaurante(pedido)
    enviar_email(pedido)
    return {"id": pedido["id"], "status": "confirmado"}
```

No `compose.yaml`, a imagem da API muda para `rango-api:0.4.0` e o atraso do e-mail é repassado, com 1 segundo como padrão:

```
# compose.yaml (trecho do serviço api)
  api:
    build: ./backend
    image: rango-api:0.4.0
    environment:
      CORS_ORIGINS: "https://rango.test:5173"
      REDIS_URL: "redis://redis:6379/0"
      ATRASO_EMAIL: "${ATRASO_EMAIL:-1}"
```

Com tudo de pé, a medição fica assim:

```
docker compose up -d --build --scale api=3
docker compose restart proxy

curl -s -o /dev/null --cacert "$CA" -w "%{http_code} em %{time_total}s\n" \
  -X POST https://api.rango.test:8443/pedidos \
  -H "Content-Type: application/json" \
  -d '{"usuario": "ana", "restaurante": "Cantina da Nona", "total": 58.9}'
```

```
200 em 5.531s
```

Agora a quinta-feira. A API é recriada com o e-mail lento, e o mesmo `curl` é repetido:

```
ATRASO_EMAIL=20 docker compose up -d --scale api=3
docker compose restart proxy
```

```
200 em 24.538s
```

Vinte e quatro segundos e meio. Nesse tempo, o cliente desiste, baixa o app do concorrente e ainda cria uma conta.

Com o e-mail lento, o teste seguinte dispara 150 pedidos em paralelo e mede a listagem de restaurantes logo em seguida:

```
for i in $(seq 150); do
  curl -s -o /dev/null --cacert "$CA" -X POST https://api.rango.test:8443/pedidos \
    -H "Content-Type: application/json" \
    -d '{"usuario": "ana", "restaurante": "Cantina da Nona", "total": 58.9}' &
done
sleep 1
curl -s -o /dev/null --cacert "$CA" -w "listagem em %{time_total}s\n" \
  https://api.rango.test:8443/restaurantes
```

As três instâncias rodam um processo Uvicorn cada, com 40 threads para rotas síncronas: 120 lugares para 150 pedidos. As threads ficam todas ocupadas esperando o e-mail, e a listagem, que respondia em milissegundos com o cache quente, entra na fila atrás dos pedidos e passa de 20 segundos. É a quinta-feira em miniatura.

Antes de seguir, é preciso esperar os pedidos concluírem.

Etapa 2: o evento vai para um stream

O Redis tem uma estrutura feita para isso: o **stream**, um log de mensagens em que só se acrescenta no fim. Cada mensagem ganha um ID com o horário em milissegundos (`1760479323512-0`). Com os **grupos de consumidores** do Redis, cada grupo recebe todas as mensagens do stream (o pub/sub) e, dentro de um grupo, cada mensagem vai para um só consumidor (a fila). O Redis mantém, para cada grupo, uma lista de mensagens entregues e ainda sem ack, a *pending entries list*. É ela que garante o "pelo menos uma vez".

Os comandos que importam são poucos: `XADD` publica, `XREADGROUP` lê como membro de um grupo, `XACK` confirma. Os nomes parecem siglas de repartição pública, mas, ao contrário das repartições, fazem exatamente o que prometem.

A versão nova substitui o trecho inteiro da etapa anterior. Ela cobra o cartão, grava o pedido, publica `pedido.criado` e responde `202 Accepted`, o código HTTP para "recebi e vou processar". É o "pode deixar comigo" do protocolo, com a diferença de que, aqui, alguém vai mesmo fazer:

```python
# backend/app/main.py (pedido assíncrono e idempotente)
import json
import uuid
from typing import Annotated

from fastapi import Header, HTTPException

from app.externos import cobrar_cartao

STREAM_PEDIDOS = "eventos:pedidos"


class NovoPedido(BaseModel):
    usuario: str
    restaurante: str
    total: float


@app.post("/pedidos", status_code=202)
def criar_pedido(
    novo: NovoPedido,
    chave: Annotated[str, Header(alias="Idempotency-Key")],
):
    chave_idem = f"idem:{chave}"
    if not cache.set(chave_idem, "processando", nx=True, ex=86400):
        anterior = cache.get(chave_idem)
        if anterior == "processando":
            raise HTTPException(409, "Pedido em processamento")
        return json.loads(anterior)

    pedido = {"id": str(uuid.uuid4()), **novo.model_dump()}
    try:
        pedido["cobranca"] = cobrar_cartao(pedido["id"], novo.total, chave)
    except Exception:
        cache.delete(chave_idem)
```

```
        raise HTTPException(502, "Pagamento indisponível, tente de novo")

    cache.hset(f"pedido:{pedido['id']}", mapping={**pedido, "status": "recebido"})
    cache.xadd(STREAM_PEDIDOS, {"tipo": "pedido.criado", "dados": json.dumps(pedido)})

    resposta = {"id": pedido["id"], "status": "recebido"}
    cache.set(chave_idem, json.dumps(resposta), ex=86400)
    return resposta
```

A idempotência está nas primeiras linhas. O `SET` com `nx=True` só grava se a chave ainda não existir, e o Redis faz isso de forma atômica: se dois cliques chegarem a duas instâncias no mesmo milissegundo, só um deles consegue gravar. O outro encontra `"processando"` e recebe `409 Conflict`. Quando o pedido termina, a chave passa a guardar a resposta, e qualquer repetição nas próximas 24 horas recebe exatamente o mesmo pedido. Se o pagamento falhar, a chave é apagada, e o cliente pode tentar de novo com a mesma chave.

Note também que a chave de idempotência vai para o gateway em `cobrar_cartao`. Essa é a proteção mais perto do efeito: mesmo que tudo do lado do Rangô falhe, o gateway não cobra duas vezes a mesma chave.

> **Armadilha:** nesta demonstração, o pedido fica num hash do Redis para não precisarmos montar o Postgres agora. Como o capítulo 4 alertou, o Redis guarda tudo em memória. Pedido confirmado mora num banco com durabilidade, e o Rangô fará essa mudança no capítulo 12, com o Amazon RDS.

Etapa 3: os workers

Cada assinante é um processo separado, usando a mesma imagem da API com outro comando. O worker fica assim:

```
# backend/app/worker.py
import json
import os
import random
import socket
import sys
import time

import redis

from app.externos import avisar_restaurante, enviar_email

STREAM = "eventos:pedidos"
DLQ = "eventos:pedidos:dlq"
TAREFAS = {"restaurante": avisar_restaurante, "email": enviar_email}
MAX_TENTATIVAS = 5

r = redis.Redis.from_url(os.environ["REDIS_URL"], decode_responses=True)
```

```
def criar_grupo(grupo: str) -> None:
    try:
        r.xgroup_create(STREAM, grupo, id="0", mkstream=True)
    except redis.ResponseError as erro:
        if "BUSYGROUP" not in str(erro):
            raise


def com_retentativas(tarefa, pedido: dict) -> None:
    for tentativa in range(1, MAX_TENTATIVAS + 1):
        try:
            tarefa(pedido)
            return
        except Exception as erro:
            if tentativa == MAX_TENTATIVAS:
                raise
            espera = random.uniform(0, min(30, 0.5 * 2**tentativa))
            print(f"tentativa {tentativa} falhou ({erro}); nova em {espera:.1f}s",
flush=True)
            time.sleep(espera)


def processar(grupo: str, msg_id: str, campos: dict) -> None:
    pedido = json.loads(campos["dados"])
    marca = f"feito:{grupo}:{pedido['id']}"
    if r.exists(marca):
        print(f"pedido {pedido['id']} já processado, ignorando", flush=True)
    else:
        try:
            com_retentativas(TAREFAS[grupo], pedido)
            r.set(marca, 1, ex=7 * 86400)
        except Exception as erro:
            r.xadd(DLQ, {**campos, "grupo": grupo, "erro": str(erro)})
            print(f"pedido {pedido['id']} foi para a DLQ: {erro}", flush=True)
    r.xack(STREAM, grupo, msg_id)


def main() -> None:
    grupo = sys.argv[1]
    consumidor = socket.gethostname()
    criar_grupo(grupo)
    print(f"worker {grupo} ({consumidor}) aguardando eventos", flush=True)
    inicio = "0"
    while True:
        resposta = r.xreadgroup(grupo, consumidor, {STREAM: inicio}, count=10,
block=5000)
        mensagens = resposta[0][1] if resposta else []
        if inicio == "0" and not mensagens:
            inicio = ">"
            continue
        for msg_id, campos in mensagens:
            processar(grupo, msg_id, campos)


if __name__ == "__main__":
    main()
```

Ao iniciar, o worker lê com o ID `"0"`, que no `XREADGROUP` significa "me devolva as mensagens entregues a mim e não confirmadas", ou seja, as que ele processava quando morreu. Só depois de esvaziar essa lista ele passa para `">"`, as mensagens novas. O nome

do consumidor vem do hostname do container, que se mantém quando o mesmo container é reiniciado. Antes de pegar trabalho novo, ele termina o que deixou pela metade, um hábito que muita gente com anos de carreira ainda não adquiriu.

Em `processar`, a ordem é: o trabalho, depois a marca `feito:email:<id>`, por último o `XACK`. Uma mensagem que vai para a DLQ também recebe ack, porque foi tratada, só que com outro destino.

O `compose.yaml` ganha os dois workers, com os atrasos e as falhas liberados por variável:

```yaml
# compose.yaml
services:
  api:
    build: ./backend
    image: rango-api:0.4.0
    environment:
      CORS_ORIGINS: "https://rango.test:5173"
      REDIS_URL: "redis://redis:6379/0"
    depends_on:
      - redis
    restart: unless-stopped

  worker-restaurante:
    build: ./backend
    image: rango-api:0.4.0
    command: ["python", "-m", "app.worker", "restaurante"]
    environment:
      REDIS_URL: "redis://redis:6379/0"
      ATRASO_RESTAURANTE: "${ATRASO_RESTAURANTE:-2}"
      FALHA_RESTAURANTE: "${FALHA_RESTAURANTE:-0}"
    depends_on:
      - redis
    restart: unless-stopped

  worker-email:
    build: ./backend
    image: rango-api:0.4.0
    command: ["python", "-m", "app.worker", "email"]
    environment:
      REDIS_URL: "redis://redis:6379/0"
      ATRASO_EMAIL: "${ATRASO_EMAIL:-1}"
      FALHA_EMAIL: "${FALHA_EMAIL:-0}"
    depends_on:
      - redis
    restart: unless-stopped

  redis:
    image: redis:7.4-alpine
    restart: unless-stopped

  proxy:
    image: nginx:1.27-alpine
    ports:
      - "127.0.0.1:8443:443"
    volumes:
      - ./nginx/rango.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on:
```

```
      - api
    restart: unless-stopped
```

Os três serviços apontam para a mesma imagem com `build`, para que o Compose saiba construí-la em vez de tentar baixá-la de um registry. Tudo sobe com o e-mail lento de novo, para comparar com a quinta-feira:

```
ATRASO_EMAIL=20 docker compose up -d --build --scale api=3
docker compose restart proxy
```

Agora, um pedido com uma chave de idempotência:

```
CHAVE=$(uuidgen)
curl -s --cacert "$CA" -w "\n%{http_code} em %{time_total}s\n" \
  -X POST https://api.rango.test:8443/pedidos \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $CHAVE" \
  -d '{"usuario": "ana", "restaurante": "Cantina da Nona", "total": 58.9}'
```

```
{"id":"8f3a2c1e-5b7d-4e9a-a1c2-3d4e5f6a7b8c","status":"recebido"}
202 em 2.524s
```

De 24,5 para 2,5 segundos, com o mesmo provedor de e-mail lento. Os logs dos workers contam o resto:

```
docker compose logs -f worker-restaurante worker-email
```

```
worker-restaurante-1  | restaurante avisado: pedido 8f3a2c1e-5b7d-4e9a-a1c2-3d4e5f6a7b8c
worker-email-1        | e-mail enviado: pedido 8f3a2c1e-5b7d-4e9a-a1c2-3d4e5f6a7b8c para
ana
```

O restaurante foi avisado 2 segundos depois, e o e-mail saiu 20 segundos depois, sem ninguém esperando por ele. Repetindo exatamente o mesmo `curl`, com a mesma `$CHAVE`, a resposta é:

```
{"id":"8f3a2c1e-5b7d-4e9a-a1c2-3d4e5f6a7b8c","status":"recebido"}
202 em 0.011s
```

Mesmo ID, 11 milissegundos, nenhuma cobrança nova e nenhum evento novo. Para simular o clique duplo, o exemplo gera outra chave e dispara duas requisições ao mesmo tempo:

```
CHAVE=$(uuidgen)
for i in 1 2; do
  curl -s --cacert "$CA" -o /dev/null -w "%{http_code}\n" \
    -X POST https://api.rango.test:8443/pedidos \
    -H "Content-Type: application/json" \
    -H "Idempotency-Key: $CHAVE" \
    -d '{"usuario": "ana", "restaurante": "Cantina da Nona", "total": 58.9}' &
```

```
done
wait
```

```
409
202
```

Uma passou, a outra foi barrada. Se essa rota existisse naquela quinta-feira, a caixa de e-mail da sua sócia teria amanhecido bem mais tranquila.

Etapa 4: a chave nasce no React

No navegador, a chave precisa nascer uma vez por tela de confirmação. Um componente mínimo:

```tsx
// frontend/src/ConfirmarPedido.tsx
import { useState } from "react";

const API = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

type Props = { usuario: string; restaurante: string; total: number };

export function ConfirmarPedido({ usuario, restaurante, total }: Props) {
  const [chave] = useState(() => crypto.randomUUID());
  const [enviando, setEnviando] = useState(false);
  const [status, setStatus] = useState<string | null>(null);

  async function confirmar() {
    setEnviando(true);
    try {
      const resposta = await fetch(`${API}/pedidos`, {
        method: "POST",
        headers: { "Content-Type": "application/json", "Idempotency-Key": chave },
        body: JSON.stringify({ usuario, restaurante, total }),
      });
      const corpo = await resposta.json();
      setStatus(resposta.ok ? corpo.status : corpo.detail);
    } finally {
      setEnviando(false);
    }
  }

  return (
    <div>
      <button onClick={confirmar} disabled={enviando}>
        {enviando ? "Enviando..." : "Confirmar pedido"}
      </button>
      {status && <p>{status}</p>}
    </div>
  );
}
```

O `disabled` no botão ajuda, mas não cobre uma requisição repetida pela rede ou por um retry. Ele é o equivalente de interface a esconder o pote de biscoitos no alto do armário: funciona até alguém achar um banquinho. A chave no `useState` com função

inicializadora é gerada uma vez e sobrevive às novas renderizações. O `crypto.randomUUID()` só existe em contextos seguros, como HTTPS ou `localhost`, e o Rangô já roda em `https://rango.test` desde o capítulo 2.

Falta um ajuste no backend. O CORS tem esse talento: some durante semanas e reaparece justo quando tudo parecia pronto. O do `main.py` ainda libera só `GET`, como ficou desde o capítulo 1, e o navegador vai barrar o `POST` na requisição de *preflight*. A linha corrigida fica assim:

```
    allow_methods=["GET", "POST"],
```

O `allow_headers=["*"]` já cobre o cabeçalho `Idempotency-Key`.

Etapa 5: derrubando um worker no meio do trabalho

Com o e-mail ainda em 20 segundos, o exemplo envia um pedido novo (com uma chave nova) e, logo depois, mata o worker de e-mail sem dar tempo de ele terminar:

```
CHAVE=$(uuidgen)
curl -s --cacert "$CA" -X POST https://api.rango.test:8443/pedidos \
  -H "Content-Type: application/json" -H "Idempotency-Key: $CHAVE" \
  -d '{"usuario": "bruno", "restaurante": "Sabor do Sertão", "total": 42.0}'
sleep 5
docker compose kill worker-email
```

O `kill` manda `SIGKILL`, o equivalente a puxar o cabo da tomada. O processo não tem direito nem a últimas palavras. O Redis, porém, anotou tudo:

```
docker compose exec redis redis-cli XPENDING eventos:pedidos email
```

```
1) (integer) 1
2) "1760479401733-0"
3) "1760479401733-0"
4) 1) 1) "9b1c4d2e8f7a"
      2) "1"
```

Uma mensagem entregue ao consumidor `9b1c4d2e8f7a` (o hostname do container) e nunca confirmada. O worker volta com os comandos abaixo, e os logs mostram:

```
docker compose start worker-email
docker compose logs -f worker-email
```

```
worker-email-1  | worker email (9b1c4d2e8f7a) aguardando eventos
worker-email-1  | e-mail enviado: pedido 4e7d1a9b-... para bruno
```

O worker leu primeiro a própria lista de pendentes e terminou o trabalho interrompido. Nada se perdeu. Lavoisier ficaria orgulhoso.

Etapa 6: a mensagem que vai para a DLQ

Agora o provedor de e-mail passa a falhar sempre, com atraso curto:

```
ATRASO_EMAIL=0.2 FALHA_EMAIL=1 docker compose up -d --scale api=3
docker compose restart proxy
```

Depois de enviar outro pedido com uma chave nova (o mesmo `curl` da etapa anterior), os logs mostram o backoff com jitter trabalhando. O worker insiste com a paciência de quem liga para o SAC e, na quinta tentativa, desiste com dignidade:

```
worker-email-1  | tentativa 1 falhou (email respondeu 503); nova em 0.7s
worker-email-1  | tentativa 2 falhou (email respondeu 503); nova em 1.3s
worker-email-1  | tentativa 3 falhou (email respondeu 503); nova em 0.4s
worker-email-1  | tentativa 4 falhou (email respondeu 503); nova em 6.2s
worker-email-1  | pedido c2a9e0f4-... foi para a DLQ: email respondeu 503
```

Os tempos variam a cada execução, por causa do jitter. A consulta à DLQ:

```
docker compose exec redis redis-cli XRANGE eventos:pedidos:dlq - +
```

A mensagem está lá, com o evento original, o grupo e o erro. O worker do restaurante, em outro grupo, avisou o restaurante normalmente.

Limpeza

Tudo roda local. Para derrubar:

```
docker compose down
```

O Redis não tem volume, então o stream, a DLQ e os pedidos somem junto. É a faxina mais rápida deste livro.

Armadilhas e casos reais

> **Armadilha:** retentativa em várias camadas. Se o React tenta 3 vezes, o Nginx tenta 3 vezes e a API tenta 3 vezes no gateway, uma única falha vira 27 chamadas ao serviço que já estava com problema. São os juros compostos, aplicados ao pior dia do seu fornecedor. Escolha uma camada para tentar de novo, normalmente a mais próxima da chamada, e faça as outras falharem rápido.
>
> **Armadilha:** timeout ausente. Uma chamada sem timeout pode esperar para sempre, e um worker preso para sempre é um worker a menos. A biblioteca `requests`, por exemplo, não define timeout nenhum por padrão. Ela confia na rede

com o otimismo de quem nunca leu a falácia número 1. Toda chamada de rede precisa de um limite explícito, menor que o timeout de quem está esperando por você.

Armadilha: worker que demora mais que o prazo do broker. No Redis Streams, uma mensagem pendente fica com o consumidor original até alguém reivindicá-la com `XAUTOCLAIM`. No SQS, a mensagem volta a ficar visível para outros workers depois de um prazo chamado *visibility timeout*. Se o processamento demorar mais que esse prazo, dois workers acabam processando a mesma mensagem ao mesmo tempo. O capítulo 21 mostra como ajustar isso.

Armadilha: o stream do Rangô mora no Redis, em memória e sem persistência. Se o container for recriado, os eventos pendentes desaparecem. Serviços como o SQS gravam as mensagens em disco e replicam entre máquinas.

Armadilha: chave de idempotência sem relação com o conteúdo. Se o cliente reenviar a mesma chave com outro total, o Rangô devolve a resposta antiga. Implementações mais cuidadosas guardam um hash do corpo junto com a chave e devolvem erro quando ele muda, como faz a Stripe.

Dica: no SQS, a idade da mensagem mais antiga é a métrica `ApproximateAgeOfOldestMessage`, e o Rangô ganha um alarme para ela no capítulo 22.

Resumo

- Comunicação síncrona amarra o tempo de resposta ao pior serviço da cadeia. A pergunta para cada etapa: o cliente precisa deste resultado para receber a resposta?
- Uma fila desacopla produtor e consumidor no tempo, absorve picos e permite escalar os workers de forma independente.
- Pub/sub entrega uma cópia de cada evento a cada assinante. Eventos descrevem fatos no passado (`pedido.criado`) e não conhecem quem os consome.
- Entre o efeito e o ack sempre existe uma janela. Quase todo sistema escolhe "pelo menos uma vez" e aceita duplicatas.
- Idempotência torna a repetição inofensiva: chave gerada pelo cliente, gravada com `SET NX` no servidor, repassada ao gateway, e marca de "já processado" nos workers.
- Retentativas precisam de backoff exponencial, jitter, limite e operação idempotente. Sem isso, viram tempestade.
- A DLQ tira do caminho mensagens que nunca vão dar certo, e só serve se alguém for avisado.

- Consistência eventual exige uma interface que tolere segundos de discordância e eventos fora de ordem.

Ponte: três da manhã

Sábado, 3h07. O Rangô está mais calmo do que nunca. Os pedidos entram, a API responde em 2,5 segundos, os workers avisam restaurantes e mandam e-mails, e a DLQ está vazia.

Às 3h12, o disco da máquina onde roda o Postgres de produção apresenta defeito. O banco volta, mas a tabela de pedidos está corrompida. Às 3h40, você está acordado, de pijama, olhando o terminal. Incidentes graves têm uma preferência inexplicável pela madrugada de sábado. Alguém lembra que existe um script de backup. Ele rodava no notebook do desenvolvedor que saiu da empresa em março. O backup, como a samambaia da recepção, era responsabilidade de alguém que não trabalha mais aqui. O último arquivo `.sql` que você encontra é de três semanas atrás, e ninguém nunca testou restaurar um deles.

Quantos pedidos você aceita perder? Quanto tempo o Rangô pode ficar fora do ar enquanto você restaura? Como saber que algo quebrou antes de um cliente reclamar no Instagram? E como fazer um deploy às 18h de uma sexta sem prender a respiração? No próximo capítulo, o último da Parte I, você vai ver o que separa um sistema que funciona de um sistema pronto para produção: backups, réplicas, RPO e RTO, infraestrutura como código, pipelines de deploy e observabilidade.

Referências

- Redis, introdução a Streams e grupos de consumidores: https://redis.io/docs/latest/develop/data-types/streams/
- Redis, comando `XREADGROUP`: https://redis.io/docs/latest/commands/xreadgroup/
- Redis, comando `XPENDING`: https://redis.io/docs/latest/commands/xpending/
- Redis, comando `XAUTOCLAIM`: https://redis.io/docs/latest/commands/xautoclaim/
- Redis, comando `SET` (opções `NX` e `EX`): https://redis.io/docs/latest/commands/set/
- FastAPI, parâmetros de cabeçalho: https://fastapi.tiangolo.com/tutorial/header-params/
- RFC 9110, métodos idempotentes: https://www.rfc-editor.org/rfc/rfc9110#section-9.2.2
- Stripe, requisições idempotentes: https://docs.stripe.com/api/idempotent_requests

- MDN, `crypto.randomUUID()`: https://developer.mozilla.org/en-US/docs/Web/API/Crypto/randomUUID
- Marc Brooker, "Exponential Backoff And Jitter", AWS Architecture Blog: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Amazon Builders' Library, "Timeouts, retries, and backoff with jitter": https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
- Amazon Builders' Library, "Making retries safe with idempotent APIs": https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/
- AWS, resumo da interrupção de 7 de dezembro de 2021 em `us-east-1`: https://aws.amazon.com/message/12721/
- Chris Richardson, padrão Transactional Outbox: https://microservices.io/patterns/data/transactional-outbox.html
- Falácias da computação distribuída: https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing

CAPÍTULO 6

Produção de verdade

No capítulo anterior, a confirmação do pedido deixou de esperar o restaurante e o e-mail. A API grava o pedido, publica `pedido.criado` num stream do Redis e responde em 2,5 segundos, enquanto workers idempotentes fazem o resto, com retentativas, backoff e uma DLQ para as mensagens que não têm conserto. O sistema ficou mais rápido e mais tolerante a falhas de terceiros. Só que toda essa engenharia parte de uma premissa silenciosa: o banco de dados continua lá amanhã. É a mesma premissa de quem deixa a marmita na geladeira do escritório na sexta e pretende almoçar na segunda.

Sábado, 3h40

Voltemos à madrugada em que o disco do Postgres falhou. Você está de pijama, com o terminal aberto, e encontrou o tal arquivo `.sql` de três semanas atrás. Decide restaurar. O primeiro `psql -f` para no meio com um erro de função inexistente, porque o dump foi gerado por uma versão do Postgres diferente da que roda hoje. Mensagem de erro de madrugada tem um efeito curioso: você lê três vezes e entende menos a cada leitura. Você ajusta, tenta de novo, e às 6h15 o banco está de pé. O sol também.

Com um buraco de três semanas.

Todos os pedidos feitos depois daquele backup sumiram. O repasse para a Cantina da Nona e para o Sabor do Sertão vai ter que ser reconstruído a partir do extrato do gateway de pagamento, linha por linha. Às 7h, o primeiro cliente reclama no Instagram que o histórico de pedidos dele está vazio. Você descobre a queda pelo Instagram, porque nada no Rangô avisou ninguém. O Rangô tinha, sim, um sistema de monitoramento: chamava-se cliente irritado.

Na segunda-feira, a sócia pergunta o que precisa mudar. A lista é longa: backups automáticos e testados, uma cópia do banco pronta para assumir, um jeito de recriar o servidor sem depender da memória de ninguém, deploys que não sejam um ato de fé e alguma forma de enxergar o sistema. A sócia ouve tudo com a calma de quem já começou a pesquisar quanto custa um curso de DevOps para o sócio. Este capítulo percorre essa lista, e cada item vira um serviço concreto na Parte II.

Backup só conta depois do primeiro restore

O script do desenvolvedor que saiu em março tinha três defeitos, e eles são os mais comuns do mundo. Rodava numa máquina que não era servidor, gravava a cópia num lugar só e nunca foi testado. Qualquer um dos três basta para você perder dados. O script conseguiu reunir os três, o que exige certo talento.

O começo são os tipos de backup de um banco relacional, porque eles se comportam de forma bem diferente.

O **backup lógico** é um retrato do conteúdo num arquivo que o próprio banco sabe reler. No Postgres, é o `pg_dump`. Ele é portável e permite restaurar só uma tabela, mas captura um instante: tudo o que aconteceu depois está perdido. Num banco grande, gerar e restaurar o dump demora horas.

O **backup físico** copia os arquivos de dados como estão no disco. O que muda o jogo é combiná-lo com o **WAL** (*write-ahead log*): antes de alterar qualquer dado, o Postgres escreve num log sequencial o que vai fazer. Com uma cópia física de domingo e todos os arquivos de WAL desde então, dá para reproduzir o histórico até qualquer segundo. Isso se chama **point-in-time recovery** (PITR), e é como ter o `git log` inteiro do banco: você faz *checkout* em "sábado, 3h11", um minuto antes do disco falhar. Ou um minuto antes de alguém rodar um `DELETE` sem `WHERE`, o desastre mais frequente na prática. Todo DBA com mais de dez anos de carreira tem uma história dessas, e quase todas começam com "foi só um ajuste rápido".

Depois vem o lugar onde a cópia mora. A regra clássica é a **3-2-1**: três cópias, em dois tipos de mídia, uma delas fora do local. Na nuvem, "fora do local" vira outra região ou outra conta, para que o mesmo erro (ou o mesmo invasor) não apague banco e backup de uma vez. Guardar o backup no mesmo disco do banco é como guardar a chave reserva de casa dentro de casa.

Por último, o teste. Restaurar precisa ser um procedimento automático e frequente, que confere o resultado: a tabela existe, a contagem bate, o pedido mais recente tem a data esperada. Descobrir que o backup está quebrado numa terça à tarde é bem melhor que num sábado às 3h40. Um backup nunca restaurado é um bilhete de loteria não conferido: pode valer tudo ou nada, e você só descobre no dia em que precisa.

RPO e RTO: quanto perder e quanto esperar

Duas perguntas da ponte do capítulo anterior têm nome técnico.

RPO (*Recovery Point Objective*) é a quantidade máxima de dados que você aceita perder, medida em tempo. Se o RPO é de cinco minutos, depois de um desastre você pode voltar a um estado de no máximo cinco minutos atrás. O RPO do Rangô naquela madrugada era, sem que ninguém tivesse escolhido, de três semanas. Foi a decisão de negócio mais importante do ano, e ninguém estava na reunião.

RTO (*Recovery Time Objective*) é quanto tempo o sistema pode ficar fora do ar até voltar a funcionar. Naquela noite, foram cerca de três horas entre a falha e o banco de pé, sem contar o trabalho de reconstruir os repasses, que exigiu uma planilha, o extrato do gateway e uma quantidade de café que não convém registrar.

Os dois números são decisões de negócio, e cada minuto a menos custa dinheiro. Um RPO de 24 horas sai com um `pg_dump` noturno; um de cinco minutos exige arquivamento contínuo de WAL; um próximo de zero exige uma réplica que confirma cada escrita. Com o RTO, o mesmo: restaurar um dump de 200 GB leva horas, promover uma réplica já ligada leva segundos.

Para o Rangô, a conversa com a sócia termina assim: perder até cinco minutos de pedidos é tolerável, porque o gateway de pagamento guarda as cobranças; ficar mais de uma hora fora numa sexta à noite, não.

Réplicas: primária, leitura e failover

Backup resolve o "quanto perder". Para encurtar o "quanto esperar", você precisa de outra cópia do banco já ligada.

Na **replicação**, um servidor é a **primária**: recebe todas as escritas. Os outros são **réplicas**: recebem o fluxo de WAL da primária e aplicam as mesmas mudanças, na mesma ordem. É parecido com um `git pull` contínuo, feito a cada poucos milissegundos.

A replicação pode ser **síncrona** ou **assíncrona**, e isso mexe direto no RPO. Na síncrona, a primária só confirma o `COMMIT` depois que a réplica recebeu a mudança: nada confirmado se perde, mas cada escrita fica mais lenta e, se a réplica travar, as escritas travam junto. Na assíncrona, a primária confirma na hora e envia logo depois, então as últimas transações podem não ter chegado à réplica no momento da falha. Esse atraso é o *replication lag*, normalmente de milissegundos, que pode virar minutos sob carga.

Uma réplica tem duas utilidades. A primeira é servir leituras, como o histórico de pedidos e os relatórios dos restaurantes, aliviando a primária. A pegadinha é o lag: o cliente confirma um pedido, a página seguinte lê da réplica, e o pedido ainda não está lá. O

cliente, desconfiado, aperta "Confirmar" de novo, e ainda bem que o capítulo 5 cuidou disso. É a consistência eventual do capítulo 5 dentro do banco. Leia da primária o que o usuário acabou de escrever.

A segunda é o **failover**: quando a primária morre, uma réplica é promovida e a aplicação passa a escrever nela. Automatizado, isso leva segundos, mas traz um risco novo, o **split brain**. Se a primária só perdeu a rede por 30 segundos e o sistema promoveu a réplica, você pode ter dois servidores aceitando escritas, cada um com uma versão dos pedidos. É o equivalente, em banco de dados, a dois gerentes dando ordens contrárias para a mesma equipe, cada um convencido de que é o chefe. Bons sistemas de failover isolam a primária antiga antes de a nova assumir.

> **Armadilha:** réplica protege contra falha de hardware, mas copia os erros com a mesma fidelidade, como o colega que cola a sua prova inteira, inclusive a resposta errada. Um `DROP TABLE pedidos` na primária chega à réplica em milissegundos. Por isso réplica e backup resolvem problemas diferentes, e você precisa dos dois.

No capítulo 12, o Amazon RDS vai fazer quase tudo isso por você (e cobrar por isso, como todo bom prestador de serviço): backups automáticos com PITR, uma réplica síncrona em outra zona para failover e réplicas de leitura assíncronas. Saber o que acontece por baixo é o que permite escolher a configuração certa.

Alta disponibilidade e recuperação de desastres

Os dois termos andam juntos e confundem. Aparecem tanto no mesmo slide que muita gente acha que são a mesma coisa com duas siglas.

Alta disponibilidade (HA, de *high availability*) é continuar funcionando quando um componente falha, de preferência sem ninguém perceber. É o tipo de trabalho que, quando bem feito, ninguém agradece, porque ninguém fica sabendo. Várias instâncias da API atrás de um balanceador, como no capítulo 4, e uma réplica pronta para failover lidam com falhas pequenas e frequentes: um processo que morre, um disco que falha.

Recuperação de desastres (DR, de *disaster recovery*) é o plano para algo grande: um data center inteiro fora do ar, um ataque de *ransomware*, um erro humano que apaga produção. DR mede-se em RPO e RTO e envolve backups em outro local, procedimentos escritos e ensaios.

Disponibilidade costuma ser expressa em "noves". Usando um mês de 730 horas, a tabela fica assim:

Disponibilidade	Indisponibilidade permitida por mês
99%	cerca de 7,3 horas
99,9%	cerca de 43,8 minutos
99,99%	cerca de 4,4 minutos
99,999%	cerca de 26 segundos

Cada nove a mais custa bem mais que o anterior. Quem promete cinco noves na reunião de vendas raramente é quem fica de plantão para cumpri-los. E existe uma conta que surpreende: disponibilidades de componentes em série se multiplicam. Se a API tem 99,9% e o banco também, o Rangô como um todo tem no máximo 99,8%, porque precisa dos dois ao mesmo tempo. Some o Redis, o gateway de pagamento e o DNS, e o número cai mais um pouco. Redundância melhora essa conta; cada dependência nova piora. Toda vez que alguém disser "é só mais uma dependência", essa conta merecia aparecer projetada na parede.

As estratégias de DR formam uma escada de custo. Na base, **backup e restauração**: você recria tudo a partir dos backups, com RTO de horas. Acima, o *pilot light* (banco replicado em outro local, servidores desligados e prontos) e o *warm standby* (uma cópia reduzida rodando o tempo todo). No topo, dois ambientes ativos ao mesmo tempo, com RTO próximo de zero e uma conta proporcional. Para o Rangô de hoje, backup e restauração bem feito já seria uma enorme melhora, desde que recriar o servidor não dependa de lembrar 40 comandos digitados há um ano e guardados com carinho no `~/.bash_history` de um servidor que acabou de morrer.

Segredos fora do código

Antes de colocar as mãos no banco, um detalhe que atrapalha quase todo projeto: onde fica a senha dele. Em muitos projetos, a resposta honesta é "num post-it, num `.env` e na memória do Carlos".

No Rangô, até agora, configuração entrou por variáveis de ambiente, e isso continua certo. O problema é de onde essas variáveis vêm. Um `.env` commitado por engano fica no histórico do git mesmo depois de apagado, e qualquer clone antigo do repositório carrega a senha. O git, por projeto, não esquece nada; é o parente que lembra de tudo o que você disse no Natal de 2009. Uma senha colada no Slack fica no histórico do Slack. E, se a senha for `rango123`, ela fica também ao alcance de qualquer pessoa com dois minutos de paciência para adivinhar. Uma senha escrita no `compose.yaml` vai para o repositório junto com ele.

A **gestão de segredos** se apoia em poucas regras. O segredo mora num lugar próprio, com controle de quem lê, e a aplicação o recebe em tempo de execução, nunca no código nem na imagem Docker. Cada ambiente tem os próprios segredos. E a **rotação** precisa ser possível sem deploy, porque um dia você vai trocar uma senha às pressas. Normalmente numa sexta-feira, minutos depois de alguém dizer "acho que commitei uma coisa".

Na AWS, esse lugar próprio será o AWS Systems Manager Parameter Store ou o AWS Secrets Manager, assunto do capítulo 16. Localmente, o Docker Compose tem um mecanismo de *secrets* que monta o conteúdo de um arquivo dentro do container, em `/run/secrets/`, sem passar por variável de ambiente. É ele que aparece na prática logo a seguir.

Na prática: backup, desastre e restore testado

O Rangô ainda guarda os pedidos no Redis, e a mudança definitiva para o Postgres acontece no capítulo 12, com o RDS. Para treinar backup e restauração, o exemplo sobe um Postgres no Compose com uma tabela de pedidos preenchida com dados de teste. A vantagem dos dados de teste é que nenhum restaurante liga reclamando quando eles somem.

Pré-requisitos: Docker com Compose v2, `openssl` e o projeto do capítulo 5. Os comandos rodam na raiz do projeto, num terminal bash (no Windows, o equivalente é o WSL ou o Git Bash).

Primeiro, a senha, gerada aleatoriamente e mantida fora do git:

```
mkdir -p secrets backups scripts
openssl rand -base64 24 | tr -d '\n' > secrets/db_password.txt
printf 'secrets/\nbackups/\n' >> .gitignore
```

Agora o `compose.yaml` ganha o serviço `db`, o volume e o segredo. As chaves `volumes` e `secrets` de nível superior ficam fora de `services`:

```
# compose.yaml (trechos novos)
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: rango
      POSTGRES_DB: rango
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - dados-db:/var/lib/postgresql/data
    restart: unless-stopped
```

```
volumes:
  dados-db:

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

A imagem oficial do Postgres aceita a variante `_FILE` de várias variáveis e lê a senha do arquivo montado. Assim, `docker inspect` não mostra a senha nas variáveis do container. Não há `ports`: o banco só é alcançável pela rede interna do Compose, a mesma lição do capítulo 1 sobre bancos expostos.

Com o banco no ar, entram 5 mil pedidos de teste. O `psql` roda dentro do container, por socket local, então não pede senha:

```
docker compose up -d db
docker compose exec -T db psql -U rango -d rango <<'SQL'
CREATE TABLE pedidos (
    id bigserial PRIMARY KEY,
    restaurante text NOT NULL,
    total numeric(10, 2) NOT NULL,
    criado_em timestamptz NOT NULL DEFAULT now()
);
INSERT INTO pedidos (restaurante, total)
SELECT (ARRAY['Cantina da Nona', 'Sabor do Sertão'])[1 + i % 2],
       round((20 + random() * 80)::numeric, 2)
FROM generate_series(1, 5000) AS i;
SQL
```

O script de backup usa o formato *custom* do `pg_dump` (`--format=custom`), que é compactado e permite restaurar tabelas isoladas com o `pg_restore`. Repare que o `pg_dump` roda dentro do container do banco. Isso garante que a versão da ferramenta é a mesma do servidor, e o caso da GitLab, mais adiante, mostra por que isso importa.

```
#!/usr/bin/env bash
# scripts/backup.sh
set -euo pipefail

arquivo="backups/rango-$(date -u +%Y%m%dT%H%M%SZ).dump"
docker compose exec -T db pg_dump -U rango -d rango --format=custom > "$arquivo"
echo "backup gravado em $arquivo ($(du -h "$arquivo" | cut -f1))"
```

O script de teste é a parte que faltava ao Rangô. Ele cria um Postgres descartável, restaura o backup nele, compara com a origem e apaga tudo no fim, dê certo ou não:

```
#!/usr/bin/env bash
# scripts/testar-restore.sh
set -euo pipefail

arquivo="${1:?uso: scripts/testar-restore.sh backups/ARQUIVO.dump}"
teste="rango-restore-teste"
```

```
docker run -d --name "$teste" \
  -e POSTGRES_USER=rango -e POSTGRES_DB=rango \
  -e POSTGRES_PASSWORD="$(openssl rand -hex 16)" \
  postgres:17-alpine > /dev/null
trap 'docker rm -f "$teste" > /dev/null' EXIT

# -h 127.0.0.1: durante a inicialização, a imagem sobe um servidor
# temporário só no socket; o TCP só responde quando o banco final está pronto
until docker exec "$teste" pg_isready -h 127.0.0.1 -U rango -d rango > /dev/null 2>&1;
do
  sleep 1
done

docker exec -i "$teste" pg_restore -U rango -d rango --no-owner < "$arquivo"

consulta="SELECT count(*) FROM pedidos"
restaurado=$(docker exec "$teste" psql -U rango -d rango -tAc "$consulta")
echo "pedidos no backup restaurado: $restaurado"

if [ "$restaurado" -lt 1 ]; then
  echo "FALHA: backup restaurado sem pedidos" >&2
  exit 1
fi
echo "OK: restore testado com sucesso"
```

Os dois em ação:

```
chmod +x scripts/*.sh
scripts/backup.sh
scripts/testar-restore.sh backups/rango-*.dump
```

```
backup gravado em backups/rango-20261016T021500Z.dump (128K)
pedidos no backup restaurado: 5000
OK: restore testado com sucesso
```

Agora dá para sentir o RPO na pele. Depois do backup, chegam mais 37 pedidos. Em seguida, acontece o desastre, provocado de propósito, o que já é mais controle do que a maioria dos desastres oferece:

```
docker compose exec -T db psql -U rango -d rango -c \
  "INSERT INTO pedidos (restaurante, total) SELECT 'Cantina da Nona', 42.90 FROM
generate_series(1, 37)"
docker compose exec -T db psql -U rango -d rango -c "DROP TABLE pedidos"
```

O backup volta para o banco principal, seguido de uma contagem:

```
docker compose exec -T db pg_restore -U rango -d rango --no-owner < backups/rango-*.dump
docker compose exec -T db psql -U rango -d rango -tAc "SELECT count(*) FROM pedidos"
```

```
5000
```

Os 37 pedidos feitos depois do backup se foram. Trinta e sete, de novo; esse número parece perseguir o Rangô. Com um `pg_dump` por hora, seu RPO é de até uma hora. Encurtar isso para minutos é o papel do arquivamento contínuo de WAL, que o RDS oferece pronto no capítulo 12.

Falta o backup rodar sozinho, ir para outro lugar e ter o restore testado periodicamente, com alerta se falhar. Na Parte II, o destino será o Amazon S3; localmente, um `cron` no servidor bastaria, desde que o servidor não seja o notebook de ninguém.

> **Dica:** um teste de restore bom mede também o tempo. Se restaurar levou 40 minutos no teste, seu RTO real nunca vai ser menor que 40 minutos, por mais que o documento de arquitetura diga outra coisa.

No fim, `docker compose down` para os containers. O volume `dados-db` continua existindo; para apagá-lo também, o comando é `docker compose down -v`.

Infraestrutura como código

Voltemos ao RTO. Para recriar o servidor do Rangô do zero, alguém precisaria instalar o Docker, criar o usuário `rango`, configurar o Nginx, instalar os certificados, ajustar o firewall e lembrar de um parâmetro do kernel que foi mudado num dia de pânico. Se isso está na cabeça de uma pessoa, o RTO depende das férias dela, e do sinal de celular na praia.

Infraestrutura como código (IaC) significa descrever servidores, redes, bancos, balanceadores e permissões em arquivos de texto versionados no git, e deixar uma ferramenta criar e atualizar tudo a partir deles. A melhor analogia para quem vem de Django ou Alembic é a *migration*: em vez de alterar o banco à mão no console, você escreve a mudança num arquivo, revisa num pull request e aplica de forma repetível. IaC faz isso com a infraestrutura inteira.

Você já usa uma forma disso. O `compose.yaml` do Rangô descreve containers, redes, volumes e segredos de forma declarativa: você diz o estado desejado e o Compose descobre o que precisa criar, recriar ou manter. A diferença para a produção é a escala do que se descreve.

As ferramentas se dividem em duas famílias. As **declarativas** descrevem o estado final ("quero dois servidores e um banco com estas configurações"), e a ferramenta calcula a diferença entre o que existe e o que foi pedido, a mesma ideia de reconciliação que você

viu no Kubernetes. Terraform, AWS CloudFormation e AWS CDK estão nessa família. As **imperativas** descrevem passos ("instale isto, depois copie aquilo"), como scripts de shell; elas funcionam bem na primeira execução e mal na segunda, como piada repetida.

Recriar o ambiente depois de um desastre vira um comando, e criar uma homologação idêntica à produção também. Toda mudança passa por revisão, então "quem abriu a porta 5432 para a internet?" tem resposta no `git blame`.

O inimigo da IaC é o **drift**: alguém muda algo direto no console, "só dessa vez" (frase dita umas quarenta vezes por ano), e o código deixa de descrever a realidade. A regra que funciona é tratar o console como somente leitura em produção.

No capítulo 15, o Rangô inteiro será descrito com o AWS CDK em Python, e a partir dali todo exemplo do livro usa código em vez de cliques. O mouse, finalmente, pode tirar as férias que nunca teve.

CI/CD: a esteira entre o commit e a produção

O deploy do Rangô ainda é uma pessoa rodando comandos. Em geral na sexta, às 18h, porque é quando a funcionalidade ficou pronta. Existe um ditado antigo entre quem opera sistemas: quem faz deploy na sexta trabalha no sábado.

Integração contínua (CI, de *continuous integration*) é a prática de, a cada push ou pull request, um servidor rodar automaticamente os testes, o *linter* e o build, para descobrir em minutos que uma mudança quebrou algo, em vez de descobrir pelo Instagram, um canal de monitoramento que já provou sair caro. **Entrega contínua** e **deploy contínuo** (ambos chamados de CD) estendem a esteira: na entrega, toda versão aprovada está pronta para produção e alguém aperta o botão; no deploy contínuo, ela vai sozinha.

O conjunto se chama *pipeline*, uma sequência de etapas em que cada uma só começa se a anterior passou: testes, build da imagem com tag única (o hash do commit, nunca `latest`), envio ao registry, homologação e produção. O ganho mais subestimado é a previsibilidade. Um deploy que roda igual todas as vezes deixa de depender de quem está de plantão, e dez deploys pequenos por dia ficam mais seguros que um gigante por mês, porque uma mudança pequena é fácil de achar e de reverter. Deploy gigante é como mudança de apartamento: sempre some uma caixa, e ninguém sabe dizer qual.

Na prática: o primeiro pipeline do Rangô

O exemplo usa o GitHub Actions, que roda de graça para repositórios públicos e tem uma cota mensal de minutos para privados. Primeiro, um teste, que precisa do arquivo `backend/requirements-dev.txt`:

```
pytest
httpx
```

O `httpx` é exigido pelo `TestClient` do FastAPI. Agora o teste em `backend/tests/test_saude.py`:

```python
# backend/tests/test_saude.py
from fastapi.testclient import TestClient

from app.main import app

cliente = TestClient(app)


def test_health_responde_ok():
    resposta = cliente.get("/health")
    assert resposta.status_code == 200
    assert resposta.json() == {"status": "ok"}


def test_restaurantes_tem_taxa_formatada():
    resposta = cliente.get("/restaurantes")
    assert resposta.status_code == 200
    assert all("taxa_entrega_fmt" in r for r in resposta.json())
```

O segundo teste precisa do Redis, porque `/restaurantes` usa cache. O pipeline vai subir um Redis como *service container*. Ele fica em `.github/workflows/ci.yaml`, na raiz do repositório:

```yaml
# .github/workflows/ci.yaml
name: ci

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  testes:
    runs-on: ubuntu-24.04
    services:
      redis:
        image: redis:7.4-alpine
        ports:
          - 6379:6379
    env:
      REDIS_URL: redis://localhost:6379/0
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
```

```
          python-version: "3.12"
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: python -m pytest -q

  imagem:
    needs: testes
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v5
      - run: docker build -t rango-api:${{ github.sha }} backend
```

O bloco `permissions` limita o token do pipeline a ler o código, menor privilégio desde o primeiro arquivo. O job `imagem` só roda se `testes` passar, por causa do `needs`, e a tag é o hash do commit, rastreável até a linha que gerou o build. O `python -m pytest` coloca o diretório atual no caminho de importação, para que `from app.main import app` funcione. Se alguém quebrar o `/health`, o pull request fica vermelho antes de qualquer revisão humana. O robô não tem pena, não tem sono e não aceita "depois eu arrumo".

O pipeline ainda para no build, porque falta o lugar para onde mandar a imagem. No capítulo 18, ele ganha o envio ao Amazon ECR e o deploy no Amazon ECS, autenticando na AWS por OIDC, sem chave de longa duração guardada no GitHub.

Estratégias de deploy: como trocar o motor com o carro andando

Com o pipeline pronto, a pergunta muda: como colocar a versão nova no ar sem derrubar a antiga de uma vez? Há três estratégias principais.

No **rolling update**, as instâncias são trocadas aos poucos: uma sai, a versão nova entra, passa no health check, e só então a próxima é trocada. É o que o Deployment do Kubernetes fazia no capítulo 3. Não exige capacidade extra, mas por alguns minutos as duas versões atendem juntas, e voltar atrás é outro rolling update no sentido inverso.

No **blue/green**, você mantém dois ambientes completos. O azul atende produção; o verde recebe a versão nova e é testado sem tráfego real. Depois, o balanceador passa a apontar para o verde de uma vez, e o rollback é apontar de volta, em segundos. O custo é o dobro da capacidade durante a troca. É como ter dois carros na garagem só para nunca ficar a pé: funciona, mas o seguro dói.

No **canary**, a versão nova recebe uma fração pequena do tráfego real, digamos 5%, enquanto você observa erros e latência. Se os números ficarem bons, a fração sobe até 100%; se piorarem, só 5% dos usuários sentiram por alguns minutos. O nome vem dos canários que mineiros levavam às minas de carvão. O pássaro sentia o gás antes das pessoas, e o mal-estar dele era o alarme. Na versão de software, o pior que acontece com o canário é um rollback. Essa estratégia depende de métricas confiáveis para decidir.

Complementam essas estratégias os *feature flags*: o código vai para produção desligado e uma configuração liga a funcionalidade para alguns usuários, separando o deploy do lançamento. É também a técnica mais elegante já inventada para esconder do chefe uma funcionalidade pela metade.

> **Armadilha:** a estratégia de deploy cuida da aplicação; o banco continua sendo um só. Se a versão nova renomeia a coluna `total` para `valor_total`, a versão antiga, que ainda atende durante o rolling ou o canary, começa a falhar. A saída é o padrão *expand/contract*: primeiro uma migration que adiciona a coluna nova e mantém a antiga, depois o código que usa a nova, e só num deploy posterior a migration que remove a antiga. Toda migration precisa ser compatível com a versão anterior do código.

Na prática: um canário com o Nginx

O Nginx do Rangô já balanceia entre as instâncias da API. Com pesos no `upstream`, ele faz um canário simples. O primeiro passo é gerar a imagem nova, que vai incluir os logs estruturados da próxima seção:

```
docker build -t rango-api:0.5.0 backend
```

No `compose.yaml`, entra um serviço `api-canario` igual ao `api`, mas com a imagem nova e um nome de host fixo. Lembrando: o middleware do capítulo 4 usa o nome do host no cabeçalho `X-Rango-Instancia`:

```
# compose.yaml (trecho novo)
  api-canario:
    image: rango-api:0.5.0
    hostname: canario
    environment:
      CORS_ORIGINS: "https://rango.test:5173"
      REDIS_URL: "redis://redis:6379/0"
    depends_on:
      - redis
    restart: unless-stopped
```

Em `nginx/rango.conf`, o canário entra no `upstream`, e as instâncias estáveis ganham peso:

```
upstream rango_api {
    server api:8000 weight=3 max_fails=2 fail_timeout=10s;
    server api-canario:8000 weight=1 max_fails=2 fail_timeout=10s;
}
```

Atenção à conta. O nome `api` resolve para três IPs, porque a API roda com `--scale api=3`, e cada IP herda o peso 3. São 9 partes para a versão estável e 1 para o canário, ou seja, 10% do tráfego. Com tudo no ar, 100 requisições mostram a divisão:

```
docker compose up -d --scale api=3
docker compose restart proxy
for i in $(seq 100); do
  curl -sk -o /dev/null -D - https://api.rango.test:8443/health | grep -i '^x-rango-instancia'
done | cut -d' ' -f2 | tr -d '\r' | sort | uniq -c
```

```
     30 3f9c1a7e2b4d
     30 8a21d4c6e0f3
     30 c7e5b2a91d08
     10 canario
```

Dez no canário, exatamente como a conta previu. A estatística raramente colabora tanto. Os IDs mudam de uma máquina para outra. Promover o canário é trocar a imagem do serviço `api` para `rango-api:0.5.0` e remover o `api-canario`. O rollback é tirar a linha do canário do `upstream` e reiniciar o proxy.

Observabilidade: saber antes do cliente

Na madrugada do disco, o Rangô ficou fora do ar e a primeira notícia veio do Instagram. Observabilidade é a capacidade de responder, olhando o que o sistema emite, a perguntas que você não previu: o que quebrou, desde quando, para quem, e por quê.

Ela se apoia em três tipos de sinal.

Os **logs** são registros de eventos individuais: "`POST /pedidos` retornou 500 em 2.380 ms". Com seis instâncias da API e dois workers, um `print` solto em cada container fica inútil. Procurar um erro assim é como procurar uma meia específica numa lavanderia industrial. Os logs precisam ser **estruturados** (JSON com campos fixos), centralizados e pesquisáveis.

As **métricas** são números agregados no tempo: requisições por segundo, percentual de erros, latência, tamanho da fila. São baratas e ótimas para alertas, mas não contam a história de uma requisição. O livro *Site Reliability Engineering*, do Google, sugere começar por quatro sinais de ouro: latência, tráfego, erros e saturação. Para latência, use percentis: uma média de 300 ms pode esconder que 1% das requisições, o *p99*, leva 9 segundos. A média é aquela velha piada de estatística: com a cabeça no forno e os pés no freezer, a pessoa está, em média, numa temperatura agradável.

Os **traces** acompanham uma requisição por todos os componentes. Um pedido do Rangô atravessa Nginx, API, gateway de pagamento, stream do Redis e workers, e o trace mostra a duração de cada trecho. É o rastreamento de encomenda que diz onde o pacote parou, em vez de "objeto em trânsito" por nove dias seguidos. Para isso, um identificador viaja junto, num cabeçalho HTTP ou num campo do evento. O padrão de mercado para os três sinais é o OpenTelemetry, que o capítulo 22 usa com a AWS.

Os sinais só servem se alguém olhar na hora certa, e por isso existem os **alertas**. Alerte sobre sintomas que o usuário sente (erros acima de 2% por cinco minutos, p99 acima de 3 segundos, mensagem mais antiga da fila parada há dez minutos), em vez de causas internas que podem ser inofensivas (CPU em 85%). Uma mensagem na DLQ do capítulo 5 também precisa disparar alerta. E todo alerta precisa exigir uma ação, senão a equipe aprende a ignorá-los, como o alarme de carro que dispara toda noite na rua e já não faz ninguém nem olhar pela janela.

Na prática: logs estruturados e um ID por requisição

A ideia é cada requisição da API gerar uma linha de log em JSON, com um identificador que também volta para o cliente. O formato fica em `backend/app/logs.py`:

```python
# backend/app/logs.py
import json
import logging
import sys
from datetime import datetime, timezone


class FormatoJson(logging.Formatter):
    def format(self, registro: logging.LogRecord) -> str:
        dados = {
            "ts": datetime.fromtimestamp(registro.created, timezone.utc).isoformat(),
            "nivel": registro.levelname,
            "msg": registro.getMessage(),
            **getattr(registro, "campos", {}),
        }
        return json.dumps(dados, ensure_ascii=False)


def configurar() -> None:
    saida = logging.StreamHandler(sys.stdout)
    saida.setFormatter(FormatoJson())
    logging.basicConfig(level=logging.INFO, handlers=[saida], force=True)
```

Em `backend/app/main.py`, os logs são configurados e o middleware entra. Os imports de `time` e `uuid` vão para o topo, junto dos outros; `INSTANCIA` já existe desde o capítulo 4:

```python
# backend/app/main.py (trechos novos)
import logging
import time
import uuid
```

```
from app import logs

logs.configurar()
log = logging.getLogger("rango.api")


@app.middleware("http")
async def registrar_requisicao(request, call_next):
    request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4())
    inicio = time.perf_counter()
    resposta = await call_next(request)
    resposta.headers["X-Request-ID"] = request_id
    log.info(
        "requisicao",
        extra={
            "campos": {
                "request_id": request_id,
                "metodo": request.method,
                "rota": request.url.path,
                "status": resposta.status_code,
                "duracao_ms": round((time.perf_counter() - inicio) * 1000, 1),
                "instancia": INSTANCIA,
            }
        },
    )
    return resposta
```

Se a requisição já trouxer um `X-Request-ID`, ele é reaproveitado; senão, a API cria um. O ID volta na resposta, e com ele você acha a linha exata do log.

Depois de reconstruir a imagem `rango-api:0.5.0` e subir o Compose (ele recria o canário com a imagem nova), o exemplo manda 20 requisições para `/restaurantes`. Com 10% do tráfego, umas duas devem cair no canário:

```
docker build -t rango-api:0.5.0 backend
docker compose up -d --scale api=3
for i in $(seq 20); do curl -sk https://api.rango.test:8443/restaurantes > /dev/null;
done
docker compose logs api api-canario --no-log-prefix | jq -cR 'fromjson? | select(.msg ==
"requisicao")'
```

A saída será parecida com esta. A primeira linha pegou o cache expirado e pagou os 300 ms simulados do banco; a segunda veio do Redis:

```
{"ts":"2026-10-
16T14:02:11.482+00:00","nivel":"INFO","msg":"requisicao","request_id":"5d0f8c2e-9b1a-
4f7e-a3c4-
1e2d3f4a5b6c","metodo":"GET","rota":"/restaurantes","status":200,"duracao_ms":304.7,"ins
tancia":"canario"}
{"ts":"2026-10-
16T14:02:11.913+00:00","nivel":"INFO","msg":"requisicao","request_id":"a17e6b90-2c4d-
4e8f-9a0b-
c1d2e3f40516","metodo":"GET","rota":"/restaurantes","status":200,"duracao_ms":1.9,"insta
ncia":"canario"}
```

Só o canário aparece porque as instâncias estáveis ainda rodam a 0.4.0. Por enquanto, ele é o único funcionário que preenche o relatório do que fez. O `fromjson?` do `jq` descarta as linhas que não são JSON, como as mensagens do Uvicorn.

Agora a pergunta "quais requisições passaram de 1 segundo?" vira um filtro, em vez de uma leitura de olhos apertados:

```
docker compose logs api api-canario --no-log-prefix \
  | jq -cR 'fromjson? | select(.duracao_ms > 1000) | {rota, status, duracao_ms,
request_id}'
```

Um limite: se a rota lançar uma exceção não tratada, `call_next` propaga o erro e a linha fica sem registro. Em produção, capture a exceção, registre com `status` 500 e relance. No capítulo 22, essas linhas vão para o Amazon CloudWatch Logs, consultadas com o Logs Insights.

Para desligar tudo, `docker compose down`.

Incidentes e postmortems

Mesmo com tudo isso, algo vai quebrar. A lei de Murphy não aceita pull request. O que muda é como a equipe reage.

Durante um **incidente**, ajuda ter papéis claros, mesmo numa equipe de três pessoas: alguém coordena, alguém mexe nos sistemas, alguém comunica clientes e restaurantes. Numa equipe de três, alguém acaba acumulando funções, e costuma ser quem já estava de pijama. A prioridade é estancar o problema. Se o deploy de 20 minutos atrás coincide com o aumento de erros, faça o rollback primeiro e investigue com o sistema estável. Anote horários e ações num canal único, porque a memória de madrugada falha. Às 4h, ninguém lembra se reiniciou o serviço uma ou três vezes, e todo mundo jura que foram duas.

Depois vem o **postmortem**: linha do tempo, impacto, causas e ações de correção com responsável e prazo. Ele precisa ser ***blameless***, sem culpados. A pergunta certa é "o que no sistema permitiu que uma pessoa apagasse produção com um comando?", e a errada é "quem apagou?". Se a conclusão for "a pessoa errou", a mesma falha volta com outra pessoa. O postmortem do sábado do Rangô, aliás, é este capítulo.

Armadilhas e casos reais

> **Na vida real:** em 31 de janeiro de 2017, um engenheiro da GitLab, cansado após horas lidando com um problema de replicação, rodou um comando para apagar o diretório de dados do banco no servidor errado: a primária, em vez da réplica. Cerca de 300 GB de dados de produção sumiram em segundos. Ao procurar os backups, a equipe descobriu que o `pg_dump` agendado falhava havia tempo sem ninguém saber: ele usava os binários do PostgreSQL 9.2 contra um servidor 9.6, e os e-mails de alerta sobre a falha eram rejeitados pelo servidor de e-mail. Os alarmes existiam; só não tinham para quem tocar. A recuperação veio de um snapshot de disco feito cerca de seis horas antes, por acaso, para popular o ambiente de homologação. Foram perdidas cerca de seis horas de dados, afetando milhares de projetos, comentários e contas de usuários. A GitLab transmitiu a restauração ao vivo e publicou um postmortem detalhado, que virou leitura obrigatória sobre backups.
>
> **Na vida real:** em 1º de agosto de 2012, a Knight Capital, grande operadora do mercado de ações americano, fez o deploy manual de um código novo em oito servidores, e um deles ficou sem a versão nova. O código novo reaproveitava uma flag que, no servidor desatualizado, ativou uma funcionalidade antiga, desativada havia anos. Em cerca de 45 minutos, o sistema enviou milhões de ordens. Segundo a SEC, o regulador americano, o prejuízo passou de 460 milhões de dólares. Dá mais de 10 milhões por minuto, uma taxa que nenhuma hora extra cobre. Um deploy automatizado e idêntico em todos os servidores teria evitado o problema.
>
> **Na vida real:** em 19 de julho de 2024, a CrowdStrike distribuiu uma atualização de configuração do seu agente de segurança para Windows a todos os clientes ao mesmo tempo. Um defeito no arquivo derrubou cerca de 8,5 milhões de computadores, segundo a Microsoft, parando companhias aéreas, hospitais e bancos. Entre as correções anunciadas pela empresa estava a distribuição gradual das atualizações, começando por um grupo pequeno: um canário.

Algumas armadilhas são menos famosas e muito mais frequentes:

- **Backup sem monitoramento.** O cron parou de rodar porque o disco encheu, e o último backup tem dois meses. Alerte quando o backup *não* acontecer, além de quando ele falhar.
- **Backup na mesma conta e com as mesmas credenciais.** Um invasor que apaga o banco apaga o backup no mesmo minuto. Cópias em outra conta, com exclusão bloqueada, resolvem isso; o capítulo 23 volta ao assunto com o AWS Backup.
- **Failover nunca ensaiado.** O dia do desastre é o pior dia para descobrir que a aplicação guarda o IP da primária num arquivo de configuração.

- **Alerta demais.** Uma equipe que recebe 200 alertas por semana deixa de ler todos, inclusive o que importava. É a fábula do menino que gritava lobo, versão plantão.

Resumo

- Backup lógico (`pg_dump`) é um retrato; físico com WAL permite point-in-time recovery. Regra 3-2-1 e restore testado.
- RPO: quanto dado perder. RTO: quanto tempo fora. Decisões de negócio.
- Réplicas servem para leitura e failover (cuidado com lag e split brain) e copiam os erros, então não substituem backup.
- HA cuida de falhas pequenas; DR, das grandes. Disponibilidades em série se multiplicam.
- Segredos ficam fora do código e da imagem.
- IaC é a migration da infraestrutura; drift é o inimigo.
- CI testa e constrói a cada mudança; CD leva a versão até produção.
- Rolling, blue/green e canary controlam o risco do deploy; migrations seguem expand/contract.
- Observabilidade: logs estruturados, métricas, traces e alertas sobre sintomas.
- Postmortems são blameless.

Ponte: hora de alugar a infraestrutura

A lista de tarefas que este capítulo deixou é comprida: um Postgres com réplica em outro prédio, arquivamento contínuo de WAL e failover sem split brain; backups em outra região; servidores recriados a partir de código; logs e métricas centralizados, com alertas que acordam a pessoa certa; um cofre para segredos; um destino para o pipeline. Montar tudo isso em servidores próprios exigiria uma equipe de infraestrutura maior que o Rangô inteiro, com um orçamento de café proporcional ao tamanho da equipe. E você ainda nem tem o computador com IP público ligado o tempo todo que o capítulo 1 prometeu.

A Parte I mostrou o que precisa existir. A Parte II responde onde e como. No próximo capítulo, o Rangô ganha uma conta na AWS e, antes de criar qualquer servidor, configura a única coisa que impede uma fatura de quatro dígitos no fim do mês: um alarme de cobrança.

Referências

- PostgreSQL, backup e restauração: https://www.postgresql.org/docs/current/backup.html
- PostgreSQL, arquivamento contínuo e point-in-time recovery: https://www.postgresql.org/docs/current/continuous-archiving.html
- PostgreSQL, `pg_dump`: https://www.postgresql.org/docs/current/app-pgdump.html
- PostgreSQL, alta disponibilidade e replicação: https://www.postgresql.org/docs/current/high-availability.html
- Imagem oficial do Postgres no Docker Hub (variáveis `_FILE`): https://hub.docker.com/_/postgres
- Docker, segredos no Compose: https://docs.docker.com/compose/how-tos/use-secrets/
- AWS, opções de recuperação de desastres na nuvem: https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html
- GitHub Actions, documentação: https://docs.github.com/en/actions
- FastAPI, testes com `TestClient`: https://fastapi.tiangolo.com/tutorial/testing/
- Nginx, módulo `upstream` (pesos e `max_fails`): https://nginx.org/en/docs/http/ngx_http_upstream_module.html
- Martin Fowler, Blue Green Deployment: https://martinfowler.com/bliki/BlueGreenDeployment.html
- Martin Fowler, Canary Release: https://martinfowler.com/bliki/CanaryRelease.html
- Google, *Site Reliability Engineering*, monitoramento de sistemas distribuídos (sinais de ouro): https://sre.google/sre-book/monitoring-distributed-systems/
- Google, *Site Reliability Engineering*, cultura de postmortem: https://sre.google/sre-book/postmortem-culture/
- OpenTelemetry, documentação: https://opentelemetry.io/docs/
- GitLab, postmortem da queda do banco de 31 de janeiro de 2017: https://about.gitlab.com/blog/2017/02/10/postmortem-of-database-outage-of-january-31/
- SEC, ordem administrativa sobre a Knight Capital (2013): https://www.sec.gov/litigation/admin/2013/34-70694.pdf
- Microsoft, sobre a queda causada pela atualização da CrowdStrike: https://blogs.microsoft.com/blog/2024/07/20/helping-our-customers-through-the-crowdstrike-outage/

PARTE II

AWS na prática

CAPÍTULO 7

Bem-vindo à AWS

Na Parte I, o Rangô saiu do `localhost` e aprendeu o que existe entre o notebook e o usuário: IPs, portas e NAT, DNS e TLS, servidores, containers e proxy reverso, balanceadores e cache, filas com idempotência e, no capítulo anterior, backups testados, RPO e RTO, réplicas, pipeline e observabilidade. Tudo isso rodou na sua máquina, dentro do Docker Compose, que já fazia o papel de data center sem nunca ter pedido aumento. A Parte II começa agora e responde à pergunta que ficou pendurada: onde essa infraestrutura vai existir de verdade, e quem vai mantê-la. A resposta deste livro é a AWS, e o primeiro passo é abrir a conta sem deixar uma porta aberta para a fatura.

Onde o Rangô vai morar

A lista que o capítulo 6 deixou na mesa continua lá. Um Postgres com réplica em outro prédio e arquivamento contínuo de WAL. Backups guardados longe dos servidores. Máquinas recriadas a partir de código. Logs e métricas centralizados, com alertas. Um cofre de segredos. Um destino para o pipeline. E, desde o capítulo 1, um computador com IP público ligado o tempo todo. Lida em voz alta, parece a lista de presentes de casamento de um engenheiro.

Você tem três caminhos.

O primeiro é comprar hardware e alugar espaço num data center. O investimento sai antes do primeiro pedido, e a capacidade é decidida com meses de antecedência. Ninguém compra um servidor às 21h14 de uma quinta-feira porque o Rangô apareceu na TV. Às 21h14, o setor de compras está, com toda a razão, em casa.

O segundo é alugar um servidor virtual num provedor de VPS. Sai barato e resolve o IP público, mas o Postgres replicado, os backups em outra cidade, o balanceador e o cofre de segredos continuam sendo trabalho seu, montado à mão. É infraestrutura artesanal, como pão de fermentação natural, e com o mesmo tempo de preparo.

O terceiro é a nuvem. Você aluga cada peça da lista como um serviço, por hora ou por uso, e cria tudo por uma API. O Postgres com réplica vira um formulário (com mais campos do que você gostaria, como todo formulário), e mais adiante algumas linhas de código.

Esse terceiro caminho tem um custo que não aparece na propaganda. Considere uma situação hipotética, mas comum. Uma desenvolvedora cria um cluster de banco de dados para testar numa sexta à tarde e esquece dele. O cluster fica ligado o mês inteiro, junto com um NAT Gateway que ela nem sabia ter criado. No dia 3 do mês seguinte, a fatura chega com quatro dígitos em dólar. Ninguém foi hackeado. A nuvem fez o que prometia: cobrou pelo que estava ligado. É a conta de luz de uma casa vazia com o ar-condicionado ligado o mês inteiro, só que em dólar.

A pergunta deste capítulo, então, tem duas metades. Onde o Rangô vai morar? E como garantir que você seja avisado antes de a conta virar um susto?

O que a nuvem vende

Tirando o folheto, computação em nuvem é isto: alguém com data centers enormes aluga pedaços deles para você, por uma API, cobrando pelo uso. Três características separam isso de alugar um servidor tradicional.

A primeira é o autosserviço. Você cria um servidor ou uma fila com uma chamada de API, em minutos, sem falar com ninguém: sem abrir chamado, sem reunião e sem esperar a aprovação de três diretores. O console, a CLI e as ferramentas de infraestrutura como código são clientes dessa mesma API: tudo que você clica no console vira uma requisição HTTP assinada para um *endpoint* da AWS. Cada clique é, no fundo, um `curl` bem vestido.

A segunda é a cobrança por uso: servidor por segundo ou hora, armazenamento por gigabyte por mês, tráfego por gigabyte que sai para a internet. Você paga pelo que deixou ligado.

A terceira é a elasticidade. Os 21 processos que o capítulo 4 calculou para o pico da TV sobem em minutos e são desligados quando o programa acaba, como as cadeiras extras de uma festa de aniversário, que aparecem quando chega gente e voltam para o depósito no fim.

Você vai ouvir as siglas IaaS (máquinas virtuais, discos e redes, como o Amazon EC2) e PaaS (um lugar para rodar código ou banco sem administrar o sistema operacional, como o Amazon RDS). A fronteira entre elas é borrada, e as siglas terminadas em "aaS" se multiplicaram tanto que hoje é difícil achar algo que não seja vendido como serviço. A pergunta prática é outra: neste serviço, o que fica comigo? É a pergunta que todo inquilino deveria fazer antes de assinar o contrato, e quase nenhum faz.

O mapa da Parte II

A AWS tem mais de duzentos serviços. O Rangô vai usar uns vinte. Aprender os duzentos antes de começar seria como decorar o cardápio inteiro de uma lanchonete de estrada antes de pedir um misto-quente. A tabela abaixo liga o que a Parte I mostrou que precisa existir ao serviço usado no livro e ao capítulo em que ele aparece.

O que precisa existir	Serviço AWS	Capítulo
Identidades e permissões	IAM, IAM Identity Center	8
Computador com IP público	Amazon EC2	9
Rede, subnets públicas e privadas, NAT	Amazon VPC	10
Fotos dos pratos e backups	Amazon S3	11
Postgres com réplica e PITR, Redis	Amazon RDS, Amazon ElastiCache	12
Balanceador e escala horizontal	Elastic Load Balancing, EC2 Auto Scaling	13
DNS, certificados, CDN	Amazon Route 53, AWS Certificate Manager, Amazon CloudFront	14
Infraestrutura como código	AWS CloudFormation, AWS CDK	15
Cofre de segredos	Parameter Store, AWS Secrets Manager, AWS KMS	16
Containers e orquestração	Amazon ECR, Amazon ECS, AWS Fargate	17
Pipeline de deploy	GitHub Actions com OIDC	18
Serverless	AWS Lambda, Amazon API Gateway	19
Banco chave-valor para dados que mudam a cada segundo	Amazon DynamoDB	20
Filas, pub/sub, eventos, fluxos com estado	Amazon SQS, Amazon SNS, Amazon EventBridge, AWS Step Functions	21
Logs, métricas, traces, alertas	Amazon CloudWatch, OpenTelemetry com AWS X-Ray	22
Auditoria, firewall, contas separadas	AWS CloudTrail, AWS WAF, Amazon GuardDuty, AWS Organizations	23
Custo sob controle	AWS Cost Explorer, AWS Budgets	24

Cada linha corresponde a uma ideia que você já entende. A Parte II mostra como a AWS implementa cada uma e onde estão as letras miúdas, que são numerosas o bastante para encher um livro inteiro. Por coincidência, este.

Quem cuida de quê

A primeira coisa que muda quando você aluga infraestrutura é a divisão de responsabilidades. A AWS chama isso de **modelo de responsabilidade compartilhada**, e a frase que ela usa para resumir é boa: a AWS cuida da segurança *da* nuvem; você cuida da segurança *na* nuvem. Uma preposição de diferença, e quase todo incidente mora do seu lado dela.

Segurança da nuvem é o prédio, os discos destruídos ao sair de uso, o hypervisor que isola a sua máquina virtual da do vizinho, a rede física. Você não mexe em nada disso, o que poupa várias tentações.

Segurança na nuvem é tudo o que você configura: quem acessa a conta, quais portas estão abertas, se o bucket das fotos está público, se o sistema operacional recebe atualizações, se a senha do banco está no repositório.

Uma analogia de desenvolvedor ajuda. Quando você usa o Django, o framework protege contra CSRF e escapa o HTML dos templates. Mas se você escrever uma view que devolve os pedidos de qualquer usuário sem conferir quem está logado, o framework não vai impedir. A fronteira existe, e o lado de lá é bem feito; o lado de cá é seu. O framework entrega a fechadura, mas não tranca a porta por você.

O detalhe que confunde iniciantes é que essa fronteira se move conforme o serviço:

Quem atualiza…	EC2 (cap. 9)	RDS (cap. 12)	Lambda (cap. 19)
Hardware e data center	AWS	AWS	AWS
Sistema operacional	Você	AWS	AWS
Versão do Postgres ou do runtime	Você	Você escolhe e agenda; AWS aplica	Você escolhe o runtime
Código da aplicação	Você	Você	Você
Quem acessa, portas abertas, dados expostos	Você	Você	Você

A última linha nunca muda de coluna. Ela é a parte do contrato que ninguém lê e todo mundo assina. Quase todos os vazamentos famosos em nuvem caíram ali: um bucket público, uma porta aberta, uma chave de acesso publicada. O capítulo 11 traz alguns casos de buckets expostos, e o capítulo 8 começa com uma chave de acesso no GitHub.

Regiões e zonas de disponibilidade

No capítulo 6, a réplica do Postgres precisava ficar "em outro prédio", e o backup, "em outra cidade". A AWS organiza a infraestrutura física exatamente nesses dois níveis. É raro a realidade combinar tão bem com uma metáfora.

Uma **Region** (região) é uma área geográfica com vários data centers, como Norte da Virgínia, Irlanda ou São Paulo. Cada região tem um código: `us-east-1` é o Norte da Virgínia, `sa-east-1` é São Paulo. Os códigos têm a poesia de uma placa de carro, mas pelo menos são fáceis de digitar. As regiões são independentes entre si. Um recurso criado em `us-east-1` existe só lá, e a maioria dos serviços não copia nada para outra região, a menos que você peça.

Uma **Availability Zone** (zona de disponibilidade, ou AZ) é um ou mais data centers dentro de uma região, com energia, refrigeração e rede próprias. As AZs ficam longe o bastante umas das outras para que um incêndio não derrube duas, e perto o bastante para que a latência entre elas seja de poucos milissegundos. Os nomes são a região mais uma letra: `us-east-1a`, `sa-east-1c`.

Em outubro de 2026, a página oficial da infraestrutura global listava 39 regiões e 124 AZs, com mais duas regiões anunciadas (Arábia Saudita e Chile). Esses números crescem todo ano, e é bem provável que a contagem já esteja velha quando você ler esta frase. É o tipo de desatualização que um autor aceita de bom grado. São Paulo existe desde 2011 e tem três AZs. A região é mais velha que muito framework JavaScript que você já adotou e abandonou.

Traduzindo para o vocabulário do capítulo 6:

- Alta disponibilidade, na AWS, quase sempre significa espalhar o sistema por **duas ou mais AZs da mesma região**. Banco primário numa AZ, réplica em outra; servidores da API divididos entre elas; balanceador nas duas.
- Recuperação de desastres contra a perda de uma região inteira significa **copiar dados para outra região**. Isso é mais caro e mais raro, e o Rangô vai tratar do assunto só para os backups.

Existem ainda as **Local Zones** (extensões de uma região perto de grandes cidades, que você pode ignorar por enquanto) e os pontos de presença do CloudFront, a CDN do capítulo 2, que entram no capítulo 14.

> **Armadilha:** o nome `us-east-1a` na sua conta pode apontar para um data center diferente do `us-east-1a` na conta de outra pessoa. A AWS embaralha os nomes por conta para distribuir a carga. Se todo mundo escolhesse a "a" por ser a primeira da lista, ela viraria o caixa rápido do supermercado. Quando precisar comparar AZs entre contas, use o **AZ ID** (como `use1-az1`), que é fixo.

Qual região escolher

Para um app brasileiro, a intuição diz São Paulo: latência menor, e alguns clientes exigem dados no Brasil. Em contrapartida, `sa-east-1` costuma ter preços mais altos que as regiões dos Estados Unidos para os mesmos serviços, e alguns serviços e tipos de máquina chegam lá mais tarde, como os filmes estrangeiros nos cinemas brasileiros dos anos 1990.

Os exemplos deste livro usam `us-east-1` por três motivos práticos. É a região padrão de quase toda a documentação, então os exemplos que você encontrar por aí vão bater com os daqui. Alguns recursos globais vivem só nela: as métricas de cobrança, que aparecem daqui a pouco, e os certificados do ACM usados pelo CloudFront, que aparecem no capítulo 14. E ela costuma ser a primeira a receber serviços novos. Entre profissionais de nuvem, ela tem fama de ser a região onde tudo acontece primeiro, inclusive os problemas.

Quando latência ou custo pesarem para o leitor brasileiro, o livro avisa. Na sua empresa, a decisão depende de onde estão os usuários, de exigências legais sobre os dados e da diferença de preço.

> **Na vida real:** `us-east-1` é a região mais antiga e mais usada, e seus incidentes têm alcance enorme. Entre 19 e 20 de outubro de 2025, um defeito latente no sistema que gerencia o DNS do Amazon DynamoDB deixou o endpoint do serviço sem resolução em `us-east-1`. Outros serviços da AWS caíram em cascata, e a criação de instâncias EC2 falhou por horas. Usar `us-east-1` é razoável; supor que ela nunca cai, não.

Na prática: a conta do Rangô

Hora de abrir a conta. O processo leva uns 40 minutos e pede um cartão de crédito internacional e o celular. O café é opcional, mas recomendado.

Antes de clicar em "Criar conta"

Três decisões merecem um minuto de reflexão antes do cadastro.

O e-mail. O e-mail da conta vira o login do usuário com mais permissões que existe nela. O ideal é não usar o seu e-mail pessoal nem o seu e-mail corporativo individual, e sim uma lista ou um alias que mais de uma pessoa da empresa receba, como `aws-root@rango.com.br`. Se você sair do Rangô, ou se a sua caixa de entrada for comprometida, a conta não vai junto. Se você está só estudando, um alias do seu e-mail pessoal (`voce+aws@gmail.com`) já resolve o problema de misturar as mensagens. O sinal de mais no endereço é um truque do Gmail que pouca gente conhece e todo mundo adora quando descobre.

O cartão. A AWS cobra em dólar, o que significa compra internacional, com câmbio e IOF, aquela sigla que lembra você, a cada compra, de onde mora. Um cartão virtual com limite baixo protege enquanto você aprende.

O nome da conta. Algo como `rango-estudos`. No capítulo 23, o Rangô vai ter várias contas, e nomes claros ajudam desde já. Nomes como `teste2-final-agora-vai` costumam envelhecer mal.

Criando a conta

O cadastro começa em `https://aws.amazon.com/`, no botão de criar conta, e pede, nesta ordem aproximada (a tela muda de tempos em tempos, como app de banco depois de atualização): e-mail e nome da conta, um código de verificação enviado ao e-mail, a senha do root user, dados de contato, dados de pagamento, verificação por telefone e a escolha do plano de suporte. O plano escolhido aqui é o **Basic**, que é gratuito. Os planos pagos de suporte servem para empresas com produção crítica e não fazem falta agora.

No meio desse processo aparece uma escolha nova, que vale uma seção própria.

Free plan ou Paid plan

O *free tier* da AWS mudou em julho de 2025. Para contas criadas a partir de 15 de julho de 2025, existem dois planos:

	Free plan	Paid plan
Créditos	Até US$ 200 (US$ 100 no cadastro, até US$ 100 por atividades no painel "Explore AWS")	Os mesmos créditos
Serviços	Um subconjunto	Todos
Cobrança	Nenhuma enquanto estiver nesse plano	Uso além dos créditos é cobrado no preço normal
Fim	6 meses após a criação ou quando os créditos acabarem, o que vier primeiro	Créditos restantes têm validade; o uso continua
Volta	Pode migrar para o Paid a qualquer momento	Não volta para o Free

O Free plan é uma caixa de areia: dá para construir castelos, mas não dá para morar neles. Nada é cobrado, e em troca nem todos os serviços estão liberados: ficam de fora os que consumiriam os créditos rápido demais e os que envolvem compra de hardware. Quando o plano acaba, a conta é suspensa. A AWS guarda os seus dados por 90 dias; se você migrar para o Paid plan nesse prazo, recupera o acesso. Depois disso, a conta e tudo dentro dela são apagados.

O Paid plan é a conta normal da AWS, com os créditos abatendo a fatura até acabarem ou expirarem. A própria página de perguntas frequentes da AWS dá prazos diferentes para a validade dos créditos em lugares diferentes (uma forma criativa de garantir que alguém leia a página até o fim), então a regra que vale para cada conta é a que aparece no console de Billing and Cost Management, na área de créditos.

Qual escolher para acompanhar este livro? Para estudar com calma, o Free plan é o começo natural. Os capítulos 8 e 9 cabem bem nele. A partir do 10, o NAT Gateway, os interface endpoints e o RDS consomem os créditos depressa, e alguns serviços que o Rangô usa podem estar fora do subconjunto liberado, e aí a migração para o Paid plan é um clique, sem volta. Para a conta de uma empresa, a escolha certa é o Paid plan desde o início.

> **Armadilha:** ouvir falar em "750 horas grátis por mês durante 12 meses" e confiar nisso. Esse era o modelo antigo, e muitos tutoriais ainda o descrevem. As regras mudaram em 2025. Para saber o que vale para a sua conta, a fonte é o console de Billing and Cost Management, e não um vídeo de 2023.

Com o cadastro concluído, chega o primeiro acesso ao console. No canto superior direito aparecem o nome da conta, um seletor de região e um número de 12 dígitos, o **account ID**. Nos exemplos do livro ele aparece como `123456789012`, o número fictício que a

própria documentação da AWS usa. É o número de conta mais famoso do mundo e, felizmente, não pertence a ninguém.

O root user

O login usado no cadastro é o **root user**. Ele é a identidade criada junto com a conta, e tem acesso total e irrestrito a tudo. Pense no `postgres` do seu banco local, o superusuário que ignora qualquer `GRANT`, multiplicado pelo cartão de crédito: o root pode criar qualquer recurso, apagar qualquer recurso, trocar o e-mail da conta, mudar o plano de suporte e fechar a conta. Num dia ruim, ele encerra o Rangô inteiro sem pedir licença a ninguém.

No capítulo 8 você vai entender por que as permissões normais da AWS não conseguem limitar o root e como criar identidades para o dia a dia. Por enquanto, as regras são três:

1. Proteger o root com MFA, agora.
2. Nunca criar uma chave de acesso para o root.
3. Usar o root só para as poucas tarefas que exigem o root. Hoje, isso inclui o que vem a seguir.

MFA no root

MFA (autenticação multifator) exige algo além da senha para entrar: um código de um aplicativo ou a confirmação de um dispositivo. Desde junho de 2025, a AWS exige MFA no root user de todos os tipos de conta. Numa conta nova, o console provavelmente conduz essa configuração logo no primeiro acesso. Se não conduzir, o caminho é este:

1. No nome da conta, no canto superior direito, fica o item **Security credentials**.
2. Na seção **Multi-factor authentication (MFA)**, o botão **Assign MFA device** inicia o cadastro.
3. O dispositivo recebe um nome, como `root-passkey-notebook`.
4. Depois vêm a escolha do tipo e as instruções da tela.

As opções são *passkey* ou chave de segurança (uma YubiKey, ou a passkey do gerenciador de senhas), aplicativo autenticador com códigos de seis dígitos e token físico. A preferência deve ir para passkey ou chave de segurança. Elas seguem o padrão FIDO, que amarra a autenticação ao domínio real da AWS, então uma página falsa que imita o console não consegue reaproveitar a resposta. A passkey confere o endereço com a desconfiança de um cachorro diante do carteiro. Um código de seis dígitos pode ser digitado numa página falsa e repassado ao site real em segundos. SMS deixou de ser opção, e poucos vão sentir saudade.

O certo é cadastrar **pelo menos dois** dispositivos. O root e cada usuário IAM aceitam até oito, e qualquer um deles basta para entrar. Se o único dispositivo for o celular, e o celular cair no rio (ou, mais provável, no balde de gelo do churrasco), recuperar o acesso ao root vira um processo com o suporte da AWS que pode levar dias. Uma chave física guardada no cofre da empresa, ou uma segunda passkey em outro gerenciador, evita isso.

O MFA do root só pode ser configurado pelo console, nunca pela CLI ou pela API. Se alguém já tivesse credenciais de API do root, o MFA chegaria tarde demais, como trancar a porta com o ladrão já sentado no sofá.

Na mesma página, a seção **Access keys** de uma conta nova está vazia, e deve continuar assim. É a única página da AWS em que vazio significa sucesso.

Faltam duas configurações rápidas na página **Account** (no menu do nome da conta). Os **Alternate contacts** de cobrança, operações e segurança são preenchidos ali, com os endereços para onde a AWS escreve quando detecta algo estranho. É o equivalente a deixar o telefone de um vizinho com o síndico. E a opção **IAM user and role access to Billing information** é ativada; sem isso, só o root vê a fatura, e o capítulo 8 vai criar outras identidades.

O alarme de cobrança, antes de tudo

Esta é a promessa do capítulo anterior. Antes de qualquer servidor, a conta ganha o aviso que chega quando a fatura passa de um valor escolhido.

A AWS publica uma métrica chamada `EstimatedCharges` no Amazon CloudWatch, o serviço de métricas e alarmes que o capítulo 22 vai detalhar. Ela representa o total estimado da fatura do mês corrente, somando todas as regiões. Um **alarme do CloudWatch** observa uma métrica e muda de estado quando ela cruza um limite; quando isso acontece, ele publica uma mensagem num tópico do Amazon SNS, o serviço de pub/sub da AWS (o mesmo conceito do capítulo 5), que entrega a mensagem por e-mail.

São três passos.

Passo 1: habilitar as métricas de cobrança. Elas vêm desligadas.

1. O ponto de partida é o console **Billing and Cost Management**.
2. No menu lateral, fica **Billing Preferences**.
3. Em **Alert preferences**, o botão **Edit** libera a edição.
4. A opção **Receive CloudWatch Billing Alerts** é marcada, e **Save preferences** grava a mudança.

Essa coleta não pode ser desligada depois, e os primeiros dados levam uns 15 minutos para aparecer. Só o root ou uma identidade com permissão para ver a cobrança consegue fazer isso.

Passo 2: trocar a região para Norte da Virgínia. No seletor de região, no canto superior direito, a escolha é **US East (N. Virginia) us-east-1**. As métricas de cobrança existem só nessa região, mesmo que todos os seus recursos estejam em São Paulo. A cobrança, pelo visto, prefere morar na Virgínia. Quem tenta criar o alarme em `sa-east-1` descobre que a métrica simplesmente não aparece na lista.

Passo 3: criar o alarme.

1. No console do **CloudWatch**, o caminho é **Alarms**, **All alarms**, **Create alarm**.
2. O botão **Select metric** leva a **Billing**, **Total Estimated Charge**. A linha marcada é a da moeda **USD**, e **Select metric** confirma.
3. A estatística fica como **Maximum** e o período como **6 hours**.
4. Em **Conditions**, as opções são **Static**, **Greater** e o limite. Para quem está estudando, `10` (dólares) é um bom começo. Para a conta de uma empresa, o ideal é um valor um pouco acima do gasto mensal esperado.
5. Em **Additional configuration**, **Datapoints to alarm** fica em `1 out of 1` e o tratamento de dados ausentes em **Treat missing data as missing**.
6. Em **Notification**, a escolha é **In alarm**, **Create new topic**. O tópico recebe o nome `alertas-cobranca` e o e-mail que deve receber o aviso (o alias da empresa, de preferência). **Create topic** cria o tópico.
7. O alarme recebe o nome `cobranca-acima-de-10-usd`, e a criação é concluída.

Em seguida, a caixa de entrada recebe um e-mail com o assunto "AWS Notification - Subscription Confirmation", e é preciso clicar em **Confirm subscription**. Sem essa confirmação, o SNS não entrega nada, e o alarme dispara para o vazio. Esse é o erro mais comum desta configuração inteira. Um alarme sem assinatura confirmada é um detector de fumaça sem pilha: bonito no teto, inútil no incêndio.

Na lista de alarmes, o novo alarme deve aparecer com o estado **Insufficient data** nas primeiras horas (o equivalente técnico de "sem comentários"), até a métrica ter dados. Depois, **OK**.

Esse alarme compara o valor **atual** da fatura estimada com o limite, sem projeção: se um recurso caro começou a rodar hoje cedo, ele só dispara quando a soma passar de US$ 10. A métrica é atualizada algumas vezes por dia, e o alarme só avisa; nada é desligado. Ele é o amigo que diz "você vai se arrepender disso" e depois fica só olhando. Se a fatura já estiver acima do limite na criação, ele vai direto para **ALARM**.

Um orçamento como segunda rede

O AWS Budgets complementa o alarme com o que falta a ele: previsão. O alarme olha pelo retrovisor; o budget olha pelo para-brisa. Um *budget* compara o gasto real e o gasto **projetado** para o mês com um valor que você define. Criar budgets que só monitoram e notificam é gratuito.

1. No console **Billing and Cost Management**, o caminho é **Budgets**, **Create budget**.
2. A opção é **Use a template**, com o modelo **Monthly cost budget**.
3. O budget recebe o nome `rango-mensal`, um valor (por exemplo, US$ 20) e o e-mail.
4. Por fim, o budget é criado.

O modelo mensal já vem com alertas para quando o gasto real passar de 85% e 100% do valor e para quando a previsão passar de 100%. Se a previsão do meio do mês disser que você vai fechar em US$ 60, o aviso chega no dia 15, e não no dia 3 do mês seguinte. É a diferença entre a previsão do tempo e a notícia da enchente.

Para quem está estudando, vale criar também um budget com o modelo **Zero spend budget**, que avisa assim que o primeiro centavo for cobrado.

No Paid plan, o console oferece ainda um limite mensal de gasto por projeto, a partir de US$ 20, que pausa o projeto ao ser atingido. O que conta como "projeto" ainda é pouco documentado; a explicação no console merece leitura antes de qualquer confiança nele.

Console, CLI e CloudShell

O console é ótimo para aprender e para olhar. Você vê os recursos, os gráficos e as opções disponíveis, e o formulário de criação ensina quais parâmetros existem. Para fazer a mesma coisa duas vezes, ele é péssimo. Quantos cliques o alarme de cobrança exigiu? Agora imagine repetir isso em três contas, ou lembrar daqui a seis meses qual opção foi marcada no passo 5. Configurar infraestrutura clicando é como fazer contabilidade numa calculadora de bolso: funciona até a segunda página.

A AWS CLI resolve isso. Ela é um programa de linha de comando que transforma cada operação da API num comando. O que você clicou no console vira uma linha que pode ir para um script, para um README ou para o pipeline. Texto, ao contrário da memória, cabe num commit e aparece no `git diff`. No capítulo 15, o CDK vai dar o passo seguinte, descrever a infraestrutura inteira em Python, mas a CLI continua sendo a ferramenta do dia a dia para consultar, testar e consertar.

Existe um jeito de usar a CLI sem instalar nada e sem configurar credenciais: o **AWS CloudShell**. É um terminal Linux no navegador (que, depois de planilhas, editores e videochamadas, agora roda também um shell), aberto a partir do console, com a CLI já instalada e autenticada com a mesma identidade que está logada no console. O uso do CloudShell não tem cobrança extra, e cada região guarda um diretório pessoal persistente de 1 GB.

Primeiros comandos no CloudShell

Com a região ainda em `us-east-1`, o ícone do CloudShell (um quadradinho com `>_`, na barra superior) abre o terminal depois de alguns segundos. O primeiro comando responde à pergunta "quem sou eu?", que a filosofia debate há milênios e a AWS resolve em JSON, em menos de um segundo:

```
aws sts get-caller-identity
```

```
{
    "UserId": "123456789012",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:root"
}
```

Repare no `Arn`, o identificador único de um recurso na AWS. Ele termina em `root`. A sessão está usando o superusuário, e esse incômodo é proposital: o capítulo 8 acaba com ele.

As zonas de disponibilidade de São Paulo aparecem assim:

```
aws ec2 describe-availability-zones \
  --region sa-east-1 \
  --query 'AvailabilityZones[].[ZoneName,ZoneId,State]' \
  --output table
```

```
-----------------------------------------------
|          DescribeAvailabilityZones          |
+-------------+-------------+-----------------+
|  sa-east-1a |  sae1-az1   |  available      |
|  sa-east-1b |  sae1-az2   |  available      |
|  sa-east-1c |  sae1-az3   |  available      |
+-------------+-------------+-----------------+
```

Na sua conta, a associação entre nome e ID pode ser outra. O `--query` usa JMESPath, uma linguagem de consulta para JSON que funciona como uma list comprehension aplicada à resposta: `AvailabilityZones[].[ZoneName,ZoneId,State]` equivale a `[[z["ZoneName"], z["ZoneId"], z["State"]] for z in resposta["AvailabilityZones"]]`.

Por fim, o alarme criado no console:

```
aws cloudwatch describe-alarms \
  --alarm-names cobranca-acima-de-10-usd \
  --region us-east-1 \
  --query 'MetricAlarms[].[AlarmName,StateValue,Threshold]' \
  --output text
```

```
cobranca-acima-de-10-usd    OK  10.0
```

Se aparecer `INSUFFICIENT_DATA`, a métrica ainda não chegou; ela aparece em algumas horas.

Para comparar com os sete passos do console, aqui está o mesmo alarme criado só pela CLI. Ele não precisa ser executado, porque o alarme já existe; o objetivo é ver como cada campo do formulário vira um parâmetro:

```
aws sns create-topic --name alertas-cobranca --region us-east-1

aws sns subscribe \
  --topic-arn arn:aws:sns:us-east-1:123456789012:alertas-cobranca \
  --protocol email \
  --notification-endpoint aws-root@rango.com.br \
  --region us-east-1

aws cloudwatch put-metric-alarm \
  --alarm-name cobranca-acima-de-10-usd \
  --namespace AWS/Billing \
  --metric-name EstimatedCharges \
  --dimensions Name=Currency,Value=USD \
  --statistic Maximum \
  --period 21600 \
  --evaluation-periods 1 \
  --threshold 10 \
  --comparison-operator GreaterThanThreshold \
  --treat-missing-data missing \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:alertas-cobranca \
  --region us-east-1
```

O período de "6 hours" do console vira `--period 21600`, em segundos. O namespace que o console mostrava como "Billing" se chama `AWS/Billing` na API. Para recriar o alarme algum dia, bastaria trocar o account ID e o e-mail pelos da conta. Esse bloco é a primeira peça de infraestrutura do Rangô escrita como texto, e ele pode ir para o repositório.

A CLI no seu computador

O trabalho diário acontece no seu terminal, junto do código do Rangô. A instalação usa a AWS CLI versão 2. No Linux (x86_64):

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
```

```
sudo ./aws/install
```

No macOS:

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

No Windows, o instalador é o `https://awscli.amazonaws.com/AWSCLIV2.msi`. Em computadores ARM com Linux, o pacote é o `awscli-exe-linux-aarch64.zip`. A conferência:

```
aws --version
```

```
aws-cli/2.x.y Python/3.x.y Linux/6.x exe/x86_64
```

O número exato da versão vai variar; o que importa é o `aws-cli/2`. A CLI v2 traz o próprio Python embutido e não interfere no Python do Rangô. Quem já brigou com três versões do Python na mesma máquina sabe o tamanho desse presente.

> **Armadilha:** `pip install awscli` instala a CLI **versão 1**, antiga, que pode conflitar com as dependências do projeto. Se `aws --version` mostrar `aws-cli/1.`, rode `pip uninstall awscli` e use o instalador oficial.

A mesma pergunta do CloudShell, agora no terminal local:

```
aws sts get-caller-identity
```

```
Unable to locate credentials. You can configure credentials by running "aws configure".
```

O CloudShell sabia quem você era porque herdava a sessão do console. O seu terminal não sabe, e admite isso com mais elegância do que muita gente em reunião. E a mensagem de erro sugere um caminho que parece óbvio. Guarde essa sugestão; ela é o assunto da ponte.

Armadilhas e casos reais

O recurso esquecido em outra região. O console mostra os recursos de uma região de cada vez. Você cria um servidor em `sa-east-1` para testar latência, troca o seletor para `us-east-1` e esquece que ele existe. Ele não aparece na tela, mas aparece na fatura. O alarme de cobrança soma todas as regiões, e é exatamente por isso que ele vem antes de tudo. Para encontrar o culpado, a página **Bills** do Billing and Cost Management detalha o gasto por serviço e por região.

O IP público que custa dinheiro. Desde fevereiro de 2024, todo endereço IPv4 público custa US$ 0,005 por hora, pouco mais de US$ 3,60 por mês (referência de outubro de 2026 em `us-east-1`; confira a página oficial), esteja ele anexado a um servidor ou parado. O IPv4 virou imóvel de luxo: escasso, disputado e com taxa de condomínio. Load balancers e NAT Gateways também usam IPs públicos cobrados.

A conta do Free plan que some. Seis meses depois da criação, ou quando os créditos acabam, a conta do Free plan é suspensa. Os avisos chegam por e-mail. Se o e-mail da conta for uma caixa que ninguém lê, como aquele `contato@` que só recebe promoção, os 90 dias passam, e a conta e os dados somem.

A métrica que não aparece. Se **Billing** não aparece na lista de métricas, a região do console provavelmente está errada ou a coleta foi habilitada há menos de 15 minutos. Contas da AWS Partner Network não publicam essas métricas; nelas, use o Budgets.

O e-mail da pessoa que saiu. Uma conta criada com o e-mail pessoal do primeiro desenvolvedor da empresa vira refém quando essa pessoa vai embora. Recuperar o root sem acesso ao e-mail exige provar a posse da conta ao suporte da AWS.

Resumo

- Nuvem é infraestrutura alugada por API e cobrada pelo que fica ligado.
- A AWS cuida da segurança *da* nuvem; você, da segurança *na* nuvem. Acesso, portas e dados expostos são sempre seus.
- Alta disponibilidade usa várias AZs; desastre regional pede outra região. O livro usa `us-east-1`.
- Contas novas escolhem entre Free plan (sem cobrança, limitado, expira) e Paid plan, com até US$ 200 em créditos.
- O root user tem poder total. MFA com passkey ou chave de segurança, pelo menos dois dispositivos, nenhuma chave de acesso.
- O alarme de cobrança usa a métrica `EstimatedCharges`, existe só em `us-east-1`, avisa sobre o valor atual e depende de uma assinatura SNS confirmada. O AWS Budgets acrescenta a previsão.
- Console para aprender e olhar; CLI para repetir; CloudShell para usar a CLI sem configurar nada.

Ponte: a sugestão perigosa

O Rangô agora tem uma casa: uma conta na AWS com o root protegido, um alarme de cobrança vigiando a fatura e a CLI instalada no seu notebook. Só que a CLI não sabe quem você é. A mensagem de erro sugeriu `aws configure`, e esse comando pede um *Access Key ID* e um *Secret Access Key*. O caminho mais curto para conseguir esse par é voltar à página **Security credentials** do root e clicar em **Create access key**.

Muita gente faz isso. Muita gente também cola a senha do Wi-Fi no próprio roteador; popularidade nunca foi medida de segurança. Copia as duas strings para o `~/.aws/credentials`, depois para um `.env`, depois para um script que vai parar num commit. E há robôs varrendo cada push público do GitHub atrás desse formato de chave. Eles são mais rápidos que qualquer revisor de código e bem menos bem-intencionados. O próximo capítulo mostra o que acontece com uma conta cuja chave vazou, abre o IAM por dentro e configura a CLI do jeito certo, com login por SSO e credenciais temporárias, sem nenhuma chave permanente no disco.

Referências

- AWS Free Tier, perguntas frequentes: https://aws.amazon.com/free/faqs/
- Anúncio do novo Free Tier (julho de 2025): https://aws.amazon.com/about-aws/whats-new/2025/07/aws-free-tier-credits-month-free-plan/
- Criar um alarme de cobrança no CloudWatch: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/monitor_estimated_charges_with_cloudwatch.html
- Preços do AWS Budgets: https://aws.amazon.com/aws-cost-management/aws-budgets/pricing
- MFA obrigatório para root em todos os tipos de conta (junho de 2025): https://aws.amazon.com/about-aws/whats-new/2025/06/aws-iam-mfa-root-users-across-all-account-types
- MFA no IAM: https://docs.aws.amazon.com/IAM/latest/UserGuide/Using_ManagingMFA.html
- Regiões e zonas de disponibilidade: https://aws.amazon.com/about-aws/global-infrastructure/regions_az/
- Cobrança de IPv4 público: https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights
- Resumo do incidente do DynamoDB em `us-east-1` (outubro de 2025): https://aws.amazon.com/message/101925/

- Modelo de responsabilidade compartilhada: https://aws.amazon.com/compliance/shared-responsibility-model/
- Instalar a AWS CLI v2: https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
- AWS CloudShell: https://docs.aws.amazon.com/cloudshell/latest/userguide/welcome.html

CAPÍTULO 8

IAM: quem pode fazer o quê

No capítulo anterior, o Rangô ganhou uma conta na AWS: root protegido com MFA em dois dispositivos, alarme de cobrança acima de 10 dólares, um orçamento mensal e a AWS CLI instalada no seu notebook. A CLI, porém, terminou respondendo `Unable to locate credentials`. Ela ainda não sabe quem você é, uma crise de identidade que muita gente só enfrenta depois dos quarenta. O atalho óbvio (criar uma chave de acesso no root e colar no `~/.aws/credentials`) foi justamente o que deixamos de fora. Agora vamos ver por quê, e qual é o caminho certo.

Dez segundos no GitHub

Em dezembro de 2025, pesquisadores da Unit 42, o time de inteligência de ameaças da Palo Alto Networks, fizeram um teste controlado. Criaram uma chave de acesso de um usuário IAM e a enviaram para um repositório público no GitHub às 18:50:05 (UTC), de propósito, o que já os coloca à frente de muita gente que faz a mesma coisa sem querer. Às 18:50:15, dez segundos depois (menos tempo do que você leva para perceber que fez o commit no branch errado), o CloudTrail (o registro de auditoria da AWS, que veremos no capítulo 23) já mostrava uma ação sobre aquele usuário: a AWS tinha anexado a ele uma política chamada `AWSCompromisedKeyQuarantineV3`. Às 18:50:17 apareceu um aviso no painel do AWS Health, às 18:50:26 chegou um e-mail e às 18:50:59 a conta tinha um caso aberto no suporte.

Dez segundos é rápido. Só que a AWS tem companhia nessa varredura. Robôs de atacantes leem o fluxo de commits públicos o tempo todo, atrás de strings no formato de uma chave da AWS. São os únicos leitores no mundo que leem com atenção todos os seus commits, inclusive os chamados "fix", "fix 2" e "agora vai". Em outubro de 2023, a mesma Unit 42 descreveu uma campanha que batizou de EleKtra-Leak: entre o fim de agosto e o início de outubro daquele ano, atacantes usaram chaves expostas no GitHub para criar 474 instâncias EC2 dedicadas a minerar Monero, uma criptomoeda. É mais servidor do que muita startup com investimento de verdade consegue justificar numa reunião de diretoria. O tempo entre a chave aparecer num repositório e ser usada chegava a cinco minutos.

A empresa japonesa Classmethod publicou um relato parecido, do ponto de vista de quem sofreu. Uma chave com a política `PowerUserAccess` foi parar num repositório público. Em cerca de dez minutos, já havia instâncias minerando Monero na conta. O GuardDuty, serviço de detecção de ameaças da AWS, deu o alerta, e a resposta envolveu apagar a chave, revisar o CloudTrail em todas as regiões (o minerador não se limita à região onde você trabalha; é o único turista que visita todas as regiões da AWS sem reclamar do fuso horário), apagar os recursos criados e trocar todas as outras credenciais, o equivalente digital a trocar todas as fechaduras da casa porque uma chave sumiu.

Agora repare num detalhe do teste da Unit 42. A política de quarentena não desativa a chave. Ela só acrescenta uma negação explícita para uma lista de ações perigosas, como lançar instâncias EC2, criar usuários e chaves, anexar políticas e apagar objetos no S3. Qualquer ação fora dessa lista continua funcionando com a chave vazada. É como trancar a porta da frente e deixar a janela da cozinha aberta, com um bilhete educado pedindo para ninguém entrar. Se a chave vazada fosse a do root, nem essa rede de proteção existiria, porque nenhuma política consegue limitar o root.

Esse é o problema do capítulo. Uma credencial de longa duração é uma senha que não expira, que cabe numa linha de texto e que pode ser copiada para qualquer lugar. E cada lugar para onde ela é copiada (um `.env`, um script, um notebook do Jupyter, uma mensagem no Slack com o texto "só para testar, depois eu apago", enviada em 2021) é um lugar de onde ela pode vazar. Para resolver isso, precisamos entender como a AWS decide quem pode fazer o quê, e trocar a chave permanente por algo que expira sozinho.

Toda chamada é uma pergunta

O capítulo 7 mostrou que o console, a CLI e o CDK são todos clientes da mesma API. Cada clique no console e cada comando `aws` vira uma requisição HTTP assinada para um endpoint da AWS. Antes de executar qualquer coisa, a AWS responde a duas perguntas sobre essa requisição.

A primeira é *quem está pedindo*. Isso é **autenticação**, e a AWS a resolve verificando a assinatura da requisição, feita com uma chave secreta que só o solicitante deveria ter. A segunda é *se esse alguém pode fazer isso*. Isso é **autorização**, e é aqui que entra o **AWS Identity and Access Management**, o IAM. As duas palavras são tão parecidas que metade dos devs troca uma pela outra em code review, e a outra metade finge que não viu.

Você já escreveu autorização muitas vezes. Num backend Python, é o decorator que protege a view:

```python
@requer_permissao("pedidos:cancelar")
def cancelar_pedido(pedido_id: int, usuario: Usuario):
    ...
```

O IAM faz o mesmo papel para a AWS inteira, com a diferença de que ninguém consegue comentar o decorator "só por enquanto" para fazer a demo funcionar. Toda requisição carrega três informações que ele avalia:

- **Principal**: quem está pedindo. Um usuário, uma role, um serviço da AWS.
- **Action**: o que está sendo pedido, no formato `servico:Operacao`, como `s3:GetObject` ou `ec2:RunInstances`.
- **Resource**: sobre o que, identificado por um ARN.

O **ARN** (*Amazon Resource Name*) é o identificador único de qualquer coisa na AWS. O formato geral é `arn:aws:servico:regiao:conta:recurso`. Alguns exemplos que vão aparecer no livro:

```
arn:aws:iam::123456789012:role/rango-ec2
arn:aws:s3:::rango-fotos-pratos/cantina-da-nona/lasanha.jpg
arn:aws:ec2:us-east-1:123456789012:instance/i-0abc123def4567890
```

Os campos vazios (os `::`) são intencionais, por mais que pareçam um emoticon que perdeu o rosto. O IAM é global, então seus ARNs não têm região. Os nomes de bucket do S3 são únicos no mundo inteiro, então o ARN de um objeto não precisa nem de região nem de conta.

O IAM em si não tem custo. Você paga pelos recursos que as identidades criam, nunca pelas identidades. É o único serviço do livro que você pode abrir sem consultar antes o Cost Explorer com o coração apertado.

O JSON por dentro

As regras do IAM são escritas em **policies** (políticas), documentos JSON que listam o que é permitido ou negado. Esta é uma policy que o Rangô vai precisar no capítulo 11, quando a API passar a guardar as fotos dos pratos no S3:

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LerEGravarFotos",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::rango-fotos-pratos/*"
    },
    {
```

```
      "Sid": "ListarFotos",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::rango-fotos-pratos"
    }
  ]
}
```

Vamos linha por linha. O `Version` parece uma data de criação, mas é a versão da linguagem de políticas, e o valor certo é sempre `2012-10-17`. É uma data de 2012 que você vai digitar até se aposentar, como o aniversário de um parente distante de quem, por algum motivo, nunca consegue esquecer. Uma versão mais antiga existe e não suporta recursos como variáveis de política, então nunca mude isso.

O `Statement` é uma lista de regras. Cada uma tem um `Sid` opcional (um rótulo para você se achar), um `Effect` que vale `Allow` ou `Deny`, uma ou mais `Action` e um ou mais `Resource`.

Repare que são duas regras, e não uma. `GetObject` e `PutObject` agem sobre objetos, então o recurso é `rango-fotos-pratos/*`, tudo que está dentro do bucket. Já `ListBucket` age sobre o bucket em si, cujo ARN não tem a barra nem o asterisco. Juntar as três ações num único statement com `rango-fotos-pratos/*` é um erro clássico: o upload funciona, a listagem dá `AccessDenied`, e você perde uma tarde convencido de que o S3 tem algo pessoal contra você.

Uma policy também pode ter um `Condition`, que restringe quando a regra vale. Esta aqui nega a criação de qualquer coisa no EC2 fora das duas regiões que o Rangô usa:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SoRegioesDoRango",
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["us-east-1", "sa-east-1"]
        }
      }
    }
  ]
}
```

Lembra do caso da Classmethod, em que foi preciso varrer todas as regiões atrás de mineradores? Com essa regra, quem tiver essa identidade não consegue subir uma instância em `ap-southeast-1` nem em nenhuma outra região além das duas. Singapura é um destino lindo, mas perde a graça quando quem viaja é uma instância que você não criou.

Como a AWS decide

Com várias policies anexadas à mesma identidade, a AWS segue uma ordem simples de lembrar:

1. Tudo começa negado. Se nenhuma policy fala da ação, a resposta é não. Isso se chama **negação implícita** (*implicit deny*).
2. Um `Allow` que case com a requisição muda a resposta para sim.
3. Um `Deny` explícito em qualquer policy vence qualquer `Allow`, em qualquer lugar. Não existe forma de um `Allow` derrubar um `Deny`. É a regra de qualquer casa com crianças: se um dos pais disse não, o sim do outro não vale nada.

A terceira regra é o que torna a policy de regiões útil. Você pode dar `AdministratorAccess` a alguém e, ao lado, anexar o `Deny` de regiões. O resultado é um administrador que não cria nada no EC2 em Singapura, um rei com o reino inteiro, menos uma ilha. A quarentena da AWS funciona do mesmo jeito: é um `Deny` explícito colado ao lado do que já existia.

Existem mais tipos de policy além das que ficam na identidade. As **resource-based policies** ficam no recurso, como a bucket policy do S3 que veremos no capítulo 11. Dentro da mesma conta, as policies da identidade e as do recurso somam: basta um `Allow` em qualquer uma delas. Entre contas diferentes, os dois lados precisam permitir. Há ainda tipos que só servem para limitar, nunca para conceder, como as *permissions boundaries* e as SCPs do AWS Organizations (capítulo 23). Para esses, vale a interseção: a ação precisa estar liberada em todos eles.

> **Dica:** Na dúvida sobre o que uma policy permite, a pergunta certa é "existe algum `Deny` explícito?" e só depois "existe algum `Allow`?". Ler na ordem contrária é como depurar um `if` começando pelo `else`.

Usuários, grupos e roles

As policies precisam ser anexadas a alguém. O IAM tem três tipos de identidade, e a diferença entre elas é a parte que mais importa neste capítulo.

Um **usuário IAM** representa uma pessoa ou uma aplicação, com credenciais permanentes: uma senha para o console, uma ou mais chaves de acesso para a API, ou as duas coisas. Foi a chave de um usuário IAM que vazou nos três casos que abriram o capítulo.

Um **grupo** é um conjunto de usuários que compartilham as mesmas policies. Você anexa a policy ao grupo `devs`, e todos os usuários dentro dele herdam aquelas permissões. Grupos não fazem login e não aparecem como principal numa requisição, e um grupo não pode conter outro grupo. O IAM não tem paciência para bonecas russas.

Uma **role** é uma identidade sem credenciais próprias. Ela tem permissões, como um usuário, mas ninguém tem uma senha ou uma chave dela. Quem precisa das permissões *assume* a role e recebe credenciais temporárias, que expiram sozinhas. Se o usuário IAM é um funcionário com crachá permanente, a role é o crachá de visitante entregue na portaria, válido por algumas horas e devolvido na saída. Como todo crachá de visitante, alguém vai tentar usá-lo no dia seguinte e descobrir que a catraca não se impressiona mais.

Uma role tem dois documentos. A **permissions policy** diz o que ela pode fazer, como qualquer outra policy. A **trust policy** diz quem pode assumi-la. Esta é a trust policy da role que aparece no fim do capítulo, para o servidor EC2 do capítulo 9:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Ela diz que o serviço EC2 pode assumir a role. Na prática, uma instância criada com essa role recebe credenciais temporárias automaticamente, renovadas pela própria AWS, e o código Python que roda nela usa essas credenciais sem que nenhuma chave apareça em arquivo, variável de ambiente ou imagem Docker, que são, por coincidência, os lugares onde alguém do seu time já guardou uma.

Credenciais que vencem

Quem entrega as credenciais temporárias é o **AWS STS** (*Security Token Service*). Uma credencial temporária tem três partes: um access key ID, uma secret access key e um session token. A sessão de uma role dura, por padrão, uma hora (mais ou menos o tempo de uma reunião que poderia ter sido um e-mail), e pode ir de 15 minutos até o máximo configurado na role (entre 1 e 12 horas).

Há um jeito rápido de saber, só de olhar, se uma credencial é permanente ou temporária. O access key ID de uma chave permanente começa com `AKIA`. O de uma credencial temporária começa com `ASIA`. Se você encontrar um `AKIA` num repositório, num log ou num ticket, alguém tem trabalho a fazer. Pela lei de Murphy, esse alguém é você.

A diferença prática é enorme. Uma chave `AKIA` que vazou vale até alguém apagá-la, o que pode levar meses se ninguém perceber. Uma credencial `ASIA` vazada vale, no pior caso, algumas horas. Ela ainda é um problema, mas um problema com prazo de validade, como o cupom de desconto que vence antes de você lembrar dele.

Menor privilégio sem paralisia

O **princípio do menor privilégio** diz que cada identidade deve ter só as permissões de que precisa para o seu trabalho, e nada além. Você já aplica isso no banco de dados: o usuário que a aplicação usa para conectar no Postgres não deveria ter permissão para `DROP DATABASE`, por maior que seja a tentação numa sexta-feira de deploy que deu errado.

O problema é que levar o princípio ao pé da letra desde o primeiro dia trava o trabalho. Escrever a policy perfeita para um sistema que ainda não existe é impossível, e cada `AccessDenied` interrompe você por dez minutos. Depois do quinto, você começa a considerar seriamente uma carreira em marcenaria. Na prática, o caminho que funciona tem algumas etapas:

1. Começar com as **AWS managed policies**, prontas e mantidas pela AWS, como `PowerUserAccess` ou `ReadOnlyAccess`. São largas, mas conhecidas.
2. Separar a identidade do dia a dia da identidade que mexe em permissões. Quem só desenvolve não precisa criar roles.
3. Para as aplicações, escrever **customer managed policies** (criadas por você) com ações e recursos específicos, como a policy das fotos que vimos acima.
4. Com o tempo, usar o *last accessed* do console do IAM, que mostra quais serviços uma identidade usou de verdade, para cortar o que sobra.

O ponto 2 é o que o Rangô vai aplicar daqui a pouco: um acesso de desenvolvimento para o dia a dia e um acesso de administrador para as poucas vezes em que for preciso mexer no IAM.

Existe ainda uma terceira forma de anexar permissões, as **inline policies**, escritas diretamente dentro de um usuário, grupo ou role, sem existir como objeto separado. Elas servem para regras que só valem para aquela identidade, como o tempero secreto que só funciona na receita da avó e que ninguém mais pode reaproveitar. Para todo o resto, prefira policies gerenciadas, que podem ser reutilizadas e versionadas.

Por que nunca usar o root

O root é o superusuário `postgres` da conta, como dissemos no capítulo 7, com uma diferença importante: nenhuma policy consegue limitar o root. Nem um `Deny` explícito, nem a quarentena da AWS. Se a chave do root vaza, a única defesa é apagá-la antes de o atacante fazer estrago. O root é aquele colega que não responde a nenhum gerente, e por isso ninguém deveria emprestar o crachá dele.

E o root quase nunca é necessário. A documentação da AWS mantém uma lista oficial das tarefas que só ele consegue executar, e ela é curta. As que um projeto como o Rangô pode encontrar algum dia são estas:

- Alterar o e-mail, a senha e as chaves de acesso do próprio root.
- Fechar a conta.
- Restaurar permissões quando o único administrador IAM se trancou para fora.
- Ativar o acesso de usuários IAM à cobrança (você fez isso no capítulo 7).
- Configurar o MFA Delete num bucket do S3.
- Editar ou apagar uma bucket policy que nega acesso a todos, inclusive aos administradores.

O item do administrador trancado para fora existe porque alguém, em algum lugar, escreveu um `Deny` contra si mesmo numa sexta-feira à tarde. O resto da lista trata de casos raros, como registrar-se como vendedor no marketplace de instâncias reservadas. Mudar o nome da conta, os contatos alternativos ou as regiões habilitadas não exige root.

O root pode ter chaves de acesso, e o console ainda oferece o botão para criá-las. Não crie. O capítulo 7 deixou o root sem chaves, com MFA em dois dispositivos, e o e-mail e a senha guardados. Ele fica assim, esquecido no cofre, como a cópia da chave de casa que mora na gaveta da sua mãe, até o dia em que uma tarefa da lista aparecer.

> **Na vida real:** Empresas que usam AWS Organizations podem centralizar o acesso root. Com isso, as contas-membro deixam de ter credenciais root, e contas novas já nascem sem elas. Voltaremos a esse modelo de várias contas no capítulo 23.

IAM Identity Center: um login para gente

Se usuário IAM tem credenciais permanentes, e role não tem credenciais próprias, como uma pessoa entra na AWS sem uma chave permanente? A resposta da AWS é o **AWS IAM Identity Center**, que já se chamou AWS SSO (o nome antigo sobrevive nos

comandos da CLI, como `aws sso login`). A AWS renomeia serviços com a mesma frequência com que você renomeia branches, e com o mesmo cuidado em atualizar todos os lugares que usavam o nome antigo.

O Identity Center é um ponto único de login para pessoas. Ele funciona assim:

- As pessoas ficam num diretório. Pode ser o diretório do próprio Identity Center ou um provedor de identidade que a empresa já usa, como Google Workspace, Microsoft Entra ID ou Okta.
- Você cria **permission sets**, que são modelos de permissão (por exemplo, "PowerUserAccess" ou "AdministratorAccess").
- Você atribui a um usuário ou grupo um permission set numa conta. O Identity Center cria, dentro da conta, uma role com aquelas permissões, com nome começando por `AWSReservedSSO_`.
- A pessoa faz login num portal (o *AWS access portal*), com senha e MFA, e escolhe qual role quer assumir. Recebe credenciais temporárias.

Para o console, isso vira um link no portal. Para a CLI, vira o comando `aws sso login`, que abre o navegador, faz você aprovar o acesso e guarda um token em cache. A CLI troca esse token por credenciais temporárias da role sempre que precisar, sem que você toque numa chave.

Compare com o caminho do `aws configure`. Lá, você teria um `AKIA` gravado em texto puro no `~/.aws/credentials`, válido para sempre, como a senha do Wi-Fi da casa dos seus pais. Aqui, o que fica no disco é um token que expira, e as credenciais da role duram uma hora.

Na prática: o Rangô sem nenhuma chave permanente

Esta seção monta o acesso do Rangô do jeito certo. O roteiro usa a conta do capítulo 7, o navegador e a AWS CLI v2 na versão 2.22.0 ou mais recente (o `aws --version` mostra qual está instalada). Nada do que aparece aqui gera cobrança.

Ativando o Identity Center

O root entra no console uma última vez nesta configuração. É a turnê de despedida dele e, como toda turnê de despedida, ele ainda volta de vez em quando. Ativar o Identity Center é uma decisão de administração da conta, e ainda não existe nenhuma outra identidade com permissão para isso.

1. A região selecionada no canto superior direito precisa ser `us-east-1`. O Identity Center vive numa região só, e é nela que você vai procurá-lo sempre.

2. A busca por **IAM Identity Center** leva ao botão **Enable**.
3. O console oferece ativar com o AWS Organizations, e essa é a opção recomendada, a que o exemplo aceita.

O passo 3 transforma a conta `rango-estudos` na conta de gerenciamento de uma organização com um único membro, ela mesma. Isso não muda nada no que você já criou e não tem custo. O modelo de várias contas fica para o capítulo 23; por enquanto, a organização existe só para o Identity Center funcionar.

Na página **Settings** do Identity Center fica o **AWS access portal URL**, que vale anotar. Ele tem o formato `https://d-xxxxxxxxxx.awsapps.com/start`, e você pode trocar o `d-xxxxxxxxxx` por um nome mais fácil de lembrar, como `rango`.

Usuário, grupo e permission sets

Ainda no Identity Center, com o root:

1. Em **Groups**, nascem dois grupos: `rango-admins` e `rango-devs`.
2. Em **Users**, nasce um usuário para você. Nos exemplos, ele se chama `dev-rango`, usa um e-mail seu diferente do e-mail do root e entra nos dois grupos.
3. Em **Permission sets**, nascem dois permission sets do tipo *predefined*: `AdministratorAccess` e `PowerUserAccess`. A duração de sessão padrão, uma hora, serve.
4. Em **AWS accounts**, com a conta `rango-estudos` selecionada, o botão **Assign users or groups** atribui o grupo `rango-admins` com `AdministratorAccess`, e depois o grupo `rango-devs` com `PowerUserAccess`.

Atribuir permissões a grupos, e não ao usuário diretamente, é o mesmo hábito do Django com `Group` e `Permission`: quando a sócia do Rangô precisar de acesso, basta criar o usuário dela e colocá-lo em `rango-devs`.

O `PowerUserAccess` permite quase tudo, menos gerenciar o IAM e a organização. É ter a chave de todas as salas do prédio, menos a do RH. É o permission set que a documentação da AWS sugere para quem desenvolve, e é o que você vai usar no dia a dia.

Falta a guarda de regiões. No permission set `PowerUserAccess`, em **Inline policy**, entra a policy `SoRegioesDoRango` da seção sobre o JSON. Depois de salvar, o Identity Center reaplica o permission set na conta automaticamente.

O Identity Center envia um e-mail de convite para `dev-rango`. Por ele, a senha é definida e, quando o sistema pede, um dispositivo MFA é cadastrado: uma passkey ou um aplicativo autenticador, como no root. No primeiro login no portal, aparecem as duas opções para a conta `rango-estudos`.

A partir daqui, o root sai de cena, com logout e tudo.

A CLI com SSO

Com o usuário criado, a configuração da CLI começa assim:

```
aws configure sso
```

O assistente faz algumas perguntas. As respostas do exemplo estão depois dos dois-pontos:

```
SSO session name (Recommended): rango
SSO start URL [None]: https://rango.awsapps.com/start
SSO region [None]: us-east-1
SSO registration scopes [sso:account:access]:
```

O *start URL* é o endereço do portal que você anotou. A *SSO region* é a região onde o Identity Center foi ativado. O escopo pode ficar no valor padrão, com um simples Enter.

Em seguida, a CLI abre o navegador, onde o login é feito como `dev-rango` (se ainda não estiver feito) e o acesso é aprovado. A tela de aprovação mostra um cliente chamado `botocore-client-` seguido do nome da sessão. Desde a versão 2.22.0, esse fluxo usa PKCE (a especificação diz que se pronuncia "pixy", como fadinha, o que faz dela a sigla mais fofa da história da segurança), um protocolo de autorização que exige que o navegador esteja na mesma máquina que a CLI.

De volta ao terminal, o assistente lista as roles disponíveis:

```
The only AWS account available to you is: 123456789012
Using the account ID 123456789012
There are 2 roles available to you.
  AdministratorAccess
> PowerUserAccess
Using the role name "PowerUserAccess"
Default client Region [None]: us-east-1
CLI default output format (json if not specified) [None]: json
Profile name [PowerUserAccess-123456789012]: rango-dev
```

No exemplo, a escolha é `PowerUserAccess`, e o perfil ganha o nome `rango-dev`. Uma segunda rodada do `aws configure sso`, com o mesmo nome de sessão (`rango`), escolhe `AdministratorAccess` e chama o perfil de `rango-admin`. O `~/.aws/config` fica assim:

```
[profile rango-dev]
sso_session = rango
sso_account_id = 123456789012
sso_role_name = PowerUserAccess
region = us-east-1
output = json

[profile rango-admin]
```

```
sso_session = rango
sso_account_id = 123456789012
sso_role_name = AdministratorAccess
region = us-east-1
output = json

[sso-session rango]
sso_start_url = https://rango.awsapps.com/start
sso_region = us-east-1
sso_registration_scopes = sso:account:access
```

O arquivo merece uma leitura atenta. Não há nenhuma chave nele. Há um endereço de portal, um número de conta e o nome de uma role. Você pode versionar esse arquivo num repositório interno sem expor nada, e por isso ele é seguro para dividir com a equipe. É o raro arquivo de configuração que pode aparecer numa apresentação ao vivo sem ninguém gritar "tira, tira, tira".

A seção `sso-session` é compartilhada pelos dois perfis. Isso quer dizer que um único login serve para os dois. O teste fica assim:

```
aws sso login --sso-session rango
aws sts get-caller-identity --profile rango-dev
```

```
{
    "UserId": "AROAEXAMPLEID7EXAMPLE:dev-rango",
    "Account": "123456789012",
    "Arn": "arn:aws:sts::123456789012:assumed-
role/AWSReservedSSO_PowerUserAccess_0a1b2c3d4e5f6a7b/dev-rango"
}
```

O mesmo comando que falhou no fim do capítulo 7 agora responde. O ARN conta a história inteira: você é `dev-rango`, e está usando uma sessão (`sts`, `assumed-role`) da role que o Identity Center criou a partir do permission set.

Para não precisar digitar `--profile` em todo comando, o perfil padrão pode ficar definido no shell:

```
export AWS_PROFILE=rango-dev
```

O token da sessão SSO fica em `~/.aws/sso/cache`. Enquanto ele valer, a CLI renova as credenciais da role sozinha. Quando ele expirar, os comandos passam a falhar com uma mensagem pedindo novo login, e basta um novo `aws sso login`. Para encerrar tudo antes disso, como ao sair de um computador emprestado (ou do computador da sala de reunião, que já viu coisas demais):

```
aws sso logout
```

```
Successfully signed out of all SSO profiles.
```

> **Dica:** Se você trabalha num servidor por SSH, ou num WSL que não consegue abrir o navegador do Windows, use `aws sso login --use-device-code`. A CLI mostra um endereço do tipo `https://device.sso.us-east-1.amazonaws.com/` e um código curto, como `QCFK-N451`. Você abre o endereço em qualquer navegador, digita o código e aprova.

O erro que você quer ver

Na raiz do repositório do Rangô, a pasta `iam/` guarda a trust policy da seção sobre roles, no arquivo `iam/rango-ec2-confianca.json`. Antes de criar a role do servidor com o perfil de administrador, o exemplo tenta com o perfil do dia a dia:

```
aws iam create-role \
  --role-name rango-ec2 \
  --assume-role-policy-document file://iam/rango-ec2-confianca.json \
  --profile rango-dev
```

```
An error occurred (AccessDenied) when calling the CreateRole operation: User:
arn:aws:sts::123456789012:assumed-
role/AWSReservedSSO_PowerUserAccess_0a1b2c3d4e5f6a7b/dev-rango is not authorized to
perform: iam:CreateRole on resource: arn:aws:iam::123456789012:role/rango-ec2 because no
identity-based policy allows the iam:CreateRole action
```

Esse erro é o sistema funcionando. Provavelmente é o único `AccessDenied` da sua carreira que vai deixar você contente. A mensagem traz o principal, a ação, o recurso e o motivo ("nenhuma policy baseada em identidade permite"), as três peças da pergunta que o IAM responde em toda requisição. Quando você receber um `AccessDenied` no futuro, leia a mensagem inteira, porque ela costuma dizer exatamente o que falta. Se o motivo citar um `explicit deny`, o problema é outro: existe uma regra negando, e acrescentar um `Allow` não vai adiantar.

O mesmo acontece com uma tentativa de subir uma instância em Singapura com o perfil `rango-dev`. O `Deny` de regiões barra a operação antes de qualquer outra checagem:

```
aws ec2 run-instances \
  --image-id ami-12345678 \
  --instance-type t3.micro \
  --region ap-southeast-1 \
  --profile rango-dev
```

O erro vem com `UnauthorizedOperation` e uma mensagem codificada, um bloco de caracteres que parece ter sido escrito por um gato andando no teclado. A AMI do exemplo é fictícia, e não importa: a policy nega a chamada antes de a AWS sequer olhar para ela, então nada é criado.

Uma role para o servidor do capítulo 9

No próximo capítulo, o Rangô vai rodar numa instância EC2. Para administrá-la sem abrir a porta 22 (a porta mais bisbilhotada da internet, com robôs batendo nela o dia inteiro feito vendedor de telemarketing) nem guardar chave SSH, usaremos o AWS Systems Manager Session Manager, e para isso a instância precisa de uma role com a managed policy `AmazonSSMManagedInstanceCore`. Vamos deixá-la pronta agora.

Agora, com o perfil de administrador:

```
aws iam create-role \
  --role-name rango-ec2 \
  --assume-role-policy-document file://iam/rango-ec2-confianca.json \
  --description "Role da EC2 do Rango" \
  --profile rango-admin

aws iam attach-role-policy \
  --role-name rango-ec2 \
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore \
  --profile rango-admin
```

Uma instância EC2 não recebe a role diretamente. Ela recebe um **instance profile**, uma espécie de envelope que carrega a role. É burocracia de cartório trazida para a nuvem: a role só entra na instância dentro de um envelope com firma reconhecida. Quando a role nasce pelo console, o instance profile é criado junto e sem aviso. Pela CLI, a criação é manual:

```
aws iam create-instance-profile \
  --instance-profile-name rango-ec2 \
  --profile rango-admin

aws iam add-role-to-instance-profile \
  --instance-profile-name rango-ec2 \
  --role-name rango-ec2 \
  --profile rango-admin
```

Antes de existir qualquer servidor, você já consegue perguntar ao IAM o que essa role pode fazer. O simulador de políticas avalia as ações sem executá-las, um ensaio geral em que ninguém se machuca:

```
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:role/rango-ec2 \
  --action-names ssm:UpdateInstanceInformation s3:ListAllMyBuckets \
  --query 'EvaluationResults[].[EvalActionName,EvalDecision]' \
  --output table \
  --profile rango-admin
```

```
-------------------------------------------------
|             SimulatePrincipalPolicy           |
+--------------------------------+----------------+
|  ssm:UpdateInstanceInformation |  allowed       |
```

```
|  s3:ListAllMyBuckets           |  implicitDeny   |
+--------------------------------+-----------------+
```

A primeira ação é uma das que o agente do Systems Manager usa para se registrar, e está liberada pela managed policy. A segunda não aparece em nenhuma policy da role, então cai na negação implícita. É o menor privilégio visto de perto: o servidor do Rangô vai conseguir ser administrado, e não vai conseguir listar os buckets da conta.

Para fechar, o resumo da conta mostra o estado do root:

```
aws iam get-account-summary \
  --query 'SummaryMap.[AccountMFAEnabled,AccountAccessKeysPresent]' \
  --profile rango-admin
```

```
[
    1,
    0
]
```

O `1` diz que o root tem MFA. O `0` diz que ele não tem chaves de acesso. São os dois números que você quer ver em toda conta. Em binário, quase dá para ler um sinal de positivo.

Python usa o mesmo caminho

O boto3, o SDK da AWS para Python, procura credenciais na mesma ordem que a CLI e entende os perfis SSO do `~/.aws/config`. Com uma versão recente do boto3 instalada no ambiente virtual do backend:

```
import boto3

identidade = boto3.client("sts").get_caller_identity()
print(identidade["Arn"])
```

```
AWS_PROFILE=rango-dev python quem_sou_eu.py
```

```
arn:aws:sts::123456789012:assumed-
role/AWSReservedSSO_PowerUserAccess_0a1b2c3d4e5f6a7b/dev-rango
```

Repare que o código não menciona perfil nem chave. São três linhas que um estagiário escreveria sem medo e que, pela primeira vez, não vão gerar um incidente de segurança. No seu notebook, a credencial vem do perfil SSO. No servidor do capítulo 9, o mesmo código vai pegar as credenciais da role `rango-ec2`, sem mudar uma linha. É assim que o código do Rangô vai falar com a AWS daqui para a frente: sem `aws_access_key_id=` em lugar nenhum.

Nada do que criamos tem custo. A role `rango-ec2` e o instance profile ficam para o capítulo 9. Para desfazer tudo, o caminho é remover a role do instance profile, apagar os dois e desativar o Identity Center nas configurações dele.

Armadilhas e casos reais

> **Armadilha:** Apagar o commit não apaga a chave. Se uma chave foi para um repositório, mesmo que por segundos e mesmo que o histórico tenha sido reescrito com `git push --force`, considere-a comprometida. Forks, clones e os robôs que leem o fluxo público já podem ter uma cópia. O `git push --force` reescreve o histórico do repositório, e não a memória da internet. O único remédio é desativar e apagar a chave na AWS, e depois procurar no CloudTrail o que foi feito com ela.
>
> **Armadilha:** A quarentena da AWS dá uma falsa sensação de alívio. Como o teste da Unit 42 mostrou, a `AWSCompromisedKeyQuarantineV3` só nega uma lista de ações. A chave continua autenticando, e tudo fora da lista segue permitido. Se você receber o e-mail da AWS sobre uma chave exposta, apague a chave. Não espere a investigação terminar para fazer isso.

Um detalhe curioso do mesmo teste: no CloudTrail, o evento `AttachUserPolicy` que aplica a quarentena aparece como se tivesse sido feito pelo próprio usuário afetado, e não pela AWS. Quem investiga um incidente pela primeira vez pode achar que o atacante anexou a política, quando foi a AWS que fez isso. É o equivalente forense a encontrar as próprias digitais na cena de um crime que você não cometeu.

> **Armadilha:** `iam:PassRole` com `"Resource": "*"` abre uma porta para escalada de privilégio. `PassRole` é a permissão de entregar uma role a um serviço, por exemplo ao criar uma instância EC2 com um instance profile. Se uma identidade pode criar instâncias e passar qualquer role, ela pode subir uma instância com a role de mais permissões da conta, entrar nela e agir como essa role. É dar a alguém a chave do carro e também o direito de escolher qual carro da garagem levar. Ao dar `PassRole`, sempre restrinja o `Resource` às roles específicas, como `arn:aws:iam::123456789012:role/rango-ec2`.
>
> **Armadilha:** Variáveis de ambiente esquecidas passam na frente do perfil. Se o seu shell tem `AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY` definidas (de um tutorial antigo, de um `.env` carregado sem querer), elas têm precedência sobre o `AWS_PROFILE`. Você acha que está usando o SSO e está usando uma chave antiga,

> como quem descobre que passou o dia inteiro editando o arquivo errado. Na dúvida, rode `aws sts get-caller-identity` antes de qualquer comando que cria ou apaga coisas.
>
> **Armadilha:** `"Action": "*"` e `"Resource": "*"` juntos numa policy de aplicação. É a forma mais rápida de fazer o erro de permissão sumir, do mesmo jeito que deixar o celular no silencioso faz a cobrança do banco sumir, e transforma qualquer falha no código (uma injeção, uma dependência comprometida) num acesso de administrador à conta. Se você se pegar escrevendo isso para "destravar" algo, pare e leia a mensagem do `AccessDenied`, que diz qual ação falta.

Há também limites que surpreendem quando as policies crescem. Por padrão, um usuário, grupo ou role aceita até 10 managed policies anexadas (o valor pode ser aumentado). Uma customer managed policy pode ter até 6.144 caracteres, sem contar espaços em branco, e esse limite não é ajustável. A trust policy de uma role tem limite padrão de 2.048 caracteres. São valores de referência consultados em outubro de 2026; confira a página de cotas do IAM antes de desenhar algo que dependa deles. Quem bate no limite de tamanho costuma estar listando ações uma a uma, com a paciência de quem copia a lista telefônica à mão, quando um curinga restrito, como `s3:Get*`, resolveria.

> **Na vida real:** O GitHub tem um programa de varredura de segredos em parceria com provedores como a AWS, e é parte do motivo de a quarentena chegar tão rápido. Repositórios privados ficam fora do fluxo público, e mesmo assim estão longe de ser cofres: um repositório privado que vira público, um colaborador externo ou um laptop roubado bastam. A regra continua sendo nenhuma chave permanente em repositório nenhum.

Resumo

- O IAM responde a duas perguntas em toda requisição: quem está pedindo (autenticação) e se pode fazer aquilo (autorização), avaliando principal, ação e recurso.
- ARN é o identificador de qualquer recurso: `arn:aws:servico:regiao:conta:recurso`, com campos vazios onde não se aplicam.
- Policies são JSON com `Version` `2012-10-17` e uma lista de `Statement` com `Effect`, `Action`, `Resource` e, opcionalmente, `Condition`.
- Tudo começa negado; um `Allow` libera; um `Deny` explícito vence sempre.
- Usuários IAM têm credenciais permanentes (`AKIA`). Roles não têm credenciais próprias e entregam credenciais temporárias (`ASIA`) pelo STS, com trust policy e

permissions policy.

- Menor privilégio na prática: managed policies no começo, acesso de dia a dia separado do de administração, policies específicas para aplicações.
- O root fica sem chaves, com MFA, e só sai do cofre para a lista curta de tarefas que exigem ele.
- O IAM Identity Center dá login com MFA para pessoas e entrega roles temporárias por meio de permission sets.
- A CLI usa `aws configure sso` e `aws sso login`; o `~/.aws/config` não guarda nenhuma chave.
- O Rangô tem agora os perfis `rango-dev` e `rango-admin` e a role `rango-ec2`, pronta para o primeiro servidor.

Para onde vamos

Você tem uma conta protegida, um login sem chaves permanentes e uma role esperando um servidor que ainda não existe. O Rangô, por sua vez, continua rodando no Docker Compose do seu notebook. Se você fechar a tampa, ele sai do ar. Hoje, a disponibilidade do Rangô depende da bateria do seu notebook e de você não levá-lo para a cozinha. A sócia em Recife ainda não consegue abrir o cardápio.

Precisamos de um computador ligado o tempo todo, com IP público, num data center da AWS. No próximo capítulo, o Rangô ganha a primeira instância EC2, que recebe a role `rango-ec2`, é acessada pelo Session Manager sem abrir a porta 22 e recebe o Rangô instalado à mão, com Uvicorn e Nginx. Vai dar trabalho, e esse trabalho é parte da lição.

Referências

- Configurar a AWS CLI com IAM Identity Center: https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-sso.html
- Tarefas que exigem o root: https://docs.aws.amazon.com/IAM/latest/UserGuide/root-user-tasks.html
- Lógica de avaliação de políticas: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html
- Cotas do IAM e do STS: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_iam-quotas.html
- Passkeys como MFA para root e usuários IAM: https://aws.amazon.com/blogs/aws/aws-adds-passkey-multi-factor-authentication-

mfa-for-root-and-iam-users/

- Unit 42, teste de exposição de credenciais e a política de quarentena: https://unit42.paloaltonetworks.com/detecting-exposed-aws-iam-credentials/
- Campanha EleKtra-Leak: https://www.darkreading.com/cloud-security/elektra-leak-attackers-harvest-aws-cloud-keys-github-campaign
- Relato da Classmethod sobre chave exposta: https://dev.classmethod.jp/articles/risk-management-aws-access-key-accidentally-exposed
- Elementos de política JSON do IAM: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_elements.html
- O que é o IAM Identity Center: https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html

CAPÍTULO 9

EC2: seu primeiro servidor

No capítulo anterior, você parou de usar o root e passou a entrar na AWS com o usuário `dev-rango`, pelo IAM Identity Center, com credenciais temporárias nos perfis `rango-dev` e `rango-admin`. No fim, criou a role `rango-ec2`, com a policy `AmazonSSMManagedInstanceCore`, e o instance profile que a embrulha. Ela ficou esperando um servidor, como quem compra a capinha antes do celular. Agora ele vai existir.

O notebook que precisa dormir

Até aqui, o Rangô inteiro roda no Docker Compose do seu notebook: API, Redis, workers, Nginx. Funciona bem enquanto a tampa está aberta. A sua sócia em Recife quer mostrar o app para o dono da Cantina da Nona numa terça às 22h, e nesse horário o seu notebook está fechado na mochila, dormindo um sono bem mais tranquilo do que o seu.

Mesmo com o notebook aberto, a conta não fecha. Lembra do capítulo 1? A sua conexão de casa provavelmente está atrás de CGNAT, o IP que o mundo vê é compartilhado com outros clientes do provedor, e nenhuma conexão de fora chega até a sua máquina sem um pedido seu antes. O roteador reinicia, o IP muda, o Windows decide instalar atualizações às 3h. O Windows Update, aliás, tem uma disponibilidade que qualquer serviço seu invejaria: ele nunca falha em aparecer.

O que o Rangô precisa é simples de descrever: um computador ligado o tempo todo, num lugar com energia e rede redundantes, com um IP público que aceite conexões de qualquer lugar. Você poderia comprar um servidor e alugar espaço num data center. Poderia contratar uma VPS de um provedor pequeno. Ou poderia pedir um à AWS por uma chamada de API e começar a usá-lo em menos de dois minutos. Este capítulo segue o terceiro caminho, e vai fazer tudo à mão, de propósito, para você sentir o peso de cada passo. É o equivalente a aprender a dirigir num carro sem direção hidráulica.

O que a AWS aluga quando você pede um servidor

O **Amazon EC2** (Elastic Compute Cloud) aluga máquinas virtuais. São duas palavras com C, por isso o "C2" da sigla, uma solução tão criativa que parece ter saído de uma reunião que passou do horário. Cada máquina alugada se chama **instância**. No capítulo 3 você viu que um hypervisor fatia uma máquina física em várias VMs. A AWS faz exatamente isso, com um hypervisor próprio (o Nitro) e hardware dedicado para rede e disco, de modo que quase toda a CPU da máquina física fica para os clientes.

A cobrança das instâncias Linux é por segundo, com mínimo de 60 segundos, enquanto a instância está no estado *running*. Uma instância parada (*stopped*) não cobra CPU nem memória, mas o disco dela continua cobrando, como você vai ver adiante.

Para criar uma instância, você responde a poucas perguntas: que tamanho de máquina, que sistema operacional, como você vai entrar nela, quem pode falar com ela pela rede, o que ela faz ao ligar e quanto disco ela tem. Cada uma dessas perguntas corresponde a um conceito do EC2.

Tipos e famílias de instância

O tamanho da máquina é o **tipo de instância**. O nome segue um padrão que parece críptico até você aprender a ler, como receita de médico. Por exemplo, `t3.micro`:

- `t` é a **família**: o perfil de uso para o qual a máquina foi pensada.
- `3` é a **geração**: números maiores são hardware mais novo, em geral mais barato pelo mesmo desempenho.
- `micro` é o **tamanho**: quantas vCPUs e quanta memória.

Entre a geração e o ponto podem aparecer letras de atributos. As mais comuns são `g` (processador Graviton, da própria AWS, arquitetura Arm), `i` (Intel), `a` (AMD) e `d` (disco local NVMe). Assim, `t4g.small` é a família T, quarta geração, Graviton, tamanho small. `m7i-flex.large` é a família M, sétima geração, Intel, na variante flex.

As famílias que você vai encontrar com mais frequência:

Família	Para que serve	Proporção memória por vCPU
T	uso intermitente, com picos curtos (sites pequenos, ambientes de teste)	variável
M	uso geral, equilibrado	4 GiB
C	CPU intensa (processamento, codificação de vídeo)	2 GiB
R	muita memória (bancos de dados, caches)	8 GiB

A família T merece atenção, porque é a mais usada por quem está começando e a que mais surpreende. Ela é *burstable*: a instância tem direito a uma fração da CPU (a linha de base) e acumula **créditos de CPU** enquanto fica abaixo dessa linha. Quando a carga sobe, ela gasta os créditos e pode usar 100% da vCPU por um tempo. É como um plano de celular com franquia que acumula, com as letrinhas miúdas no parágrafo seguinte.

O que acontece quando os créditos acabam depende do modo. No modo *standard*, a CPU volta à linha de base, e a sua API fica lenta. No modo *unlimited*, a instância continua usando CPU acima da linha de base e a AWS cobra pelo excedente. As instâncias T3 sobem em modo unlimited por padrão. Para um servidor de estudo, prefira standard: lentidão você nota, cobrança extra você só descobre na fatura.

Lembra do erro `exec format error` do capítulo 3, quando uma imagem feita para amd64 rodou num Mac arm64? A mesma questão aparece aqui. Instâncias Graviton (`t4g`, `m7g` e similares) são Arm e costumam sair mais baratas pelo mesmo desempenho. Para o Rangô em Python puro, a diferença é invisível, porque as dependências têm *wheels* para as duas arquiteturas. Para começar, vamos de x86_64, que é a arquitetura do seu notebook (salvo se você usa um Mac com Apple Silicon, e nesse caso você já mencionou isso em pelo menos três conversas esta semana) e evita uma variável a mais no primeiro deploy.

O free tier no EC2

O que entra no free tier depende de quando a conta foi criada. Ele tem mais regras do que jogo de tabuleiro alemão, e as regras mudam conforme o ano em que você comprou a caixa. Contas criadas antes de 15 de julho de 2025 têm o modelo antigo, de 12 meses, em que `t2.micro` e `t3.micro` são elegíveis. Contas criadas a partir dessa data, como a `rango-estudos` do capítulo 7, funcionam com créditos: US$ 100 no cadastro e até US$ 100 a mais ao completar algumas atividades, por até seis meses. Parece missão de videogame, só que a recompensa é desconto em servidor. Nelas, os tipos elegíveis são `t3.micro`, `t3.small`, `t4g.micro`, `t4g.small`, `c7i-flex.large` e `m7i-flex.large`. Você pode conferir a lista na sua conta pela CLI, como vai fazer daqui a pouco.

O Rangô vai rodar numa `t3.micro`: 2 vCPUs, 1 GiB de memória. É pouco (o seu navegador gasta mais do que isso só para dizer bom-dia), mas suficiente para a API, dois workers e o cache, e é elegível nos dois modelos.

AMI: o sistema operacional de fábrica

Uma instância nasce de uma **AMI** (Amazon Machine Image): um molde com o sistema operacional instalado e o disco inicial pronto. Pense numa imagem Docker, só que de uma máquina inteira, com kernel e tudo. A AWS publica AMIs próprias, fabricantes de

sistemas publicam as deles (Ubuntu, Debian, Red Hat) e você pode criar as suas a partir de uma instância configurada.

Vamos usar o **Amazon Linux 2023** (AL2023), a distribuição da própria AWS. Ela usa `dnf` como gerenciador de pacotes, tem suporte previsto até junho de 2029, e o agente de que vamos precisar para entrar na máquina já vem instalado. A versão anterior, o Amazon Linux 2, chegou ao fim do suporte em 30 de junho de 2026. Muitos tutoriais na internet ainda usam o AL2 e comandos como `amazon-linux-extras`, que não existem no AL2023. Se um tutorial manda rodar `yum install` e `amazon-linux-extras`, ele é do sistema antigo, e provavelmente também ensina a instalar o Python 2.

Cada AMI tem um ID, como `ami-0abcd1234ef567890`, e esse ID muda a cada nova versão. A AWS marca cada AMI do AL2023 como obsoleta 90 dias depois do lançamento. Por isso, nunca escreva um ID de AMI fixo num script: ele envelhece mais rápido do que biblioteca de JavaScript da moda. A AWS mantém um parâmetro público no Systems Manager Parameter Store que sempre aponta para a AMI mais recente:

```
/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
```

Para Graviton, o final vira `arm64`. Na hora de criar a instância, você passa `resolve:ssm:` seguido desse caminho, e a AWS resolve o ID na hora. É o equivalente a depender de `python:3.12-slim` em vez de um digest fixo, com a diferença de que aqui você quer mesmo a versão mais nova.

> **Na vida real:** Muita gente escolhe Ubuntu no EC2 porque o Ubuntu 24.04 já vem com Python 3.12. O AL2023 traz o 3.9 como `python3` padrão, uma escolha conservadora que faz qualquer dev Python sentir saudade do futuro, e oferece o 3.12 como pacote separado (`python3.12`), que o Rangô instala logo mais.

Como você entra na máquina

A pergunta parece trivial e é a que mais causa incidentes. O jeito tradicional é SSH com um **key pair**: a AWS guarda a chave pública, coloca-a no servidor na criação, e você baixa um arquivo `.pem` com a chave privada. Esse arquivo vai morar na pasta Downloads, ao lado de `rango(1).pem` e `rango-final-agora-vai.pem`. Para entrar, você abre a porta 22 no firewall e roda `ssh -i rango.pem ec2-user@IP`.

Funciona, e tem três problemas. A porta 22 aberta para a internet recebe tentativas de login de robôs minutos depois de a instância subir. O `.pem` é uma credencial de longa duração que mora num arquivo, e você já viu no capítulo 8 o que acontece com credenciais de longa duração em arquivos. E, se três pessoas usam a mesma chave, ninguém sabe quem entrou e fez o quê.

A alternativa é o **Session Manager**, parte do AWS Systems Manager. Um agente instalado na instância (o SSM Agent) abre uma conexão de saída, por HTTPS na porta 443, até os endpoints do Systems Manager. Quando você pede uma sessão, a AWS confere as suas permissões no IAM e encaminha o terminal por essa conexão. Nenhuma porta de entrada fica aberta, não existe chave para perder, e cada sessão fica registrada com o nome de quem a abriu. É o fim do clássico "não fui eu" na reunião de análise de incidente.

Para funcionar, três coisas precisam existir:

1. O SSM Agent rodando na instância (vem na AMI do AL2023).
2. Um instance profile com uma role que tenha a policy `AmazonSSMManagedInstanceCore`. É a `rango-ec2` do capítulo 8.
3. Saída HTTPS da instância até os endpoints do Systems Manager, seja pela internet, seja por VPC endpoints (capítulo 10).

Do seu lado, você precisa do Session Manager plugin instalado junto da AWS CLI. O livro não usa key pair em lugar nenhum. Os arquivos `.pem` da sua pasta Downloads podem descansar em paz.

Security Group: o primeiro contato

Quem pode falar com a instância pela rede é decidido pelo **Security Group**, um firewall virtual ligado à placa de rede da instância. Pense nele como um middleware de autorização que roda antes da sua aplicação, só que no nível de pacotes: ele olha protocolo, porta e origem, e decide se a conexão entra.

Três características explicam quase todo o comportamento dele:

- Só existem regras de permissão. Tudo o que nenhuma regra libera fica bloqueado, como o implicit deny do IAM.
- Ele é *stateful*: se uma conexão de entrada foi permitida, a resposta sai sem precisar de regra de saída. E se a instância abre uma conexão de saída, a resposta volta sem regra de entrada.
- A origem de uma regra pode ser uma faixa CIDR (`0.0.0.0/0` é a internet inteira, `198.51.100.7/32` é um único IP) ou outro Security Group.

O *stateful* é o detalhe que mais poupa trabalho. O Security Group lembra quem saiu e deixa a resposta voltar, uma gentileza que muita catraca de metrô nunca teve.

Um Security Group recém-criado não aceita nenhuma entrada e permite toda a saída. É o adolescente da rede: sai quando quer e não deixa ninguém entrar no quarto. Se você criar uma instância sem dizer qual grupo usar, ela entra no Security Group *default* da VPC,

que só aceita tráfego de instâncias do próprio grupo. Na prática, nada de fora consegue chegar. O capítulo 10 volta a esse assunto com a rede completa, e lá você vai comparar o Security Group com as Network ACLs.

User data: o script do primeiro boot

Ao criar a instância, você pode entregar um script chamado **user data**. Na primeira inicialização, o *cloud-init*, um programa que vem na AMI, executa esse script como root, com a autoridade de quem nunca precisou pedir `sudo` na vida. É o lugar natural para instalar pacotes e criar usuários, coisas que você faria em todo servidor novo e que, feitas à mão, você esqueceria em pelo menos um.

Dois detalhes evitam muita confusão. O user data roda só no primeiro boot: se você parar a instância, editar o script e ligá-la de novo, nada é reexecutado. Como primeira impressão em entrevista, não existe segunda chance. E o conteúdo fica legível de dentro da instância pelo *Instance Metadata Service* (IMDS), um endereço HTTP local que qualquer processo da máquina consegue consultar. Qualquer senha colocada ali pode ser lida por qualquer processo da máquina. Segredos vão para um cofre gerenciado, assunto do capítulo 16: o Parameter Store para a configuração e o Secrets Manager para a senha do banco, que precisa girar. O limite de tamanho é 16 KB, o que é suficiente para um script de instalação e insuficiente para alguém tentar colar um `node_modules` ali, o que é uma bênção.

EBS: um disco que mora na rede

O disco da instância é um volume **EBS** (Elastic Block Store). Para o sistema operacional ele se comporta como um disco comum, mas fisicamente está em outro lugar do data center, ligado à instância pela rede. É um HD externo com um cabo muito comprido, no qual, felizmente, ninguém consegue tropeçar. Isso tem consequências práticas:

- O volume sobrevive quando a instância é parada. Você desliga a máquina, liga de novo, e os arquivos estão lá.
- O volume pertence a uma zona de disponibilidade. Uma instância em `us-east-1a` não monta um volume de `us-east-1b`.
- Você paga pelo tamanho provisionado, esteja a instância ligada ou não.
- Você pode tirar *snapshots*, cópias de um momento guardadas pela AWS, e criar volumes novos a partir delas.

O tipo padrão hoje é o `gp3`. Ele custa cerca de US$ 0,08 por GB-mês e inclui 3.000 IOPS e 125 MiB/s de vazão sem custo adicional (us-east-1, consulta de outubro de 2026; confira a página de preços do EBS). Um disco de 10 GiB sai por uns US$ 0,80 por mês.

Uma opção do volume decide o que acontece quando a instância é destruída (*terminated*): `DeleteOnTermination`. Com `true`, o volume vai junto com a instância. Com `false`, ele fica órfão, cobrando todo mês, até alguém lembrar dele, como a assinatura da academia contratada numa empolgação de janeiro. Vamos declarar o valor explicitamente na criação, para não depender do padrão.

Na prática: o Rangô no EC2

O plano é este: criar um Security Group que libere só a porta 80, escrever o script de user data, subir a instância com a role `rango-ec2`, entrar pelo Session Manager e instalar o Rangô à mão, com systemd e Nginx, como no capítulo 3. Desta vez a máquina fica num data center, com IP público, longe do seu gato e da sua xícara de café.

Pré-requisitos

- AWS CLI v2 com o perfil `rango-dev` do capítulo 8, região `us-east-1`.
- Session Manager plugin instalado. A página da documentação tem instaladores para Windows, macOS e Linux. Este comando mostra a versão instalada:

```
session-manager-plugin --version
```

- O repositório do Rangô no GitHub, com o backend em `backend/`.

Antes de criar qualquer coisa, o exemplo faz login e confere quem está do outro lado, um hábito que poupa mais vergonha do que conferir o microfone antes de falar mal de alguém numa chamada de vídeo:

```
export AWS_PROFILE=rango-dev
aws sso login
aws sts get-caller-identity --query Arn --output text
```

```
arn:aws:sts::123456789012:assumed-
role/AWSReservedSSO_PowerUserAccess_0a1b2c3d4e5f6a7b/dev-rango
```

Em seguida, a lista dos tipos de instância elegíveis no free tier da conta:

```
aws ec2 describe-instance-types \
  --filters Name=free-tier-eligible,Values=true \
  --query "InstanceTypes[*].[InstanceType]" \
  --output text | sort
```

Com a `t3.micro` na lista, o caminho está livre.

O Security Group

Toda conta vem com uma **VPC default** em cada região, uma rede pronta com uma subnet pública por zona de disponibilidade. Ela existe para que você consiga criar uma instância sem antes desenhar uma rede. Vamos usá-la neste capítulo e substituí-la no próximo. Ela é o apartamento mobiliado da AWS: ótimo para chegar, ninguém deveria morar nele para sempre.

```
VPC_ID=$(aws ec2 describe-vpcs \
  --filters Name=is-default,Values=true \
  --query "Vpcs[0].VpcId" --output text)

SG_ID=$(aws ec2 create-security-group \
  --group-name rango-web \
  --description "HTTP publico da API do Rango" \
  --vpc-id "$VPC_ID" \
  --query GroupId --output text)

aws ec2 authorize-security-group-ingress \
  --group-id "$SG_ID" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

echo "$SG_ID"
```

Repare na descrição sem acento. O campo de descrição do Security Group aceita só um conjunto restrito de caracteres ASCII, e um "Rangô" ali faz o comando falhar com `InvalidParameterValue`. Em 2026, o acento circunflexo ainda é tratado como ameaça por certos campos de formulário.

Note também o que ficou de fora: nenhuma regra para a porta 22. O grupo aceita HTTP de qualquer lugar e mais nada. A saída continua liberada, e é por ela que o SSM Agent, o `dnf` e o `git` vão falar com o mundo.

O script de user data

O script fica na raiz do repositório, em `infra/ec2/user-data.sh`:

```
#!/bin/bash
set -euxo pipefail

dnf install -y nginx git python3.12 python3.12-pip valkey
systemctl enable --now valkey nginx

useradd --system --create-home --home-dir /opt/rango \
  --shell /sbin/nologin rango
install -d -o root -g rango -m 750 /etc/rango
```

O script instala o Nginx, o git, o Python 3.12 e o **Valkey**, o fork do Redis que você conheceu no capítulo 4. Ele nasceu em 2024, quando o Redis mudou de licença e a comunidade open source fez o que sabe fazer de melhor quando fica contrariada: um fork. No AL2023, o Valkey é o substituto do antigo pacote `redis6`, e fala o mesmo

protocolo: o cliente `redis` do Python conecta sem mudança. A configuração padrão dele escuta só em `127.0.0.1`, o que basta, porque só a API e os workers da própria máquina falam com ele.

O `set -euxo pipefail` faz o script parar no primeiro erro e registrar cada comando executado. Sem ele, um `dnf` que falha passa em silêncio, o script segue em frente otimista como dev estimando prazo, e você só descobre na hora de rodar o Python.

Subindo a instância (e tropeçando no IAM)

Com o grupo e o script prontos, a instância nasce com este comando:

```
aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
x86_64 \
  --instance-type t3.micro \
  --credit-specification CpuCredits=standard \
  --iam-instance-profile Name=rango-ec2 \
  --security-group-ids "$SG_ID" \
  --metadata-options HttpTokens=required,HttpEndpoint=enabled \
  --block-device-mappings '[{"DeviceName":"/dev/xvda","Ebs":
{"VolumeSize":10,"VolumeType":"gp3","Encrypted":true,"DeleteOnTermination":true}}]' \
  --user-data file://infra/ec2/user-data.sh \
  --tag-specifications \
    'ResourceType=instance,Tags=[{Key=Name,Value=rango-api-01},
{Key=Projeto,Value=rango}]' \
    'ResourceType=volume,Tags=[{Key=Projeto,Value=rango}]' \
  --query "Instances[0].InstanceId" --output text
```

Cada opção responde a uma pergunta da seção anterior. Não há `--key-name`, então nenhum key pair. O `HttpTokens=required` exige a versão 2 do serviço de metadados, que você vai entender na seção de armadilhas. O volume raiz tem 10 GiB, é `gp3`, criptografado e morre junto com a instância. A tag `Name` é o nome que aparece no console.

E o resultado:

```
An error occurred (UnauthorizedOperation) when calling the RunInstances operation:
You are not authorized to perform this operation. User: arn:aws:sts::123456789012:
assumed-role/AWSReservedSSO_PowerUserAccess_0a1b2c3d4e5f6a7b/dev-rango is not
authorized to perform: iam:PassRole on resource: arn:aws:iam::123456789012:role/rango-
ec2
because no identity-based policy allows the iam:PassRole action. Encoded authorization
failure message: ...
```

(A mensagem real termina com um bloco codificado longo, omitido aqui.)

Você já viu esse filme no capítulo 8, e ele não melhorou na reprise. O `PowerUserAccess` libera quase tudo no EC2, mas nada no IAM, e entregar uma role a uma instância exige `iam:PassRole`. A AWS trata isso como uma ação de IAM, porque quem pode passar uma

role pode, na prática, agir como ela. É o velho golpe de pedir emprestado o crachá do diretor só para pegar um café. A armadilha do capítulo anterior dizia para nunca liberar `PassRole` com `"Resource": "*"`. Agora é a hora de aplicar.

A correção acontece no portal do Identity Center, com o perfil de administrador: no permission set `PowerUserAccess`, em **Inline policy**, entra um segundo statement ao lado do `SoRegioesDoRango`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SoRegioesDoRango",
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["us-east-1", "sa-east-1"]
        }
      }
    },
    {
      "Sid": "PassarSoARoleDoServidor",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::123456789012:role/rango-ec2",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": "ec2.amazonaws.com"
        }
      }
    }
  ]
}
```

O `Resource` limita a uma única role, e a condição `iam:PassedToService` garante que ela só pode ser entregue ao EC2. O `123456789012` é o ID da conta do exemplo; numa conta real, entra o ID dela. Depois de salvar, é preciso esperar o Identity Center reaplicar o permission set e fazer login de novo (`aws sso login`), porque a sessão antiga ainda carrega as permissões antigas. Na segunda tentativa, o `run-instances` responde:

```
i-0a1b2c3d4e5f67890
```

O ID vai para uma variável, e o comando seguinte espera a instância ficar pronta:

```
INSTANCE_ID=i-0a1b2c3d4e5f67890
aws ec2 wait instance-running --instance-ids "$INSTANCE_ID"

aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].
[State.Name,Placement.AvailabilityZone,PublicIpAddress,PrivateIpAddress]" \
  --output text
```

```
running us-east-1c  198.51.100.23   172.31.23.104
```

Repare nos dois endereços. O `172.31.23.104` é privado, da faixa `172.16.0.0/12` do capítulo 1, e é o endereço que a instância enxerga em si mesma. O `198.51.100.23` é o público, e quem faz a tradução entre os dois é a rede da AWS, num NAT de um para um. Um `ip addr` dentro da instância não mostra o IP público em lugar nenhum. A instância não sabe o próprio endereço público, como aquele amigo que nunca decorou o número do próprio celular.

Uma instância *running* ainda pode estar no meio do boot, como quem avisa que já está a caminho e ainda nem saiu do banho. O SSM Agent precisa subir e se registrar antes de aceitar sessões. O estado dele aparece assim:

```
aws ssm describe-instance-information \
  --filters Key=InstanceIds,Values="$INSTANCE_ID" \
  --query "InstanceInformationList[0].PingStatus" --output text
```

Quando a resposta for `Online`, a instância está pronta para receber você. Se depois de uns cinco minutos ela ainda não aparecer, as causas costumam ser duas: o instance profile não foi associado, ou a instância não consegue sair para a internet na porta 443.

Entrando sem abrir a porta 22

```
aws ssm start-session --target "$INSTANCE_ID"
```

```
Starting session with SessionId: dev-rango-0f1e2d3c4b5a69788
sh-5.2$
```

Você está dentro, e o Security Group não tem nenhuma regra de SSH. É como entrar numa casa sem porta, o que soa assustador até você lembrar que o ladrão também não tem por onde entrar. A sessão abre como o usuário `ssm-user`, num `sh`. O passo seguinte é virar root e conferir se o user data terminou:

```
sudo -i
cloud-init status --wait
```

```
status: done
```

Se aparecer `status: error`, o script de user data falhou em algum comando. Com o `set -x`, cada comando executado ficou registrado no log do cloud-init, e o último antes do erro é o culpado. Em logs, como em reunião, quem falou por último antes do silêncio costuma ser o responsável.

Os serviços do script também precisam estar de pé:

```
systemctl is-active nginx valkey
valkey-cli ping
python3.12 --version
```

```
active
active
PONG
Python 3.12.12
```

A versão exata do patch do Python vai variar conforme a data da AMI. Repare no `PONG`: responder "pong" a um "ping" é a piada mais antiga dos bancos em memória, e ela continua funcionando.

Trazendo o código

O código está no GitHub. Para o servidor ler o repositório sem que você coloque um token pessoal na máquina (aquele com permissão para tudo, criado há anos e marcado como "sem expiração"), a solução é uma **deploy key**: uma chave SSH que dá acesso de leitura a um único repositório. A chave privada vai ser gerada dentro do servidor e nunca sai dele.

```
cd /opt/rango
sudo -u rango mkdir -p -m 700 /opt/rango/.ssh
sudo -u rango ssh-keygen -t ed25519 -N "" \
  -f /opt/rango/.ssh/id_ed25519 -C "rango-api-01"
cat /opt/rango/.ssh/id_ed25519.pub
```

O `cd /opt/rango` existe por um motivo prosaico: o git aborta quando o diretório atual é ilegível para o usuário que o executa, e o `/root` é ilegível para o `rango`.

A linha que começa com `ssh-ed25519` vai para o GitHub, no repositório, em **Settings > Deploy keys > Add deploy key**, com **Allow write access** desmarcado. Depois vem o clone:

```
sudo -u rango git clone git@github.com:SEU-USUARIO/rango.git /opt/rango/codigo
```

Na primeira conexão o SSH mostra a impressão digital do servidor do GitHub e pede confirmação. O certo é comparar com as impressões digitais publicadas na documentação do GitHub antes de responder `yes`, em vez de fazer o que quase todo mundo faz, que é aceitar sem ler, como termos de uso.

Agora o ambiente virtual e as dependências:

```
sudo -u rango python3.12 -m venv /opt/rango/venv
sudo -u rango /opt/rango/venv/bin/pip install \
  -r /opt/rango/codigo/backend/requirements.txt
```

Com 1 GiB de memória, a instalação leva um ou dois minutos, tempo exato para se perguntar se não deveria ter escolhido uma máquina maior.

Configuração e serviços

A configuração da API vai num arquivo de ambiente, legível só pelo root e pelo grupo `rango`, como no capítulo 3:

```
cat > /etc/rango/api.env <<'EOF'
REDIS_URL=redis://127.0.0.1:6379/0
CORS_ORIGINS=http://localhost:5173
EOF
chown root:rango /etc/rango/api.env
chmod 640 /etc/rango/api.env
```

O `CORS_ORIGINS` aponta para o front local por enquanto. O React ganha casa própria, com domínio e HTTPS, no capítulo 14. Até lá, continua morando de favor no notebook.

A unit do systemd é quase a mesma do capítulo 3. A diferença está nos caminhos e na dependência do Valkey. A unit fica em `/etc/systemd/system/rango-api.service`:

```
[Unit]
Description=API do Rango (Uvicorn)
After=network-online.target valkey.service
Wants=network-online.target valkey.service

[Service]
User=rango
Group=rango
WorkingDirectory=/opt/rango/codigo/backend
EnvironmentFile=/etc/rango/api.env
ExecStart=/opt/rango/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000 --
workers 2 --proxy-headers
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Dois processos do Uvicorn, um por vCPU, por causa do GIL que você viu no capítulo 4. Basta mencionar o GIL numa conferência de Python para a sala inteira suspirar ao mesmo tempo.

Os workers de pedidos do capítulo 5 são dois processos iguais que mudam só o grupo de consumo. O systemd tem um recurso para isso: a *template unit*. Um arquivo com `@` no nome vira um molde, e o que vier depois do `@` na hora de ativar entra no lugar de `%i`. O molde fica em `/etc/systemd/system/rango-worker@.service`:

```
[Unit]
Description=Worker do Rango (grupo %i)
After=network-online.target valkey.service
Wants=network-online.target valkey.service
```

```
[Service]
User=rango
Group=rango
WorkingDirectory=/opt/rango/codigo/backend
EnvironmentFile=/etc/rango/api.env
ExecStart=/opt/rango/venv/bin/python -m app.worker %i
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

É o mesmo raciocínio de uma função com parâmetro em vez de duas funções copiadas (que, como todo código copiado, começam a divergir em silêncio na terceira semana). A ativação de tudo fica assim:

```
systemctl daemon-reload
systemctl enable --now rango-api rango-worker@restaurante rango-worker@email
systemctl is-active rango-api rango-worker@restaurante rango-worker@email
curl -s http://127.0.0.1:8000/health
```

Se algum serviço não estiver `active`, o `journalctl -u rango-api -n 50` mostra os logs em JSON do capítulo 6, com o traceback do Python, que se lê de baixo para cima, como manda a tradição.

O Nginx na frente

Falta o proxy reverso, em `/etc/nginx/conf.d/rango.conf`:

```
upstream rango_api {
    server 127.0.0.1:8000;
}

server {
    listen 80 default_server;
    server_name _;

    location / {
        proxy_pass http://rango_api;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 2s;
    }
}
```

O `default_server` faz este bloco atender requisições que chegam pelo IP, sem nome de host conhecido, que é como vamos acessar por enquanto. O `server_name _` é o jeito do Nginx de dizer "qualquer um", com um sublinhado que parece alguém dando de ombros. O teste da configuração e a recarga vêm juntos:

```
nginx -t && systemctl reload nginx
```

Se o `nginx -t` reclamar de `duplicate default server for 0.0.0.0:80`, é porque o `/etc/nginx/nginx.conf` da sua versão já declara um servidor padrão. A saída é remover o `default_server` de lá e testar de novo.

O teste que importa

Fora da sessão (são dois `exit`), a chamada à API pelo IP público, feita do notebook, fica assim:

```
curl -i http://198.51.100.23/restaurantes
```

```
HTTP/1.1 200 OK
Server: nginx/1.28.0
Content-Type: application/json
X-Rango-Instancia: ip-172-31-23-104.ec2.internal
X-Request-ID: 6f3c1a2e-...

[{"nome":"Cantina da Nona","taxa_entrega":6.9,"taxa_entrega_fmt":"R$ 6,90"},...]
```

(A versão do Nginx e os campos exatos do JSON podem variar.)

O header `X-Rango-Instancia` do capítulo 4 agora mostra o hostname da instância, que a AWS monta a partir do IP privado, um nome que só uma mãe muito apaixonada por redes daria a um filho. A primeira chamada leva uns 300 ms, por causa do `sleep` que simula o Postgres; a segunda vem do cache no Valkey e responde bem mais rápido. No navegador do celular, fora do Wi-Fi de casa, `http://198.51.100.23/docs` mostra a documentação interativa do FastAPI. A sua sócia em Recife pode fazer o mesmo, com o seu notebook fechado. Desta vez, o link que você manda para ela abre mesmo com você dormindo.

Agora o teste que separa um servidor de um processo esquecido num terminal. O exemplo entra de novo e reinicia a máquina:

```
aws ssm start-session --target "$INSTANCE_ID"
sudo reboot
```

A sessão cai. Um minuto depois, o mesmo `curl`, feito do notebook, recebe resposta da API, porque o systemd subiu o Valkey, o Uvicorn, os workers e o Nginx na ordem certa, sem ninguém logado. No capítulo 3, fechar o SSH matava o Uvicorn com SIGHUP. Aqui, ninguém precisa estar conectado para o Rangô estar no ar. Reiniciar um servidor de propósito e vê-lo voltar sozinho é uma das poucas alegrias puras da profissão.

Quanto isso custa e como desligar

Com preços aproximados de referência em `us-east-1`, consultados em outubro de 2026, a conta fica assim:

- A instância `t3.micro` cobra por segundo enquanto está ligada. O preço por hora muda com a região e o tempo; consulte a página de preços sob demanda do EC2.
- O volume `gp3` de 10 GiB custa cerca de US$ 0,80 por mês, ligado ou desligado.
- O IPv4 público custa US$ 0,005 por hora, o que dá uns US$ 3,65 por mês se ficar ligado o mês inteiro.

Numa conta do plano gratuito, tudo isso sai dos créditos. Numa conta antiga dentro dos 12 meses, a instância e 750 horas de IPv4 público entram na franquia. De qualquer forma, o alarme de cobrança do capítulo 7 está de olho.

O código de criação está todo no repositório (`infra/ec2/user-data.sh` e os comandos acima), e o servidor vai mudar de rede no próximo capítulo. Por isso, a recomendação é destruir tudo ao terminar os testes. Dói um pouco desmontar o que levou um capítulo inteiro para ficar de pé, mas a velha máxima da área ajuda: servidor é gado, não bicho de estimação.

```
aws ec2 terminate-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-terminated --instance-ids "$INSTANCE_ID"
aws ec2 delete-security-group --group-id "$SG_ID"
```

O volume vai junto, por causa do `DeleteOnTermination`. A deploy key no GitHub também deve sair. A role `rango-ec2` e o statement de `PassRole` ficam, porque vão servir de novo.

Se preferir manter o servidor por uns dias, `aws ec2 stop-instances` interrompe a cobrança da instância e libera o IPv4 público. Ao ligar de novo, ela volta com outro IP.

Armadilhas e casos reais

> **Armadilha:** A porta 22 aberta para `0.0.0.0/0` é a regra mais comum em Security Groups de tutoriais, e também a mais varrida por robôs. Se em algum momento você realmente precisar de SSH, restrinja a origem ao seu IP com `/32` (o `checkip.amazonaws.com` do capítulo 1 diz qual é) e remova a regra ao terminar. Com o Session Manager, essa regra simplesmente deixa de existir.
>
> **Armadilha:** O IP público muda quando a instância é parada e ligada de novo. Quem colocou o IP num registro DNS ou num `VITE_API_URL` descobre isso no dia seguinte, normalmente às 8h, por uma mensagem que começa com "bom dia, o site caiu?". Um **Elastic IP** é um IPv4 público reservado para a conta, que você associa a uma instância e que sobrevive a stop e start. Ele custa os mesmos US$ 0,005 por hora, associado ou não. Um Elastic IP esquecido sem instância continua cobrando todo mês. Para o Rangô, a solução definitiva é um load balancer com um nome DNS, nos capítulos 13 e 14.

Armadilha: "Free tier" no tipo de instância não quer dizer servidor grátis. O IPv4 público cobra à parte (com franquia só no modelo antigo), o modo unlimited da família T cobra pelo excedente de CPU (US$ 0,05 por vCPU-hora nas T3 Linux, us-east-1, outubro de 2026), e um volume grande ou com IOPS extras cobra pelo que passar do incluso. O painel de free tier do console atualiza só três vezes por dia; ele não mostra o consumo em tempo real.

Armadilha: Editar o user data e reiniciar não reinstala nada, porque o script só roda no primeiro boot. Se você mudou o script, crie uma instância nova a partir dele. Isso tem um efeito colateral bom: força você a manter o script funcionando, em vez de acumular ajustes feitos à mão numa máquina que ninguém sabe recriar.

Armadilha: Uma `t3.micro` tem 1 GiB de memória e nenhuma partição de swap no AL2023. Um `pip install` pesado, um terceiro worker ou um vazamento de memória fazem o kernel matar processos com o OOM killer (um nome que deixa claro que ninguém ali está para conversa), e o `journalctl` mostra só que o serviço morreu. Se a memória apertar, suba para `t3.small`: pare a instância, troque o tipo com `aws ec2 modify-instance-attribute --instance-id "$INSTANCE_ID" --instance-type Value=t3.small` e ligue de novo. É a escala vertical do capítulo 4, com um minuto de indisponibilidade.

Na vida real: Em julho de 2019, o Capital One revelou um vazamento de dados de cerca de 100 milhões de clientes nos Estados Unidos e 6 milhões no Canadá. A invasora usou um firewall de aplicação mal configurado, rodando numa instância EC2, para fazer o servidor chamar o serviço de metadados em nome dela, uma técnica chamada SSRF (*server-side request forgery*). O serviço de metadados respondeu com as credenciais temporárias da role da instância, e com elas a invasora leu dados de buckets S3. Na versão 1 do serviço de metadados, uma simples requisição GET bastava para obter as credenciais. Em novembro de 2019, a AWS lançou o IMDSv2, que exige antes um PUT para obter um token de sessão, algo que a maioria das falhas de SSRF não consegue fazer. É por isso que o `run-instances` deste capítulo leva `HttpTokens=required`. O custo dessa escolha: bibliotecas muito antigas que só sabem falar IMDSv1 deixam de achar as credenciais. As versões atuais do boto3 e da AWS CLI usam o v2.

Resumo

- Uma instância EC2 é uma máquina virtual cobrada por segundo enquanto está ligada. O nome do tipo (`t3.micro`) diz família, geração, atributos e tamanho.

- A família T é burstable, com créditos de CPU. O modo unlimited, padrão nas T3, cobra pelo excedente; o standard só desacelera.
- O free tier depende da data de criação da conta: créditos no modelo novo, 12 meses no antigo.
- A AMI é o molde do sistema. Use o Amazon Linux 2023 pelo parâmetro público do SSM, nunca por um ID fixo. O AL2 chegou ao fim do suporte em 30 de junho de 2026.
- O Session Manager substitui o SSH com key pair: sem porta de entrada, sem chave para perder, com autorização pelo IAM. Exige SSM Agent, a policy `AmazonSSMManagedInstanceCore` e saída HTTPS.
- Passar uma role a uma instância exige `iam:PassRole`, que deve ser restrito à role certa e ao serviço certo.
- O Security Group só tem regras de permissão e é stateful. O `rango-web` libera só a porta 80.
- O user data roda uma vez, como root, no primeiro boot, e não deve conter segredos.
- O EBS é um disco de rede preso a uma zona de disponibilidade, cobrado pelo tamanho mesmo com a instância parada. `DeleteOnTermination` decide se ele morre junto.
- O Rangô rodou no EC2 com Valkey, Uvicorn e workers no systemd e o Nginx na frente, sobreviveu a um reboot e foi destruído no fim, com o script guardado em `infra/ec2/user-data.sh`.

O próximo passo, e o perigo que vem com ele

Os pedidos do Rangô ainda moram no Redis, um arranjo provisório desde o capítulo 5. O próximo passo óbvio é trazer o PostgreSQL para o servidor. O atalho também é óbvio: instalar o Postgres na mesma instância e, para conferir os dados do notebook com o seu cliente SQL favorito, acrescentar ao `rango-web` uma regra para a porta 5432 com origem `0.0.0.0/0`. "Só até terminar o teste." Famosas últimas palavras, logo depois de "em produção vai funcionar igual".

Antes disso, repare onde a instância está. A VPC default coloca tudo numa subnet pública, com IP público, alcançável por qualquer um. Não existe hoje nenhum lugar na sua conta onde um banco possa ficar fora do alcance da internet. Os robôs que varrem a porta 22 também varrem a 5432, e você já viu no capítulo 1 o que eles fizeram com milhares de bancos MongoDB abertos. No próximo capítulo, a rede do Rangô é desenhada do zero: uma VPC própria, subnets públicas e privadas, rotas, Internet Gateway e NAT Gateway, este último com um custo que pega muita gente de surpresa.

Referências

- Amazon EC2, tipos de instância: https://aws.amazon.com/ec2/instance-types/
- Amazon EC2, preços sob demanda e créditos de CPU: https://aws.amazon.com/ec2/pricing/on-demand/
- Free tier no EC2 (contas antes e depois de 15/07/2025): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-free-tier-usage.html
- Amazon Linux 2023 no EC2 (parâmetros SSM, AMIs, kernel): https://docs.aws.amazon.com/linux/al2023/ug/ec2.html
- Amazon Linux 2, perguntas frequentes e fim do suporte: https://aws.amazon.com/amazon-linux-2/faqs/
- Transição de Redis 6 para Valkey no AL2023: https://docs.aws.amazon.com/linux/al2023/ug/redis6-to-valkey-al2023.html
- SSM Agent no Amazon Linux 2 e 2023: https://docs.aws.amazon.com/systems-manager/latest/userguide/agent-install-al2.html
- Session Manager no EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/session-manager.html
- Instalação do Session Manager plugin: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager-working-with-install-plugin.html
- User data no EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html
- Instance metadata e IMDSv2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instancedata-data-retrieval.html
- Preços do Amazon EBS: https://aws.amazon.com/ebs/pricing/
- Cobrança de IPv4 público: https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/
- Free tier com 750 horas de IPv4 público: https://aws.amazon.com/about-aws/whats-new/2024/02/aws-free-tier-750-hours-free-public-ipv4-addresses/
- Lançamento do IMDSv2 (novembro de 2019): https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/
- Capital One, comunicado sobre o incidente de 2019: https://www.capitalone.com/digital/facts2019/
- GitHub, deploy keys: https://docs.github.com/en/authentication/connecting-to-github-with-ssh/managing-deploy-keys

- GitHub, impressões digitais das chaves SSH: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/githubs-ssh-key-fingerprints

CAPÍTULO 10

VPC: a rede na AWS

No capítulo anterior, o Rangô rodou numa instância EC2 de verdade, com Valkey, Uvicorn e workers no systemd e o Nginx na frente, e você entrou nela pelo Session Manager sem abrir a porta 22. Tudo isso aconteceu na VPC default, uma rede pronta em que toda subnet é pública. No fim, a instância foi destruída, e a role `rango-ec2` e o script `infra/ec2/user-data.sh` ficaram guardados. A ponte deixou um atalho perigoso no ar: instalar o PostgreSQL na mesma máquina e abrir a porta 5432 para o mundo, "só até terminar o teste". A frase tem o mesmo prazo de validade de "depois eu escrevo os testes".

A porta 5432 aberta para o mundo

Suponha que você tenha seguido o atalho. É uma situação hipotética, mas acontece todos os dias. Numa terça à noite, você instala o PostgreSQL na instância, cria uma senha razoável para o usuário `postgres`, acrescenta ao `rango-web` a regra da porta 5432 com origem `0.0.0.0/0` e conecta pelo DBeaver do notebook. Funciona. Você vai dormir o sono tranquilo de quem ainda não leu o log.

Na manhã seguinte, o log do Postgres tem isto:

```
2026-10-14 02:13:47.112 UTC [48213] FATAL:  password authentication failed for user
"postgres"
2026-10-14 02:13:47.894 UTC [48214] FATAL:  password authentication failed for user
"postgres"
2026-10-14 02:13:48.401 UTC [48215] FATAL:  no pg_hba.conf entry for host
"198.51.100.77", user "admin", database "postgres", SSL off
2026-10-14 02:13:49.020 UTC [48216] FATAL:  password authentication failed for user
"postgres"
```

São milhares de linhas, vindas de dezenas de IPs que você nunca viu. É o maior número de visitantes que o Rangô já teve, e nenhum deles quer pedir comida. Ninguém mirou no Rangô. Robôs varrem a faixa inteira de IPs públicos da AWS o tempo todo, com uma dedicação que nenhum funcionário seu teria num domingo, procurando portas conhecidas: 22 para SSH, 3306 para MySQL, 5432 para Postgres, 6379 para Redis, 27017 para MongoDB, 9200 para Elasticsearch. Mecanismos de busca como o Shodan

catalogam o que respondem. É uma espécie de Google das coisas que não deveriam estar na internet. Uma porta aberta numa instância nova costuma ser encontrada em minutos ou poucas horas, bem antes de qualquer cliente de verdade encontrar o seu site.

Desta vez a senha aguentou. Basta um usuário de teste com senha fraca, uma extensão com falha conhecida ou um Postgres que ficou dois anos sem atualização para a história terminar mal.

> **Na vida real:** Em julho de 2020, um ataque que ficou conhecido como "Meow" começou a apagar bancos Elasticsearch e MongoDB expostos na internet sem autenticação. O robô não pedia resgate nem deixava recado: só destruía os dados e escrevia a palavra "meow" no lugar, com a mesma tranquilidade de um gato que derruba o copo da mesa olhando nos seus olhos. Em poucos dias, o BleepingComputer contou quase 4 mil bancos apagados. Um dos primeiros casos públicos foi encontrado pelo pesquisador Bob Diachenko, num banco ligado ao serviço UFO VPN.

O problema de fundo vai além da regra do Security Group. Na VPC default, toda instância nasce numa subnet pública, com IP público. O Security Group é a única coisa entre o banco e a internet, e uma regra errada desfaz tudo. Não existe na sua conta nenhum lugar onde uma máquina fique fora do alcance da internet *por construção*, de forma que nem uma regra errada a exponha. É esse lugar que o capítulo constrói agora.

Uma rede só sua dentro da AWS

A **Amazon VPC** (*Virtual Private Cloud*) é uma rede privada isolada dentro de uma região da AWS. Você escolhe a faixa de IPs, divide em subnets, decide as rotas e decide o que conversa com a internet. É ser o administrador de rede da própria empresa sem precisar passar cabo por baixo do carpete. Fisicamente, as suas instâncias dividem hardware com as de milhares de outros clientes, como numa república em que cada morador tem a própria chave e ninguém ouve a música do vizinho. Logicamente, a VPC garante que nenhum pacote de outra rede chegue nelas, a menos que você monte o caminho.

Lembra da rede da sua casa, do capítulo 1? Ela tem uma faixa privada (`192.168.0.0/24`), um roteador que é o gateway padrão e um NAT que deixa os aparelhos saírem para a internet sem que ninguém de fora consiga iniciar uma conexão com eles. A VPC tem as mesmas peças, com nomes diferentes e uma vantagem: cada peça é um recurso que você cria por API e paga (ou não) separadamente. O "ou não" é a parte mais bonita desta frase.

A VPC em si não custa nada. Subnets, route tables, Internet Gateway, Security Groups e Network ACLs também não. A conta começa a subir com o NAT Gateway, os IPv4 públicos, os endpoints de interface e o tráfego. Guarde essa lista, porque ela vai decidir o que fica e o que vai embora no fim do capítulo.

Escolher a faixa antes de tudo

Toda VPC começa com um bloco CIDR IPv4. O tamanho permitido vai de `/16` (65.536 endereços) a `/28` (16 endereços). A AWS recomenda as faixas privadas da RFC 1918 que você conheceu no capítulo 1: `10.0.0.0/8`, `172.16.0.0/12` e `192.168.0.0/16`.

Essa escolha merece cinco minutos de conversa (em reunião de arquitetura, isso quer dizer uma hora e meia e um quadro branco cheio de setas), porque o tamanho de um bloco CIDR não muda depois de criado. Você pode associar blocos secundários à VPC (até cinco por padrão), mas o primário fica para sempre. E o critério mais importante é outro: a faixa não pode colidir com as redes com que a VPC um dia vai conversar. Conexões entre VPCs (*peering*) e com a rede de um escritório exigem faixas sem sobreposição. Se o Rangô um dia tiver uma VPC de produção, outra de homologação e uma VPN para o escritório, as três precisam de faixas diferentes.

Para o Rangô, a escolha é `10.20.0.0/16`, e cada descartada tem um motivo:

- `10.0.0.0/16` é a faixa de todos os tutoriais, o "senha123" das redes. Quanto mais gente usa, maior a chance de colidir com a VPC de um parceiro ou com a rede de uma empresa que você adquirir.
- `172.31.0.0/16` é a faixa da VPC default em todas as regiões.
- `172.17.0.0/16` é a rede *bridge* padrão do Docker. A documentação da AWS pede para evitá-la, porque serviços como o SageMaker AI usam essa faixa e o conflito de IPs aparece em lugares inesperados. Você já viu esse tipo de colisão com a VPN no capítulo 1.

Um `/16` é muito mais do que o Rangô precisa (65 mil endereços para um app de entrega de bairro é otimismo de fundador em estado puro), e é justamente essa sobra que permite fatiar a rede com folga.

Subnets: a rede fatiada por zona

Uma **subnet** é um pedaço do CIDR da VPC que vive inteiro dentro de uma única zona de disponibilidade. Essa amarração tem uma consequência direta: para ter alta disponibilidade, você precisa de subnets em pelo menos duas AZs. Ovos, cestas: você conhece o ditado. O RDS, no capítulo 12, exige um grupo de subnets em duas AZs, e o Application Load Balancer, no capítulo 13, também.

O capítulo 1 adiantou que a AWS reserva cinco endereços em toda subnet. Numa subnet `10.20.10.0/24`, são estes:

- `10.20.10.0`: o endereço da rede.
- `10.20.10.1`: o roteador da VPC.
- `10.20.10.2`: o servidor DNS da AWS (o resolver da VPC, que na prática responde em qualquer endereço base da VPC mais dois).
- `10.20.10.3`: reservado para uso futuro.
- `10.20.10.255`: o último endereço, reservado porque a VPC não usa broadcast.

O "reservado para uso futuro" é a gaveta de bagunça da cozinha da AWS: ninguém sabe o que vai parar ali, mas também ninguém mexe. Sobram 251 endereços utilizáveis por `/24`, mais do que suficiente para o Rangô por um bom tempo.

O plano tem duas camadas, cada uma em duas AZs. A numeração deixa buracos de propósito, para caber uma terceira AZ ou uma camada nova sem bagunçar a contagem, como quem deixa espaço no armário para as roupas que ainda vai comprar:

Subnet	CIDR	AZ	Para quê
`rango-publica-a`	`10.20.0.0/24`	us-east-1a	API, NAT, futuro load balancer
`rango-publica-b`	`10.20.1.0/24`	us-east-1b	idem, na outra AZ
`rango-privada-a`	`10.20.10.0/24`	us-east-1a	banco, workers, endpoints
`rango-privada-b`	`10.20.11.0/24`	us-east-1b	idem, na outra AZ

Repare que nada na criação de uma subnet diz se ela é pública ou privada. Essa diferença mora em outro lugar.

Route tables: quem decide o caminho

Cada subnet está associada a uma **route table**, uma tabela que diz para onde vai cada pacote conforme o destino. É o mesmo roteamento por *longest prefix match* do capítulo 1, aquele que lembra a escolha de rota do FastAPI: vence a regra mais específica.

Toda route table de uma VPC nasce com uma rota que você não pode apagar:

```
Destino          Alvo
10.20.0.0/16     local
```

Ela garante que qualquer máquina da VPC alcance qualquer outra. É uma das raras configurações à prova de estagiário em toda a AWS. A partir daí, você acrescenta rotas. Uma subnet sem associação explícita usa a *main route table* da VPC, que no início tem só

a rota `local`. É o padrão para quem não escolheu nada, como o toque de celular que veio de fábrica.

Internet Gateway: a porta da rua

O **Internet Gateway** é o componente que liga a VPC à internet. Você cria, anexa à VPC (uma VPC aceita um só; nesse ponto, ela é rigorosamente monogâmica) e ele passa a fazer duas coisas: encaminhar o tráfego entre a VPC e a internet e traduzir, um para um, o IP público de cada instância para o IP privado dela. É o NAT de um para um que você viu no capítulo 9, quando o `198.51.100.23` não aparecia no `ip addr` da instância.

Ele não cobra por hora e não tem limite de banda que você precise dimensionar, um caso raro de componente da AWS que não exige planilha. Anexar um Internet Gateway, porém, não muda nada sozinho. É como instalar uma porta na parede e esquecer de construir o corredor até ela. Nenhum pacote vai para ele até que uma route table aponte para lá.

O que torna uma subnet pública

Agora dá para responder à pergunta que ficou aberta. Uma subnet é **pública** quando a route table dela tem uma rota padrão para o Internet Gateway:

```
Destino          Alvo
10.20.0.0/16     local
0.0.0.0/0        igw-0a1b2c3d4e5f67890
```

Uma subnet **privada** é aquela que não tem essa rota. É só isso, uma linha a menos numa tabela, e o efeito é forte. Uma instância numa subnet privada pode até ganhar um Security Group que libera a porta 5432 para `0.0.0.0/0`, e mesmo assim nenhum pacote da internet chega até ela, porque simplesmente não existe caminho. O atacante pode bater o quanto quiser numa porta que não fica em rua nenhuma. Ela também não tem IP público, e a resposta de qualquer coisa que ela tentar enviar para fora morre na rota `local`.

Essa é a proteção por construção que faltava, e ela não depende de ninguém acertar a regra às duas da manhã. O Security Group continua sendo a regra fina; a subnet privada é a parede.

Para a instância numa subnet pública ser alcançável, ela ainda precisa de um IPv4 público. A subnet tem um atributo que atribui um IP público automaticamente a toda instância que nasce nela. Cada um desses IPs custa US$ 0,005 por hora desde fevereiro de 2024, como você viu nos capítulos anteriores.

NAT Gateway: sair sem ser encontrado

A subnet privada resolve a entrada, e cria um problema na saída. É um clássico da engenharia: todo problema resolvido costuma deixar um filhote. O banco do Rangô precisa baixar pacotes com o `dnf`. O SSM Agent precisa falar com o Systems Manager pela porta 443, senão você não entra na máquina. Um worker numa subnet privada vai precisar chamar o gateway de pagamento. Sem rota para fora, nada disso funciona.

A resposta é o mesmo truque do roteador de casa: um NAT que deixa sair e não deixa entrar. O **NAT Gateway** fica numa subnet pública, com um Elastic IP. A route table da subnet privada ganha a rota `0.0.0.0/0` apontando para ele. Quando o banco abre uma conexão para fora, o NAT Gateway troca o IP de origem pelo dele, guarda a conversão na tabela (o dicionário do capítulo 1) e devolve a resposta. Uma conexão iniciada de fora não encontra nenhuma entrada na tabela e é descartada.

O NAT Gateway é gerenciado: a AWS cuida da disponibilidade dentro da AZ e da escala até dezenas de Gbps, banda de sobra mesmo para uma noite de promoção de pizza no Rangô. E ele cobra de dois jeitos ao mesmo tempo. Em `us-east-1`, com preços de referência consultados em outubro de 2026:

- US$ 0,045 por hora em que existe, com ou sem tráfego. São cerca de US$ 32,85 por mês.
- US$ 0,045 por GB processado, em qualquer direção, independentemente da origem ou do destino.
- O Elastic IP dele, US$ 0,005 por hora, cerca de US$ 3,65 por mês.

Em `sa-east-1`, os valores são mais altos. Consulte a página de preços da VPC para a sua região antes de decidir.

Agora vem a parte que pega as pessoas. Um NAT Gateway vive numa AZ só. Se a AZ dele cair, as subnets privadas de todas as AZs que dependem dele ficam sem saída. O desenho recomendado para produção é um NAT Gateway por AZ, cada subnet privada apontando para o da própria zona. Com três AZs, são cerca de US$ 98 por mês só de horas, antes de passar um único byte. Para um projeto que fatura pouco, é muitas vezes a maior linha da fatura. Em muita startup pequena, o NAT Gateway é o funcionário mais bem pago da empresa, e ninguém se lembra de tê-lo contratado.

A alternativa de economizar com um único NAT para todas as AZs tem outro custo: o tráfego de uma subnet privada na AZ `b` atravessa até o NAT na AZ `a`, e tráfego entre AZs cobra cerca de US$ 0,01 por GB em cada direção.

Desde novembro de 2025, existe uma terceira opção, o **NAT Gateway regional**. A AWS ouviu as reclamações sobre a complexidade e respondeu do jeito de sempre: com mais uma opção. Um único gateway atende várias AZs, cresce e encolhe sozinho conforme

surgem recursos em novas zonas e nem precisa de subnet pública. A cobrança é por hora para cada AZ ativa, mais o mesmo valor por GB, então ela simplifica a operação sem necessariamente baratear. Um artigo do AWS re:Post observa que, quando aparece um recurso numa AZ nova, a expansão pode levar até cerca de uma hora, e nesse intervalo o tráfego sai por outra AZ. Este capítulo usa o NAT Gateway zonal, que é o modelo que a maior parte da documentação e dos exemplos ainda assume.

Security Groups e Network ACLs

Você conheceu o Security Group no capítulo 9 como um middleware de autorização na rede: só tem regras de permissão, é *stateful* (a resposta de uma conexão permitida volta sozinha) e fica preso à interface de rede da instância. Na VPC própria, ele ganha um recurso que vai mudar a forma como você escreve regras: a origem pode ser outro Security Group.

Em vez de "libere a porta 5432 para `10.20.0.0/24`", você escreve "libere a porta 5432 para quem estiver no grupo `rango-web`". A regra acompanha as instâncias. Quando o Auto Scaling do capítulo 13 criar uma API nova com outro IP, ela entra no grupo e o banco a aceita, sem ninguém editar faixa nenhuma. Acabou a era da planilha `ips-liberados-NAO-APAGAR.xlsx`. É a mesma ideia de checar `request.user.groups` em vez de uma lista de IDs de usuário.

A VPC tem uma segunda camada de filtro, as **Network ACLs** (NACLs). Elas funcionam no nível da subnet e se comportam de outro jeito:

	Security Group	Network ACL
Onde age	Interface de rede da instância	Subnet inteira
Regras	Só *allow*	*Allow* e *deny*
Estado	Stateful	Stateless
Avaliação	Todas as regras juntas	Em ordem numérica, a primeira que casar vence
Origem	CIDR, prefix list ou outro SG	Só CIDR
Padrão	Novo grupo: nada entra, tudo sai	NACL default da VPC: tudo entra, tudo sai

O detalhe que mais confunde é a palavra *stateless*. A NACL não lembra conexões; tem a memória proverbial de um peixinho dourado. Se ela libera a entrada na porta 443, a resposta sai para a porta efêmera do cliente, um número alto e aleatório, e precisa de uma regra de saída própria, em geral para as portas `1024-65535`. Esquecer essa regra produz o timeout mais difícil de diagnosticar da VPC, daqueles que fazem um dev sênior duvidar do próprio diploma, porque o Security Group está certo e o pacote chega.

Para o Rangô, a recomendação é deixar a NACL default como está, liberando tudo, e fazer o controle no Security Group. A NACL serve bem a um caso específico: bloquear uma faixa de IPs que está atacando, algo que o Security Group não sabe fazer, porque não tem regra de negação. Uma regra de número baixo com `deny` para `203.0.113.0/24` resolve na hora para a subnet inteira. É o botão "bloquear contato" da rede. Para filtrar ataques HTTP de forma mais inteligente, o capítulo 23 vai apresentar o AWS WAF.

VPC endpoints: falar com a AWS sem passar pela internet

Há uma pegadinha no desenho com NAT. O SSM Agent da instância privada fala com `ssm.us-east-1.amazonaws.com`. Um worker que lê fotos do S3 fala com `s3.us-east-1.amazonaws.com`. São serviços da própria AWS, na mesma região, e mesmo assim o tráfego sai pelo NAT Gateway, como se fosse para a internet, e paga US$ 0,045 por GB no caminho.

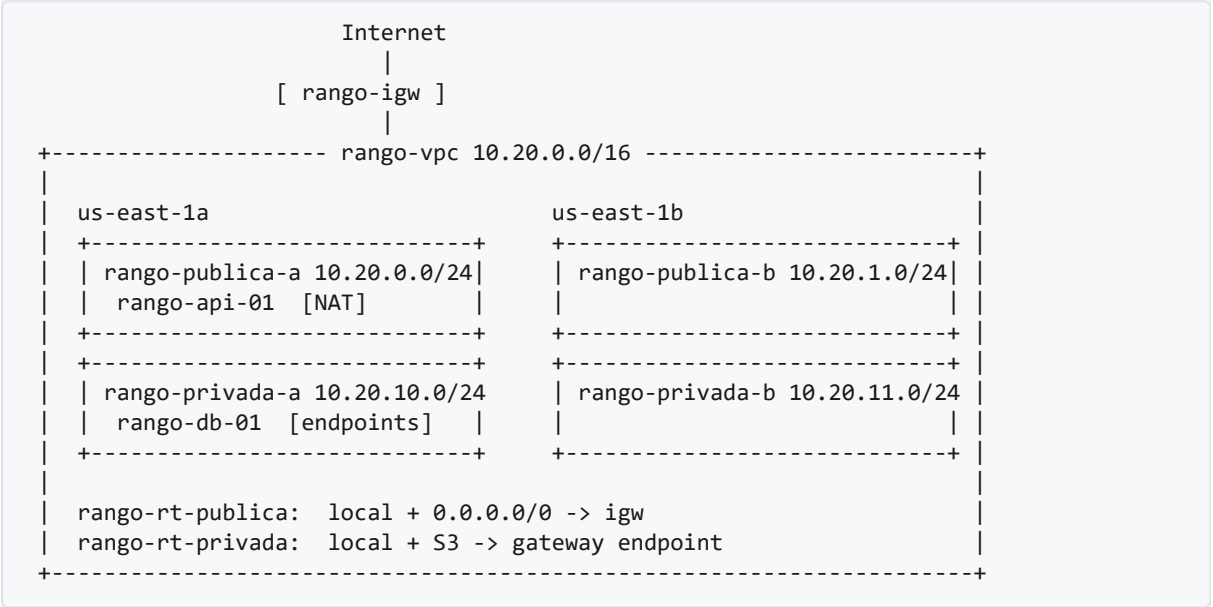
Os **VPC endpoints** criam um caminho privado da VPC até os serviços da AWS, um corredor de serviço nos fundos, longe da avenida movimentada da internet. Existem dois tipos, com custos bem diferentes:

- **Gateway endpoints**, só para S3 e DynamoDB. Viram uma rota na route table, apontando o prefixo do serviço para o endpoint. Não cobram nada, nem por hora nem por GB.
- **Interface endpoints**, para quase todos os outros serviços (SSM, Secrets Manager, ECR, CloudWatch e dezenas mais). Cada um cria uma interface de rede com IP privado em cada subnet que você escolher, e o DNS privado faz o nome do serviço resolver para esse IP. Cobram por hora por AZ (cerca de US$ 0,01) e por GB processado (US$ 0,01 nos primeiros petabytes), com preços de referência de outubro de 2026.

O gateway endpoint do S3 é a decisão mais fácil do capítulo, a única decisão de arquitetura em que todo mundo na reunião concorda de primeira: não custa nada e tira do NAT um tráfego que pode ser enorme. Os endpoints de interface pedem conta. Para o Session Manager funcionar sem nenhuma saída para a internet, a instância precisa de três: `ssm`, `ssmmessages` e `ec2messages`. Em duas AZs, são seis horas de endpoint por hora, cerca de US$ 44 por mês, mais caro que um NAT Gateway. Endpoints de interface compensam quando o volume de dados é alto ou quando a regra é que a subnet não pode ter saída nenhuma para a internet, como em ambientes regulados, onde a auditoria quer saber até para onde vai cada ping.

Na prática: a rede do Rangô

O plano é este: criar a `rango-vpc` com quatro subnets, o Internet Gateway e duas route tables; subir a API numa subnet pública e um PostgreSQL 17 numa privada; dar ao banco uma saída temporária por NAT Gateway para instalar os pacotes; e, por fim, trocar o NAT por endpoints, deixando o banco sem nenhum caminho para a internet. O desenho abaixo é ASCII, a técnica de ilustração preferida de quem tem medo de ferramenta de diagrama.



Pré-requisitos

- AWS CLI v2 com o perfil `rango-dev` dos capítulos anteriores, com login feito.
- Session Manager plugin instalado (capítulo 9).
- A role `rango-ec2` e o statement `PassarSoARoleDoServidor` no permission set, também do capítulo 9.
- O arquivo `infra/ec2/user-data.sh` no repositório.

Os comandos usam variáveis de shell para guardar IDs, então tudo roda no mesmo terminal. Se a sessão cair, os IDs voltam com `describe-*` filtrando pela tag `Name`. E ela vai cair, porque sessão de terminal sempre cai na hora exata em que você não anotou nada.

```
export AWS_PROFILE=rango-dev
export AWS_REGION=us-east-1
aws sso login
```

A VPC e as subnets

```
VPC_ID=$(aws ec2 create-vpc \
  --cidr-block 10.20.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=rango-vpc},
{Key=Projeto,Value=rango}]' \
  --query Vpc.VpcId --output text)

aws ec2 modify-vpc-attribute --vpc-id "$VPC_ID" \
  --enable-dns-hostnames '{"Value":true}'
```

O resolver DNS da VPC já vem ligado. O segundo comando liga os nomes DNS das instâncias, que os endpoints de interface com DNS privado exigem. Sem ele, a criação dos endpoints falha mais adiante, longe o bastante para ninguém mais lembrar da causa.

Para as quatro subnets, uma função curta evita repetir o mesmo comando quatro vezes, com a chance de errar o CIDR em uma delas e só descobrir no capítulo 12:

```
criar_subnet() {
  aws ec2 create-subnet --vpc-id "$VPC_ID" \
    --cidr-block "$2" --availability-zone "$3" \
    --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=$1},
{Key=Projeto,Value=rango}]" \
    --query Subnet.SubnetId --output text
}

PUB_A=$(criar_subnet rango-publica-a 10.20.0.0/24 us-east-1a)
PUB_B=$(criar_subnet rango-publica-b 10.20.1.0/24 us-east-1b)
PRIV_A=$(criar_subnet rango-privada-a 10.20.10.0/24 us-east-1a)
PRIV_B=$(criar_subnet rango-privada-b 10.20.11.0/24 us-east-1b)
```

A conferência do que existe fica assim:

```
aws ec2 describe-subnets --filters Name=vpc-id,Values="$VPC_ID" \
  --query "Subnets[].[Tags[?Key=='Name']|
[0].Value,CidrBlock,AvailabilityZone,AvailableIpAddressCount]" \
  --output table
```

```
-------------------------------------------------------------
|                     DescribeSubnets                       |
+------------------+----------------+-------------+-------+
|  rango-publica-a |  10.20.0.0/24  |  us-east-1a |  251  |
|  rango-publica-b |  10.20.1.0/24  |  us-east-1b |  251  |
|  rango-privada-a |  10.20.10.0/24 |  us-east-1a |  251  |
|  rango-privada-b |  10.20.11.0/24 |  us-east-1b |  251  |
+------------------+----------------+-------------+-------+
```

Os 251 da última coluna são os cinco reservados em ação. A ordem das linhas pode variar.

O Internet Gateway e as rotas

```
IGW_ID=$(aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=rango-igw},
```

```
{Key=Projeto,Value=rango}]' \
  --query InternetGateway.InternetGatewayId --output text)

aws ec2 attach-internet-gateway --internet-gateway-id "$IGW_ID" --vpc-id "$VPC_ID"
```

Agora a route table pública, com a rota padrão para o Internet Gateway, associada às duas subnets públicas:

```
RT_PUB=$(aws ec2 create-route-table --vpc-id "$VPC_ID" \
  --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=rango-rt-
publica},{Key=Projeto,Value=rango}]' \
  --query RouteTable.RouteTableId --output text)

aws ec2 create-route --route-table-id "$RT_PUB" \
  --destination-cidr-block 0.0.0.0/0 --gateway-id "$IGW_ID"

for SUBNET in "$PUB_A" "$PUB_B"; do
  aws ec2 associate-route-table --route-table-id "$RT_PUB" --subnet-id "$SUBNET"
  aws ec2 modify-subnet-attribute --subnet-id "$SUBNET" --map-public-ip-on-launch
done
```

O `--map-public-ip-on-launch` faz toda instância criada nessas subnets ganhar um IPv4 público. As privadas ficam sem esse atributo.

E a route table privada, por enquanto só com a rota `local`:

```
RT_PRIV=$(aws ec2 create-route-table --vpc-id "$VPC_ID" \
  --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=rango-rt-
privada},{Key=Projeto,Value=rango}]' \
  --query RouteTable.RouteTableId --output text)

for SUBNET in "$PRIV_A" "$PRIV_B"; do
  aws ec2 associate-route-table --route-table-id "$RT_PRIV" --subnet-id "$SUBNET"
done
```

Por que criar uma route table privada, se a main route table da VPC também só tem a rota `local`? Porque a associação explícita deixa a intenção registrada. É o comentário `# NÃO MEXA` da rede, com a vantagem de que o computador obedece. Se alguém um dia acrescentar uma rota para o Internet Gateway na main route table, as subnets privadas continuam privadas.

Já que a route table privada existe, ela recebe também o gateway endpoint do S3. Ele não cobra e vai servir às fotos dos pratos no próximo capítulo:

```
aws ec2 create-vpc-endpoint --vpc-id "$VPC_ID" \
  --vpc-endpoint-type Gateway \
  --service-name com.amazonaws.us-east-1.s3 \
  --route-table-ids "$RT_PRIV" \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[{Key=Name,Value=rango-s3},
{Key=Projeto,Value=rango}]' \
  --query VpcEndpoint.VpcEndpointId --output text
```

Os Security Groups

Três grupos: o da API, o do banco e o dos endpoints.

```
criar_sg() {
  aws ec2 create-security-group --vpc-id "$VPC_ID" \
    --group-name "$1" --description "$2" \
    --tag-specifications "ResourceType=security-group,Tags=[{Key=Name,Value=$1},
{Key=Projeto,Value=rango}]" \
    --query GroupId --output text
}

SG_WEB=$(criar_sg rango-web "HTTP publico da API do Rango")
SG_DB=$(criar_sg rango-db "Postgres do Rango, so a partir da API")
SG_ENDP=$(criar_sg rango-endpoints "HTTPS da VPC para os VPC endpoints")

aws ec2 authorize-security-group-ingress --group-id "$SG_WEB" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress --group-id "$SG_DB" \
  --protocol tcp --port 5432 --source-group "$SG_WEB"

aws ec2 authorize-security-group-ingress --group-id "$SG_ENDP" \
  --protocol tcp --port 443 --cidr 10.20.0.0/16
```

As descrições continuam sem acento, pelo mesmo `InvalidParameterValue` do capítulo 9. O circunflexo do Rangô segue banido da infraestrutura, exilado no front-end. Repare na regra do banco: nenhum CIDR, só o grupo `rango-web` como origem. O `rango-web` é um grupo novo, com outro ID, porque Security Groups pertencem a uma VPC e não atravessam para outra.

A API na subnet pública

O comando é o do capítulo 9 com dois acréscimos: `--subnet-id` e o grupo novo. Reaproveitar o comando do capítulo anterior é a forma mais pura de preguiça produtiva.

```
API_ID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
x86_64 \
  --instance-type t3.micro \
  --credit-specification CpuCredits=standard \
  --subnet-id "$PUB_A" \
  --security-group-ids "$SG_WEB" \
  --iam-instance-profile Name=rango-ec2 \
  --metadata-options HttpTokens=required,HttpEndpoint=enabled \
  --block-device-mappings '[{"DeviceName":"/dev/xvda","Ebs":
{"VolumeSize":10,"VolumeType":"gp3","Encrypted":true,"DeleteOnTermination":true}}]' \
  --user-data file://infra/ec2/user-data.sh \
  --tag-specifications \
    'ResourceType=instance,Tags=[{Key=Name,Value=rango-api-01},
{Key=Projeto,Value=rango}]' \
    'ResourceType=volume,Tags=[{Key=Projeto,Value=rango}]' \
  --query "Instances[0].InstanceId" --output text)
```

Para ter o Rangô completo rodando nela, valem os passos da seção "Trazendo o código" do capítulo 9; nada muda ali. Para este capítulo, basta a máquina.

O NAT Gateway, por uma hora

O banco precisa de saída para instalar o PostgreSQL e para o SSM Agent se registrar. O NAT Gateway nasce antes da instância, na subnet pública `a`:

```
EIP_ALLOC=$(aws ec2 allocate-address --domain vpc \
  --tag-specifications 'ResourceType=elastic-ip,Tags=[{Key=Name,Value=rango-nat-a},
{Key=Projeto,Value=rango}]' \
  --query AllocationId --output text)

NAT_ID=$(aws ec2 create-nat-gateway \
  --subnet-id "$PUB_A" --allocation-id "$EIP_ALLOC" \
  --tag-specifications 'ResourceType=natgateway,Tags=[{Key=Name,Value=rango-nat-a},
{Key=Projeto,Value=rango}]' \
  --query NatGateway.NatGatewayId --output text)

aws ec2 wait nat-gateway-available --nat-gateway-ids "$NAT_ID"

aws ec2 create-route --route-table-id "$RT_PRIV" \
  --destination-cidr-block 0.0.0.0/0 --nat-gateway-id "$NAT_ID"
```

A partir do `create-nat-gateway`, o relógio está correndo. O `wait` costuma levar de um a dois minutos, o tempo de esquentar um café no micro-ondas e esquecê-lo lá dentro.

O PostgreSQL na subnet privada

O script do banco fica em `infra/ec2/user-data-db.sh`:

```
#!/bin/bash
set -euxo pipefail

dnf install -y postgresql17-server
postgresql-setup --initdb
systemctl enable --now postgresql
```

O Amazon Linux 2023 tem o PostgreSQL 17 no repositório oficial, a mesma versão do `postgres:17-alpine` que o Compose usava no capítulo 6. A instância sobe na subnet privada:

```
DB_ID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
x86_64 \
  --instance-type t3.micro \
  --credit-specification CpuCredits=standard \
  --subnet-id "$PRIV_A" \
  --security-group-ids "$SG_DB" \
  --iam-instance-profile Name=rango-ec2 \
  --metadata-options HttpTokens=required,HttpEndpoint=enabled \
  --block-device-mappings '[{"DeviceName":"/dev/xvda","Ebs":
{"VolumeSize":10,"VolumeType":"gp3","Encrypted":true,"DeleteOnTermination":true}}]' \
```

```
  --user-data file://infra/ec2/user-data-db.sh \
  --tag-specifications \
    'ResourceType=instance,Tags=[{Key=Name,Value=rango-db-01},
{Key=Projeto,Value=rango}]' \
    'ResourceType=volume,Tags=[{Key=Projeto,Value=rango}]' \
  --query "Instances[0].InstanceId" --output text)

aws ec2 wait instance-running --instance-ids "$API_ID" "$DB_ID"

aws ec2 describe-instances --instance-ids "$API_ID" "$DB_ID" \
  --query "Reservations[].Instances[].[Tags[?Key=='Name']|
[0].Value,PublicIpAddress,PrivateIpAddress]" \
  --output text
```

```
rango-api-01    198.51.100.41   10.20.0.183
rango-db-01 None    10.20.10.57
```

O banco não tem IP público. Ele é o introvertido da VPC: conversa só com quem conhece e não aparece em lista nenhuma. Não existe endereço na internet que leve até ele, e nenhuma regra de Security Group muda isso.

Com o SSM Agent `Online`, como no capítulo 9, a sessão no banco abre assim:

```
aws ssm start-session --target "$DB_ID"
```

Dentro da sessão, um `curl` mostra por onde a instância sai para a internet:

```
curl -s https://checkip.amazonaws.com
```

```
203.0.113.80
```

Esse endereço é o Elastic IP do NAT Gateway. Para o mundo, todas as máquinas das subnets privadas aparecem com esse endereço (um único rosto público para a família inteira, como a conta compartilhada de streaming), como os aparelhos da sua casa aparecem com o IP do roteador.

Agora o Postgres precisa aceitar conexões da VPC. Ainda como `ssm-user`:

```
sudo -u postgres psql -c "ALTER SYSTEM SET listen_addresses = '*';"
sudo -u postgres psql -c "CREATE ROLE rango LOGIN;"
sudo -u postgres psql -c "CREATE DATABASE rango OWNER rango;"
sudo -u postgres psql -c "\password rango"
```

O `\password` pede a senha duas vezes, porque ninguém confia na própria digitação, e com razão. Tudo acontece no terminal, e ele grava só o hash. A senha não passa pelo histórico do shell nem por nenhum arquivo. Por enquanto, ela fica no seu gerenciador de senhas (e não num post-it colado no monitor, por mais tradicional que seja). O capítulo

16 tira as senhas dos arquivos e as leva para cofres gerenciados: o Parameter Store fica com a configuração, e o Secrets Manager, com a senha do banco, que passa a girar sozinha.

O `listen_addresses = '*'` parece assustador depois da cena do início do capítulo, mas aqui o `*` quer dizer "todas as interfaces da máquina", e a única interface tem o IP `10.20.10.57`, inalcançável de fora da VPC. Mesmo assim, a restrição se repete no `pg_hba.conf`, a camada de autorização do próprio Postgres:

```
echo "host    rango    rango    10.20.0.0/16    scram-sha-256" \
  | sudo tee -a /var/lib/pgsql/data/pg_hba.conf
sudo systemctl restart postgresql
```

Agora são três camadas, cada uma com uma função: a subnet privada impede que a internet chegue; o Security Group `rango-db` restringe a quem está no `rango-web`; o `pg_hba.conf` exige usuário, banco e senha. O Shrek explicaria melhor: ogros, cebolas e bancos de dados bem protegidos têm camadas.

O teste que importa

Outra sessão, desta vez na API, instala só o cliente do Postgres:

```
aws ssm start-session --target "$API_ID"
```

```
sudo dnf install -y postgresql17
psql "host=10.20.10.57 dbname=rango user=rango" -c "SELECT version();"
```

```
Password for user rango:
                   version
-------------------------------------------------
 PostgreSQL 17.6 on x86_64-amazon-linux-gnu, ...
(1 row)
```

A versão exata do patch vai variar. O importante é que a API, no grupo `rango-web`, chega ao banco. Uma conexão de banco que funciona de primeira merece, no mínimo, um print para a posteridade. E se ela sair do grupo? No terminal do notebook, o Security Group da API é trocado temporariamente por outro qualquer da VPC, como o `rango-endpoints`:

```
aws ec2 modify-instance-attribute --instance-id "$API_ID" --groups "$SG_ENDP"
```

Na sessão da API, o mesmo `psql` agora fica parado até desistir:

```
psql: error: connection to server at "10.20.10.57", port 5432 failed: Connection timed
out
    Is the server running on that host and accepting TCP/IP connections?
```

Um *timeout*: o pacote foi descartado em silêncio pelo Security Group do banco, como você aprendeu a distinguir no capítulo 1. O banco não diz "não"; ele finge que não ouviu, a forma mais antiga de recusa educada. Depois do teste, o grupo certo volta:

```
aws ec2 modify-instance-attribute --instance-id "$API_ID" --groups "$SG_WEB"
```

Trocando o NAT por endpoints

O banco já tem o que precisa instalado. A partir daqui, a única saída que ele usa é a do Session Manager. Em vez de pagar um NAT Gateway por mês para isso, entram os três endpoints de interface, só na subnet privada `a`, onde o banco está:

```
for SERVICO in ssm ssmmessages ec2messages; do
  aws ec2 create-vpc-endpoint --vpc-id "$VPC_ID" \
    --vpc-endpoint-type Interface \
    --service-name "com.amazonaws.us-east-1.$SERVICO" \
    --subnet-ids "$PRIV_A" \
    --security-group-ids "$SG_ENDP" \
    --private-dns-enabled \
    --tag-specifications "ResourceType=vpc-endpoint,Tags=
[{Key=Name,Value=rango-$SERVICO},{Key=Projeto,Value=rango}]" \
    --query VpcEndpoint.VpcEndpointId --output text
done
```

Quando os três estiverem `available` (o `aws ec2 describe-vpc-endpoints --filters Name=vpc-id,Values="$VPC_ID" --query "VpcEndpoints[].[ServiceName,State]" --output table` mostra o estado), a rota do NAT sai e o gateway é destruído:

```
aws ec2 delete-route --route-table-id "$RT_PRIV" --destination-cidr-block 0.0.0.0/0
aws ec2 delete-nat-gateway --nat-gateway-id "$NAT_ID"
aws ec2 wait nat-gateway-deleted --nat-gateway-ids "$NAT_ID"
aws ec2 release-address --allocation-id "$EIP_ALLOC"
```

O Elastic IP só pode ser liberado depois que o NAT Gateway termina de ser apagado; por isso o `wait` no meio. Não dá para devolver a chave antes de o inquilino sair. Esquecer o `release-address` deixa um IP cobrando US$ 3,65 por mês sem fazer nada.

Uma sessão nova no banco abre normalmente, agora pelos endpoints. O SSM Agent detecta a mudança sozinho, mas pode levar alguns minutos; se a instância aparecer `ConnectionLost`, um reboot acelera. O velho "já tentou desligar e ligar de novo?" continua invicto. Dentro da sessão, a tentativa de chegar à internet termina assim:

```
curl -s -m 5 https://checkip.amazonaws.com || echo "sem saida"
```

```
sem saida
```

Dentro da VPC, o nome do Systems Manager agora resolve para um IP privado:

```
getent hosts ssm.us-east-1.amazonaws.com
```

```
10.20.10.92     ssm.us-east-1.amazonaws.com
```

O mesmo nome devolve outro endereço conforme quem pergunta; o DNS também sabe ser diplomático.

O banco do Rangô não recebe conexões da internet e também não consegue abrir nenhuma. Virou um monge da VPC, com voto de silêncio e sem rede social. O que ele alcança é a API, o próprio S3 pelo gateway endpoint e o Systems Manager pelos endpoints de interface.

O preço dessa pureza aparece no primeiro `dnf upgrade`: sem NAT, não há atualização. Isolamento total tem disso: nem as boas notícias chegam. Você teria de recriar o NAT por uma hora a cada janela de manutenção, ou manter um. Esse trabalho de cuidar de um banco numa EC2, com atualizações, backups e discos, é o assunto do capítulo 12.

Quanto isso custa e o que destruir

Com preços de referência em `us-east-1`, outubro de 2026:

Recurso	Custo aproximado	Destino
VPC, subnets, route tables, IGW, SGs	Zero	Ficam
Gateway endpoint do S3	Zero	Fica
NAT Gateway + Elastic IP (uma hora)	~US$ 0,05	Já destruídos
3 endpoints de interface em 1 AZ	~US$ 0,03/h (~US$ 22/mês)	Destruir
2 instâncias `t3.micro` + 2 volumes de 10 GiB	Créditos ou free tier	Destruir
IPv4 público da API	US$ 0,005/h	Vai com a instância

A rede fica, porque não cobra nada e o RDS do capítulo 12 vai morar nas subnets privadas dela. O que cobra por hora vai embora:

```
aws ec2 terminate-instances --instance-ids "$API_ID" "$DB_ID"
aws ec2 wait instance-terminated --instance-ids "$API_ID" "$DB_ID"

for ENDP in $(aws ec2 describe-vpc-endpoints \
    --filters Name=vpc-id,Values="$VPC_ID" Name=vpc-endpoint-type,Values=Interface \
    --query "VpcEndpoints[].VpcEndpointId" --output text); do
  aws ec2 delete-vpc-endpoints --vpc-endpoint-ids "$ENDP"
done
```

Por fim, o console da VPC, em **NAT gateways** e em **Elastic IPs**, deve mostrar que nada sobrou. Um NAT apagado aparece como `Deleted` por um tempo e não cobra. Fica ali como um fantasma educado, que só assusta quem abre o console.

Armadilhas e casos reais

Armadilha: O NAT Gateway cobra por hora mesmo sem tráfego, e ninguém olha para ele depois de criado. Um ambiente de homologação com um NAT por AZ em três AZs custa perto de US$ 100 por mês só de existir. Em contas de estudo, crie o NAT quando precisar e apague ao terminar. Quando o capítulo 15 trouxer o AWS CDK, preste atenção: o construto `Vpc` cria, por padrão, um NAT Gateway por AZ. O parâmetro `nat_gateways` reduz esse número.

Armadilha: Tráfego para o S3 pelo NAT paga US$ 0,045 por GB. Numa situação hipotética, um worker numa subnet privada que reprocessa 5 TB de fotos por mês gera cerca de US$ 225 só de processamento no NAT. O gateway endpoint do S3 leva esse número a zero e se cria com um único comando. Não há motivo para uma VPC com subnets privadas ficar sem ele.

Armadilha: Uma faixa CIDR escolhida às pressas é para sempre. `10.0.0.0/16` em todas as VPCs da empresa parece inofensivo até o dia de conectar duas delas, quando o peering é recusado por sobreposição e a única saída é migrar uma rede inteira, tarefa que costuma ser estimada em duas semanas e entregue em dois trimestres. Anote as faixas de cada VPC numa planilha ou no README da infraestrutura, e nunca repita uma.

Armadilha: Esquecer as portas efêmeras numa NACL personalizada. Uma NACL nova nega tudo, nos dois sentidos, até receber regras. Liberar a entrada na 443 sem liberar a saída para `1024-65535` gera timeouts que parecem defeito do Security Group. Se você ainda não precisa de NACL, não mexa nela.

Armadilha: Instância "privada" numa subnet pública. Uma instância sem IP público numa subnet cuja route table aponta para o Internet Gateway fica inalcançável hoje, mas basta alguém associar um Elastic IP para ela aparecer na internet. Ela é privada do mesmo jeito que um diário com cadeado de plástico. Quem define se uma máquina é privada é a route table da subnet dela.

Na vida real: As cotas padrão por região surpreendem quem cria ambientes demais: 5 VPCs, 5 Elastic IPs e 5 NAT Gateways por AZ. Por VPC, são 200 subnets e 60 regras de entrada por Security Group. Todas podem ser aumentadas pelo Service

> Quotas, mas o pedido leva tempo. Descobrir o limite de Elastic IPs no meio de um deploy é uma forma desagradável de aprender.

Resumo

- A VPC é uma rede privada isolada numa região. Ela, as subnets, as route tables, o Internet Gateway, os Security Groups e as NACLs não cobram nada.
- O CIDR da VPC vai de `/16` a `/28`, não muda de tamanho e não deve se sobrepor a outras redes com que ela vá conversar. O Rangô usa `10.20.0.0/16`, longe de `10.0.0.0/16`, `172.31.0.0/16` e `172.17.0.0/16`.
- Cada subnet vive numa AZ e perde cinco endereços para a AWS. Alta disponibilidade exige subnets em pelo menos duas AZs.
- Uma subnet é pública quando a route table dela tem `0.0.0.0/0` para o Internet Gateway. Sem essa rota, ela é privada, e nada da internet chega, qualquer que seja o Security Group.
- O NAT Gateway dá saída às subnets privadas sem permitir entrada. Cobra por hora e por GB, vive numa AZ e, em produção, costuma ser um por AZ. O NAT Gateway regional, de novembro de 2025, cobre várias AZs com um único recurso.
- Security Groups são stateful, só permitem e podem usar outro grupo como origem. NACLs são stateless, aceitam negação, valem para a subnet inteira e exigem regras para as portas efêmeras.
- Gateway endpoints (S3, DynamoDB) são gratuitos. Endpoints de interface cobram por hora por AZ e por GB, e o Session Manager sem internet precisa de três.
- O Rangô ganhou a `rango-vpc`, com duas subnets públicas e duas privadas, o banco ficou atrás de três camadas (subnet, Security Group e `pg_hba.conf`), e só a rede, que é gratuita, continua de pé.

As fotos que não cabem no disco

Com a rede pronta, o Rangô voltou a crescer. Os restaurantes querem mostrar os pratos, e a primeira versão do upload salva as fotos em `/opt/rango/fotos`, no disco da instância. A Cantina da Nona sozinha mandou 340 fotos tiradas no celular, de 4 MB cada. O volume de 10 GiB começa a encher, e em breve o `df -h` vai mostrar 100%, a porcentagem favorita de nenhum administrador de sistemas.

Aumentar o disco adia o problema, como comprar um armário maior para quem não joga nada fora. Mas o capítulo 4 já avisou: quando a API tiver três instâncias atrás de um load balancer, uma foto enviada para a primeira não existe na segunda. E toda foto que o

React exibe passa pelo Uvicorn, ocupando um processo Python para servir bytes estáticos, um trabalho que ele faz com o entusiasmo de quem carrega caixa de mudança. No próximo capítulo, as fotos saem do servidor e vão para o Amazon S3, com buckets, classes de armazenamento e uploads que vão direto do navegador para a AWS sem passar pela API. Você também vai ver por que tantas empresas acabaram nas manchetes por causa de um bucket esquecido aberto.

Referências

- Amazon VPC, guia do usuário: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html
- Blocos CIDR de VPC (tamanhos, faixas recomendadas e restrições): https://docs.aws.amazon.com/vpc/latest/userguide/vpc-cidr-blocks.html
- Cotas da Amazon VPC: https://docs.aws.amazon.com/vpc/latest/userguide/amazon-vpc-limits.html
- Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html
- Preços da Amazon VPC (NAT Gateway, IPv4 público, tráfego): https://aws.amazon.com/vpc/pricing/
- Preços do AWS PrivateLink (endpoints de interface): https://aws.amazon.com/privatelink/pricing/
- Lançamento do NAT Gateway regional (novembro de 2025): https://aws.amazon.com/about-aws/whats-new/2025/11/aws-nat-gateway-regional-availability
- AWS CLI, `create-nat-gateway`: https://docs.aws.amazon.com/cli/latest/reference/ec2/create-nat-gateway.html
- AWS re:Post, NAT Gateway regional: https://www.repost.aws/articles/AR77CnVH2zR6KjoD4SvgNVAQ/understanding-amazon-vpc-regional-nat-gateway
- AWS re:Post, endpoints de VPC para o Systems Manager: https://www.repost.aws/knowledge-center/systems-manager-vpc-connectivity
- Cobrança de IPv4 público: https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/
- AWS CDK v2, `VpcProps` (NAT por AZ): https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_ec2/VpcProps.html
- PostgreSQL 17 no Amazon Linux 2023 (pacote `postgresql17`): https://computingforgeeks.com/how-to-install-postgresql-17-on-amazon-linux/

- PostgreSQL, `pg_hba.conf`: https://www.postgresql.org/docs/17/auth-pg-hba-conf.html
- Ataque "Meow" (BleepingComputer): https://www.bleepingcomputer.com/news/security/new-meow-attack-has-deleted-almost-4-000-unsecured-databases/

CAPÍTULO 11

S3: armazenamento de objetos

No capítulo anterior, o Rangô ganhou uma rede própria, a `rango-vpc`, com subnets públicas para o que precisa receber tráfego da internet e subnets privadas para o banco. Testamos o desenho com uma API e um Postgres em instâncias separadas, destruímos as máquinas e o NAT Gateway, e deixamos de pé só o que é gratuito. Entre essas peças ficou um gateway endpoint do S3 na route table privada, criado de propósito para este capítulo. Ele ficou lá, quieto, como aquele cabo HDMI que você guarda há dez anos porque um dia vai precisar. O dia chegou.

Trezentas e quarenta fotos num disco de 10 GiB

A primeira versão do upload de fotos era a mais óbvia possível. O formulário do painel do restaurante enviava o arquivo num `multipart/form-data`, a rota do FastAPI recebia o `UploadFile` e gravava em `/opt/rango/fotos/cantina-da-nona/lasanha.jpg`. Uma rota estática devolvia a imagem quando o React pedia. Funcionou no primeiro dia, com três fotos de teste. Toda arquitetura ruim funciona no primeiro dia; é assim que ela consegue o emprego.

Aí a Cantina da Nona mandou 340 fotos tiradas no celular, de 4 MB cada. A Nona, pelo visto, fotografou cada lasanha de três ângulos, como perito documentando cena de crime. São cerca de 1,3 GB de um único restaurante, num volume EBS de 10 GiB que já carrega o sistema operacional, o Python, o código e os logs. Mais seis restaurantes com o mesmo entusiasmo e o `df -h` mostra 100%. Quando o disco enche, a foto deixa de ser o problema principal: o Valkey não consegue gravar, o journald para de registrar e o Nginx começa a devolver erro 500 sem deixar pista no log. Disco cheio é o único bug que derruba tudo e ainda apaga as provas, como o mordomo dos romances policiais.

Aumentar o volume adiaria o problema, e os outros são piores:

- Quando a API tiver três instâncias atrás de um load balancer (o capítulo 4 já avisou), a foto enviada para a instância A não existe nas instâncias B e C. Dois terços das vezes, o prato aparece sem imagem, e o cliente pede a lasanha por fé.
- Cada foto exibida ocupa um processo Python servindo bytes que não mudam. Com dois workers do Uvicorn, uma página com 20 pratos disputa os mesmos processos

que estão tentando confirmar pedidos.

- O upload também passa pela API. Um dono de restaurante num 4G fraco leva 40 segundos para enviar 4 MB, e durante esses 40 segundos uma conexão fica presa no seu servidor, segurando a porta para alguém que ainda está no estacionamento.
- Se a instância for destruída, as fotos vão junto, a menos que alguém se lembre de fazer snapshot do volume. "Alguém se lembrar" é a estratégia de backup mais popular do mundo e a que menos funciona.

O disco local da instância é o lugar errado para guardar arquivos que os usuários enviam. Precisamos de um armazenamento que fique fora de qualquer servidor, que todas as instâncias enxerguem igual, que cresça sem ninguém redimensionar nada e que o navegador consiga acessar sem passar pelo Python. Ou seja, um lugar onde a Nona possa fotografar à vontade.

Um dicionário gigante acessado por HTTP

O Amazon S3 (Simple Storage Service) foi um dos primeiros serviços da AWS, lançado em 2006 (o que, em anos de nuvem, faz dele o avô que ainda bate ponto), e resolve exatamente isso. A forma mais fácil de entendê-lo é esquecer a ideia de disco e pensar num dicionário Python muito grande, acessado por HTTP:

```
# O modelo mental do S3
bucket = {
    "fotos/cantina-da-nona/lasanha.jpg": b"\xff\xd8\xff\xe0...",
    "fotos/sabor-do-sertao/baiao.jpg": b"\xff\xd8\xff\xe0...",
}
```

O *bucket* é o dicionário. Cada entrada é um **objeto**: os bytes do arquivo mais alguns metadados (tipo de conteúdo, tamanho, data, cabeçalhos que você definir). A chave do dicionário se chama, sem surpresa, **key**. A AWS reserva a criatividade para os nomes dos serviços, não para os conceitos. Você grava com `PUT`, lê com `GET`, apaga com `DELETE` e lista as chaves com um prefixo. Quase não há mais operações do que isso. É a API enxuta que todo backend promete ser na primeira sprint e esquece na terceira.

Repare que a key tem barras, mas o S3 não tem pastas. `fotos/cantina-da-nona/lasanha.jpg` é uma string única. O console mostra "pastas" porque agrupa as chaves pelo caractere `/`, por conveniência. Na prática, isso significa que não existe "renomear pasta": mudar um prefixo é copiar cada objeto para uma chave nova e apagar a antiga. Quem já tentou "só renomear a pasta" com dois milhões de objetos costuma contar a história com o olhar perdido. Também não existe editar um pedaço do arquivo. Você substitui o objeto inteiro ou não mexe nele.

Essa simplicidade tem contrapartidas que interessam ao Rangô:

- **Capacidade**: o bucket não tem limite de tamanho nem de número de objetos. Você não provisiona espaço e paga pelo que está guardado, por GB por mês.
- **Durabilidade**: na classe padrão, cada objeto é gravado em dispositivos de pelo menos três zonas de disponibilidade da região. A AWS projeta essa classe para 99,999999999% de durabilidade (os famosos "onze noves") em um ano. Perder um arquivo por falha de hardware deixa de ser uma preocupação sua. Apagá-lo por engano continua sendo, e voltamos a isso no versionamento. Contra o dedo humano, nenhuma quantidade de noves ajuda.
- **Acesso por HTTP**: qualquer cliente que fale HTTPS e saiba assinar uma requisição consegue ler e gravar. Isso inclui a CLI, o boto3 e, com a autorização certa, o navegador do seu usuário.
- **Consistência**: desde dezembro de 2020, o S3 tem consistência forte de leitura após escrita. Se o `PUT` retornou 200, o próximo `GET` de qualquer lugar já devolve a versão nova. Antes disso, havia casos em que o S3 se comportava como aquele colega que confirma o recebimento do e-mail e só lê na semana seguinte.

O nome do bucket tem uma regra que pega todo mundo na primeira vez: ele é único no mundo inteiro, entre todas as contas da AWS. Se alguém já criou `fotos`, você não cria. É como tentar registrar um e-mail só com o seu primeiro nome hoje em dia. Por isso o livro usa `rango-fotos-pratos`, o nome que apareceu na policy do capítulo 8, e você vai precisar acrescentar um sufixo seu, como o número da conta. O nome também vira parte da URL (`https://rango-fotos-pratos.s3.amazonaws.com/...`), então não coloque nele nada que você não queira ver num log de terceiros.

Cada conta pode ter até 10.000 buckets de uso geral por padrão, valor que subiu de 100 em novembro de 2024 e pode chegar a 1 milhão pelo Service Quotas. Os primeiros 2.000 são gratuitos. Para o Rangô, um bucket basta. Se um dia você precisar de um milhão de buckets, este livro já não tem muito a lhe ensinar.

Quanto custa guardar as fotos

Pelos valores de referência que encontramos em outubro de 2026 para `us-east-1` (vindos de levantamentos de terceiros, porque a tabela oficial não abriu na consulta), a classe padrão custa cerca de US$ 0,023 por GB por mês. As 340 fotos da Cantina da Nona ocupam 1,3 GB, ou uns 3 centavos de dólar por mês. A Nona gastou mais com o papel-toalha que usou para limpar a lente do celular. As requisições também são cobradas: cerca de US$ 0,005 por mil `PUT` e US$ 0,0004 por mil `GET`. Confira os números atuais na página de preços do S3 ou no AWS Pricing Calculator antes de fazer qualquer projeção.

O item que costuma surpreender é a transferência para a internet. O S3 funciona como estacionamento de shopping: entrar é tranquilo, o que custa é sair. Os primeiros 100 GB por mês são gratuitos, somando todos os serviços e regiões da conta. Depois disso, cada GB que sai da AWS para o navegador de alguém é cobrado. Uma foto de 4 MB vista 50 mil vezes por mês já são 200 GB. O capítulo 14 coloca o CloudFront na frente do bucket, e o capítulo 24 volta ao assunto de tráfego de saída.

Classes de armazenamento: a despensa e o depósito

Nem todo objeto é lido com a mesma frequência. A foto do prato do dia é lida milhares de vezes. A foto de um prato que saiu do cardápio há dois anos talvez nunca mais seja aberta. O S3 permite pagar menos pelo armazenamento de quem é pouco lido, em troca de cobrar mais quando alguém lê. Pense numa cozinha: o que se usa todo dia fica na bancada, o que se usa uma vez por mês vai para a despensa, e a louça de Natal vai para o depósito no fundo do quintal.

Classe	Para quê	Duração mínima cobrada	Leitura
S3 Standard	Dados lidos com frequência	Nenhuma	Imediata, sem taxa por GB lido
S3 Intelligent-Tiering	Padrão de acesso desconhecido	Nenhuma	Imediata nos níveis frequentes; taxa mensal de monitoramento por objeto
S3 Standard-IA	Lido raramente, mas precisa estar disponível na hora	30 dias	Imediata, com taxa por GB lido
S3 One Zone-IA	Igual ao anterior, em uma só AZ	30 dias	Imediata, com taxa por GB lido
S3 Glacier Instant Retrieval	Arquivo lido poucas vezes por ano	90 dias	Imediata, taxa por GB maior
S3 Glacier Flexible Retrieval	Arquivo que pode esperar minutos ou horas	90 dias	Exige restauração antes
S3 Glacier Deep Archive	Retenção longa, quase nunca lida	180 dias	Restauração em horas

Duas letras miúdas dessa tabela mudam a conta. A primeira é a duração mínima: se você mover um objeto para Glacier Instant Retrieval e apagá-lo no dia 10, paga os 90 dias do mesmo jeito, como quem paga o pacote de três meses do curso de inglês e vai a duas aulas. A segunda é o tamanho mínimo: Standard-IA, One Zone-IA e Glacier Instant Retrieval cobram cada objeto como se tivesse pelo menos 128 KB. Uma miniatura de 20 KB nessas classes custa como uma de 128 KB. Nas duas classes de arquivo com restauração, cada objeto ainda carrega uns 40 KB extras de metadados cobrados.

Como ordem de grandeza (valores de terceiros, outubro de 2026), Standard-IA fica em torno de US$ 0,0125 por GB por mês e Deep Archive perto de US$ 0,001: o depósito sai mais de vinte vezes mais barato que a bancada. Em compensação, buscar a louça de Natal lá no fundo leva horas e envolve uma escada.

One Zone-IA guarda o objeto numa só AZ, então use apenas para dados que você consegue gerar de novo, como miniaturas. Intelligent-Tiering serve para quando você não sabe o padrão de acesso: o S3 move cada objeto sozinho entre níveis, sem cobrar pela leitura, mas cobra uma taxa mensal de monitoramento por objeto.

Para o Rangô de hoje, tudo fica em Standard. Otimizar custo de armazenamento de 1,3 GB seria como contratar um consultor para reorganizar uma gaveta de talheres. O volume é pequeno e as fotos são lidas o tempo todo. As outras classes vão importar quando os backups do Postgres começarem a ocupar terabytes.

Lifecycle: regras que trabalham enquanto você dorme

Mover objetos de classe à mão não escala. O S3 tem *lifecycle rules*: regras declarativas no bucket que dizem o que fazer com os objetos conforme a idade deles. Lembra um job agendado de limpeza que você não escreve nem monitora. Talvez seja o único cron da história que nunca falhou em silêncio numa segunda-feira. Cada regra tem um filtro (prefixo, tags ou tamanho) e ações, como mudar `backups/` para Glacier Flexible Retrieval após 30 dias ou apagar `tmp/` após 1 dia.

Há duas restrições que evitam frustração. Um objeto precisa ter pelo menos 30 dias para ser transicionado para Standard-IA ou One Zone-IA. E, desde setembro de 2024, as regras novas, por padrão, não transicionam objetos menores que 128 KB para nenhuma classe, porque o custo da requisição de transição seria maior que a economia. Se você quiser mudar isso, existe o filtro `ObjectSizeGreaterThan` (e o `ObjectSizeLessThan`). Regras criadas antes de setembro de 2024 mantêm o comportamento antigo até serem editadas. É o tipo de detalhe que ninguém lê no anúncio e todo mundo descobre do pior jeito.

Versionamento: o `git log` do bucket

Lembra que um `PUT` na mesma key substitui o objeto? Com o bucket no modo padrão, substituir é apagar. Um bug que grava a foto da lasanha na key da foto do baião destrói o baião para sempre. Um `aws s3 rm --recursive` digitado no terminal errado leva tudo. Quem nunca rodou um comando destrutivo no terminal de produção achando que era o de teste só não teve tempo de carreira suficiente.

O versionamento muda esse comportamento. Com ele ligado, cada `PUT` cria uma nova versão do objeto e guarda as anteriores, cada uma com um `VersionId`. Um `DELETE` comum não apaga nada: ele coloca um **delete marker** por cima, uma espécie de lápide que faz o objeto parecer inexistente. Remover o delete marker traz o objeto de volta. Para apagar de verdade, você precisa pedir a exclusão de uma versão específica.

Se você pensou em `git`, acertou: cada versão é um commit, o delete marker é um commit que remove o arquivo, e restaurar é fazer checkout de uma versão anterior. A diferença é que o S3 cobra armazenamento por todas essas versões. Sobrescrever uma foto de 4 MB dez vezes deixa 40 MB guardados. É um repositório git que nunca roda `gc` e manda boleto.

Daí a combinação clássica: versionamento ligado mais uma lifecycle rule que apaga versões antigas (as *noncurrent versions*) depois de alguns dias. Você ganha uma janela para desfazer erros sem pagar para sempre por eles.

Uma vez ligado, o versionamento não pode ser desligado, apenas suspenso. É como foto antiga na internet: dá para esconder, sumir não.

Quem pode ler o bucket

Até aqui falamos de onde os bytes ficam. A parte que derrubou empresas inteiras é outra: quem tem permissão para lê-los. Nenhum diretor jurídico jamais perdeu o sono por causa de onze noves.

O capítulo 8 mostrou que a AWS avalia permissões juntando dois tipos de policy. A **identity-based policy** fica presa a quem pede (um usuário, uma role). A **resource-based policy** fica presa ao recurso. No S3, essa segunda é a **bucket policy**: um JSON no próprio bucket, com o mesmo formato que você já conhece, mais o campo `Principal` dizendo a quem a regra se aplica. Na mesma conta, um `Allow` em qualquer uma das duas basta, e um `Deny` explícito em qualquer uma das duas vence.

O problema histórico está no `Principal`. Uma bucket policy com `"Principal": "*"` e `"Action": "s3:GetObject"` significa "qualquer pessoa na internet pode ler qualquer objeto". Muitas vezes é alguém tentando fazer uma imagem aparecer no front e copiando a primeira resposta que encontrou, aquela com 400 votos e data de 2014. Havia ainda as ACLs, permissões por objeto anteriores ao IAM, com "AllUsers: READ" a um clique de distância no console. Dezenas de empresas foram parar nas manchetes por um bucket aberto para "facilitar" uma integração e nunca mais fechado.

A AWS respondeu em camadas. A primeira foi o **S3 Block Public Access**, uma trava acima de todas as policies e ACLs, com quatro configurações:

Configuração	O que faz
`BlockPublicAcls`	Rejeita qualquer requisição que tente criar uma ACL pública
`IgnorePublicAcls`	Ignora ACLs públicas que já existam
`BlockPublicPolicy`	Rejeita bucket policies que concedam acesso público
`RestrictPublicBuckets`	Se uma policy pública já existir, restringe o acesso a principals da AWS e da própria conta

O Block Public Access pode ser ligado em cada bucket e também na conta inteira. Ligado na conta, nenhum bucket fica público, nem que alguém escreva `"Principal": "*"` por engano, mesmo que esse alguém seja você às seis da tarde de uma sexta-feira.

A segunda camada veio como padrão. Desde abril de 2023, todo bucket novo nasce com as quatro configurações do Block Public Access ligadas e com as ACLs desabilitadas (a configuração *Object Ownership* fica em `BucketOwnerEnforced`: tudo que entra no bucket pertence à conta dona dele, e qualquer tentativa de definir uma ACL falha). Isso vale para buckets criados pelo console, pela CLI, pelo SDK ou pelo CloudFormation, em todas as regiões. Buckets antigos não mudaram, então uma conta com histórico pode ter buckets criados em 2019 com ACLs ativas, guardados como sítio arqueológico com risco de desabamento.

Uma terceira proteção, menos ligada à exposição, também virou padrão: desde 5 de janeiro de 2023, todo objeto novo é criptografado em repouso com SSE-S3 (AES-256, com chaves gerenciadas pela AWS), sem custo. Criptografia em repouso protege contra um disco roubado do data center. Ela não protege contra uma policy aberta: o S3 descriptografa com prazer para quem tem permissão de leitura. É um cofre blindado com um atendente muito simpático na frente. As chaves gerenciadas por você, com o KMS, ficam para o capítulo 16.

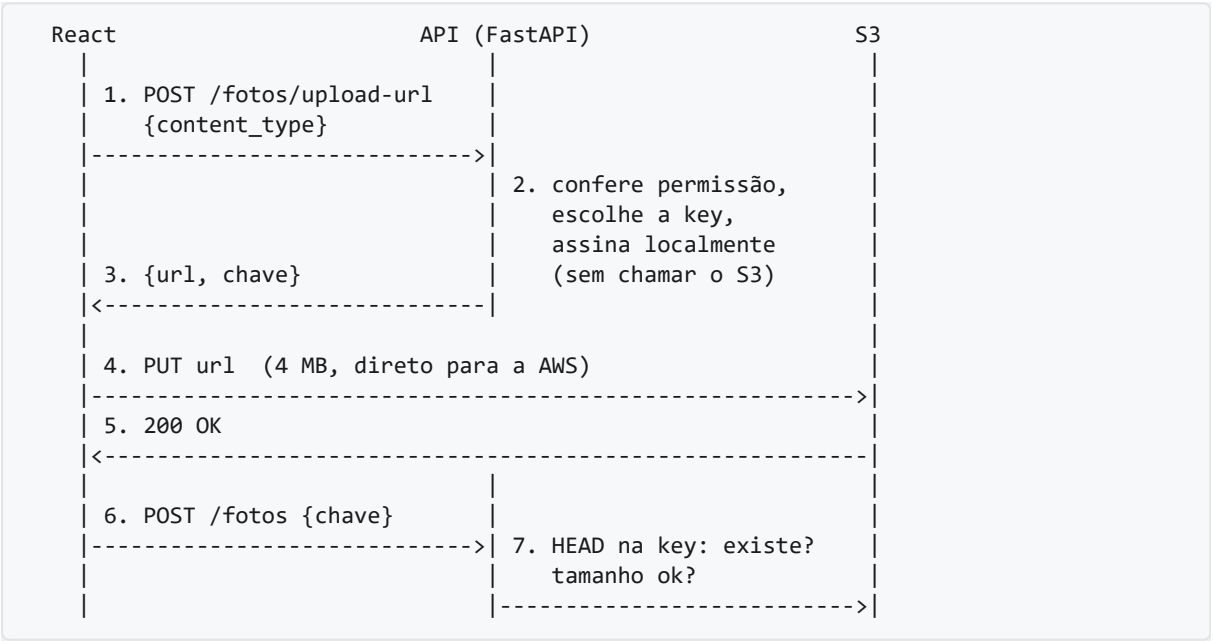
O bucket do Rangô nasce fechado, e vai continuar assim. Só que, fechado, o React não consegue mostrar nem enviar fotos, e mandar tudo pela API seria voltar ao começo do capítulo. Existe uma terceira saída.

Presigned URLs: um ingresso com hora marcada

Uma *presigned URL* é uma URL do S3 que já carrega a assinatura de uma requisição específica. Quem gera é alguém com permissão (a API do Rangô, usando a role dela). Quem usa pode ser qualquer pessoa, sem credencial nenhuma. A URL diz exatamente o que pode ser feito (método, bucket, key, às vezes o `Content-Type`), até quando, e em nome de quem.

A analogia é um ingresso de show com assento marcado e horário. A bilheteria (a API) confere se você pode entrar e emite o ingresso. Na porta (o S3), ninguém pergunta seu nome, só confere o ingresso. Se você tentar entrar em outra sessão ou sentar em outro lugar, a assinatura não bate e a porta fecha. E, ao contrário dos shows de verdade, não tem fila de quatro horas nem taxa de conveniência.

O fluxo de upload direto do navegador fica assim:



Repare no passo 2: gerar a URL é um cálculo local com a credencial da API, sem chamada ao S3. Os 4 MB vão do celular direto para a AWS, e o Uvicorn só cuida de duas requisições pequenas de JSON. Pela primeira vez na vida, o Python fica com a parte leve do serviço.

Agora as letras miúdas, todas importantes:

Validade. Uma URL assinada com credenciais de um usuário IAM pode valer até 7 dias. Só que o Rangô não usa usuário IAM (o capítulo 8 cuidou disso), e aqui mora a surpresa: uma URL assinada com credenciais temporárias expira quando a credencial expira, mesmo que você tenha pedido mais tempo. Na EC2, as credenciais da role (o instance profile) costumam durar umas 6 horas. Numa sessão com `AssumeRole`, o padrão é 1 hora. No ECS, as credenciais são trocadas a cada 1 a 6 horas. Na sua máquina, com o SSO do capítulo 8, a URL morre junto com a sessão. Peça sempre expirações curtas, e você nem chega perto desse limite.

Reuso. A URL pode ser usada quantas vezes alguém quiser até expirar. Um `PUT` repetido na mesma key substitui o objeto. Por isso a key é sempre gerada pela API com um UUID, e nunca escolhida pelo navegador. Deixar o cliente escolher a key é como deixar o hóspede escolher o número do quarto, inclusive os que já estão ocupados.

Portador. A presigned URL funciona como um bearer token. Quem tiver a URL tem o acesso, então ela não deve ir para logs, analytics ou mensagens de erro detalhadas. Colar uma dessas no canal geral do time equivale a ler a senha do cofre no microfone da festa da firma.

O boto3 também oferece a presigned POST (`generate_presigned_post`), pensada para formulários HTML. No Rangô vamos de presigned PUT, mais simples com `fetch`, e conferimos o tamanho depois do upload. Formulário HTML puro, em 2026, só no painel do condomínio.

Na prática: as fotos do Rangô no S3

Pré-requisitos: AWS CLI v2 com o profile `rango-dev` do capítulo 8 (e o `rango-admin` para um passo de IAM), o projeto do Rangô com o Compose do capítulo 6 e uma foto qualquer chamada `lasanha.jpg` na pasta atual. Todos os comandos usam `us-east-1`. A lasanha é opcional, mas melhora muito a leitura.

```
export AWS_PROFILE=rango-dev
aws sso login --sso-session rango
```

O bucket

O nome precisa ser único no mundo, então usamos o número da conta como sufixo:

```
CONTA=$(aws sts get-caller-identity --query Account --output text)
BUCKET="rango-fotos-pratos-$CONTA"
echo "$BUCKET"

aws s3api create-bucket --bucket "$BUCKET" --region us-east-1
```

```
{
    "Location": "/rango-fotos-pratos-123456789012"
}
```

Em `us-east-1`, o `create-bucket` não leva `--create-bucket-configuration`. Em outras regiões, como `sa-east-1`, é preciso acrescentar `--create-bucket-configuration LocationConstraint=sa-east-1`. O bucket fica em `us-east-1`, junto da nossa VPC e do gateway endpoint. Por que a região padrão dispensa o parâmetro e as outras não? Porque a `us-east-1` é a filha mais velha da AWS e tem regalias que as irmãs nunca vão ter.

O que veio ligado de fábrica

Antes de mexer em qualquer coisa, vale conferir os padrões de segurança:

```
aws s3api get-public-access-block --bucket "$BUCKET"
aws s3api get-bucket-ownership-controls --bucket "$BUCKET"
aws s3api get-bucket-encryption --bucket "$BUCKET" \
  --query
'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault'
```

```
{
    "PublicAccessBlockConfiguration": {
        "BlockPublicAcls": true,
        "IgnorePublicAcls": true,
        "BlockPublicPolicy": true,
        "RestrictPublicBuckets": true
    }
}
{
    "OwnershipControls": {
        "Rules": [
            {
                "ObjectOwnership": "BucketOwnerEnforced"
            }
        ]
    }
}
{
    "SSEAlgorithm": "AES256"
}
```

As quatro travas estão ligadas, as ACLs estão desabilitadas e a criptografia SSE-S3 está ativa. O próximo passo é subir uma foto e tentar lê-la como um estranho leria:

```
aws s3 cp lasanha.jpg "s3://$BUCKET/teste/lasanha.jpg"
curl -s "https://$BUCKET.s3.amazonaws.com/teste/lasanha.jpg"
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied</Message>...</Error>
```

Esse `AccessDenied` é o comportamento que queremos. Raramente um erro traz tanta alegria, então aproveite. O `curl` não assinou nada, então é um anônimo pedindo, e o bucket recusa. A diferença aparece com uma URL assinada de leitura, gerada pela CLI:

```
URL=$(aws s3 presign "s3://$BUCKET/teste/lasanha.jpg" --expires-in 120)
curl -s -o /dev/null -w "%{http_code}\n" "$URL"
```

```
200
```

No navegador, a mesma URL mostra a foto. Dois minutos depois, ao recarregar a página, o S3 responde com `AccessDenied` e a mensagem `Request has expired`. A Cinderela tinha até meia-noite; a lasanha tem 120 segundos.

A bucket policy

O bucket já é privado. A policy que vamos colocar só acrescenta proibições, com dois `Deny`. O primeiro recusa qualquer acesso que não use HTTPS. O segundo usa a condição `s3:signatureAge`, a idade da assinatura em milissegundos, para recusar uploads assinados há mais de 10 minutos. Mesmo que um bug na API gere uma URL de upload com validade de um dia, ela para de funcionar em 10 minutos. A policy fica em `infra/s3/rango-fotos-policy.json`, com o nome do bucket trocado pelo seu:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SoHttps",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::rango-fotos-pratos-123456789012",
        "arn:aws:s3:::rango-fotos-pratos-123456789012/*"
      ],
      "Condition": {
        "Bool": { "aws:SecureTransport": "false" }
      }
    },
    {
      "Sid": "UploadComAssinaturaRecente",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::rango-fotos-pratos-123456789012/*",
      "Condition": {
        "NumericGreaterThan": { "s3:signatureAge": "600000" }
      }
    }
  ]
}
```

```
aws s3api put-bucket-policy --bucket "$BUCKET" \
  --policy file://infra/s3/rango-fotos-policy.json
```

O `"Principal": "*"` aqui é seguro porque o efeito é `Deny`: a regra vale para todo mundo, inclusive você, e só tira permissões. É o único asterisco que deixa o time de segurança dormir tranquilo. O Block Public Access só barra policies que concedem acesso público. Restringimos a idade da assinatura apenas no `PutObject` porque as URLs de leitura que a API vai gerar valem uma hora, e o navegador precisa delas durante todo esse tempo.

Versionamento e lifecycle

Primeiro, o versionamento é ligado:

```
aws s3api put-bucket-versioning --bucket "$BUCKET" \
  --versioning-configuration Status=Enabled
```

Agora, o delete marker em ação. A sequência abaixo sobrescreve a foto de teste, apaga e lista as versões:

```
aws s3 cp lasanha.jpg "s3://$BUCKET/teste/lasanha.jpg"
aws s3 rm "s3://$BUCKET/teste/lasanha.jpg"
aws s3api list-object-versions --bucket "$BUCKET" --prefix teste/ \
  --query '{Versoes: Versions[].{Id: VersionId, Atual: IsLatest}, Marcadores:
DeleteMarkers[].{Id: VersionId, Atual: IsLatest}}'
```

```
{
    "Versoes": [
        {
            "Id": "Rb5t0jEXAMPLEqv3N8d1Wc2x",
            "Atual": false
        },
        {
            "Id": "null",
            "Atual": false
        }
    ],
    "Marcadores": [
        {
            "Id": "kQ7mXEXAMPLEzT4pLa9sVb0e",
            "Atual": true
        }
    ]
}
```

A versão com `Id` `null` é a primeira foto, enviada antes de o versionamento ser ligado. A outra é a cópia que acabamos de subir. O marcador é o `rm`. A foto parece apagada (um `aws s3 ls` não mostra nada), mas as duas versões continuam lá. É o Ctrl+Z que a vida real nunca ofereceu. Desfazer o `rm` é apagar o marcador:

```
aws s3api delete-object --bucket "$BUCKET" --key teste/lasanha.jpg \
  --version-id kQ7mXEXAMPLEzT4pLa9sVb0e
```

```
aws s3 ls "s3://$BUCKET/teste/"
```

A foto voltou. Falta a lifecycle rule, para que esse histórico não cresça para sempre. Ela fica em `infra/s3/rango-fotos-lifecycle.json`:

```
{
  "Rules": [
    {
      "ID": "limpar-historico",
      "Filter": { "Prefix": "" },
      "Status": "Enabled",
      "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
      "Expiration": { "ExpiredObjectDeleteMarker": true },
      "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
    }
  ]
}
```

```
aws s3api put-bucket-lifecycle-configuration --bucket "$BUCKET" \
  --lifecycle-configuration file://infra/s3/rango-fotos-lifecycle.json
```

A regra faz três coisas. Versões antigas somem 30 dias depois de deixarem de ser a atual, o que dá um mês para perceber e desfazer um erro. Delete markers que ficaram sozinhos, sem nenhuma versão embaixo, são removidos. E uploads em partes (*multipart uploads*) que começaram e nunca terminaram são abortados em 7 dias. Esse último item merece atenção: um único `PUT` aceita objetos de até 5 GB, e arquivos maiores são enviados em partes. Se o envio for interrompido, as partes já recebidas ficam guardadas, invisíveis no `aws s3 ls`, e cobradas todo mês. São os fantasmas do bucket: ninguém vê, mas eles aparecem religiosamente na fatura.

CORS: o navegador também precisa de permissão

A API do Rangô já tem CORS configurado desde o capítulo 1. O bucket é outra origem, e o navegador vai fazer uma requisição *preflight* (`OPTIONS`) antes do `PUT`. Sem uma configuração de CORS no bucket, o upload falha no console do navegador com a mensagem `has been blocked by CORS policy`, mesmo com a URL assinada correta. Se existisse um ranking de frases em inglês mais lidas por quem faz front-end, essa disputaria o primeiro lugar com "undefined is not a function". A configuração fica em `infra/s3/rango-fotos-cors.json`:

```
{
  "CORSRules": [
    {
      "AllowedOrigins": ["https://rango.test:5173"],
      "AllowedMethods": ["PUT"],
      "AllowedHeaders": ["content-type"],
      "MaxAgeSeconds": 3000
    }
```

```
  ]
}
```

```
aws s3api put-bucket-cors --bucket "$BUCKET" \
  --cors-configuration file://infra/s3/rango-fotos-cors.json
```

Só `PUT` e só a origem do front local. Lista de permissões curta é como lista de convidados de casamento pequeno: menos gente, menos confusão no final. As fotos vão ser exibidas com a tag `<img>`, que não precisa de CORS para mostrar uma imagem. Quando o front tiver domínio próprio, no capítulo 14, a origem entra nesta lista.

Permissão para a role do servidor

Quando a API rodar na EC2, quem assina as URLs é a role `rango-ec2` do capítulo 8. Ela precisa das ações que a API usa, e só no prefixo `fotos/`. A policy fica em `iam/rango-ec2-fotos.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "FotosDosPratos",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::rango-fotos-pratos-123456789012/fotos/*"
    }
  ]
}
```

Criar policies é tarefa de IAM, e o `PowerUserAccess` do profile `rango-dev` não inclui IAM. O profile de administrador entra só neste comando:

```
aws iam put-role-policy --profile rango-admin \
  --role-name rango-ec2 --policy-name RangoFotos \
  --policy-document file://iam/rango-ec2-fotos.json
```

Não incluímos `s3:ListBucket`. A API sempre sabe qual key quer, porque guarda as keys no Redis, e não precisa listar o bucket. Uma consequência curiosa: sem `ListBucket`, um `HEAD` numa key inexistente devolve 403 em vez de 404, para não revelar se o objeto existe. O código abaixo trata os dois casos do mesmo jeito.

Uma presigned URL só funciona se quem a assinou tiver a permissão. Se a role não tiver `s3:PutObject`, a API gera a URL sem reclamar (é só um cálculo), e o erro aparece no navegador como `403 AccessDenied`. A API, nesse caso, é o amigo que passa o endereço da festa sem saber que você não foi convidado.

A API: gerar, registrar e listar

O `boto3` entra no `backend/requirements.txt`, e o código fica em `backend/app/fotos.py`:

```
import os
import uuid

import boto3
import redis
from botocore.config import Config
from botocore.exceptions import ClientError
from fastapi import APIRouter, HTTPException, Path
from pydantic import BaseModel

BUCKET = os.environ["FOTOS_BUCKET"]
TIPOS_ACEITOS = {"image/jpeg": "jpg", "image/png": "png", "image/webp": "webp"}
TAMANHO_MAXIMO = 10 * 1024 * 1024  # 10 MB

s3 = boto3.client(
    "s3",
    region_name=os.environ.get("AWS_REGION", "us-east-1"),
    config=Config(
        signature_version="s3v4",
        # Não acrescenta checksums opcionais à URL assinada; o navegador não os enviaria
        request_checksum_calculation="when_required",
    ),
)
r = redis.Redis.from_url(os.environ["REDIS_URL"], decode_responses=True)

router = APIRouter()
Restaurante = Path(pattern=r"^[a-z0-9-]{1,60}$")


class PedidoDeUpload(BaseModel):
    content_type: str


class FotoEnviada(BaseModel):
    chave: str


@router.post("/restaurantes/{restaurante}/fotos/upload-url")
def gerar_url_de_upload(pedido: PedidoDeUpload, restaurante: str = Restaurante):
    extensao = TIPOS_ACEITOS.get(pedido.content_type)
    if extensao is None:
        raise HTTPException(status_code=415, detail="Envie JPEG, PNG ou WebP")
    chave = f"fotos/{restaurante}/{uuid.uuid4()}.{extensao}"
    url = s3.generate_presigned_url(
        "put_object",
        Params={"Bucket": BUCKET, "Key": chave, "ContentType": pedido.content_type},
        ExpiresIn=300,
    )
    return {"url": url, "chave": chave}


@router.post("/restaurantes/{restaurante}/fotos", status_code=201)
def registrar_foto(foto: FotoEnviada, restaurante: str = Restaurante):
    if not foto.chave.startswith(f"fotos/{restaurante}/"):
        raise HTTPException(status_code=400, detail="Chave de outro restaurante")
    try:
        info = s3.head_object(Bucket=BUCKET, Key=foto.chave)
```

```
    except ClientError:
        raise HTTPException(status_code=404, detail="Foto não encontrada no bucket")
    if info["ContentLength"] > TAMANHO_MAXIMO:
        s3.delete_object(Bucket=BUCKET, Key=foto.chave)
        raise HTTPException(status_code=413, detail="Foto maior que 10 MB")
    r.sadd(f"fotos:{restaurante}", foto.chave)
    return {"chave": foto.chave}


@router.get("/restaurantes/{restaurante}/fotos")
def listar_fotos(restaurante: str = Restaurante):
    return [
        s3.generate_presigned_url(
            "get_object", Params={"Bucket": BUCKET, "Key": chave}, ExpiresIn=3600
        )
        for chave in sorted(r.smembers(f"fotos:{restaurante}"))
    ]
```

O router é registrado no `backend/app/main.py`:

```
from app.fotos import router as fotos_router

app.include_router(fotos_router)
```

Alguns detalhes desse código merecem comentário.

O `ContentType` entra na assinatura. O navegador precisa enviar exatamente esse `Content-Type` no `PUT`, ou o S3 recusa. Isso impede que alguém peça uma URL para `image/jpeg` e suba um HTML no lugar, que depois seria servido com o tipo errado. Ninguém quer uma lasanha que, ao ser aberta, vira página de phishing.

O padrão no parâmetro `restaurante` evita keys estranhas. Falta conferir se quem pede é dono do restaurante, checagem que entra quando o Rangô tiver login de restaurante: gerar uma presigned URL é conceder uma permissão.

A configuração `request_checksum_calculation="when_required"` evita que as versões recentes do boto3, que calculam checksums de integridade por padrão, coloquem na assinatura algo que o `fetch` não vai enviar.

Por fim, as keys ficam no Redis (`fotos:{restaurante}`), como os pedidos do capítulo 5. É o mesmo arranjo provisório, e todo mundo que trabalha com software sabe que nada dura tanto quanto uma solução provisória. No próximo capítulo, os pedidos ganham um banco de verdade; o registro das fotos continua no Redis até o capítulo 24, quando a API passa a perguntar ao próprio bucket.

O React: envio direto para a AWS

O componente fica em `frontend/src/EnviarFoto.tsx`:

```
import { useState, type ChangeEvent } from "react";
```

```tsx
const API = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

export function EnviarFoto({ restaurante }: { restaurante: string }) {
  const [status, setStatus] = useState("");

  async function enviar(evento: ChangeEvent<HTMLInputElement>) {
    const arquivo = evento.target.files?.[0];
    if (!arquivo) return;

    setStatus("Pedindo autorização...");
    const pedido = await fetch(`${API}/restaurantes/${restaurante}/fotos/upload-url`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ content_type: arquivo.type }),
    });
    if (!pedido.ok) {
      setStatus("Formato não aceito. Use JPEG, PNG ou WebP.");
      return;
    }
    const { url, chave } = await pedido.json();

    setStatus("Enviando a foto...");
    const upload = await fetch(url, {
      method: "PUT",
      headers: { "Content-Type": arquivo.type },
      body: arquivo,
    });
    if (!upload.ok) {
      setStatus(`O envio falhou (HTTP ${upload.status}).`);
      return;
    }

    const registro = await fetch(`${API}/restaurantes/${restaurante}/fotos`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ chave }),
    });
    setStatus(registro.ok ? "Foto publicada." : "A foto foi recusada.");
  }

  return (
    <label>
      Foto do prato{" "}
      <input type="file" accept="image/jpeg,image/png,image/webp" onChange={enviar} />
      <span>{status}</span>
    </label>
  );
}
```

O `body: arquivo` envia o `File` em binário, sem `FormData`, com o mesmo `Content-Type` usado na assinatura.

O Compose com as suas credenciais

Na EC2, o boto3 encontraria as credenciais da role sozinho. No Compose local, o container não tem credencial, então emprestamos a sessão SSO da sua máquina, montando o `~/.aws` como somente leitura. No `compose.yaml`, o serviço `api` (e o `api-canario`, se ele estiver ligado) ganha estas linhas:

```
    image: rango-api:0.6.0
    environment:
      CORS_ORIGINS: "https://rango.test:5173"
      REDIS_URL: "redis://redis:6379/0"
      FOTOS_BUCKET: "rango-fotos-pratos-123456789012"
      AWS_REGION: "us-east-1"
      AWS_PROFILE: "rango-dev"   # só em desenvolvimento
    volumes:
      - ~/.aws:/home/rango/.aws:ro
```

O caminho `/home/rango` vem do `useradd --create-home rango` do Dockerfile. Esse volume serve só para a sua máquina; em produção, credencial vem da role, nunca de arquivo. Depois vêm o build e o Compose:

```
docker build -t rango-api:0.6.0 backend
docker compose up -d --scale api=3
```

Com o `<EnviarFoto restaurante="cantina-da-nona" />` colocado em alguma tela do `App.tsx`, a foto é enviada pela página `https://rango.test:5173`. Na aba Network do navegador aparecem quatro requisições: o `POST` para a API, o `OPTIONS` e o `PUT` para `rango-fotos-pratos-...s3.amazonaws.com`, e o `POST` de registro. O `PUT` de 4 MB nunca passou pelo Nginx nem pelo Uvicorn, que seguiram a vida sem saber da lasanha. O terminal confirma:

```
aws s3 ls "s3://$BUCKET/fotos/cantina-da-nona/"
curl -s https://api.rango.test:8443/restaurantes/cantina-da-nona/fotos
```

O erro antes do acerto

Vale a pena quebrar o upload de propósito, para reconhecer a mensagem quando ela aparecer sem querer. Quebrar de propósito é a única forma de quebrar que não termina em postmortem. O exemplo pede uma URL para `image/jpeg` e envia dizendo que é PNG:

```
RESPOSTA=$(curl -s -X POST https://api.rango.test:8443/restaurantes/cantina-da-
nona/fotos/upload-url \
  -H "Content-Type: application/json" -d '{"content_type": "image/jpeg"}')
URL=$(echo "$RESPOSTA" | python3 -c 'import json, sys; print(json.load(sys.stdin)
["url"])')

curl -s -X PUT -H "Content-Type: image/png" --upload-file lasanha.jpg "$URL"
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>SignatureDoesNotMatch</Code><Message>The request signature we calculated
does not match the signature you provided. Check your key and signing method.
</Message>...
```

Com `image/jpeg`, o mesmo comando devolve 200. `SignatureDoesNotMatch` quase sempre significa que a requisição usada é diferente da que foi assinada: outro método, outro cabeçalho, um proxy que mexeu em algo no caminho, ou o relógio da máquina que assinou fora de sincronia. A criptografia, ao contrário de muita reunião, exige pontualidade. Outro erro que você vai encontrar é `ExpiredToken`: a URL ainda estava no prazo, mas a credencial temporária que a assinou expirou. No desenvolvimento local, isso acontece quando a sessão SSO acaba; um `aws sso login` resolve.

O que fica ligado

O bucket fica, porque é a casa das fotos do Rangô daqui em diante. O custo de algumas dezenas de MB é uma fração de centavo por mês, menos do que a moeda de cinco centavos que ninguém mais aceita na padaria. A limpeza apaga a foto de teste e as versões dela, e derruba o Compose:

```
aws s3 rm "s3://$BUCKET/teste/lasanha.jpg"
docker compose down
```

O `rm` só cria um delete marker, e a lifecycle rule limpa o resto em 30 dias. Para apagar o bucket inteiro um dia, esvazie-o antes pela opção *Empty* do console: num bucket versionado, `aws s3 rm --recursive` deixa as versões antigas para trás.

Armadilhas e casos reais

> **Na vida real:** Em junho de 2017, Chris Vickery, pesquisador da UpGuard, encontrou um bucket do S3 aberto com dados de clientes da Verizon. O bucket pertencia à NICE Systems, fornecedora do call center da operadora, e continha nomes, endereços e PINs usados para confirmar a identidade do cliente por telefone. A UpGuard falou em 14 milhões de clientes; a Verizon, em cerca de 6 milhões. Bastava conhecer a URL para baixar os arquivos, e o bucket ficou aberto por mais de uma semana depois do aviso.

O vazamento da Capital One, citado no capítulo 9, foi de outro tipo: nenhum bucket estava público, e o atacante usou credenciais de uma role com acesso a buckets demais. Por isso a `RangoFotos` dá acesso só ao prefixo `fotos/`. Privilégio mínimo é como sal: ninguém elogia quando está na medida, todo mundo percebe quando passou da conta.

> **Armadilha:** O Block Public Access do bucket pode ser desligado com um clique, e o console avisa, mas deixa. Ligue também o Block Public Access na conta (`aws s3control put-public-access-block --account-id ... --public-access-block-`

> `configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true`). Quando o Rangô precisar de um bucket realmente público, a resposta vai ser CloudFront com OAC, no capítulo 14, que serve o conteúdo sem abrir o bucket.
>
> **Armadilha:** Key escolhida pelo cliente. Se o React mandasse `fotos/cantina-da-nona/lasanha.jpg` e a API assinasse essa key, qualquer pessoa poderia sobrescrever a foto de outro prato, já que um `PUT` na mesma key substitui o objeto. A API sempre gera a key, com UUID.
>
> **Armadilha:** Presigned URL em log. O middleware de log do capítulo 6 registra o path das requisições, e a resposta de `/fotos/upload-url` contém uma URL que funciona como senha por alguns minutos. Não registre corpos de resposta, e não mande essas URLs para ferramentas de analytics do front.
>
> **Armadilha:** Versões acumulando. Uma rotina que regrava todas as fotos toda noite multiplica o armazenamento por 30 em um mês num bucket versionado sem lifecycle, e o `aws s3 ls` continua mostrando o tamanho de sempre, porque só lista as versões atuais.
>
> **Dica:** A EC2 nas subnets privadas da `rango-vpc` fala com o S3 pelo gateway endpoint que criamos no capítulo 10, sem NAT e sem custo por GB. O upload do navegador não passa por ali: ele vai pela internet direto para o S3. São dois caminhos diferentes para o mesmo bucket.

Resumo

- O S3 guarda objetos (bytes mais metadados) em buckets, acessados por HTTP por key. Não há pastas, só prefixos, e não há edição parcial.
- Nomes de bucket são únicos no mundo. Não há limite de tamanho do bucket, e a classe padrão replica cada objeto em várias AZs.
- Classes mais frias custam menos para guardar e mais para ler, com duração mínima cobrada (30, 90 ou 180 dias) e tamanho mínimo de 128 KB em algumas delas.
- Lifecycle rules movem e apagam objetos por idade. Desde setembro de 2024, por padrão, não transicionam objetos menores que 128 KB.
- Versionamento transforma sobrescrita e exclusão em operações reversíveis. Com lifecycle, o histórico tem prazo.
- Bucket policy é a resource-based policy do S3. Block Public Access fica acima dela e, desde abril de 2023, vem ligado em todo bucket novo, com ACLs desabilitadas.

Objetos novos são criptografados com SSE-S3 desde janeiro de 2023.

- Presigned URLs dão acesso temporário a uma operação específica sem abrir o bucket. Expiram junto com a credencial que as assinou e funcionam como senha enquanto valem.
- O Rangô ganhou o bucket `rango-fotos-pratos-<conta>`, privado, versionado, com lifecycle, CORS e uma policy que exige HTTPS e assinaturas recentes. O React envia fotos direto para o S3, e a API só assina e registra.

O banco que ninguém quer acordar para cuidar

As fotos saíram do servidor. O próximo inquilino é bem mais delicado. No capítulo 10, o Postgres do Rangô rodou numa EC2 numa subnet privada, protegido por três camadas de rede. Só que alguém precisa rodar o `scripts/backup.sh` do capítulo 6 e guardar o dump em lugar seguro, aplicar o próximo patch do PostgreSQL 17 sem derrubar os pedidos e vigiar o disco do banco, que enche de madrugada do mesmo jeito que o das fotos encheu (discos, por algum motivo, preferem encher às três da manhã de domingo). E os pedidos continuam no Redis, esperando um banco de verdade. No próximo capítulo, o Amazon RDS assume essa rotina: backups automáticos com point-in-time recovery, uma réplica pronta em outra AZ e o banco do Rangô morando nas subnets privadas que construímos.

Referências

- Amazon S3, guia do usuário: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html
- Restrições e limites de buckets: https://docs.aws.amazon.com/AmazonS3/latest/dev/BucketRestrictions.html
- Até 1 milhão de buckets por conta (novembro de 2024): https://aws.amazon.com/about-aws/whats-new/2024/11/amazon-s3-up-1-million-buckets-per-aws-account/
- Preços do Amazon S3: https://aws.amazon.com/s3/pricing/
- Levantamento de preços do S3 por terceiros (valores de referência usados no capítulo): https://cloudburn.io/blog/amazon-s3-pricing
- Considerações sobre transições de lifecycle (30 dias, 128 KB): https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-transition-general-considerations.html
- Novos padrões de segurança em buckets (abril de 2023): https://aws.amazon.com/about-aws/whats-new/2023/04/amazon-s3-security-best-

practices-buckets-default

- Anúncio das mudanças de abril de 2023: https://aws.amazon.com/blogs/aws/heads-up-amazon-s3-security-changes-are-coming-in-april-of-2023
- Block Public Access e ACLs desabilitadas em qualquer método de criação: https://aws.amazon.com/about-aws/whats-new/2022/12/amazon-s3-automatically-enable-block-public-access-disable-access-control-lists-buckets-april-2023
- Criptografia automática de objetos novos (janeiro de 2023): https://aws.amazon.com/about-aws/whats-new/2023/01/amazon-s3-automatically-encrypts-new-objects
- FAQ de criptografia padrão: https://docs.aws.amazon.com/AmazonS3/latest/userguide/default-encryption-faq.html
- Presigned URLs (validade, credenciais temporárias, `s3:signatureAge`, erros comuns): https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html
- boto3, `generate_presigned_url`: https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3/client/generate_presigned_url.html
- Vazamento de dados de clientes da Verizon via bucket da NICE Systems (TechTarget): https://www.techtarget.com/searchsecurity/news/450422709/Misconfigured-AWS-S3-bucket-exposes-millions-of-Verizon-customers-data
- Vazamento da Capital One, controles a revisar (Rackspace): https://www.rackspace.com/pt-br/blog/capital-one-data-breach-two-security-controls-you-should-review

CAPÍTULO 12

RDS: banco de dados gerenciado

No capítulo anterior, as fotos dos pratos saíram do disco do servidor e foram para o bucket `rango-fotos-pratos-<conta>`, privado e versionado, com o React enviando os arquivos direto para o S3 por presigned URLs. Sobrou o inquilino mais delicado, aquele que nunca atrasa o aluguel e, quando tem um problema, liga às três da manhã: o PostgreSQL, que no capítulo 10 rodou numa EC2 da subnet privada, e os pedidos, que até hoje moram no Redis.

Duas e quarenta e sete da manhã

Suponha que você tivesse mantido a `rango-db-01` do capítulo 10 em produção. A cena é hipotética, mas cada peça dela acontece com quem cuida de banco na mão.

Sábado, 2h47. O disco de 10 GiB do banco chega a 100%. Quem encheu foi o WAL, o log de escrita do Postgres, porque uma réplica que você testou na semana anterior ficou registrada como *replication slot* e o servidor guardou cada segmento de log esperando por ela, com a lealdade do Hachiko na estação de Shibuya. O Postgres não tem onde escrever e para:

```
PANIC:  could not write to file "pg_wal/xlogtemp.48213": No space left on device
```

A API começa a responder 500 em todo `POST /pedidos`. Você acorda às 7h com mensagens da Cantina da Nona: dezessete textos, quatro áudios e uma foto de lasanha esfriando no balcão. Para aumentar o disco, precisa modificar o volume EBS, esperar, entrar pelo Session Manager e expandir o sistema de arquivos. Aí descobre a segunda surpresa: o `scripts/backup.sh` que você agendou no cron rodava como um usuário que perdeu a permissão de escrita no diretório de destino há doze dias. O log do cron registrou o erro, e ninguém leu. Log que ninguém lê é diário íntimo do servidor.

E ainda tem a pendência da semana: um patch de segurança do PostgreSQL 17, que exige parar o banco, atualizar o pacote e torcer para nada quebrar. Torcer, aliás, é a ferramenta de operação mais usada e menos documentada do mercado.

Nada disso tem relação com o produto do Rangô. São tarefas de DBA: vigiar disco, garantir e testar backup, aplicar patches, manter uma réplica pronta. Quando um trabalho é idêntico em milhares de empresas, a AWS costuma transformá-lo em serviço, e em mais uma sigla de três letras para você decorar.

O que o RDS assume por você

O **Amazon RDS** (*Relational Database Service*) roda bancos relacionais gerenciados: PostgreSQL, MySQL, MariaDB, Oracle, SQL Server e Db2. Sim, o Db2 continua vivo, e é bem possível que algum esteja calculando o seu salário neste momento. Você escolhe a engine, a versão, o tamanho da máquina e o disco. A AWS cuida do sistema operacional, da instalação, dos backups, dos patches, da replicação e da troca automática para a réplica quando a máquina principal falha.

No capítulo 7, a tabela de responsabilidade compartilhada já mostrava o RDS no meio do caminho entre a EC2 e o Lambda. Do seu lado ficam o esquema, as queries, os índices, os usuários do banco, a rede que chega até ele e as decisões de configuração (quantos dias de backup, se tem réplica, quando aplicar atualização). A AWS cuida da cozinha; o tempero ruim continua sendo culpa sua.

O que você perde é o acesso ao servidor: nada de SSH ou Session Manager. Você fala com o banco pela porta 5432 e com a AWS pela API. O usuário mestre que você cria recebe a role `rds_superuser`, que tem quase todos os poderes, mas fica abaixo do superusuário do PostgreSQL. Tem crachá de gerente e nenhuma chave da sala do servidor. Algumas extensões e configurações que exigem acesso ao sistema de arquivos ficam fora do seu alcance.

Uma **DB instance** é um banco rodando numa máquina gerenciada, com classes parecidas com as da EC2: `db.t4g.micro`, `db.m7g.large`, `db.r7g.xlarge`. Depois do prefixo `db.`, as letras são as do capítulo 9: `t` para burstable, `m` para uso geral, `r` para memória, `g` para Graviton. Quem decorou a sopa de letrinhas da EC2 ganha aqui o único retorno sobre investimento garantido deste livro.

Qual versão do PostgreSQL

O RDS segue o calendário da comunidade. Uma versão major fica em *standard support* pelo menos até o fim do suporte da comunidade. Depois disso, ela não para de funcionar: entra em **RDS Extended Support**, que é pago à parte e dura até três anos além do fim do standard support. É a garantia estendida dos bancos de dados, oferecida por um vendedor que já sabe que você vai precisar dela.

O PostgreSQL 18 foi lançado pela comunidade em 13 de novembro de 2025 e chegou ao RDS no dia seguinte. A AWS publica o fim do standard support do 18 no RDS em 28 de fevereiro de 2031, e o fim do Extended Support em 28 de fevereiro de 2034. Para comparar, o PostgreSQL 14 sai do standard support por volta de fevereiro de 2027. Banco novo começa na major mais recente: é a que vai demorar mais para exigir um upgrade. O Rangô vai de PostgreSQL 18.

As versões minor (18.1, 18.2, 18.4) trazem correções e patches de segurança. No RDS, você pode deixar a opção de atualização automática de minor ligada, e a AWS aplica a nova versão na **janela de manutenção**, um intervalo semanal de 30 minutos que você escolhe. A pendência do patch de sábado vira uma configuração. Deve ser a primeira vez na história que uma madrugada de sábado vira um checkbox. Uma minor também pode perder suporte antes da major, então não vale a pena desligar essa opção e esquecer.

Upgrade de major continua sendo decisão sua, porque pode quebrar compatibilidade. O RDS faz a parte mecânica e oferece os *Blue/Green Deployments*: uma cópia do banco com a versão nova, mantida em sincronia, para a qual você troca depois dos testes, como no blue/green do capítulo 6. Upgrade de major sem teste é como trocar o pneu com o carro andando: alguns conseguem, e todos se arrependem de ter tentado.

Disco que cresce sozinho

O armazenamento do RDS é um volume EBS que você nunca vê. O tipo recomendado para começar é o gp3, com mínimo de 20 GiB no PostgreSQL. O recurso que resolve o PANIC das 2h47 se chama **storage autoscaling**: você define um teto (`--max-allocated-storage`), e o RDS aumenta o disco sozinho quando o espaço livre fica abaixo de 10% por pelo menos cinco minutos. Entre um aumento e outro, ele espera seis horas.

O autoscaling nunca diminui o disco e não descobre por que ele está enchendo. Se uma replication slot esquecida estiver segurando WAL, o autoscaling só adia o problema até o teto, como comprar uma gaveta maior em vez de jogar fora as sacolinhas de plástico. Por isso, o alarme no espaço livre (métrica `FreeStorageSpace`, que veremos no capítulo 22) continua necessário.

Backups automáticos e point-in-time recovery

No capítulo 6, você viu a diferença entre um dump (um retrato do banco) e o backup físico com WAL arquivado, que permite voltar a qualquer instante, como um `git checkout` para um commit específico. O RDS faz o segundo tipo por padrão.

Uma vez por dia, dentro da **janela de backup** que você escolhe, ele tira um snapshot do volume. Além disso, envia os logs de transação para o S3 a cada cinco minutos. Com o snapshot mais os logs, consegue reconstruir o banco em qualquer segundo do período de

retenção, que vai de 1 a 35 dias (0 desliga os backups automáticos, e com eles o point-in-time recovery). Por causa desse envio, o `LatestRestorableTime`, o momento mais recente para o qual você pode restaurar, costuma ficar uns cinco minutos atrás do relógio. Lembra do RPO de 5 minutos que definimos para o Rangô no capítulo 6? Ele vem pronto. É raro um requisito chegar pronto, então aproveite antes que alguém mude de ideia.

O detalhe que mais surpreende quem chega ao RDS: uma restauração **nunca sobrescreve** o banco existente. O RDS não entrega a ninguém um botão de "sobrescrever produção", e, sendo honesto, ninguém deveria ter um. O *point-in-time recovery* (PITR) cria uma instância nova, com outro nome e outro endereço. Você pode comparar os dois bancos e copiar só o que precisa, mas a aplicação continua apontando para o banco antigo até alguém mudar a configuração. A parte prática do capítulo faz isso.

Há também os **snapshots manuais**, que você tira quando quiser (antes de uma migration arriscada, por exemplo). Eles ficam até você apagá-los, mesmo depois que a instância some. Os automáticos, ao contrário, vão embora com a instância, a menos que você peça para mantê-los. São como os amigos do trabalho que somem quando você muda de emprego: estavam lá por causa da instância.

Multi-AZ: a réplica que fica esperando

Backup resolve o "perdi dados". Não resolve o "a máquina morreu e o banco está fora". Para isso, o capítulo 6 apresentou a replicação com failover. No RDS, isso se chama **Multi-AZ** e tem duas formas.

Na **Multi-AZ DB instance**, o RDS mantém uma segunda instância, a *standby*, em outra AZ. Cada escrita é replicada de forma síncrona: o commit só volta para a aplicação depois que os dois lados gravaram. Se a primária falhar, o RDS promove a standby e aponta o mesmo nome DNS para ela. A aplicação reconecta no mesmo endereço e segue em frente. A AWS não publica um tempo oficial para esse failover; o jeito é medir o seu, como a parte prática faz.

O ponto que confunde: essa standby não atende leitura nem escrita. Você paga por uma segunda máquina que fica parada esperando um problema, como um gerador de hospital. Ou como o goleiro reserva, que recebe salário para assistir ao jogo e torcer para não ser chamado.

Na **Multi-AZ DB cluster**, disponível só para MySQL e PostgreSQL, são três instâncias em três AZs: uma escritora e duas standbys que **aceitam leitura**. A AWS informa failover típico abaixo de 35 segundos, sem perda de dados, e latência de commit até duas vezes menor que a da versão com uma standby.

	Single-AZ	Multi-AZ DB instance	Multi-AZ DB cluster
Instâncias	1	2 (1 standby)	3 (2 standbys)
AZs	1	2	3
Standby atende leitura	não há standby	não	sim
Engines	todas	todas	MySQL e PostgreSQL
Custo de instância	1×	cerca de 2×	cerca de 3×

Para o Rangô em produção, a Multi-AZ DB instance é o ponto de partida razoável: protege contra a perda de uma AZ e não exige mudança no código. Mesmo assim, o Rangô começa em Single-AZ, para conter o custo enquanto você aprende, e a parte prática liga o Multi-AZ só por alguns minutos para ver o failover. A conta volta no capítulo 24, na lista de pendências de confiabilidade.

Réplicas de leitura

Uma **read replica** é outra coisa. Ela recebe as mudanças da primária de forma assíncrona e tem um endereço próprio, que a aplicação usa para consultas. Serve para tirar carga de leitura da primária: relatórios, a listagem de restaurantes, o painel administrativo. Pode estar em qualquer AZ ou em outra região, e pode ser promovida a banco independente num plano de recuperação de desastre.

Por ser assíncrona, ela atrasa. O atraso aparece na métrica `ReplicaLag`, em segundos. O problema que o capítulo 6 descreveu continua valendo: o usuário que acabou de fazer um pedido e lê "meus pedidos" na réplica pode não ver o pedido novo e ligar para o suporte convencido de que o dinheiro sumiu. Leituras logo depois de uma escrita do mesmo usuário vão para a primária.

Multi-AZ responde "e se a máquina cair?"; réplica responde "e se a primária não der conta das leituras?". Um banco de produção com bastante tráfego costuma ter as duas. Confundir uma com a outra é o jeito mais caro de descobrir a diferença.

Parameter groups: o `postgresql.conf` que você não edita

Sem acesso ao servidor, as configurações ficam num **DB parameter group**, um conjunto de parâmetros que você associa a uma ou mais instâncias. Cada versão major tem uma família (`postgres18`), e cada família tem um grupo padrão, que não pode ser editado. Para mudar qualquer coisa, você cria um grupo seu e associa à instância.

Cada parâmetro é dinâmico (vale na hora) ou estático (exige reinício). Trocar o grupo associado a uma instância que já está rodando também exige reinício. Por isso, crie o grupo customizado antes do banco, mesmo sem nada para mudar ainda, como a route

table privada explícita do capítulo 10.

Um parâmetro merece atenção desde o início: `rds.force_ssl`. Com ele em 1, o banco recusa conexões sem TLS. No PostgreSQL 15 em diante, ele já vem ligado; no 14 e anteriores, vem desligado. A senha do banco viaja pela rede da VPC, e o TLS do capítulo 2 vale aqui também.

O outro número que você precisa conhecer é o `max_connections`. No RDS, o padrão é calculado pela memória da classe: `LEAST({DBInstanceClassMemory/9531392},5000)`. Numa `db.t4g.micro`, com 1 GiB, isso dá no máximo pouco mais de cem conexões, e menos na prática, porque parte da memória fica reservada. O gargalo do capítulo 4 (`FATAL: sorry, too many clients already`) chega rápido quando cada processo da API abre seu próprio pool. Cem conexões parecem muito até alguém contar quantos workers sobem num deploy.

E o Aurora?

O **Amazon Aurora** é o banco relacional que a AWS construiu, compatível com PostgreSQL e MySQL. Para a aplicação, ele é um Postgres, com o mesmo driver e o mesmo SQL, como um dublê tão parecido que nem o diretor percebe a troca. A diferença está embaixo. Em vez de cada instância ter seu volume EBS, todas compartilham um volume de cluster distribuído, que guarda seis cópias dos dados em três AZs. Paranoia, quando bem implementada, ganha o nome de arquitetura. As réplicas leem do mesmo armazenamento que a escritora, então o atraso entre elas costuma ficar na casa dos milissegundos, e o failover não precisa copiar dados.

O **Aurora Serverless v2** acrescenta escala automática de capacidade, medida em ACUs (*Aurora Capacity Units*). Com versões recentes (Aurora PostgreSQL 16.3, 15.7, 14.12 ou 13.15 em diante), a capacidade mínima pode ser 0 ACU: depois de um tempo sem conexões (de 300 a 86.400 segundos, padrão 300), a instância pausa e para de cobrar computação. O armazenamento continua sendo cobrado. A primeira conexão depois da pausa espera a retomada, que leva uns 15 segundos, e 30 segundos ou mais se o banco ficou pausado por mais de um dia. Qualquer conexão aberta, mesmo ociosa, impede a pausa, e um pool da API mantém o banco acordado para sempre, como criança que tomou refrigerante depois das dez da noite.

Para o Rangô de hoje, o RDS PostgreSQL é a escolha mais simples e previsível. O Aurora começa a compensar com muitas réplicas de pouco atraso, ou num ambiente de desenvolvimento que pode pausar à noite. O preço dele tem lógica própria, então faça a conta no AWS Pricing Calculator antes de trocar.

Cache gerenciado: ElastiCache

O banco guarda só uma parte dos dados do Rangô. Os carrinhos, as chaves de idempotência e o stream de eventos vivem num Valkey instalado na mesma EC2 da API desde o capítulo 9. Se a instância some, somem os carrinhos e os eventos ainda não processados. E no capítulo 13, com várias instâncias, cada uma teria o seu Valkey, e o problema da Ana e dos três carrinhos voltaria.

O **Amazon ElastiCache** roda Valkey, Redis OSS e Memcached gerenciados. Para o Rangô, a escolha é o Valkey, o fork open source criado em 2024, que já usamos na EC2. Ele nasceu depois que o Redis mudou de licença; no mundo open source, quando a licença muda, alguém faz um fork antes do café esfriar. Há dois modelos.

No modelo baseado em nós, você escolhe a classe e o número de réplicas e paga por hora de cada nó. Em contas antigas, a `cache.t3.micro` entra no free tier de 12 meses, um raro caso em que ter uma conta velha conta pontos.

No **ElastiCache Serverless**, você não escolhe máquina. Paga pelo volume de dados guardado e pelas operações, medidas em ECPUs. Para Valkey, em `us-east-1`, a referência em outubro de 2026 era de US$ 0,084 por GB-hora e US$ 0,0023 por milhão de ECPUs, com mínimo medido de 100 MB por cache. Esse mínimo dá um piso de uns US$ 6 por mês, só de armazenamento, o preço de um sanduíche para guardar quase nada. A AWS informa que o Serverless com Valkey sai 33% mais barato que com as outras engines e tem um mínimo 90% menor (o do Redis OSS é de 1 GB). Os valores são aproximados; confira a página de preços antes de decidir. O free tier do ElastiCache não cobre o Serverless.

O Serverless sempre exige TLS e sempre funciona em *cluster mode*, que divide os dados em partes (*shards*). Para o cliente, porém, tudo aparece como um único nó virtual, e o cliente Python comum funciona para comandos de uma chave só, que são todos os que o Rangô usa. Comandos que tocam várias chaves em pedaços diferentes falham com erro `CROSSSLOT`, um nome que parece golpe de luta e dói parecido.

Na prática: o Rangô ganha um banco de verdade

Pré-requisitos: AWS CLI v2 com o profile `rango-dev`, a `rango-vpc` do capítulo 10 com as subnets e os Security Groups `rango-web` e `rango-db`, e a role `rango-ec2`. A parte que roda no servidor usa a `rango-api-01` criada de novo na subnet `rango-publica-a`, com o comando do capítulo 10, e com a seção "Trazendo o código" do capítulo 9 repetida. Tudo em `us-east-1`.

Sobre custo: a `db.t4g.micro` Single-AZ entra no free tier de 12 meses das contas criadas antes de 15 de julho de 2025; nas contas novas, consome os créditos do Free plan. Para o preço por hora, use o AWS Pricing Calculator. O ElastiCache Serverless cobra desde o primeiro minuto. No fim, tudo é desligado.

Os IDs da VPC de volta

Os IDs do capítulo 10 ficaram em variáveis de um terminal que já foi fechado. Variável de shell tem a durabilidade de um post-it em dia de vento. As tags `Name` resolvem:

```
export AWS_PROFILE=rango-dev
aws sso login --sso-session rango

VPC_ID=$(aws ec2 describe-vpcs --filters Name=tag:Name,Values=rango-vpc \
  --query "Vpcs[0].VpcId" --output text)

id_subnet() {
  aws ec2 describe-subnets --filters Name=tag:Name,Values="$1" \
    --query "Subnets[0].SubnetId" --output text
}
id_sg() {
  aws ec2 describe-security-groups \
    --filters Name=vpc-id,Values="$VPC_ID" Name=group-name,Values="$1" \
    --query "SecurityGroups[0].GroupId" --output text
}

PRIV_A=$(id_subnet rango-privada-a)
PRIV_B=$(id_subnet rango-privada-b)
SG_WEB=$(id_sg rango-web)
SG_DB=$(id_sg rango-db)
echo "$VPC_ID $PRIV_A $PRIV_B $SG_WEB $SG_DB"
```

Se algum valor sair `None`, o recurso correspondente não existe mais, e precisa ser recriado com os comandos do capítulo 10.

Subnet group e parameter group

O RDS não recebe uma subnet: recebe um **DB subnet group**, uma lista de subnets em pelo menos duas AZs, onde ele pode colocar a primária e, se houver, a standby. É por isso que o capítulo 10 avisou que o RDS exige duas AZs, mesmo num banco Single-AZ. É o restaurante que exige reserva para dois quando você vai jantar sozinho.

```
aws rds create-db-subnet-group \
  --db-subnet-group-name rango-db-subnets \
  --db-subnet-group-description "Subnets privadas do Rango" \
  --subnet-ids "$PRIV_A" "$PRIV_B" \
  --tags Key=Projeto,Value=rango
```

Agora o parameter group, criado antes do banco para evitar o reinício depois. O mesmo passo pede ao Postgres que registre no log toda query que passar de 500 ms, o que transforma o log no dedo-duro oficial das queries lentas:

```
aws rds create-db-parameter-group \
  --db-parameter-group-name rango-pg18 \
  --db-parameter-group-family postgres18 \
  --description "Parametros do Postgres do Rango" \
  --tags Key=Projeto,Value=rango

aws rds modify-db-parameter-group \
  --db-parameter-group-name rango-pg18 \
  --parameters
"ParameterName=log_min_duration_statement,ParameterValue=500,ApplyMethod=immediate"
```

A consulta abaixo confirma o TLS obrigatório:

```
aws rds describe-db-parameters --db-parameter-group-name rango-pg18 \
  --query "Parameters[?ParameterName=='rds.force_ssl'].ParameterValue" --output text
```

```
1
```

O banco

Primeiro, a versão minor mais recente do PostgreSQL 18 disponível na região:

```
VERSAO=$(aws rds describe-db-engine-versions --engine postgres \
  --query "DBEngineVersions[?starts_with(EngineVersion, '18.')].EngineVersion | [-1]" \
  --output text)
echo "$VERSAO"
```

```
18.4
```

O seu número pode ser maior. Agora o comando principal do capítulo, com mais bandeiras do que desfile de Sete de Setembro:

```
aws rds create-db-instance \
  --db-instance-identifier rango-db \
  --engine postgres \
  --engine-version "$VERSAO" \
  --db-instance-class db.t4g.micro \
  --storage-type gp3 \
  --allocated-storage 20 \
  --max-allocated-storage 100 \
  --storage-encrypted \
  --master-username rango_admin \
  --manage-master-user-password \
  --db-name rango \
  --db-subnet-group-name rango-db-subnets \
  --vpc-security-group-ids "$SG_DB" \
  --no-publicly-accessible \
  --db-parameter-group-name rango-pg18 \
  --backup-retention-period 7 \
  --preferred-backup-window 06:00-06:30 \
  --preferred-maintenance-window sun:07:00-sun:07:30 \
  --auto-minor-version-upgrade \
  --copy-tags-to-snapshot \
  --deletion-protection \
```

```
  --tags Key=Projeto,Value=rango \
  --query "DBInstance.DBInstanceStatus" --output text
```

```
creating
```

Cada opção responde a um pedaço da madrugada do início do capítulo. O par `--allocated-storage 20` e `--max-allocated-storage 100` liga o storage autoscaling: começa com 20 GiB e cresce sozinho até 100. O `--backup-retention-period 7` mantém uma semana de point-in-time recovery. As janelas estão em UTC: 06:00 UTC são 3h em Brasília, a hora com menos pedidos, porque até a fome tem horário de descanso; a manutenção fica no domingo, logo depois, sem sobrepor o backup. O `--auto-minor-version-upgrade` cuida dos patches de segurança.

O `--no-publicly-accessible` impede que o RDS dê um IP público ao banco. O Security Group é o `rango-db`, que já aceita 5432 só de quem está no `rango-web`. O `--deletion-protection` faz o `delete-db-instance` falhar até alguém desligar a proteção de propósito. É a tampinha de plástico em cima do botão vermelho.

E o `--manage-master-user-password` faz o RDS gerar a senha do usuário mestre e guardá-la no AWS Secrets Manager. A senha nunca aparece no seu terminal, no histórico do shell ou neste livro. O segredo tem uma cobrança mensal pequena, e o capítulo 16 explica por que essa senha fica no Secrets Manager enquanto o resto da configuração do Rangô vai para o Parameter Store.

A criação leva uns dez minutos, tempo para um café e meia reunião que poderia ter sido um e-mail:

```
aws rds wait db-instance-available --db-instance-identifier rango-db

aws rds describe-db-instances --db-instance-identifier rango-db \
  --query "DBInstances[0].
[Endpoint.Address,AvailabilityZone,MasterUserSecret.SecretArn]" \
  --output text
```

```
rango-db.c1abcd2efgh3.us-east-1.rds.amazonaws.com   us-east-1a
arn:aws:secretsmanager:us-east-1:123456789012:secret:rds!db-0f1e2d3c-...
```

O endereço é um nome DNS, e a aplicação sempre se conecta por ele, nunca pelo IP. É esse nome que muda de destino num failover. Quem guarda o IP no código descobre isso no primeiro failover, quase sempre de madrugada.

A conexão a partir da API, com TLS

O acesso à `rango-api-01` é pelo Session Manager:

```
aws ssm start-session --target "$API_ID"
```

Dentro da sessão, o primeiro passo instala o cliente e baixa o pacote de certificados das autoridades do RDS. Com ele, o `psql` confere que está falando com o banco de verdade, e não com alguém no meio do caminho, exatamente como o navegador faz com um site no capítulo 2:

```
sudo dnf install -y postgresql17
curl -sS -o /tmp/rds-ca.pem https://truststore.pki.rds.amazonaws.com/global/global-
bundle.pem
sudo install -m 644 /tmp/rds-ca.pem /etc/rango/rds-ca.pem
```

Em seguida vem a senha do mestre, lida no **seu terminal** (a role `rango-ec2` não tem permissão para ler segredos, e assim deve continuar):

```
aws secretsmanager get-secret-value \
  --secret-id "$(aws rds describe-db-instances --db-instance-identifier rango-db \
    --query 'DBInstances[0].MasterUserSecret.SecretArn' --output text)" \
  --query SecretString --output text
```

O resultado é um JSON com `username` e `password`. A senha vai só para o prompt do `psql` a seguir, e o terminal deve ser limpo depois. Senha esquecida no histórico de rolagem do terminal é chave deixada do lado de fora da fechadura. De volta à sessão da API:

```
DB_HOST=rango-db.c1abcd2efgh3.us-east-1.rds.amazonaws.com
psql "host=$DB_HOST dbname=rango user=rango_admin sslmode=verify-full
sslrootcert=/etc/rango/rds-ca.pem"
```

```
Password for user rango_admin:
psql (17.6, server 18.4)
WARNING: psql major version 17, server major version 18.
         Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

rango=>
```

O aviso vem do cliente 17 do Amazon Linux 2023 falando com o servidor 18, e não atrapalha aqui. A linha que importa é a do `SSL connection`. O teste negativo mostra o outro lado: saindo com `\q` e tentando de novo com `sslmode=disable` no lugar de `sslmode=verify-full`, o banco recusa a conexão com um erro de `pg_hba.conf` citando `no encryption`. É o `rds.force_ssl` trabalhando, com a flexibilidade de um fiscal de prova de concurso.

O usuário da aplicação e a tabela de pedidos

A API usa um usuário próprio, com menos poder, como a role `rango-ec2` tem menos poder que o seu permission set. Conectado como `rango_admin`:

```
CREATE ROLE rango LOGIN;
\password rango
GRANT CREATE ON SCHEMA public TO rango;
\q
```

O `\password` funciona como no capítulo 10: pede a senha duas vezes e envia só o hash. O `GRANT CREATE` existe porque, desde o PostgreSQL 15, usuários comuns não podem mais criar tabelas no schema `public` sem permissão explícita. O schema `public` deixou de ser praça pública e virou condomínio fechado.

No repositório, a migração fica em `backend/sql/001_pedidos.sql`:

```
CREATE TABLE IF NOT EXISTS pedidos (
    id          uuid PRIMARY KEY,
    usuario     text NOT NULL,
    restaurante text NOT NULL,
    total       numeric(10, 2) NOT NULL,
    cobranca    text,
    status      text NOT NULL DEFAULT 'recebido',
    criado_em   timestamptz NOT NULL DEFAULT now()
);
```

Depois de um `git pull` no servidor, ela é aplicada como `rango`, para que a tabela pertença ao usuário da aplicação:

```
psql "host=$DB_HOST dbname=rango user=rango sslmode=verify-full
sslrootcert=/etc/rango/rds-ca.pem" \
  -f /opt/rango/codigo/backend/sql/001_pedidos.sql
```

Num projeto maior, essa pasta vira um Alembic. Para uma tabela, um arquivo numerado basta. Ferramenta de migração para uma tabela só seria como contratar caminhão de mudança para levar uma cadeira.

Os pedidos saem do Redis

O driver entra no `backend/requirements.txt`:

```
psycopg[binary,pool]
```

O pool de conexões fica em `backend/app/banco.py`:

```
import os

from psycopg.rows import dict_row
from psycopg_pool import ConnectionPool

pool = ConnectionPool(
    os.environ["DATABASE_URL"],
    min_size=1,
    max_size=5,
    kwargs={"row_factory": dict_row},
```

```
    open=True,
)
```

O `max_size=5` é a conta do capítulo 4 feita ao contrário: dois workers do Uvicorn por instância, cinco conexões cada, dez conexões por servidor. Com o `max_connections` de uma `db.t4g.micro`, sobra espaço para os workers de eventos, para você no `psql` e para algumas instâncias a mais no capítulo 13. Planejar conexões de banco é dividir pizza em festa: quem chega por último sempre acha que a conta foi mal feita.

Em `backend/app/main.py`, a linha do `cache.hset` dá lugar a um `INSERT`, e uma rota nova lê o pedido:

```
from app.banco import pool

# dentro de criar_pedido, no lugar de cache.hset(f"pedido:{pedido['id']}", ...)
    with pool.connection() as conn:
        conn.execute(
            """
            INSERT INTO pedidos (id, usuario, restaurante, total, cobranca)
            VALUES (%s, %s, %s, %s, %s)
            """,
            (pedido["id"], novo.usuario, novo.restaurante, novo.total,
pedido["cobranca"]),
        )
    cache.xadd(STREAM_PEDIDOS, {"tipo": "pedido.criado", "dados": json.dumps(pedido)})


@app.get("/pedidos/{pedido_id}")
def ver_pedido(pedido_id: uuid.UUID):
    with pool.connection() as conn:
        pedido = conn.execute(
            "SELECT id, restaurante, total, status, criado_em FROM pedidos WHERE id =
%s",
            (pedido_id,),
        ).fetchone()
    if pedido is None:
        raise HTTPException(404, "Pedido não encontrado")
    return pedido
```

O `with pool.connection()` pega uma conexão emprestada, faz commit ao sair do bloco sem erro e devolve a conexão ao pool. A idempotência continua no Valkey, onde o `SET NX` atômico funciona bem. Repare que gravar no Postgres e depois publicar no stream ainda é a escrita dupla do capítulo 5: se a API morrer entre as duas linhas, o pedido existe e o evento não. É o tipo de inconsistência que espera a sexta-feira de casa cheia para aparecer. O transactional outbox resolve isso, e o capítulo 21 volta ao assunto com filas da AWS.

No servidor, o arquivo de ambiente é editado com `sudoedit /etc/rango/api.env` (assim a senha não passa pelo histórico) e ganha duas linhas:

```
DATABASE_URL=postgresql://rango@rango-db.c1abcd2efgh3.us-east-
1.rds.amazonaws.com:5432/rango?sslmode=verify-full&sslrootcert=/etc/rango/rds-ca.pem
```

```
PGPASSWORD=a-senha-que-voce-definiu-no-password
```

O driver lê o `PGPASSWORD` do ambiente, como o `psql`. O arquivo já é `640`, dono `root:rango`, desde o capítulo 9. A senha fica nesse arquivo até o capítulo 16 tirá-la dali. Depois vêm as dependências e o reinício dos serviços:

```
sudo -u rango /opt/rango/venv/bin/pip install -r
/opt/rango/codigo/backend/requirements.txt
sudo systemctl restart rango-api rango-worker@restaurante rango-worker@email
```

Três pedidos entram pelo Nginx local, feitos por Ana, Bruno e Carla, a clientela fictícia mais fiel do país:

```
for USUARIO in ana bruno carla; do
  curl -s -X POST http://127.0.0.1/pedidos \
    -H "Content-Type: application/json" \
    -H "Idempotency-Key: $(uuidgen)" \
    -d "{\"usuario\": \"$USUARIO\", \"restaurante\": \"cantina-da-nona\", \"total\":
58.90}"
  echo
done
```

```
{"id":"6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b","status":"recebido"}
{"id":"0a9b8c7d-6e5f-4a3b-8c2d-1e0f9a8b7c6d","status":"recebido"}
{"id":"d4e5f6a7-b8c9-4d0e-9f1a-2b3c4d5e6f70","status":"recebido"}
```

A conferência é um `curl -s http://127.0.0.1/pedidos/6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b` (com o ID que voltou para você). Os pedidos agora estão num banco com backup contínuo.

O DELETE sem WHERE

Hora de testar o backup do jeito que o capítulo 6 recomendou: restaurando. Backup que nunca foi restaurado é só um arquivo com autoestima. O primeiro passo anota o horário atual, em UTC:

```
ANTES=$(date -u +%Y-%m-%dT%H:%M:%SZ)
echo "$ANTES"
```

```
2026-10-15T14:32:10Z
```

Agora o erro clássico, cometido por quem está conectado como `rango`:

```
psql "host=$DB_HOST dbname=rango user=rango sslmode=verify-full
sslrootcert=/etc/rango/rds-ca.pem" \
  -c "DELETE FROM pedidos;"
```

```
DELETE 3
```

Os três pedidos sumiram. Em algum lugar do mundo, um DBA sentiu um calafrio sem saber por quê. No seu terminal (os comandos de `rds` rodam com o profile `rango-dev`, de fora do servidor), é preciso esperar o `LatestRestorableTime` passar do horário anotado, o que leva até uns cinco minutos:

```
aws rds describe-db-instances --db-instance-identifier rango-db \
  --query "DBInstances[0].LatestRestorableTime" --output text
```

Quando ele passa, a restauração vai para um banco novo, com o horário de antes do `DELETE`, que é o valor de `ANTES` anotado acima:

```
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier rango-db \
  --target-db-instance-identifier rango-db-resgate \
  --restore-time 2026-10-15T14:32:10Z \
  --db-instance-class db.t4g.micro \
  --db-subnet-group-name rango-db-subnets \
  --vpc-security-group-ids "$SG_DB" \
  --db-parameter-group-name rango-pg18 \
  --no-publicly-accessible \
  --no-multi-az \
  --tags Key=Projeto,Value=rango \
  --query "DBInstance.DBInstanceStatus" --output text

aws rds wait db-instance-available --db-instance-identifier rango-db-resgate
```

Essa espera merece cronômetro: ela é a maior parte do seu RTO num acidente desse tipo, e costuma passar de dez minutos mesmo num banco quase vazio. Sem os parâmetros de rede, o banco restaurado iria para o Security Group padrão da VPC, apareceria como `available`, e a API receberia timeout. Às 3h, isso custa mais meia hora de investigação e um pedaço considerável da sua fé na humanidade.

Com o resgate disponível, os pedidos voltam por cópia. Na sessão da API, o endpoint novo (ele tem o mesmo sufixo, com `rango-db-resgate` no começo) entra numa variável:

```
RESGATE=rango-db-resgate.c1abcd2efgh3.us-east-1.rds.amazonaws.com
TLS="sslmode=verify-full sslrootcert=/etc/rango/rds-ca.pem"

psql "host=$RESGATE dbname=rango user=rango $TLS" \
  -c "\copy pedidos TO '/tmp/pedidos.csv' CSV"
psql "host=$DB_HOST dbname=rango user=rango $TLS" \
  -c "\copy pedidos FROM '/tmp/pedidos.csv' CSV"
psql "host=$DB_HOST dbname=rango user=rango $TLS" \
  -c "SELECT usuario, total FROM pedidos ORDER BY usuario;"
```

```
 usuario | total
---------+-------
 ana     | 58.90
 bruno   | 58.90
 carla   | 58.90
(3 rows)
```

O `\copy` roda do lado do cliente, então o arquivo fica na EC2. Num incidente real, é o caminho mais seguro: a produção segue recebendo pedidos, e você recupera só as linhas perdidas. Apontar a API para o banco restaurado perderia tudo o que foi escrito depois do horário escolhido. Seria consertar o vazamento da pia inundando a cozinha.

Depois, o resgate e o CSV são apagados:

```
rm /tmp/pedidos.csv
```

```
aws rds delete-db-instance --db-instance-identifier rango-db-resgate \
  --skip-final-snapshot --delete-automated-backups
```

Opcional: o failover acontecendo

Este passo dobra o custo de instância enquanto estiver ligado. Converter para Multi-AZ não derruba o banco, mas leva vários minutos:

```
aws rds modify-db-instance --db-instance-identifier rango-db \
  --multi-az --apply-immediately
aws rds wait db-instance-available --db-instance-identifier rango-db
```

Na sessão da API, o `dig` mostra para onde o nome aponta:

```
dig +short "$DB_HOST"
```

```
10.20.10.84
```

Um IP da `rango-privada-a`. Agora, no seu terminal, uma falha forçada. Derrubar o próprio banco de propósito traz uma satisfação que nenhum terapeuta recomendaria:

```
aws rds reboot-db-instance --db-instance-identifier rango-db --force-failover
```

Enquanto isso, na API, um laço tenta ler um pedido a cada segundo e mostra o horário de cada resposta:

```
while true; do
  printf '%s ' "$(date +%T)"
  curl -s -o /dev/null -w '%{http_code}\n' --max-time 5 \
    http://127.0.0.1/pedidos/6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b
  sleep 1
done
```

A saída mostra `200`, depois alguns `500` (o pool usando conexões que morreram com a primária antiga), e de novo `200`. O intervalo entre eles é o seu tempo real de indisponibilidade. Depois de um Ctrl+C, o `dig` repetido mostra:

```
10.20.11.37
```

O mesmo nome agora leva à `rango-privada-b`, sem nenhuma mudança na aplicação. Os `500` no meio pedem um pool que teste conexões antes de usá-las e retentativa com backoff, como no capítulo 5. Por fim, o banco volta para Single-AZ:

```
aws rds modify-db-instance --db-instance-identifier rango-db \
  --no-multi-az --apply-immediately
```

O Valkey sai da EC2

Primeiro, um Security Group para o cache, aceitando só a API:

```
SG_CACHE=$(aws ec2 create-security-group --vpc-id "$VPC_ID" \
  --group-name rango-cache --description "Valkey do Rango, so a partir da API" \
  --tag-specifications 'ResourceType=security-group,Tags=[{Key=Name,Value=rango-cache},
{Key=Projeto,Value=rango}]' \
  --query GroupId --output text)

aws ec2 authorize-security-group-ingress --group-id "$SG_CACHE" \
  --protocol tcp --port 6379 --source-group "$SG_WEB"
```

E o cache serverless, com limites que funcionam como teto de custo:

```
aws elasticache create-serverless-cache \
  --serverless-cache-name rango-cache \
  --engine valkey \
  --subnet-ids "$PRIV_A" "$PRIV_B" \
  --security-group-ids "$SG_CACHE" \
  --cache-usage-limits 'DataStorage={Maximum=1,Unit=GB},ECPUPerSecond={Maximum=5000}' \
  --tags Key=Projeto,Value=rango
```

O status costuma chegar a `available` em poucos minutos, e a consulta abaixo mostra quando:

```
aws elasticache describe-serverless-caches --serverless-cache-name rango-cache \
  --query "ServerlessCaches[0].[Status,Endpoint.Address]" --output text
```

```
available   rango-cache-a1b2c3.serverless.use1.cache.amazonaws.com
```

Na API, o teste usa o próprio Python do Rangô. Repare no `rediss://`, com dois esses: é o esquema com TLS. Uma letra de diferença entre conversa aberta e criptografada é o tipo de detalhe que consome uma tarde inteira de depuração.

```
sudo -u rango /opt/rango/venv/bin/python -c \
  "import redis; print(redis.Redis.from_url('rediss://rango-cache-
a1b2c3.serverless.use1.cache.amazonaws.com:6379').ping())"
```

```
True
```

Com o `REDIS_URL` em `/etc/rango/api.env` trocado para esse endereço, o Valkey local é desligado e os serviços, reiniciados:

```
sudo systemctl disable --now valkey
sudo systemctl restart rango-api rango-worker@restaurante rango-worker@email
```

As units do capítulo 9 têm `Wants=valkey.service`, e o systemd tentaria subir o Valkey local de novo no próximo boot. Essa dependência sai das units (e o `valkey` sai do `infra/ec2/user-data.sh`) quando a mudança for consolidada.

Um `POST /pedidos` enviado duas vezes com o mesmo `Idempotency-Key` recebe na segunda vez a mesma resposta da primeira: a idempotência agora vive fora do servidor.

O Compose na mesma versão do RDS

O código da API agora exige um `DATABASE_URL`, e o Compose da sua máquina ainda roda o PostgreSQL 17 do capítulo 6. Com uma major de diferença, algo pode funcionar no RDS e quebrar no notebook. O serviço `db` do `compose.yaml` sobe para o 18:

```
  db:
    image: postgres:18-alpine
    # ... environment e secrets como no capítulo 6
    volumes:
      - dados-db:/var/lib/postgresql
```

A linha do volume também mudou. A partir do 18, a imagem oficial guarda os dados num subdiretório por versão, dentro de `/var/lib/postgresql`, e se recusa a subir quando encontra um volume montado no antigo `/var/lib/postgresql/data`. Uma linha de YAML mudou, e é sempre uma linha de YAML.

O volume `dados-db` guarda arquivos no formato do 17, que o servidor 18 não abre. Ali só há dados de teste, incluindo a `pedidos` do capítulo 6 com `id bigserial`, incompatível com a tabela nova, então o caminho mais curto é apagar o volume. Se quiser guardar algo, faça antes um `pg_dump` com o script de backup do capítulo 6 e restaure no 18 com `pg_restore`; restaurar o dump de uma versão anterior numa mais nova é suportado.

```
docker compose down -v
```

No serviço `api` (e no `api-canario`, se ele estiver ligado), a imagem muda e a conexão entra. A senha continua no arquivo do capítulo 6 e entra pela variável do shell, sem ser escrita no `compose.yaml`:

```
    image: rango-api:0.7.0
    environment:
```

```
      # ... as variáveis do capítulo 11
      DATABASE_URL: "postgresql://rango@db:5432/rango"
      PGPASSWORD: "${DB_PASSWORD:?defina DB_PASSWORD}"
```

A sequência constrói a imagem nova, que já traz o `psycopg`, sobe o banco, aplica a migração e sobe o resto:

```
docker build -t rango-api:0.7.0 backend
export DB_PASSWORD="$(cat secrets/db_password.txt)"
docker compose up -d db
docker compose exec -T db psql -U rango -d rango < backend/sql/001_pedidos.sql
docker compose up -d --scale api=3
```

Um `POST /pedidos` em `https://api.rango.test:8443` agora grava no Postgres local, na mesma major do `rango-db`. No fim, um `docker compose down` encerra tudo.

A desmontagem

O banco vai ser recriado nos próximos capítulos, a partir de um snapshot. A proteção contra exclusão é desligada (a tampinha sai do botão vermelho, agora de propósito), e a instância é apagada com um snapshot final:

```
aws rds modify-db-instance --db-instance-identifier rango-db \
  --no-deletion-protection --apply-immediately

aws rds delete-db-instance --db-instance-identifier rango-db \
  --final-db-snapshot-identifier rango-db-final-cap12

aws elasticache delete-serverless-cache --serverless-cache-name rango-cache

aws ec2 terminate-instances --instance-ids "$API_ID"
```

O snapshot final fica e cobra armazenamento de backup por GB-mês, poucos centavos para um banco desse tamanho. O subnet group, o parameter group e o Security Group `rango-cache` não cobram nada e ficam para o capítulo 13. A última checagem confirma que não sobrou instância rodando (instância esquecida é torneira pingando na nuvem, que ninguém ouve até a fatura chegar cheia):

```
aws rds describe-db-instances --query "DBInstances[].
[DBInstanceIdentifier,DBInstanceStatus]" --output text
aws rds describe-db-snapshots --db-snapshot-identifier rango-db-final-cap12 \
  --query "DBSnapshots[0].Status" --output text
```

Armadilhas e casos reais

> **Armadilha:** Versão esquecida vira cobrança extra. Quando uma major sai do standard support, o RDS inscreve o banco no Extended Support automaticamente e começa a cobrar por vCPU-hora. Para o PostgreSQL 12, a AWS lista, em `us-east-2`, US$ 0,100 por vCPU-hora de março de 2025 a fevereiro de 2027 e US$ 0,200 a partir de março de 2027 (valores de referência, consultados em outubro de 2026). Num `db.r6g.xlarge`, com 4 vCPUs, isso dá cerca de US$ 290 por mês só no primeiro período, além do preço da instância. Coloque o fim do standard support da sua versão no calendário do time.
>
> **Armadilha:** A `db.t4g.micro` também é burstable. No RDS, as classes T3 e T4g rodam em modo *unlimited*: quando os créditos de CPU acabam, o banco continua rápido e a AWS cobra o excedente, a US$ 0,075 por vCPU-hora em todas as regiões. Um relatório pesado rodando no banco de produção aparece na fatura. Se o uso de CPU fica alto o tempo todo, a família `m` sai mais barata. O modo *unlimited* faz jus ao nome nos dois sentidos.
>
> **Armadilha:** Apagar a instância apaga os backups automáticos. O `delete-db-instance` remove os snapshots automáticos e os logs de PITR, a menos que você use `--no-delete-automated-backups`. Só sobrevivem os snapshots manuais e o snapshot final.
>
> **Armadilha:** "Multi-AZ é minha réplica de leitura." Na Multi-AZ DB instance, a standby não atende nenhuma consulta. Quem quer aliviar leituras precisa de read replica ou de Multi-AZ DB cluster. E nenhuma das duas protege contra um `DELETE` sem `WHERE`: o erro é replicado em milissegundos para todas as cópias. A replicação é fiel: copia o acerto e o desastre com o mesmo capricho. Para isso, só o PITR.
>
> **Armadilha:** Banco público "só para conectar do notebook". O `--publicly-accessible` mais uma regra de 5432 para `0.0.0.0/0` recria o banco exposto do capítulo 10, agora com o seu endpoint num domínio conhecido da AWS. Para acessar do seu computador, use o Session Manager com *port forwarding* para um host remoto (`AWS-StartPortForwardingSessionToRemoteHost`), passando pela `rango-api-01`.

Resumo

- O RDS assume sistema operacional, instalação, backups, patches e failover. Esquema, queries, usuários, rede e configurações continuam com você.

- Sem acesso ao servidor, as configurações ficam num DB parameter group customizado, criado antes do banco.
- Depois do standard support (até 28 de fevereiro de 2031 para o PostgreSQL 18), vem o Extended Support, pago e automático. Minors chegam pela janela de manutenção.
- Storage autoscaling aumenta o disco até um teto definido, e nunca diminui.
- Backups automáticos guardam de 1 a 35 dias, com logs enviados a cada cinco minutos. O point-in-time recovery sempre cria uma instância nova.
- Multi-AZ DB instance: uma standby síncrona que não atende consultas. Multi-AZ DB cluster: duas standbys legíveis, failover típico abaixo de 35 segundos. Read replica: assíncrona, para aliviar leituras, com `ReplicaLag`.
- Aurora usa um volume de cluster com seis cópias em três AZs. O Serverless v2 pode pausar com 0 ACU, e a retomada leva uns 15 segundos.
- ElastiCache Serverless com Valkey cobra por GB-hora e ECPU, exige TLS e tem piso de uns US$ 6 por mês.
- O Rangô gravou pedidos no `rango-db` (PostgreSQL 18, `db.t4g.micro`, subnets privadas, TLS obrigatório), recuperou um `DELETE` sem `WHERE` com PITR e levou o Valkey para o `rango-cache`. Tudo foi desligado, com o snapshot final `rango-db-final-cap12` guardado.

Nenhum dado mora mais na API

Olhe o que sobrou dentro da `rango-api-01`: o código, o Nginx e o Uvicorn. As fotos estão no S3, os pedidos no RDS, os carrinhos, as chaves de idempotência e os eventos no ElastiCache. Se a instância sumir agora, nada se perde. Essa é a aplicação stateless do capítulo 4, e ela finalmente existe na AWS.

E chega a notícia: o programa de culinária vai fazer um especial com a Cantina da Nona, no sábado à noite. Da última vez, um servidor só caiu em cinco minutos, e a Nona ainda não esqueceu. Agora dá para ter dez cópias da API, mas alguém precisa distribuir os pedidos entre elas, tirar da roda a que travar e criar mais cópias quando o tráfego crescer, sem você acordado ao lado do console. No próximo capítulo, o Rangô ganha um Application Load Balancer e um Auto Scaling Group, e voltamos à TV para aguentar o tráfego desta vez.

Referências

- Amazon RDS, guia do usuário: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html

- Calendário de versões do PostgreSQL no RDS (standard e Extended Support): https://docs.aws.amazon.com/AmazonRDS/latest/PostgreSQLReleaseNotes/postgresql-release-calendar.html
- SSL/TLS com PostgreSQL no RDS (`rds.force_ssl`): https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/PostgreSQL.Concepts.General.SSL.html
- Período de retenção de backup: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.BackupRetention.html
- Recursos do Multi-AZ: https://aws.amazon.com/rds/features/multi-az
- Single-AZ, Multi-AZ instance ou Multi-AZ cluster (AWS Database Blog): https://aws.amazon.com/blogs/database/choose-the-right-amazon-rds-deployment-option-single-az-instance-multi-az-instance-or-multi-az-database-cluster/
- Preços do RDS para PostgreSQL (Extended Support, créditos de CPU): https://aws.amazon.com/rds/postgresql/pricing/
- Aurora Serverless v2, pausa automática: https://docs.aws.eu/AmazonRDS/latest/AuroraUserGuide/aurora-serverless-v2-auto-pause.html
- Preços do ElastiCache: https://aws.amazon.com/elasticache/pricing/
- Configuração e limites de Valkey e Redis OSS no ElastiCache (cluster mode no Serverless): https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/RedisConfiguration.html
- Primeiros passos com ElastiCache para Valkey (AWS Database Blog): https://aws.amazon.com/blogs/database/get-started-with-amazon-elasticache-for-valkey/
- Free tier de bancos de dados da AWS: https://aws.amazon.com/free/database/

CAPÍTULO 13

Load Balancer e Auto Scaling

No capítulo anterior, os pedidos foram para o RDS e o Valkey saiu da EC2 para o ElastiCache Serverless. Com as fotos no S3 desde o capítulo 11, a `rango-api-01` ficou só com o código, o Nginx e o Uvicorn: se ela sumir, nada se perde. Terminamos com o banco desligado, guardado no snapshot `rango-db-final-cap12`, e com uma notícia: o especial de TV com a Cantina da Nona vai ao ar no sábado à noite. A Nona já comprou roupa nova.

O plano das dez instâncias

Quinta-feira, você abre uma planilha e faz a conta do capítulo 4. Na última vez em que a Cantina da Nona apareceu na TV, o tráfego foi de 5 para 250 requisições por segundo em poucos minutos. Cada instância `t3.micro` roda dois workers do Uvicorn, e cada worker atende umas 12 requisições por segundo. Dez instâncias, com folga. Planilha nunca erra; quem erra é a realidade, que insiste em não ler a planilha.

O primeiro plano que vem à cabeça é subir dez `rango-api` à mão no sábado às 20h, com o comando do capítulo 10. Vale a pena seguir esse plano no papel até o fim da noite, porque cada passo dele quebra em algum lugar.

Às 20h, você tem dez IPs públicos, e o capítulo 4 já descartou colocar dez registros A no DNS: quem guardou o IP de uma instância travada continua mandando pedidos para ela até o TTL vencer. Às 21h20, a instância 7 fica sem memória. Alguém precisa perceber, tirar ela da lista e subir outra. Esse alguém é você, na sala, com o programa passando ao fundo e a família perguntando por que você não larga o notebook nem no sábado.

Às 21h40, o apresentador repete o cupom e o tráfego dobra. Você sobe mais cinco instâncias, cada uma levando uns três minutos para ficar pronta. À meia-noite, o tráfego volta para 5 requisições por segundo, e quinze instâncias continuam ligadas até a segunda-feira, junto com a fatura. Você assistiu ao programa inteiro e não lembra de nenhuma receita.

O que falta tem três partes bem separadas: um endereço único que distribui as requisições e sabe quem está saudável, uma receita de como criar uma instância do Rangô sem ninguém entrar nela, e um processo que olha a carga e decide quantas

instâncias precisam existir. A AWS tem um serviço para cada uma. Ela tem serviço para quase tudo, inclusive um catálogo para organizar os próprios serviços.

As peças da AWS

Elastic Load Balancing

O Elastic Load Balancing é a família de balanceadores gerenciados da AWS. Dela, dois tipos interessam aqui, e eles correspondem às camadas que você viu no capítulo 4. O **Application Load Balancer** (ALB) trabalha na camada 7: lê o HTTP, sabe o path e o cabeçalho `Host`, e pode decidir para onde mandar cada requisição com base neles. O **Network Load Balancer** (NLB) trabalha na camada 4: repassa conexões TCP e UDP sem abrir o conteúdo, com latência menor e IPs fixos por AZ. Existe ainda o Gateway Load Balancer, para appliances de rede, e o Classic Load Balancer, a geração antiga, que você só vai encontrar em contas legadas, geralmente ao lado de um README de 2014 assinado por alguém que já saiu da empresa.

Para uma API HTTP como a do Rangô, a escolha é o ALB. O NLB fica para quando você precisar de velocidade e não fizer questão de ler a correspondência dos outros.

O ALB tem uma característica que surpreende quem vem do Nginx do capítulo 4: por trás dele existe um conjunto de nós que a AWS mantém em cada AZ onde você o habilita, e que cresce e encolhe conforme o tráfego. Por isso o ALB recebe um **nome DNS**, algo como `rango-alb-1234567890.us-east-1.elb.amazonaws.com`, e os IPs por trás desse nome mudam com o tempo. A regra é a mesma do endpoint do RDS no capítulo 12: a aplicação e o DNS apontam para o nome, nunca para o IP. O balanceador é um só no nome e vários na prática, como o Papai Noel de shopping em dezembro.

Por dentro, o ALB tem três conceitos encaixados:

- O **listener** escuta uma porta e um protocolo (HTTP na 80, HTTPS na 443).
- As **regras** do listener olham a requisição e escolhem uma ação. Se o path começa com `/admin`, manda para um lugar; se o `Host` é `api.rango.com.br`, para outro. Sempre existe uma ação padrão.
- O **target group** é a lista de destinos, com o seu próprio health check. É o bloco `upstream rango_api` do Nginx, só que com API, métricas e registro automático. É o `upstream` que fez faculdade.

Se você pensar no `APIRouter` do FastAPI, o listener é a aplicação, as regras são os decoradores de rota, e o target group é a função que atende. O Rangô vai usar só a ação padrão, mandando tudo para um único target group.

O algoritmo padrão do target group é o round robin. Dá para trocar para *least outstanding requests*, que escolhe o alvo com menos requisições em andamento, útil quando algumas rotas demoram muito mais que outras, mudando o atributo `load_balancing.algorithm.type` para `least_outstanding_requests`. É a lógica de quem escolhe o caixa do mercado com a fila mais curta, em vez de visitar todos por rodízio.

Health checks e draining

O target group pergunta periodicamente a cada alvo se ele está bem, com uma requisição HTTP para um path que você escolhe. É o health check ativo do capítulo 4. Os valores padrão, para alvos do tipo instância, são um intervalo de 30 segundos, timeout de 5 segundos e cinco respostas boas seguidas para considerar um alvo saudável. Com esses números, uma instância nova leva dois minutos e meio até receber a primeira requisição. Para quem está com fome no meio de um pico, dois minutos e meio são uma eternidade. Para o Rangô, vamos usar 10 segundos de intervalo e duas checagens, o que cai para uns 20 segundos.

O path é o `/health` que existe desde o capítulo 1, e ele continua raso de propósito. O capítulo 4 mostrou o que acontece com um health check que testa o banco: um soluço do Postgres faz todos os alvos falharem juntos, e o balanceador tira da roda as instâncias que estavam funcionando. Health check ambicioso é como demitir a cozinha inteira porque o fornecedor atrasou a entrega.

Quando todos os alvos estão doentes, o ALB faz algo que parece estranho na primeira vez: passa a mandar tráfego para todos eles. A documentação chama isso de *fail open*. A lógica é que, se tudo parece quebrado, o problema provavelmente está no health check, e mandar requisições para alvos talvez bons é melhor do que devolver erro para todo mundo. É o otimismo transformado em configuração: na dúvida, confia.

O outro lado é a saída. Quando um alvo é removido do target group, ele entra no estado `draining`: o ALB para de mandar requisições novas, mas deixa as que estão em andamento terminarem. Esse prazo se chama *deregistration delay*, tem padrão de 300 segundos e vai de 0 a 3.600. Se não houver nenhuma requisição em aberto, a remoção termina na hora, mesmo que o console continue mostrando `draining` até o prazo acabar. Se a instância fechar a conexão antes, o cliente recebe um erro 5xx. Para uma API cujas requisições duram milissegundos, cinco minutos de espera só atrasam cada redução de capacidade, e 30 segundos bastam. Com o padrão, o alvo vira aquela visita que se despede na porta e continua conversando por mais meia hora.

Launch Template

A segunda peça é a receita da instância. Um **Launch Template** guarda tudo o que você passava ao `run-instances`: AMI, tipo, Security Groups, instance profile, opções de metadados, disco, tags e user data. Ele tem versões numeradas, e cada alteração cria uma versão nova, como um commit. Se o `run-instances` do capítulo 10 era um `docker run` com quinze flags, o Launch Template é o serviço no `compose.yaml`. Ninguém sente falta de digitar quinze flags às nove da noite de sábado.

> **Armadilha:** Tutoriais antigos usam *launch configurations*, a versão anterior desse recurso. Contas criadas a partir de 1º de outubro de 2024 não conseguem mais criá-las por nenhum meio (console, API, CLI ou CloudFormation), e elas não recebem tipos de instância novos desde 1º de janeiro de 2023. A conta `rango-estudos` é de 2025. Se um `create-launch-configuration` falhar, é isso.

Auto Scaling Group

A terceira peça é o **Auto Scaling Group** (ASG), do Amazon EC2 Auto Scaling. Você informa três números: mínimo, máximo e capacidade desejada. É o orçamento da reforma: o que dá para fazer, o teto que você jurou não passar e o que você queria de verdade. O ASG cria instâncias a partir do Launch Template até chegar à capacidade desejada, espalhando-as pelas subnets e AZs que você indicou, e registra cada uma no target group.

Daí em diante, ele fica comparando o que existe com o que deveria existir. Uma instância morreu? Ele cria outra. Alguém mudou a capacidade desejada de 2 para 4? Ele cria duas. Tem a persistência de um despertador com soneca infinita. É a reconciliação do Kubernetes que você viu no capítulo 3, aplicada a máquinas virtuais: você declara o estado, e o serviço corre atrás dele.

Para saber se uma instância morreu, o ASG precisa de um critério. Com a opção `--health-check-type ELB`, ele passa a considerar também o veredito do target group, além do estado da instância no EC2. Uma máquina ligada, mas com o Nginx travado, é marcada como doente e substituída, sem atestado e sem segunda chance.

Esse critério precisa de uma pausa no começo. Uma instância recém-criada ainda está iniciando o sistema e os serviços, e falharia no health check. A *health check grace period* é o tempo em que o ASG ignora o veredito do balanceador para uma instância nova. Durante esse período, só uma coisa derruba a instância: sair do estado `running` no EC2. É o período de experiência, com a diferença de que ninguém avalia o recém-chegado antes de ele tomar café.

Políticas de escala

Falta decidir quando a capacidade desejada muda. O EC2 Auto Scaling oferece alguns tipos de política, e dois deles resolvem o sábado do Rangô.

A **target tracking** funciona como o termostato de um ar-condicionado, sem a briga de escritório sobre quem mexeu na temperatura. Você escolhe uma métrica e um valor alvo, e o ASG cria e remove instâncias para manter a métrica perto do alvo. A métrica mais direta para uma API atrás de um ALB é `ALBRequestCountPerTarget`: quantas requisições cada instância recebeu, em média, no último minuto. Se o alvo é 600 e as instâncias estão recebendo 1.200, o ASG dobra a frota. Por baixo, ele cria alarmes no Amazon CloudWatch e os ajusta sozinho; você não mexe neles. A redução é mais conservadora que o aumento, para evitar o efeito sanfona de criar e destruir máquinas a cada minuto.

A **ação agendada** muda o mínimo, o máximo ou a capacidade desejada num horário marcado. O capítulo 4 avisou que auto scaling leva minutos e que, para um pico com hora marcada, o certo é escalar antes. O programa começa às 21h. Às 20h30, o Rangô já pode estar com quatro instâncias, e a target tracking cuida do que passar disso. Escalar antes do pico é como chegar ao restaurante antes da fila do domingo: parece exagero até você ver a fila. Uma instância nova não fica útil no instante em que é criada. O *default instance warmup* diz ao ASG quantos segundos esperar antes de contar as métricas dela na média do grupo. Sem isso, uma instância que acabou de subir e ainda não recebeu tráfego puxa a média para baixo, e a política pode concluir que já existem máquinas suficientes, como o chefe que conta o estagiário do primeiro dia como mais um par de mãos na entrega.

O que ainda guarda estado

Antes de multiplicar o Rangô, vale repassar a lista de estado escondido do capítulo 4, agora contra o código real.

As sessões não existem no Rangô: o carrinho fica no Valkey, num hash por usuário, e qualquer instância lê o mesmo carrinho. Os uploads vão direto do navegador para o S3 com presigned URLs, e a instância só gera a URL e confere o objeto. O cache de `/restaurantes` mora no Valkey, compartilhado. As chaves de idempotência também. Os pedidos estão no RDS. Até aqui, a lista parece aquele check-up em que o médico diz que está tudo ótimo.

Sobram três itens menos óbvios, que são os exames que ainda não voltaram.

O primeiro são os workers. Cada instância roda `rango-worker@restaurante` e `rango-worker@email`, e o nome do consumidor no stream é o hostname da máquina. Com várias instâncias, os workers disputam as mensagens do mesmo grupo, que é o *competing*

consumers do capítulo 5. O problema aparece quando o ASG remove uma instância: as mensagens pendentes do consumidor que sumiu ficam esperando alguém que nunca mais vai voltar com aquele hostname, como prato pronto no balcão de um garçom que pediu demissão. O `XAUTOCLAIM`, citado no capítulo 5, existe para isso, e o capítulo 21 troca o stream pelo Amazon SQS, que devolve a mensagem para a fila sozinho.

O segundo são os logs. O `journalctl -u rango-api` mostra os logs em JSON, guardados no disco da instância. Quando o ASG destrói uma máquina, os logs dela vão junto com o volume. O capítulo 22 manda os logs para o CloudWatch Logs; até lá, um erro numa instância já substituída fica sem explicação, como crime sem testemunha, porque a testemunha foi reciclada.

O terceiro é a configuração, que vamos resolver agora. Até aqui, cada servidor do Rangô foi montado à mão: deploy key, `git clone`, ambiente virtual, `api.env`, units do systemd. Nenhum ASG vai fazer isso às 21h40 por você. Ele é automático, e automação ainda não inclui ler a sua mente.

E se a aplicação tivesse sessão em memória? O ALB oferece *sticky sessions* por target group, com um cookie `AWSALB` que prende o cliente a um alvo por 1 segundo a 7 dias. Se esse alvo ficar doente ou sair, o ALB escolhe outro, e a sessão em memória se perde. É o remendo do capítulo 4, útil para segurar uma aplicação legada enquanto o estado sai da memória. Remendo, aliás, é o material mais usado na construção de sistemas legados.

Na prática: o Rangô atrás de um balanceador

Pré-requisitos: AWS CLI v2 com o profile `rango-dev`, a `rango-vpc` com as quatro subnets, os Security Groups `rango-web`, `rango-db` e `rango-cache`, o subnet group `rango-db-subnets`, o parameter group `rango-pg18`, o snapshot `rango-db-final-cap12` e a role `rango-ec2` com a policy `RangoFotos`. Tudo em `us-east-1`. Para o teste de carga, o k6 instalado no seu computador. A lista é longa porque o Rangô já cresceu; capítulos atrás, o pré-requisito era um notebook e boa vontade.

Sobre custo: o ALB cobra por hora desde a criação, mais o uso, e cada nó dele tem um IPv4 público cobrado à parte. O banco, o cache e as instâncias também. A seção de armadilhas traz os números. No fim, tudo o que cobra por hora é desligado.

Banco e cache de volta

As variáveis voltam com as funções `id_subnet` e `id_sg` do capítulo 12, agora com as subnets públicas e o grupo do cache:

```
export AWS_PROFILE=rango-dev
aws sso login --sso-session rango
```

```
VPC_ID=$(aws ec2 describe-vpcs --filters Name=tag:Name,Values=rango-vpc \
  --query "Vpcs[0].VpcId" --output text)
id_subnet() {
  aws ec2 describe-subnets --filters Name=tag:Name,Values="$1" \
    --query "Subnets[0].SubnetId" --output text
}
id_sg() {
  aws ec2 describe-security-groups \
    --filters Name=vpc-id,Values="$VPC_ID" Name=group-name,Values="$1" \
    --query "SecurityGroups[0].GroupId" --output text
}

PUB_A=$(id_subnet rango-publica-a)
PUB_B=$(id_subnet rango-publica-b)
PRIV_A=$(id_subnet rango-privada-a)
PRIV_B=$(id_subnet rango-privada-b)
SG_WEB=$(id_sg rango-web)
SG_DB=$(id_sg rango-db)
SG_CACHE=$(id_sg rango-cache)
```

O banco volta do snapshot, com o mesmo identificador:

```
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier rango-db \
  --db-snapshot-identifier rango-db-final-cap12 \
  --db-instance-class db.t4g.micro \
  --db-subnet-group-name rango-db-subnets \
  --vpc-security-group-ids "$SG_DB" \
  --db-parameter-group-name rango-pg18 \
  --no-publicly-accessible \
  --no-multi-az \
  --deletion-protection \
  --tags Key=Projeto,Value=rango \
  --query "DBInstance.DBInstanceStatus" --output text

aws rds wait db-instance-available --db-instance-identifier rango-db

aws rds modify-db-instance --db-instance-identifier rango-db \
  --backup-retention-period 7 --max-allocated-storage 100 --apply-immediately
```

Como o identificador é o mesmo, o endpoint também é: `rango-db.c1abcd2efgh3.us-east-1.rds.amazonaws.com`. O sufixo depende da conta e da região, e o `DATABASE_URL` do capítulo 12 continua valendo. O usuário `rango`, a senha dele e os três pedidos de teste vieram no snapshot. Ana, Bruno e Carla sobreviveram a mais uma destruição completa de infraestrutura sem perceber nada. O `modify-db-instance` garante a retenção de backup e o teto de disco que o capítulo 12 tinha configurado.

O cache serverless é criado com o mesmo comando do capítulo 12. O endpoint novo precisa ser anotado, porque ele muda a cada criação:

```
aws elasticache create-serverless-cache \
  --serverless-cache-name rango-cache \
  --engine valkey \
  --subnet-ids "$PRIV_A" "$PRIV_B" \
  --security-group-ids "$SG_CACHE" \
```

```
  --cache-usage-limits 'DataStorage={Maximum=1,Unit=GB},ECPUPerSecond={Maximum=5000}' \
  --tags Key=Projeto,Value=rango
```

Duas mudanças no repositório

O capítulo 12 deixou duas pendências, e o ASG torna as duas urgentes. Pendência é como louça na pia: só vira urgente quando chega visita.

A primeira é o `valkey` no `infra/ec2/user-data.sh`. As instâncias novas não precisam dele. As duas primeiras linhas de comando do script ficam assim:

```
dnf install -y nginx git python3.12 python3.12-pip
systemctl enable --now nginx
```

A segunda é o pool de conexões. No failover do capítulo 12, a API devolveu `500` porque o pool entregou conexões que tinham morrido com a primária antiga. Num grupo com seis instâncias, isso vira `500` em seis lugares ao mesmo tempo. O `psycopg_pool` a partir da versão 3.2 aceita uma função que testa a conexão antes de entregá-la. Em `backend/app/banco.py`:

```
pool = ConnectionPool(
    os.environ["DATABASE_URL"],
    min_size=1,
    max_size=5,
    kwargs={"row_factory": dict_row},
    check=ConnectionPool.check_connection,
    open=True,
)
```

O `check_connection` faz uma ida e volta ao banco a cada empréstimo, coisa de fração de milissegundo dentro da VPC, e troca a conexão se ela estiver morta. É o garçom que confere se o copo está limpo antes de servir, e não depois que o cliente reclamou. Uma conexão que cai no meio de uma consulta ainda gera erro: nas leituras, uma retentativa com o backoff do capítulo 5 resolve; no `INSERT` do pedido, a retentativa fica com o cliente, que já manda o `Idempotency-Key`.

As duas mudanças vão para o repositório com um commit e um push.

A imagem do Rangô

Como uma instância criada pelo ASG ganha código, dependências e configuração? Existem dois caminhos. No primeiro, o user data faz tudo a cada boot: instala pacotes, baixa o código, monta o ambiente. No segundo, você monta uma máquina uma vez, tira uma AMI dela e o ASG cria todas as instâncias a partir dessa imagem. O mercado chama a segunda de *golden AMI*, nome pomposo para "uma foto da máquina depois de arrumada".

O Rangô vai pela AMI. Uma instância que nasce pronta fica saudável em segundos, e no meio de um pico cada minuto de boot pesa. Além disso, as instâncias do ASG vão para as subnets privadas, sem NAT Gateway: não há como rodar `dnf install` nem `git clone` no boot. É a imagem Docker do capítulo 3 aplicada à máquina inteira.

A montagem acontece numa instância na subnet pública, criada com o comando do capítulo 10, com o nome trocado para `rango-base` e o ID guardado em `BASE_ID`. Nela, as seções "Trazendo o código" e "Configuração e serviços" do capítulo 9 se repetem, com duas diferenças.

As units do systemd não dependem mais do Valkey. Depois de criadas, elas passam por este ajuste:

```
sudo sed -i 's/ valkey.service//' \
  /etc/systemd/system/rango-api.service \
  /etc/systemd/system/rango-worker@.service
sudo systemctl daemon-reload
```

E o `/etc/rango/api.env` (editado com `sudoedit`) recebe a configuração completa: a do capítulo 12, o endpoint novo do cache e as variáveis de fotos do capítulo 11, que até agora só existiam no Compose. A conta e o endpoint do cache do exemplo são os do livro; na sua conta, são outros:

```
REDIS_URL=rediss://rango-cache-d4e5f6.serverless.use1.cache.amazonaws.com:6379
CORS_ORIGINS=http://localhost:5173
DATABASE_URL=postgresql://rango@rango-db.c1abcd2efgh3.us-east-
1.rds.amazonaws.com:5432/rango?sslmode=verify-full&sslrootcert=/etc/rango/rds-ca.pem
PGPASSWORD=a-senha-que-voce-definiu-no-password
FOTOS_BUCKET=rango-fotos-pratos-123456789012
AWS_REGION=us-east-1
```

O `/etc/rango/rds-ca.pem` também é baixado, como no capítulo 12. O teste dentro da instância lê um pedido que veio do snapshot:

```
curl -s http://127.0.0.1/pedidos/6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b
```

```
{"id":"6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b","restaurante":"cantina-da-
nona","total":"58.90","status":"recebido","criado_em":"2026-10-15T14:28:41.512309Z"}
```

Antes de tirar a foto da máquina, sai dela o que não pode ir junto. A deploy key serviu para o clone, e as instâncias do ASG nunca vão falar com o GitHub. Credencial que não serve para mais nada é a primeira coisa a sair da mala:

```
sudo rm -rf /opt/rango/.ssh
```

A deploy key também é removida no GitHub. Fora da sessão, no seu terminal, a imagem é criada:

```
AMI_ID=$(aws ec2 create-image \
  --instance-id "$BASE_ID" \
  --name rango-api-20261015 \
  --description "Rango API, cap. 13" \
  --tag-specifications \
    'ResourceType=image,Tags=[{Key=Projeto,Value=rango}]' \
    'ResourceType=snapshot,Tags=[{Key=Projeto,Value=rango}]' \
  --query ImageId --output text)

aws ec2 wait image-available --image-ids "$AMI_ID"
aws ec2 terminate-instances --instance-ids "$BASE_ID"
```

O `create-image` reinicia a instância para que o disco seja copiado num estado consistente, e leva alguns minutos. O snapshot da AMI herda a criptografia do volume. A máquina precisa ficar parada para a foto, como nos retratos de família do século XIX.

> **Armadilha:** Esta AMI carrega o `api.env`, com a senha do banco dentro. Ela é privada da sua conta e o snapshot está criptografado, mas qualquer pessoa da conta com permissão para criar instâncias consegue subir uma cópia e ler o arquivo. Nunca torne uma AMI dessas pública nem a compartilhe com outra conta. O capítulo 16 tira a senha da imagem, e a instância passa a buscá-la no boot.

Security Groups, e um erro de propósito

O ALB precisa de um grupo próprio, aberto para a internet na porta 80:

```
SG_ALB=$(aws ec2 create-security-group --vpc-id "$VPC_ID" \
  --group-name rango-alb --description "ALB do Rango, HTTP publico" \
  --tag-specifications 'ResourceType=security-group,Tags=[{Key=Name,Value=rango-alb},
{Key=Projeto,Value=rango}]' \
  --query GroupId --output text)

aws ec2 authorize-security-group-ingress --group-id "$SG_ALB" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
```

E o `rango-web` deixa de aceitar a internet. Desde o capítulo 9, ele tem a porta 80 aberta para `0.0.0.0/0`. As instâncias vão para subnets privadas e nem teriam como receber esse tráfego, mas a regra não precisa existir:

```
aws ec2 revoke-security-group-ingress --group-id "$SG_WEB" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
```

Falta a regra que deixa o ALB falar com as instâncias. Vamos esquecê-la por enquanto, para ver o sintoma. Esquecer de propósito é muito mais confortável do que do jeito tradicional.

Target group e ALB

```
TG_ARN=$(aws elbv2 create-target-group \
  --name rango-api \
  --protocol HTTP --port 80 \
  --vpc-id "$VPC_ID" \
  --target-type instance \
  --health-check-path /health \
  --health-check-interval-seconds 10 \
  --health-check-timeout-seconds 5 \
  --healthy-threshold-count 2 \
  --unhealthy-threshold-count 2 \
  --matcher HttpCode=200 \
  --tags Key=Projeto,Value=rango \
  --query "TargetGroups[0].TargetGroupArn" --output text)

aws elbv2 modify-target-group-attributes --target-group-arn "$TG_ARN" \
  --attributes Key=deregistration_delay.timeout_seconds,Value=30
```

A porta 80 do target group é a do Nginx em cada instância. O ALB fica nas subnets públicas, uma por AZ:

```
ALB_ARN=$(aws elbv2 create-load-balancer \
  --name rango-alb \
  --type application \
  --scheme internet-facing \
  --subnets "$PUB_A" "$PUB_B" \
  --security-groups "$SG_ALB" \
  --tags Key=Projeto,Value=rango \
  --query "LoadBalancers[0].LoadBalancerArn" --output text)

aws elbv2 wait load-balancer-available --load-balancer-arns "$ALB_ARN"

aws elbv2 create-listener --load-balancer-arn "$ALB_ARN" \
  --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn="$TG_ARN"

ALB_DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns "$ALB_ARN" \
  --query "LoadBalancers[0].DNSName" --output text)
echo "$ALB_DNS"
```

```
rango-alb-1234567890.us-east-1.elb.amazonaws.com
```

Os nós do ALB, nas subnets públicas, recebem a requisição e abrem uma conexão nova até a instância, nas privadas, pela rota `local` da VPC. A instância dispensa IP público e os US$ 3,65 mensais dele. Nunca uma economia de três dólares foi tão elegante.

Launch Template

O Launch Template recebe a AMI e o resto do que estava no `run-instances`. O arquivo `infra/asg/launch-template.json` é gerado a partir das variáveis:

```
mkdir -p infra/asg
cat > infra/asg/launch-template.json <<EOF
```

```
{
  "ImageId": "$AMI_ID",
  "InstanceType": "t3.micro",
  "CreditSpecification": {"CpuCredits": "standard"},
  "IamInstanceProfile": {"Name": "rango-ec2"},
  "SecurityGroupIds": ["$SG_WEB"],
  "MetadataOptions": {"HttpTokens": "required", "HttpEndpoint": "enabled"},
  "BlockDeviceMappings": [{
    "DeviceName": "/dev/xvda",
    "Ebs": {"VolumeSize": 10, "VolumeType": "gp3", "Encrypted": true,
"DeleteOnTermination": true}
  }],
  "TagSpecifications": [
    {"ResourceType": "instance", "Tags": [{"Key": "Name", "Value": "rango-api"}, {"Key":
"Projeto", "Value": "rango"}]},
    {"ResourceType": "volume", "Tags": [{"Key": "Projeto", "Value": "rango"}]}
  ]
}
EOF

aws ec2 create-launch-template \
  --launch-template-name rango-api \
  --version-description "AMI rango-api-20261015" \
  --launch-template-data file://infra/asg/launch-template.json \
  --tag-specifications 'ResourceType=launch-template,Tags=[{Key=Projeto,Value=rango}]'
```

Não há user data: a imagem já sabe tudo, como o aluno que chega no primeiro dia de aula tendo lido a apostila. Repare que o template não diz em qual subnet a instância nasce. Essa decisão é do ASG.

O Auto Scaling Group

```
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name rango-api \
  --launch-template 'LaunchTemplateName=rango-api,Version=$Default' \
  --min-size 2 --max-size 6 --desired-capacity 2 \
  --vpc-zone-identifier "$PRIV_A,$PRIV_B" \
  --target-group-arns "$TG_ARN" \
  --health-check-type ELB \
  --health-check-grace-period 90 \
  --default-instance-warmup 60 \
  --tags Key=Projeto,Value=rango,PropagateAtLaunch=false
```

O mínimo de 2, com duas subnets, deixa uma instância em cada AZ: se uma zona cair, o Rangô segue com metade da capacidade. O `$Default`, entre aspas simples para o shell não expandir, faz o ASG usar a versão marcada como padrão no template.

O máximo de 6 tem a ver com o banco. Cada instância abre até dez conexões (dois workers do Uvicorn com cinco cada), e seis instâncias somam sessenta. A fórmula do `max_connections` do capítulo 12 dá pouco mais de cem para uma `db.t4g.micro`, e uma parte disso fica com os workers, com você no `psql` e com o próprio RDS. Um ASG com

máximo de 20 derrubaria o banco com `FATAL: sorry, too many clients already` exatamente no pico. O teto do grupo protege o banco e a fatura. Escalar a API sem olhar o banco é alargar a estrada que termina numa ponte de uma pista só.

A grace period de 90 segundos dá folga para uma instância que nasce pronta. Pelo console, o padrão é 300 segundos. Pela CLI e pelos SDKs, o padrão é zero, e a seção de armadilhas mostra o que isso causa.

O PassRole da role `rango-ec2` já está liberado no seu permission set desde o capítulo 9. O ASG só aceita um Launch Template com instance profile se quem cria o grupo puder passar aquela role ao EC2, e o statement `PassarSoARoleDoServidor` cobre exatamente isso.

O sintoma do Security Group esquecido

Um minuto depois, a saúde dos alvos conta a história:

```
aws elbv2 describe-target-health --target-group-arn "$TG_ARN" \
  --query "TargetHealthDescriptions[].
[Target.Id,TargetHealth.State,TargetHealth.Reason]" \
  --output text
```

```
i-0a1b2c3d4e5f60718 unhealthy   Target.Timeout
i-0f9e8d7c6b5a40321 unhealthy   Target.Timeout
```

Timeout, como no capítulo 10: o `rango-web` descarta em silêncio o pacote do ALB. Um pouco mais tarde, o histórico do grupo mostra o resto:

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name rango-api \
  --max-items 4 --query "Activities[].Description" --output text
```

Passada a grace period, o ASG confia no veredito do target group, conclui que as duas instâncias estão doentes e as substitui por outras duas, que vão falhar pelo mesmo motivo. Um erro de rede vira um ciclo de criar e destruir máquinas. É Sísifo com cartão de crédito corporativo. A correção é a regra que faltava, tendo o grupo do ALB como origem:

```
aws ec2 authorize-security-group-ingress --group-id "$SG_WEB" \
  --protocol tcp --port 80 --source-group "$SG_ALB"
```

Em menos de um minuto:

```
i-0c4d5e6f7a8b90123 healthy None
i-0d7e8f9a0b1c23456 healthy None
```

As duas instâncias atendem só a quem está no grupo `rango-alb`, como restaurante que só aceita pedido pelo aplicativo. É a mesma regra que o `rango-db` usa para aceitar só a API, um andar acima.

O primeiro curl

Do seu computador, algumas chamadas ao ALB mostram o cabeçalho que o middleware do capítulo 4 coloca em cada resposta:

```
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -D - "http://$ALB_DNS/restaurantes" | grep -i x-rango-instancia
done
```

```
X-Rango-Instancia: ip-10-20-10-37.ec2.internal
X-Rango-Instancia: ip-10-20-11-142.ec2.internal
X-Rango-Instancia: ip-10-20-10-37.ec2.internal
X-Rango-Instancia: ip-10-20-11-142.ec2.internal
X-Rango-Instancia: ip-10-20-10-37.ec2.internal
X-Rango-Instancia: ip-10-20-11-142.ec2.internal
```

Round robin, uma instância em cada AZ. Um `GET /pedidos/6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b` pelo ALB devolve o pedido da Ana, gravado no capítulo 12, de qualquer uma das duas. A Ana, a esta altura, já deve estar com fome de novo.

Uma instância sai da roda

Num terminal, um laço mostra o código de status e a instância de cada resposta:

```
while true; do
  printf '%s ' "$(date +%T)"
  curl -s -o /dev/null -w '%{http_code} %header{x-rango-instancia}\n' \
    --max-time 5 "http://$ALB_DNS/restaurantes"
  sleep 0.5
done
```

O `%header{}` exige `curl` 7.84 ou mais novo. Em outro terminal, o ASG recebe o pedido de encerrar uma das instâncias sem reduzir a capacidade desejada:

```
aws autoscaling terminate-instance-in-auto-scaling-group \
  --instance-id i-0c4d5e6f7a8b90123 \
  --no-should-decrement-desired-capacity
```

O laço continua com `200` o tempo todo. O ASG primeiro tira a instância do target group, ela entra em `draining`, e só depois é encerrada. Em seguida, o ASG percebe que tem uma instância a menos do que o desejado e cria outra. Um minuto e pouco depois, um hostname novo aparece no laço. A instância morreu, viva a instância.

No failover do banco, no capítulo 12, o laço mostrou alguns `500`. A diferença está no draining: o ALB sabe que a instância vai sair antes de ela sair. É a diferença entre aviso prévio e sumiço sem explicação.

A política de escala

A política precisa saber de qual ALB e de qual target group contar as requisições. Esse par vai num campo chamado `ResourceLabel`, montado com o final dos dois ARNs. ARN é como nome completo de família real europeia: comprido, e quase ninguém lembra a parte do meio:

```
LABEL="${ALB_ARN#*:loadbalancer/}/${TG_ARN##*:}"
echo "$LABEL"
```

```
app/rango-alb/50dc6c495c0c9188/targetgroup/rango-api/943f017f100becff
```

O alvo vem da conta do capítulo 4. Cada worker atende umas 12 requisições por segundo, e dois workers dão umas 24 por segundo, ou cerca de 1.400 por minuto por instância. Um alvo de 600 por minuto mantém cada máquina abaixo da metade da capacidade, com espaço para absorver o crescimento enquanto as instâncias novas sobem. Trabalhar a meia carga parece desperdício até o apresentador repetir o cupom.

```
cat > infra/asg/politica-requisicoes.json <<EOF
{
  "TargetValue": 600,
  "PredefinedMetricSpecification": {
    "PredefinedMetricType": "ALBRequestCountPerTarget",
    "ResourceLabel": "$LABEL"
  }
}
EOF

aws autoscaling put-scaling-policy \
  --auto-scaling-group-name rango-api \
  --policy-name requisicoes-por-instancia \
  --policy-type TargetTrackingScaling \
  --target-tracking-configuration file://infra/asg/politica-requisicoes.json
```

A noite da TV, em quinze minutos

O teste de carga fica em `carga/tv.js`, escrito para o k6, a ferramenta citada no capítulo 4:

```
import http from "k6/http";

export const options = {
  vus: 30,
  duration: "15m",
};
```

```
export default function () {
  http.get(`http://${__ENV.ALB}/restaurantes`);
}
```

Trinta usuários virtuais pedindo a lista sem parar, os clientes mais indecisos do mundo: olham o cardápio por quinze minutos e não pedem nada. Do Brasil até `us-east-1`, isso dá na casa de 200 requisições por segundo, perto do pico do programa de TV do capítulo 4. A execução:

```
k6 run -e ALB="$ALB_DNS" carga/tv.js
```

Em outro terminal, a frota é acompanhada a cada minuto:

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names rango-api \
  --query "AutoScalingGroups[0].[DesiredCapacity,length(Instances)]" --output text
```

Nos primeiros minutos, nada muda: o alarme da target tracking precisa ver alguns minutos de métrica acima do alvo. É a hora em que todo mundo atualiza o terminal três vezes e jura que está quebrado. Depois, o histórico conta a decisão:

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name rango-api \
  --max-items 1 --query "Activities[0].Cause" --output text
```

```
At 2026-10-15T18:21:40Z a monitor alarm TargetTracking-rango-api-AlarmHigh-7f3a2b1c in
state ALARM triggered policy requisicoes-por-instancia changing the desired capacity
from 2 to 6. At 2026-10-15T18:21:52Z instances were launched in response to a difference
between desired and running capacity, increasing the capacity from 2 to 6.
```

Cerca de 12.000 requisições por minuto divididas por um alvo de 600 dariam 20 instâncias, e o máximo de 6 cortou o pedido. Raramente alguém diz "não" a um gráfico subindo e acerta, mas aqui acertou. Quando o k6 terminar, o ASG remove instâncias aos poucos até voltar a 2, bem mais devagar do que subiu.

Agendado para sábado

O especial de sábado tem hora marcada. A API de ações agendadas trabalha em UTC, e Brasília está em UTC−3, então 20h30 de sábado, 17 de outubro, são 23h30 UTC. Você provavelmente vai ler isto em outro dia (se estiver lendo no próprio sábado, às 20h29, talvez seja um pouco tarde), e uma data que já passou faz o comando falhar. Por isso, o shell calcula o próximo sábado:

```
SABADO=$(date -u -d 'next saturday' +%Y-%m-%d)
INICIO="${SABADO}T23:30:00Z"
FIM=$(date -u -d "$SABADO 23:30 UTC + 4 hours" +%Y-%m-%dT%H:%M:%SZ)
echo "$INICIO $FIM"

aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name rango-api \
```

```
  --scheduled-action-name especial-tv-inicio \
  --start-time "$INICIO" \
  --min-size 4 --desired-capacity 4

aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name rango-api \
  --scheduled-action-name especial-tv-fim \
  --start-time "$FIM" \
  --min-size 2
```

O `-d` é do `date` do GNU, o do Linux e do CloudShell. No macOS, o trecho roda no CloudShell, ou com o `coreutils` instalado (`brew install coreutils`) e `gdate` no lugar de `date`. As datas que aparecem nas saídas deste capítulo são as da história; no seu terminal, aparecem as suas.

Às 20h30, o mínimo sobe para 4, e a target tracking continua podendo ir até 6. À meia-noite e meia, o mínimo volta a 2, e a política remove o excesso conforme o tráfego cai. Desta vez, quem fica acordado no sábado é só o ASG, e você pode finalmente prestar atenção nas receitas.

A desmontagem

O ALB, o ASG, o banco e o cache cobram por hora. O Launch Template, os Security Groups e a AMI ficam, porque o capítulo 14 reaproveita tudo: o template e os grupos não cobram nada, e o snapshot da AMI cobra centavos por mês. O grupo é apagado primeiro, porque ele traria as instâncias de volta se só elas fossem apagadas. Com o despertador da soneca infinita, o jeito é tirar da tomada:

```
aws autoscaling delete-auto-scaling-group \
  --auto-scaling-group-name rango-api --force-delete

aws elbv2 delete-load-balancer --load-balancer-arn "$ALB_ARN"
aws elbv2 wait load-balancers-deleted --load-balancer-arns "$ALB_ARN"
aws elbv2 delete-target-group --target-group-arn "$TG_ARN"

aws elasticache delete-serverless-cache --serverless-cache-name rango-cache

aws rds modify-db-instance --db-instance-identifier rango-db \
  --no-deletion-protection --apply-immediately
aws rds delete-db-instance --db-instance-identifier rango-db \
  --final-db-snapshot-identifier rango-db-final-cap13
aws rds wait db-instance-deleted --db-instance-identifier rango-db
aws rds delete-db-snapshot --db-snapshot-identifier rango-db-final-cap12
```

O `--force-delete` encerra as instâncias junto com o grupo, e apagar o ASG apaga também a política, os alarmes que ela criou e as ações agendadas. O plano de sábado vai junto, e tudo bem: o capítulo 14 recria o grupo e as duas ações antes do especial. A última consulta confirma que não sobrou instância:

```
aws ec2 describe-instances \
  --filters Name=tag:Projeto,Values=rango Name=instance-state-
```

```
name,Values=pending,running \
  --query "Reservations[].Instances[].InstanceId" --output text
```

Armadilhas e casos reais

Armadilha: Grace period zero pela CLI. Num caso hipotético, mas comum, um time cria o ASG pela CLI sem `--health-check-grace-period`, com `--health-check-type ELB` e um user data que leva quatro minutos instalando pacotes. O padrão pela CLI é zero, então o ASG passa a ouvir o target group assim que a instância nasce. Ela falha no health check porque o Nginx nem foi instalado, é marcada como doente e substituída. A substituta tem o mesmo destino. O grupo fica girando máquinas por horas, cada uma cobrando no mínimo 60 segundos. A AWS recomenda configurar também o `--default-instance-warmup`, como fizemos.

Armadilha: O ALB cobra parado. Em `us-east-1`, o ALB custa US$ 0,0225 por hora (hora parcial conta como cheia), cerca de US$ 16 por mês, mais US$ 0,008 por LCU-hora. A LCU mede o uso em quatro dimensões (25 conexões novas por segundo, 3.000 conexões ativas por minuto, 1 GB por hora processado, 1.000 avaliações de regra por segundo, com as 10 primeiras regras grátis), e só a dimensão de maior uso na hora entra na conta. Cada nó do ALB também tem um IPv4 público, a US$ 0,005 por hora: com duas AZs, mais uns US$ 7 por mês. Um ALB esquecido com zero tráfego passa de US$ 23 por mês. O free tier antigo cobria 750 horas e 15 LCUs por mês; nas contas criadas a partir de 15 de julho de 2025, como a `rango-estudos`, o ALB consome os créditos do Free plan. Valores de referência, consultados em outubro de 2026. **Armadilha:** Instância burstable no ASG. As instâncias do Rangô usam `CpuCredits=standard` desde o capítulo 9, para nunca gerar cobrança extra. Num pico longo, os créditos acabam, a CPU cai para a linha de base, e a instância fica lenta sem ficar doente: o `/health` continua respondendo em milissegundos, enquanto as rotas reais demoram segundos. A política por requisições não percebe. Para produção com tráfego constante, uma família `m` ou `c` evita a surpresa.

Armadilha: O máximo do ASG é a sua última proteção de custo e de banco. Um bot fazendo milhões de requisições também aumenta o `ALBRequestCountPerTarget`, e a target tracking não diferencia cliente de ataque. Com um máximo alto, o bot escala a sua fatura e esgota as conexões do RDS. O WAF, no capítulo 23, filtra o tráfego antes que ele chegue à política.

Armadilha: AZ sem instância. O cross-zone load balancing do ALB fica sempre ligado no nível do balanceador, e por isso um nó numa AZ sem instâncias repassa as requisições para a outra. O target group pode desligar essa opção, com o atributo

> `load_balancing.cross_zone.enabled`. Se você fizer isso e uma AZ ficar sem alvos, quem cair no nó daquela AZ recebe `503`. As sticky sessions também deixam de funcionar com o cross-zone desligado. **Na vida real:** Na véspera de Natal de 2012, às 12h24 no horário do Pacífico, um processo de manutenção rodou por engano contra os dados de estado do Elastic Load Balancing em produção, na região `us-east-1`. Esses dados dizem ao plano de controle do ELB como cada balanceador está configurado, inclusive para quais instâncias mandar tráfego. No pior momento, 6,8% dos balanceadores da região foram afetados, e os demais continuaram funcionando sem poder ser alterados ou escalados. A Netflix ficou fora do ar em vários dispositivos durante a noite de Natal nos Estados Unidos. O resumo publicado pela AWS conta a recuperação: parte dos balanceadores foi restaurada à mão, e o restante voltou com uma restauração dos dados de estado para o ponto logo antes das 12h24. É point-in-time recovery, o mesmo do capítulo 12, aplicado à infraestrutura da própria AWS.

Resumo

- O Elastic Load Balancing tem o ALB (camada 7, HTTP) e o NLB (camada 4, TCP e UDP). O ALB é um conjunto de nós por AZ, acessado por um nome DNS.
- Listener escuta a porta, regras escolhem a ação, target group guarda os alvos e o health check.
- Health check padrão: 30 segundos de intervalo, 5 de timeout, 5 sucessos para ficar saudável. Sem alvos saudáveis, o ALB faz fail open.
- Deregistration delay (padrão de 300 segundos) deixa as requisições em andamento terminarem antes de um alvo sair.
- Launch Template é a receita versionada da instância. Launch configurations não podem mais ser criadas em contas novas.
- O ASG mantém a capacidade desejada entre o mínimo e o máximo, substitui instâncias doentes e registra as novas no target group. Com `--health-check-type ELB`, ele também ouve o target group.
- Grace period: 300 segundos pelo console, zero pela CLI. Defina sempre, junto com o default instance warmup.
- Target tracking com `ALBRequestCountPerTarget` mantém as requisições por instância perto de um alvo. Ações agendadas escalam antes de um pico com hora marcada.
- O Rangô ganhou uma golden AMI (`rango-api-20261015`), o Launch Template `rango-api`, o ALB `rango-alb` nas subnets públicas e o ASG `rango-api` com 2 a 6 instâncias nas privadas. Sobreviveu à remoção de uma instância sem nenhum erro e escalou sozinho num teste de carga.

Um endereço que ninguém vai digitar

Olhe de novo o endereço que o Rangô tem agora: `http://rango-alb-1234567890.us-east-1.elb.amazonaws.com`. Ele aguenta a TV, mas o apresentador não vai ler isso no ar. Se ler, ninguém vai conseguir anotar antes do intervalo. E ainda tem o `http://` na frente: os pedidos e as presigned URLs das fotos, que funcionam como senha, passando em texto puro, com o navegador avisando "não seguro" para quem abrir o link.

O React continua rodando no seu computador, apontando para a API por um `VITE_API_URL`. Quando ele for para a nuvem, a sócia em Recife vai carregar cada arquivo JavaScript de um servidor na Virgínia, com cada byte cruzando a linha do equador para chegar até ela. O capítulo 2 explicou DNS, certificados e CDN na teoria. No próximo capítulo, o Rangô ganha o domínio `rango.com.br` no Amazon Route 53, um certificado gratuito do AWS Certificate Manager no ALB e o React servido pelo Amazon CloudFront, perto de quem está pedindo comida.

Referências

- Health checks dos target groups do ALB: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html
- Atributos dos target groups (deregistration delay, algoritmos, slow start, cross-zone, sticky sessions): https://docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-target-group-attributes.html
- Cotas do Application Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-limits.html
- Preços do Elastic Load Balancing: https://aws.amazon.com/elasticloadbalancing/pricing/
- Cobrança de IPv4 público (AWS News Blog): https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/
- Health check grace period do EC2 Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/health-check-grace-period.html
- Target tracking no EC2 Auto Scaling: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html

- API `PutScalingPolicy` (warmup): https://docs.aws.amazon.com/autoscaling/ec2/APIReference/API_PutScalingPolicy.html
- Launch configurations (fim da criação): https://docs.aws.amazon.com/autoscaling/latest/userguide/LaunchConfiguration.html
- IAM role para aplicações em instâncias do ASG (PassRole): https://docs.aws.amazon.com/autoscaling/ec2/userguide/us-iam-role.html
- Resumo do evento do ELB de 24 de dezembro de 2012: https://aws.amazon.com/message/680587/
- psycopg_pool, verificação de conexões: https://www.psycopg.org/psycopg3/docs/advanced/pool.html

CAPÍTULO 14

Route 53, CloudFront e certificados

No capítulo 13, o Rangô passou a rodar atrás do Application Load Balancer `rango-alb`, com um Auto Scaling Group de 2 a 6 instâncias criadas a partir da AMI `rango-api-20261015`, nas subnets privadas. Ele sobreviveu à remoção de uma instância sem erro e escalou sozinho no teste de carga. No fim, desmontamos o que cobra por hora e guardamos o Launch Template, os Security Groups, a AMI e o snapshot `rango-db-final-cap13`. Ficou a promessa: um nome de verdade, HTTPS e o React perto de quem pede comida. Promessa de fim de capítulo costuma ter a mesma confiabilidade de estimativa de sprint, mas esta vai ser cumprida.

Um endereço para a TV

Quinta-feira, 15h. A produtora do especial de sábado manda uma mensagem: "Qual endereço a gente coloca na tela?". Você abre o terminal e copia o que tem: `http://rango-alb-1234567890.us-east-1.elb.amazonaws.com`. Ninguém vai digitar isso, e a produtora nem responde. O silêncio dela diz tudo: um endereço com dez dígitos no meio e `elb.amazonaws.com` no fim tem a mesma chance de aparecer na TV que um stack trace num comercial de margarina.

O nome é o problema mais visível. O segundo está no `http://`. Os pedidos, com endereço de entrega, e as presigned URLs das fotos, que funcionam como senha, atravessam a internet em texto puro. Qualquer pessoa no Wi-Fi da praça de alimentação do capítulo 2 consegue ler, e o navegador mostra "não seguro" ao lado do endereço. Para quem está com fome, esse aviso soa como uma placa de "cozinha sem alvará" na porta do restaurante.

O terceiro aparece quando o React sai do seu notebook. A primeira ideia é deixar o Nginx das instâncias servir os arquivos do `npm run build`, como no capítulo 3. Funciona, mas pense na sócia em Recife: cada arquivo vai até a Virgínia e volta. Num teste hipotético, mas com números típicos, a ida e volta (o RTT) entre Recife e `us-east-1` fica entre 120 e 150 milissegundos. Antes do primeiro byte do `index.html`, o navegador gasta uma ida e volta no TCP, outra no TLS 1.3 e mais uma na requisição: quase meio segundo. Depois vem o bundle JavaScript, com algumas centenas de kilobytes. O TCP começa enviando

pouco e aumenta aos poucos (o mecanismo se chama *slow start*), e cada aumento custa mais uma ida e volta. Num celular em 4G, a tela branca passa de dois segundos sem que nenhum servidor esteja lento. O culpado é a física, e a física não aceita ticket de suporte.

E essa carga cairia nas instâncias que atendem `POST /pedidos`, com arquivos iguais para todo mundo que só mudam no deploy, o caso ideal para cache.

O capítulo 2 explicou a teoria de DNS, certificados e CDN. Agora ela vira prática, com três serviços da AWS.

Os três serviços

Amazon Route 53

O Amazon Route 53 é o serviço de DNS da AWS. O 53 do nome é a porta do DNS, e o resto lembra uma estrada americana, o que combina com um serviço que passa o dia ensinando caminho para os outros. No vocabulário do capítulo 2, ele faz o papel de servidor autoritativo: guarda a zona do `rango.com.br` e responde aos resolvers do mundo inteiro quando alguém pergunta pelo endereço.

A zona, no Route 53, se chama **hosted zone**. Ao criar uma, a AWS atribui quatro servidores de nome a ela, espalhados por domínios de topo diferentes (algo como `ns-1234.awsdns-26.org` e `ns-567.awsdns-06.net`). Se um desses domínios de topo tiver problemas, os outros continuam respondendo. É o velho conselho de não pôr todos os ovos na mesma cesta, aplicado a ponto com, ponto net e companhia. Esses quatro nomes são os que você entrega ao registrador.

O domínio continua registrado no Registro.br, como no capítulo 2. O registrador e o provedor de DNS são papéis separados: o Registro.br diz à hierarquia do `.br` quais servidores respondem pelo `rango.com.br`, e o Route 53 dá as respostas. Para trocar de provedor, basta trocar os quatro nomes no painel do registrador, como quem troca de contador sem precisar mudar de CPF.

Uma hosted zone pode ser pública, visível na internet, ou privada, visível só dentro das VPCs que você associar a ela. A do Rangô é pública.

Os registros são os mesmos do capítulo 2 (A, AAAA, CNAME, MX, TXT), com uma adição que resolve um problema deixado lá atrás: o CNAME é proibido no apex da zona, o nome "pelado" `rango.com.br`. O apex precisa ter os registros SOA e NS, e o CNAME não pode coexistir com nenhum outro registro no mesmo nome. O CNAME é ciumento: não divide o nome com ninguém. Isso seria contornável se o ALB e o CloudFront tivessem IP fixo, mas eles só têm nome DNS, com IPs que mudam.

A resposta da AWS é o **registro alias**. Você cria um registro do tipo A (ou AAAA) e, em vez de um IP, aponta para um recurso da AWS: um load balancer, uma distribuição do CloudFront, um bucket configurado como site. Quando um resolver pergunta pelo `rango.com.br`, o Route 53 consulta os IPs atuais do recurso e responde com eles, como se fosse um A comum. Para o mundo de fora, é um registro A. Por dentro, ele se comporta como um CNAME que o próprio Route 53 resolve antes de responder.

Pense numa property do Python. Quem lê `pedido.total` recebe um número, sem saber que por trás existe uma função calculando o valor a cada acesso. O alias é a property do DNS, com a vantagem de que nenhum colega vai abrir um comentário no PR reclamando de efeito colateral escondido no getter.

O alias tem duas vantagens além de funcionar no apex. Consultas a alias que apontam para recursos da AWS como ELB e CloudFront não são cobradas. Num provedor que cobra até pelo ar que a instância respira, "não são cobradas" merece ser lido duas vezes, devagar. E, num alias para load balancer, a opção `EvaluateTargetHealth` faz o Route 53 levar em conta a saúde do alvo, o que importa quando há mais de um destino.

Isso leva aos **health checks** e às **políticas de roteamento**. Verificadores do Route 53 em várias regiões chamam o seu endpoint a cada 30 segundos (ou 10, pagando mais), e ele é considerado saudável quando mais de 18% deles concordam. É um quórum menor que o de reunião de condomínio, e mesmo assim decide coisas com mais rapidez. A política decide a resposta: a simples devolve sempre o mesmo valor; a ponderada divide por pesos, útil para um canary; a de latência escolhe a região mais próxima; a de failover troca para o secundário quando o health check do primário falha. É o DR do capítulo 6 feito no DNS, com o TTL definindo quanto os clientes demoram para perceber a troca.

O Rangô tem uma região só e usa a política simples. O health check entra como vigia externo, olhando a API como um cliente veria.

AWS Certificate Manager

O AWS Certificate Manager (ACM) emite certificados TLS públicos e cuida da renovação. Para certificados usados em serviços integrados, como ALB e CloudFront, a emissão não custa nada.

O ACM faz o papel de CA e de Certbot ao mesmo tempo. Você pede um certificado para uma lista de nomes, prova que controla o domínio e recebe um ARN. A chave privada fica guardada na AWS, e você nunca a vê: não há arquivo `.pem` para vazar num commit nem para copiar entre servidores. Nem para alguém mandar por e-mail com o assunto "segue a chave, apagar depois de ler", o que sempre acontece com alguém, e nunca é apagado. O certificado é instalado no ALB ou na distribuição pelo ARN.

A prova de posse é a validação DV do capítulo 2. O ACM aceita três métodos: DNS, e-mail e, mais recentemente, HTTP. A escolha certa é o DNS. O ACM informa um registro CNAME com nomes aleatórios, algo como `_3f1c9a.rango.com.br` apontando para `_8b2e7d.acm-validations.aws.`, e você o cria na zona. Enquanto esse registro existir, o ACM consegue revalidar sozinho e renovar o certificado sem ninguém fazer nada. A validação por HTTP não aceita certificados curinga, e a por e-mail depende de alguém clicar num link a cada renovação. Numa caixa de entrada com quatro mil e-mails não lidos, isso é menos um processo e mais um bilhete de loteria.

As regras de validade mudaram em 2026, seguindo o calendário do CA/Browser Forum do capítulo 2: a partir de 15 de março, no máximo 200 dias. Desde fevereiro de 2026, o ACM emite e renova certificados públicos com 198 dias, em vez de 395; os antigos valem até a próxima renovação. A renovação começa 45 dias antes do vencimento, mais ou menos a cada cinco meses. Com renovação automática, ninguém percebe a mudança. Sem ela, alguém perceberia, provavelmente num domingo à noite.

Há uma regra que surpreende todo mundo uma vez (e algumas pessoas, duas): o CloudFront só aceita certificados do ACM emitidos na região `us-east-1`, seja qual for a região da sua aplicação. O CloudFront é global, e a configuração dele vive na Virgínia. Na AWS, todos os caminhos, cedo ou tarde, passam pela Virgínia. Se a API do Rangô estivesse em `sa-east-1`, você precisaria de dois certificados: um em São Paulo para o ALB e outro em `us-east-1` para o CloudFront. Como tudo do Rangô está em `us-east-1`, um único certificado com `rango.com.br`, `www.rango.com.br` e `api.rango.com.br` serve aos dois. Esse é um dos motivos que o capítulo 7 listou para começar pela Virgínia.

Amazon CloudFront

O Amazon CloudFront é a CDN da AWS, com centenas de pontos de presença pelo mundo, vários deles no Brasil. O que o capítulo 2 chamou de borda, a AWS chama de *edge location*, que é a mesma coisa, só que soa bem mais caro.

A unidade de configuração é a **distribuição**. Ela recebe um nome como `d1a2b3c4d5e6f7.cloudfront.net`, aceita nomes alternativos (os seus domínios, com o certificado do ACM) e liga um ou mais comportamentos de cache a uma ou mais origens. Uma origem pode ser um bucket S3, um load balancer ou qualquer servidor HTTP. Um comportamento diz que caminhos ele atende (`/assets/*`, por exemplo), qual política de cache usar e o que fazer com HTTP sem criptografia.

Para o React, a origem natural é um bucket S3. O build do Vite gera um `index.html` e uma pasta `assets/` com arquivos cujo nome carrega um hash do conteúdo. São arquivos estáticos, e o S3 do capítulo 11 guarda arquivos estáticos por centavos. O CloudFront fica na frente: a sócia em Recife fala com uma borda no Brasil, a dezenas de milissegundos ou

menos, e só a primeira requisição de cada arquivo em cada borda vai até o bucket na Virgínia. As seguintes saem do cache da borda. O handshake TLS também acontece na borda, perto dela.

Falta um detalhe de segurança. O bucket precisa ficar privado, com o Block Public Access do capítulo 11 ligado, e mesmo assim entregar arquivos ao CloudFront. A peça que resolve isso se chama **Origin Access Control (OAC)**. A distribuição assina cada requisição ao S3 com SigV4, a mesma assinatura que a AWS CLI usa, e a bucket policy libera `s3:GetObject` para o serviço `cloudfront.amazonaws.com`, com uma condição: o ARN de origem precisa ser o da sua distribuição. Qualquer outra distribuição, de qualquer outra conta, recebe `403`. Quem tentar o endereço direto do bucket também. É festa com lista de convidados e conferência de documento na entrada.

Tutoriais antigos usam OAI, o mecanismo anterior, que a AWS mantém como legado, ou o *static website hosting* do S3 com bucket público. O OAC só funciona com o endpoint REST do bucket (`bucket.s3.us-east-1.amazonaws.com`), e o bucket público é o que o capítulo 11 ensinou a evitar. A internet está cheia de tutorial de 2019 com cinco estrelas, milhares de curtidas e um bucket aberto para o mundo.

Sobre custo, o CloudFront tem hoje dois modelos. No pay-as-you-go, você paga por GB entregue e por requisição, com uma franquia mensal permanente de 1 TB de saída e 10 milhões de requisições. Uma distribuição parada não cobra nada. Existem também planos de preço fixo (Free, Pro, Business, Premium, nomes dignos de cartão de crédito), sem cobrança de excedente, cada um cobrindo uma distribuição e um domínio apex, com WAF, DNS e certificado incluídos. O Free custa zero, com 1 milhão de requisições e 100 GB por mês. Os comandos deste capítulo usam o pay-as-you-go, que no volume do Rangô também não deve gerar cobrança. Preços e franquias de referência, `us-east-1`, consultados em outubro de 2026; confira a página oficial antes de decidir.

Na prática: rango.com.br no ar

Os comandos e as saídas estão todos aqui, então dá para acompanhar a história inteira só lendo, sem gastar nem os US$ 0,50 da hosted zone.

Pré-requisitos: tudo o que o capítulo 13 deixou (Launch Template `rango-api`, AMI `rango-api-20261015`, Security Groups `rango-web`, `rango-alb`, `rango-db` e `rango-cache`, snapshot `rango-db-final-cap13`, role `rango-ec2`, bucket `rango-fotos-pratos-<conta>`), o profile `rango-dev`, e o domínio `rango.com.br` registrado no Registro.br com acesso ao painel. Numa conta de verdade, o seu próprio domínio entra no lugar do `rango.com.br` em todos os comandos. Para o build do front, Node.js e o repositório do Rangô.

Sobre custo: a hosted zone cobra US$ 0,50 por mês, e o mês começa a contar na criação. O banco, o cache, o ALB e as instâncias voltam a cobrar por hora até a seção "A desmontagem". O taxímetro liga agora.

A zona e a delegação

O começo é pela zona, porque a delegação pode levar algum tempo para valer e o certificado depende dela:

```
export AWS_PROFILE=rango-dev
aws sso login --sso-session rango

ZONE_ID=$(aws route53 create-hosted-zone \
  --name rango.com.br \
  --caller-reference "rango-$(date +%s)" \
  --hosted-zone-config Comment="Zona publica do Rango" \
  --query "HostedZone.Id" --output text)
ZONE_ID=${ZONE_ID##*/}

aws route53 get-hosted-zone --id "$ZONE_ID" \
  --query "DelegationSet.NameServers" --output text
```

```
ns-1234.awsdns-26.org  ns-567.awsdns-06.net  ns-1890.awsdns-44.co.uk  ns-89.awsdns-
11.com
```

O ID volta no formato `/hostedzone/Z0123456789ABCDEFGHIJ`, e o `${ZONE_ID##*/}` corta tudo até a última barra. O `--caller-reference` precisa ser único a cada chamada: ele impede que uma retentativa da CLI crie duas zonas iguais. É o `Idempotency-Key` do capítulo 5, do lado da AWS. O `date +%s` garante a unicidade com o método mais antigo da computação: torcer para que ninguém rode duas vezes no mesmo segundo.

A zona nasce com dois registros: o SOA e o NS com os quatro servidores. Os seus quatro nomes vão ser outros. Eles vão para o painel do Registro.br, na tela de servidores DNS do domínio. O Registro.br costuma conferir se os servidores informados respondem pela zona antes de aceitar a troca, e como a zona já existe no Route 53, a conferência passa.

O teste vai direto num servidor do `.br`, que mostra a delegação sem passar por nenhum cache:

```
dig @a.dns.br rango.com.br NS +norec
```

```
;; AUTHORITY SECTION:
rango.com.br.       3600    IN  NS  ns-1234.awsdns-26.org.
rango.com.br.       3600    IN  NS  ns-567.awsdns-06.net.
...
```

Quando os quatro nomes da AWS aparecerem, a hierarquia já manda as perguntas para o Route 53. Os resolvers que guardaram a delegação antiga só vão perceber quando o TTL deles vencer, o fenômeno que o capítulo 2 explicou como "propagação". É a palavra mágica do suporte de hospedagem: "aguarde a propagação, até 48 horas" resolve qualquer chamado, inclusive os que não têm nada a ver com DNS.

O certificado

O certificado é pedido em `us-east-1`, com os três nomes:

```
CERT_ARN=$(aws acm request-certificate \
  --region us-east-1 \
  --domain-name rango.com.br \
  --subject-alternative-names www.rango.com.br api.rango.com.br \
  --validation-method DNS \
  --tags Key=Projeto,Value=rango \
  --query CertificateArn --output text)
```

Os três nomes viram SANs do mesmo certificado, com chave RSA 2048, o padrão do ACM.

Uns dez segundos depois, chega a vez dos registros de validação. Uma função pequena evita repetir o JSON de mudança do Route 53 a cada registro. A quantidade de barras invertidas denuncia JSON dentro de string de shell, a boneca russa da tipografia:

```
upsert() {
  aws route53 change-resource-record-sets --hosted-zone-id "$ZONE_ID" \
    --change-batch "{\"Changes\":[{\"Action\":\"UPSERT\",\"ResourceRecordSet\":$1}]}" \
    --query "ChangeInfo.Status" --output text
}

aws acm describe-certificate --certificate-arn "$CERT_ARN" \
  --query "Certificate.DomainValidationOptions[].ResourceRecord.[Name,Value]" \
  --output text | sort -u |
while read -r NOME VALOR; do
  upsert "{\"Name\":\"$NOME\",\"Type\":\"CNAME\",\"TTL\":300,\"ResourceRecords\":
[{\"Value\":\"$VALOR\"}]}"
done

aws acm wait certificate-validated --certificate-arn "$CERT_ARN"
```

O `UPSERT` cria o registro ou o substitui se já existir, e torna o comando seguro para rodar de novo. Cada chamada responde `PENDING` e, segundos depois, a mudança está em todos os servidores do Route 53. O `wait` consulta o ACM a cada minuto até o status virar `ISSUED`, o que costuma levar poucos minutos depois que a delegação valeu. Se ele estourar o tempo, a suspeita recai sobre a delegação, e o `dig` da seção anterior tira a dúvida: o ACM pergunta pela internet, e sem delegação ele não encontra os registros.

Não apague esses três CNAMEs nunca. Eles são a autorização permanente para as renovações. Têm toda a cara de lixo esquecido por um teste antigo, e é exatamente por isso que alguém, um dia, vai querer apagá-los.

A API de volta, com configuração nova

O banco e o cache voltam com os comandos de "Banco e cache de volta", do capítulo 13, trocando o snapshot por `rango-db-final-cap13`. As variáveis `VPC_ID`, `PUB_A`, `PUB_B`, `PRIV_A`, `PRIV_B`, `SG_WEB` e `SG_ALB` também são recuperadas com as funções `id_subnet` e `id_sg` de lá.

Existe um problema escondido aqui. A AMI carrega o `/etc/rango/api.env` do dia em que foi criada, com o endpoint do cache daquele dia. O cache serverless ganha endpoint novo a cada criação, e as instâncias novas tentariam falar com um cache que não existe mais, como quem liga para o número antigo de um amigo e cai numa pizzaria. E o `CORS_ORIGINS` ainda aponta para `http://localhost:5173`, então o React em `https://rango.com.br` seria barrado pelo navegador.

Em vez de gerar outra AMI, o remendo é um user data curto numa versão nova do Launch Template, que ajusta as duas linhas no primeiro boot. Nenhum dos valores é segredo:

```
CACHE_HOST=$(aws elasticache describe-serverless-caches \
  --serverless-cache-name rango-cache \
  --query "ServerlessCaches[0].Endpoint.Address" --output text)

cat > infra/asg/user-data-config.sh <<EOF
#!/bin/bash
set -euo pipefail
sed -i \\
  -e 's|^REDIS_URL=.*|REDIS_URL=rediss://$CACHE_HOST:6379|' \\
  -e 's|^CORS_ORIGINS=.*|CORS_ORIGINS=https://rango.com.br,https://www.rango.com.br|' \\
  /etc/rango/api.env
systemctl restart rango-api rango-worker@restaurante rango-worker@email
EOF

aws ec2 create-launch-template-version \
  --launch-template-name rango-api \
  --source-version 1 \
  --version-description "cap. 14: cache novo e CORS do dominio" \
  --launch-template-data "{\"UserData\":\"$(base64 -w0 infra/asg/user-data-
config.sh)\"}"

aws ec2 modify-launch-template --launch-template-name rango-api --default-version 2
```

O `--source-version 1` copia a versão anterior e acrescenta o user data. No macOS, o `base64 -w0` vira `base64 -i`. O `CORS_ORIGINS` aceita uma lista separada por vírgula, porque o `main.py` faz `split(",")` desde o capítulo 3. Como o ASG do capítulo 13 usa `$Default`, as instâncias que ele criar a partir de agora já nascem com a versão 2.

O capítulo 16 troca esse remendo por algo melhor: a instância vai buscar a configuração e a senha no Parameter Store a cada boot, e a AMI deixa de carregar o `api.env`. Remendo com data marcada para sair é a forma mais honesta de gambiarra que existe.

Em seguida, o target group, o ALB e o ASG voltam com os comandos do capítulo 13, com uma diferença: o `create-listener` da porta 80 fica de fora. O ALB ganha dois listeners novos. Antes deles, a porta 443 precisa estar aberta no grupo do balanceador:

```
aws ec2 authorize-security-group-ingress --group-id "$SG_ALB" \
  --protocol tcp --port 443 --cidr 0.0.0.0/0

aws elbv2 create-listener --load-balancer-arn "$ALB_ARN" \
  --protocol HTTPS --port 443 \
  --certificates CertificateArn="$CERT_ARN" \
  --ssl-policy ELBSecurityPolicy-TLS13-1-2-2021-06 \
  --default-actions Type=forward,TargetGroupArn="$TG_ARN"

aws elbv2 create-listener --load-balancer-arn "$ALB_ARN" \
  --protocol HTTP --port 80 \
  --default-actions 'Type=redirect,RedirectConfig=
{Protocol=HTTPS,Port=443,StatusCode=HTTP_301}'
```

O listener HTTPS termina o TLS no ALB, como o Nginx do capítulo 3, e repassa às instâncias em HTTP pela rede privada. A *security policy* escolhida aceita só TLS 1.3 e 1.2. A porta 80 responde com um `301` para o HTTPS, sem incomodar as instâncias, como a recepcionista que aponta a entrada certa sem precisar chamar o gerente.

Com o ASG de volta, as ações `especial-tv-inicio` e `especial-tv-fim` também precisam ser recriadas, com o cálculo do próximo sábado de "Agendado para sábado", no capítulo 13. O `--force-delete` de lá levou as duas junto com o grupo.

api.rango.com.br

O nome da API é um alias para o ALB. O alias precisa do ID da hosted zone interna que a AWS usa para os load balancers da região, e o próprio ALB informa esse valor:

```
read -r ALB_DNS ALB_ZONE <<< "$(aws elbv2 describe-load-balancers \
  --load-balancer-arns "$ALB_ARN" \
  --query "LoadBalancers[0].[DNSName,CanonicalHostedZoneId]" --output text)"

upsert "{\"Name\":\"api.rango.com.br\",\"Type\":\"A\",\"AliasTarget\":
{\"HostedZoneId\":\"$ALB_ZONE\",\"DNSName\":\"$ALB_DNS\",\"EvaluateTargetHealth\":true}}
"
```

Não há TTL: no alias, quem decide é o recurso de destino. Os testes mostram o resultado:

```
dig +short api.rango.com.br
curl -sI http://api.rango.com.br/health | head -3
curl -s https://api.rango.com.br/restaurantes
```

```
203.0.113.81
198.51.100.140

HTTP/1.1 301 Moved Permanently
Server: awselb/2.0
Location: https://api.rango.com.br:443/health

[{"id":"cantina-da-nona","nome":"Cantina da
Nona","taxa_entrega":6.9,"taxa_entrega_fmt":"R$ 6,90"},...]
```

Dois IPs, um por nó do ALB. Amanhã podem ser outros dois, e o alias nem pisca. O `http://` foi redirecionado pelo próprio balanceador, e o `https://` respondeu com o certificado do ACM. O `openssl` do capítulo 2 confere a cadeia:

```
openssl s_client -connect api.rango.com.br:443 -servername api.rango.com.br </dev/null
2>/dev/null \
  | openssl x509 -noout -subject -issuer -enddate
```

```
subject=CN=rango.com.br
issuer=C=US, O=Amazon, CN=Amazon RSA 2048 M02
notAfter=May  1 23:59:59 2027 GMT
```

O emissor é uma CA intermediária da Amazon (o nome exato pode variar), e o vencimento fica a 198 dias de hoje. Tempo de sobra para esquecer completamente que ele existe, que é justamente a ideia.

O React num bucket privado

O front precisa saber onde a API mora. O Vite lê variáveis de arquivos `.env` por modo, e o `npm run build` roda no modo `production`. O arquivo que resolve é o `frontend/.env.production`:

```
VITE_API_URL=https://api.rango.com.br
```

Esse arquivo pode ir para o git, porque toda variável `VITE_` acaba dentro do bundle, legível no DevTools: segredo nenhum entra aqui. Tudo o que vai para o navegador deve ser tratado como se estivesse escrito num outdoor na avenida principal. Ele tem prioridade sobre o `.env.local` só no build, e o desenvolvimento continua apontando para o Compose.

Depois vem o bucket. Buckets novos já nascem com Block Public Access ligado e com o Object Ownership em `BucketOwnerEnforced`, que o OAC exige:

```
CONTA=$(aws sts get-caller-identity --query Account --output text)
FRONT=rango-front-$CONTA

aws s3api create-bucket --bucket "$FRONT" --region us-east-1
```

```
aws s3api put-bucket-tagging --bucket "$FRONT" \
  --tagging 'TagSet=[{Key=Projeto,Value=rango}]'
```

Em seguida, o build e o envio. A ordem e os cabeçalhos importam:

```
cd frontend
npm ci
npm run build

aws s3 sync dist/ "s3://$FRONT/" \
  --exclude index.html \
  --cache-control "public, max-age=31536000, immutable"

aws s3 cp dist/index.html "s3://$FRONT/index.html" \
  --cache-control "no-cache"
cd ..
```

Os arquivos de `assets/` têm o hash no nome, como `index-BxQ3k9aZ.js`. Se o conteúdo muda, o nome muda, e por isso eles podem ficar um ano em qualquer cache. O `index.html` tem sempre o mesmo nome e aponta para os hashes da versão atual, então ele vai com `no-cache`: o navegador pode guardá-lo, mas precisa perguntar se mudou antes de usar, como quem cheira o leite antes de colocar no café. É a regra de `Cache-Control` do capítulo 2.

Os assets sobem primeiro. Se o `index.html` novo chegasse antes, um usuário poderia receber uma página apontando para um JavaScript que ainda não existe no bucket. E o `sync` vai sem `--delete` de propósito: os arquivos da versão anterior ficam no bucket, e quem está com a aba aberta desde ontem continua conseguindo carregar os pedaços da versão antiga. A seção de armadilhas mostra o que acontece sem isso.

A distribuição

Primeiro vem o OAC. Ele é só uma configuração de assinatura, reaproveitável por várias distribuições:

```
OAC_ID=$(aws cloudfront create-origin-access-control \
  --origin-access-control-config \
    Name=rango-front-
oac,SigningProtocol=sigv4,SigningBehavior=always,OriginAccessControlOriginType=s3 \
  --query "OriginAccessControl.Id" --output text)
```

A configuração da distribuição é um JSON grande, do tipo que faz qualquer pessoa sentir saudade do YAML por uns três segundos. Ela fica em `infra/cloudfront/rango-front.json`, gerado a partir das variáveis:

```
mkdir -p infra/cloudfront
cat > infra/cloudfront/rango-front.json <<EOF
{
  "CallerReference": "rango-front-$(date +%s)",
  "Comment": "Front do Rango",
```

```
  "Enabled": true,
  "Aliases": {"Quantity": 2, "Items": ["rango.com.br", "www.rango.com.br"]},
  "DefaultRootObject": "index.html",
  "Origins": {"Quantity": 1, "Items": [{
    "Id": "s3-front",
    "DomainName": "$FRONT.s3.us-east-1.amazonaws.com",
    "S3OriginConfig": {"OriginAccessIdentity": ""},
    "OriginAccessControlId": "$OAC_ID"
  }]},
  "DefaultCacheBehavior": {
    "TargetOriginId": "s3-front",
    "ViewerProtocolPolicy": "redirect-to-https",
    "CachePolicyId": "658327ea-f89d-4fab-a63d-7e88639e58f6",
    "Compress": true,
    "AllowedMethods": {"Quantity": 2, "Items": ["GET", "HEAD"],
      "CachedMethods": {"Quantity": 2, "Items": ["GET", "HEAD"]}}
  },
  "CustomErrorResponses": {"Quantity": 2, "Items": [
    {"ErrorCode": 403, "ResponsePagePath": "/index.html", "ResponseCode": "200",
"ErrorCachingMinTTL": 10},
    {"ErrorCode": 404, "ResponsePagePath": "/index.html", "ResponseCode": "200",
"ErrorCachingMinTTL": 10}
  ]},
  "ViewerCertificate": {
    "ACMCertificateArn": "$CERT_ARN",
    "SSLSupportMethod": "sni-only",
    "MinimumProtocolVersion": "TLSv1.2_2021"
  },
  "HttpVersion": "http2and3",
  "IsIPV6Enabled": true,
  "PriceClass": "PriceClass_All"
}
EOF

read -r DIST_ID DIST_DNS <<< "$(aws cloudfront create-distribution \
  --distribution-config file://infra/cloudfront/rango-front.json \
  --query "Distribution.[Id,DomainName]" --output text)"
echo "$DIST_ID $DIST_DNS"
```

```
E1ABCDEF2GHIJK d1a2b3c4d5e6f7.cloudfront.net
```

Vale passar pelas linhas menos óbvias. O `S3OriginConfig` com `OriginAccessIdentity` vazio é obrigatório na API mesmo quando você usa OAC, porque o campo é do mecanismo antigo. É o campo "número do fax" de formulário de cadastro: ninguém usa, mas sem ele o formulário não passa. O `CachePolicyId` é o da política gerenciada `CachingOptimized`, que respeita o `Cache-Control` do objeto dentro de limites de 1 segundo a 1 ano e não inclui cookies nem query strings na chave de cache. O `sni-only` usa o SNI do capítulo 2 para escolher o certificado. O `PriceClass_All` (o nome soa como passagem de classe executiva, e o que entrega é a lista completa de bordas) inclui as bordas da América do Sul, e a seção de armadilhas explica por que isso pesa para o Rangô.

O `CustomErrorResponses` é o equivalente do `try_files` do Nginx do capítulo 3. Uma SPA tem rotas que só existem no React: quando o Rangô ganhar o React Router, um link direto para `https://rango.com.br/pedidos/6f1c2a9e` vai pedir ao bucket um objeto chamado `pedidos/6f1c2a9e`, que não existe. O OAC só libera `s3:GetObject`, sem `s3:ListBucket`, e nessas condições o S3 responde `403` para uma chave inexistente, para não revelar o que existe ou não no bucket. Sem o mapeamento, o usuário veria isto:

```
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access Denied</Message></Error>
```

Uma mensagem de erro em XML, a esta altura do século, é o tipo de coisa que faz o cliente fechar a aba e pedir pizza pelo telefone.

Com o mapeamento, o CloudFront troca o `403` e o `404` pelo `index.html` com status `200`, e o React lê a URL e desenha a tela certa.

Agora a bucket policy, que libera a leitura só para essa distribuição. Ela fica em `infra/s3/rango-front-policy.json`:

```
cat > infra/s3/rango-front-policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SoADistribuicaoDoRango",
      "Effect": "Allow",
      "Principal": {"Service": "cloudfront.amazonaws.com"},
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::$FRONT/*",
      "Condition": {
        "StringEquals": {"AWS:SourceArn":
"arn:aws:cloudfront::$CONTA:distribution/$DIST_ID"}
      }
    }
  ]
}
EOF

aws s3api put-bucket-policy --bucket "$FRONT" \
  --policy file://infra/s3/rango-front-policy.json

aws cloudfront wait distribution-deployed --id "$DIST_ID"
```

O Block Public Access aceita essa policy, porque o principal é um serviço com condição sobre um ARN específico. O ARN da distribuição não tem região. O `wait` leva alguns minutos, até a configuração chegar a todas as bordas. É o intervalo perfeito para ir buscar um café e descobrir que o café acabou.

O apex, e um erro de propósito

Falta apontar `rango.com.br` para a distribuição. O reflexo de quem conhece DNS é criar um CNAME. O Route 53, que também conhece DNS, tem outra opinião:

```
upsert "{\"Name\":\"rango.com.br\",\"Type\":\"CNAME\",\"TTL\":300,\"ResourceRecords\":
[{\"Value\":\"$DIST_DNS\"}]}"
```

```
An error occurred (InvalidChangeBatch) when calling the ChangeResourceRecordSets
operation:
[RRSet of type CNAME with DNS name rango.com.br. is not permitted at apex in zone
rango.com.br.]
```

É a regra do capítulo 2, aplicada pelo próprio Route 53. A saída é o alias, em A e em AAAA, já que a distribuição tem IPv6 ligado. O `HostedZoneId` de um alias para CloudFront é sempre o mesmo, `Z2FDTNDATAQYW2`, em qualquer conta. É o único ID da AWS que vale a pena decorar, e mesmo assim ninguém decora:

```
for NOME in rango.com.br www.rango.com.br; do
  for TIPO in A AAAA; do
    upsert "{\"Name\":\"$NOME\",\"Type\":\"$TIPO\",\"AliasTarget\":
{\"HostedZoneId\":\"Z2FDTNDATAQYW2\",\"DNSName\":\"$DIST_DNS\",\"EvaluateTargetHealth\":
false}}"
  done
done
```

O `www` poderia ser um CNAME comum, porque não fica no apex. Com alias, as consultas a ele também saem de graça.

O teste do caminho todo pede primeiro o `index.html`, depois o mesmo arquivo de assets duas vezes (o nome do arquivo muda a cada build):

```
curl -sI https://rango.com.br/ | grep -iE "^(http|cache-control|x-cache|x-amz-cf-pop)"
curl -sI https://rango.com.br/assets/index-BxQ3k9aZ.js | grep -iE "^(x-cache|x-amz-cf-
pop)"
curl -sI https://rango.com.br/assets/index-BxQ3k9aZ.js | grep -iE "^(x-cache|x-amz-cf-
pop)"
```

```
HTTP/2 200
cache-control: no-cache
x-cache: Miss from cloudfront
x-amz-cf-pop: GRU3-P2

x-cache: Miss from cloudfront
x-amz-cf-pop: GRU3-P2

x-cache: Hit from cloudfront
x-amz-cf-pop: GRU3-P2
```

O `x-amz-cf-pop` mostra a borda que atendeu, pelo código do aeroporto: `GRU` é Guarulhos, em São Paulo. Pela primeira vez na história, alguma coisa saiu de Guarulhos e chegou ao destino em poucos milissegundos. Do Recife, a borda pode ser outra. O primeiro pedido do JavaScript foi um *miss* e buscou no bucket da Virgínia; o segundo saiu do cache da borda. Só o primeiro visitante de cada borda paga a viagem até `us-east-1`.

No navegador, `https://rango.com.br` abre com o cadeado, a lista de restaurantes vem de `https://api.rango.com.br`, e o console do DevTools não mostra erro de CORS, porque o `api.env` das instâncias já aceita a origem nova. Um console do DevTools sem nada vermelho é uma visão tão rara que merecia print e moldura.

Já o endereço direto do bucket precisa falhar, e falha. Poucas vezes um `403` trouxe tanta alegria:

```
curl -s -o /dev/null -w "%{http_code}\n" "https://$FRONT.s3.us-east-
1.amazonaws.com/index.html"
```

```
403
```

Num deploy futuro do front, o `index.html` com `no-cache` já faz o navegador perguntar pela versão nova, e a política `CachingOptimized` só o guarda na borda por instantes. Se você precisar forçar uma borda a esquecer algo, existe a invalidação, que remove caminhos do cache em todas as bordas:

```
aws cloudfront create-invalidation --distribution-id "$DIST_ID" --paths "/index.html"
```

Os primeiros 1.000 caminhos invalidados por mês são gratuitos. O capítulo 18 coloca esse passo no pipeline.

As fotos e o domínio novo

O capítulo 11 deixou o CORS do bucket de fotos aceitando só `https://rango.test:5173`, e o upload direto do navegador agora parte de `https://rango.com.br`. A atualização vai em `infra/s3/rango-fotos-cors.json`:

```
{
  "CORSRules": [
    {
      "AllowedOrigins": [
        "https://rango.test:5173",
        "https://rango.com.br",
        "https://www.rango.com.br"
      ],
      "AllowedMethods": ["PUT"],
      "AllowedHeaders": ["content-type"],
      "MaxAgeSeconds": 3000
```

```
    }
  ]
}
```

```
aws s3api put-bucket-cors --bucket "rango-fotos-pratos-$CONTA" \
  --cors-configuration file://infra/s3/rango-fotos-cors.json
```

As fotos continuam sendo exibidas por presigned URLs de leitura, direto do S3. Servi-las pelo CloudFront exigiria trocar essas URLs por URLs assinadas do CloudFront, e fica para quando o volume justificar, uma frase que todo projeto tem escrita em algum lugar e que quase ninguém volta para ler.

Um vigia do lado de fora

O ALB e o ASG cuidam das instâncias, mas não sabem se o Rangô está acessível para quem está fora da AWS. Elogio do chef à própria comida vale pouco; o que conta é a opinião de quem está na mesa. Um health check do Route 53 testa o mesmo caminho que o cliente:

```
HC_ID=$(aws route53 create-health-check \
  --caller-reference "rango-api-$(date +%s)" \
  --health-check-config
'Type=HTTPS,FullyQualifiedDomainName=api.rango.com.br,Port=443,ResourcePath=/health,Requ
estInterval=30,FailureThreshold=3' \
  --query "HealthCheck.Id" --output text)

aws route53 change-tags-for-resource --resource-type healthcheck \
  --resource-id "$HC_ID" --add-tags Key=Name,Value=rango-api Key=Projeto,Value=rango
```

Depois de um ou dois minutos:

```
aws route53 get-health-check-status --health-check-id "$HC_ID" \
  --query "HealthCheckObservations[0:3].[Region,StatusReport.Status]" --output text
```

```
us-east-1   Success: HTTP Status Code 200, OK
sa-east-1   Success: HTTP Status Code 200, OK
eu-west-1   Success: HTTP Status Code 200, OK
```

Verificadores em São Paulo, na Irlanda e na Virgínia veem o `/health` responder. O check publica uma métrica no CloudWatch, que aceita um alarme como qualquer outra; o capítulo 22 mostra como montar alarmes. Numa arquitetura com duas regiões, é este check que a política de failover consulta.

Um health check com HTTPS para endpoint na AWS custa US$ 1,00 por mês, mas a página de preços informa que até 50 health checks de endpoints da AWS por conta não são cobrados (referência, outubro de 2026).

Sábado, 21h

Na quinta à noite, a produtora recebe a resposta: `rango.com.br`. No sábado, às 20h30, a ação agendada sobe o mínimo para 4, e as duas instâncias novas já estão saudáveis no target group quando o programa começa. Às 21h12, a Cantina da Nona aparece na tela com o endereço embaixo. Em três minutos, o ALB passa de 900 para 7.600 requisições por minuto, e a target tracking leva o grupo ao máximo de 6. O pico fica em 7.800 por minuto, umas 1.300 por instância, abaixo das 1.400 que cada uma aguenta pela conta do capítulo 13. O front quase não pesa nessa conta: o CloudFront entrega o `index.html` e o bundle pela borda de São Paulo, com 97% de acerto no cache. Até a meia-noite, são 1.140 pedidos, sem nenhum `502` no ALB e sem nenhum `too many clients` no banco. Você passa a noite olhando os gráficos como quem assiste a uma final de campeonato, com a diferença de que o seu time ganha. À 0h30, a ação `especial-tv-fim` devolve o mínimo a 2, e a política remove o excesso ao longo da madrugada.

A desmontagem

No domingo de manhã, com o especial encerrado, o que cobra por hora volta a ser desligado. Quem está acompanhando sem um programa de TV marcado não precisa esperar o sábado para desmontar, e o `--force-delete` leva as ações agendadas junto com o grupo. A ordem começa pelo health check e pelo registro da API, que apontariam para um ALB inexistente. Para apagar um registro, o Route 53 exige o registro exato, igual ao que foi criado. É o único sistema que pede uma descrição perfeita da coisa que você quer jogar fora:

```
aws route53 delete-health-check --health-check-id "$HC_ID"

aws route53 change-resource-record-sets --hosted-zone-id "$ZONE_ID" \
  --change-batch "{\"Changes\":[{\"Action\":\"DELETE\",\"ResourceRecordSet\":
{\"Name\":\"api.rango.com.br\",\"Type\":\"A\",\"AliasTarget\":
{\"HostedZoneId\":\"$ALB_ZONE\",\"DNSName\":\"$ALB_DNS\",\"EvaluateTargetHealth\":true}}
}]}"
```

Depois, a mesma sequência do capítulo 13, com o snapshot novo:

```
aws autoscaling delete-auto-scaling-group \
  --auto-scaling-group-name rango-api --force-delete

aws elbv2 delete-load-balancer --load-balancer-arn "$ALB_ARN"
aws elbv2 wait load-balancers-deleted --load-balancer-arns "$ALB_ARN"
aws elbv2 delete-target-group --target-group-arn "$TG_ARN"

aws elasticache delete-serverless-cache --serverless-cache-name rango-cache

aws rds modify-db-instance --db-instance-identifier rango-db \
  --no-deletion-protection --apply-immediately
aws rds delete-db-instance --db-instance-identifier rango-db \
  --final-db-snapshot-identifier rango-db-final-cap14
```

```
aws rds wait db-instance-deleted --db-instance-identifier rango-db
aws rds delete-db-snapshot --db-snapshot-identifier rango-db-final-cap13
```

Ficam a hosted zone (US$ 0,50 por mês), o certificado, o bucket `rango-front-<conta>`, o OAC e a distribuição, que no pay-as-you-go não cobra nada parada. O site `https://rango.com.br` continua carregando, só que a lista de restaurantes vai falhar por falta de API. A versão 2 do Launch Template também fica, com um endpoint de cache que já não existe dentro do user data. O capítulo 15 reconstrói tudo isso de outro jeito.

Armadilhas e casos reais

Armadilha: Certificado na região errada. Você pede o certificado em `sa-east-1`, porque o ALB está lá, e tenta usá-lo no CloudFront. Ele nem aparece na lista do console, e pela CLI a criação falha com `InvalidViewerCertificate`. Certificados do ACM não podem ser movidos entre regiões: peça outro em `us-east-1`. Com o certificado certo, o mesmo erro aparece quando um nome alternativo da distribuição não está entre os SANs.

Armadilha: Hosted zone recriada, delegação antiga. Ao apagar e recriar uma hosted zone, a AWS atribui outros quatro servidores de nome. O registrador continua apontando para os antigos, que agora não respondem pelo seu domínio, e o site some para quem não tiver a resposta em cache. É mudar de casa e esquecer de avisar os Correios, só que quem fica sem receber é o cliente. Sempre que recriar a zona, atualize o Registro.br. O capítulo 15 traz a zona para o código sem apagá-la.

Armadilha: Registro de validação apagado. Alguém faz uma "limpeza" na zona, com o entusiasmo de quem arruma a gaveta dos outros, e remove aqueles CNAMEs de nome estranho. Nada acontece por meses. Quando o ACM tenta renovar, a validação falha, e ele avisa por e-mail e pelo AWS Health Dashboard. Se ninguém olhar, o certificado vence. Com validade de 198 dias, a chance de alguém esquecer o que aqueles registros fazem entre uma renovação e outra só aumenta.

Armadilha: Domínio preso em outra distribuição. Um nome alternativo só pode estar em uma distribuição de cada vez. Se você criar uma segunda distribuição com `rango.com.br`, a operação falha com `CNAMEAlreadyExists`. Isso vai aparecer no capítulo 15, quando o código tentar criar a distribuição que hoje existe só pelo console: a antiga precisa liberar o nome primeiro.

Armadilha: Price class barata demais. O `PriceClass_100` custa menos por GB porque usa só bordas da América do Norte, da Europa e de Israel. O `PriceClass_200` acrescenta Ásia, Oriente Médio e África. Nenhum dos dois inclui a América do Sul. Com eles, a sócia em Recife seria atendida por uma borda nos

Estados Unidos, como pedir comida na lanchonete da esquina e ela sair de Miami, e o ganho de latência desapareceria quase todo. Para um público brasileiro, use `PriceClass_All`.

Armadilha: O mapeamento de erro esconde erros. O `CustomErrorResponses` vale para a distribuição inteira. Se um arquivo de `assets/` sumir do bucket, o CloudFront entrega o `index.html` com status `200` no lugar dele, e o Chrome mostra `Failed to load module script: Expected a JavaScript module script but the server responded with a MIME type of "text/html"`. É o que acontece se você usar `aws s3 sync --delete` num deploy enquanto alguém está com a versão anterior aberta: o navegador pede um pedaço antigo, sob demanda, e recebe HTML. Mantenha os assets antigos por alguns dias e, quando o tráfego crescer, troque o mapeamento por uma CloudFront Function que reescreve para `/index.html` só os caminhos sem extensão.

Armadilha: Registro órfão. Um `cdn.empresa.com.br` aponta para o endpoint de site de um bucket S3 que foi apagado. Como nomes de bucket são globais, qualquer conta da AWS pode criar um bucket com aquele nome e servir conteúdo no seu subdomínio, um ataque chamado *subdomain takeover*. Ao apagar um recurso, apague o registro junto, como fizemos com o `api.rango.com.br`.

Na vida real: Em 6 de dezembro de 2018, milhões de clientes da operadora O2, no Reino Unido, ficaram sem dados móveis por quase um dia inteiro, e a SoftBank, no Japão, teve uma queda de mais de quatro horas. A causa, segundo a análise inicial da Ericsson, que fornecia o software do núcleo das duas redes, foi um certificado vencido embutido em duas versões do software dos nós SGSN-MME. Operadoras em cerca de 11 países foram afetadas. A renovação automática do ACM existe para tirar essa data das mãos das pessoas, desde que o registro de validação continue na zona.

Resumo

- O Route 53 hospeda a zona do `rango.com.br` (hosted zone pública, US$ 0,50 por mês) com quatro servidores de nome, informados ao Registro.br. Registrador e provedor de DNS são papéis separados.
- O registro alias responde como um A, mas aponta para um recurso da AWS e acompanha os IPs dele. Ele resolve o CNAME proibido no apex, e as consultas a alias para ELB e CloudFront não são cobradas.
- Health checks do Route 53 testam o endpoint de fora, de várias regiões. As políticas de roteamento (simples, ponderada, latência, failover) decidem a resposta.

- O ACM emite certificados públicos gratuitos para ALB e CloudFront e os renova sozinho com validação DNS. Desde fevereiro de 2026, eles saem com 198 dias de validade, e a renovação começa 45 dias antes. Para o CloudFront, o certificado precisa estar em `us-east-1`.
- O CloudFront entrega o front a partir de bordas próximas do usuário. O bucket fica privado, e o OAC assina as requisições da distribuição, com uma bucket policy restrita ao ARN dela.
- SPA no CloudFront: `index.html` com `no-cache`, assets com hash e `max-age` de um ano, `403` e `404` mapeados para o `index.html`, e `PriceClass_All` para ter bordas no Brasil.
- O Rangô tem agora `https://rango.com.br` (CloudFront) e `https://api.rango.com.br` (ALB com listener HTTPS e redirecionamento da porta 80), com um único certificado para os três nomes.

Arqueologia

Vale contar os comandos usados para colocar o Rangô no ar desde o capítulo 9: subnets, rotas, Security Groups que dependem uns dos outros, um banco restaurado de um snapshot com nome de capítulo, uma AMI, duas versões de Launch Template, uma distribuição com um JSON de quarenta linhas, registros de DNS com aspas escapadas. Cada recriação "com os comandos do capítulo anterior" trouxe uma surpresa, como o endpoint do cache que mudou.

Agora você precisa de uma homologação idêntica à produção. Qual porta o `rango-endpoints` liberou? Que versão do Launch Template está valendo, e por quê? Ninguém lembra o que foi clicado, e recriar o ambiente virou arqueologia, com pincelzinho de escavação e muito mais aspas escapadas.

No próximo capítulo, o Rangô inteiro vira código Python, com o AWS CDK por cima do AWS CloudFormation. Um `cdk deploy` cria a VPC, o banco, o ALB, o ASG, a distribuição e os registros de DNS, e um `cdk destroy` apaga tudo sem deixar sobra. A partir dele, todo exemplo do livro usa CDK.

Referências

- Preços do Amazon Route 53 (hosted zones, consultas, alias, health checks): https://aws.amazon.com/route53/pricing/
- Escolha entre registros alias e não alias: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-

sets-choosing-alias-non-alias.html

- Health checks e failover de DNS no Route 53: https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html
- Exemplos de Route 53 no CloudFormation (hosted zone ID `Z2FDTNDATAQYW2` para CloudFront): https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/quickref-route53.html
- Anúncio da validade padrão de 198 dias no ACM (fevereiro de 2026): https://aws.amazon.com/about-aws/whats-new/2026/02/aws-certificate-manager-updates-default
- Características dos certificados do ACM (validação e renovação): https://docs.aws.amazon.com/acm/latest/userguide/acm-certificate-characteristics.html
- Nomes alternativos e HTTPS no CloudFront (certificado em `us-east-1`): https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cnames-and-https-procedures.html
- Restringir acesso a uma origem S3 com OAC: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html
- Políticas de cache gerenciadas do CloudFront: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-managed-cache-policies.html
- Planos de preço fixo do CloudFront: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/flat-rate-pricing-plan.html
- Preços do Amazon CloudFront e price classes: https://aws.amazon.com/cloudfront/pricing/
- Listeners HTTPS do Application Load Balancer: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-https-listener.html
- Variáveis de ambiente e modos no Vite: https://vite.dev/guide/env-and-mode
- O que causou as quedas da O2 e da SoftBank (TechCrunch, 7 de dezembro de 2018): https://techcrunch.com/2018/12/07/heres-what-caused-yesterdays-o2-and-softbank-outages
- Ericsson pede desculpas pela queda da O2 (Computing): https://www.computing.co.uk/news/3067847/ericsson-apologises-for-o2-network-outage

CAPÍTULO 15

Infraestrutura como código com CDK

No capítulo 14, o Rangô ganhou endereço de verdade. O Route 53 hospeda a zona `rango.com.br`, o ACM emitiu um certificado para o apex, o `www` e o `api`, e o React passou a sair de um bucket privado pelas bordas do CloudFront. A API ganhou listener HTTPS no ALB e aguentou o especial de sábado com seis instâncias. No domingo, desligamos o que cobra por hora: o ASG, o ALB, o banco e o cache foram apagados, e ficaram a zona, o certificado, a distribuição, o bucket do front, a versão 2 do Launch Template e o snapshot `rango-db-final-cap14`.

Homologação para quinta-feira

Segunda-feira, a sócia manda uma mensagem. O Bruno, do Sabor do Sertão, quer testar o cardápio novo antes de ele ir para o ar, e ela prometeu um ambiente de homologação até quinta. "É só copiar o que já existe, né?" Toda frase de trabalho que termina em "né?" esconde, em média, três dias de esforço não previsto.

Você abre os capítulos anteriores e começa a contar. A VPC do capítulo 10 tem quatro subnets, duas route tables, um Internet Gateway, um gateway endpoint para o S3 e três Security Groups que dependem uns dos outros. O banco do capítulo 12 precisa de subnet group, parameter group e um snapshot com nome de capítulo. O capítulo 13 trouxe AMI, Launch Template, target group, ALB, ASG e política de escala. O 14 acrescentou listener HTTPS, distribuição, OAC e registros de DNS com aspas escapadas. Passa de cem comandos, e cada um devolve um ID que o próximo precisa receber. É uma corrida de revezamento em que o bastão é uma string como `sg-0a1b2c3d4e5f67890`, e quem deixa o bastão cair só descobre na quarta-feira.

Na terça à noite, a homologação sobe. Na quarta de manhã, o Bruno avisa que a lista de restaurantes dá erro. O log da instância mostra:

```
redis.exceptions.ConnectionError: Error -2 connecting to rango-cache-
x7k2pq.serverless.use1.cache.amazonaws.com:6379. Name or service not known.
```

A versão 2 do Launch Template ainda carrega no user data o endpoint de um cache que foi apagado no capítulo 14. Você corrige. Depois descobre que esqueceu o gateway endpoint do S3, e o upload de fotos trava num timeout. Depois percebe que a porta 443

do grupo do ALB foi aberta na produção e não na homologação. Na quinta, os dois ambientes funcionam, mas ninguém sabe dizer em quantos detalhes eles diferem. É o jogo dos sete erros, sem a garantia de que são sete.

Esse é o problema que o capítulo 6 chamou de *drift* e de servidor floco de neve, agora na escala de uma conta inteira. O estado da infraestrutura mora em três lugares: na AWS, no seu histórico de terminal e na sua memória, um armazenamento sem backup, sem versionamento e que perde dados toda vez que você dorme mal. Só o primeiro é confiável, e ele não diz como chegou ali.

A infraestrutura como uma migration

O capítulo 6 apresentou a ideia: descrever a infraestrutura num arquivo versionado, como uma migration descreve o banco. Você declara o estado que quer, e uma ferramenta compara com o que existe e faz a diferença. Na AWS, a ferramenta que faz isso desde 2011 é o **AWS CloudFormation**, que, em anos de nuvem, já tem idade para ser chamado de senhor.

CloudFormation por dentro

O CloudFormation recebe um *template*, um arquivo JSON ou YAML que lista recursos. Escolher entre os dois é escolher entre chaves demais e espaços demais. Cada recurso tem um tipo, como `AWS::EC2::VPC` ou `AWS::RDS::DBInstance`, e propriedades. Um pedaço de template com um bucket fica assim:

```yaml
Resources:
  FrontBucket:
    Type: AWS::S3::Bucket
    Properties:
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        BlockPublicPolicy: true
        IgnorePublicAcls: true
        RestrictPublicBuckets: true
```

Você entrega o template, e o CloudFormation cria uma *stack*: o conjunto de recursos nascidos daquele template, gerenciados juntos. A stack é a unidade de tudo. Você cria, atualiza e apaga a stack, e os recursos vão junto.

Duas palavras vão aparecer o capítulo inteiro. O **logical ID** é o nome do recurso dentro do template (`FrontBucket`, no exemplo). O **physical ID** é o nome real na AWS, como `rangoproducao-frontbucket-1a2b3c4d5e6f`, um nome que só um computador consegue amar. O CloudFormation guarda o mapa entre os dois. É por esse mapa que ele sabe que o `FrontBucket` do template de hoje é o mesmo bucket de ontem.

Quando um recurso cita outro (a subnet cita a VPC, o banco cita a subnet), o CloudFormation monta um grafo de dependências. Ele cria em paralelo o que é independente e espera o que depende. Você não escreve a ordem, igual a uma migration que não precisa saber em que ordem o Postgres grava as páginas no disco.

Na atualização, o CloudFormation compara o template novo com o anterior e calcula o que muda. Você pode pedir essa lista antes de aplicar, num *change set*. Para cada recurso, a mudança cai em uma de três categorias: atualizar sem interrupção, atualizar com interrupção (um reboot, por exemplo) ou substituir. Substituir quer dizer criar um recurso novo e apagar o antigo. Para um bucket vazio de front, tanto faz. Para o banco de pedidos, é a diferença entre uma terça tranquila e um postmortem.

Rollback

Se qualquer recurso falha durante uma operação, o CloudFormation desfaz o que fez e volta ao último estado bom. Isso é o *rollback*, e ele é automático: o Ctrl+Z da infraestrutura, sem que ninguém precise apertar nada. Uma atualização que falha termina em `UPDATE_ROLLBACK_COMPLETE`, com a stack como estava antes.

Uma criação que falha termina em `ROLLBACK_COMPLETE`. A stack fica lá, vazia, e não aceita atualização. A única saída é apagá-la e tentar de novo. É o equivalente em nuvem de desligar e ligar de novo: ninguém se orgulha, mas resolve.

O caso feio é o `UPDATE_ROLLBACK_FAILED`: o rollback não conseguiu desfazer alguma coisa, quase sempre porque alguém mexeu num recurso por fora. Se uma pessoa apaga pelo console um Security Group que a stack gerencia, e uma atualização depois falha, o rollback tenta restaurar o grupo antigo e não encontra. É como tentar devolver a cadeira à mesa e descobrir que alguém levou a mesa. A saída é corrigir a causa e rodar `aws cloudformation continue-update-rollback`. Em último caso, a opção `--resources-to-skip` pula os recursos travados (só aceita os que estão em `UPDATE_FAILED`), e eles ficam marcados como atualizados, mesmo que a realidade não bata com o template.

Drift

Isso leva ao drift. O CloudFormation só sabe o que ele mesmo fez. Se alguém abre a porta 22 de um Security Group pelo console, o template continua dizendo que ela está fechada. O CloudFormation tem uma detecção de drift que compara as propriedades declaradas no template com as reais e aponta as diferenças. Ela detecta, mas não corrige, como o exame de sangue, que aponta o colesterol e deixa a dieta com você: a decisão de voltar ao template ou de atualizar o template fica com você.

A regra que faz tudo isso funcionar é a do capítulo 6: depois que um recurso nasce pelo código, o console vira somente leitura para ele. O botão de editar continua lá, brilhando, e resistir a ele exige a disciplina de um monge. Daqui em diante, todo exemplo usa CDK, com poucas exceções avisadas no próprio capítulo: uma variável do GitHub no 18, o SLO e o Transaction Search no 22, o Security Hub no 23 e a policy do bucket de fotos, que mora fora da stack, no 24.

CloudFormation não cobra pelos tipos `AWS::*`; você paga só pelos recursos criados. Os limites que importam para o Rangô são 500 recursos por template e 2.000 stacks por região por conta (referência, outubro de 2026; confira os valores da sua conta com `aws cloudformation describe-account-limits`).

Por que não escrever o YAML direto

Dá para escrever o Rangô inteiro em YAML. Muita gente faz isso, e funciona. Muita gente também atravessa o Atlântico a remo. O YAML, aliás, tem as suas manias: na versão 1.1 da especificação, o código de país da Noruega, `NO`, é lido como o booleano falso, e há quem tenha descoberto isso numa lista de países. O problema aparece no tamanho. A VPC do capítulo 10, com subnets, rotas, associações e gateway, passa de 150 linhas de YAML. O Security Group do banco que aceita conexões do grupo da API vira dois recursos e uma referência cruzada. Quer dois ambientes? Você copia o arquivo ou aprende a linguagem de condições e funções intrínsecas do CloudFormation (`Fn::If`, `Fn::Sub`, `Fn::GetAtt`), que tem tudo de uma linguagem de programação, menos o conforto.

O **AWS CDK** (Cloud Development Kit) resolve isso escrevendo a infraestrutura numa linguagem de verdade, Python no nosso caso, e gerando o template do CloudFormation a partir dela. O CDK não fala com as APIs dos serviços para criar recursos. Ele produz um template e entrega ao CloudFormation, que faz o resto. Stack, rollback, drift e change set continuam valendo exatamente como acabamos de ver.

A relação lembra a do Django ORM com o SQL. Você escreve `class Pedido(models.Model)`, e o `makemigrations` gera o SQL. No CDK, você escreve `s3.Bucket(...)`, e o `cdk synth` gera o template. Dá para olhar o resultado gerado sempre que quiser, e às vezes você vai precisar, quase sempre numa sexta-feira às seis da tarde.

Constructs em três níveis

Tudo no CDK é um *construct*: um objeto Python que representa um ou mais recursos. Eles vêm em três níveis.

O **L1** é o recurso do CloudFormation traduzido um a um. As classes começam com `Cfn`, como `CfnBucket` ou `CfnServerlessCache`, e as propriedades são as mesmas do template, sem padrão nenhum. É o YAML com sintaxe de Python: todo o desconforto de antes, agora com `import`.

O **L2** é a camada que faz o CDK valer a pena. Ele vem com opiniões, como todo colega sênior, com a diferença de que as opiniões dele costumam estar certas. O `s3.Bucket` já nasce com Block Public Access, o `ec2.Vpc` cria subnets, rotas e gateways de uma vez, e o `rds.DatabaseInstance` gera uma senha e guarda no Secrets Manager se você não informar uma. Os L2 também trazem métodos que geram permissões e regras de rede, como `bucket.grant_read(role)` e `db.connections.allow_default_port_from(api)`.

O **L3**, chamado de *pattern*, junta vários recursos numa arquitetura pronta. O capítulo 17 vai usar um deles para subir um serviço de containers atrás de um ALB em poucas linhas.

Na prática, você usa L2 quase sempre e desce para L1 quando o serviço ainda não tem L2. O cache do Rangô vai ser exatamente esse caso.

O que o CDK precisa para rodar

O CDK tem duas partes. A biblioteca, `aws-cdk-lib`, é o pacote Python que o seu código importa. A CLI, o comando `cdk`, roda em Node.js, mesmo quando o projeto é Python. Você vai precisar dos dois. Para quem é do Python, é como descobrir que o carro elétrico precisa de um galão de gasolina para dar a partida. O CDK v1 teve o suporte encerrado em junho de 2023; tudo aqui usa o v2, que vem num pacote único.

Antes do primeiro deploy, cada par de conta e região passa por um *bootstrap*. O comando `cdk bootstrap` cria uma stack chamada `CDKToolkit` com um bucket S3 e um repositório ECR, onde o CDK guarda templates e arquivos (os *assets*) antes de entregá-los ao CloudFormation, e algumas roles IAM que o deploy usa. O custo é o armazenamento desses arquivos, alguns centavos por mês. O bootstrap é como a mudança de casa: ninguém quer fazer, mas sem ele não dá para pendurar quadro nenhum.

Na prática: o Rangô inteiro em Python

Como nos capítulos anteriores, o código e as saídas aparecem por inteiro, e dá para acompanhar tudo só lendo. O plano: um projeto CDK com uma stack, `RangoProducao`, que cria a rede, o banco restaurado do snapshot, o cache, a API com ALB e ASG, o bucket do front, a distribuição e os registros de DNS. A zona e o certificado ficam de fora da stack. Eles vivem mais que qualquer ambiente, e recriar a zona trocaria os servidores de nome que estão no Registro.br.

Pré-requisitos

Node.js 22 ou 24. O suporte do CDK ao Node 20 termina em 30 de outubro de 2026, daqui a duas semanas na data em que escrevo, e o 18 já saiu em novembro de 2025. No ritmo do Node.js, quem piscar perde uma versão LTS. Python 3.12, a AWS CLI v2 e os profiles do capítulo 8.

```
node --version
npm install -g aws-cdk
cdk --version
```

Bootstrap com o profile certo

O bootstrap cria roles IAM. O `rango-dev`, com `PowerUserAccess`, não pode criar roles, então este é um dos raros momentos para o `rango-admin`, o profile que, como a louça boa da casa, só sai do armário em ocasião especial:

```
aws sso login --profile rango-admin
cdk bootstrap aws://123456789012/us-east-1 --profile rango-admin
```

```
⏳  Bootstrapping environment aws://123456789012/us-east-1...
✅  Environment aws://123456789012/us-east-1 bootstrapped.
```

Numa conta real, o `123456789012` dá lugar ao ID da conta (`aws sts get-caller-identity`). Vale saber o que esse comando criou. Entre as roles da `CDKToolkit` há uma role de execução que o CloudFormation assume para criar os recursos, e ela recebe `AdministratorAccess` por padrão. Na prática, quem pode fazer deploy pelo CDK pode criar qualquer coisa na conta. Voltamos a isso nas armadilhas.

O projeto

O projeto de infraestrutura mora no mesmo repositório, ao lado de `backend/` e `frontend/`. A família cresceu, e o novo irmão já chega com opinião sobre a rede de todo mundo:

```
mkdir rango-infra && cd rango-infra
cdk init app --language python
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
```

O `cdk init` exige a pasta vazia (ele não divide espaço com o seu `rascunho-final-v3.py`) e gera esta estrutura (o hífen do nome vira sublinhado no pacote Python):

```
rango-infra/
├── app.py
├── cdk.json
├── requirements.txt
├── requirements-dev.txt
```

```
├── rango_infra/
│   ├── __init__.py
│   └── rango_infra_stack.py
└── tests/
```

O `requirements.txt` já vem com a versão do `aws-cdk-lib` fixada com `==`, e é bom que continue assim. O pip não tem lockfile, e um `pip install` de outra pessoa daqui a três meses traria outra versão, com outros padrões. É o clássico "ontem funcionava", agora com versionamento semântico. Na hora de atualizar, o caminho é editar o número à mão, em vez de usar `pip freeze`, que despejaria no arquivo todas as dependências transitivas.

O `cdk.json` diz à CLI como rodar o app (`python3 app.py`) e guarda o *context*, um dicionário de configuração. É ali que entra o ARN do certificado do capítulo 14, que não é segredo:

```
{
  "app": "python3 app.py",
  "context": {
    "certificado_arn": "arn:aws:acm:us-east-1:123456789012:certificate/0a1b2c3d-4e5f-
6789-abcd-ef0123456789"
  }
}
```

O arquivo gerado tem mais linhas dentro de `context`, com as *feature flags* do CDK. Elas ficam todas, e a única novidade é a chave do certificado.

O app

O `app.py` instancia a stack. É aqui que, no futuro, entra a homologação: outra chamada à mesma classe, com outros parâmetros. Um ambiente novo passa a custar uma chamada de função, em vez de uma semana e um pedido de desculpas ao Bruno.

```python
#!/usr/bin/env python3
import os

import aws_cdk as cdk

from rango_infra.rango_infra_stack import RangoInfraStack

app = cdk.App()

RangoInfraStack(
    app,
    "RangoProducao",
    dominio="rango.com.br",
    certificado_arn=app.node.try_get_context("certificado_arn"),
    snapshot_db="rango-db-final-cap14",
    env=cdk.Environment(
        account=os.environ["CDK_DEFAULT_ACCOUNT"],
        region="us-east-1",
    ),
)
```

```
cdk.Tags.of(app).add("Projeto", "rango")

app.synth()
```

A CLI preenche `CDK_DEFAULT_ACCOUNT` a partir das suas credenciais. Declarar conta e região deixa a stack amarrada a um lugar, o que permite ao CDK consultar a conta durante a síntese, como veremos na zona e na AMI. O `Tags.of(app)` aplica a tag `Projeto=rango` em todo recurso que aceita tags, de uma vez só, a mesma tag que colocamos à mão desde o capítulo 7.

A rede

Agora a stack, em `rango_infra/rango_infra_stack.py`. O começo traz os imports e a VPC:

```
from aws_cdk import (
    CfnOutput,
    Duration,
    RemovalPolicy,
    Stack,
    aws_autoscaling as autoscaling,
    aws_certificatemanager as acm,
    aws_cloudfront as cloudfront,
    aws_cloudfront_origins as origins,
    aws_ec2 as ec2,
    aws_elasticache as elasticache,
    aws_elasticloadbalancingv2 as elbv2,
    aws_iam as iam,
    aws_rds as rds,
    aws_route53 as route53,
    aws_route53_targets as targets,
    aws_s3 as s3,
)
from constructs import Construct


class RangoInfraStack(Stack):
    def __init__(
        self,
        scope: Construct,
        construct_id: str,
        *,
        dominio: str,
        certificado_arn: str,
        snapshot_db: str,
        **kwargs,
    ) -> None:
        super().__init__(scope, construct_id, **kwargs)

        vpc = ec2.Vpc(
            self,
            "Rede",
            ip_addresses=ec2.IpAddresses.cidr("10.20.0.0/16"),
            max_azs=2,
            nat_gateways=0,
            subnet_configuration=[
                ec2.SubnetConfiguration(
```

```
                name="publica", subnet_type=ec2.SubnetType.PUBLIC, cidr_mask=24
            ),
            ec2.SubnetConfiguration(
                name="privada",
                subnet_type=ec2.SubnetType.PRIVATE_ISOLATED,
                cidr_mask=24,
            ),
        ],
    )
    vpc.add_gateway_endpoint(
        "S3",
        service=ec2.GatewayVpcEndpointAwsService.S3,
        subnets=[ec2.SubnetSelection(subnet_type=ec2.SubnetType.PRIVATE_ISOLATED)],
    )
```

Essas linhas fazem o trabalho de uma seção inteira do capítulo 10: duas subnets públicas com rota para um Internet Gateway, duas privadas sem rota para a internet, as route tables, as associações e o gateway endpoint do S3 nas privadas. Quem sofreu no capítulo 10 tem todo o direito de se sentir levemente traído.

O detalhe que importa é o `nat_gateways=0`. Sem ele, o `ec2.Vpc` cria por padrão um NAT Gateway em cada zona de disponibilidade, com subnets privadas do tipo `PRIVATE_WITH_EGRESS`. É o padrão sensato para muitas aplicações e a promessa que ficou no capítulo 10: com duas AZs, seriam dois NAT Gateways cobrando por hora sem que o Rangô precise deles. Um NAT Gateway esquecido é a forma mais cara que existe de aprender a ler a fatura. As instâncias da API não saem para a internet desde o capítulo 13. A AMI já tem tudo instalado, o S3 chega pelo gateway endpoint, e o banco e o cache estão dentro da VPC. Por isso as privadas são `PRIVATE_ISOLATED`.

O CIDR é o mesmo `10.20.0.0/16` da VPC feita à mão, que vai ser apagada no fim do capítulo. Duas VPCs com a mesma faixa podem coexistir enquanto não forem conectadas.

Permissões e grupos de segurança

```
    sg_api = ec2.SecurityGroup(
        self, "SgApi", vpc=vpc, description="Instancias da API do Rango"
    )
    sg_cache = ec2.SecurityGroup(
        self, "SgCache", vpc=vpc, description="Valkey serverless do Rango"
    )
    sg_cache.add_ingress_rule(sg_api, ec2.Port.tcp(6379), "API no Valkey")

    role_api = iam.Role(
        self,
        "RoleApi",
        assumed_by=iam.ServicePrincipal("ec2.amazonaws.com"),
        managed_policies=[
            iam.ManagedPolicy.from_aws_managed_policy_name(
                "AmazonSSMManagedInstanceCore"
            )
        ],
    )
    role_api.add_to_policy(
```

```
            iam.PolicyStatement(
                actions=["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
                resources=[f"arn:aws:s3:::rango-fotos-pratos-{self.account}/fotos/*"],
            )
        )
```

As descrições ficam sem acento pelo mesmo motivo do capítulo 9: a API de Security Groups recusa caracteres fora do ASCII nesse campo. O til e a cedilha ainda precisam de visto para entrar em certos cantos da AWS.

A role repete a `rango-ec2` dos capítulos 8 e 11: Session Manager e as três ações em `fotos/*`, sem `ListBucket`. O bucket de fotos fica fora da stack de propósito. Ele guarda dados que não podem sumir num `cdk destroy`, e a stack só precisa saber o nome dele. As fotos dos pratos são o álbum de família do Rangô, e álbum de família não entra na caçamba da reforma. O L2 de bucket tem métodos como `grant_read_write`, mas eles concedem ações a mais, inclusive listagem; aqui o `PolicyStatement` explícito reproduz exatamente a policy do capítulo 11.

Banco e cache

```
        db = rds.DatabaseInstanceFromSnapshot(
            self,
            "Banco",
            snapshot_identifier=snapshot_db,
            instance_identifier="rango-db",
            engine=rds.DatabaseInstanceEngine.postgres(
                version=rds.PostgresEngineVersion.of("18.4", "18")
            ),
            instance_type=ec2.InstanceType.of(
                ec2.InstanceClass.BURSTABLE4_GRAVITON, ec2.InstanceSize.MICRO
            ),
            vpc=vpc,
            vpc_subnets=ec2.SubnetSelection(
                subnet_type=ec2.SubnetType.PRIVATE_ISOLATED
            ),
            credentials=rds.SnapshotCredentials.from_generated_secret("rango_admin"),
            parameter_group=rds.ParameterGroup(
                self,
                "ParametrosPg",
                engine=rds.DatabaseInstanceEngine.postgres(
                    version=rds.PostgresEngineVersion.of("18.4", "18")
                ),
                parameters={"log_min_duration_statement": "500"},
            ),
            storage_type=rds.StorageType.GP3,
            max_allocated_storage=100,
            backup_retention=Duration.days(7),
            deletion_protection=False,
            removal_policy=RemovalPolicy.SNAPSHOT,
        )
        db.connections.allow_default_port_from(sg_api, "API no Postgres")

        cache = elasticache.CfnServerlessCache(
            self,
            "Cache",
```

```
            serverless_cache_name="rango-cache",
            engine="valkey",
            subnet_ids=vpc.select_subnets(
                subnet_type=ec2.SubnetType.PRIVATE_ISOLATED
            ).subnet_ids,
            security_group_ids=[sg_cache.security_group_id],
            cache_usage_limits=elasticache.CfnServerlessCache.CacheUsageLimitsProperty(
                data_storage=elasticache.CfnServerlessCache.DataStorageProperty(
                    maximum=1, unit="GB"
                ),
                ecpu_per_second=elasticache.CfnServerlessCache.ECPUPerSecondProperty(
                    maximum=5000
                ),
            ),
        )
```

O banco nasce do snapshot do capítulo 14, como nos capítulos 13 e 14, com o usuário `rango`, a senha dele e os pedidos de teste. O `from_generated_secret("rango_admin")` gera uma senha nova para o usuário mestre e a guarda no Secrets Manager. O CDK também cria o subnet group e o parameter group (família `postgres18`, deduzida da versão), que antes eram comandos separados.

A `removal_policy` diz o que acontece com o recurso quando ele sai da stack. Com `SNAPSHOT`, o CloudFormation tira um snapshot final antes de apagar a instância. Essa política deve aparecer sempre, declarada, no banco, em vez de ficar por conta do padrão: é a linha que separa um `cdk destroy` de uma perda de dados, e de um currículo atualizado às pressas.

A linha `db.connections.allow_default_port_from(sg_api, ...)` merece atenção. No capítulo 10, você criou o grupo `rango-db` e uma regra de entrada na porta 5432 com origem no grupo `rango-web`. Aqui, o banco sabe a própria porta, e o método gera a mesma regra. Se a porta mudar um dia, a regra acompanha.

O cache é o L1 do capítulo. O ElastiCache Serverless não tem construct L2 estável na `aws-cdk-lib`, então usamos o `CfnServerlessCache`, que é o recurso `AWS::ElastiCache::ServerlessCache` com nomes de propriedade em snake_case. Ao lado do comando do capítulo 12, são os mesmos campos, um a um, e sem nenhuma ajuda. O L1 é aquele colega que faz exatamente o que você pediu, nem uma vírgula a mais, e assiste em silêncio enquanto você erra. As subnets e o Security Group precisam ser passados como listas de IDs, e nada impede que você esqueça a regra de entrada.

A API

```
        user_data = ec2.UserData.for_linux()
        user_data.add_commands(
            "set -euo pipefail",
            "sed -i"
            f" -e
's|^REDIS_URL=.*|REDIS_URL=rediss://{cache.attr_endpoint_address}:6379|'"
```

```
            f" -e 's|@rango-db[^:]*:5432|@{db.db_instance_endpoint_address}:5432|'"
            f" -e 's|^CORS_ORIGINS=.*|CORS_ORIGINS=https://{dominio},https://www.
{dominio}|'"
            " /etc/rango/api.env",
            "systemctl restart rango-api rango-worker@restaurante rango-worker@email",
        )

        modelo = ec2.LaunchTemplate(
            self,
            "ModeloApi",
            machine_image=ec2.MachineImage.lookup(
                name="rango-api-20261015", owners=["self"]
            ),
            instance_type=ec2.InstanceType("t3.micro"),
            role=role_api,
            security_group=sg_api,
            user_data=user_data,
            require_imdsv2=True,
        )

        asg = autoscaling.AutoScalingGroup(
            self,
            "Api",
            vpc=vpc,
            vpc_subnets=ec2.SubnetSelection(
                subnet_type=ec2.SubnetType.PRIVATE_ISOLATED
            ),
            launch_template=modelo,
            min_capacity=2,
            max_capacity=6,
            health_checks=autoscaling.HealthChecks.with_additional_checks(
                additional_types=[autoscaling.AdditionalHealthCheckType.ELB],
                grace_period=Duration.seconds(90),
            ),
            default_instance_warmup=Duration.seconds(60),
        )

        certificado = acm.Certificate.from_certificate_arn(
            self, "Certificado", certificado_arn
        )

        alb = elbv2.ApplicationLoadBalancer(
            self, "Alb", vpc=vpc, internet_facing=True
        )
        https = alb.add_listener(
            "Https",
            port=443,
            certificates=
[elbv2.ListenerCertificate.from_certificate_manager(certificado)],
            ssl_policy=elbv2.SslPolicy.RECOMMENDED_TLS,
        )
        alb.add_redirect()

        https.add_targets(
            "AlvoApi",
            port=80,
            protocol=elbv2.ApplicationProtocol.HTTP,
            targets=[asg],
            health_check=elbv2.HealthCheck(
                path="/health",
                interval=Duration.seconds(10),
                healthy_threshold_count=2,
```

```
                unhealthy_threshold_count=2,
            ),
            deregistration_delay=Duration.seconds(30),
        )
        asg.scale_on_request_count("Requisicoes", target_requests_per_minute=600)
```

O ponto de partida é o user data, porque ele resolve o erro que abriu o capítulo. O endpoint do cache e o do banco entram no script como `cache.attr_endpoint_address` e `db.db_instance_endpoint_address`. No momento em que o Python roda, esses valores ainda não existem: o cache nem foi criado. O CDK coloca no lugar um *token*, uma marca que vira uma referência no template (um `Fn::GetAtt`), e o CloudFormation troca a referência pelo valor real na hora do deploy. O token funciona como um cheque pré-datado: o Python assina a promessa, e o CloudFormation desconta no dia certo. Isso cria também uma dependência: o Launch Template só nasce depois do cache e do banco. O endpoint velho preso no user data deixa de ser possível, porque nenhum endpoint fica escrito à mão.

O `sed` do banco troca só o host dentro do `DATABASE_URL` do capítulo 12, preservando usuário, porta e parâmetros de TLS. Com o `instance_identifier` fixo, o endpoint sai igual ao de antes, mas o script não depende disso. A senha do usuário `rango` continua no `api.env` da AMI, um remendo que o capítulo 16 remove.

O `MachineImage.lookup` procura a AMI `rango-api-20261015` na sua conta durante a síntese. Esse tipo de consulta, um *lookup*, é a razão de declarar conta e região no `app.py`. O resultado fica guardado em `cdk.context.json`, e as sínteses seguintes leem dali, sem consultar a AWS de novo. Esse arquivo vai para o commit: ele garante que todo mundo gera o mesmo template, mesmo que surja uma AMI nova com nome parecido.

O resto é o capítulo 13 em outra forma. O `HealthChecks.with_additional_checks` liga a verificação do ELB com o grace period de 90 segundos. O `add_listener` com certificado e `RECOMMENDED_TLS` é o listener HTTPS do capítulo 14, com a mesma política `ELBSecurityPolicy-TLS13-1-2-2021-06`. O `add_redirect()` cria o listener da porta 80 com o `301`. O `add_targets` cria o target group, registra o ASG nele e, sem você pedir, gera a regra no `sg_api` que aceita a porta 80 vindo do grupo do ALB. E o `scale_on_request_count` monta o target tracking por `ALBRequestCountPerTarget`, com o `ResourceLabel` que você teve de montar à mão no capítulo 13. Aquele `ResourceLabel`, com as suas barras e os seus pedaços de ARN, agora pode descansar em paz.

Front e DNS

```
        front = s3.Bucket(
            self,
            "Front",
            block_public_access=s3.BlockPublicAccess.BLOCK_ALL,
            enforce_ssl=True,
            removal_policy=RemovalPolicy.DESTROY,
```

```
            auto_delete_objects=True,
        )

        spa = [
            cloudfront.ErrorResponse(
                http_status=codigo,
                response_http_status=200,
                response_page_path="/index.html",
                ttl=Duration.seconds(0),
            )
            for codigo in (403, 404)
        ]

        distribuicao = cloudfront.Distribution(
            self,
            "Distribuicao",
            default_behavior=cloudfront.BehaviorOptions(
                origin=origins.S3BucketOrigin.with_origin_access_control(front),

viewer_protocol_policy=cloudfront.ViewerProtocolPolicy.REDIRECT_TO_HTTPS,
                cache_policy=cloudfront.CachePolicy.CACHING_OPTIMIZED,
            ),
            domain_names=[dominio, f"www.{dominio}"],
            certificate=certificado,
            default_root_object="index.html",
            error_responses=spa,
            price_class=cloudfront.PriceClass.PRICE_CLASS_ALL,
            http_version=cloudfront.HttpVersion.HTTP2_AND_3,
        )

        zona = route53.HostedZone.from_lookup(self, "Zona", domain_name=dominio)
        alvo_front = route53.RecordTarget.from_alias(
            targets.CloudFrontTarget(distribuicao)
        )
        for nome, sufixo in ((None, "Apex"), ("www", "Www")):
            route53.ARecord(
                self, f"Front{sufixo}A", zone=zona, record_name=nome, target=alvo_front
            )
            route53.AaaaRecord(
                self, f"Front{sufixo}Aaaa", zone=zona, record_name=nome,
target=alvo_front
            )
        route53.ARecord(
            self,
            "ApiA",
            zone=zona,
            record_name="api",
            target=route53.RecordTarget.from_alias(targets.LoadBalancerTarget(alb)),
        )

        CfnOutput(self, "FrontBucket", value=front.bucket_name)
        CfnOutput(self, "DistribuicaoId", value=distribuicao.distribution_id)
        CfnOutput(self, "AlbDns", value=alb.load_balancer_dns_name)
```

O `S3BucketOrigin.with_origin_access_control` cria o OAC e escreve a bucket policy com o `AWS:SourceArn` da distribuição, o JSON que você montou à mão em `infra/s3/rango-front-policy.json`. Uma linha de Python substitui o arquivo JSON

inteiro e todas as vírgulas que você esqueceu nele. O mapeamento de `403` e `404` para o `index.html` também está aqui, assim como a `PriceClass_All`, que mantém as bordas da América do Sul.

O bucket do front leva `RemovalPolicy.DESTROY` e `auto_delete_objects=True` porque o conteúdo dele é um build, que se gera de novo em um minuto. O CloudFormation não apaga bucket com objetos, então o CDK acrescenta um *custom resource*, uma pequena função Lambda que esvazia o bucket antes do delete. É o faxineiro contratado só para tirar as coisas do apartamento antes da entrega das chaves. O nome do bucket fica a cargo do CloudFormation, e ele sai na saída `FrontBucket`.

A zona entra por `from_lookup`. A stack não cria nem apaga a zona; ela só descobre o ID para escrever registros nela. A stack entra na zona como visita bem-educada: usa o que precisa e não mexe nos móveis. Com o certificado acontece o mesmo, pelo ARN.

Síntese

Antes de criar qualquer coisa, vale ver o que o Python vira:

```
export AWS_PROFILE=rango-dev
aws sso login
cdk synth > /dev/null
ls cdk.out/
grep -c '"Type": "AWS::' cdk.out/RangoProducao.template.json
```

O `cdk synth` executa o `app.py`, faz os lookups e grava o *cloud assembly* em `cdk.out/`, com um template por stack. Algumas dezenas de linhas de Python viram um template com dezenas de recursos e mais de mil linhas de JSON, longe do limite de 500. É como mandar uma frase para um advogado e receber de volta um contrato de trinta páginas. Ali aparecem os `AWS::EC2::Route`: são as rotas que você criou com `create-route` no capítulo 10.

O primeiro deploy, e o erro previsto

```
cdk deploy
```

Antes de criar, o CDK mostra as mudanças que ampliam permissões, numa tabela de statements IAM e outra de regras de Security Group, e pergunta `Do you wish to deploy these changes (y/n)?`. Essa tabela merece leitura de verdade, e não a leitura de termos de uso, em que o dedo desce direto até o botão de aceitar. É ali que aparece o `0.0.0.0/0` na porta 443 do ALB e a policy da role da API. A resposta, aqui, é `y`.

Uns quinze minutos depois, o deploy para:

```
RangoProducao | CREATE_FAILED | AWS::CloudFront::Distribution | Distribuicao...
Resource handler returned message: "Invalid request provided: One or more of the CNAMEs
```

```
you provided are already associated with a different resource."
...
RangoProducao | ROLLBACK_COMPLETE | AWS::CloudFormation::Stack | RangoProducao
```

É o `CNAMEAlreadyExists` que o capítulo 14 avisou. A distribuição criada pelo console, `E1ABCDEF2GHIJK`, ainda segura `rango.com.br` e `www.rango.com.br`, e um nome alternativo só pode estar em uma distribuição. O mais interessante é o que o CloudFormation fez: ao primeiro erro, ele cancelou o que estava em andamento e apagou tudo o que tinha criado, inclusive o banco restaurado. Nenhum recurso pela metade ficou para trás. Poucos hóspedes deixam o quarto tão arrumado.

A stack ficou em `ROLLBACK_COMPLETE`, e nesse estado ela não aceita atualização. A casca vazia precisa ser apagada:

```
aws cloudformation delete-stack --stack-name RangoProducao
aws cloudformation wait stack-delete-complete --stack-name RangoProducao
```

Depois, os nomes precisam ser liberados. Para apagar uma distribuição, o CloudFront exige que ela esteja desabilitada, e toda alteração precisa do `ETag` da versão atual. É a burocracia da nuvem, com direito a firma reconhecida:

```
DIST_VELHA=E1ABCDEF2GHIJK

ETAG=$(aws cloudfront get-distribution-config --id "$DIST_VELHA" \
  --query ETag --output text)
aws cloudfront get-distribution-config --id "$DIST_VELHA" \
  --query DistributionConfig > /tmp/dist.json
jq '.Enabled = false' /tmp/dist.json > /tmp/dist-off.json

aws cloudfront update-distribution --id "$DIST_VELHA" \
  --if-match "$ETAG" --distribution-config file:///tmp/dist-off.json
aws cloudfront wait distribution-deployed --id "$DIST_VELHA"

ETAG=$(aws cloudfront get-distribution --id "$DIST_VELHA" \
  --query ETag --output text)
aws cloudfront delete-distribution --id "$DIST_VELHA" --if-match "$ETAG"
```

Os registros alias de `rango.com.br` e `www` também precisam sair, porque a stack vai criá-los, e o Route 53 recusa criar um registro que já existe. Em vez de escrever cada um com aspas escapadas, o comando filtra pelo zone ID do CloudFront e transforma a lista em exclusões:

```
ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name rango.com.br \
  --query "HostedZones[0].Id" --output text)
ZONE_ID=${ZONE_ID##*/}

aws route53 list-resource-record-sets --hosted-zone-id "$ZONE_ID" \
  --query "ResourceRecordSets[?AliasTarget.HostedZoneId=='Z2FDTNDATAQYW2']" \
  | jq '{Changes: map({Action: "DELETE", ResourceRecordSet: .})}' \
  > /tmp/apagar-alias.json
```

```
aws route53 change-resource-record-sets --hosted-zone-id "$ZONE_ID" \
  --change-batch file:///tmp/apagar-alias.json
```

A partir daqui, `https://rango.com.br` fica fora do ar até o próximo deploy terminar. Para o Rangô de estudos, uns vinte minutos sem site são aceitáveis. Numa produção com clientes, você evitaria a janela trazendo a distribuição existente para dentro da stack com `cdk import`, que usa o *resource import* do CloudFormation para adotar um recurso sem recriá-lo.

Segunda tentativa, agora com o caminho livre:

```
cdk deploy
```

```
 ✅  RangoProducao

Outputs:
RangoProducao.AlbDns = Rango-AlbXX-1A2B3C4D5E6F-123456789.us-east-1.elb.amazonaws.com
RangoProducao.DistribuicaoId = E2KLMNOP3QRSTU
RangoProducao.FrontBucket = rangoproducao-front1a2b3c4d-5e6f7g8h9i0j
```

O tempo fica entre quinze e vinte e cinco minutos, quase todo esperando o banco e a distribuição. Dá para ver um episódio de série e ainda sobrar tempo para reclamar da barra de progresso. Falta o conteúdo do front, que fica fora da stack: ele muda a cada commit, e o capítulo 18 vai colocá-lo no pipeline. Por enquanto, os mesmos comandos do capítulo 14, com o bucket novo:

```
FRONT=$(aws cloudformation describe-stacks --stack-name RangoProducao \
  --query "Stacks[0].Outputs[?OutputKey=='FrontBucket'].OutputValue" --output text)

cd ../frontend
npm ci
npm run build
aws s3 sync dist/ "s3://$FRONT/" \
  --exclude index.html \
  --cache-control "public, max-age=31536000, immutable"
aws s3 cp dist/index.html "s3://$FRONT/index.html" \
  --cache-control "no-cache"
cd ../rango-infra
```

E o teste de ponta a ponta:

```
curl -s -o /dev/null -w "%{http_code}\n" https://rango.com.br/
curl -s https://api.rango.com.br/health
curl -sI https://api.rango.com.br/restaurantes | grep -i x-rango-instancia
```

```
200
{"status":"ok"}
x-rango-instancia: ip-10-20-2-87.ec2.internal
```

O `10.20.2.x` mostra a instância numa subnet privada nova: o CDK distribui as faixas em sequência, com as públicas em `10.20.0.0/24` e `10.20.1.0/24` e as privadas logo depois. No navegador, o site abre e a lista de restaurantes aparece, servida pelo cache que a instância encontrou sem que ninguém escrevesse o endpoint dele.

Mudar com segurança

Agora a parte do dia a dia. Toda alteração passa por `cdk diff`, o equivalente a olhar o SQL de uma migration antes de rodar, o hábito que separa os desenvolvedores experientes dos desenvolvedores com histórias para contar. Suponha que alguém, organizando o código, renomeie o construct do banco de `"Banco"` para `"BancoDeDados"`:

```
cdk diff
```

```
Stack RangoProducao
Resources
[-] AWS::RDS::DBInstance Banco6A9B6A33 destroy
[+] AWS::RDS::DBInstance BancoDeDados4C1D2E3F
```

O CDK monta o logical ID a partir do caminho do construct, e um id novo no Python é um recurso novo no template. Para o CloudFormation, o banco antigo saiu e um novo entrou: ele apagaria a instância e criaria outra a partir do snapshot do capítulo 14, perdendo todos os pedidos feitos desde então. Nesse caso, o `instance_identifier` fixo acabaria travando o deploy, porque o nome `rango-db` ainda estaria em uso pela instância antiga, mas não conte com essa sorte em outros recursos. A saída é desfazer a troca de nome. Quando for mesmo preciso reorganizar o código, o comando `cdk refactor` move recursos para logical IDs novos preservando o que já está implantado.

A leitura do diff é simples: `[~]` muda, `[+]` cria, `[-]` apaga. As palavras a caçar são `destroy` e `replace`, em tudo que guarda dados.

Drift na prática

Suponha que, numa sexta à noite, alguém abra a porta 22 do grupo do ALB pelo console "só para testar uma coisa". Nenhuma frase na história da computação abriu tantas portas que deviam continuar fechadas. A detecção fica assim:

```
DRIFT_ID=$(aws cloudformation detect-stack-drift --stack-name RangoProducao \
  --query StackDriftDetectionId --output text)
aws cloudformation describe-stack-drift-detection-status \
  --stack-drift-detection-id "$DRIFT_ID" --query DetectionStatus

aws cloudformation describe-stack-resource-drifts --stack-name RangoProducao \
  --stack-resource-drift-status-filters MODIFIED \
  --query "StackResourceDrifts[].
[LogicalResourceId,PropertyDifferences[0].DifferenceType,PropertyDifferences[0].Property
```

```
Path]" \
  --output text
```

```
AlbSecurityGroup86A59E99    ADD /SecurityGroupIngress/2
```

A CLI do CDK também tem um `cdk drift`, que faz a mesma pergunta a partir do projeto. Agora o detalhe que surpreende: rodar `cdk deploy` em seguida não fecha a porta. O template não mudou, então o CloudFormation conclui que não há nada a fazer. Ele compara template com template, de costas para a realidade, como quem confere a lista de compras olhando para a própria lista. A correção é revogar a regra à mão, ou levá-la para o código se ela tivesse motivo para existir, e reforçar a regra de que o console é somente leitura para o que nasceu na stack.

O fim do trabalho manual

Com a stack no ar, sobram recursos dos capítulos anteriores que nenhum código descreve. Esta é a última demolição à mão do livro. Alguns leitores vão sentir saudade; a maioria vai sentir alívio:

```
aws ec2 delete-launch-template --launch-template-name rango-api

FRONT_VELHO=rango-front-123456789012
aws s3 rm "s3://$FRONT_VELHO" --recursive
aws s3api list-object-versions --bucket "$FRONT_VELHO" --query "Versions[0]"
aws s3 rb "s3://$FRONT_VELHO"

OAC_ID=$(aws cloudfront list-origin-access-controls \
  --query "OriginAccessControlList.Items[?Name=='rango-front-oac'].Id" --output text)
ETAG=$(aws cloudfront get-origin-access-control --id "$OAC_ID" \
  --query ETag --output text)
aws cloudfront delete-origin-access-control --id "$OAC_ID" --if-match "$ETAG"

aws rds delete-db-subnet-group --db-subnet-group-name rango-db-subnets
aws rds delete-db-parameter-group --db-parameter-group-name rango-pg18
aws rds delete-db-snapshot --db-snapshot-identifier rango-db-final-cap14
```

O `list-object-versions` confere que o bucket do front não tinha versionamento; se aparecer alguma versão, o `rb` vai falhar e as versões precisam ser apagadas antes. O snapshot do capítulo 14 pode sair porque o banco atual nasceu dele e já tem os próprios backups.

A role `rango-ec2` e o instance profile dela também ficaram sem uso. Como a remoção mexe em IAM, a louça boa sai do armário de novo, e o profile é o `rango-admin`:

```
aws iam remove-role-from-instance-profile --instance-profile-name rango-ec2 \
  --role-name rango-ec2 --profile rango-admin
aws iam delete-instance-profile --instance-profile-name rango-ec2 --profile rango-admin
aws iam delete-role-policy --role-name rango-ec2 --policy-name RangoFotos --profile
rango-admin
aws iam detach-role-policy --role-name rango-ec2 \
```

```
  --policy-arn arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore --profile rango-
admin
aws iam delete-role --role-name rango-ec2 --profile rango-admin
```

Por último, a `rango-vpc`. Pelo console da VPC, a opção *Delete VPC* sobre a `rango-vpc` lista e apaga junto as subnets, as route tables, o Internet Gateway, o gateway endpoint e os Security Groups `rango-web`, `rango-alb`, `rango-db`, `rango-cache` e `rango-endpoints`. A lista merece uma conferida antes da confirmação: a VPC certa é a que não tem a tag `aws:cloudformation:stack-name`.

Desligar sem deixar sobra

O que cobra por hora agora está numa stack só. Antes de destruir, o banco precisa ter backup garantido:

```
cdk destroy
```

O CDK pede confirmação e apaga tudo na ordem inversa das dependências. Poucos comandos têm um nome tão honesto quanto `destroy`. O banco sai com um snapshot final, por causa da `RemovalPolicy.SNAPSHOT`, com nome gerado pelo CloudFormation. Para achar o mais recente:

```
aws rds describe-db-snapshots --db-instance-identifier rango-db \
  --query "reverse(sort_by(DBSnapshots, &SnapshotCreateTime))[0].
[DBSnapshotIdentifier,Status]" \
  --output text
```

Ficam a zona, o certificado, o bucket de fotos, a AMI, o snapshot e a stack `CDKToolkit`. O site sai do ar junto com a distribuição. No próximo capítulo, o caminho de volta é um `cdk deploy` com o nome desse snapshot no `snapshot_db`, e o front volta com os dois comandos de sync.

Para a homologação do Bruno, o caminho agora é outra chamada a `RangoInfraStack` no `app.py`, com outro nome de stack, outro snapshot e outro identificador de banco. O domínio exige um pouco mais: `homolog.rango.com.br` precisa de um certificado que cubra o nome, e a stack pode pedir esse certificado ao ACM com validação na própria zona. Os detalhes de configuração por ambiente ficam para o capítulo 16. A sócia já pode prometer a próxima homologação para quinta-feira sem que ninguém precise perder a quarta.

Armadilhas e casos reais

Armadilha: Trocar o snapshot de um banco que já existe. A propriedade `snapshot_identifier` só vale na criação. Se você muda o valor numa stack já implantada, o CloudFormation entende que precisa de um banco novo e substitui a instância, jogando fora tudo o que foi gravado depois da restauração. Depois do primeiro deploy, não mexa nesse parâmetro da stack que está no ar; use outro nome de stack para restaurar um backup diferente.

Armadilha: Renomear o que tem nome fixo. Com `instance_identifier="rango-db"`, qualquer mudança que exija substituição falha com `CloudFormation cannot update a stack when a custom-named resource requires replacing`. O CloudFormation cria o novo antes de apagar o velho, e dois recursos não podem ter o mesmo nome. Nomes físicos fixos facilitam a vida de quem lê o console e atrapalham a de quem atualiza a stack; use só onde algo de fora depende do nome.

Armadilha: Quem faz deploy é administrador. A role de execução que o bootstrap cria tem `AdministratorAccess`. O `rango-dev` não pode criar roles IAM, mas o `cdk deploy` dele criou uma, porque quem cria é o CloudFormation, com a role de execução. Restrinja com `cdk bootstrap --cloudformation-execution-policies` apontando para policies menores, quando o time crescer. O capítulo 23 volta a isso com várias contas.

Armadilha: Bootstrap na região esquecida. O bootstrap vale por conta e por região. Ao criar uma stack em `sa-east-1` pela primeira vez, o deploy falha pedindo bootstrap. Evite o atalho `cdk bootstrap --express`, que usa um modo do CloudFormation sem rollback automático.

Armadilha: Erro de tipo só em tempo de execução. O Python não confere tipos antes da síntese, e o CDK roda sobre uma camada chamada JSII, que traduz as chamadas para a biblioteca original em TypeScript. Passar uma string onde se espera uma lista, ou misturar um `CfnXxx` onde se espera um L2, só estoura no `cdk synth`, com um traceback que mistura Python e JavaScript, uma leitura bilíngue que ninguém pediu. Rode `mypy` ou `pyright` no CI para pegar isso antes.

Armadilha: Mexer no recurso, não no código. Alguém aumenta o `max_capacity` do ASG pelo console num pico. No próximo deploy que tocar no ASG, o valor do template volta, com a pontualidade de um boleto. A mudança de emergência precisa ir para o código logo depois, ou ela some sem aviso.

Na vida real: Uma stack em `UPDATE_ROLLBACK_FAILED` costuma ter a mesma história: alguém mexeu pelo console num recurso que a stack gerenciava. Recrie o que sumiu e rode `continue-update-rollback`. O `--resources-to-skip` deixa a stack

> acreditando num estado que não existe, e a conta chega no deploy seguinte.

Resumo

- O CloudFormation cria stacks a partir de templates. Logical ID é o nome no template, physical ID é o nome na AWS. Ele monta o grafo de dependências, faz rollback automático e detecta drift, sem corrigir.
- Uma criação que falha termina em `ROLLBACK_COMPLETE` e precisa ser apagada. Uma atualização que não consegue voltar termina em `UPDATE_ROLLBACK_FAILED`, resolvido com `continue-update-rollback`.
- O CDK gera templates do CloudFormation a partir de Python. L1 (`Cfn*`) é o recurso cru, L2 traz padrões seguros e métodos de permissão e rede, L3 monta arquiteturas inteiras.
- A CLI do CDK roda em Node.js (use 22 ou 24). Cada conta e região precisa de `cdk bootstrap`, que cria a stack `CDKToolkit` com uma role de execução administradora por padrão.
- Tokens viram referências no template, e é por isso que o user data recebe o endpoint do cache e do banco sem ninguém digitar.
- `cdk synth` gera, `cdk diff` mostra, `cdk deploy` aplica, `cdk destroy` apaga. Renomear um construct troca o logical ID e pode substituir o recurso.
- `removal_policy` decide o destino dos dados: `SNAPSHOT` no banco, `DESTROY` com `auto_delete_objects` só no que se regenera.
- O Rangô inteiro, da VPC sem NAT Gateway aos registros de DNS, está em `rango-infra/`, na stack `RangoProducao`.

O que ainda está escrito num arquivo

A infraestrutura agora cabe num `git diff`. Só que, no meio dela, a AMI `rango-api-20261015` ainda leva o `/etc/rango/api.env` com o `PGPASSWORD` do usuário `rango` em texto puro. Quem tiver permissão para lançar uma instância dessa AMI lê a senha. A senha mestre que o CDK gerou foi parar no Secrets Manager, mas a do usuário da aplicação continua num arquivo, e a versão de desenvolvimento dela ainda está no `.env` de alguém, num commit antigo e numa conversa do Slack de março, espremida entre um bom-dia e uma figurinha de gato.

No próximo capítulo, os segredos saem dos arquivos. O Parameter Store e o Secrets Manager guardam configuração e senhas, a instância busca o que precisa a cada boot com a própria role, a senha do banco passa a girar sozinha, e o KMS entra para explicar

quem guarda as chaves que cifram tudo isso.

Referências

- AWS CDK v2, guia do desenvolvedor: https://docs.aws.amazon.com/cdk/v2/guide/home.html
- Trabalhar com o CDK em Python (Python 3.9+, ambiente virtual, imports, versões fixadas): https://docs.aws.amazon.com/cdk/v2/guide/work-with-cdk-python.html
- Versões de Node.js suportadas pelo CDK: https://docs.aws.amazon.com/cdk/v2/guide/node-versions.html
- Referência dos comandos da CLI do CDK (`synth`, `diff`, `deploy`, `destroy`, `drift`, `import`, `refactor`): https://docs.aws.amazon.com/cdk/v2/guide/ref-cli-cmd.html
- `cdk bootstrap` (stack `CDKToolkit`, role de execução, `--cloudformation-execution-policies`, fim do CDK v1): https://docs.aws.amazon.com/cdk/v2/guide/ref-cli-cmd-bootstrap.html
- `HealthChecks` do Auto Scaling na API Python do CDK: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_autoscaling/HealthChecks.html
- `aws_rds` na API Python do CDK: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_rds/README.html
- `ec2.Vpc` na API Python do CDK: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_ec2/Vpc.html
- Cotas do CloudFormation: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/cloudformation-limits.html
- Preços do CloudFormation: https://aws.amazon.com/cloudformation/pricing
- `continue-update-rollback` na AWS CLI: https://docs.aws.amazon.com/cli/latest/reference/cloudformation/continue-update-rollback.html
- Detecção de drift no CloudFormation: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-stack-drift.html
- Importar recursos existentes para uma stack: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/resource-import.html

CAPÍTULO 16

Segredos e criptografia

No capítulo anterior, o Rangô inteiro virou código. A stack `RangoProducao` cria a VPC sem NAT Gateway, o banco a partir de um snapshot, o cache, o Auto Scaling Group atrás do ALB, o front no CloudFront e os registros no Route 53. O user data recebe os endpoints do banco e do cache por tokens, sem ninguém digitar nada. Só que a senha do usuário `rango` continua num arquivo dentro da AMI, e o capítulo terminou com a stack destruída e um snapshot esperando o próximo `cdk deploy`.

Onde mora a senha do banco

Vale um exercício rápido: listar todos os lugares onde a senha do usuário `rango` está escrita hoje.

O primeiro você conhece: o `/etc/rango/api.env` da AMI `rango-api-20261015`, na linha `PGPASSWORD=`. Quem pode lançar uma instância dessa AMI lê o arquivo, e cada cópia da AMI leva a senha junto. É uma senha que viaja mais do que muito diretor comercial.

O segundo está no Slack. Em março, o Bruno precisava rodar uma consulta para conferir os pedidos de um restaurante, e alguém colou a senha na conversa "só desta vez". A mensagem está lá, indexada pela busca do Slack, visível para quem entrar no canal depois. "Só desta vez" é a expressão mais duradoura da língua portuguesa: tudo o que foi feito só desta vez continua lá, anos depois.

O terceiro está no git. Na raiz do repositório, uma busca no histórico mostra:

```
git log -S 'PGPASSWORD' --oneline --all
```

```
8c41e2a ajusta conexao com o banco
3f09b7d remove .env do repositorio
a12d6c0 testes locais contra o RDS
```

O `-S` procura commits que adicionaram ou removeram aquele texto. O commit `a12d6c0` trouxe um `backend/.env` com a senha de produção para testar contra o RDS, e o `3f09b7d` apagou o arquivo. Apagar num commit novo deixa a senha intacta no histórico:

basta um `git show a12d6c0:backend/.env` para ler. E cada pessoa que clonou o repositório nesse intervalo tem uma cópia. O git tem memória de elefante e discrição nenhuma.

Nenhum desses lugares tem controle de acesso pensado para uma senha de produção, e nenhum registra quem leu. Trocar a senha exigiria mexer na AMI e caçar cada cópia. Na prática, ela nunca vai ser trocada (senha que ninguém sabe onde está escrita vira senha vitalícia), e dá acesso de leitura e escrita a todos os pedidos do Rangô.

> **Na vida real:** Em outubro de 2022, a Toyota informou que uma empresa contratada tinha publicado, em dezembro de 2017, parte do código do T-Connect num repositório público do GitHub. Nesse código estava uma chave de acesso ao servidor de dados, escrita direto no código-fonte. A chave ficou exposta até 15 de setembro de 2022, quase cinco anos, e dava acesso a dados de cerca de 296 mil clientes (e-mails e números de controle). A Toyota trocou as chaves dois dias depois. Disse não ter evidência de acesso de terceiros, sem poder descartar a hipótese. Repare na ordem dos fatos: a chave só mudou quando alguém a encontrou.

O capítulo 6 já deu a regra: segredo fica fora do código e da imagem, separado por ambiente, e gira de tempos em tempos. No Compose, isso virou um arquivo em `secrets/` fora do git. Na AWS, a pergunta é onde o segredo fica e como a instância, que nasce sozinha às três da manhã numa subnet privada, chega até ele sem ninguém acordado para ajudar.

Configuração, segredo e chave

Antes dos serviços, vale separar três coisas que costumam morar juntas no mesmo `.env`, numa convivência tão pacífica quanto a de três colegas dividindo uma república com uma geladeira só.

Configuração é o que muda entre ambientes e pode ser lido sem estrago: o endereço do cache, o nome do bucket de fotos, as origens do CORS. Se vazar, ninguém ganha acesso a nada.

Segredo é o que dá acesso a alguma coisa: a senha do banco, a chave da API de pagamento, o token de um webhook. Se vazar, alguém entra. Uma régua prática: configuração pode aparecer no slide da reunião com o cliente; segredo não aparece nem em compartilhamento de tela por engano.

Chave criptográfica é o que transforma dados legíveis em ilegíveis e vice-versa. Ela protege os segredos e os dados guardados em disco. É o segredo que guarda os outros segredos, a matrioska do mundo da segurança.

A AWS tem um serviço para cada uma, o que, para os padrões da AWS, é uma economia admirável de serviços. O AWS Systems Manager Parameter Store guarda configuração (e segredos simples). O AWS Secrets Manager guarda segredos e sabe trocá-los sozinho. O AWS Key Management Service (KMS) guarda as chaves que cifram os outros dois, e quase tudo o mais na conta.

Parameter Store

O Parameter Store é um dicionário de chave e valor dentro do Systems Manager, o mesmo serviço do Session Manager do capítulo 9. Você já usou um parâmetro dele sem perceber: o `resolve:ssm:` que buscava o ID da AMI mais recente do Amazon Linux 2023 apontava para um parâmetro público mantido pela AWS. Era o Parameter Store fazendo o seu trabalho em silêncio, como aquele colega que resolve tudo e nunca aparece nas reuniões de resultado.

Os nomes seguem uma hierarquia com barras, como caminhos de arquivo:

```
/rango/producao/cors_origins
/rango/producao/redis_url
/rango/homologacao/cors_origins
/rango/homologacao/redis_url
```

A hierarquia serve para duas coisas. Uma chamada `GetParametersByPath` com `/rango/producao/` devolve todos os parâmetros daquele ambiente de uma vez, como um `os.environ` remoto filtrado por prefixo. E uma policy IAM pode liberar só `/rango/producao/*` para a role de produção, sem que ela enxergue a homologação. Cada ambiente com a sua gaveta, e ninguém mexe na gaveta do outro.

Cada parâmetro tem um tipo. `String` é texto puro. `StringList` é uma lista separada por vírgulas. `SecureString` é texto cifrado com uma chave do KMS: o valor é guardado cifrado, e quem lê precisa de permissão no parâmetro e na chave.

Existem dois *tiers*, escolhidos por parâmetro. No standard, que é o padrão, você tem até 10.000 parâmetros por conta e região, valores de até 4 KB e nenhum custo de armazenamento. No advanced, o limite sobe para 100.000 parâmetros e 8 KB por valor, e aparecem recursos como políticas de expiração e compartilhamento com outras contas, com cobrança por parâmetro por mês e por chamada de API. Dá para promover um parâmetro de standard para advanced, mas nunca o caminho inverso; para voltar, só apagando e criando de novo. Funciona como plano de celular: subir é um clique, descer é

uma saga. O Rangô cabe com folga no standard. A taxa de requisições é outra configuração, por conta e região, chamada *higher throughput*; quando ligada, cobra cada parâmetro lido, em qualquer tier.

Secrets Manager

O Secrets Manager guarda segredos de forma parecida, com três diferenças que importam.

A primeira é a rotação automática. Você associa ao segredo uma função AWS Lambda e uma agenda, e o Secrets Manager chama essa função para gerar uma senha nova, aplicá-la no banco, testá-la e só então marcá-la como a atual. É um gerenciador de senhas com um cron, e o pior inimigo do post-it amarelo colado no monitor.

A segunda são as versões com rótulos, os *staging labels*. Cada troca cria uma versão nova do segredo, e os rótulos dizem o papel de cada uma: `AWSCURRENT` é a que a aplicação deve usar, `AWSPENDING` é a que está sendo criada durante uma rotação, e `AWSPREVIOUS` é a anterior, guardada para emergências, como o estepe no porta-malas. Quem lê o segredo sem pedir versão recebe a `AWSCURRENT`.

A terceira é o preço, e aqui a fatura da AWS mostra o seu lado mais generoso. Cada segredo custa cerca de US$ 0,40 por mês, mais US$ 0,05 a cada 10.000 chamadas de API (valores de referência da página oficial em outubro de 2026; a página não indica região, então confirme na calculadora de preços). A rotação não tem taxa própria: criar versões novas é grátis, e as chamadas que a função de rotação faz entram como chamadas comuns. Somam-se só as invocações da Lambda, que numa rotação mensal ficam em frações de centavo.

Você já tem um segredo lá. No capítulo 12, a senha mestre do `rango_admin` foi para o Secrets Manager, e no capítulo 15 o CDK passou a gerá-la com `from_generated_secret`. O que falta é o usuário da aplicação.

Rotação sem derrubar ninguém

Trocar a senha de um banco em uso tem um problema de tempo. A aplicação abriu conexões com a senha antiga e guardou essa senha para abrir as próximas. No instante em que a senha muda no banco, toda conexão nova falha até a aplicação descobrir a senha nova. É o mesmo problema de trocar a fechadura da casa com a família inteira na rua. Alguém sempre volta da padaria e fica do lado de fora, segurando o pão e uma chave que não serve mais.

O Secrets Manager oferece duas estratégias para bancos.

Na **rotação de usuário único**, a função troca a senha do próprio `rango`. É simples, e existe uma janela, de segundos, em que o banco já aceita só a senha nova e parte da aplicação ainda usa a antiga. Segundos parecem pouco até que alguém, nesses segundos, esteja pagando um pedido.

Na **rotação de usuários alternados** (*multi-user*), existem dois usuários com as mesmas permissões, e a função alterna entre eles. No PostgreSQL, a primeira rotação cria o `rango_clone` com senha nova e roda `GRANT rango TO rango_clone`, o que faz o clone herdar tudo o que o `rango` pode fazer. É o gêmeo idêntico que assume o turno enquanto o outro vai trocar de senha. A `AWSCURRENT` passa a apontar para o clone. Na rotação seguinte, quem ganha senha nova é o `rango`, e a `AWSCURRENT` volta para ele. Enquanto isso, a senha anterior continua valendo: quem leu o segredo antes da rotação segue conectando até a rotação seguinte. Para trocar a senha mestre e criar o clone, a função usa o segredo do `rango_admin`.

O Rangô vai usar a rotação de usuários alternados, a cada 30 dias. A agenda aceita expressões `rate()` em horas ou dias, com mínimo de 4 horas, ou `cron()` para escolher uma janela fina. Os horários são sempre em UTC, o fuso horário de ninguém, que garante pelo menos uma conta errada de três horas por ano em qualquer time brasileiro.

Qual dos dois

	Parameter Store (standard)	Secrets Manager
Preço	Sem custo de armazenamento	~US$ 0,40/segredo/mês + chamadas
Tamanho máximo	4 KB	64 KB
Valor cifrado	Só no tipo `SecureString`	Sempre
Rotação automática	Não	Sim, com Lambda
Versões	Histórico de versões	Versões com `AWSCURRENT` / `AWSPREVIOUS`
Criado pelo CloudFormation	`String` e `StringList`, sem `SecureString`	Sim, com valor gerado

O padrão do Rangô é o Parameter Store, que é mais simples e não cobra por parâmetro no tier standard. Em arquitetura, como em restaurante, o prato do dia resolve a maior parte dos pedidos. Configuração vai para lá como `String`. Um segredo de terceiro que você nunca consegue girar sozinho, como a chave de um parceiro que só muda por e-mail, fica num `SecureString`, criado pela CLI. O Secrets Manager é a exceção, reservada para o segredo que precisa de rotação automática gerenciada, como a senha do banco: o

Parameter Store não tem rotação nativa, e refazer a alternância de usuários com uma Lambda e um agendamento próprios custaria bem mais que os US$ 0,40 por mês do segredo, sem contar as horas de quem iria manter essa Lambda caseira para sempre.

KMS: quem guarda a chave

O Secrets Manager guarda o segredo cifrado. Cifrado com qual chave? E onde fica essa chave?

Se a chave ficasse num arquivo ao lado dos dados, a criptografia serviria só de enfeite. Quem rouba o disco rouba a chave junto. É a chave debaixo do tapete da porta, em versão digital. O KMS existe para que a chave nunca fique ao lado de nada. As chaves do KMS vivem em módulos de hardware de segurança (HSMs) operados pela AWS e nunca saem de lá em texto puro. Você nunca recebe a chave. Você manda os dados, e o KMS devolve o resultado.

Pense no gerente do banco que guarda a chave do cofre: ele confere seu documento (o IAM), abre o cofre e anota o pedido no livro de registro (o AWS CloudTrail, do capítulo 23). Um gerente que nunca entra de férias, nunca esquece a chave em casa e nunca aceita um "é rapidinho, depois eu assino".

Envelope encryption

Mandar cada byte para o KMS cifrar não funciona para um arquivo de 2 GB. A operação `Encrypt` aceita no máximo 4 KB, e cada chamada atravessa a rede. Seria como mandar cada página de um livro, uma por vez, ao cartório para autenticar. A solução tem nome: *envelope encryption*.

O fluxo para cifrar é este:

1. A aplicação pede ao KMS uma *data key* com `GenerateDataKey`.
2. O KMS devolve duas cópias da mesma data key: uma em texto puro e outra cifrada com a chave do KMS.
3. A aplicação cifra os dados localmente, com a cópia em texto puro e um algoritmo como AES-256-GCM, rápido e sem limite de tamanho.
4. A aplicação joga fora a cópia em texto puro e guarda a cópia cifrada ao lado dos dados.

Para decifrar, a aplicação manda a data key cifrada ao KMS com `Decrypt`, recebe a data key em texto puro e decifra os dados localmente.

É como fechar uma caixa com um cadeado comum e guardar a chave do cadeado num envelope lacrado que só o gerente do banco abre. A caixa pode ficar em qualquer lugar. Sem o gerente, ninguém abre o envelope.

Quase todo serviço da AWS que diz "cifrado com KMS" funciona assim por baixo. O volume EBS cifrado do capítulo 9, o RDS, o Secrets Manager e o S3 com SSE-KMS geram data keys e guardam a versão cifrada junto dos dados. O KMS só entra em cena para abrir a data key, e o trabalho pesado acontece na máquina. É a agência dos Correios mais movimentada que você nunca viu.

Chaves da AWS e chaves suas

No KMS existem dois tipos de chave que você vai encontrar.

As **chaves gerenciadas pela AWS** têm nomes como `aws/secretsmanager`, `aws/ssm`, `aws/rds` e `aws/ebs`. Cada serviço cria a sua na primeira vez que precisa. Elas não cobram armazenamento, giram sozinhas uma vez por ano, e você não consegue mudar a policy delas nem desligar a rotação. São como apartamento alugado mobiliado: tudo funciona, mas ninguém deixa você trocar o sofá. Qualquer um na conta que tenha permissão no serviço consegue usá-las por meio dele.

As **chaves gerenciadas pelo cliente** (*customer managed keys*) são criadas por você. Custam US$ 1 por mês cada uma, proporcional por hora, e as requisições custam cerca de US$ 0,03 a cada 10.000, com 20.000 requisições grátis por mês (referência de outubro de 2026). Em troca, você controla a *key policy*, que é uma policy baseada em recurso como a bucket policy do capítulo 11: ela decide quem pode cifrar, decifrar ou administrar a chave, independente das permissões no serviço que usa a chave.

Essa independência é o motivo para pagar o dólar. Com uma chave sua, ler um segredo exige duas permissões: `secretsmanager:GetSecretValue` no segredo e `kms:Decrypt` na chave. Alguém que ganhe acesso amplo ao Secrets Manager por engano continua sem conseguir abrir o que foi cifrado com ela. É uma segunda tranca, controlada num lugar diferente. Quem já perdeu as chaves de casa sabe que duas trancas irritam; quem já foi assaltado sabe por que elas existem.

A rotação automática de uma chave sua vem desligada e você liga com uma chamada. O período padrão é de 365 dias e pode ser configurado. Rotacionar uma chave do KMS cria material criptográfico novo, mas mantém o antigo guardado: dados cifrados há três anos continuam decifráveis, sem você recifrar nada. O ID e o ARN da chave não mudam. A cobrança sobe US$ 1 por mês na primeira rotação e mais US$ 1 na segunda, e depois para de subir. Fora da agenda, a rotação sob demanda permite até 25 rotações por chave. A rotação automática só existe para chaves simétricas de criptografia, as deste livro.

Apagar uma chave tem um freio. O `ScheduleKeyDeletion` coloca a chave em `PendingDeletion` por 7 a 30 dias (padrão de 30), período em que ela fica desabilitada e não cobra armazenamento. Até o prazo acabar, um `CancelKeyDeletion` desfaz tudo. Depois, todo dado cifrado com aquela chave vira ruído para sempre. Nem o suporte da AWS nem o sobrinho que entende de computador conseguem trazer de volta.

Na prática: o Rangô sem senha em arquivo

O código, os comandos e as saídas aparecem por inteiro, para que tudo possa ser acompanhado só lendo. O plano tem cinco partes. Uma chave do KMS para os segredos do Rangô. Um segredo novo para o usuário `rango`, com rotação de usuários alternados. A configuração não secreta no Parameter Store, separada por ambiente. Os VPC endpoints para que as instâncias, nas subnets isoladas, alcancem os dois serviços. E um user data que monta o `api.env` a cada instância nova, a partir do que está na AWS.

A senha que vazou não vai ser copiada para lugar nenhum. Ela vai morrer na segunda rotação, e ninguém vai precisar de atestado de óbito: o banco simplesmente para de reconhecê-la.

Pré-requisitos

O projeto `rango-infra/` do capítulo 15, com o ambiente virtual ativo, e o profile de desenvolvimento:

```
cd rango-infra
source .venv/bin/activate
export AWS_PROFILE=rango-dev
aws sso login
```

No `app.py`, o `snapshot_db` recebe o nome do snapshot final que o `describe-db-snapshots` mostrou no fim do capítulo 15, e a stack ganha um parâmetro novo, o ambiente:

```
RangoInfraStack(
    app,
    "RangoProducao",
    ambiente="producao",
    dominio="rango.com.br",
    certificado_arn=app.node.try_get_context("certificado_arn"),
    snapshot_db="rangoproducao-snapshot-banco-a1b2c3d4e5f6",
    env=cdk.Environment(
        account=os.environ["CDK_DEFAULT_ACCOUNT"],
        region="us-east-1",
    ),
)
```

Numa conta real, o nome do snapshot é o que o `describe-db-snapshots` mostrar. Em `rango_infra/rango_infra_stack.py`, entram os imports e o parâmetro:

```
from pathlib import Path

from aws_cdk import (
    # ... os imports do capítulo 15 ...
    aws_kms as kms,
    aws_ssm as ssm,
)
```

```
    def __init__(
        self,
        scope: Construct,
        construct_id: str,
        *,
        ambiente: str,
        dominio: str,
        certificado_arn: str,
        snapshot_db: str,
        **kwargs,
    ) -> None:
```

A chave e o caminho até as APIs

Logo depois da criação do cache, entram a chave e os endpoints:

```
        chave = kms.Key(
            self,
            "ChaveSegredos",
            alias=f"alias/rango-{ambiente}-segredos",
            description="Segredos do Rango",
            enable_key_rotation=True,
            pending_window=Duration.days(7),
            removal_policy=RemovalPolicy.DESTROY,
        )

        privadas = ec2.SubnetSelection(subnet_type=ec2.SubnetType.PRIVATE_ISOLATED)
        endpoint_ssm = vpc.add_interface_endpoint(
            "Ssm", service=ec2.InterfaceVpcEndpointAwsService.SSM, subnets=privadas
        )
        endpoint_sm = vpc.add_interface_endpoint(
            "SecretsManager",
            service=ec2.InterfaceVpcEndpointAwsService.SECRETS_MANAGER,
            subnets=privadas,
        )
```

A chave tem rotação anual ligada. O `removal_policy=RemovalPolicy.DESTROY` com `pending_window` de 7 dias faz o `cdk destroy` agendar a exclusão com a janela mínima, em vez de deixar a chave cobrando para sempre. Numa chave que protege dados que precisam sobreviver à stack, o certo é o padrão do CDK, que mantém a chave. Aqui ela protege só o segredo, que é apagado junto. Chave sem nada para abrir é chaveiro de lembrancinha de viagem: bonito e inútil.

Os endpoints resolvem um problema que você descobriria do jeito difícil, ou seja, com um timeout silencioso num domingo à noite e um log que termina no meio da frase. As instâncias da API estão em subnets `PRIVATE_ISOLATED`, sem rota para a internet, e as APIs do Parameter Store e do Secrets Manager são endereços públicos. Sem os endpoints, o primeiro comando do user data ficaria preso tentando conectar em `ssm.us-east-1.amazonaws.com` até estourar o tempo. Os *interface endpoints* do capítulo 10 colocam uma interface de rede desses serviços dentro das suas subnets, e o DNS privado da VPC faz o nome público apontar para ela. O código da instância não muda nada.

O preço é o do capítulo 10: cerca de US$ 0,01 por hora por endpoint por AZ, mais o tráfego. Dois serviços em duas AZs dão perto de US$ 29 por mês (us-east-1, outubro de 2026), ainda menos que dois NAT Gateways. O L2 cria para cada endpoint um Security Group que aceita a porta 443 vindo da VPC.

O segredo e a rotação

```
        segredo_app = rds.DatabaseSecret(
            self,
            "SegredoApp",
            username="rango",
            master_secret=db.secret,
            encryption_key=chave,
        ).attach(db)

        rotacao = db.add_rotation_multi_user(
            "RotacaoApp",
            secret=segredo_app,
            automatically_after=Duration.days(30),
            endpoint=endpoint_sm,
            vpc_subnets=privadas,
        )
```

O `DatabaseSecret` gera uma senha aleatória para o usuário `rango` e guarda um JSON com `username` e `password`, cifrado com a chave nova. O `master_secret` acrescenta o campo `masterarn`, que diz à função de rotação onde está a senha do `rango_admin`. O `.attach(db)` completa o JSON com `host`, `port` e `engine`, os campos que a função usa para encontrar o banco. Por padrão, o CDK exclui da senha gerada caracteres que costumam quebrar URLs e shells, como aspas, `@`, `/` e `$`. Uma senha com aspas dentro de uma URL dentro de um shell rende um dia inteiro de depuração e um desabafo no canal do time.

O `add_rotation_multi_user` monta o resto: uma função Lambda de rotação, publicada pela AWS no Serverless Application Repository, dentro das subnets privadas; um Security Group para ela, com a regra de entrada na porta 5432 do banco; as permissões de leitura e escrita no segredo e de leitura no segredo mestre; e a agenda de 30 dias. O

`rotate_immediately_on_update`, que não aparece porque o padrão já é `True`, faz a primeira rotação acontecer assim que a agenda é criada. A Lambda mal nasce e já vai trabalhar, sem nem uma semana de integração.

A senha gerada nesse momento é inútil: o `rango` do snapshot continua com a senha antiga. A primeira rotação cria o `rango_clone` e passa a `AWSCURRENT` para ele. A senha vazada só muda na segunda rotação, quando chega a vez do `rango`. Vamos forçar essa segunda logo depois do deploy.

A configuração por ambiente

```
        configuracao = {
            "aws_region": self.region,
            "cors_origins": f"https://{dominio},https://www.{dominio}",
            "database_url": (
                f"postgresql://{db.db_instance_endpoint_address}:5432/rango"
                "?sslmode=verify-full&sslrootcert=/etc/rango/rds-ca.pem"
            ),
            "fotos_bucket": f"rango-fotos-pratos-{self.account}",
            "redis_url": f"rediss://{cache.attr_endpoint_address}:6379",
        }
        parametros = [
            ssm.StringParameter(
                self,
                f"Config_{nome}",
                parameter_name=f"/rango/{ambiente}/{nome}",
                string_value=valor,
            )
            for nome, valor in configuracao.items()
        ]
```

Cada entrada do dicionário vira um parâmetro `String` no tier standard. Os endpoints do banco e do cache continuam chegando por tokens, como no capítulo 15, e agora vão para o Parameter Store em vez de entrar num `sed`. O `sed` do capítulo anterior pode, finalmente, voltar para o lugar de onde ninguém sabe direito como ele funciona.

Um detalhe no `database_url`: ele perdeu o `rango@`. Com a rotação de usuários alternados, o nome do usuário muda a cada mês, entre `rango` e `rango_clone`. Se o usuário estivesse fixo na URL, ela ganharia da variável de ambiente, porque a libpq dá preferência ao que vem na string de conexão. Sem ele, o driver usa o `PGUSER`, que o boot vai preencher a partir do segredo, como já faz com o `PGPASSWORD`.

Aqui também se cumpre o que ficou pendente no capítulo 15 sobre a homologação do Bruno. Uma segunda chamada a `RangoInfraStack` no `app.py`, com `ambiente="homologacao"`, cria `/rango/homologacao/*`, uma chave `alias/rango-homologacao-segredos` e um segredo próprio. As duas stacks compartilham o código e nada mais, como irmãos que dividem o sobrenome e nenhuma escova de dentes.

O boot que busca a configuração

O user data do capítulo 15 trocava trechos de um `api.env` que vinha pronto na AMI. Agora ele escreve o arquivo inteiro do zero. Para não espremer um script de shell dentro de strings Python, receita garantida para uma tarde inteira contando aspas, ele mora num arquivo próprio, `rango-infra/user_data/configurar.sh`:

```
# Definidas antes deste trecho pela stack: AMBIENTE, REGIAO, SEGREDO_DB_ARN
set -euo pipefail

CONFIG=$(mktemp)

aws ssm get-parameters-by-path --region "$REGIAO" \
  --path "/rango/$AMBIENTE/" \
  --query "Parameters[].[Name,Value]" --output text \
  | awk -F '\t' '{ n = $1; sub(".*/", "", n); print toupper(n) "=" $2 }' > "$CONFIG"

aws secretsmanager get-secret-value --region "$REGIAO" \
  --secret-id "$SEGREDO_DB_ARN" --query SecretString --output text \
  | python3 -c 'import json, sys; s = json.load(sys.stdin); print("PGUSER=" +
s["username"]); print("PGPASSWORD=" + s["password"])' \
  >> "$CONFIG"

install -m 640 -o root -g rango "$CONFIG" /etc/rango/api.env
rm -f "$CONFIG"

systemctl restart rango-api rango-worker@restaurante rango-worker@email
```

O `get-parameters-by-path` traz todos os parâmetros do ambiente, e o `awk` transforma `/rango/producao/redis_url` em `REDIS_URL=...`. O segundo comando lê o segredo e extrai usuário e senha com o Python do sistema, que o Amazon Linux 2023 já traz. O `mktemp` cria o arquivo temporário com permissão só para o root, e o `install` grava o `api.env` final com o mesmo dono e permissão do capítulo 9, `root:rango` e `640`. O arquivo antigo da AMI, com a senha vazada, é sobrescrito. Ele sai de cena sem discurso de despedida.

O user data roda no primeiro boot de cada instância, como vimos no capítulo 9. Num Auto Scaling Group, cada instância nova passa por ele, e as instâncias que já estão rodando não mudam. Esse detalhe vai importar na rotação.

Na stack, o bloco de user data fica assim:

```
        user_data = ec2.UserData.for_linux()
        user_data.add_commands(
            f"AMBIENTE={ambiente}",
            f"REGIAO={self.region}",
            f"SEGREDO_DB_ARN={segredo_app.secret_arn}",
            (Path(__file__).parent.parent / "user_data" / "configurar.sh").read_text(),
        )
```

Nenhum valor secreto passa pelo user data. Ele leva só o ARN do segredo, e isso importa, porque o user data fica legível pelo serviço de metadados para qualquer processo da instância e para quem tiver `ec2:DescribeInstanceAttribute` na conta.

Permissões e ordem

A role da API precisa ler os parâmetros do seu ambiente e o seu segredo:

```
        role_api.add_to_policy(
            iam.PolicyStatement(
                actions=["ssm:GetParametersByPath"],
                resources=[
                    f"arn:aws:ssm:{self.region}:
{self.account}:parameter/rango/{ambiente}",
                    f"arn:aws:ssm:{self.region}:
{self.account}:parameter/rango/{ambiente}/*",
                ],
            )
        )
        segredo_app.grant_read(role_api)
```

O `grant_read` gera duas coisas. Na role, `secretsmanager:GetSecretValue` e `DescribeSecret` só nesse segredo. E, como o segredo usa uma chave sua, `kms:Decrypt` na chave, condicionado a chamadas que venham pelo Secrets Manager. A role consegue decifrar o segredo e nada mais com aquela chave. É a chave do quarto de hotel que abre o seu quarto e não abre a cozinha do restaurante.

No `AutoScalingGroup`, duas mudanças. A primeira é um parâmetro novo:

```
            max_instance_lifetime=Duration.days(7),
```

Ele faz o ASG trocar toda instância que completar sete dias, de forma gradual. O motivo é a rotação. Uma instância lê o segredo no boot e guarda a senha no `api.env`. Com usuários alternados, essa senha continua válida até a rotação seguinte à próxima, ou seja, por pelo menos 30 dias. Uma instância que vive no máximo sete dias nunca chega a ver a sua senha expirar. Uma semana de trabalho e aposentadoria compulsória: nenhuma instância do Rangô fica velha o bastante para reclamar.

A segunda mudança é a ordem de criação, logo depois do ASG:

```
        asg.node.add_dependency(endpoint_ssm, endpoint_sm, rotacao, *parametros)
```

No capítulo 15, os tokens criavam as dependências sozinhos: o Launch Template usava o endpoint do banco, logo vinha depois do banco. Aqui, o user data busca os parâmetros pelo caminho, em tempo de execução, e não referencia nenhum deles no template. Para o

CloudFormation, nada impede que as instâncias subam antes dos parâmetros existirem ou antes dos endpoints ficarem prontos. O `add_dependency` deixa essa ordem explícita. Sem ele, a instância chegaria à festa antes de alguém montar as mesas.

Por fim, duas saídas para os comandos seguintes:

```
        CfnOutput(self, "AsgNome", value=asg.auto_scaling_group_name)
        CfnOutput(self, "SegredoAppArn", value=segredo_app.secret_arn)
```

Deploy e a segunda rotação

```
cdk diff
cdk deploy
```

O `cdk diff` mostra a tabela de mudanças de IAM e pede confirmação no deploy, como no capítulo 15. Aparece também uma stack aninhada, criada pelo Serverless Application Repository com a função de rotação. O deploy leva o tempo de restaurar o banco do snapshot, uns 15 minutos, o suficiente para preparar um café e tentar lembrar quem colou aquela senha no Slack.

Quando ele termina, as saídas vão para variáveis:

```
saida() {
  aws cloudformation describe-stacks --stack-name RangoProducao \
    --query "Stacks[0].Outputs[?OutputKey=='$1'].OutputValue" --output text
}
SEGREDO=$(saida SegredoAppArn)
ASG=$(saida AsgNome)
```

O estado da rotação aparece assim:

```
aws secretsmanager describe-secret --secret-id "$SEGREDO" \
  --query "[RotationEnabled, LastRotatedDate, NextRotationDate]" --output text
```

```
True    2026-10-16T14:22:08.512000-03:00    2026-11-15T20:59:59-03:00
```

A primeira rotação já aconteceu. Agora vem a segunda, forçada à mão, a que troca a senha do `rango`:

```
aws secretsmanager rotate-secret --secret-id "$SEGREDO"
```

Meio minuto depois, dá para ver quem é quem, sem nunca imprimir a senha no terminal:

```
for estagio in AWSCURRENT AWSPREVIOUS; do
  printf '%s ' "$estagio"
  aws secretsmanager get-secret-value --secret-id "$SEGREDO" \
    --version-stage "$estagio" --query SecretString --output text \
```

```
    | python3 -c 'import json, sys; print(json.load(sys.stdin)["username"])'
done
```

```
AWSCURRENT rango
AWSPREVIOUS rango_clone
```

O `rango` voltou a ser o usuário atual, agora com uma senha que ninguém viu: nem você, nem o Bruno, nem o Slack. A senha do Slack, do commit `a12d6c0` e da AMI parou de funcionar neste momento. O `rango_clone` continua válido, com a senha da primeira rotação.

Falta uma coisa. As duas instâncias que subiram durante o deploy leram o segredo uma vez, no boot, e não dá para saber se foi antes ou depois da primeira rotação terminar, porque o CloudFormation dispara a rotação e segue em frente. Uma instância que leu a versão gerada pelo CDK, antes da rotação, está com uma senha que o banco nunca conheceu. A forma limpa de garantir que todo mundo tem credencial válida é trocar todas as instâncias:

```
aws autoscaling start-instance-refresh --auto-scaling-group-name "$ASG" \
  --preferences '{"MinHealthyPercentage": 50}'
```

O *instance refresh* substitui as instâncias aos poucos, mantendo pelo menos metade saudável. O mesmo comando vale depois de qualquer mudança de Launch Template, porque o ASG não troca sozinho as instâncias que estão rodando.

Com o front de volta pelos dois comandos de sync do capítulo 14, o teste da API é feito de fora:

```
curl -s https://api.rango.com.br/pedidos/6f1c2a9e-3b4d-4e8f-9a1b-2c3d4e5f6a7b
```

O pedido do snapshot volta como antes. O cliente não percebe nada, que é o maior elogio que uma mudança de segurança pode receber. O código do Rangô ficou igual; mudou de onde vêm as variáveis que ele lê. O FastAPI nem ficou sabendo da mudança, e ninguém pretende contar.

Dentro de cada instância, o `api.env` ainda tem a senha em texto, só que ela nasce no boot, vem de um lugar com controle de acesso e deixa de valer sozinha.

A faxina dos outros lugares

A mensagem do Slack e o `.env` do notebook também precisam ser apagados; o `.env` local aponta só para o Postgres do Compose, com a senha de `secrets/db_password.txt` do capítulo 6. Reescrever o histórico do git com o `git filter-repo` limpa o repositório

daqui para frente, e as cópias que já existem continuam com o commit. Reescrever o histórico depois do vazamento é trocar o tapete da porta depois que alguém já tirou cópia da chave. Quem resolveu o vazamento foi a troca da senha.

O *push protection* do GitHub bloqueia pushes com credenciais de formato conhecido, como chaves de acesso da AWS. Senhas aleatórias de banco ele não reconhece (para um robô, `kX9#pQ2v` é indistinguível de um nome de variável mal escolhido), por isso a senha precisa nascer longe do repositório, como agora.

Envelope encryption com as próprias mãos

Para ver o envelope funcionando, um script curto em `scripts/envelope.py`, usando a chave nova. Ele precisa do `boto3` e do pacote `cryptography` instalados no ambiente:

```
import os

import boto3
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

kms = boto3.client("kms", region_name="us-east-1")


def cifrar(texto: bytes) -> dict:
    chave = kms.generate_data_key(
        KeyId="alias/rango-producao-segredos", KeySpec="AES_256"
    )
    nonce = os.urandom(12)
    dados = AESGCM(chave["Plaintext"]).encrypt(nonce, texto, None)
    return {"chave_cifrada": chave["CiphertextBlob"], "nonce": nonce, "dados": dados}


def decifrar(pacote: dict) -> bytes:
    chave = kms.decrypt(CiphertextBlob=pacote["chave_cifrada"])["Plaintext"]
    return AESGCM(chave).decrypt(pacote["nonce"], pacote["dados"], None)


pacote = cifrar(b"Interfone quebrado, ligar ao chegar")
print(len(pacote["dados"]), "bytes cifrados")
print(decifrar(pacote).decode())
```

```
python scripts/envelope.py
```

```
51 bytes cifrados
Interfone quebrado, ligar ao chegar
```

O pacote guarda só a versão cifrada da data key (`CiphertextBlob`); o `Plaintext` é usado e descartado. Os 51 bytes são os 35 do texto mais os 16 da etiqueta de autenticação do GCM, que detecta qualquer alteração nos dados. No `decifrar`, o `kms.decrypt` nem recebe o ID da chave: o `CiphertextBlob` já carrega a informação de qual chave o KMS deve usar. O envelope traz o endereço do remetente, por assim dizer. E o entregador, ao contrário do interfone do exemplo, funciona.

Esse script é didático, e é o tipo de código em que um detalhe errado (um nonce repetido) quebra a segurança sem nenhum erro aparente. Para cifrar dados de verdade na aplicação, o caminho é o AWS Encryption SDK, que implementa o envelope, guarda os metadados num formato padrão e reaproveita data keys com segurança. E, antes de ir para produção, qualquer código de criptografia precisa passar pela revisão de alguém com experiência em segurança. Criptografia caseira é como cirurgia caseira: dá para fazer, e é exatamente por isso que não se deve.

O desligamento

```
cdk destroy
```

O banco sai com snapshot final, como no capítulo 15; o nome aparece no mesmo `describe-db-snapshots`. A chave do KMS entra em `PendingDeletion` por sete dias, sem cobrar, uma espécie de aviso prévio que a chave cumpre sem trabalhar. Os endpoints, os parâmetros, os segredos e a função de rotação saem com a stack. Ficam a zona, o certificado, o bucket de fotos, a AMI, o snapshot e a stack `CDKToolkit`.

Para quem muda de ideia sobre a chave dentro da janela, o caminho é `aws kms cancel-key-deletion --key-id <id>` e depois `aws kms enable-key`, porque a chave volta desabilitada.

Armadilhas e casos reais

> **Armadilha:** O template guarda o que você escreve. Uma senha escrita no código do CDK, com `SecretValue.unsafe_plain_text` ou num `CfnOutput`, vai parar no template sintetizado, no bucket de assets do bootstrap e no console do CloudFormation, legível para quem tiver `cloudformation:GetTemplate`. O nome do método avisa, com uma honestidade que deveria vir acompanhada de sirene. Deixe o valor ser gerado pelo serviço, como no `DatabaseSecret`.
>
> **Armadilha:** O CloudFormation cria parâmetros `String` e `StringList`, mas nunca `SecureString`. Quem procura `SecureString` no `ssm.StringParameter` não acha. Para segredo em código de infraestrutura, use o Secrets Manager. Um `SecureString` avulso sai da CLI com `aws ssm put-parameter --type SecureString`, digitado por alguém, fora do repositório.
>
> **Armadilha:** Tabela com dono errado. O `rango_clone` herda as permissões do `rango` por `GRANT`, mas o que ele cria fica sendo dele. Se uma migration rodar enquanto a `AWSCURRENT` for o clone, a tabela nova pertence ao `rango_clone`, e o

`rango` não consegue alterá-la no mês seguinte. Gêmeos idênticos, mas cada um com a sua escritura. Rode as migrations com o `rango_admin` e dê as permissões ao `rango`, ou comece a migration com `SET ROLE rango`.

Armadilha: Duas rotações seguidas derrubam quem ficou para trás. A rotação de usuários alternados protege quem leu a versão atual ou a anterior. Quem leu a versão de antes disso perde o acesso. Ao forçar rotações manuais em sequência, como fizemos, faça um instance refresh em seguida.

Armadilha: Apagar uma chave é apagar os dados. Tudo o que foi cifrado com uma chave do KMS depende dela para sempre. Antes de agendar a exclusão de uma chave antiga, desabilite-a com `aws kms disable-key` por algumas semanas e veja se algo quebra. Desabilitar se desfaz em um segundo; a exclusão, depois da janela, nunca. É a diferença entre deixar alguém de castigo e apagar o contato.

Armadilha: `PowerUserAccess` lê todos os segredos. O permission set dos devs, do capítulo 8, inclui `secretsmanager:*` e `kms:*`, e a key policy padrão do CDK confia na conta, o que deixa o IAM decidir. Na prática, qualquer dev lê a senha de produção, inclusive o que entrou ontem e ainda nem sabe onde fica o banheiro. A saída é uma conta separada para produção, assunto do capítulo 23.

Na vida real: O caso da Toyota mostra o vazamento que ninguém nota por anos. Uma credencial que gira sozinha a cada 30 dias limita o estrago desse tipo de vazamento sem depender de alguém descobri-lo.

Resumo

- Configuração vai para o Parameter Store (caminhos como `/rango/producao/...`, tier standard sem custo), segredo que gira vai para o Secrets Manager, e as chaves ficam no KMS.
- O Secrets Manager versiona com `AWSCURRENT`, `AWSPENDING` e `AWSPREVIOUS` e gira senhas com uma Lambda. A rotação de usuários alternados mantém a senha anterior válida até a rotação seguinte.
- O KMS nunca entrega a chave. Envelope encryption cifra os dados localmente com data keys, cifradas por uma chave do KMS.
- Uma chave sua custa US$ 1 por mês e traz a key policy, uma segunda tranca independente do serviço.
- O Rangô monta o `api.env` no boot, por VPC endpoints, e cada instância vive no máximo sete dias. A senha vazada morreu na segunda rotação.

Uma imagem que envelhece

O segredo saiu dos arquivos, mas a AMI `rango-api-20261015` merece um segundo olhar. Ela ainda carrega o `api.env` antigo, com uma senha morta, e o código do Rangô do dia em que foi criada. Para mudar uma linha do `banco.py`, o caminho é subir uma instância de montagem numa subnet pública, cadastrar de novo a deploy key, rodar `git pull` e `pip install`, criar uma AMI nova, trocar o nome no `MachineImage.lookup` e disparar um instance refresh. É uma receita com mais etapas que bolo de casamento, para mudar uma linha. E o Amazon Linux 2023 recebe atualizações de segurança com frequência, que só chegam às instâncias quando alguém refaz a imagem.

No próximo capítulo, o Rangô troca a AMI por uma imagem de container. O Amazon ECR guarda as imagens, o Amazon ECS roda os containers e o AWS Fargate tira de cena o servidor que você precisava manter. E o segredo do banco vira um campo na definição da tarefa, injetado pelo próprio ECS quando o container sobe.

Referências

- Parameter Store, tiers standard e advanced: https://docs.aws.amazon.com/systems-manager/latest/userguide/parameter-store-advanced-parameters.html
- Preços do Systems Manager (Parameter Store, higher throughput, interações de API): https://aws.amazon.com/systems-manager/pricing/
- Preços do Secrets Manager: https://aws.amazon.com/secrets-manager/pricing/
- Regras de agenda da rotação (`rate()`, `cron()`, `Duration`): https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-secretsmanager-rotationschedule-rotationrules.html
- `RotationScheduleOptions` na API Python do CDK (`rotate_immediately_on_update`): https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_secretsmanager/RotationScheduleOptions.html
- `aws_rds` na API Python do CDK (`DatabaseSecret`, `add_rotation_multi_user`): https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_rds/README.html
- Rotação de usuários alternados para o RDS (AWS Database Blog): https://aws.amazon.com/blogs/database/multi-user-secrets-rotation-for-amazon-rds/
- Código das funções de rotação publicadas pela AWS: https://github.com/aws-samples/aws-secrets-manager-rotation-lambdas
- Conceitos do KMS: https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html

- Rotação de chaves do KMS: https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html
- `enable-key-rotation` na AWS CLI (período de rotação): https://docs.aws.amazon.com/cli/latest/reference/kms/enable-key-rotation.html
- Rotação sob demanda no KMS: https://docs.aws.amazon.com/kms/latest/developerguide/rotating-keys-on-demand.html
- `schedule_key_deletion` no boto3 (janela de 7 a 30 dias): https://docs.aws.amazon.com/boto3/latest/reference/services/kms/client/schedule_key_deletion.html
- Preços do KMS: https://aws.amazon.com/kms/pricing/
- AWS Encryption SDK: https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/introduction.html
- Caso Toyota T-Connect (GitGuardian): https://blog.gitguardian.com/toyota-accidently-exposed-a-secret-key-publicly-on-github-for-five-years/

CAPÍTULO 17

Containers com ECR, ECS e Fargate

No capítulo 16, a senha do banco saiu dos arquivos. A configuração foi para o Parameter Store, em `/rango/producao/...`, o segredo do usuário `rango` passou a girar sozinho no Secrets Manager com usuários alternados, e uma chave do KMS protege esse segredo. Cada instância monta o `api.env` no boot, pelos VPC endpoints, e vive no máximo sete dias. No fim, o `cdk destroy` desligou tudo, e ficaram a zona, o certificado, o bucket de fotos, um snapshot do banco e a AMI `rango-api-20261015`, que ainda guarda o código antigo e uma senha que não funciona mais. É a versão em nuvem do molho de chaves do zelador, metade delas de fechaduras que já nem existem.

Uma linha, uma hora e quarenta

Terça-feira, 10h. O Bruno, da Sabor do Sertão, avisa que a listagem de pedidos mostra a hora em UTC, três horas adiantada. A correção é uma linha no `banco.py`. Você testa no Compose, com a imagem `rango-api:0.7.0` que roda no seu notebook desde o capítulo 12, e funciona na primeira tentativa. Isso, por si só, já deveria ter levantado suspeitas.

Agora a parte de levar essa linha para produção. Você sobe uma instância de montagem numa subnet pública, a partir da AMI atual. Cadastra de novo a deploy key no GitHub, porque ela foi removida por segurança no capítulo 13. Roda `git pull` e `pip install`. O `pip` reclama de uma dependência que mudou de versão desde outubro. Você resolve, cria a AMI `rango-api-20261103`, troca o nome no `MachineImage.lookup`, roda `cdk deploy`, dispara o instance refresh e espera as instâncias serem trocadas uma a uma. Apaga a instância de montagem e remove a deploy key de novo. Uma hora e quarenta minutos para uma linha. No mesmo tempo daria para assistir a um filme inteiro, com os trailers, e a sensação de produtividade seria parecida.

Repare numa coisa estranha nesse processo. A imagem Docker que você constrói e testa desde o capítulo 3 nunca chegou a produção. Em produção roda outra coisa: um Amazon Linux com Python instalado por `dnf`, units do systemd e um `venv` em `/opt/rango`. São dois ambientes diferentes, mantidos por dois caminhos diferentes, e o "na minha máquina funciona" do capítulo 3 voltou pela porta dos fundos.

E tem o sistema operacional. Suponha que, na mesma semana, saia um aviso de segurança do Amazon Linux 2023 para o OpenSSL (situação hipotética, mas avisos assim são frequentes). As instâncias só recebem a correção quando alguém refaz a AMI, e ninguém tem isso na agenda. A agenda, aliás, já está tomada por três reuniões de alinhamento sobre a próxima reunião de alinhamento.

O que você quer é o que já tem no notebook: uma imagem com o código e as dependências, construída uma vez, com uma tag, e que roda igual em qualquer lugar. Falta onde guardar essa imagem na AWS e quem rode os containers por você.

Três serviços para uma imagem

O capítulo 3 apresentou as peças: imagem, registry, container e orquestrador. A AWS separou cada uma num serviço.

O **Amazon ECR** (*Elastic Container Registry*) é o registry. Funciona como o Docker Hub, com o mesmo `docker push` e `docker pull`, mas privado, dentro da sua conta, com acesso controlado pelo IAM. Cada imagem mora num repositório, e cada versão é identificada por uma tag e por um *digest*, o hash SHA-256 do conteúdo. A tag é o apelido da imagem; o digest é o CPF dela, e esse ninguém troca.

O **Amazon ECS** (*Elastic Container Service*) é o orquestrador. Você declara quantas cópias de qual imagem quer rodar, com quanta CPU e memória, e ele reconcilia: sobe o que falta, substitui o que morreu, registra cada container no load balancer. É a mesma ideia de estado desejado do Kubernetes, com menos peças e menos palestras de conferência sobre ele. O ECS em si não cobra nada. Você paga pela capacidade onde os containers rodam. É o modelo de negócio do restaurante que não cobra o couvert e compensa no preço do refrigerante.

E essa capacidade pode vir de dois lugares. No modo EC2, os containers rodam em instâncias suas, que você escolhe, atualiza e escala. No **AWS Fargate**, não há instância à vista. Você diz "0,5 vCPU e 1 GB" e a AWS arranja a máquina, roda o container e cobra pelo tempo de uso. O sistema operacional por baixo, o patch do kernel, o agente do ECS: tudo isso passa a ser problema da AWS, na divisão do modelo de responsabilidade compartilhada do capítulo 7. Poucas frases em computação são tão reconfortantes quanto "isso agora é problema de outra pessoa". O aviso de segurança do OpenSSL no Amazon Linux deixa de ser seu. O da imagem `python:3.12-slim` continua sendo, porque a imagem é sua.

Para o Rangô, a escolha é Fargate. O motivo de sair da AMI é parar de cuidar de servidor, e o modo EC2 traria o servidor de volta.

O vocabulário do ECS

Quatro termos aparecem o tempo todo.

O **cluster** é um agrupamento lógico de serviços e tarefas. Com Fargate, ele não tem máquinas; é um nome e um lugar para pendurar configurações. Um cabide com crachá.

A **task definition** é a receita: imagem, CPU, memória, portas, variáveis de ambiente, segredos, comando, destino dos logs e as roles do IAM. Se você conhece o `compose.yaml`, a task definition é a entrada de um serviço dele, só que em JSON e versionada. Quem sentiu falta do YAML pode levantar a mão; até agora, ninguém levantou. Cada alteração cria uma revisão nova, como `rango-api:7`, e as antigas continuam lá, o que torna o rollback uma questão de apontar para a revisão anterior. No Kubernetes, o equivalente é a especificação do pod.

A **task** é uma execução da task definition: um ou mais containers rodando juntos, com o mesmo endereço de rede. Ela é para a task definition o que o container é para a imagem. Quem sofreu com questões de analogia no colégio acaba de recuperar um ponto.

O **service** mantém um número de tasks rodando o tempo todo, troca as que caem, faz o deploy gradual de uma revisão nova e registra cada task num target group do ALB. Ele junta o papel do Deployment e do Service do Kubernetes, e faz pelas tasks o que o Auto Scaling Group fazia pelas instâncias no capítulo 13.

Cada task com seu próprio IP

No Fargate, toda task usa o modo de rede `awsvpc`: ela ganha uma interface de rede própria numa subnet da sua VPC, com um IP privado e um Security Group. Para a rede, cada task parece uma instância pequena. O target group passa a ser do tipo `ip`, apontando para esses endereços, em vez do tipo `instance` do capítulo 13.

Isso tem uma consequência que vai aparecer na parte prática. Tudo o que a task precisa buscar antes de o container nascer, como a imagem no ECR e os segredos, sai por essa interface, na subnet onde a task está. Se a subnet não tiver caminho até esses serviços, a task não sobe, como passageiro que chega ao aeroporto e descobre que o passaporte ficou em casa.

As duas roles de uma task

Uma task tem duas roles, e confundir as duas é um dos erros mais comuns do ECS. Os nomes ajudam pouco: "task role" e "task execution role" parecem ter sido batizadas por alguém com pressa para sair para o almoço.

A **task execution role** é usada pelo próprio ECS, antes e em volta do seu código. Com ela, o agente baixa a imagem do ECR, busca os segredos no Secrets Manager e no Parameter Store e escreve os logs no CloudWatch. O seu código nunca a vê.

A **task role** é a do seu código. É ela que o `boto3` encontra dentro do container, pela mesma cadeia de credenciais do capítulo 8, quando a API gera uma presigned URL para o bucket de fotos. Ela substitui o instance profile da EC2.

Se a permissão de S3 for parar na execution role, a API recebe `AccessDenied` ao gerar a URL. Se a permissão do segredo for parar na task role, a task nem chega a subir. Nos dois casos, o primeiro impulso humano é dar `AdministratorAccess` para as duas roles e ir para casa. Resista.

Segredos como campo da task definition

O capítulo 16 terminou com uma promessa: o segredo do banco vira um campo na definição da tarefa. Na task definition, além de `environment`, com valores em texto, existe a lista `secrets`. Cada item diz o nome da variável e de onde o ECS deve buscar o valor:

```
"secrets": [
  {
    "name": "PGPASSWORD",
    "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:SegredoApp-
AbCdEf:password::"
  },
  {
    "name": "REDIS_URL",
    "valueFrom": "arn:aws:ssm:us-east-1:123456789012:parameter/rango/producao/redis_url"
  }
]
```

O `valueFrom` do Secrets Manager segue o formato `arn:...:secret:nome:json-key:version-stage:version-id`. O `password` depois do nome escolhe uma chave dentro do JSON do segredo. Os dois campos seguintes ficaram vazios, o que significa a versão `AWSCURRENT`. Os dois-pontos finais são obrigatórios mesmo vazios, e esquecê-los é uma fonte clássica de erro quando se escreve o JSON à mão. Dois-pontos vazios no fim de uma string são o tipo de detalhe que separa um deploy tranquilo de uma tarde inteira olhando para o mesmo ARN. O `valueFrom` também aceita o ARN de um parâmetro do Parameter Store, como no segundo item.

Na hora em que a task sobe, o agente usa a execution role, busca cada valor e entrega ao container como variável de ambiente. O segredo não passa pelo user data, não fica em disco e não aparece na task definition. Ele chega, entrega o recado e não deixa endereço. Para pedir uma chave específica do JSON, como `password`, o Fargate precisa da *platform version* 1.4.0 ou mais nova no Linux, que é a versão padrão hoje.

A busca acontece uma única vez, no início da task. Uma rotação depois disso não chega aos containers que já estão rodando. Guarde essa frase, porque ela vai voltar duas vezes neste capítulo, como bordão de programa de auditório.

Na prática: o Rangô em containers

O plano: criar um repositório no ECR numa stack separada, construir e enviar a imagem, trocar o Auto Scaling Group e a AMI por um cluster ECS com Fargate, e colocar os workers em containers também. Todo plano que cabe numa frase cabe também em umas trezentas linhas de Python, como se verá.

Pré-requisitos

O projeto `rango-infra/` do capítulo 16, o Docker com o `buildx` (que vem no Docker Desktop e nas instalações recentes do Docker Engine) e o profile de desenvolvimento:

```
cd rango-infra
source .venv/bin/activate
export AWS_PROFILE=rango-dev
aws sso login
```

No `app.py`, o `snapshot_db` recebe o nome do snapshot final do capítulo 16, que você encontra com o mesmo `describe-db-snapshots`. Os nomes que o CloudFormation gera, com aquele sufixo hexadecimal no fim, foram claramente pensados para que nenhum ser humano consiga decorá-los.

A imagem de produção

O Dockerfile do capítulo 3 serve quase sem mudanças, o que em software beira o milagre: código de quatorze capítulos atrás que ainda presta. Falta o certificado da CA do RDS. A `database_url` do Parameter Store aponta para `/etc/rango/rds-ca.pem`, e antes esse arquivo era colocado na instância pelo processo da AMI. Agora ele precisa estar na imagem. O bundle vai para a pasta do backend:

```
curl -sS -o backend/rds-ca.pem \
  https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

E o Dockerfile ganha uma linha:

```
# backend/Dockerfile
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY rds-ca.pem /etc/rango/rds-ca.pem
COPY app ./app

RUN useradd --create-home rango
USER rango

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

O certificado de uma CA é público; colocá-lo na imagem não expõe nada. Ele é tão secreto quanto o cardápio pendurado na porta do restaurante. A mesma imagem vai servir à API e aos dois workers, que só trocam o `CMD`.

O repositório, numa stack à parte

O repositório precisa existir e já ter uma imagem antes de o serviço do ECS ser criado. Se ele nascesse na mesma stack, o serviço tentaria subir tasks com uma imagem que ninguém enviou ainda. Além disso, as imagens precisam sobreviver a um `cdk destroy` da produção. Por isso o repositório vai para uma stack própria, `rango_infra/registro_stack.py`:

```
from aws_cdk import Duration, RemovalPolicy, Stack, aws_ecr as ecr
from constructs import Construct


class RegistroStack(Stack):
    def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        ecr.Repository(
            self,
            "Api",
            repository_name="rango-api",
            image_scan_on_push=True,
            image_tag_mutability=ecr.TagMutability.IMMUTABLE,
            lifecycle_rules=[
                ecr.LifecycleRule(
                    description="Apaga imagens sem tag",
                    tag_status=ecr.TagStatus.UNTAGGED,
                    max_image_age=Duration.days(1),
                ),
                ecr.LifecycleRule(
                    description="Guarda as 30 imagens mais recentes",
                    max_image_count=30,
                ),
            ],
            removal_policy=RemovalPolicy.RETAIN,
        )
```

Três decisões estão nessas linhas.

As tags são imutáveis. Uma vez que `rango-api:3f9c2b1` foi enviada, ninguém consegue enviar outra imagem com a mesma tag. Nem você, nem o colega que jura que só ia "corrigir uma coisinha". No capítulo 3, a armadilha era a `latest`, que aponta para coisas diferentes ao longo do tempo. Aqui, a tag é o hash do commit, como o pipeline do capítulo 6 já fazia, e o ECR garante que ela sempre signifique a mesma imagem.

O `image_scan_on_push` faz a varredura básica de vulnerabilidades a cada push, comparando os pacotes da imagem com bancos de CVEs conhecidos. É assim que você descobre que a `python:3.12-slim` de três meses atrás tem pacotes do Debian com correção disponível. Imagem de container envelhece como leite, e não como vinho.

As regras de ciclo de vida são o lifecycle do S3, do capítulo 11, aplicado a imagens: as sem tag somem depois de um dia, e o repositório guarda as 30 mais recentes. O armazenamento no ECR custa cerca de US$ 0,10 por GB por mês (us-east-1, outubro de 2026; confirme na página oficial de preços), e um pipeline que envia uma imagem a cada commit enche o repositório rápido. Sem as regras, a fatura costuma ser a primeira a lembrar que esse repositório existe.

No `app.py`, as duas stacks dividem o mesmo ambiente, e a produção ganha a versão da imagem como parâmetro:

```
import os

import aws_cdk as cdk

from rango_infra.rango_infra_stack import RangoInfraStack
from rango_infra.registro_stack import RegistroStack

app = cdk.App()
env = cdk.Environment(account=os.environ["CDK_DEFAULT_ACCOUNT"], region="us-east-1")

RegistroStack(app, "RangoRegistro", env=env)

RangoInfraStack(
    app,
    "RangoProducao",
    ambiente="producao",
    dominio="rango.com.br",
    certificado_arn=app.node.try_get_context("certificado_arn"),
    snapshot_db="rangoproducao-snapshot-banco-f6e5d4c3b2a1",
    versao_api=app.node.try_get_context("versao_api"),
    env=env,
)

app.synth()
```

Com o nome do snapshot trocado pelo da sua conta, o primeiro deploy cria só o registro, por enquanto:

```
cdk deploy RangoRegistro
```

Build e push

O ECR autentica o Docker com um token temporário, gerado a partir da sua sessão SSO. Nenhuma senha fica guardada:

```
CONTA=$(aws sts get-caller-identity --query Account --output text)
REGISTRO="$CONTA.dkr.ecr.us-east-1.amazonaws.com"

aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin "$REGISTRO"
```

O token vale 12 horas, o suficiente para um dia de trabalho com hora extra, que é o expediente não oficial da área. Agora o build, com uma escolha de arquitetura:

```
VERSAO=$(git rev-parse --short HEAD)

docker buildx build --platform linux/arm64 \
  -t "$REGISTRO/rango-api:$VERSAO" --push backend/
```

O `linux/arm64` é para rodar no Graviton, que no Fargate custa cerca de 20% menos que x86 (us-east-1, outubro de 2026). Num notebook x86, o `buildx` usa emulação e fica mais lento, mas funciona. Lento o bastante para um café, não o bastante para um almoço. A task definition vai declarar ARM explicitamente, para não repetir o `exec format error` do capítulo 3.

Depois do push, a varredura já tem resultado:

```
aws ecr describe-image-scan-findings --repository-name rango-api \
  --image-id imageTag="$VERSAO" \
  --query "imageScanFindings.findingSeverityCounts"
```

```
{
    "MEDIUM": 6,
    "LOW": 19
}
```

Seis achados médios e dezenove baixos: é o boletim de quem passou de ano, mas com bilhete para os pais assinarem. Os números variam com a data do build. Se aparecer `CRITICAL` ou `HIGH`, comece reconstruindo com a base atualizada (`--pull`).

A produção sem servidor à vista

Em `rango_infra/rango_infra_stack.py`, muita coisa sai: o bloco do user data e o arquivo `user_data/configurar.sh`, o Launch Template, o Auto Scaling Group com o `max_instance_lifetime` e o `add_dependency`, a `role_api`, o ALB com os listeners e o `add_targets`, o registro `ApiA` e a saída `AsgNome`. Apagar código é a parte mais gostosa de qualquer refatoração, e esta é generosa. Nos imports, saem `aws_autoscaling` e o `Path`, e entram estes:

```
from aws_cdk import (
    # ... os imports do capítulo 16, sem aws_autoscaling ...
    aws_ecr as ecr,
    aws_ecs as ecs,
    aws_ecs_patterns as ecs_patterns,
    aws_logs as logs,
)
```

O construtor ganha o parâmetro `versao_api: str`, ao lado de `snapshot_db`.

Duas mudanças pequenas no que já existia. A primeira: os parâmetros do Parameter Store passam a ser um dicionário, para que cada um vire uma variável do container. Os IDs dos constructs continuam os mesmos:

```
        parametros = {
            nome: ssm.StringParameter(
                self,
                f"Config_{nome}",
                parameter_name=f"/rango/{ambiente}/{nome}",
                string_value=valor,
            )
            for nome, valor in configuracao.items()
        }
```

A segunda: a busca da zona (`route53.HostedZone.from_lookup`) e o `certificado` sobem para antes do código novo, porque o serviço da API vai usar os dois.

Os endpoints que faltavam

As tasks vão rodar nas subnets `PRIVATE_ISOLATED`, sem NAT, como as instâncias rodavam. Elas já alcançam o Parameter Store e o Secrets Manager pelos endpoints do capítulo 16, e o S3 pelo gateway endpoint do capítulo 15. Faltam três caminhos:

```
        endpoints = [endpoint_ssm, endpoint_sm]
        for nome, servico in (
            ("EcrApi", ec2.InterfaceVpcEndpointAwsService.ECR),
            ("EcrDocker", ec2.InterfaceVpcEndpointAwsService.ECR_DOCKER),
            ("Logs", ec2.InterfaceVpcEndpointAwsService.CLOUDWATCH_LOGS),
        ):
            endpoints.append(
                vpc.add_interface_endpoint(nome, service=servico, subnets=privadas)
            )
```

O `ecr.api` atende as chamadas de autenticação e de metadados das imagens. O `ecr.dkr` atende o protocolo do Docker, o manifesto da imagem. E as camadas propriamente ditas, os bytes pesados, o ECR serve a partir do S3, e por isso o gateway endpoint do S3 também entra na conta. Pedir uma imagem ao ECR lembra pedir uma certidão no cartório: um guichê confere quem você é, outro entrega o protocolo, e o papel mesmo sai de um terceiro lugar. O endpoint de logs leva a saída do container ao CloudWatch Logs.

Sem esses três, o deploy fica preso com tasks que nascem e morrem, e o motivo aparece nos eventos do serviço. Esse erro está nas armadilhas, com a mensagem.

Cluster, logs, role e imagem

```
        cluster = ecs.Cluster(self, "Cluster", vpc=vpc, cluster_name=f"rango-
{ambiente}")

        logs_app = logs.LogGroup(
            self,
            "LogsApp",
            retention=logs.RetentionDays.ONE_MONTH,
            removal_policy=RemovalPolicy.DESTROY,
        )

        role_app = iam.Role(
            self, "RoleApp", assumed_by=iam.ServicePrincipal("ecs-tasks.amazonaws.com")
        )
        role_app.add_to_policy(
            iam.PolicyStatement(
                actions=["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
                resources=[f"arn:aws:s3:::rango-fotos-pratos-{self.account}/fotos/*"],
            )
        )

        repositorio = ecr.Repository.from_repository_name(self, "RepoApi", "rango-api")
        imagem = ecs.ContainerImage.from_ecr_repository(repositorio, tag=versao_api)

        plataforma = ecs.RuntimePlatform(
            cpu_architecture=ecs.CpuArchitecture.ARM64,
            operating_system_family=ecs.OperatingSystemFamily.LINUX,
        )
```

O grupo de logs resolve uma pendência do capítulo 13: os logs viviam no journal de cada instância e morriam com ela, como diário de bordo guardado no navio que afundou. Agora o JSON do `logs.py` vai para o CloudWatch Logs por 30 dias, e o capítulo 22 vai consultá-lo a sério.

A `role_app` é a task role. Ela tem só as três ações no bucket de fotos, as mesmas da `role_api`, e é assumida pelo principal `ecs-tasks.amazonaws.com` em vez do `ec2.amazonaws.com`. Saíram a policy do Session Manager, que não tem uso sem instância (não há mais onde entrar, e isso é ótima notícia), e as permissões de leitura de parâmetros e segredos, que agora são da execution role.

O `from_repository_name` aponta para o repositório da outra stack pelo nome, do mesmo jeito que a zona entra por lookup.

As variáveis do container

```
        variaveis = {
            nome.upper(): ecs.Secret.from_ssm_parameter(parametro)
            for nome, parametro in parametros.items()
```

```
        }
        variaveis["PGUSER"] = ecs.Secret.from_secrets_manager(segredo_app,
field="username")
        variaveis["PGPASSWORD"] = ecs.Secret.from_secrets_manager(
            segredo_app, field="password"
        )
```

Compare com o `configurar.sh` do capítulo 16. Lá, um `awk` transformava `/rango/producao/redis_url` em `REDIS_URL=...` e um trecho de Python extraía usuário e senha do JSON. Aqui, o `nome.upper()` faz o papel do `awk`, e o `field` escolhe a chave do JSON, gerando o `:password::` que você viu no `valueFrom`. O `awk`, que nasceu em 1977, perde mais um emprego neste projeto sem nunca ter reclamado.

Os parâmetros de configuração não são secretos. Eles passam pela lista `secrets` porque assim o valor continua morando no Parameter Store, por ambiente, e a task definition guarda só o ARN. Isso tem um efeito colateral bom: o CDK dá à execution role permissão de leitura em cada parâmetro e no segredo, inclusive o `kms:Decrypt` na chave do capítulo 16, sem uma linha a mais.

O serviço da API

O capítulo 15 prometeu os constructs L3, os *patterns*, e promessa de capítulo anterior, ao contrário do "semana que vem eu documento", se cumpre. O `ApplicationLoadBalancedFargateService` monta de uma vez o ALB, os listeners, o target group, a task definition, o serviço e o registro no Route 53:

```
        api = ecs_patterns.ApplicationLoadBalancedFargateService(
            self,
            "ServicoApi",
            cluster=cluster,
            cpu=512,
            memory_limit_mib=1024,
            desired_count=2,
            min_healthy_percent=100,
            runtime_platform=plataforma,
            task_image_options=ecs_patterns.ApplicationLoadBalancedTaskImageOptions(
                image=imagem,
                container_port=8000,
                secrets=variaveis,
                task_role=role_app,
                log_driver=ecs.LogDrivers.aws_logs(
                    stream_prefix="api", log_group=logs_app
                ),
            ),
            task_subnets=privadas,
            security_groups=[sg_api],
            public_load_balancer=True,
            certificate=certificado,
            ssl_policy=elbv2.SslPolicy.RECOMMENDED_TLS,
            redirect_http=True,
            domain_name=f"api.{dominio}",
            domain_zone=zona,
            circuit_breaker=ecs.DeploymentCircuitBreaker(rollback=True),
```

```
        )
        api.target_group.configure_health_check(
            path="/health",
            interval=Duration.seconds(10),
            healthy_threshold_count=2,
            unhealthy_threshold_count=2,
        )
        api.target_group.set_attribute("deregistration_delay.timeout_seconds", "30")

        escala = api.service.auto_scale_task_count(min_capacity=2, max_capacity=6)
        escala.scale_on_request_count(
            "Requisicoes", requests_per_target=600, target_group=api.target_group
        )
        api.node.add_dependency(rotacao, *endpoints)
```

Quase tudo aqui é o capítulo 13 com outro nome, como remake de filme com elenco novo e o mesmo roteiro. O health check em `/health` com 10 segundos e 2 de 2, o deregistration delay de 30 segundos, o target tracking de 600 requisições por minuto por alvo, o mínimo de 2 e o máximo de 6 (o limite que as conexões do RDS impuseram no capítulo 13). A diferença é o que escala: tasks, que sobem em segundos, em vez de instâncias, que levavam minutos.

O tamanho da task, 0,5 vCPU e 1 GB, segue o raciocínio do GIL do capítulo 4: um processo Uvicorn usa no máximo um núcleo, e a escala vem de mais tasks. O Fargate só aceita algumas combinações; com 0,5 vCPU, a memória vai de 1 a 4 GB.

O `min_healthy_percent=100` diz como o deploy acontece. Com o máximo padrão de 200%, o ECS sobe as tasks novas ao lado das antigas, espera o health check passar e só então drena e para as antigas. Nunca há menos de duas tasks saudáveis.

O `circuit_breaker` com `rollback=True` vigia esse deploy. Se as tasks novas falham repetidamente, o ECS marca o deploy como falho e volta para a revisão anterior da task definition. Como o deploy está dentro do CloudFormation, a stack também reverte.

O `add_dependency` repete a lição do capítulo 16: o serviço só nasce depois que os endpoints e a agenda de rotação existem.

Quando a task para, num deploy ou numa redução de escala, o ECS manda `SIGTERM` ao processo principal e espera 30 segundos por padrão antes do `SIGKILL`. O `CMD` em forma de lista do capítulo 3 garante que o sinal chegue ao Uvicorn, que termina as requisições em andamento. O `SIGTERM` é o garçom avisando que a cozinha fechou; o `SIGKILL` é ele apagando as luzes com você ainda na mesa.

Os workers

Os workers do capítulo 5 não recebem tráfego HTTP. Eles leem o stream no Valkey e precisam só de um serviço que os mantenha vivos:

```
        for grupo in ("restaurante", "email"):
            tarefa = ecs.FargateTaskDefinition(
                self,
                f"TarefaWorker_{grupo}",
                cpu=256,
                memory_limit_mib=512,
                runtime_platform=plataforma,
            )
            tarefa.add_container(
                "worker",
                image=imagem,
                command=["python", "-m", "app.worker", grupo],
                secrets=variaveis,
                logging=ecs.LogDrivers.aws_logs(
                    stream_prefix=f"worker-{grupo}", log_group=logs_app
                ),
            )
            worker = ecs.FargateService(
                self,
                f"Worker_{grupo}",
                cluster=cluster,
                task_definition=tarefa,
                desired_count=1,
                min_healthy_percent=100,
                vpc_subnets=privadas,
                security_groups=[sg_api],
                circuit_breaker=ecs.DeploymentCircuitBreaker(rollback=True),
            )
            worker.node.add_dependency(rotacao, *endpoints)
```

É a template unit `rango-worker@.service` do capítulo 9, com o `%i` virando o `grupo` do laço. A imagem é a mesma da API; o `command` troca o Uvicorn pelo worker. Os workers não recebem a `role_app`, porque não mexem nas fotos. O CDK cria para eles uma task role vazia. Um crachá que não abre porta nenhuma, a forma mais honesta de privilégio mínimo.

O `sg_api` serve aos três serviços porque é ele que o banco e o cache aceitam desde o capítulo 15. O pattern acrescenta nele a regra que libera a porta 8000 vindo do Security Group do ALB.

Por fim, as saídas:

```
        CfnOutput(self, "Cluster", value=cluster.cluster_name)
        CfnOutput(self, "ServicoApiNome", value=api.service.service_name)
        CfnOutput(self, "SegredoAppArn", value=segredo_app.secret_arn)
```

Deploy

```
cdk diff RangoProducao -c versao_api="$VERSAO"
cdk deploy RangoProducao -c versao_api="$VERSAO"
```

O deploy leva uns 15 minutos, quase todos do banco restaurando do snapshot. Restaurar dados é a única etapa que ninguém consegue apressar gritando com a tela, e muita gente já tentou. Quando ele termina, as saídas ficam guardadas com a função `saida` do capítulo 16:

```
CLUSTER=$(saida Cluster)
SERVICO=$(saida ServicoApiNome)
```

Lembra da frase sobre os segredos? A situação do capítulo 16 se repete. A stack criou um segredo novo com uma senha gerada pelo CDK, e a primeira rotação acontece em paralelo com a subida das tasks. Uma task que leu o segredo antes da rotação terminar ficou com uma senha que o banco nunca conheceu. O `/health` não toca no banco (o capítulo 4 explicou por quê), então essa task parece saudável e devolve erro 500 em `/pedidos`, como o paciente que passa no check-up e desmaia na escada do consultório. A saída é esperar a rotação terminar, conferir com o `describe-secret` do capítulo 16 e trocar todas as tasks:

```
aws ecs update-service --cluster "$CLUSTER" --service "$SERVICO" \
  --force-new-deployment > /dev/null
```

O mesmo vale para os dois workers. O *force new deployment* é o instance refresh do ECS: as tasks são substituídas com as regras do deploy normal, e as novas leem o segredo atual.

O andamento do deploy aparece assim:

```
aws ecs describe-services --cluster "$CLUSTER" --services "$SERVICO" \
  --query "services[0].deployments[].[status,rolloutState,runningCount]" \
  --output table
```

```
-----------------------------------------
|           DescribeServices            |
+----------+--------------+-------------+
|  PRIMARY |  IN_PROGRESS |  2          |
|  ACTIVE  |  COMPLETED   |  2          |
+----------+--------------+-------------+
```

Por um momento, há quatro tasks: duas novas (`PRIMARY`) e duas antigas (`ACTIVE`). Um minuto depois, sobra só a linha `PRIMARY` com `COMPLETED`.

Com o front no ar, pelos dois comandos de sync do capítulo 14, o teste é este:

```
for i in 1 2 3 4; do
  curl -s -o /dev/null -D - https://api.rango.com.br/health | grep -i x-rango-instancia
done
```

```
x-rango-instancia: ip-10-20-10-87.ec2.internal
x-rango-instancia: ip-10-20-11-142.ec2.internal
x-rango-instancia: ip-10-20-10-87.ec2.internal
x-rango-instancia: ip-10-20-11-142.ec2.internal
```

O cabeçalho do capítulo 4 mostra cada task pelo IP privado dela, uma em cada AZ. E os logs estão num lugar só. Acabou a caça ao log de instância em instância, que lembrava procurar o controle remoto entre as almofadas do sofá:

```
aws logs tail "$(aws logs describe-log-groups \
  --log-group-name-prefix RangoProducao-LogsApp \
  --query 'logGroups[0].logGroupName' --output text)" --since 5m
```

A linha do Bruno, de novo

De volta à correção do fuso horário. Agora o caminho é este:

```
git commit -am "Listagem de pedidos no fuso de Sao Paulo"
VERSAO=$(git rev-parse --short HEAD)
docker buildx build --platform linux/arm64 \
  -t "$REGISTRO/rango-api:$VERSAO" --push backend/
cdk deploy RangoProducao -c versao_api="$VERSAO"
```

O `cdk diff` mostraria uma única mudança: a tag da imagem nas três task definitions, que ganham revisões novas. O deploy troca as tasks da API e dos workers e termina em poucos minutos. Ninguém entrou em servidor nenhum, e a imagem que roda em produção é a mesma que você testou com `docker run` no notebook. A hora e quarenta virou poucos minutos, e o filme com trailers fica para outro dia.

O circuit breaker aparece trabalhando quando alguém constrói uma versão com `--platform linux/amd64`, envia e faz o deploy com essa tag. Esse alguém, quase sempre, é você numa segunda-feira de manhã. As tasks novas morrem logo depois de subir, com `exec format error` no log, enquanto as antigas continuam atendendo. Depois de algumas falhas, o ECS marca o deploy como `FAILED`, volta para a revisão anterior, e o CloudFormation reverte a stack. Em nenhum momento o `api.rango.com.br` saiu do ar. O Bruno nem ficou sabendo, que é exatamente a quantidade certa de informação para um cliente.

O capítulo 18 vai tirar esses comandos do seu terminal e colocá-los num pipeline.

A AMI pode ir embora

A AMI `rango-api-20261015` não tem mais função, e ainda guarda o `api.env` com a senha morta, a versão em nuvem do post-it com a senha antiga que fica colado no monitor "só por garantia". Uma AMI tem um snapshot de EBS por trás, que continua cobrando depois que a AMI some, então os dois vão embora:

```
AMI=$(aws ec2 describe-images --owners self \
  --filters Name=name,Values=rango-api-20261015 \
  --query "Images[0].ImageId" --output text)
SNAP=$(aws ec2 describe-images --image-ids "$AMI" \
  --query "Images[0].BlockDeviceMappings[0].Ebs.SnapshotId" --output text)

aws ec2 deregister-image --image-id "$AMI"
aws ec2 delete-snapshot --snapshot-id "$SNAP"
```

Um `cdk context --clear` completa a limpeza, tirando do `cdk.context.json` o lookup da AMI que não existe mais.

Quanto custa

Com os preços de Fargate em ARM em us-east-1 (outubro de 2026, aproximados; confira a página oficial), a conta fica assim:

Serviço	Tasks	Tamanho	Por mês (730 h)
API	2	0,5 vCPU, 1 GB	~US$ 29
Workers	2	0,25 vCPU, 0,5 GB	~US$ 14
Endpoints novos	3 × 2 AZs		~US$ 44

O cálculo da API: 0,5 × US$ 0,032380 por vCPU-hora mais 1 × US$ 0,003560 por GB-hora dá cerca de US$ 0,0198 por hora por task. O Fargate cobra por segundo, com mínimo de um minuto, desde o início do download da imagem até a task terminar.

Duas `t3.micro` custavam perto de US$ 15 por mês e rodavam API e workers juntos. O Fargate cobra mais pelo mesmo processamento; a diferença paga o tempo que você deixou de gastar com AMI e patches. Se o seu tempo vale menos do que essa diferença, o problema é outro, e nenhum livro de AWS resolve.

Os endpoints merecem atenção. Cada um parece barato sozinho, como o cafezinho da padaria; o susto vem na soma do mês. Somados aos dois do capítulo 16, são cinco interface endpoints em duas AZs, cerca de US$ 73 por mês, praticamente o preço de dois NAT Gateways sem contar o tráfego. Para o Rangô, a balança ainda pende para os endpoints: nada sai para a internet, e não há cobrança por GB processado no NAT. Mas, se a aplicação precisasse chamar APIs externas, como um gateway de pagamento, o NAT voltaria de qualquer jeito, e aí muitos endpoints deixariam de compensar.

Hora de desligar

```
cdk destroy RangoProducao
```

O banco sai com snapshot final, como nos capítulos anteriores; anote o nome. O cluster, os serviços, o ALB, os endpoints e o grupo de logs saem junto. A stack `RangoRegistro` fica, com o repositório e as imagens; algumas centenas de megabytes custam centavos por mês, o aluguel mais barato que você vai pagar na vida. A chave do KMS entra em `PendingDeletion` por sete dias, como no capítulo 16.

E o Kubernetes?

O capítulo 3 apresentou o Kubernetes, e a AWS tem o Amazon EKS (*Elastic Kubernetes Service*). O EKS gerencia o control plane: a API do Kubernetes, o etcd e os controladores rodam em várias AZs por conta da AWS. Você continua com o `kubectl`, os manifests e as ferramentas do mundo Kubernetes, como Helm e os operators.

O control plane custa US$ 0,10 por hora por cluster, cerca de US$ 73 por mês, antes de rodar qualquer pod (us-east-1, outubro de 2026). Cada versão do Kubernetes tem 14 meses de suporte padrão. Depois disso, o cluster entra em suporte estendido por mais 12 meses e passa a custar US$ 0,60 por hora, seis vezes mais. É a multa por atraso da biblioteca, só que cobrada por hora. É o mesmo mecanismo do Extended Support do RDS, do capítulo 12: quem não atualiza paga. E atualizar um cluster Kubernetes envolve também os add-ons, os nós e as APIs que foram removidas entre versões.

Os pods podem rodar em instâncias que você gerencia, no próprio Fargate ou no EKS Auto Mode, em que a AWS escolhe e mantém as instâncias em troca de uma taxa extra por instância sobre o preço da EC2.

Quando o EKS vale a pena? Quando a equipe já domina Kubernetes e tem manifests, charts e processos prontos. Quando a empresa precisa da mesma plataforma em mais de uma nuvem ou no próprio data center. Quando a aplicação depende de algo que só existe no mundo Kubernetes, como um operator de banco ou uma malha de serviços específica. E quando são dezenas de serviços e várias equipes, em que o investimento numa plataforma se paga.

Para o Rangô, com uma API, dois workers e uma pessoa cuidando da infraestrutura, o ECS faz o mesmo trabalho sem o custo fixo e sem as atualizações de versão a cada ano. Montar Kubernetes para uma API e dois workers seria como contratar uma transportadora com frota de caminhões para entregar um envelope.

> **Dica:** Existe um caminho ainda mais curto no ECS. O ECS Express Mode, lançado em novembro de 2025, recebe só a imagem e cria cluster, task definition, ALB, auto scaling e uma URL. Ele não cobra nada além dos recursos criados, que ficam visíveis e editáveis na sua conta, e até 25 serviços Express podem dividir o mesmo ALB. É

uma boa porta de entrada para um protótipo. Para o Rangô, que já tem VPC, certificado e regras de rede próprias, o CDK dá o controle que o Express Mode esconde.

Armadilhas e casos reais

Armadilha: Subnet sem caminho para o ECR. Sem os endpoints `ecr.api` e `ecr.dkr` (ou sem NAT), a task fica em `PROVISIONING` por um tempo e para com um motivo que começa com `ResourceInitializationError: unable to pull secrets or registry auth`. A mensagem fala de segredos e de registry ao mesmo tempo porque as duas buscas acontecem na mesma etapa, pela interface da task. Confira os endpoints, o gateway endpoint do S3 e o Security Group dos endpoints, que precisa aceitar a porta 443 vindo das tasks.

Armadilha: Rotação não chega à task. O ECS lê o segredo uma vez, quando a task sobe. Com usuários alternados, a senha lida continua válida até a rotação seguinte à próxima, ou seja, por pelo menos 30 dias. Uma task da API que viva mais que isso perde o acesso ao banco sem aviso. O `max_instance_lifetime` do capítulo 16 não tem equivalente direto no ECS. Na prática, deploys frequentes resolvem, e o pipeline do capítulo 18 vai garantir que eles aconteçam; até lá, um `--force-new-deployment` por semana cumpre o papel.

Armadilha: Permissão na role errada. `AccessDenied` ao gerar a presigned URL quer dizer que a permissão do S3 está na execution role. Task parada antes de subir, com erro de segredo, quer dizer que a permissão do segredo está na task role, ou falta o `kms:Decrypt`. Antes de mexer em policy, descubra qual das duas roles está envolvida.

Armadilha: Variável de ambiente é visível para tudo dentro do container: ferramentas de debug, bibliotecas que despejam o ambiente em mensagens de erro, um `print(os.environ)` esquecido (que, quando foi escrito, era só temporário). Nunca registre o ambiente em log; o CloudWatch Logs guardaria a senha por 30 dias.

Armadilha: Imagem gorda custa dinheiro. O Fargate cobra a partir do início do download da imagem, e uma imagem de 2 GB demora a baixar a cada task nova, inclusive nas que o auto scaling cria no meio de um pico. Mantenha a base `slim`, o `.dockerignore` em dia e as ferramentas de build fora da imagem final. Cada task tem 20 GB de armazenamento temporário sem custo; o que passar disso é cobrado à parte.

> **Dica:** Fargate Spot roda tasks em capacidade ociosa da AWS com desconto de até 70%, só no ECS e só em Linux. A AWS pode interromper a task com dois minutos de aviso, o que já é mais antecedência do que muita reunião urgente costuma dar. A API precisa estar sempre de pé e fica no Fargate normal, mas os workers são bons candidatos: eles processam de forma idempotente desde o capítulo 5, e uma mensagem interrompida volta a ser processada. O cuidado é o mesmo do capítulo 13: as mensagens pendentes ficam presas ao nome do consumidor que sumiu até alguém reivindicá-las. O capítulo 21 troca esses workers por filas do SQS, e o Fargate Spot volta no capítulo 24, no ambiente de homologação.
>
> **Na vida real:** Muitas equipes usam o `ecs.ContainerImage.from_asset("../backend")` do CDK, que constrói e envia a imagem dentro do `cdk deploy`. É prático para começar. O repositório próprio, com tags imutáveis, separa o build do deploy: a imagem testada no CI é a que vai para produção, e voltar uma versão é fazer o deploy de uma tag antiga.

Resumo

- O ECR guarda imagens com acesso pelo IAM. Tags imutáveis com o hash do commit, varredura no push e regras de ciclo de vida evitam a `latest` e o repositório inchado.
- O ECS orquestra: task definition é a receita versionada, task é a execução, service mantém o número de tasks, faz o deploy gradual e registra no ALB. O ECS não cobra; a capacidade, sim.
- O Fargate tira o servidor de cena e cobra por vCPU e memória por segundo. Em ARM, sai cerca de 20% mais barato.
- Execution role é do ECS (imagem, segredos, logs); task role é do seu código.
- Segredos entram pela lista `secrets` da task definition, lidos uma vez quando a task sobe. Rotação exige tasks novas.
- O Rangô roda em três serviços Fargate em subnets isoladas, com cinco interface endpoints, circuit breaker e logs no CloudWatch. A AMI foi apagada.
- O EKS vale a pena com equipe que já domina Kubernetes, muitas aplicações ou necessidade de portabilidade, e custa US$ 0,10 por hora por cluster antes do primeiro pod.

Sexta-feira, 18h

O deploy encolheu para quatro comandos, mas eles ainda rodam no seu notebook, com a sua sessão SSO e o seu Docker. Na sexta-feira às 18h, o Bruno pede uma correção urgente no cálculo da taxa de entrega antes do movimento do fim de semana. Você está no aniversário da sua sobrinha, com um pedaço de bolo numa mão e o celular na outra, e a outra pessoa da equipe não tem o `buildx` configurado nem permissão para rodar `cdk deploy`. Ou pior: tem, e roda a partir de uma branch que não passou pelos testes. Deploy na sexta às 18h já é esporte radical; feito do notebook de outra pessoa, é esporte radical sem capacete.

Quem garante que a imagem em produção veio do commit que passou no CI? Quem decide que o deploy pode seguir, e quem aperta o botão de voltar quando a versão nova começa a devolver erro 500? No próximo capítulo, o GitHub Actions assume o build, o push para o ECR e o deploy no ECS, autenticado na AWS por OIDC, sem nenhuma chave de longa duração guardada no GitHub. E o deploy ganha o blue/green nativo do ECS, com tráfego de teste antes da troca e rollback automático por alarme.

Referências

- Amazon ECR, preços: https://aws.amazon.com/ecr/pricing
- AWS Fargate, preços: https://aws.amazon.com/fargate/pricing/
- Amazon ECS, preços: https://aws.amazon.com/ecs/pricing/
- Amazon EKS, preços: https://aws.amazon.com/eks/pricing/
- Passar segredos do Secrets Manager como variáveis de ambiente no ECS: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/secrets-envvar-secrets-manager.html
- CloudFormation, `AWS::ECS::TaskDefinition` Secret: https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-properties-ecs-taskdefinition-secret.html
- Anúncio do ECS Express Mode (novembro de 2025): https://aws.amazon.com/about-aws/whats-new/2025/11/announcing-amazon-ecs-express-mode/
- Anúncio do blue/green nativo do ECS (julho de 2025): https://aws.amazon.com/about-aws/whats-new/2025/07/amazon-ecs-built-in-blue-green-deployments/
- Blue/green nativo no ECS, post do AWS News Blog: https://aws.amazon.com/blogs/aws/accelerate-safe-software-releases-with-new-built-in-blue-green-deployments-in-amazon-ecs

- Lifecycle hooks no blue/green do ECS: https://aws.amazon.com/blogs/containers/extending-deployment-pipelines-with-amazon-ecs-blue-green-deployments-and-lifecycle-hooks
- Documentação do Amazon ECS: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html
- Referência da API do AWS CDK, módulo `aws_ecs_patterns`: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_ecs_patterns.html

CAPÍTULO 18

CI/CD na AWS

No capítulo 17, o Rangô trocou as instâncias por três serviços no Fargate. A imagem mora no ECR, com tags imutáveis que carregam o hash do commit, e o serviço da API tem circuit breaker com rollback. O deploy ficou curto: login no ECR, `buildx`, push e `cdk deploy RangoProducao -c versao_api=...`. Ao fim, a stack `RangoProducao` foi destruída e a `RangoRegistro`, com o repositório e as imagens, ficou, como a caixa de fotos de família que sobrevive a todas as mudanças de casa.

Sexta-feira, 18h07

O Bruno manda mensagem: a taxa de entrega da Sabor do Sertão aparece como R$ 6,90 no app, mas o combinado é R$ 4,50. O erro entrou no último commit, numa linha copiada da Cantina da Nona. Ctrl+C e Ctrl+V: a dupla responsável por boa parte dos bugs do mundo e por quase todo o software que funciona. Sábado é o dia de maior movimento, e cada pedido com a taxa errada é um cliente que desiste no checkout ou um reembolso para fazer na segunda.

A correção é uma linha. O deploy, quatro comandos. Na teoria, coisa de cinco minutos; na prática, a teoria nunca foi a um aniversário de criança. Só que os quatro comandos dependem de coisas que estão no seu notebook: a sessão SSO do profile `rango-dev`, o Docker com o `buildx` configurado para ARM, o Node com a CLI do CDK e o `cdk.context.json` atualizado. Você está no aniversário da sua sobrinha, a uma hora de casa, e acabaram de acender as velinhas.

A outra pessoa da equipe tem acesso ao repositório, mas não tem o `buildx`. Se ela conseguir configurar tudo, qual commit vai usar? O da `main`, que passou nos testes, ou o da branch dela, com a correção e mais duas mudanças pela metade? O ECR não sabe a diferença: a tag é o que a pessoa digitou. Ele guarda o que receber, com a indiferença de um cofre que não pergunta de onde veio o conteúdo.

E se a versão nova devolver erro 500 em `/restaurantes`? O `/health` não toca no banco, então o circuit breaker não percebe nada. Alguém precisa notar, descobrir a tag anterior e rodar o `cdk deploy` de novo. Esse alguém, por todas as leis do universo, vai estar no meio do parabéns.

O CI do capítulo 6 testa e constrói a imagem, mas joga o resultado fora junto com o runner, como castelo de areia na maré alta.

O pipeline completo

A solução é a entrega contínua do capítulo 6, levada até o fim. Um *pipeline* recebe cada commit na `main`, roda os testes, constrói a imagem uma única vez, envia para o ECR e faz o deploy dessa imagem exata. Se você quiser um ponto de controle humano, ele vira um botão de aprovação, e não um terminal aberto. Um botão é a interface favorita de qualquer gerente, e desta vez com razão.

O Rangô já usa o GitHub, e o GitHub Actions já roda os testes. A AWS tem um serviço próprio de pipelines, o AWS CodePipeline, e vamos falar dele no fim do capítulo; como todo parente que se preze, ele chega na hora da sobremesa. Para um projeto que mora no GitHub, porém, o Actions resolve com menos peças. O ponto difícil é outro: como o runner do GitHub, uma máquina que não é sua e que só existe durante o job, prova para a AWS que pode fazer deploy? É um desconhecido batendo à porta e dizendo que veio a mando do dono da casa.

O jeito errado de autenticar

Muitos tutoriais antigos criam um usuário IAM, geram uma access key e guardam o par `AKIA...` nos *secrets* do repositório. Esses tutoriais seguem vivos porque a internet nunca apaga nada, principalmente o que deveria. O próprio README da action de deploy do ECS ainda mostra os parâmetros `aws-access-key-id` e `aws-secret-access-key`.

Funciona, e é o problema do capítulo 8. A chave não expira e vale para quem a tiver. Uma action de terceiros comprometida ou um `run: env` esquecido num passo de debug a vazam, e você só descobre pela fatura. Access key nos secrets do repositório é a chave reserva debaixo do tapete: todo mundo sabe onde procurar.

OIDC, ou o crachá emitido na hora

O GitHub resolve isso com OIDC (*OpenID Connect*). A cada job, o GitHub pode emitir um token assinado, um JWT, que diz quem está rodando: qual repositório, qual branch, qual environment, qual evento disparou o workflow. É o mesmo tipo de token que o "Entrar com Google" entrega ao seu backend para provar a identidade de um usuário, aquele botão que poupou a humanidade de criar a senha número 347.

A AWS, por sua vez, sabe confiar em quem emite esses tokens. É uma relação de confiança mais saudável do que muitas por aí, baseada em assinatura conferida e não em promessa. Você cadastra o GitHub como provedor de identidade no IAM e cria uma role

cuja trust policy diz: aceito tokens emitidos por `token.actions.githubusercontent.com`, desde que tragam estas informações. O runner apresenta o token ao STS com a chamada `AssumeRoleWithWebIdentity` e recebe credenciais temporárias, as `ASIA...` do capítulo 8, que valem uma hora.

O fluxo inteiro fica assim:

```
Runner do GitHub                 GitHub OIDC                  AWS STS
      |                               |                          |
      |-- pede token (id-token) ----->|                          |
      |<-- JWT assinado --------------|                          |
      |     sub: repo:SEU-USUARIO/rango:environment:producao     |
      |     aud: sts.amazonaws.com                               |
      |                                                          |
      |-- AssumeRoleWithWebIdentity(role, JWT) ----------------->|
      |                      confere assinatura, aud e sub       |
      |<-- credenciais temporárias (ASIA..., 1 h) ---------------|
```

Nenhum segredo fica guardado no GitHub. Não há nada para vazar além de um token que expira em minutos e de credenciais que expiram em uma hora. É como perder um bilhete de metrô já usado: quem achar não vai longe.

A parte que exige atenção está na trust policy. Ela precisa conferir dois campos do token. O `aud`, a audiência, tem de ser `sts.amazonaws.com`. O `sub`, o assunto, diz de onde o job veio, e o formato depende do contexto:

Contexto do job	Valor do `sub`
push na branch `main`	`repo:SEU-USUARIO/rango:ref:refs/heads/main`
job com `environment: producao`	`repo:SEU-USUARIO/rango:environment:producao`
pull request	`repo:SEU-USUARIO/rango:pull_request`

Se a trust policy não conferir o `sub`, qualquer repositório do GitHub, de qualquer pessoa, consegue assumir a sua role, porque o token dele é assinado pela mesma chave do GitHub que o seu. Inclusive o `hello-world` de um desconhecido do outro lado do planeta.

> **Armadilha:** O GitHub passou a usar um formato novo de `sub` com IDs numéricos imutáveis, como `repo:octo-org@123456/octo-repo@456789:ref:refs/heads/main`, para repositórios criados a partir de 15 de julho de 2026 ou que fizeram opt-in. Se o seu repositório está nesse grupo, a condição com o formato antigo nunca casa, e o job falha com `Not authorized to perform sts:AssumeRoleWithWebIdentity`. Confira o formato que vale para o seu repositório na documentação do GitHub antes de escrever a condição.

Na prática

O plano é este:

1. Uma stack nova, `RangoPipeline`, com o provedor OIDC e duas roles.
2. Ajustes na `RangoProducao`: permissões do front para a role do pipeline e o blue/green nativo do ECS na API.
3. Um environment `producao` no GitHub, com aprovação e restrito à `main`.
4. O workflow, que passa a fazer build, push e deploy.

Pré-requisitos: o repositório `rango-infra/` do capítulo 17, a stack `RangoRegistro` no ar, o snapshot do fim do capítulo 17 com o nome anotado no `app.py`, a CLI do GitHub (`gh`) autenticada e o profile `rango-dev`.

Duas roles, dois crachás

Por que duas? O pipeline faz duas coisas com riscos diferentes. Enviar uma imagem ao ECR acontece a cada push na `main` e só mexe no repositório de imagens. Fazer deploy muda a produção e deve passar pela aprovação. Cada uma ganha a sua role, e cada role aceita um `sub` diferente. É a mesma lógica de não entregar a chave do cofre para quem só veio regar as plantas.

As duas moram em `rango_infra/pipeline_stack.py`:

```
from aws_cdk import Stack, aws_ecr as ecr, aws_iam as iam
from constructs import Construct

GITHUB = "token.actions.githubusercontent.com"


class PipelineStack(Stack):
    def __init__(
        self, scope: Construct, construct_id: str, *, repositorio_github: str, **kwargs
    ) -> None:
        super().__init__(scope, construct_id, **kwargs)

        provedor = iam.OpenIdConnectProvider(
            self,
            "GitHub",
            url=f"https://{GITHUB}",
            client_ids=["sts.amazonaws.com"],
        )

        def confia_em(sub: str) -> iam.IPrincipal:
            return iam.OpenIdConnectPrincipal(
                provedor,
                conditions={
                    "StringEquals": {
                        f"{GITHUB}:aud": "sts.amazonaws.com",
                        f"{GITHUB}:sub": sub,
                    }
                },
```

```
            )

        role_imagem = iam.Role(
            self,
            "RoleImagem",
            role_name="rango-github-imagem",
            assumed_by=confia_em(f"repo:{repositorio_github}:ref:refs/heads/main"),
        )
        repositorio = ecr.Repository.from_repository_name(self, "RepoApi", "rango-api")
        repositorio.grant_pull_push(role_imagem)
        repositorio.grant(role_imagem, "ecr:DescribeImages")

        role_deploy = iam.Role(
            self,
            "RoleDeploy",
            role_name="rango-github-deploy",
            assumed_by=confia_em(f"repo:{repositorio_github}:environment:producao"),
        )
        role_deploy.add_to_policy(
            iam.PolicyStatement(
                actions=["sts:AssumeRole"],
                resources=[
                    f"arn:aws:iam::{self.account}:role/cdk-hnb659fds-*-{self.account}-
{self.region}"
                ],
            )
        )
        role_deploy.add_to_policy(
            iam.PolicyStatement(
                actions=["cloudformation:DescribeStacks"],
                resources=[
                    f"arn:aws:cloudformation:{self.region}:
{self.account}:stack/RangoProducao/*"
                ],
            )
        )
        role_deploy.add_to_policy(
            iam.PolicyStatement(
                actions=["ecs:UpdateService"],
                resources=[
                    f"arn:aws:ecs:{self.region}:{self.account}:service/rango-producao/*"
                ],
            )
        )
```

O `OpenIdConnectProvider` cadastra o GitHub no IAM; existe um por URL em cada conta. O `OpenIdConnectPrincipal` monta a trust policy com a ação `sts:AssumeRoleWithWebIdentity` e as duas condições com `StringEquals`. Você vai ver exemplos com `StringLike` e `repo:ORG/REPO:*`. O asterisco aceita qualquer branch, pull request ou environment do repositório, inclusive a branch que alguém criou ontem para testar uma ideia e batizou de `teste-nao-mexer`.

A `role_imagem` recebe o login no ECR, o envio de camadas e manifestos ao `rango-api` e o `DescribeImages`, que o workflow usa para saber se uma tag já existe.

A `role_deploy` pede mais cuidado, porque o `cdk deploy` cria VPC, banco e roles IAM. Muitos guias dão `AdministratorAccess` à role do GitHub, o que equivale a entregar a chave mestra do prédio ao motoboy que veio trazer o pedido. O caminho mais estreito usa as roles que o bootstrap do capítulo 15 criou, com o qualificador padrão `hnb659fds` no nome: `cdk-hnb659fds-deploy-role-...`, `...-file-publishing-role-...`, `...-image-publishing-role-...` e `...-lookup-role-...`. A CLI do CDK assume essas roles sozinha; a role do GitHub só precisa de permissão para assumi-las.

Isso desloca o risco em vez de eliminá-lo. A role de execução do bootstrap continua administradora, como o capítulo 15 avisou. A proteção está em quem assume a `role_deploy`: só um job no environment `producao`, a partir da `main`, depois de uma aprovação. O capítulo 23 volta a isso com contas separadas.

As duas últimas permissões atendem passos fora do CDK: ler as saídas da stack e forçar deploys no cluster `rango-producao`.

No `app.py`, a stack nova entra ao lado das outras:

```
from rango_infra.pipeline_stack import PipelineStack

PipelineStack(app, "RangoPipeline", repositorio_github="SEU-USUARIO/rango", env=env)
```

Com `SEU-USUARIO` trocado pelo dono do repositório, respeitando maiúsculas e minúsculas, o deploy sai do notebook:

```
cdk deploy RangoPipeline -c versao_api=nenhuma
```

O `-c versao_api=nenhuma` existe porque o CDK sintetiza o app inteiro, e a `RangoProducao` precisa de algum valor para a tag, mesmo sem ser implantada. Este é o último deploy que sai do seu terminal, e só podia ser este: a role que autoriza o pipeline não pode ser criada pelo próprio pipeline. O Barão de Münchhausen, na lenda, saiu do pântano puxando os próprios cabelos; na AWS, esse truque não funciona.

A produção abre a porta do front

O conteúdo do front fica fora da stack desde o capítulo 15. Quem vai enviá-lo agora é a `role_deploy`, que precisa escrever no bucket e invalidar a distribuição. As duas coisas estão na `RangoProducao`, e o nome do bucket é gerado pelo CDK, então a permissão nasce lá. Em `rango_infra/rango_infra_stack.py`, depois da distribuição:

```
        role_pipeline = iam.Role.from_role_name(self, "RolePipeline", "rango-github-
deploy")
        front.grant_read(role_pipeline)
        front.grant_put(role_pipeline)
        distribuicao.grant_create_invalidation(role_pipeline)
```

Os `grant_*` criam uma policy dentro da `RangoProducao` e a anexam à role importada; quando a stack for destruída, a policy vai junto. O `grant_read` traz o `s3:ListBucket`, de que o `aws s3 sync` precisa. Nenhum `grant` permite apagar, já que o deploy do front nunca usa `--delete` (capítulo 14). Pipeline que não consegue apagar nada é pipeline que não vira notícia.

É nesses passos fora do CDK que os pipelines quebram com `AccessDenied`: o `cdk deploy` passa, com as roles do bootstrap, e o `aws s3 sync` seguinte falha, porque roda com a role do GitHub. Costuma acontecer depois de quinze minutos de deploy bem-sucedido, para dar tempo de você criar esperança.

Blue/green na API

O deploy *rolling* do capítulo 17 tem um circuit breaker que volta atrás quando as tasks novas não ficam de pé. Quando elas ficam de pé e respondem errado, que é a história da sexta-feira, ele não faz nada. É o airbag que só abre depois que o carro já foi para o ferro-velho.

O ECS tem quatro estratégias de deploy: rolling (o padrão), blue/green, canary e linear, estas duas desde outubro de 2025. No blue/green, a versão atual (azul) continua atendendo enquanto a nova (verde) sobe inteira num segundo target group. O ALB tem uma regra de produção e uma de teste. A de teste manda tráfego ao verde primeiro, para você conferir antes de qualquer cliente. Depois o ECS vira a regra de produção para o verde de uma vez, e começa o *bake time*, nome que vem da cozinha, com a ideia junto: o bolo fica mais um pouco no forno antes de alguém declarar que está pronto. No bake time, azul e verde rodam juntos por um período configurável. Se um alarme do CloudWatch disparar nessa janela, o tráfego volta ao azul, que ainda está de pé, no tempo de trocar o peso de uma regra.

O deploy passa por estágios que aparecem no console, como `TEST_TRAFFIC_SHIFT` e `PRODUCTION_TRAFFIC_SHIFT`. Em vários deles cabe um *lifecycle hook*: uma função Lambda que decide se o deploy continua, ou uma pausa que espera alguém chamar `ContinueServiceDeployment`. Um hook no `POST_TEST_TRAFFIC_SHIFT` com testes de fumaça é o próximo passo natural (na eletrônica, o teste de fumaça era literal: ligar o aparelho e ver se saía fumaça); para o Rangô, o alarme basta por enquanto. O custo do bake time é dobrar as tasks durante a janela, frações de centavo por deploy.

O `ApplicationLoadBalancedFargateService` do capítulo 17 cria um target group só. Para ter dois, e regras de listener, a API sai do pattern. Os patterns são ótimos até o dia em que você precisa de algo que eles não previram, e esse dia sempre chega. Saem o bloco `api = ecs_patterns...` com o health check e a escala que vinham depois, e o import de `aws_ecs_patterns`. Entra `aws_cloudwatch as cloudwatch`. Primeiro, o ALB e as regras:

```
        alb = elbv2.ApplicationLoadBalancer(
            self,
            "AlbApi",
            vpc=vpc,
            internet_facing=True,
            vpc_subnets=ec2.SubnetSelection(subnet_type=ec2.SubnetType.PUBLIC),
        )
        alb.add_redirect()
        https = alb.add_listener(
            "Https",
            port=443,
            certificates=[certificado],
            ssl_policy=elbv2.SslPolicy.RECOMMENDED_TLS,
            default_action=elbv2.ListenerAction.fixed_response(404),
        )
        sg_api.connections.allow_from(alb, ec2.Port.tcp(8000), "ALB para a API")

        def grupo_alvo(nome: str) -> elbv2.ApplicationTargetGroup:
            return elbv2.ApplicationTargetGroup(
                self,
                nome,
                vpc=vpc,
                port=8000,
                protocol=elbv2.ApplicationProtocol.HTTP,
                target_type=elbv2.TargetType.IP,
                deregistration_delay=Duration.seconds(30),
                health_check=elbv2.HealthCheck(
                    path="/health",
                    interval=Duration.seconds(10),
                    healthy_threshold_count=2,
                    unhealthy_threshold_count=2,
                ),
            )

        azul = grupo_alvo("GrupoAzul")
        verde = grupo_alvo("GrupoVerde")

        def regra(nome: str, prioridade: int, condicoes) ->
elbv2.ApplicationListenerRule:
            return elbv2.ApplicationListenerRule(
                self,
                nome,
                listener=https,
                priority=prioridade,
                conditions=condicoes,
                action=elbv2.ListenerAction.weighted_forward(
                    [
                        elbv2.WeightedTargetGroup(target_group=azul, weight=100),
                        elbv2.WeightedTargetGroup(target_group=verde, weight=0),
                    ]
                ),
            )

        regra_teste = regra(
            "RegraTeste", 10, [elbv2.ListenerCondition.http_header("X-Rango-Teste",
["verde"])]
        )
        regra_producao = regra(
            "RegraProducao", 20, [elbv2.ListenerCondition.path_patterns(["/*"])]
        )
```

Os target groups repetem o health check do capítulo 13, e o `add_redirect` recria o redirecionamento da porta 80. As regras são avaliadas da menor prioridade para a maior: com o cabeçalho `X-Rango-Teste: verde`, a requisição cai na regra de teste; o resto cai na de produção. As duas começam com 100% no azul, e o ECS muda os pesos durante o deploy.

A regra de teste fica no listener público, e quem descobrir o cabeçalho alcança a versão verde durante um deploy, uma espécie de easter egg involuntário para curiosos. Como ela é o próximo código de produção, isso expõe pouco; rotas que ainda não podem ser vistas pedem uma feature flag (capítulo 6).

Agora o alarme que decide a volta, a task definition e o serviço:

```
        erros_5xx = cloudwatch.Alarm(
            self,
            "ErrosApi",
            metric=alb.metrics.http_code_target(
                elbv2.HttpCodeTarget.TARGET_5XX_COUNT,
                period=Duration.minutes(1),
                statistic="Sum",
            ),
            threshold=10,
            evaluation_periods=1,

comparison_operator=cloudwatch.ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD,
            treat_missing_data=cloudwatch.TreatMissingData.NOT_BREACHING,
        )

        tarefa_api = ecs.FargateTaskDefinition(
            self,
            "TarefaApi",
            cpu=512,
            memory_limit_mib=1024,
            runtime_platform=plataforma,
            task_role=role_app,
        )
        tarefa_api.add_container(
            "api",
            image=imagem,
            secrets=variaveis,
            port_mappings=[ecs.PortMapping(container_port=8000)],
            logging=ecs.LogDrivers.aws_logs(stream_prefix="api", log_group=logs_app),
        )

        api = ecs.FargateService(
            self,
            "ServicoApi",
            cluster=cluster,
            task_definition=tarefa_api,
            desired_count=2,
            vpc_subnets=privadas,
            security_groups=[sg_api],
            deployment_strategy=ecs.DeploymentStrategy.BLUE_GREEN,
            bake_time=Duration.minutes(10),
            circuit_breaker=ecs.DeploymentCircuitBreaker(rollback=True),
            deployment_alarms=ecs.DeploymentAlarmConfig(
                alarm_names=[erros_5xx.alarm_name],
```

```
                behavior=ecs.AlarmBehavior.ROLLBACK_ON_ALARM,
            ),
        )
        api.load_balancer_target(
            container_name="api",
            container_port=8000,
            alternate_target=ecs.AlternateTarget(
                "Verde",
                alternate_target_group=verde,

production_listener=ecs.ListenerRuleConfiguration.application_listener_rule(
                    regra_producao
                ),
                test_listener=ecs.ListenerRuleConfiguration.application_listener_rule(
                    regra_teste
                ),
            ),
        ).attach_to_application_target_group(azul)

        escala = api.auto_scale_task_count(min_capacity=2, max_capacity=6)
        escala.scale_on_cpu_utilization("Cpu", target_utilization_percent=60)
        api.node.add_dependency(rotacao, *endpoints)
```

O alarme dispara com dez ou mais respostas 5xx das tasks num minuto. O `NOT_BREACHING` trata minuto sem dados como normal, porque minuto sem erro não gera ponto na métrica. O número 10 é um ponto de partida, escolhido com o rigor científico de quem chuta um número redondo e promete revisar depois; o capítulo 22 troca contagens por taxas e SLOs. O `deployment_alarms` faz o ECS observar o alarme durante o deploy e o bake time, e o `ROLLBACK_ON_ALARM` devolve o tráfego ao azul. O circuit breaker fica para a task que nem sobe.

O `load_balancer_target` com o `AlternateTarget` registra as tasks no azul e entrega ao ECS o verde e as duas regras. Para mexer nelas, o ECS usa uma role de infraestrutura com a policy gerenciada `AmazonECSInfrastructureRolePolicyForLoadBalancers`, que o CDK cria sozinho.

A escala mudou de métrica. Com blue/green, o target group ativo se alterna a cada deploy, e uma política de requisições presa ao azul veria zero depois do primeiro deploy. A CPU média do serviço não depende do grupo ativo.

Por fim, o registro `api.rango.com.br` volta a ser explícito, como no capítulo 15, e a saída `ServicoApiNome` passa a usar `api.service_name`:

```
        route53.ARecord(
            self,
            "ApiA",
            zone=zona,
            record_name="api",
            target=route53.RecordTarget.from_alias(targets.LoadBalancerTarget(alb)),
        )
```

Os workers continuam com o deploy rolling. Eles não recebem tráfego HTTP, então não há o que testar antes nem para onde devolver.

Antes do commit, um `cdk synth` local devolve o lookup da zona ao `cdk.context.json`, apagado no fim do capítulo 17, e esse arquivo entra no commit. Sem ele, o pipeline usa a lookup role do bootstrap a cada execução. Funciona, mas o template passa a depender do que a conta responde naquele momento.

O environment no GitHub

O `sub` da role de deploy fala de um environment chamado `producao`. No GitHub, um *environment* é um alvo de deploy com regras próprias. O do Rangô nasce em Settings → Environments → New environment, com o nome `producao`, e recebe duas regras:

- **Required reviewers**: o seu usuário. O job que usar esse environment para e espera alguém aprovar.
- **Deployment branches and tags**: "Selected branches and tags", com a regra `main`.

A segunda regra é a que fecha a porta. Sem ela, o environment é uma catraca com placa de "proibido passar" e nenhuma trava. O `sub` diz qual environment o job declarou, e não qual branch o disparou. Sem a restrição, alguém com permissão de push cria uma branch, escreve um workflow com `environment: producao` e recebe as credenciais de deploy. Com a restrição, o GitHub recusa o job antes de emitir o token.

> **Armadilha:** Em repositórios privados, as regras de proteção de environment dependem do plano do GitHub. Confira se as suas aparecem na tela antes de contar com elas, porque o `sub` com environment sem a regra de branch é quase tão frouxo quanto um curinga.

O ID da conta também fica numa variável do repositório. Ele dispensa sigilo (sozinho, o ID da conta abre tantas portas quanto o número do seu sapato), e assim o ARN das roles não fica repetido no workflow:

```
gh variable set AWS_ACCOUNT_ID --body "123456789012"
```

O workflow

O `.github/workflows/ci.yaml` do capítulo 6 tinha dois jobs: `testes` e `imagem`. O primeiro continua igual. O segundo passa a enviar a imagem, e entra um terceiro, `deploy`:

```
# .github/workflows/ci.yaml
name: ci
```

```yaml
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
    inputs:
      versao:
        description: "Hash completo de um commit cuja imagem já está no ECR"
        required: true

permissions:
  contents: read

env:
  VERSAO: ${{ inputs.versao || github.sha }}
  AWS_REGION: us-east-1

jobs:
  testes:
    # igual ao capítulo 6

  imagem:
    needs: testes
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-24.04-arm
    permissions:
      contents: read
      id-token: write
    steps:
      - uses: actions/checkout@v5
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: arn:aws:iam::${{ vars.AWS_ACCOUNT_ID }}:role/rango-github-
imagem
          role-session-name: imagem-${{ github.run_id }}
          aws-region: ${{ env.AWS_REGION }}
      - id: ecr
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build e push, se a tag ainda não existe
        env:
          REPO: ${{ steps.ecr.outputs.registry }}/rango-api
        run: |
          if aws ecr describe-images --repository-name rango-api \
               --image-ids imageTag="$VERSAO" > /dev/null 2>&1; then
            echo "A imagem $VERSAO já está no ECR"
            exit 0
          fi
          docker build -t "$REPO:$VERSAO" backend
          docker push "$REPO:$VERSAO"

  deploy:
    needs: imagem
    runs-on: ubuntu-24.04
    environment: producao
    concurrency: producao
    permissions:
      contents: read
      id-token: write
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
```

```
          python-version: "3.12"
      - uses: actions/setup-node@v5
        with:
          node-version: 22
      - run: npm install -g aws-cdk@2
      - run: pip install -r rango-infra/requirements.txt
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: arn:aws:iam::${{ vars.AWS_ACCOUNT_ID }}:role/rango-github-
deploy
          role-session-name: deploy-${{ github.run_id }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Infraestrutura e API
        working-directory: rango-infra
        run: |
          cdk deploy RangoRegistro RangoProducao \
            --require-approval never -c versao_api="$VERSAO"

      - name: Front
        if: github.event_name == 'push'
        working-directory: frontend
        run: |
          saida() {
            aws cloudformation describe-stacks --stack-name RangoProducao \
              --query "Stacks[0].Outputs[?OutputKey=='$1'].OutputValue" --output text
          }
          npm ci
          npm run build
          FRONT=$(saida FrontBucket)
          aws s3 sync dist/ "s3://$FRONT/" \
            --exclude index.html \
            --cache-control "public, max-age=31536000, immutable"
          aws s3 cp dist/index.html "s3://$FRONT/index.html" \
            --cache-control "no-cache"
          aws cloudfront create-invalidation \
            --distribution-id "$(saida DistribuicaoId)" --paths "/index.html"
```

Vamos por partes, como diria Jack, o Estripador, porque cada linha tem um motivo.

O `id-token: write` autoriza o job a pedir o JWT ao GitHub e nada mais; quem decide o acesso é a trust policy. O job `testes` não pede essa permissão, então um teste comprometido nem consegue tentar. Teste mal-intencionado é o pior tipo de teste, logo depois do teste que passa sempre.

O `VERSAO` é o hash completo do commit, que o capítulo 6 já usava como tag; as imagens de hash curto do capítulo 17 convivem com ele. Num disparo manual, o `inputs.versao` toma o lugar do hash, e isso vai servir para voltar versões.

O job `imagem` roda no runner `ubuntu-24.04-arm`, disponível em repositórios privados desde janeiro de 2026, com duas vCPUs e consumindo os minutos do plano. Na máquina arm64, um `docker build` comum gera a imagem do Graviton, sem `buildx` nem emulação. A checagem antes do build existe por causa das tags imutáveis: num re-run do mesmo commit, o push falharia com `ImageTagAlreadyExistsException`, e a imagem que já está lá é, por definição, a mesma.

O `environment: producao` muda o `sub` do token e faz o GitHub pedir aprovação. O `concurrency: producao` enfileira deploys: se chega um terceiro commit, o GitHub descarta o que esperava e fica com o mais novo. Na fila do pipeline, quem chega por último fura a fila, e todo mundo acha justo.

A `configure-aws-credentials` está na versão 6, que roda em Node 24; exemplos com `@v1`, `@v2` ou `@v4` estão defasados (tutorial de internet envelhece mais rápido que banana na fruteira), e runners próprios precisam do agente 2.327.1 ou mais novo. Ela pede o token, chama o STS e exporta as credenciais como variáveis de ambiente, onde a CLI e o CDK as encontram.

O `cdk deploy` recebe as stacks pelo nome em vez de `--all`, que incluiria a `RangoPipeline`: um commit que ampliasse a `role_deploy` seria aplicado pela própria `role_deploy`, como funcionário assinando o próprio aumento. O `--require-approval never` desliga a pergunta sobre mudanças de IAM, que travaria o job; a aprovação humana já aconteceu no environment.

O passo do front é o do capítulo 14 com as saídas da stack, e só roda em push, para que voltar a API não mexa no front. As primeiras 1.000 invalidações de caminho por mês são gratuitas, somando todas as distribuições da conta, e `/*` conta como um caminho (us-east-1, outubro de 2026; confira a página de preços). Um caminho por deploy fica longe disso, a não ser que o Bruno descubra que pode pedir uma cor nova de botão a cada hora.

> **Dica:** Os exemplos acima usam tags como `@v6` para facilitar a leitura. A documentação do GitHub sobre OIDC na AWS fixa as actions pelo hash do commit (`uses: aws-actions/configure-aws-credentials@<hash de 40 caracteres>`), e você deve fazer o mesmo, inclusive no `ci.yaml` do capítulo 6. Uma tag pode ser movida para outro commit por quem controla o repositório da action; um hash não. O Dependabot sabe atualizar actions fixadas por hash e abre pull requests quando sai versão nova.

Por que não as actions de deploy do ECS?

A AWS mantém duas actions para o ECS: `amazon-ecs-render-task-definition`, que troca a imagem num JSON de task definition, e `amazon-ecs-deploy-task-definition`, hoje na versão 2, que registra a revisão nova e atualiza o serviço. Ela aceita `task-definition`, `service`, `cluster` e `wait-for-service-stability: true`, e o README mostra o fluxo completo: credenciais, login no ECR, build e push com a tag `${{ github.sha }}`, render e deploy.

É um bom caminho quando infraestrutura e aplicação têm donos separados, cada um com opiniões muito firmes sobre o outro. No Rangô, a task definition pertence ao CDK. Revisões registradas por fora seriam o drift do capítulo 15: no próximo `cdk deploy`, a tag antiga voltaria para produção sem ninguém pedir, como aquele bug que você jura que já corrigiu.

O primeiro deploy pelo pipeline

Com tudo no commit (a stack nova, as mudanças na produção, o `cdk.context.json` e o workflow) e enviado para a `main`, o acompanhamento pode ser pelo terminal, em vez da aba Actions:

```
gh run watch
```

Os testes passam, a imagem é construída e enviada, e o job `deploy` para com o status "Waiting". O GitHub manda um e-mail pedindo a revisão. Pela primeira vez na história, um e-mail automático do GitHub vai ser lido com atenção. A aprovação é feita pela página do run, que também abre a partir da linha de comando:

```
gh run view --web
```

Este primeiro deploy cria a produção do zero, com o banco restaurado do snapshot, e leva os 15 a 20 minutos de sempre. Repare no que aparece no log do passo de infraestrutura: as credenciais usadas são da role `rango-github-deploy`, e a CLI do CDK anuncia que vai assumir a `cdk-hnb659fds-deploy-role`.

Como no capítulo 17, a primeira rotação do segredo corre em paralelo com a subida das tasks. A saída é esperar ela terminar e forçar tasks novas, agora com a permissão que a `role_deploy` tem no cluster. Isso nos leva à pendência do capítulo 17: a task que vive mais que duas rotações perde o acesso ao banco, e o `--force-new-deployment` semanal precisava de alguém para lembrar. Um segundo workflow, `.github/workflows/renovar-tasks.yaml`, lembra por você, e não adia, ao contrário do lembrete no celular que você já empurrou dezessete vezes:

```yaml
# .github/workflows/renovar-tasks.yaml
name: renovar-tasks

on:
  schedule:
    - cron: "0 9 * * 1"
  workflow_dispatch:

permissions:
  contents: read
  id-token: write

jobs:
```

```
  renovar:
    runs-on: ubuntu-24.04
    environment: producao
    concurrency: producao
    steps:
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: arn:aws:iam::${{ vars.AWS_ACCOUNT_ID }}:role/rango-github-
deploy
          role-session-name: renovar-${{ github.run_id }}
          aws-region: us-east-1
      - run: |
          for servico in $(aws ecs list-services --cluster rango-producao \
                           --query "serviceArns[]" --output text); do
            aws ecs update-service --cluster rango-producao --service "$servico" \
              --force-new-deployment > /dev/null
            echo "Deploy forçado: $servico"
          done
```

Toda segunda-feira às 9h em UTC (6h em Brasília), as tasks são trocadas e leem o segredo atual, num horário em que quase nenhum cliente pede comida e nenhum humano está acordado para reclamar. Para isso, a `role_deploy` ganha um statement com `ecs:ListServices` e `resources=["*"]`, aplicado com um novo deploy manual da `RangoPipeline`. Este job também pede aprovação; se incomodar, a saída é dar a ele um environment e uma role próprios, só com essas duas ações.

A correção do Bruno, pelo celular

De volta à sexta-feira. A correção da taxa vai num pull request:

```
git switch -c corrige-taxa-sertao
# edite a taxa da Sabor do Sertão em backend/app/main.py
git commit -am "Corrige taxa de entrega da Sabor do Sertao"
git push -u origin corrige-taxa-sertao
gh pr create --fill
```

O job `testes` roda no pull request. Os jobs `imagem` e `deploy` aparecem como ignorados, por causa do `if` e do `needs`. A outra pessoa da equipe revisa, aprova e faz o merge pelo navegador. O push na `main` dispara o pipeline. Você recebe o e-mail de aprovação antes do bolo, abre o link no celular e aprova, e ainda sobra tempo para garantir o último pedaço de bolo.

Enquanto o deploy roda, a regra de teste já mostra a versão verde. Qualquer um com o `curl` pode conferir antes da troca:

```
curl -s -H "X-Rango-Teste: verde" https://api.rango.com.br/restaurantes \
  | python3 -m json.tool | grep -A1 Sertao
```

Depois da troca, o estado do deploy aparece no ECS:

```
aws ecs list-service-deployments --cluster rango-producao \
  --service "$(saida ServicoApiNome)" \
  --query "serviceDeployments[0].[status,targetServiceRevisionArn]"
```

Durante os dez minutos de bake time, o status fica `IN_PROGRESS` e o serviço tem quatro tasks. Depois, as azuis são drenadas, o status vira `SUCCESSFUL` e o `cdk deploy` termina. O job inteiro leva uns 15 minutos, a maior parte esperando o bake time. Ninguém abriu um terminal. O notebook passou a noite fechado, que é onde notebook deve ficar numa festa de aniversário.

Quando a versão nova quebra

Agora o teste que interessa: uma versão que sobe, passa no `/health` e devolve erro. Numa branch de teste, alguém troca o nome de uma coluna na consulta de `/restaurantes`, para que o Postgres responda `column "taxa_entrga" does not exist`, faz o merge e aprova o deploy. É o tipo de erro de digitação que o corretor do celular conserta e o Postgres não perdoa.

Durante o bake time, entra tráfego. O script do k6 do capítulo 13 serve, ou um laço simples:

```
for i in $(seq 1 300); do
  curl -s -o /dev/null -w "%{http_code}\n" https://api.rango.com.br/restaurantes
  sleep 0.5
done | sort | uniq -c
```

```
    169 200
    131 500
```

Em um ou dois minutos, o alarme `ErrosApi` passa para `ALARM`. O ECS devolve a regra de produção ao azul, e os 500 param. Os sucessos do resultado são as respostas que chegaram depois da volta, já servidas pelas tasks azuis. O deploy termina como `ROLLBACK_SUCCESSFUL`, o CloudFormation reverte a stack para a revisão anterior da task definition, o `cdk deploy` sai com erro e o run fica vermelho no GitHub.

O vermelho é o aviso. Na versão do capítulo 17, ninguém teria percebido até um cliente reclamar, e cliente reclamando tem um alcance que nenhum alarme do CloudWatch consegue igualar.

Para os casos que o alarme não pega, voltar uma versão é rodar o mesmo pipeline com uma tag antiga:

```
gh workflow run ci -f versao=3f9c2b1e8d7a6c5b4a39281706f5e4d3c2b1a098
```

Os testes rodam sobre a `main`, o job `imagem` encontra a tag no ECR e não constrói nada, e o `deploy` aplica a infraestrutura atual com a imagem antiga, em blue/green, depois da aprovação. O hash vem do histórico de runs ou do `git log`. As 30 imagens que o lifecycle do ECR guarda são as suas versões disponíveis para voltar.

Voltar a imagem não desfaz uma migration de banco. Se a versão nova renomeou uma coluna, a antiga vai procurar a coluna com o nome antigo e falhar. É por isso que o capítulo 6 insistiu no expand/contract: cada deploy precisa funcionar com o esquema do deploy anterior e do seguinte. Máquina do tempo para código existe; para banco de dados, ainda não inventaram.

Desligar sem deixar o pipeline religar

A produção é destruída a partir do notebook, como nos capítulos anteriores:

```
cdk destroy RangoProducao -c versao_api=nenhuma
```

O banco sai com snapshot final; o nome dele vai para o `app.py`, num commit, quando a produção for subir de novo. A `RangoRegistro` e a `RangoPipeline` ficam. O provedor OIDC e as roles não custam nada, uma das raras frases sobre a AWS que dispensam asterisco.

Há um detalhe novo. Com a produção destruída, qualquer push na `main` recria tudo, com banco e NAT de endpoints cobrando por hora. Um commit inocente que corrige uma vírgula no README vira uma produção inteira ligada. Por isso os workflows ficam desligados enquanto você estuda outra coisa:

```
gh workflow disable ci
gh workflow disable renovar-tasks
```

E voltam com `gh workflow enable` quando a produção precisar voltar.

E o CodePipeline?

O AWS CodePipeline é o orquestrador de pipelines da AWS. Ele encadeia estágios (origem, build, aprovação, deploy) e delega o trabalho a outros serviços: o AWS CodeBuild roda os comandos, como o runner do GitHub, e o deploy pode ir direto para o ECS, para o CloudFormation ou para um bucket. O código pode vir do GitHub, por meio de uma conexão autorizada uma vez no console.

Ele tem dois tipos de pipeline com cobranças diferentes (us-east-1, outubro de 2026; confira a página oficial). O V1 custa US$ 1,00 por mês por pipeline ativo, sendo ativo o pipeline com mais de 30 dias e pelo menos uma mudança no mês; o primeiro é gratuito

no free tier. O V2 cobra US$ 0,002 por minuto de execução de ação, arredondado para cima, com 100 minutos gratuitos por mês, e a aprovação manual não conta, o que é ótimo, porque humanos aprovam devagar. Nos dois, CodeBuild, S3 e outros serviços usados são cobrados à parte.

A vantagem é ficar tudo na AWS, para quem gosta de morar, trabalhar e fazer compras no mesmo condomínio: só roles IAM, sem provedor OIDC, e o CodeBuild pode rodar dentro da VPC, alcançando um banco privado para testes de integração. O CDK tem um construct para isso, o CDK Pipelines, em que o pipeline se atualiza quando a definição dele muda. Para o Rangô, com pull requests e revisões no GitHub, seria um segundo lugar para olhar quando algo der errado. Dois painéis de CI são como dois controles remotos para a mesma TV: um deles sempre está no lugar errado.

Armadilhas e casos reais

> **Armadilha:** Trust policy sem `sub`, ou com curinga. É o erro mais grave deste capítulo, porque não dá sinal nenhum: tudo funciona, e qualquer repositório do GitHub consegue assumir a role. Revise a trust policy de cada role federada com o mesmo cuidado de uma bucket policy pública. O IAM Access Analyzer aponta roles que podem ser assumidas de fora da conta.
>
> **Armadilha:** Dois provedores para a mesma URL. Só existe um provedor OIDC por URL em cada conta. Se alguém já criou o do GitHub pelo console, seguindo um tutorial (provavelmente de madrugada e sem anotar em lugar nenhum), o deploy da `RangoPipeline` falha com `EntityAlreadyExists`. Apague o provedor antigo, se nenhuma role depender dele, ou importe-o com `iam.OpenIdConnectProvider.from_open_id_connect_provider_arn`.
>
> **Armadilha:** `cdk deploy` sem a versão. Sem o `-c versao_api`, a tag chega como `None` e o `from_ecr_repository` usa `latest`, que não existe num repositório de tags imutáveis enviado só com hashes. As tasks novas não conseguem baixar a imagem, o circuit breaker volta atrás e você perde 15 minutos para descobrir o motivo, e mais uns cinco para admitir o motivo para si mesmo. Por isso os comandos manuais deste capítulo passam um valor qualquer; nenhum deles faz deploy da `RangoProducao`.
>
> **Armadilha:** Mexer nas regras do ALB no CDK depois do primeiro deploy. O ECS altera os pesos das regras a cada deploy, e depois de um número ímpar de deploys o tráfego está no verde. Paridade da contagem de deploys: mais um conceito que você não esperava ter de acompanhar. O CloudFormation não sabe disso. Se você mudar a definição de uma regra no código, o update reescreve os pesos com os valores do

template, 100 para o azul, e o tráfego vai para um target group que pode estar vazio. Se precisar mudar uma regra, faça isso numa janela em que possa conferir o resultado com o `curl` logo em seguida.

Armadilha: O bake time ocupa o runner. O `cdk deploy` só termina depois dele, e em repositórios privados são minutos do plano. Com deploys muito frequentes, reduza o bake time ou use a estratégia linear, com degraus de 3% a 100% e espera de 0 a 1.440 minutos entre eles.

Na vida real: Em março de 2025, a action `tj-actions/changed-files`, usada por milhares de repositórios, foi comprometida. Os atacantes apontaram as tags existentes, inclusive as de versão, para um commit malicioso que imprimia os segredos do runner no log do workflow. Em repositórios públicos, esses logs podiam ser lidos por qualquer pessoa. Quem fixava a action por hash de commit não foi afetado, e quem autenticava na AWS por OIDC não tinha access key para perder: o token vazado expirava em minutos. Foi registrada como CVE-2025-30066.

Na vida real: O deploy agora depende do GitHub Actions estar no ar. Mantenha documentado o caminho manual do capítulo 17, com o profile `rango-admin`, para emergências, e trate cada uso dele como incidente com postmortem.

Resumo

- O GitHub Actions autentica na AWS por OIDC: o job pede um JWT, o STS confere assinatura, `aud` e `sub`, e devolve credenciais temporárias. Nenhuma access key fica guardada.
- A trust policy precisa sempre do `sub` com valor exato. Um environment no GitHub só protege com a regra de branch e a aprovação configuradas.
- O Rangô tem duas roles: `rango-github-imagem` (push na `main`, só o repositório `rango-api`) e `rango-github-deploy` (environment `producao`, assume as roles do bootstrap do CDK, lê saídas da stack, força deploys).
- O workflow testa, constrói a imagem uma vez num runner ARM, envia ao ECR com o hash completo do commit, roda `cdk deploy` com esse hash e envia o front com invalidação só do `/index.html`.
- A task definition continua com o CDK. Atualizar o serviço por fora causaria drift.
- A API usa o blue/green nativo do ECS: regra de teste por cabeçalho, troca de uma vez, bake time de dez minutos e rollback por alarme de 5xx. Os workers seguem com rolling.

- Voltar uma versão é rodar o workflow com uma tag antiga. Migrations precisam de expand/contract.
- O CodePipeline faz o mesmo de dentro da AWS e vale a pena quando o código ou as regras da empresa já estão lá.

Três vezes por dia

Com o pipeline no ar, o Bruno pega gosto por pedir coisas. Cliente que descobre que o deploy ficou fácil é criança que descobriu onde fica o pote de biscoitos. A próxima: um resumo de vendas para cada restaurante, enviado às 11h, às 15h e às 23h, com os pedidos do período, o ticket médio e os pratos mais pedidos. A lógica é uma consulta ao Postgres e um e-mail, coisa de quarenta linhas de Python. Todo pedido de cliente vem com a frase "é rapidinho", e este, por incrível que pareça, é.

Onde ela roda? O caminho mais curto com o que você tem é um terceiro worker no ECS, com um laço que dorme até a próxima hora cheia. Uma task de 0,25 vCPU ligada o mês inteiro para trabalhar uns poucos minutos por dia, como um funcionário contratado em tempo integral para apertar um botão três vezes ao dia. O Rangô passaria a pagar 24 horas por dia por uma rotina que roda três vezes por dia, e o mesmo raciocínio vale para os próximos pedidos do Bruno: o relatório semanal, a limpeza de carrinhos abandonados, o webhook do gateway de pagamento que chega uma vez por pedido.

No próximo capítulo, o AWS Lambda roda código só quando algo acontece e cobra pelos milissegundos de execução. O capítulo mostra como ele executa uma função por dentro, o que é o cold start que o capítulo 4 apresentou, os limites que surpreendem, e quando o API Gateway na frente dele substitui um servidor inteiro. E também quando nada disso compensa.

Referências

- GitHub, configurar OIDC na AWS (provedor, `aud`, formatos de `sub`, `id-token: write`, actions fixadas por hash): https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-aws
- Releases da action `configure-aws-credentials`: https://github.com/aws-actions/configure-aws-credentials/releases
- Action `amazon-ecs-deploy-task-definition`: https://github.com/aws-actions/amazon-ecs-deploy-task-definition
- Blue/green nativo no Amazon ECS (bake time, lifecycle hooks, rollback): https://docs.aws.amazon.com/AmazonECS/latest/developerguide/deployment-type-

blue-green.html

- Estágios do deploy e lifecycle hooks, referência da API do ECS: https://docs.aws.eu/AmazonECS/latest/APIReference/API_DeploymentLifecycleHook.html
- Deploy linear no ECS: https://docs.aws.eu/AmazonECS/latest/developerguide/deployment-type-linear.html
- Anúncio dos deploys linear e canary no ECS (outubro de 2025): https://aws.amazon.com/about-aws/whats-new/2025/10/amazon-ecs-built-in-linear-canary-deployments
- AWS CDK, classe `AlternateTarget` do `aws_ecs`: https://docs.aws.amazon.com/cdk/api/v2/docs/aws-cdk-lib.aws_ecs.AlternateTarget.html
- AWS CDK, `DeploymentStrategy` em Python: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_ecs/DeploymentStrategy.html
- CloudFront, cobrança de invalidações: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PayingForInvalidation.html
- Preços do AWS CodePipeline: https://aws.amazon.com/codepipeline/pricing
- `cdk deploy` no GitHub Actions com OIDC e roles do bootstrap (Classmethod): https://dev.classmethod.jp/articles/github-actions-oidc-aws-cdk-deploy/
- Permissões para passos fora do CDK no pipeline (DEV Community): https://dev.to/simonireilly/secure-aws-cdk-deployments-with-github-actions-3jfk
- Runners ARM padrão em repositórios privados (changelog do GitHub, janeiro de 2026): https://github.blog/changelog/2026-01-29-arm64-standard-runners-are-now-available-in-private-repositories
- Comprometimento da action `tj-actions/changed-files`, CVE-2025-30066: https://github.com/advisories/GHSA-mrrh-fwg8-r2c3

CAPÍTULO 19

Serverless com Lambda e API Gateway

No capítulo anterior, o deploy do Rangô saiu das suas mãos. Um push na `main` dispara o GitHub Actions, que se autentica na AWS por OIDC, constrói a imagem num runner ARM, envia ao ECR e roda o `cdk deploy`, com blue/green no ECS e rollback automático quando o alarme de erros 5xx dispara. No fim, a stack `RangoProducao` foi destruída, com snapshot do banco, e os workflows ficaram desabilitados. E o Bruno, animado com a facilidade, já tinha um pedido novo. Ele é o tipo de cliente que transforma cada melhoria do sistema em três pedidos novos, e é por isso que o Rangô ainda existe.

Seis minutos de trabalho, 24 horas de conta

O pedido é simples: um resumo de vendas para cada restaurante às 11h, às 15h e às 23h. Quantos pedidos entraram no período, quanto faturaram e qual foi o ticket médio. Os pratos mais pedidos ficam para depois, porque os itens ainda não estão numa tabela. Quarenta linhas de Python, uma consulta ao Postgres e um e-mail. Coisa de fim de tarde, se ninguém marcar reunião.

Com o que você tem, o caminho óbvio é um terceiro worker no ECS. Um laço que calcula quanto falta até a próxima hora cheia, dorme, acorda, gera o resumo e volta a dormir. Funciona. Agora, a conta. A consulta e o envio levam uns dois segundos por execução. Três execuções por dia dão seis segundos de trabalho. A task de 0,25 vCPU e 512 MB fica ligada 86.400 segundos por dia. Você estaria pagando por 99,99% de sono. Nem gato de apartamento tem uma taxa de ocupação tão baixa.

O valor é pequeno: o capítulo 17 estimou uns US$ 14 por mês para os dois workers, então uma task dessas fica perto de US$ 7. O problema aparece quando o padrão se repete. O Bruno já quer um relatório semanal, uma limpeza dos carrinhos abandonados e, o mais urgente, um endereço para receber os avisos do gateway de pagamento.

O gateway acaba de liberar o PIX, que funciona diferente do cartão. O Rangô gera o código, o cliente paga no aplicativo do banco quando quiser, e minutos depois o gateway faz um POST num endereço do Rangô dizendo "a cobrança tal foi aprovada". Esse POST

chama-se *webhook*. É um telefonema às avessas: em vez de você ligar a cada minuto perguntando se o pagamento caiu, o gateway liga quando cai. Ele chega em rajadas no almoço e no jantar, e quase nunca de madrugada.

O webhook poderia ser mais uma rota do FastAPI, e seria uma escolha razoável. Mas repare no padrão dos pedidos do Bruno: código pequeno, que roda de vez em quando, disparado por um evento. Manter algo ligado o tempo todo para isso é alugar um salão de festas para um jantar de três pessoas.

Código que só existe quando é chamado

O capítulo 4 apresentou *serverless* como conceito: você entrega o código, o provedor executa quando algo acontece e cobra pelo tempo de execução. O AWS Lambda é a implementação da AWS. Existe servidor, claro; você só deixa de escolhê-lo e de pagar por ele quando nada está rodando.

O que você entrega é uma função Python com uma assinatura fixa:

```
def handler(event, context):
    return {"ok": True}
```

O `event` é um dicionário com os dados de quem chamou. Se foi uma regra de agendamento, vem o que você configurou nela. Se foi uma requisição HTTP, vêm o método, o caminho, os cabeçalhos e o corpo. O `context` traz metadados da execução, como o ID da requisição e quanto tempo resta antes do limite, uma ampulheta que muita reunião deveria ter. Pense numa view do Django em que o `request` virou um dicionário e o framework foi substituído pela AWS.

O ambiente de execução por dentro

Quando a primeira chamada chega, a Lambda cria um **ambiente de execução**: uma micro máquina virtual (Firecracker, a mesma tecnologia por trás do Fargate) com o runtime Python, o seu código e a memória que você configurou. A fase **INIT** acontece uma vez: a Lambda inicia o interpretador e importa o seu módulo, e tudo o que está fora do `handler` roda aqui (imports, clientes do boto3, leitura de segredos). A fase **INVOKE** acontece a cada chamada. A fase **SHUTDOWN** vem quando a Lambda decide descartar o ambiente ocioso, num momento que você não controla, como o garçom que recolhe o prato quando ainda sobrava uma batata.

Entre uma chamada e outra, o ambiente fica congelado. Se outra chamada chega, a Lambda descongela e chama o `handler` de novo, sem repetir o INIT, como a pizza da noite anterior, que só precisa de um minuto no micro-ondas. As variáveis globais, a

conexão aberta com o banco e os arquivos em `/tmp` continuam lá. A chamada que precisa de um ambiente novo é o *cold start*, que o capítulo 4 prometeu explicar por dentro: o INIT somado à primeira chamada. A que encontra um ambiente pronto é o *warm start*.

Um detalhe muda o modelo mental: **cada ambiente atende uma chamada por vez**. Se dez requisições chegam juntas, a Lambda precisa de dez ambientes. Lembra o atendente de padaria que só faz um pão na chapa por vez, com a diferença de que a Lambda contrata outro atendente na hora. O número de ambientes ocupados ao mesmo tempo é a **concorrência** da função. Um processo Uvicorn atende dezenas de requisições ao mesmo tempo com `async`; na Lambda, a concorrência vem de multiplicar processos Python inteiros.

O que pesa no cold start

O tempo do INIT depende do tamanho do pacote, dos imports (`import pandas` sozinho custa centenas de milissegundos; o pandas é o convidado que chega com três malas para passar o fim de semana), do código de inicialização e da memória. Na Lambda, você escolhe a memória, entre 128 MB e 10.240 MB, e a CPU vem junto, em proporção: com 1.769 MB, a função recebe o equivalente a uma vCPU. Uma função com imports pesados pode ficar mais rápida e até mais barata com mais memória, porque o tempo cai mais do que o preço por milissegundo sobe.

Muito texto antigo recomenda evitar VPC por causa de cold starts de vários segundos. A AWS mudou esse mecanismo em 2019, e hoje as interfaces de rede são criadas quando a função é configurada, não a cada cold start. O conselho antigo sobrevive em respostas de fórum como mito urbano, o jacaré no esgoto da computação em nuvem.

Até julho de 2025, o INIT de funções em zip com runtime gerenciado não era cobrado. Desde 1º de agosto de 2025, ele é cobrado em todas as configurações, pelo preço normal de duração, e aparece somado no `Billed Duration` dos logs. Para quem inicializava coisas pesadas fora do handler contando com o INIT grátis, a conta mudou. Almoço grátis, como sempre, tinha data de validade.

Quando o cold start incomoda, existem duas saídas pagas. A *provisioned concurrency* mantém um número fixo de ambientes já inicializados, prontos para atender, e você paga por eles mesmo parados; é a recomendação da documentação quando a latência precisa ser previsível. O *SnapStart*, disponível para Python 3.12 em diante desde novembro de 2024, tira uma fotografia da memória do ambiente depois do INIT e restaura a partir dela nos cold starts seguintes. Ele não funciona com imagens de container, e o código precisa tomar cuidado com o que fica na fotografia: uma conexão aberta com o banco, por exemplo, não sobrevive à restauração. Uma foto sua ao telefone não mantém a ligação ativa.

Os limites que você precisa conhecer

A Lambda tem limites fixos que moldam o que cabe nela, como a mala de mão na companhia aérea: o que não cabe vai despachado para outro serviço. Os valores abaixo são da documentação de cotas, consultada em outubro de 2026; confira a página oficial antes de projetar algo perto deles.

Limite	Valor
Tempo máximo de execução	900 segundos (15 minutos)
Memória	128 MB a 10.240 MB
Payload de chamada síncrona	6 MB na requisição e 6 MB na resposta
Payload de chamada assíncrona	1 MB
Pacote zip	50 MB compactado; 250 MB descompactado, somando as layers
Imagem de container	10 GB
Layers por função	5
Espaço em `/tmp`	512 MB a 10.240 MB
Variáveis de ambiente	4 KB no total
Concorrência por região	1.000 por padrão (contas novas começam com menos)

Os 15 minutos deixam de fora processamentos longos, que ficam melhor numa task do ECS. Função que precisa de mais de 15 minutos está pedindo para virar outra coisa quando crescer. Os 4 KB de variáveis de ambiente reforçam o caminho do capítulo 16: o Parameter Store e o Secrets Manager guardam os valores, e a função recebe só os nomes.

O limite de 1.000 de concorrência vale para a região inteira, somando todas as funções, e cada função ganha no máximo 1.000 ambientes novos a cada 10 segundos. A documentação aponta um descompasso: o API Gateway aceita por padrão 10.000 requisições por segundo, a Lambda roda 1.000 ambientes. Se cada requisição leva um segundo, o excesso volta como erro de *throttling*. A solução é pedir aumento da cota antes de precisar, como o capítulo 4 recomendou pré-escalar antes do programa de TV. Pedir aumento de cota no dia do pico é como reservar mesa às 20h do Dia dos Namorados.

Runtimes e layers

Em outubro de 2026, os runtimes Python de uso geral são `python3.14`, `python3.13` e `python3.12`, todos sobre Amazon Linux 2023 e disponíveis em x86_64 e arm64 (Graviton, cerca de 20% mais barato por GB-segundo). O `python3.12` tem descontinuação marcada para 31 de outubro de 2028, uma antecedência que, no ritmo da área, equivale a planejar a aposentadoria. O Rangô usa essa versão, a mesma da imagem do container desde o capítulo 3. Fidelidade de versão é raridade em software e merece registro.

Uma *layer* é um zip extra que a Lambda descompacta em `/opt`. Se ele tiver uma pasta `python/`, ela entra no `sys.path`. O uso clássico é separar dependências do código: a layer leva o `psycopg` e o `boto3`, o pacote da função leva só os seus `.py`, e duas funções podem compartilhar a mesma layer. São no máximo cinco por função, dentro dos 250 MB descompactados. Cada publicação gera uma versão imutável, e layers não resolvem conflito de dependências: se duas trazem a mesma biblioteca, vence a primeira na ordem, sem aviso. É o inferno de dependências servido em camadas, como uma lasanha em que cada andar traz o próprio molho.

O runtime já vem com um `boto3`, mas a versão muda quando a AWS atualiza o runtime. A documentação recomenda empacotar o seu, com versão conhecida.

As formas de chamar uma função

Uma função Lambda não escuta nenhuma porta. Ela não atende o interfone; alguém precisa subir e bater. Na **chamada síncrona**, quem chama espera a resposta; é o que o API Gateway faz. Na **chamada assíncrona**, quem chama entrega o evento e vai embora, a Lambda executa quando puder e, se a função falhar, tenta de novo duas vezes por padrão; é o que acontece quando o Amazon EventBridge dispara uma função num horário. No **mapeamento de origem de eventos**, a própria Lambda lê uma fila do SQS e chama a função com lotes de mensagens, modelo que o capítulo 21 usa.

A AWS lançou variações recentes, como as Lambda Managed Instances e as durable functions, e passou a chamar as funções clássicas de *Event Functions*. A relação da AWS com nomes de produto é complicada e renderia um livro à parte. Este capítulo fica com elas.

A porta HTTP da função

Para o webhook do PIX, o gateway de pagamento precisa de uma URL HTTPS. A Lambda não tem porta, então alguma coisa precisa receber a requisição HTTP, terminar o TLS, transformar tudo num `event` e chamar a função. Esse é o papel do Amazon API

Gateway, o recepcionista que atende a porta, confere quem chegou e só então chama a função na sala dos fundos.

O API Gateway faz para a Lambda o que o ALB faz para as tasks do ECS, com autenticação, limite de requisições e rotas declaradas. Ele cobra por requisição, sem custo fixo por hora, enquanto o ALB do capítulo 13 custa uns US$ 23 por mês parado, como ingresso de show comprado para um show a que você não foi. As *function URLs*, que dão à função um endereço HTTPS sem API Gateway, servem para protótipos, mas não oferecem rotas nem limite de requisições.

HTTP API ou REST API

O API Gateway tem dois produtos para APIs HTTP comuns, com nomes que confundem: a REST API e a HTTP API. As duas recebem requisições REST. É como ter dois restaurantes, um chamado "Comida" e outro chamado "Restaurante", ambos servindo comida. A diferença está nos recursos e no preço.

	HTTP API	REST API
Preço (primeira faixa, us-east-1, out/2026)	US$ 1,00 por milhão de chamadas	US$ 3,50 por milhão de chamadas
Autorizadores	JWT, Lambda, IAM	Lambda, IAM, Cognito
Chaves de API e limite por cliente	Não	Sim
Validação da requisição	Não	Sim
Integração com AWS WAF	Não	Sim
Endpoint privado (só dentro da VPC)	Não	Sim
Cache de respostas	Não	Sim, cobrado por hora

A REST API compensa quando você vende acesso à API para terceiros, com chave e cota por cliente, ou precisa do WAF na frente (capítulo 23). Para o webhook, que tem um único cliente e valida a assinatura no código, a HTTP API basta e custa menos de um terço. Os preços são aproximados, de referência, da página oficial; contas criadas desde 15 de julho de 2025 usam os créditos do plano gratuito do capítulo 7.

A HTTP API tem dois limites fixos. A integração tem no máximo 30 segundos: se a função demorar mais, o cliente recebe erro, mesmo que a Lambda continue rodando, trabalhando com dedicação para alguém que já foi embora. E o payload vai até 10 MB. Um relatório pesado sob demanda precisa responder logo com um "estou processando", a versão HTTP do "seu pedido está sendo preparado", e entregar depois, no modelo assíncrono do capítulo 5.

O evento que chega na função

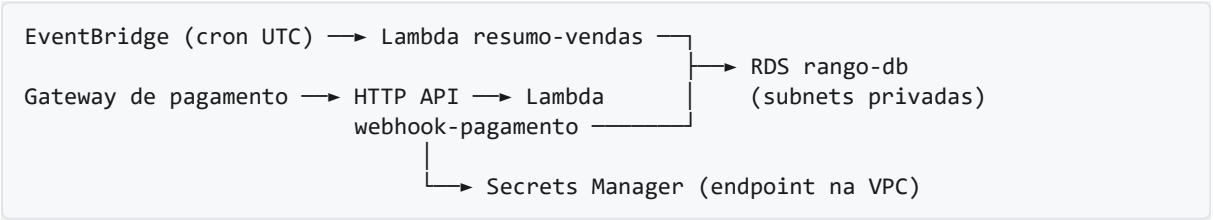
Quando a HTTP API chama a Lambda, ela usa o formato de payload 2.0. O `event` fica mais ou menos assim, resumido:

```
{
  "version": "2.0",
  "routeKey": "POST /webhooks/pagamento",
  "rawPath": "/webhooks/pagamento",
  "headers": {
    "content-type": "application/json",
    "x-assinatura": "sha256=4f1c..."
  },
  "requestContext": {
    "http": {"method": "POST", "sourceIp": "203.0.113.77"}
  },
  "body": "{\"pedido_id\": \"...\", \"status\": \"aprovada\"}",
  "isBase64Encoded": false
}
```

O corpo chega como texto cru, ou em base64 quando o `isBase64Encoded` vem `true`. A resposta da função é outro dicionário, com `statusCode`, `headers` e `body`. Na Lambda, até a resposta HTTP é um dicionário, e o Python nunca se sentiu tão em casa.

Na prática

O plano é colocar duas funções no Rangô. A `resumo-vendas` roda três vezes por dia, disparada por regras do EventBridge. A `webhook-pagamento` atende o POST do gateway, atrás de uma HTTP API. As duas leem o Postgres, então ficam dentro da VPC, nas subnets privadas, e compartilham uma layer com as dependências. O diagrama abaixo é o mais perto de arte que este livro vai chegar.



Pré-requisitos: o repositório do capítulo 18, a AWS CLI v2 com o profile `rango-dev`, o Node e o CDK, o Docker rodando (o CDK usa um container para montar a layer) e o `jq`, o canivete suíço do JSON na linha de comando, cuja sintaxe todo mundo consulta toda vez.

O ponto de partida

A stack `RangoProducao` foi destruída no fim do capítulo 18. Ficaram a `RangoRegistro`, com as imagens no ECR, a `RangoPipeline`, com as roles do GitHub, a zona, o certificado, o bucket de fotos e o snapshot final do banco. A lista do que sobrevive a cada capítulo já parece testamento. O nome do snapshot aparece com:

```
aws rds describe-db-snapshots --snapshot-type manual \
  --query "reverse(sort_by(DBSnapshots,&SnapshotCreateTime))[0].DBSnapshotIdentifier" \
  --output text
```

Esse nome vai no `snapshot_db` do `rango-infra/app.py`, como nos capítulos anteriores. O deploy vai acontecer pelo pipeline, no fim das mudanças.

O código das funções

As funções ficam numa pasta própria no backend:

```
backend/lambdas/
├── camada/
│   └── requirements.txt
└── rango_lambda/
    ├── banco.py
    ├── rds-ca.pem
    ├── resumo_vendas.py
    └── webhook_pagamento.py
```

O certificado da CA do RDS que o capítulo 17 baixou é copiado para lá:

```
mkdir -p backend/lambdas/camada backend/lambdas/rango_lambda
cp backend/rds-ca.pem backend/lambdas/rango_lambda/
```

O `backend/lambdas/camada/requirements.txt` tem as dependências da layer. As versões exatas devem ser fixadas com um `pip freeze`, como no capítulo 3; os intervalos abaixo são o ponto de partida:

```
psycopg[binary]~=3.2
boto3~=1.40
```

O `banco.py` concentra a conexão. Repare que quase tudo acontece no nível do módulo, ou seja, na fase INIT:

```
import json
import os

import boto3
import psycopg
from psycopg.rows import dict_row

_segredo = json.loads(
    boto3.client("secretsmanager").get_secret_value(
```

```
        SecretId=os.environ["SEGREDO_DB_ARN"]
    )["SecretString"]
)
_conexao = None


def conexao() -> psycopg.Connection:
    global _conexao
    if _conexao is None or _conexao.closed or _conexao.broken:
        _conexao = psycopg.connect(
            host=os.environ["DB_HOST"],
            dbname="rango",
            user=_segredo["username"],
            password=_segredo["password"],
            sslmode="verify-full",
            sslrootcert=os.path.join(os.environ["LAMBDA_TASK_ROOT"], "rds-ca.pem"),
            connect_timeout=5,
            autocommit=True,
            row_factory=dict_row,
        )
    return _conexao
```

O segredo é lido uma vez por ambiente, e a conexão aberta na primeira chamada é reaproveitada nas seguintes. Uma conexão por ambiente, sem pool, porque cada ambiente atende uma chamada por vez. Pool para uma chamada só seria piscina olímpica para um patinho de borracha. Se o Postgres derrubou a conexão durante o congelamento, o `broken` acusa e a próxima chamada reconecta. E a rotação do capítulo 16 não atrapalha: na estratégia multi-user, a senha anterior vale até a rotação seguinte, e nenhum ambiente vive 30 dias. O `LAMBDA_TASK_ROOT` é a pasta do código, `/var/task`.

Agora o `resumo_vendas.py`:

```
import json

from banco import conexao

CONSULTA = """
    SELECT restaurante,
           count(*) AS pedidos,
           sum(total) AS faturamento,
           round(avg(total), 2) AS ticket_medio
    FROM pedidos
    WHERE criado_em >= now() - %s * interval '1 hour'
      AND status <> 'recusado'
    GROUP BY restaurante
    ORDER BY restaurante
"""


def enviar_resumo(linha: dict, horas: int) -> None:
    # O envio de e-mail segue simulado, como no externos.py do capítulo 5.
    print(json.dumps({"evento": "resumo.enviado", "horas": horas, **linha},
default=str))


def handler(event, context):
    horas = int(event.get("horas", 8))
```

```
    linhas = conexao().execute(CONSULTA, (horas,)).fetchall()
    for linha in linhas:
        enviar_resumo(linha, horas)
    return {"restaurantes": len(linhas), "horas": horas}
```

O período chega no evento: às 11h, 12 horas desde as 23h da véspera; às 15h, 4 horas; às 23h, 8 horas. O `print` vai para o CloudWatch Logs, que captura a saída padrão. É a primeira vez no livro em que um `print` é a solução oficial, e não a vergonha do code review.

E o `webhook_pagamento.py`. Gateways assinam o corpo do webhook com uma chave compartilhada, para que ninguém marque um pedido como pago com um POST qualquer. Sem isso, qualquer pessoa com `curl` e fome comeria de graça. O formato varia de gateway para gateway; aqui, o gateway hipotético manda um HMAC-SHA256 do corpo no cabeçalho `X-Assinatura`:

```
import base64
import hashlib
import hmac
import json
import os
import uuid

import boto3

from banco import conexao

_chave = boto3.client("secretsmanager").get_secret_value(
    SecretId=os.environ["SEGREDO_WEBHOOK_ARN"]
)["SecretString"].encode()

STATUS = {"aprovada": "pago", "recusada": "recusado"}


def resposta(codigo: int, corpo: dict) -> dict:
    return {
        "statusCode": codigo,
        "headers": {"content-type": "application/json"},
        "body": json.dumps(corpo),
    }


def handler(event, context):
    corpo = event.get("body") or ""
    bruto = base64.b64decode(corpo) if event.get("isBase64Encoded") else corpo.encode()

    cabecalhos = {k.lower(): v for k, v in (event.get("headers") or {}).items()}
    recebida = cabecalhos.get("x-assinatura", "").encode()
    esperada = ("sha256=" + hmac.new(_chave, bruto,
hashlib.sha256).hexdigest()).encode()
    if not hmac.compare_digest(esperada, recebida):
        return resposta(401, {"erro": "assinatura inválida"})

    aviso = json.loads(bruto)
    status = STATUS.get(aviso.get("status"))
    try:
```

```
        pedido_id = uuid.UUID(str(aviso.get("pedido_id")))
    except ValueError:
        return resposta(422, {"erro": "pedido_id inválido"})
    if status is None:
        return resposta(422, {"erro": "status desconhecido"})

    with conexao().cursor() as cur:
        cur.execute(
            "UPDATE pedidos SET status = %s WHERE id = %s AND cobranca = %s",
            (status, pedido_id, aviso.get("cobranca")),
        )
        if cur.rowcount == 0:
            return resposta(404, {"erro": "pedido não encontrado"})

    return resposta(200, {"pedido_id": str(pedido_id), "status": status})
```

A assinatura é conferida sobre os bytes crus, antes de interpretar o JSON, porque reformatar o JSON muda o HMAC. O `compare_digest` compara em tempo constante, para a demora não revelar quantos caracteres batem. É um cofre que não faz clique quando você acerta um número. E o `UPDATE` com `status = 'pago'` é idempotente, como no capítulo 5: gateways reenviam webhooks quando não recebem 200, e o aviso repetido deixa o pedido no mesmo estado.

> **Armadilha:** Código de autenticação, verificação de assinatura e criptografia é o lugar onde um detalhe errado vira brecha. Use o esquema exato que o seu gateway documenta (muitos incluem um timestamp no cálculo, para impedir que alguém reenvie um aviso antigo), prefira a biblioteca oficial dele quando existir e peça a revisão de alguém de segurança antes de ligar o webhook em produção.

A stack

No `rango-infra/rango_infra/rango_infra_stack.py`, entram os imports:

```
from aws_cdk import (
    # ... os imports do capítulo 18 ...
    BundlingOptions,
    DockerImage,
    aws_apigatewayv2 as apigwv2,
    aws_apigatewayv2_integrations as integracoes,
    aws_events as events,
    aws_events_targets as alvos,
    aws_lambda as lambda_,
    aws_secretsmanager as secretsmanager,
)
```

Primeiro a layer. O CDK monta o zip dentro de um container:

```
        camada_banco = lambda_.LayerVersion(
            self,
            "CamadaBanco",
            code=lambda_.Code.from_asset(
```

```
                "../backend/lambdas/camada",
                bundling=BundlingOptions(
                    image=DockerImage.from_registry("python:3.12-slim"),
                    command=[
                        "pip", "install", "--no-cache-dir",
                        "-r", "requirements.txt",
                        "-t", "/asset-output/python",
                        "--platform", "manylinux2014_aarch64",
                        "--platform", "manylinux_2_28_aarch64",
                        "--implementation", "cp",
                        "--python-version", "3.12",
                        "--only-binary=:all:",
                    ],
                ),
            ),
            compatible_runtimes=[lambda_.Runtime.PYTHON_3_12],
            compatible_architectures=[lambda_.Architecture.ARM_64],
            removal_policy=RemovalPolicy.DESTROY,
        )
```

Os parâmetros `--platform`, `--implementation`, `--python-version` e `--only-binary` pedem ao pip as *wheels* para Linux arm64 com CPython 3.12, qualquer que seja a máquina do deploy. É o primo do `exec format error` do capítulo 3, da família de erros que só aparecem depois do deploy e nunca faltam a uma festa. A mensagem de erro está nas armadilhas.

Depois, o Security Group das funções e a liberação no banco:

```
        sg_lambdas = ec2.SecurityGroup(
            self, "SgLambdas", vpc=vpc, description="Funcoes Lambda do Rango"
        )
        db.connections.allow_default_port_from(sg_lambdas, "Lambdas no Postgres")
```

A descrição vai sem acento pelo motivo do capítulo 9; a AWS e o til ainda não fizeram as pazes. As funções alcançam o Secrets Manager pelo endpoint do capítulo 16, mas não a internet: as subnets são `PRIVATE_ISOLATED`, sem NAT. Uma função que precisasse chamar a API do gateway traria de volta a conta do NAT Gateway do capítulo 10, o fantasma que este livro passou capítulos tentando exorcizar.

Uma função auxiliar evita repetir a configuração comum, e evita também repetir a história da linha copiada da Cantina da Nona:

```
        codigo_lambdas = lambda_.Code.from_asset("../backend/lambdas/rango_lambda")

        def funcao(id_: str, nome: str, handler: str, **extras) -> lambda_.Function:
            variaveis = {
                "DB_HOST": db.instance_endpoint.hostname,
                "SEGREDO_DB_ARN": segredo_app.secret_arn,
                **extras.pop("environment", {}),
            }
            fn = lambda_.Function(
                self,
                id_,
                function_name=f"rango-{ambiente}-{nome}",
```

```
                runtime=lambda_.Runtime.PYTHON_3_12,
                architecture=lambda_.Architecture.ARM_64,
                handler=handler,
                code=codigo_lambdas,
                layers=[camada_banco],
                vpc=vpc,
                vpc_subnets=privadas,
                security_groups=[sg_lambdas],
                environment=variaveis,
                log_group=logs.LogGroup(
                    self,
                    f"Logs{id_}",
                    retention=logs.RetentionDays.ONE_MONTH,
                    removal_policy=RemovalPolicy.DESTROY,
                ),
                **extras,
            )
            segredo_app.grant_read(fn)
            return fn
```

Cada função ganha uma role própria, com a policy gerenciada de logs e de interfaces de rede na VPC. O `grant_read` soma a leitura do segredo e o `kms:Decrypt` na chave do capítulo 16. O grupo de logs explícito evita uma armadilha que aparece mais adiante, num suspense de novela, com a diferença de que aqui a revelação sai no mesmo capítulo.

A função de resumo e os três horários:

```
        resumo = funcao(
            "ResumoVendas",
            "resumo-vendas",
            "resumo_vendas.handler",
            memory_size=512,
            timeout=Duration.seconds(60),
            retry_attempts=0,
        )
        for hora_utc, horas in ((14, 12), (18, 4), (2, 8)):
            events.Rule(
                self,
                f"Resumo{hora_utc:02d}hUtc",
                schedule=events.Schedule.cron(minute="0", hour=str(hora_utc)),
                targets=[
                    alvos.LambdaFunction(
                        resumo, event=events.RuleTargetInput.from_object({"horas":
horas})
                    )
                ],
            )
```

As regras do EventBridge entendem cron apenas em UTC. Brasília está em UTC−3 o ano todo desde o fim do horário de verão, em 2019: 11h vira 14h, 15h vira 18h e 23h vira 2h. O EventBridge Scheduler, do capítulo 21, aceita fuso horário e evita essa conta. Fuso horário é aquele problema que todo programador acha simples até a primeira vez.

O `retry_attempts=0` é deliberado. Com as duas novas tentativas padrão, uma falha depois de alguns e-mails enviados mandaria tudo de novo, e o dono da Cantina da Nona receberia dois resumos iguais, e ligaria para perguntar se tinha vendido o dobro. O Bruno prefere um a menos a um duplicado; a falha vai precisar de alarme, assunto do capítulo 22. O `timeout` de 60 segundos também: o padrão de 3 segundos é curto para um cold start que busca segredo e abre conexão TLS.

Agora o webhook, o segredo dele e a HTTP API:

```
        segredo_webhook = secretsmanager.Secret(
            self,
            "SegredoWebhook",
            encryption_key=chave,
            generate_secret_string=secretsmanager.SecretStringGenerator(
                password_length=48, exclude_punctuation=True
            ),
            removal_policy=RemovalPolicy.DESTROY,
        )

        webhook = funcao(
            "WebhookPagamento",
            "webhook-pagamento",
            "webhook_pagamento.handler",
            memory_size=256,
            timeout=Duration.seconds(10),
            reserved_concurrent_executions=10,
            environment={"SEGREDO_WEBHOOK_ARN": segredo_webhook.secret_arn},
        )
        segredo_webhook.grant_read(webhook)

        api_webhooks = apigwv2.HttpApi(
            self, "ApiWebhooks", api_name=f"rango-{ambiente}-webhooks"
        )
        api_webhooks.add_routes(
            path="/webhooks/pagamento",
            methods=[apigwv2.HttpMethod.POST],
            integration=integracoes.HttpLambdaIntegration("IntegracaoWebhook", webhook),
        )

        CfnOutput(self, "UrlWebhooks", value=api_webhooks.api_endpoint)
        CfnOutput(self, "SegredoWebhookArn", value=segredo_webhook.secret_arn)
```

O segredo nasce com valor aleatório, útil nos testes: 48 caracteres que nenhum ser humano vai decorar, e é bom que seja assim. Em produção, o gateway gera a chave e você grava com `aws secretsmanager put-secret-value`, fora do código e do template. O nome fica a cargo do CDK porque um segredo apagado passa dias em exclusão agendada, e um nome fixo impediria recriar a stack nesse intervalo.

O `reserved_concurrent_executions=10` reserva dez ambientes só para o webhook e, ao mesmo tempo, limita o webhook a dez, protegendo o banco. Um `db.t4g.micro` aceita pouco mais de cem conexões, teto que o capítulo 13 já encontrou. Com o limite, o excesso de uma rajada recebe erro 429, e o gateway tenta de novo depois. O 429 é o "volte mais

tarde" educado da web: ninguém gosta de ouvir, mas é melhor do que a cozinha pegar fogo. A permissão para o API Gateway chamar a função, o CDK cria na resource-based policy dela, a mesma ideia da bucket policy do capítulo 11.

Antes do deploy, a cota da conta

A reserva de concorrência tem uma regra: a conta precisa manter pelo menos 100 execuções não reservadas. Numa conta nova, com cota reduzida, essa conta não fecha. Sim, a conta da conta; o português também tem seus problemas de namespace. Para conferir:

```
aws lambda get-account-settings \
  --query "AccountLimit.[ConcurrentExecutions,UnreservedConcurrentExecutions]" \
  --output text
```

```
1000    1000
```

Se aparecer `10 10`, a reserva vai falhar no deploy com esta mensagem:

```
Specified ReservedConcurrentExecutions for function decreases account's
UnreservedConcurrentExecution below its minimum value of [100].
```

Nesse caso, a saída é pedir o aumento pelo Service Quotas (cota "Concurrent executions" do serviço AWS Lambda) e, enquanto ele não sai, comentar a linha `reserved_concurrent_executions`.

O deploy pelo pipeline

Primeiro, os workflows desabilitados no fim do capítulo 18 voltam a valer:

```
gh workflow enable ci.yaml
gh workflow enable renovar-tasks.yaml
```

O commit de `backend/lambdas/` e `rango-infra/` vai para a `main`. Depois da sua aprovação no environment `producao`, o `cdk deploy` roda o container que monta a layer e publica os zips no bucket do bootstrap. O runner `ubuntu-24.04-arm` já tem Docker, e a role `rango-github-deploy` não precisa de nada novo, porque as roles do bootstrap que ela assume fazem o trabalho. A stack inteira volta, com o banco do snapshot, a API no ECS e o front. Ressuscitar a produção virou rotina neste livro, e ela já nem reclama. No fim, as saídas ficam em variáveis:

```
saida() {
  aws cloudformation describe-stacks --stack-name RangoProducao \
    --query "Stacks[0].Outputs[?OutputKey=='$1'].OutputValue" --output text
}
URL_WEBHOOKS=$(saida UrlWebhooks)
```

```
SEGREDO_WEBHOOK_ARN=$(saida SegredoWebhookArn)
echo "$URL_WEBHOOKS"
```

```
https://k3x9q2w7ab.execute-api.us-east-1.amazonaws.com
```

O primeiro cold start

A primeira chamada da função de resumo é feita à mão, com um período longo para pegar os pedidos que vieram do snapshot:

```
aws lambda invoke \
  --function-name rango-producao-resumo-vendas \
  --cli-binary-format raw-in-base64-out \
  --payload '{"horas": 720}' \
  --log-type Tail \
  --query LogResult --output text \
  resposta.json | base64 -d
cat resposta.json
```

O `--log-type Tail` devolve os últimos 4 KB do log da execução, em base64, e o `base64 -d` decodifica. O `--cli-binary-format` diz à CLI v2 que o payload é JSON puro.

```
START RequestId: 8b1f0c2e-4d7a-4c55-9f0e-2a6b9d3c7e11 Version: $LATEST
{"evento": "resumo.enviado", "horas": 720, "restaurante": "cantina-da-nona", "pedidos":
41, "faturamento": "2414.90", "ticket_medio": "58.90"}
{"evento": "resumo.enviado", "horas": 720, "restaurante": "sabor-do-sertao", "pedidos":
17, "faturamento": "736.30", "ticket_medio": "43.31"}
END RequestId: 8b1f0c2e-4d7a-4c55-9f0e-2a6b9d3c7e11
REPORT RequestId: 8b1f0c2e-4d7a-4c55-9f0e-2a6b9d3c7e11  Duration: 812.45 ms Billed
Duration: 1290 ms    Memory Size: 512 MB Max Memory Used: 96 MB  Init Duration: 476.83
ms
```

```
{"restaurantes": 2, "horas": 720}
```

Os seus números vão ser outros. Na linha `REPORT`, o `Init Duration` de 477 ms é a fase INIT: imports e busca do segredo. O `Duration` de 812 ms é a primeira chamada, quase toda gasta abrindo a conexão TLS com o Postgres. O `Billed Duration` soma as duas: é a cobrança do INIT na prática. Numa segunda chamada, logo em seguida:

```
REPORT RequestId: 1c7e5a90-...  Duration: 38.12 ms  Billed Duration: 39 ms  Memory Size:
512 MB Max Memory Used: 97 MB
```

Sem `Init Duration`. O ambiente foi reaproveitado com a conexão aberta, e a chamada caiu de 1,3 segundo para 39 milissegundos. Para a rotina das 11h, tanto faz. Para uma API com usuário esperando, faz muita diferença. Um segundo e meio, para quem está com fome, dá tempo de abrir o aplicativo da concorrência.

O webhook, assinado

Primeiro, um pedido criado pela API. O gateway simulado do capítulo 5 forma o código da cobrança com os oito primeiros caracteres da chave de idempotência:

```
CHAVE=$(uuidgen | tr 'A-Z' 'a-z')
PEDIDO=$(curl -s -X POST https://api.rango.com.br/pedidos \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $CHAVE" \
  -d '{"usuario": "ana", "restaurante": "cantina-da-nona", "total": 58.90}' | jq -r .id)
COBRANCA="cobranca-${CHAVE:0:8}"
```

Agora o terminal faz o papel do gateway, numa pequena peça de teatro em que o `openssl` interpreta o gateway de pagamento: lê a chave do webhook, monta o aviso e assina:

```
CHAVE_WEBHOOK=$(aws secretsmanager get-secret-value \
  --secret-id "$SEGREDO_WEBHOOK_ARN" --query SecretString --output text)
CORPO="{\"pedido_id\": \"$PEDIDO\", \"cobranca\": \"$COBRANCA\", \"status\":
\"aprovada\"}"
ASSINATURA="sha256=$(printf '%s' "$CORPO" \
  | openssl dgst -sha256 -hmac "$CHAVE_WEBHOOK" | awk '{print $NF}')"

curl -s -X POST "$URL_WEBHOOKS/webhooks/pagamento" \
  -H "Content-Type: application/json" \
  -H "X-Assinatura: $ASSINATURA" \
  -d "$CORPO"
```

```
{"pedido_id": "5f0d2c1a-9e3b-4b7f-8a61-0c4e2d9b7a35", "status": "pago"}
```

A chave do webhook passou pela sua variável de ambiente local só para este teste. Ao terminar, um `unset CHAVE_WEBHOOK` a apaga.

A API, que lê o mesmo banco, confirma:

```
curl -s https://api.rango.com.br/pedidos/$PEDIDO | jq .status
```

```
"pago"
```

Agora, o mesmo corpo com uma assinatura errada, com o último caractere trocado:

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST "$URL_WEBHOOKS/webhooks/pagamento" \
  -H "Content-Type: application/json" \
  -H "X-Assinatura: ${ASSINATURA%?}0" \
  -d "$CORPO"
```

```
401
```

E quando o aviso original é reenviado, com a assinatura certa, como um gateway faria depois de um timeout, a resposta volta `200` com o mesmo `"status": "pago"`. O pedido continua igual. Idempotência é isso: apertar o botão do elevador dez vezes não faz o

elevador chegar dez vezes mais rápido.

Quanto isso custa

Com os preços de referência de outubro de 2026 em us-east-1 (US$ 0,20 por milhão de chamadas e cerca de US$ 0,0000133 por GB-segundo em arm64; confira a página oficial), o resumo dá 90 execuções por mês de uns 2 segundos com 0,5 GB: 90 GB-segundos, menos de um centavo de dólar, contra os US$ 7 da task do ECS. A diferença é tão grande que dá vontade de conferir a conta duas vezes, e eu conferi. Três mil webhooks por mês custam meio centavo na HTTP API. O que custa nesta arquitetura já existia antes: o banco e os endpoints da VPC.

Desmontando

```
cd rango-infra
cdk destroy RangoProducao
gh workflow disable ci.yaml
gh workflow disable renovar-tasks.yaml
```

A exclusão pode demorar vários minutos a mais, porque as interfaces de rede da Lambda na VPC são liberadas aos poucos e travam a exclusão do Security Group e das subnets. A Lambda sai da festa devagar, despedindo-se de cada convidado. O segredo do webhook entra em exclusão agendada, e o banco deixa um snapshot novo para o próximo capítulo.

Quando serverless vale a pena, e quando não

Depois do resumo a menos de um centavo, a tentação é levar a API inteira para a Lambda. É a fase de quem acabou de ganhar um martelo e passa a ver prego em tudo. O Lambda Web Adapter permite isso, rodando o FastAPI dentro da função sem mudar o código. Antes, a conta.

A API do Rangô recebe tráfego o dia inteiro. Suponha uma média de 50 requisições por segundo, cada uma com 100 ms, numa função de 512 MB em arm64. Num mês de 30 dias, são cerca de 130 milhões de chamadas. A duração dá 6,5 milhões de GB-segundos, uns US$ 86. As chamadas da Lambda somam uns US$ 26. A HTTP API cobra mais uns US$ 130. Total: perto de US$ 240 por mês. As duas tasks do ECS que o capítulo 17 estimou custam uns US$ 29, e mesmo somando o ALB a conta fica abaixo de US$ 60. Os valores são aproximados, mas a ordem de grandeza não muda. Serverless para tráfego constante é como ir de táxi ao trabalho todos os dias: confortável e, no fim do mês, a explicação de por que tanta gente compra carro.

O ECS cobra pela capacidade reservada, e um Uvicorn ocupado atende dezenas de requisições na mesma vCPU paga. A Lambda cobra por ambiente ocupado, inclusive o tempo esperando o banco responder. Com tráfego constante, você paga a conveniência em cada milissegundo.

Os critérios, para o Rangô e para quase todo projeto:

- **Tráfego intermitente ou imprevisível** favorece a Lambda. Zero chamadas custam zero.
- **Tráfego constante e alto** favorece containers.
- **Trabalho disparado por evento** (um horário, um arquivo no S3, uma mensagem, um webhook) é o terreno natural da Lambda.
- **Trabalhos longos** ficam no ECS: 15 minutos de teto, 30 segundos atrás do API Gateway.
- **Latência previsível** é difícil com cold starts, e a provisioned concurrency traz de volta a capacidade parada.
- **Muitas conexões com banco relacional** são um problema. A Lambda escala para centenas de ambientes em segundos, e o Postgres não acompanha. O RDS Proxy, um PgBouncer gerenciado e cobrado à parte, existe para isso, e serviços sem conexão persistente, como S3, SQS e DynamoDB, combinam melhor com funções.

O Rangô fica híbrido, como muita coisa em produção, e como carro flex, que bebe o que estiver mais barato no posto: a API e os workers do stream seguem no ECS, onde o tráfego é constante; o que roda de vez em quando ou reage a eventos externos vai para a Lambda.

Armadilhas e casos reais

> **Armadilha:** Wheel do sistema errado. Quem instala as dependências da layer com um `pip install` simples num Mac ou num Linux x86 recebe binários que não rodam na Lambda arm64. O deploy funciona, e a primeira chamada falha com `Runtime.ImportModuleError: Unable to import module 'resumo_vendas': no pq wrapper available.` O `--platform` e o `--only-binary` do bundling existem para isso. Bibliotecas em Python puro não sofrem com o problema, por isso ele costuma aparecer só quando entra a primeira dependência com código compilado.
>
> **Armadilha:** Timeout padrão de 3 segundos. Uma função criada sem `timeout` explícito morre em 3 segundos, e um cold start que busca segredo e abre conexão TLS com o banco pode passar disso. O log mostra `Task timed out after 3.00`

`seconds`, e a função funciona na segunda tentativa, já aquecida. Esse "falha só às vezes" costuma consumir uma tarde de investigação e um pedaço da sua fé na profissão.

Armadilha: Grupo de logs sem retenção. Sem o `log_group` explícito, a Lambda cria sozinha o grupo `/aws/lambda/nome-da-funcao` na primeira execução, com retenção infinita. O CloudFormation não sabe que ele existe, então o grupo sobrevive ao `cdk destroy` e cresce para sempre, cobrando armazenamento. Defina o grupo na stack, com retenção.

Armadilha: Laço recursivo. Uma função disparada por upload no S3 gera uma miniatura e grava no mesmo bucket. A gravação dispara a função de novo, que gera a miniatura da miniatura, e assim por diante, como dois espelhos frente a frente, com a concorrência subindo até o limite da conta. A AWS hoje detecta e interrompe esse tipo de laço para algumas combinações de serviços, mas não conte com isso: grave a saída em outro bucket ou em outro prefixo, e filtre o gatilho pelo prefixo de entrada.

Armadilha: Estado que vaza entre chamadas. Variáveis globais sobrevivem entre invocações, o que serve para conexões e, por engano, para dados de uma requisição. Num exemplo hipotético, uma função guarda o usuário autenticado numa global, a chamada seguinte de outro usuário falha antes de sobrescrever, e a resposta sai com os dados do primeiro. O que é da requisição fica dentro do `handler`, inclusive arquivos em `/tmp`.

Armadilha: Cron em UTC. Um `cron(0 11 * * ? *)` numa regra do EventBridge dispara às 8h em Brasília, antes de a cozinha abrir, e o dono do restaurante lê o resumo de vendas ainda de pijama.

Dica: O `event` é só um dicionário. No pytest, chame o handler com um evento de exemplo no formato 2.0 e você cobre a maior parte dos testes sem ferramenta extra.

Resumo

- A Lambda executa uma função Python (`handler(event, context)`) quando um evento acontece e cobra por chamada e por GB-segundo. Zero chamadas custam zero.
- Cada ambiente de execução passa por INIT uma vez, atende uma chamada por vez e fica congelado entre chamadas. Cold start é o INIT somado à primeira chamada; desde agosto de 2025, o INIT é cobrado.
- Memória vai de 128 MB a 10.240 MB, com CPU proporcional (1.769 MB equivalem a uma vCPU). O teto de execução é de 15 minutos; o padrão de timeout, 3 segundos.

- A concorrência padrão é de 1.000 ambientes por região, menor em contas novas. Reservar concorrência protege o banco e exige deixar 100 execuções livres na conta.
- Layers levam dependências para `/opt/python`, no máximo cinco por função. As wheels precisam ser montadas para a arquitetura certa.
- O API Gateway transforma HTTP em evento. A HTTP API (US$ 1,00 por milhão) serve à maioria dos casos e corta integrações em 30 segundos; a REST API (US$ 3,50) traz chaves de API, validação, WAF, endpoints privados e cache.
- O Rangô ganhou a `resumo-vendas`, disparada por três regras do EventBridge em UTC, e a `webhook-pagamento`, atrás de uma HTTP API, com assinatura HMAC e `UPDATE` idempotente. As duas rodam na VPC e falam com o RDS.
- Serverless vale para trabalho intermitente e disparado por evento. Para tráfego constante, muitas conexões com banco relacional ou tarefas longas, containers saem mais baratos e mais previsíveis.

Entregadores a cada cinco segundos

Com o PIX funcionando, o Bruno volta com a próxima ideia, e essa ele chama de prioridade da empresa, categoria em que, curiosamente, cabem todos os pedidos do Bruno: o cliente quer ver o entregador no mapa, em tempo real. O aplicativo do entregador manda a posição a cada cinco segundos, e o React do cliente pergunta "onde está meu pedido?" com a mesma frequência.

Com 200 entregadores no jantar, são 40 gravações por segundo só de posição, mais as leituras dos clientes esperando. Cada posição vira uma linha numa tabela que cresce milhões de linhas por semana e que ninguém consulta depois de uma hora. Agora imagine isso no `db.t4g.micro` dos pedidos, que mal aguentou seis instâncias no capítulo 13, com Lambdas na frente, cada uma com a sua conexão. O Postgres passa o jantar gravando coordenadas que vão para o lixo em seguida, como um escrivão de cartório anotando a posição de cada formiga do quintal.

O próximo capítulo apresenta o Amazon DynamoDB, um banco acessado por HTTP, sem conexão persistente, que aguenta milhares de gravações por segundo e apaga sozinho o que expirou. Ele obriga você a desenhar as tabelas começando pelas perguntas que a aplicação vai fazer. E o capítulo mostra também quando ele é a escolha errada.

Referências

- AWS Lambda, cotas e limites: https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html

- AWS Lambda, runtimes e calendário de descontinuação: https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtimes.html
- AWS Lambda, preços: https://aws.amazon.com/lambda/pricing/
- AWS Compute Blog, cobrança da fase INIT padronizada (2025): https://aws.amazon.com/blogs/compute/aws-lambda-standardizes-billing-for-init-phase/
- AWS What's New, SnapStart para Python e .NET (nov/2024): https://aws.amazon.com/about-aws/whats-new/2024/11/aws-lambda-snapstart-python-net-functions/
- Lambda Web Adapter, SnapStart: https://aws.github.io/aws-lambda-web-adapter/features/snapstart.html
- Amazon API Gateway, HTTP API versus REST API: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html
- Amazon API Gateway, cotas da HTTP API: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-quotas.html
- Amazon API Gateway, preços: https://aws.amazon.com/api-gateway/pricing/

CAPÍTULO 20

DynamoDB

No capítulo anterior, o resumo de vendas e o webhook do PIX saíram do servidor e viraram funções Lambda: código que só existe quando é chamado, cobrado por milissegundo, atrás de uma HTTP API do API Gateway. O capítulo terminou com um aviso que vale para este: Lambda escala para centenas de ambientes em segundos, e o Postgres não acompanha. A stack `RangoProducao` foi destruída, e o banco deixou um snapshot novo esperando por nós, fiel como cachorro na porta de casa.

Quarenta coordenadas por segundo

O pedido do Bruno parece simples. Os pedidos do Bruno sempre parecem simples; ninguém nunca pediu nada complicado numa frase só. O cliente quer ver o entregador andando no mapa. O aplicativo do entregador manda a posição a cada cinco segundos, e a tela do React pergunta "onde está meu pedido?" no mesmo ritmo.

A primeira versão que vem à cabeça é uma tabela no Postgres que já existe:

```
CREATE TABLE posicoes (
    pedido_id     uuid NOT NULL REFERENCES pedidos (id),
    lat           numeric(9, 6) NOT NULL,
    lon           numeric(9, 6) NOT NULL,
    registrada_em timestamptz NOT NULL,
    PRIMARY KEY (pedido_id, registrada_em)
);
```

Funciona no notebook. Agora vem a conta do jantar. Com 200 entregadores na rua, são 40 gravações por segundo, mais 40 leituras dos clientes olhando o mapa. Em quatro horas de pico por dia, isso dá 576 mil linhas por noite, uns 4 milhões por semana. Cada `INSERT` mexe no índice da chave primária, gera WAL, entra no backup e na réplica. E quase nenhuma dessas linhas vai ser lida de novo depois que o pedido chegar: passada uma hora, ninguém quer saber onde o entregador estava às 20h41. É um dado com a validade de um "chego em cinco minutos".

Agora coloque essa carga no `db.t4g.micro`, que já mostrou o `FATAL: sorry, too many clients already` no capítulo 13. Se a gravação de posição for para uma Lambda, cada ambiente abre a sua conexão. Num sábado de chuva, com o dobro de entregadores, as

conexões acabam, e quem sofre junto é o `POST /pedidos`, que divide o mesmo banco. Um mapa derrubando a confirmação de pedidos rende um postmortem constrangedor. Ninguém quer explicar numa reunião que a pizza não foi confirmada porque o bonequinho da moto estava ocupado demais andando na tela.

Daria para remendar com instância maior, RDS Proxy e um job noturno apagando linhas velhas, cada peça custando dinheiro e atenção. O mapa, que era um detalhe charmoso, viraria a parte mais cara da arquitetura, como o enfeite que custa mais do que o bolo. O rastreamento pede um lugar que aceite muitas gravações pequenas, responda a perguntas simples em milissegundos, dispense conexão aberta e jogue fora sozinho o que envelheceu.

Um dicionário do tamanho que você precisar

O **Amazon DynamoDB** é um banco de dados chave-valor e de documentos, totalmente gerenciado. Não existe instância para escolher, disco para aumentar ou versão para atualizar. E não existe conexão: cada operação é uma chamada HTTPS assinada com SigV4, autorizada pelo IAM. Mil Lambdas gravando nele não seguram nada aberto. Para quem passou o capítulo 13 contando conexões como quem conta moedas no fim do mês, é um alívio quase físico.

Para quem programa em Python, pense num dicionário de dicionários distribuído por muitas máquinas:

```
tabela = {
    ("PEDIDO#5f0d2c1a", "ATUAL"): {"status": "em_rota", "entregador": "joao-p"},
    ("PEDIDO#5f0d2c1a", "POS#1792094460000"): {"posicao": {"lat": -8.05, "lon": -34.9}},
}
```

Cada linha é um **item**, e os campos dele são **atributos**. Fora da chave, nenhum atributo é obrigatório, e dois itens da mesma tabela podem ter atributos diferentes. Não existe `ALTER TABLE`: você grava o atributo novo, e ele passa a existir naquele item. O DBA que mora dentro de você acaba de desmaiar. Deixe-o descansar; ele acorda no fim do capítulo, e com razão.

Partition key e sort key

A chave primária tem uma ou duas partes, e entender as duas é metade do capítulo. A outra metade é aprender a conviver com a escolha.

A **partition key** decide onde o item mora. O DynamoDB calcula um hash dela para escolher a **partição**, um pedaço da tabela guardado em servidores e replicado em três zonas de disponibilidade. É o `hash()` que escolhe a posição de uma chave num `dict`, só

que o balde é uma máquina, refrigerada e com crachá na porta. Conforme a tabela cresce, o DynamoDB divide partições sem você ver, como o garçom que troca a toalha da mesa sem interromper a conversa.

A **sort key** é opcional e ordena os itens com a mesma partition key. Esses itens formam uma **item collection** e ficam guardados juntos, em ordem: uma lista ordenada pendurada em cada chave do dicionário. É isso que permite pedir "as últimas 120 posições do pedido X" numa única leitura sequencial. A combinação das duas identifica o item, e gravar de novo a mesma combinação substitui o item.

Cada partição tem um teto de cerca de 3.000 unidades de leitura e 1.000 de escrita por segundo. A *adaptive capacity* remaneja capacidade para uma partição muito procurada, sem passar desse teto. Por isso a partition key precisa espalhar o tráfego. O `id` do pedido espalha bem; a data do dia concentra todo mundo na mesma chave, a *hot partition*. É o único banheiro aberto no intervalo do show, com os outros vinte interditados.

As operações, e a que você deve evitar

A API é pequena. Para um item, você tem `PutItem` (grava ou substitui), `GetItem` (lê pela chave completa), `UpdateItem` (altera atributos, criando o item se ele não existir) e `DeleteItem`. Para vários, tem `Query` e `Scan`.

A `Query` recebe uma partition key exata e, opcionalmente, uma condição sobre a sort key: igual, maior, menor, entre dois valores ou "começa com". Ela lê só aquela item collection, em ordem, para frente ou para trás: um `SELECT ... WHERE pk = ? AND sk BETWEEN ? AND ? ORDER BY sk` que sempre usa índice, porque o próprio armazenamento é o índice.

O `Scan` lê a tabela inteira, e você paga pela leitura de tudo. Ele existe para exportações e manutenção. Usar `Scan` no dia a dia é procurar a chave do carro esvaziando a casa inteira na calçada, e pagando por gaveta aberta.

A pegadinha para quem vem do SQL é a *filter expression*. `Query` e `Scan` aceitam um filtro sobre qualquer atributo, que parece um `WHERE`, só que roda depois da leitura. O DynamoDB lê os itens, cobra por eles e então descarta os que não passam. Um `Scan` com filtro `restaurante = cantina-da-nona` custa o mesmo que o `Scan` sem filtro. É pagar o rodízio inteiro e comer só a salada. Também não há `JOIN`, `GROUP BY` ou subconsulta: se a pergunta não cabe numa `Query`, você muda o desenho das chaves ou cria um índice.

As leituras são *eventualmente consistentes* por padrão: logo depois de uma gravação, uma leitura pode devolver a versão anterior por uma fração de segundo. "Eventualmente", aqui, quer dizer milissegundos, e não o "eventualmente" das tarefas do backlog. A leitura *fortemente consistente* (`ConsistentRead=True`) custa o dobro. E toda escrita aceita uma *condition expression*, "grave só se o atributo X tiver o valor Y"; se a

condição falhar, nada é gravado e você recebe `ConditionalCheckFailedException`. É o `UPDATE ... WHERE status = 'pago'` do capítulo 19, com checagem e escrita atômicas no mesmo item.

Capacidade: pagar por requisição ou reservar

O DynamoDB mede o trabalho em unidades. Na escrita, uma unidade cobre um item de até 1 KB; um item de 2,5 KB consome 3. O DynamoDB arredonda para cima, como a balança do restaurante a quilo, que nunca erra a seu favor. Na leitura, uma unidade cobre até 4 KB com consistência forte; a leitura eventualmente consistente consome meia. As unidades usam KB binário, 1.024 bytes, e escritas dentro de transação contam em dobro.

No modo **on-demand**, você paga por requisição. Não há nada para configurar: a tabela aceita o tráfego que chegar, até a cota padrão de 40.000 unidades de leitura e 40.000 de escrita por segundo por tabela, que pode ser aumentada. Os preços de referência em us-east-1, consultados em outubro de 2026, são de cerca de US$ 0,625 por milhão de unidades de escrita e US$ 0,125 por milhão de unidades de leitura. São valores aproximados; confira a página oficial e a calculadora antes de decidir. Em novembro de 2024, a AWS cortou pela metade o preço do on-demand em todas as regiões e passou a tratá-lo como o modo padrão para a maioria das cargas. Metade do preço de verdade, sem o "a partir de" em letra miúda.

No modo **provisionado**, você declara quantas unidades por segundo a tabela suporta (RCU para leitura, WCU para escrita) e paga por hora pela reserva, use ou não. É a mesa reservada no restaurante: o lugar é seu, se você vai aparecer é problema seu. O excedente recebe `ProvisionedThroughputExceededException`. O auto scaling ajusta a capacidade por política de alvo, como o ASG do capítulo 13, mas reduzir tem limite: quatro vezes no início do dia em UTC e mais uma por hora, no máximo 27 por dia.

Comece em on-demand. O provisionado compensa com tráfego alto, estável e já medido. O nível gratuito vale só para o provisionado (25 WCU e 25 RCU) e para os primeiros 25 GB de armazenamento, que custa cerca de US$ 0,25 por GB-mês na classe Standard.

Começar pelas perguntas

No Postgres, você modela as entidades, normaliza e depois escreve as consultas; o planejador responde qualquer `SELECT` que você inventar amanhã. O DynamoDB inverte a ordem. Você começa listando os **padrões de acesso**: as perguntas exatas que a aplicação vai fazer, com que frequência e com que dados na mão. Depois desenha as chaves para que cada pergunta vire um `GetItem` ou uma `Query`. No Postgres, você enche a despensa e cozinha o que pedirem. No DynamoDB, você recebe o cardápio antes e compra só os ingredientes daqueles pratos.

Para o rastreamento do Rangô, as perguntas são estas:

#	Quem pergunta	Pergunta	Frequência
1	App do entregador	Gravar a posição atual do pedido X	a cada 5 s por entregador
2	Cliente	Onde está o entregador do pedido X agora?	a cada 5 s por cliente
3	Cliente	Qual o trajeto dos últimos minutos do pedido X?	ao abrir o mapa
4	Restaurante	Quais entregas minhas estão na rua agora?	a cada 30 s
5	Entregador	Iniciar e concluir a entrega do pedido X	2 vezes por pedido

Repare no que falta: nenhuma pergunta do tipo "quanto tempo em média os entregadores levaram em outubro". Essa pergunta existe, o Bruno vai fazê-la um dia (provavelmente numa sexta, às seis da tarde), e o lugar dela é outro. Voltamos a isso na seção sobre quando não usar.

Uma tabela, vários tipos de item

As perguntas 2, 3 e 5 giram em torno do pedido. Então a partition key é o pedido, e dentro da item collection dele moram dois tipos de item:

```
pk                    sk                   atributos
--------------------  -------------------  ------------------------------------------
--
PEDIDO#5f0d2c1a-...   ATUAL                status, entregador, restaurante, posicao,
                                           registrada_em, iniciada_em
PEDIDO#5f0d2c1a-...   POS#1792094455000    posicao, expira_em
PEDIDO#5f0d2c1a-...   POS#1792094460000    posicao, expira_em
PEDIDO#5f0d2c1a-...   POS#1792094465000    posicao, expira_em
```

O item `ATUAL` guarda o estado da entrega e a última posição. Ele é sobrescrito a cada cinco segundos e responde à pergunta 2 com um `GetItem`. Os itens `POS#...` são o histórico: um por posição, com a sort key formada pelo instante em milissegundos. A pergunta 3 vira uma `Query` com `begins_with(sk, "POS#")`, em ordem decrescente, limitada aos últimos 120 pontos (dez minutos).

O número na sort key tem 13 dígitos, completados com zeros à esquerda. A sort key é texto, e texto ordena caractere a caractere: sem o preenchimento, `"POS#999"` viria depois de `"POS#1000"`. Toda chave numérica dentro de uma string precisa desse cuidado, o mesmo que você já teve ao nomear arquivos `01-`, `02-` para o `ls` mostrar na ordem. Quem já viu `capitulo-10` aparecer antes de `capitulo-2` sabe que a ordem alfabética não tem nenhum respeito pela matemática.

Os nomes genéricos `pk` e `sk`, com prefixos como `PEDIDO#` e `POS#`, são a marca de uma técnica chamada **single-table design**: guardar entidades diferentes na mesma tabela, com as chaves arranjadas para que itens lidos juntos fiquem na mesma item collection. Para quem vem do SQL, é como guardar o resultado de um `JOIN` já pronto. Em vez de juntar `entregas` e `posicoes` na hora da leitura, você grava as duas coisas lado a lado e lê de uma vez. É a marmita montada no domingo: dá trabalho na véspera, e na segunda é só abrir.

A técnica tem custo. Uma tabela com dez tipos de item é difícil de ler no console, de explicar para quem chega no time e de mudar quando aparece um padrão de acesso imprevisto. Toda empresa tem uma tabela assim, que só uma pessoa entende, e essa pessoa está de férias. Para o Rangô, a regra é modesta: uma tabela por domínio, com os itens lidos juntos. O rastreamento ganha a sua, `rango-entregas`.

Um índice para o restaurante

A pergunta 4 tem o restaurante na mão, e a partition key da tabela é o pedido. Sobraria o `Scan` com filtro, que lê tudo.

A resposta é um **Global Secondary Index** (GSI): uma cópia de parte da tabela, mantida pelo DynamoDB de forma assíncrona, com outra chave primária. O índice `por-restaurante` usa `restaurante` como partition key e `iniciada_em` como sort key, e a `Query` nele responde "entregas da Cantina da Nona, da mais antiga para a mais nova". Leituras em GSI são sempre eventualmente consistentes, o que o painel do restaurante nem percebe. Um mapa meio segundo atrasado não faz nenhum cozinheiro largar a panela.

Um item só entra no GSI se tiver todos os atributos da chave do índice. Os itens `POS#...` não têm `restaurante` e ficam de fora; quando a entrega termina, remover `restaurante` do item `ATUAL` tira a entrega do índice. Esse padrão se chama **índice esparso** (*sparse index*): o GSI guarda só as entregas em andamento. É uma lista de convidados que risca sozinha quem já foi embora da festa.

O custo: toda escrita que mexe em atributos projetados no índice também é uma escrita no índice, cobrada à parte. A projeção padrão é `ALL`; há também `KEYS_ONLY` e `INCLUDE`. Como o restaurante quer ver a posição, fica `ALL`, e cada posição custa duas escritas, uma na tabela e uma no índice.

> **Dica:** Existe também o Local Secondary Index (LSI), que troca só a sort key. Ele nasce com a tabela, no máximo cinco, e impede o DynamoDB de dividir uma item collection entre partições. Na dúvida, use GSI (até 20 por tabela, na cota padrão).

> **Dica:** Desde novembro de 2025, a chave de um GSI pode ter até quatro atributos na partition key e quatro na sort key, sem custo adicional. Antes disso, um índice por "restaurante e status" exigia gravar um atributo sintético como `cantina-da-nona#em_rota`. Se você encontrar esse padrão de concatenação em código antigo, saiba que hoje ele tem alternativa. Quem escreveu não estava confuso; só escreveu antes de novembro de 2025.

Na prática

Esta parte acrescenta ao Rangô a tabela `rango-entregas`, as rotas de rastreamento na API do ECS e uma Lambda que avisa o Postgres quando a entrega termina. A tabela cuida do que muda a cada cinco segundos; o pedido, com status, valor e cobrança, continua no Postgres, que é o lugar dele. Dinheiro dos outros fica em banco relacional, com chave estrangeira e cinto de segurança.

O ponto de partida

Como no capítulo anterior, o ponto de partida é o snapshot deixado pelo `cdk destroy`, que este comando encontra:

```
aws rds describe-db-snapshots --snapshot-type manual \
  --query "reverse(sort_by(DBSnapshots,&SnapshotCreateTime))[0].DBSnapshotIdentifier" \
  --output text
```

O nome dele vai no `snapshot_db` do `rango-infra/app.py`. O pedido pago no teste do webhook do capítulo 19 está nesse snapshot, e vai servir de cobaia. É o pedido mais testado da história do Rangô, e até hoje ninguém comeu.

A tabela na stack

No `rango-infra/rango_infra/rango_infra_stack.py`, entram os imports:

```
from aws_cdk import (
    # ... os imports do capítulo 19 ...
    aws_dynamodb as dynamodb,
    aws_lambda_event_sources as fontes,
)
```

E a tabela, logo depois da VPC:

```
        tabela_entregas = dynamodb.Table(
            self,
            "TabelaEntregas",
            partition_key=dynamodb.Attribute(name="pk",
type=dynamodb.AttributeType.STRING),
            sort_key=dynamodb.Attribute(name="sk", type=dynamodb.AttributeType.STRING),
```

```
            billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
            time_to_live_attribute="expira_em",
            stream=dynamodb.StreamViewType.NEW_AND_OLD_IMAGES,
            removal_policy=RemovalPolicy.DESTROY,
        )
        tabela_entregas.add_global_secondary_index(
            index_name="por-restaurante",
            partition_key=dynamodb.Attribute(name="restaurante",
type=dynamodb.AttributeType.STRING),
            sort_key=dynamodb.Attribute(name="iniciada_em",
type=dynamodb.AttributeType.NUMBER),
        )
```

Repare no que a declaração tem e no que ela não tem. Só os atributos das chaves são declarados, com tipo (`STRING` ou `NUMBER`); todo o resto do item é livre. Quem já escreveu uma migration de cem linhas no Alembic sente uma pontinha de inveja. O `PAY_PER_REQUEST` é o on-demand. O nome fica a cargo do CDK, pelo mesmo motivo dos segredos: nome fixo impede a substituição do recurso. O `DESTROY` faz sentido aqui porque posições de entrega são descartáveis por natureza; numa tabela com dados que importam, use `RETAIN` e ligue o point-in-time recovery, que custa cerca de US$ 0,20 por GB-mês e restaura a tabela para qualquer segundo dos últimos 35 dias, como o PITR do RDS no capítulo 12.

O `time_to_live_attribute` e o `stream` são os dois recursos que ainda não apresentei. Eles merecem uma seção cada, e chegam logo depois do código.

O caminho de rede

As tarefas do ECS rodam nas subnets `PRIVATE_ISOLATED`, sem NAT e sem internet. Até aqui, cada serviço da AWS que a API usa ganhou um endpoint na VPC. O DynamoDB é um dos dois serviços com *gateway endpoint*, o tipo gratuito que você criou para o S3 no capítulo 10:

```
        vpc.add_gateway_endpoint(
            "DynamoDb",
            service=ec2.GatewayVpcEndpointAwsService.DYNAMODB,
            subnets=[privadas],
        )
```

Sem essa linha, o log mostra `ConnectTimeoutError` para `dynamodb.us-east-1.amazonaws.com`, depois de algumas retentativas do boto3. Leva uma tarde para entender, porque a permissão está certa e a tabela existe. A tarde termina com alguém dizendo "era só uma linha", num tom que mistura alívio e vergonha.

A API ganha acesso à tabela

Na task role da API e no container:

```
        tabela_entregas.grant_read_write_data(role_app)
```

```
        tarefa_api.add_container(
            "api",
            image=imagem,
            secrets=variaveis,
            environment={"TABELA_ENTREGAS": tabela_entregas.table_name},
            port_mappings=[ecs.PortMapping(container_port=8000)],
            logging=ecs.LogDrivers.aws_logs(stream_prefix="api", log_group=logs_app),
        )
```

O `grant_read_write_data` cria uma policy com as ações de leitura e escrita de itens, no ARN da tabela e no ARN dos índices dela (`.../index/*`). Esquecer os índices é um erro comum quando se escreve a policy à mão: a `Query` na tabela funciona, a `Query` no GSI devolve `AccessDeniedException`. A tabela deixa você entrar na sala, e o índice pede o crachá de novo. O nome da tabela é configuração, então vai como variável de ambiente comum.

O código da API

O código fica em `backend/app/entregas.py`:

```
import os
import time
import uuid
from decimal import Decimal

import boto3
from boto3.dynamodb.conditions import Attr, Key
from fastapi import APIRouter, HTTPException, Path
from pydantic import BaseModel, Field

from app.banco import pool

tabela = boto3.resource(
    "dynamodb", region_name=os.environ.get("AWS_REGION", "us-east-1")
).Table(os.environ.get("TABELA_ENTREGAS", "rango-entregas-local"))
CondicaoFalhou = tabela.meta.client.exceptions.ConditionalCheckFailedException

router = APIRouter()
Restaurante = Path(pattern=r"^[a-z0-9-]{1,60}$")
HISTORICO_SEGUNDOS = 24 * 3600


class Inicio(BaseModel):
    entregador: str = Field(pattern=r"^[a-z0-9-]{1,40}$")


class Posicao(BaseModel):
    lat: float = Field(ge=-90, le=90)
    lon: float = Field(ge=-180, le=180)
    registrada_em: int = Field(gt=0)  # epoch em milissegundos, do celular


def chave(pedido_id: uuid.UUID) -> dict:
```

```
    return {"pk": f"PEDIDO#{pedido_id}", "sk": "ATUAL"}


@router.post("/entregas/{pedido_id}/iniciar", status_code=201)
def iniciar(pedido_id: uuid.UUID, inicio: Inicio):
    with pool.connection() as con:
        pedido = con.execute(
            "SELECT restaurante FROM pedidos WHERE id = %s AND status = 'pago'",
            (pedido_id,),
        ).fetchone()
    if pedido is None:
        raise HTTPException(404, "pedido pago não encontrado")
    try:
        tabela.put_item(
            Item={
                **chave(pedido_id),
                "status": "em_rota",
                "entregador": inicio.entregador,
                "restaurante": pedido["restaurante"],
                "iniciada_em": int(time.time()),
                "registrada_em": 0,
            },
            ConditionExpression="attribute_not_exists(pk)",
        )
    except CondicaoFalhou:
        raise HTTPException(409, "entrega já iniciada")
    return {"pedido_id": str(pedido_id), "status": "em_rota"}


@router.post("/entregas/{pedido_id}/posicao", status_code=204)
def registrar_posicao(pedido_id: uuid.UUID, p: Posicao):
    posicao = {"lat": Decimal(str(p.lat)), "lon": Decimal(str(p.lon))}
    try:
        tabela.update_item(
            Key=chave(pedido_id),
            UpdateExpression="SET posicao = :pos, registrada_em = :t",
            ConditionExpression="#st = :em_rota AND registrada_em < :t",
            ExpressionAttributeNames={"#st": "status"},
            ExpressionAttributeValues={
                ":pos": posicao,
                ":t": p.registrada_em,
                ":em_rota": "em_rota",
            },
        )
    except CondicaoFalhou:
        return  # posição atrasada ou entrega encerrada: descarta
    tabela.put_item(
        Item={
            "pk": f"PEDIDO#{pedido_id}",
            "sk": f"POS#{p.registrada_em:013d}",
            "posicao": posicao,
            "expira_em": int(time.time()) + HISTORICO_SEGUNDOS,
        }
    )


@router.get("/entregas/{pedido_id}")
def posicao_atual(pedido_id: uuid.UUID):
    item = tabela.get_item(Key=chave(pedido_id)).get("Item")
    if item is None:
        raise HTTPException(404, "entrega não encontrada")
    return {k: item.get(k) for k in ("status", "entregador", "posicao",
```

```
"registrada_em")}


@router.get("/entregas/{pedido_id}/trajeto")
def trajeto(pedido_id: uuid.UUID):
    resposta = tabela.query(
        KeyConditionExpression=Key("pk").eq(f"PEDIDO#{pedido_id}")
        & Key("sk").begins_with("POS#"),
        FilterExpression=Attr("expira_em").gt(int(time.time())),
        ScanIndexForward=False,
        Limit=120,
    )
    return [item["posicao"] for item in reversed(resposta["Items"])]


@router.get("/restaurantes/{restaurante}/entregas")
def entregas_do_restaurante(restaurante: str = Restaurante):
    resposta = tabela.query(
        IndexName="por-restaurante",
        KeyConditionExpression=Key("restaurante").eq(restaurante),
    )
    return [
        {
            "pedido_id": item["pk"].removeprefix("PEDIDO#"),
            "entregador": item["entregador"],
            "posicao": item.get("posicao"),
        }
        for item in resposta["Items"]
    ]


@router.post("/entregas/{pedido_id}/concluir")
def concluir(pedido_id: uuid.UUID):
    agora = int(time.time())
    try:
        tabela.update_item(
            Key=chave(pedido_id),
            UpdateExpression="SET #st = :entregue, concluida_em = :agora, "
            "expira_em = :expira REMOVE restaurante",
            ConditionExpression="#st = :em_rota",
            ExpressionAttributeNames={"#st": "status"},
            ExpressionAttributeValues={
                ":entregue": "entregue",
                ":em_rota": "em_rota",
                ":agora": agora,
                ":expira": agora + HISTORICO_SEGUNDOS,
            },
        )
    except CondicaoFalhou:
        raise HTTPException(409, "entrega não está em rota")
    return {"pedido_id": str(pedido_id), "status": "entregue"}
```

O `iniciar` lê o pedido no Postgres uma única vez, para conferir o pagamento e descobrir o restaurante; daí em diante, as posições não tocam no Postgres. O `attribute_not_exists(pk)` faz o `PutItem` falhar se o item `ATUAL` já existir, o que impede dois entregadores de iniciarem a mesma entrega. Sem isso, dois motoqueiros disputariam a mesma pizza na porta da cantina, cena que nenhum cliente merece assistir.

O `registrar_posicao` usa `UpdateItem` com condição dupla. A primeira parte recusa posições de entregas encerradas ou nunca iniciadas. A segunda (`registrada_em < :t`) resolve um problema do capítulo 5: a rede do celular reordena mensagens, e a posição das 20h41min05s pode chegar depois da das 20h41min10s. Sem a condição, o mapa pularia para trás, e o cliente veria o entregador dando ré pela avenida. Com ela, o próprio banco descarta a posição velha, sem ler antes de escrever. O instante vem do celular, que conhece a ordem real das leituras do GPS.

O `#st` existe porque `status` é palavra reservada do DynamoDB, e usá-lo direto numa expressão dá `ValidationException: ... Attribute name is a reserved keyword; reserved keyword: status`. A lista tem centenas de palavras, e lendo-a inteira dá a impressão de que alguém a montou olhando o seu modelo de dados; a posição vive num mapa `posicao` justamente para que `lat` e `lon` entrem como valor, sem placeholders.

O `Decimal(str(...))` é obrigatório, porque o boto3 se recusa a gravar `float`:

```
TypeError: Float types are not supported. Use Decimal types instead.
```

O número do DynamoDB tem precisão decimal, e o `float` do Python é binário: `-8.05` vira `-8.050000000000007105...`. Passar pelo `str` gera o `Decimal("-8.05")` esperado. O `float` é o colega que arredonda a conta do bar "mais ou menos"; o DynamoDB exige nota fiscal. Na leitura, o FastAPI converte os `Decimal` para JSON sozinho.

O `trajeto` usa a filter expression do jeito certo: sobre itens que a `Query` já leu pela chave, para esconder posições expiradas (o motivo aparece na seção sobre TTL). O `Limit` conta os itens lidos antes do filtro. E o `concluir` faz três coisas numa escrita condicional: muda o status, remove `restaurante` (tirando a entrega do índice esparso) e dá ao item `ATUAL` um prazo de expiração.

O router entra no `backend/app/main.py`, como o das fotos:

```
from app.entregas import router as entregas_router

app.include_router(entregas_router)
```

> **Armadilha:** As rotas `iniciar`, `posicao` e `concluir` aceitam qualquer chamada, e o Rangô ainda não tem login de entregador. Do jeito que está, qualquer pessoa que descubra o `id` de um pedido pode mover o entregador no mapa ou marcar a entrega como concluída. A rota do restaurante ganha login ainda neste capítulo, mas o entregador fica para depois. Antes de produção de verdade, essas rotas precisam autenticar o entregador e conferir que ele é o dono da entrega. Autenticação é código que pede uma biblioteca estabelecida (um provedor OIDC, como o Amazon Cognito, ou o que a empresa já usa) e revisão de alguém de segurança.

> **Dica:** O `TABELA_ENTREGAS` tem um valor padrão para os testes do CI e o Compose subirem sem AWS. Para desenvolver contra uma tabela local, existe a imagem oficial `amazon/dynamodb-local`, e o boto3 aceita a variável de ambiente `AWS_ENDPOINT_URL_DYNAMODB` para apontar para ela sem mudar o código.

No React, o mapa do cliente pergunta a posição a cada cinco segundos. O componente de mapa fica por sua conta (desenhar a motinha andando consome uma sprint inteira de alguém muito feliz); a parte que conversa com a API é esta:

```
import { useEffect, useState } from "react";

type Posicao = { lat: number; lon: number };

export function usePosicaoEntregador(pedidoId: string) {
  const [posicao, setPosicao] = useState<Posicao | null>(null);

  useEffect(() => {
    const buscar = async () => {
      const resposta = await
fetch(`${import.meta.env.VITE_API_URL}/entregas/${pedidoId}`);
      if (resposta.ok) setPosicao((await resposta.json()).posicao);
    };
    buscar();
    const intervalo = setInterval(buscar, 5000);
    return () => clearInterval(intervalo);
  }, [pedidoId]);

  return posicao;
}
```

TTL: o banco joga fora o que envelheceu

O **Time to Live** (TTL) é o job de limpeza que você não precisa escrever. Você escolhe um atributo da tabela, aqui o `expira_em`, e grava nele um instante no formato epoch Unix em segundos, como `Number`. Quando esse instante passa, o DynamoDB apaga o item por conta própria, sem cobrar escrita por isso. Se o atributo não existir no item ou tiver outro tipo (uma string com data ISO, por exemplo), o item é ignorado e vive para sempre, como o pote de sorvete com feijão no fundo do congelador.

É o `EXPIRE` do Redis do capítulo 4, com uma diferença que pega muita gente: o apagamento acontece "tipicamente dentro de alguns dias" depois da expiração, nas palavras da documentação. É a faxina gratuita que não marca dia: um dia ela vem. Até lá, o item expirado continua aparecendo em `GetItem`, `Query` e `Scan`, e por isso o `trajeto` filtra por `expira_em`. TTL controla o tamanho da tabela; regra de negócio confere a data no código.

Atualizar um item expirado antes da limpeza, estendendo ou removendo o `expira_em`, cancela a expiração; a documentação recomenda uma condition expression quando o item não deveria ressuscitar. Sessão zumbi é um filme que ninguém quer ver. E as exclusões do TTL aparecem no Stream marcadas como feitas pelo serviço.

No Rangô, o histórico fica 24 horas, e o item `ATUAL` some 24 horas depois da conclusão. Localização de entregador é dado pessoal, e a LGPD agradece quando ele não fica guardado sem motivo.

O painel do restaurante ganha um login

Hoje, quem adivinhar o nome `cantina-da-nona` vê onde estão os entregadores do restaurante. A segurança depende de ninguém pensar no nome que está escrito na fachada. O painel precisa de login, e sessão é um caso clássico para o DynamoDB: chave conhecida, `GetItem` e prazo de validade. Uma tabela do domínio do painel guarda usuários e sessões, logo abaixo da `TabelaEntregas`:

```
        tabela_painel = dynamodb.Table(
            self,
            "TabelaPainel",
            partition_key=dynamodb.Attribute(name="pk",
type=dynamodb.AttributeType.STRING),
            billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
            time_to_live_attribute="expira_em",
            removal_policy=RemovalPolicy.DESTROY,
        )
        tabela_painel.grant_read_write_data(role_app)
```

No container da API, o `environment` ganha `"TABELA_PAINEL": tabela_painel.table_name`. O `backend/requirements.txt` ganha o `argon2-cffi`, com a versão fixada, e o código fica em `backend/app/painel.py`:

```
import hashlib
import os
import secrets
import time

import boto3
from argon2 import PasswordHasher
from argon2.exceptions import VerificationError
from fastapi import APIRouter, Header, HTTPException
from pydantic import BaseModel

tabela = boto3.resource(
    "dynamodb", region_name=os.environ.get("AWS_REGION", "us-east-1")
).Table(os.environ.get("TABELA_PAINEL", "rango-painel-local"))

router = APIRouter()
hasher = PasswordHasher()
HASH_DESCARTAVEL = hasher.hash(secrets.token_urlsafe(16))
SESSAO_SEGUNDOS = 8 * 3600
```

```
class Login(BaseModel):
    email: str
    senha: str


def chave_sessao(token: str) -> dict:
    return {"pk": "SESSAO#" + hashlib.sha256(token.encode()).hexdigest()}


@router.post("/painel/entrar")
def entrar(login: Login):
    usuario = tabela.get_item(Key={"pk": f"USUARIO#{login.email.lower()}"}).get("Item")
    try:
        hasher.verify(usuario["hash_senha"] if usuario else HASH_DESCARTAVEL,
login.senha)
    except VerificationError:
        usuario = None
    if usuario is None:
        raise HTTPException(401, "e-mail ou senha inválidos")
    token = secrets.token_urlsafe(32)
    tabela.put_item(
        Item={
            **chave_sessao(token),
            "restaurante": usuario["restaurante"],
            "expira_em": int(time.time()) + SESSAO_SEGUNDOS,
        }
    )
    return {"token": token, "restaurante": usuario["restaurante"]}


def restaurante_logado(authorization: str = Header(default="")) -> str:
    token = authorization.removeprefix("Bearer ")
    sessao = tabela.get_item(Key=chave_sessao(token)).get("Item") if token else None
    if sessao is None or sessao["expira_em"] < time.time():
        raise HTTPException(401, "sessão inválida")
    return sessao["restaurante"]
```

A tabela guarda só o hash Argon2 da senha, feito por uma biblioteca estabelecida. Para e-mail inexistente, o código verifica contra um hash descartável, e a resposta tem o mesmo tempo e a mesma mensagem. Do token, a tabela guarda só o SHA-256; quem a ler não se passa por ninguém. E, como o TTL apaga com atraso, a sessão vencida é recusada pelo código. Expulsar visitante não pode depender da faxina que não marca dia.

No `entregas.py`, a rota do restaurante passa a exigir a sessão e a conferir o dono:

```
from fastapi import Depends

from app.painel import restaurante_logado


@router.get("/restaurantes/{restaurante}/entregas")
def entregas_do_restaurante(
    restaurante: str = Restaurante, logado: str = Depends(restaurante_logado)
):
    if logado != restaurante:
        raise HTTPException(403, "sessão de outro restaurante")
```

```
    resposta = tabela.query(
        # ... igual ao anterior ...
```

O `painel_router` entra no `main.py`. No React, o painel guarda o token em memória e o manda no cabeçalho `Authorization: Bearer`. É um login mínimo, sem MFA nem limite de tentativas, e precisa de revisão de alguém de segurança antes da produção; o capítulo 23 volta a ele.

Streams: a entrega concluída chega ao Postgres

O pedido no Postgres precisa saber que foi entregue. O caminho óbvio é o `concluir` fazer um `UPDATE` no Postgres depois de gravar no DynamoDB. É a escrita dupla do capítulo 5: se o processo morrer entre as duas, os dois bancos discordam para sempre. É o velho problema de quem tem dois relógios em casa e nunca sabe que horas são.

O **DynamoDB Streams** resolve isso do lado do banco. Ele registra cada alteração de item num log ordenado por item, disponível por 24 horas. Cada registro carrega só as chaves (`KEYS_ONLY`), o item depois (`NEW_IMAGE`), o item antes (`OLD_IMAGE`) ou os dois (`NEW_AND_OLD_IMAGES`). Escolhemos os dois para detectar a transição de `em_rota` para `entregue`.

É o *transactional outbox* do capítulo 5 entregue pelo banco: a gravação e o registro no Stream acontecem juntos. A Lambda lê o Stream por um *event source mapping*, um processo do serviço Lambda que junta registros em lotes e chama a função, e essas leituras não são cobradas.

A função fica em `backend/lambdas/rango_lambda/entrega_concluida.py`:

```
from banco import conexao


def handler(event, context):
    with conexao().cursor() as cur:
        for registro in event["Records"]:
            pedido_id = registro["dynamodb"]["Keys"]["pk"]["S"].removeprefix("PEDIDO#")
            cur.execute(
                "UPDATE pedidos SET status = 'entregue' WHERE id = %s AND status =
'pago'",
                (pedido_id,),
            )
    return {"processados": len(event["Records"])}
```

O registro chega no formato bruto do DynamoDB, com o tipo de cada valor explícito (`{"S": "PEDIDO#..."}` , onde `S` quer dizer string). O `UPDATE` com `AND status = 'pago'` é idempotente: o Stream entrega cada registro pelo menos uma vez, e uma reentrega não faz nada.

Na stack, a função usa o auxiliar `funcao` do capítulo 19, que já cuida da layer, da VPC e do segredo do banco:

```
        entrega_concluida = funcao(
            "EntregaConcluida",
            "entrega-concluida",
            "entrega_concluida.handler",
            memory_size=256,
            timeout=Duration.seconds(30),
        )
        entrega_concluida.add_event_source(
            fontes.DynamoEventSource(
                tabela_entregas,
                starting_position=lambda_.StartingPosition.LATEST,
                batch_size=25,
                retry_attempts=5,
                bisect_batch_on_error=True,
                filters=[
                    lambda_.FilterCriteria.filter(
                        {
                            "eventName": lambda_.FilterRule.is_equal("MODIFY"),
                            "dynamodb": {
                                "OldImage": {"status": {"S":
lambda_.FilterRule.is_equal("em_rota")}},
                                "NewImage": {"status": {"S":
lambda_.FilterRule.is_equal("entregue")}},
                            },
                        }
                    )
                ],
            )
        )
```

O filtro economiza dinheiro. Sem ele, a função seria chamada 40 vezes por segundo para ignorar posições: uma Lambda paga para passar o dia dizendo "não é comigo", emprego com que muita gente sonha. Com ele, o event source mapping descarta antes de chamar, e só a transição `em_rota` → `entregue` passa.

O `bisect_batch_on_error` divide o lote ao meio quando a função falha, para isolar o registro problemático. Depois da quinta tentativa, o registro é descartado em silêncio, porque não há destino para falhas. Guarde esse detalhe para a ponte. O `DynamoEventSource` dá à role a permissão de ler o Stream, e o `sg_lambdas` do capítulo 19 já alcança o banco.

O deploy e o primeiro trajeto

O deploy começa reativando os workflows e enviando o commit de `backend/` e `rango-infra/` para a `main`:

```
gh workflow enable ci.yaml
gh workflow enable renovar-tasks.yaml
```

Depois da sua aprovação no environment `producao`, o pipeline publica a imagem nova da API e o `cdk deploy` cria a tabela, o endpoint e a função. Os passos do webhook do capítulo 19 deixam um pedido pago em `$PEDIDO`. Com ele, o script abaixo faz o papel do entregador, sem capacete e sem pegar chuva:

```
API=https://api.rango.com.br

curl -s -X POST "$API/entregas/$PEDIDO/iniciar" \
  -H "Content-Type: application/json" -d '{"entregador": "joao-p"}'

agora_ms() { echo $(( $(date +%s) * 1000 )); }
for ponto in "-8.0631 -34.8711" "-8.0612 -34.8730" "-8.0598 -34.8752"; do
  set -- $ponto
  curl -s -X POST "$API/entregas/$PEDIDO/posicao" \
    -H "Content-Type: application/json" \
    -d "{\"lat\": $1, \"lon\": $2, \"registrada_em\": $(agora_ms)}"
  sleep 2
done
```

Para o papel do restaurante, o painel precisa de uma usuária cadastrada. O hash é gerado no terminal de quem faz o cadastro (com `pip install argon2-cffi`), e a senha não aparece no histórico do shell:

```
HASH=$(python3 -c "import getpass; from argon2 import PasswordHasher;
print(PasswordHasher().hash(getpass.getpass()))")
PAINEL=$(aws dynamodb list-tables \
  --query "TableNames[?contains(@, 'TabelaPainel')] | [0]" --output text)
aws dynamodb put-item --table-name "$PAINEL" --item \
  "{\"pk\":{\"S\":\"USUARIO#ana@cantinadanona.com.br\"},\"restaurante\":
{\"S\":\"cantina-da-nona\"},\"hash_senha\":{\"S\":\"$HASH\"}}"

read -rs -p "Senha: " SENHA; echo
TOKEN=$(jq -n --arg s "$SENHA" '{email: "ana@cantinadanona.com.br", senha: $s}' \
  | curl -s -X POST "$API/painel/entrar" -H "Content-Type: application/json" -d @- | jq
-r .token)
unset SENHA
```

E os papéis do cliente e do restaurante:

```
curl -s "$API/entregas/$PEDIDO" | jq .
curl -s "$API/entregas/$PEDIDO/trajeto" | jq -c .
curl -s "$API/restaurantes/cantina-da-nona/entregas" -H "Authorization: Bearer $TOKEN" |
jq -c .
```

```
{
  "status": "em_rota",
  "entregador": "joao-p",
  "posicao": {"lat": -8.0598, "lon": -34.8752},
  "registrada_em": 1792094465000
}
```

```
[{"lat":-8.0631,"lon":-34.8711},{"lat":-8.0612,"lon":-34.873},
{"lat":-8.0598,"lon":-34.8752}]
```

```
[{"pedido_id":"5f0d2c1a-9e3b-4b7f-8a61-0c4e2d9b7a35","entregador":"joao-p","posicao":
{"lat":-8.0598,"lon":-34.8752}}]
```

Agora o teste do capítulo 5: uma posição com um instante anterior ao da última.

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST "$API/entregas/$PEDIDO/posicao" \
  -H "Content-Type: application/json" \
  -d '{"lat": -8.1000, "lon": -34.9000, "registrada_em": 1000000000000}'
curl -s "$API/entregas/$PEDIDO" | jq -c .posicao
```

```
204
{"lat":-8.0598,"lon":-34.8752}
```

A posição atrasada foi recusada pela condição, e o mapa não pulou. O instante `1000000000000` cai em setembro de 2001, um atraso de uns 25 anos, demais até para entrega em dia de chuva. Por fim, a entrega é concluída, e a API de pedidos mostra o que ficou no Postgres:

```
curl -s -X POST "$API/entregas/$PEDIDO/concluir" | jq -c .
sleep 5
curl -s "$API/pedidos/$PEDIDO" | jq .status
curl -s "$API/restaurantes/cantina-da-nona/entregas" -H "Authorization: Bearer $TOKEN"
```

```
{"pedido_id":"5f0d2c1a-9e3b-4b7f-8a61-0c4e2d9b7a35","status":"entregue"}
"entregue"
[]
```

O Stream levou a mudança até a Lambda, que atualizou o Postgres. E a entrega saiu do índice do restaurante, porque perdeu o atributo `restaurante`. Uma segunda chamada ao `concluir` recebe 409 (a mesma pizza não se entrega duas vezes, por mais que o cliente gostasse); a Lambda não é chamada, porque o filtro só aceita a transição a partir de `em_rota`.

Olhando a tabela pela CLI

A CLI mostra quanto cada operação custa, o que ajuda a fixar o modelo. O exemplo pega o nome da tabela e faz a `Query` do trajeto, pedindo o consumo:

```
TABELA=$(aws dynamodb list-tables \
  --query "TableNames[?contains(@, 'TabelaEntregas')] | [0]" --output text)

aws dynamodb query --table-name "$TABELA" \
  --key-condition-expression "pk = :pk AND begins_with(sk, :pos)" \
  --expression-attribute-values "{\":pk\": {\"S\": \"PEDIDO#$PEDIDO\"}, \":pos\":
{\"S\": \"POS#\"}}" \
  --return-consumed-capacity TOTAL \
  --query "{itens: Count, consumo: ConsumedCapacity.CapacityUnits}"
```

```
{
    "itens": 3,
    "consumo": 0.5
}
```

Três itens pequenos cabem em 4 KB, então a leitura eventualmente consistente custou meia unidade. Com o preço de referência, dá para repetir essa leitura mais de dez milhões de vezes antes de gastar um dólar, e mesmo assim alguém vai perguntar no Slack se não dá para cachear. Agora o jeito errado de responder a pergunta do restaurante, com `Scan` e filtro:

```
aws dynamodb scan --table-name "$TABELA" \
  --filter-expression "restaurante = :r" \
  --expression-attribute-values '{":r": {"S": "cantina-da-nona"}}' \
  --return-consumed-capacity TOTAL \
  --query "{itens: Count, lidos: ScannedCount, consumo: ConsumedCapacity.CapacityUnits}"
```

A comparação que importa é entre `Count` (itens devolvidos) e `ScannedCount` (itens lidos e cobrados). Numa noite de jantar, o `ScannedCount` seria 576 mil para devolver uma dúzia, que é o que a `Query` no índice esparso lê. Quase ninguém tem mais lista telefônica em casa, mas o `Scan` mantém viva a experiência de folhear todas as páginas para achar um número.

Quanto isso custa

Com os preços de referência de outubro de 2026 em us-east-1 (aproximados; confira a página oficial), e assumindo 200 entregadores durante quatro horas por dia, todo dia:

Item	Conta	Por mês
Posições recebidas	40/s × 14.400 s × 30 dias	17,3 milhões
Escritas	3 por posição (`ATUAL` , índice, histórico) = 51,8 milhões de unidades × US$ 0,625/milhão	~US$ 32
Leituras do cliente	17,3 milhões × 0,5 unidade × US$ 0,125/milhão	~US$ 1
Armazenamento	< 1 GB, graças ao TTL	dentro dos 25 GB gratuitos
Lambda e Stream	algumas centenas de entregas por dia	centavos

Uns US$ 33 por mês, quase tudo em escritas, um terço delas no índice. Menos do que a pizza de sexta do time, o que, para uma linha da fatura da AWS, é quase um elogio. Para baixar, grave o histórico a cada três posições ou projete só as chaves no índice.

Desmontando

```
cd rango-infra
cdk destroy RangoProducao
gh workflow disable ci.yaml
gh workflow disable renovar-tasks.yaml
```

As duas tabelas vão embora com o `DESTROY`, junto com o índice, o Stream e a usuária do painel, que você cadastra de novo quando recriar a stack. O banco deixa um snapshot novo, como sempre. A coleção de snapshots do Rangô já rivaliza com o álbum de figurinhas da Copa.

Quando não usar o DynamoDB

Depois da API respondendo em milissegundos, aparece a tentação de mover os pedidos para lá também. Resista. O DBA interior acordou do desmaio e trouxe argumentos.

O resumo de vendas do capítulo 19 é um `GROUP BY restaurante` com `sum` e `avg`. No DynamoDB, ele vira um `Scan` com agregação em Python, ou uma tabela de totais mantida por um Stream. Relatórios e perguntas que o Bruno ainda não fez pedem SQL. E ele não vai aceitar "preciso de um GSI e duas semanas" como resposta para "quanto vendemos ontem?".

Padrões de acesso mudam. No Postgres, uma pergunta nova é um `CREATE INDEX`; no DynamoDB, pode ser um GSI ou um backfill em milhões de itens.

Relações e restrições também. Chave estrangeira, `UNIQUE` fora da chave, `CHECK` e cascata viram código seu, e código seu vem com bugs seus. Existem transações (`TransactWriteItems`), que custam o dobro e têm limite de itens; para pedido com cobrança, cupom e estoque, o Postgres faz melhor.

O DynamoDB brilha com acesso sempre por chave, padrões conhecidos, escritas em volume alto ou imprevisível e vontade de zero operação: sessões, carrinhos, rastreamento, contadores, chaves de idempotência. O Rangô agora usa os dois bancos, cada um onde é bom.

Armadilhas e casos reais

> **Armadilha:** Partition key com pouca variedade. Com `pk = "2026-10-15"`, todas as escritas do dia caem numa partição, que aguenta cerca de 1.000 unidades de escrita por segundo. Com 5.000 entregadores, vem throttling mesmo em on-demand, com

a tabela longe da cota. Acrescente um sufixo (`2026-10-15#7`) e leia os sufixos em paralelo.

Armadilha: Esquecer a paginação. `Query` e `Scan` devolvem páginas limitadas por tamanho, com `LastEvaluatedKey` quando há mais. O `entregas_do_restaurante` assume que cabe numa página, o que vale para o Rangô; em consultas que crescem, use os paginators do boto3.

Armadilha: O Stream de uma tabela comum aceita no máximo dois leitores simultâneos por shard (em global tables, um). A terceira Lambda pendurada nele sofre throttling. Para muitos consumidores, um leitor publica num barramento de eventos, assunto do próximo capítulo.

Na vida real: Em 20 de setembro de 2015, o DynamoDB de us-east-1 passou horas com erros que chegaram a cerca de 55% das requisições. Os servidores de armazenamento consultam um serviço interno de metadados para saber de quais partições cuidam, e a adoção rápida de GSIs tinha inchado essas respostas até perto do limite de tempo. Uma falha breve de rede fez muitos servidores perguntarem ao mesmo tempo, e as retentativas mantiveram o serviço sobrecarregado. É a tempestade de retentativas do capítulo 5 dentro da própria AWS. Saber que acontece com eles também é um consolo estranho, mas é um consolo. "Gerenciado" quer dizer que outra equipe cuida da operação; a falha continua possível.

Resumo

- O DynamoDB é um banco chave-valor gerenciado, acessado por HTTPS com IAM, sem conexão persistente: combina com Lambda e com cargas de muitas escritas pequenas.
- A partition key escolhe a partição pelo hash e precisa espalhar o tráfego (teto de cerca de 3.000 leituras e 1.000 escritas por segundo por partição). A sort key ordena os itens dentro da item collection.
- O desenho começa pelos padrões de acesso. Single-table design guarda junto o que é lido junto, com chaves como `PEDIDO#id` e `POS#instante`.
- GSI responde perguntas com outra chave; índice esparso indexa só os itens que têm o atributo da chave. Escritas no índice são cobradas à parte.
- On-demand cobra por requisição e é o ponto de partida; provisionado reserva capacidade e compensa com tráfego estável e conhecido.
- Condition expressions fazem checagem e escrita atômicas, e resolveram a posição fora de ordem.

- TTL apaga sem custo de escrita, em até alguns dias; filtre os expirados na leitura.
- Streams registram cada alteração por 24 horas; com Lambda e filtro, viram um outbox pronto.
- O Rangô usa Postgres para pedidos e relatórios e DynamoDB para o rastreamento e as sessões do painel, com gateway endpoint gratuito na VPC.

O pedido que ninguém enxerga inteiro

O caminho que um pedido do Rangô percorre hoje é este. O `POST /pedidos` grava no Postgres e chama o gateway de pagamento, esperando a resposta. Um stream no Valkey, inventado no capítulo 5, avisa o restaurante e manda o e-mail, com mensagens pendentes presas ao nome de tarefas que já não existem, como recados na geladeira de uma casa que já mudou de endereço. O webhook do PIX chega por uma Lambda. A conclusão da entrega chega por outra, pelo Stream do DynamoDB, e se ela falhar cinco vezes o registro some sem deixar rastro.

Cada peça funciona, e ninguém enxerga o pedido de ponta a ponta. Se o restaurante recusar um pedido já pago, quem devolve o dinheiro? Se o e-mail falhar, quem tenta de novo, e quando alguém fica sabendo? O Bruno já pediu um SMS para o cliente quando o entregador sair, e isso seria mais uma chamada pendurada no meio da cadeia. A arquitetura, neste ponto, lembra uma extensão ligada em outra extensão, ligada num benjamim.

O próximo capítulo troca essa cadeia por mensagens e eventos de verdade: Amazon SQS com dead letter queue e visibility timeout, Amazon SNS para avisar vários interessados de uma vez, Amazon EventBridge como barramento e agendador com fuso horário, e AWS Step Functions para desenhar o fluxo do pedido como uma máquina de estados que mostra, a qualquer momento, em que passo cada pedido está.

Referências

- Amazon DynamoDB, Developer Guide: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html
- Partições e chaves de partição, burst e adaptive capacity: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/burst-adaptive-capacity.html
- Cotas do DynamoDB (tabelas, índices, throughput, Streams): https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ServiceQuo

tas.html

- Time to Live: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html
- TTL e Streams, filtro para exclusões do serviço: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/time-to-live-ttl-streams.html
- DynamoDB Streams: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html
- StreamSpecification no CloudFormation: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-dynamodb-table-streamspecification.html
- Preços on-demand: https://aws.amazon.com/dynamodb/pricing/on-demand/
- Redução de preços do on-demand e das global tables (nov/2024): https://aws.amazon.com/about-aws/whats-new/2024/11/amazon-dynamo-db-reduces-prices-on-demand-throughput-global-tables
- Blog da AWS sobre a redução de preços: https://aws.amazon.com/blogs/database/new-amazon-dynamodb-lowers-pricing-for-on-demand-throughput-and-global-tables/
- Chaves compostas de vários atributos em GSIs (nov/2025): https://aws.amazon.com/about-aws/whats-new/2025/11/amazon-dynamodb-multi-attribute-composite-keys-global-secondary-indexes
- Palavras reservadas do DynamoDB: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ReservedWords.html
- Filtros de eventos da Lambda: https://docs.aws.amazon.com/lambda/latest/dg/invocation-eventfiltering.html
- Resumo do incidente do DynamoDB em us-east-1 (20/set/2015): https://aws.amazon.com/message/5467D2/

CAPÍTULO 21

SQS, SNS, EventBridge e Step Functions

No capítulo anterior, o rastreamento das entregas saiu do Postgres e foi para o DynamoDB, e o Stream da tabela passou a avisar uma Lambda quando a entrega termina. Na despedida, olhamos o caminho de um pedido e vimos um quebra-cabeça: chamada síncrona na API, um stream no Valkey com mensagens órfãs, um webhook numa Lambda, um Stream que descarta registros depois de cinco falhas. Neste capítulo, o pedido ganha um fluxo único, que alguém consegue enxergar de ponta a ponta. Hoje, quem enxerga o pedido inteiro é só o cliente, com fome e irritado.

Terça, 20h40, sem gás na cozinha

O botijão do Sabor do Sertão acabou no meio do jantar. O Bruno ligou para o restaurante, confirmou que a cozinha só voltaria no dia seguinte e foi olhar os pedidos. Eram seis, todos pagos por PIX, todos com status `pago` no banco, e nenhum ia sair. O sistema, impecável, continuava mostrando seis pedidos pagos e felizes.

O worker `restaurante` tinha avisado o restaurante e marcado a mensagem como feita. Missão cumprida, do ponto de vista dele. Ninguém no sistema sabe o que fazer quando o restaurante recusa, então o Bruno abriu o painel do gateway e estornou os seis pagamentos à mão, um por um. Errou um deles: estornou duas vezes o mesmo pedido de R$ 62,40 e não estornou o de R$ 48,00. Ficou provado que uma pessoa às nove da noite, com fome e seis abas abertas, é o pior sistema distribuído já inventado.

Na mesma noite, um deploy da API substituiu as tasks do worker de e-mail. Três mensagens ficaram pendentes no stream do Valkey, presas ao nome de uma task que já não existia, e três clientes nunca receberam a confirmação. Ninguém ficou sabendo até a reclamação chegar pelo Instagram, o sistema de monitoramento mais confiável do mercado e, de longe, o mais barulhento.

E o Bruno quer mais. Ele pediu um SMS para o cliente quando o entregador sair, e a única forma de fazer isso hoje é pendurar mais uma chamada no meio do código, com mais um jeito de falhar. O código já parece varal de roupa em dia de ventania.

Os conceitos você já tem desde o capítulo 5. Falta a versão gerenciada de cada um e uma peça nova: alguém que conduza o pedido passo a passo e saiba desfazer o que já foi feito quando um passo falha.

Quatro serviços, quatro trabalhos

A AWS tem um serviço para cada papel do capítulo 5, e a confusão mais comum é achar que eles competem entre si. Os nomes ajudam pouco: SQS e SNS diferem por uma letra, e ninguém lembra qual é qual antes do segundo café. Eles se completam:

Pergunta	Serviço	Analogia do capítulo 5
Quem guarda o trabalho até alguém processar?	Amazon SQS	a fila, a comanda no trilho
Como avisar vários interessados de uma vez?	Amazon SNS	pub/sub, fan-out
Como rotear eventos pelo conteúdo e agendar tarefas?	Amazon EventBridge	barramento de eventos, cron
Quem conduz um processo de vários passos, com espera e compensação?	AWS Step Functions	(novo) o orquestrador

Amazon SQS: a fila

O Amazon SQS (Simple Queue Service) é a fila gerenciada mais antiga da AWS. Você cria uma fila, produtores mandam mensagens, consumidores pedem mensagens. Sem servidor, sem cluster, sem versão para atualizar. Ele é mais velho que o próprio EC2: na família AWS, é o avô que já viu de tudo e não se impressiona com mais nada.

O detalhe que muda tudo é como o consumidor confirma o processamento. No SQS, receber uma mensagem não a remove da fila. Ela fica invisível para os outros consumidores durante um prazo, o *visibility timeout*. Se o consumidor terminar e chamar `DeleteMessage` dentro do prazo, a mensagem some. Se ele morrer, travar ou demorar demais, o prazo vence e a mensagem reaparece para qualquer outro consumidor. É a comanda que volta para o trilho quando o cozinheiro sai para atender o telefone e não volta.

Você já usou essa ideia no capítulo 5, com outro nome: é um `SET NX EX` aplicado à mensagem, um lock com expiração. E ela resolve sozinha o problema dos pendentes órfãos do Valkey. No stream, a mensagem pendente pertencia a um consumidor com

nome, e se esse consumidor desaparecesse alguém precisava rodar `XAUTOCLAIM`. No SQS, a mensagem não pertence a ninguém; o prazo vence e ela volta para a fila, sem inventário nem briga de herança.

O prazo padrão é de 30 segundos e vai de zero a 12 horas. Ele precisa ser maior que o tempo de processamento, ou a mesma mensagem vai ser processada duas vezes em paralelo: dois cozinheiros fazendo o mesmo prato ao mesmo tempo, e o cliente pagando um.

Cada mensagem guarda um contador, o `ApproximateReceiveCount`. Na fila, você configura uma *redrive policy*: depois de N recebimentos sem `DeleteMessage`, a mensagem vai para outra fila, a dead letter queue. É a DLQ do capítulo 5, agora como recurso de configuração em vez de código seu.

Alguns números para ter na cabeça (aproximados, conferidos em outubro de 2026):

Item	Valor
Tamanho máximo da mensagem	1 MiB (era 256 KB até pouco tempo)
Retenção	padrão de 4 dias, de 60 segundos a 14 dias
Visibility timeout	padrão de 30 segundos, máximo de 12 horas
Atraso na entrega (*delay*)	até 15 minutos
Lote	até 10 mensagens por chamada

Para mensagens maiores, guarde o conteúdo no S3 e mande só a referência. No Rangô, a mensagem carrega só o ID do pedido. Ninguém precisa mandar a foto da pizza pela fila.

Existem dois tipos de fila. A *standard* tem vazão praticamente ilimitada e entrega cada mensagem pelo menos uma vez, sem garantia de ordem. A *FIFO* ordena dentro de um grupo: mensagens com o mesmo `MessageGroupId` saem em ordem, e grupos diferentes andam em paralelo. O preço é a vazão: sem o modo de alta vazão, 300 chamadas por segundo por ação; com ele, até 70.000 em us-east-1 e 4.500 em sa-east-1. Para o Rangô, a standard basta: ordem entre e-mails de pedidos diferentes não importa, e a duplicata, possível nos dois tipos, a idempotência resolve. Quem já recebeu a mesma notificação de entrega três vezes sabe que o "pelo menos uma vez" leva o "pelo menos" muito a sério.

O consumidor mais simples de uma fila é uma Lambda, pelo *event source mapping* prometido no capítulo 19: um processo da AWS consulta a fila com *long polling* (a consulta espera até 20 segundos por mensagens), monta lotes e chama a função. Sem erro, o mapping apaga o lote. Com exceção, o lote inteiro volta para a fila, e uma mensagem estragada faz as outras nove serem processadas de novo. É a turma inteira de castigo por causa de um aluno. A saída é o `ReportBatchItemFailures`: a função devolve a

lista das que falharam, e só elas voltam. A documentação da Lambda ainda pede um visibility timeout de pelo menos seis vezes o timeout da função, mais a janela de agrupamento do lote.

Amazon SNS: um aviso, vários interessados

O Amazon SNS (Simple Notification Service) faz pub/sub. Você cria um *tópico*, publica mensagens nele, e cada *assinatura* recebe uma cópia. Assinaturas podem ser filas SQS, Lambdas, endpoints HTTPS, e-mail e SMS. Você já criou um tópico no capítulo 7, o `alertas-cobranca`, com uma assinatura de e-mail.

O SNS empurra a mensagem e não guarda nada; com um assinante em apuros, ele tenta por um tempo e desiste, como o entregador que toca o interfone algumas vezes e vai embora sem deixar aviso. Por isso o padrão de mercado é SNS na frente e uma fila SQS atrás de cada consumidor, que guarda até ele dar conta, como a caixa de correio que dispensa o carteiro de esperar você chegar em casa. A entrega do SNS para o SQS não tem cobrança de SNS; você paga só as requisições da fila.

Cada assinatura pode ter uma *filter policy*. A publicação leva atributos (pense em cabeçalhos HTTP da mensagem), e a assinatura declara quais quer receber:

```
{"tipo": ["pedido.confirmado", "pedido.cancelado"]}
```

A fila de e-mail recebe confirmações e cancelamentos. A de SMS, só a saída do entregador. Quem publica não conhece os assinantes, que é o desacoplamento do capítulo 5: amanhã, uma fila para o programa de fidelidade assina o mesmo tópico sem ninguém mexer no publicador. É o nível de desacoplamento que todo time sonha ter com o time vizinho.

> **Na vida real:** O SNS envia SMS de verdade, mas por trás dele está o AWS End User Messaging SMS, com registro de remetente e regras próprias de cada país. Numa conta nova, o envio começa restrito a números verificados: a AWS pede referências antes de deixar você mandar mensagem para estranhos. No livro, o SMS fica simulado, como o e-mail está desde o capítulo 5.

Amazon EventBridge: barramento e agenda

O Amazon EventBridge também entrega eventos a vários destinos, e a pergunta inevitável é por que existe mais um. Lançar mais um serviço que entrega mensagens é, ao que tudo indica, a especialidade da casa. A diferença está em quem decide o roteamento e com base em quê.

Um evento do EventBridge tem formato fixo: `source` (quem emitiu), `detail-type` (o tipo) e `detail` (o conteúdo, um JSON livre). Os eventos chegam a um *barramento* (*event bus*), e *regras* no barramento comparam cada evento com um padrão e mandam os que batem para até alguns alvos. O padrão olha qualquer campo, inclusive dentro do `detail`:

```
{
  "source": ["aws.states"],
  "detail-type": ["Step Functions Execution Status Change"],
  "detail": {"status": ["FAILED", "TIMED_OUT", "ABORTED"]}
}
```

Toda conta tem um barramento `default`, e nele a própria AWS publica eventos dos serviços: uma task do ECS que parou, um objeto novo no S3 (se você ligar), uma execução do Step Functions que falhou. Esses eventos de serviço não custam nada para entrar. Você também cria barramentos próprios e publica eventos seus, por US$ 1,00 por milhão (cada 64 KB contam como um evento). Entregar a um alvo na mesma conta não custa. Duas gratuidades seguidas no mesmo parágrafo da AWS é um acontecimento raro, digno de foto.

Comparando com o SNS:

	SNS	EventBridge
Filtro	atributos da mensagem (ou corpo, se configurado)	qualquer campo do evento
Eventos dos serviços da AWS	não	sim, no barramento `default`
Arquivar e reprocessar eventos	não	sim (archive e replay)
Alvos	filas, Lambdas, HTTPS, e-mail, SMS	dezenas de serviços da AWS, APIs externas
Latência e volume	menor latência, vazão muito alta	um pouco mais de latência

A regra prática que uso: SNS quando o publicador é seu e você quer fan-out simples e rápido; EventBridge quando o evento vem da AWS ou quando o roteamento depende do conteúdo. No Rangô, os dois aparecem.

O EventBridge Pipes liga uma fonte (um Stream do DynamoDB, uma fila) a um alvo, com filtro no meio, por US$ 0,40 por milhão de eventos. É a resposta à armadilha do capítulo 20: quando o Stream precisa de muitos leitores, um pipe publica num barramento, e as regras distribuem. O nome é literal: um cano com filtro, ligando uma ponta à outra, e nenhum encanador reclamaria.

E o EventBridge Scheduler. O capítulo 19 agendou o resumo de vendas com regras de cron em UTC, e você converteu 11h em 14h de cabeça. Converter fuso de cabeça é a segunda maior fonte de bugs da humanidade; a primeira é converter fuso no código. O Scheduler aceita fuso horário (`America/Sao_Paulo`), agendamentos únicos e recorrentes, e tem 14 milhões de invocações gratuitas por mês; depois, US$ 1,00 por milhão.

AWS Step Functions: o pedido como máquina de estados

SQS, SNS e EventBridge levam mensagens de um lugar a outro. Nenhum deles responde à pergunta do botijão: o restaurante recusou um pedido pago, quem estorna? Para isso é preciso alguém que lembre em que ponto o pedido está e decida o próximo passo, inclusive o passo de desfazer. Em outras palavras, um adulto na sala.

O AWS Step Functions faz isso com uma *máquina de estados*. Você descreve o processo em JSON, na Amazon States Language: estados `Task` (fazer algo, como chamar uma Lambda ou publicar no SNS), `Choice` (um `if`), `Wait`, `Parallel`, `Map`, `Succeed` e `Fail`. Cada pedido vira uma *execução*, com histórico de cada passo, entrada e saída, visível no console como um diagrama com o passo atual pintado. É o rastreio de encomenda que você sempre quis ter para os próprios sistemas.

Pense num reducer do React com efeitos colaterais, cujo estado vive na AWS e sobrevive a deploys, quedas de task e dias de espera: um `useReducer` que não perde nada quando alguém aperta F5 no datacenter. O tratamento de erro é declarativo. Um estado `Task` ganha `Retry` e `Catch`:

```
"ConfirmarRestaurante": {
  "Type": "Task",
  "Resource": "arn:aws:states:::lambda:invoke",
  "Retry": [{
    "ErrorEquals": ["ErroTemporario"],
    "IntervalSeconds": 5,
    "MaxAttempts": 4,
    "BackoffRate": 2,
    "JitterStrategy": "FULL"
  }],
  "Catch": [{"ErrorEquals": ["States.ALL"], "ResultPath": "$.erro", "Next":
"Estornar"}],
  "Next": "RestauranteAceitou"
}
```

É o backoff exponencial com full jitter do capítulo 5, sem uma linha de Python. Só JSON, o que, dependendo do seu dia, pode ser pior. O nome do erro é o nome da classe da exceção que a Lambda lançou. Se as tentativas acabam, o `Catch` desvia para outro estado.

Desviar para o estorno tem nome: *compensação*. Num banco de dados, uma transação desfaz tudo com `ROLLBACK`. Entre sistemas diferentes (o seu banco, o gateway, o restaurante), não existe rollback; cada passo feito precisa de um passo contrário, executado quando algo adiante falha. Essa sequência de passos com compensações é chamada de *saga*. O estorno é a compensação do pagamento. O nome saga é bem escolhido: quem já tentou cancelar uma assinatura por telefone sabe que desfazer um passo pode virar uma epopeia.

A outra peça que o pedido precisa é esperar. O pagamento chega por webhook, minutos depois; o entregador sai quando o prato fica pronto. O Step Functions resolve com o *callback*: um estado `Task` com o sufixo `.waitForTaskToken` gera um token, entrega esse token a quem você indicar e fica parado até alguém chamar `SendTaskSuccess` (ou `SendTaskFailure`) com ele. A execução esperando não consome nada além do registro. É o único funcionário que espera dois dias sem reclamar e sem pedir hora extra.

Existem dois tipos de máquina:

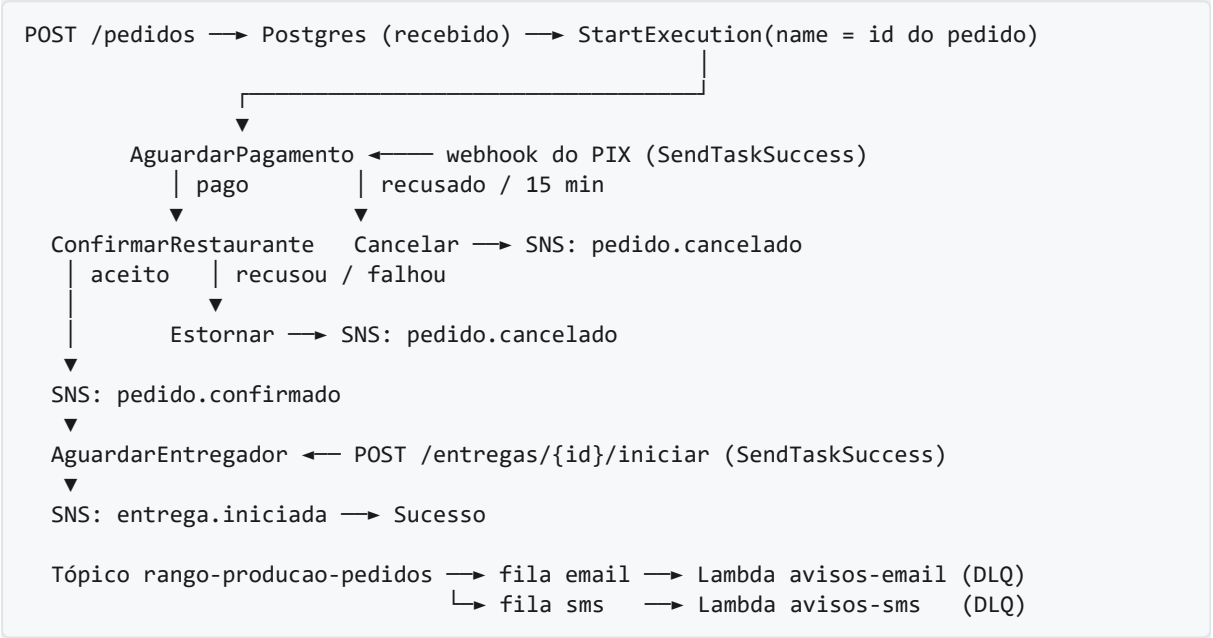
	Standard	**Express**
Para quê	processos longos, com espera e auditoria	fluxos curtos e de volume muito alto
Cobrança	US$ 0,025 por 1.000 transições de estado	US$ 1,00 por milhão de execuções, mais duração
Gratuito	4.000 transições por mês, sem prazo	não tem
Callback (`.waitForTaskToken`)	sim	não
Histórico por passo no console	sim	via CloudWatch Logs
`StartExecution` com mesmo nome	idempotente	não idempotente

O pedido do Rangô espera webhook e entregador, então vai de Standard. Express, para quem espera entregador, seria otimismo demais. A conta é por transição: cada passo concluído conta, e cada nova tentativa de um `Retry` também. Insistir custa, aqui como na vida.

Na prática

O desenho

O pedido passa a ter um fluxo só:



O stream do Valkey e os dois workers no ECS vão embora. O Valkey continua com a idempotência da API e o cache. Perde um cargo e mantém dois, o que, no mercado de hoje, é uma vitória.

O ponto de partida

Como no capítulo anterior, o ponto de partida é o snapshot deixado pelo `cdk destroy`, que este comando encontra:

```
aws rds describe-db-snapshots --snapshot-type manual \
  --query "reverse(sort_by(DBSnapshots,&SnapshotCreateTime))[0].DBSnapshotIdentifier" \
  --output text
```

O nome dele vai no `snapshot_db` do `rango-infra/app.py`. No `rango-infra/cdk.json`, o `context` ganha o e-mail que vai receber alertas:

```
"email_alertas": "alertas@rango.com.br"
```

A espera, guardada no DynamoDB

O callback cria um problema de corrida. O fluxo começa, chega ao `AguardarPagamento` e entrega o token a uma Lambda, que o guarda. Se o webhook do pagamento chegar antes de o token estar guardado (um cartão aprovado em dois segundos, uma Lambda em cold

start), quem avisa o fluxo? É o convidado que chega à festa antes do aniversariante e não tem a quem entregar o presente.

A solução é um item no DynamoDB por espera, com dois campos: `token`, gravado pelo fluxo, e `resultado`, gravado por quem traz a notícia. Cada lado grava o seu campo e lê o item inteiro na mesma operação atômica. Quem chegar por último encontra os dois e chama o `SendTaskSuccess`. Pela primeira vez na história, chegar atrasado vem com responsabilidade. O código fica em `backend/lambdas/rango_lambda/fluxo.py`:

```
import json
import os
import time

import boto3

REGIAO = os.environ.get("AWS_REGION", "us-east-1")
tabela = boto3.resource("dynamodb", region_name=REGIAO).Table(
    os.environ.get("TABELA_FLUXO", "rango-fluxo-local")
)
sfn = boto3.client("stepfunctions", region_name=REGIAO)
EsperaEncerrada = (
    sfn.exceptions.TaskTimedOut,
    sfn.exceptions.TaskDoesNotExist,
    sfn.exceptions.InvalidToken,
)
PRAZO_SEGUNDOS = 2 * 86400


def _registrar(pedido_id, etapa: str, campo: str, valor: str) -> dict:
    return tabela.update_item(
        Key={"pk": f"ESPERA#{pedido_id}#{etapa}"},
        UpdateExpression="SET #c = :v, expira_em = :e",
        ExpressionAttributeNames={"#c": campo},
        ExpressionAttributeValues={":v": valor, ":e": int(time.time()) +
PRAZO_SEGUNDOS},
        ReturnValues="ALL_NEW",
    )["Attributes"]


def _concluir(item: dict) -> None:
    if "token" in item and "resultado" in item:
        try:
            sfn.send_task_success(taskToken=item["token"], output=item["resultado"])
        except EsperaEncerrada:
            pass  # aviso repetido ou prazo vencido: nada a fazer


def guardar_token(pedido_id, etapa: str, token: str) -> None:
    _concluir(_registrar(pedido_id, etapa, "token", token))


def avisar(pedido_id, etapa: str, resultado: dict) -> None:
    _concluir(_registrar(pedido_id, etapa, "resultado", json.dumps(resultado)))
```

O `UpdateItem` com `ReturnValues="ALL_NEW"` grava e devolve o item como ficou, de uma vez; como o DynamoDB serializa gravações no mesmo item, só a segunda vê os dois campos. Um webhook reenviado gera um `SendTaskSuccess` repetido, que recebe erro de token encerrado e é ignorado. A Lambda da espera, `guardar_token.py`:

```
from fluxo import guardar_token


def handler(event, context):
    guardar_token(event["pedido_id"], event["etapa"], event["token"])
```

Sem Postgres, ela roda fora da VPC e sem layer. A API também avisa o fluxo, então o `fluxo.py` vai ser copiado para `backend/app/`, a mesma duplicação que o `banco.py` já tem.

Os passos que mexem no banco

`backend/lambdas/rango_lambda/confirmar_restaurante.py` simula a conversa com o tablet do restaurante, como o `externos.py` do capítulo 5:

```
import os
import random

from banco import conexao


class ErroTemporario(Exception):
    pass


def perguntar_ao_restaurante(restaurante: str) -> bool:
    # Simulado. Com FALHA_RESTAURANTE=0.5, metade das chamadas falha.
    if random.random() < float(os.environ.get("FALHA_RESTAURANTE", "0")):
        raise ErroTemporario("tablet do restaurante não respondeu")
    return restaurante != os.environ.get("RECUSAR", "")


def handler(event, context):
    pedido_id = event["pedido_id"]
    pedido = conexao().execute(
        "SELECT restaurante, status FROM pedidos WHERE id = %s", (pedido_id,)
    ).fetchone()
    if pedido["status"] == "confirmado":
        return {"aceito": True}  # repetição de um passo que já deu certo
    aceito = perguntar_ao_restaurante(pedido["restaurante"])
    if aceito:
        conexao().execute(
            "UPDATE pedidos SET status = 'confirmado' WHERE id = %s AND status =
'pago'",
            (pedido_id,),
        )
    return {"aceito": aceito}
```

A primeira checagem cobre o passo repetido: a Lambda grava `confirmado`, morre antes de responder, e o `Retry` chama de novo. Sem ela, o restaurante ouviria "aceita o pedido?" outra vez, como o garçom que pergunta se está tudo bem pela quarta vez.

E `cancelar_pedido.py`, que serve aos dois caminhos de cancelamento:

```
import json

from banco import conexao


def estornar_cobranca(pedido_id: str) -> None:
    # Simulado. No gateway real, a chave de idempotência impede estorno em dobro.
    print(json.dumps({"estorno": pedido_id, "idempotency_key": f"estorno-{pedido_id}"}))


def handler(event, context):
    pedido_id, estornar = event["pedido_id"], event["estornar"]
    if estornar:
        estornar_cobranca(pedido_id)
    novo = "estornado" if estornar else "cancelado"
    conexao().execute(
        "UPDATE pedidos SET status = %s WHERE id = %s"
        " AND status IN ('recebido', 'pago', 'recusado')",
        (novo, pedido_id),
    )
    return {"status": novo}
```

A chave `estorno-{pedido_id}` é o que teria salvado o Bruno na terça: se o passo rodar duas vezes, o gateway reconhece a chave e devolve o primeiro estorno em vez de fazer outro. E o cliente dos R$ 48,00 teria recebido o dinheiro sem precisar reclamar no Instagram.

Para os avisos, `avisos.py`, que atende o e-mail e o SMS:

```
import json
import os
import time

import boto3

tabela = boto3.resource("dynamodb").Table(os.environ["TABELA_FLUXO"])
JaEnviado = tabela.meta.client.exceptions.ConditionalCheckFailedException
CANAL = os.environ["CANAL"]


def enviar(tipo: str, pedido: dict) -> None:
    # Simulado, como o enviar_email do capítulo 5.
    if os.environ.get("FALHA_AVISOS") == "1":
        raise RuntimeError(f"provedor de {CANAL} fora do ar")
    print(json.dumps({"canal": CANAL, "tipo": tipo, "pedido_id": pedido["pedido_id"]}))


def processar(registro: dict) -> None:
    tipo = registro["messageAttributes"]["tipo"]["stringValue"]
    pedido = json.loads(registro["body"])
```

```
    chave = {"pk": f"AVISO#{CANAL}#{tipo}#{pedido['pedido_id']}"}
    try:
        tabela.put_item(
            Item={**chave, "expira_em": int(time.time()) + 7 * 86400},
            ConditionExpression="attribute_not_exists(pk)",
        )
    except JaEnviado:
        return
    try:
        enviar(tipo, pedido)
    except Exception:
        tabela.delete_item(Key=chave)
        raise


def handler(event, context):
    falhas = []
    for registro in event["Records"]:
        try:
            processar(registro)
        except Exception as erro:
            print(f"falha em {registro['messageId']}: {erro!r}")
            falhas.append({"itemIdentifier": registro["messageId"]})
    return {"batchItemFailures": falhas}
```

O `put_item` condicional é o `SET NX` do capítulo 5 no DynamoDB: marca o aviso antes de enviar, e a duplicata encontra a marca e sai. Se o envio falha, a marca é apagada. Sobra uma janela pequena (o ambiente morrer entre enviar e terminar) para um e-mail duplicado, o que é aceitável. Ninguém nunca abriu chamado por ter sido avisado duas vezes de que a comida está a caminho.

> **Dica:** O Powertools for AWS Lambda (Python), biblioteca de código aberto mantida pela AWS, tem um utilitário de idempotência que faz esse controle com uma tabela do DynamoDB e um decorator. Conheça antes de escrever o seu terceiro controle à mão.

A máquina de estados na stack

No `rango-infra/rango_infra/rango_infra_stack.py`, entram os imports:

```
from aws_cdk import (
    # ... os imports do capítulo 20 ...
    ArnFormat,
    TimeZone,
    aws_scheduler as scheduler,
    aws_scheduler_targets as agendados,
    aws_sns as sns,
    aws_sns_subscriptions as assinaturas,
    aws_sqs as sqs,
    aws_stepfunctions as sfn,
    aws_stepfunctions_tasks as tarefas,
)
```

O laço dos workers do capítulo 17 (o `for grupo in ("restaurante", "email")` inteiro) e as três `events.Rule` do resumo de vendas do capítulo 19 saem da stack. Depois do bloco do webhook, a tabela do fluxo, o endpoint e o tópico:

```
        tabela_fluxo = dynamodb.Table(
            self,
            "TabelaFluxo",
            partition_key=dynamodb.Attribute(name="pk",
type=dynamodb.AttributeType.STRING),
            billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
            time_to_live_attribute="expira_em",
            removal_policy=RemovalPolicy.DESTROY,
        )

        vpc.add_interface_endpoint(
            "StepFunctions",
            service=ec2.InterfaceVpcEndpointAwsService.STEP_FUNCTIONS,
            subnets=privadas,
        )

        topico_pedidos = sns.Topic(self, "TopicoPedidos", topic_name=f"rango-{ambiente}-
pedidos")
```

O endpoint serve à API e ao webhook, que chamam o Step Functions de dentro das subnets sem internet. Quando o Step Functions publica no SNS, a conversa não passa pela VPC.

As Lambdas que não usam o banco ficam fora da VPC, com um auxiliar próprio:

```
        def funcao_leve(id_: str, nome: str, handler: str, **extras) ->
lambda_.Function:
            fn = lambda_.Function(
                self,
                id_,
                function_name=f"rango-{ambiente}-{nome}",
                runtime=lambda_.Runtime.PYTHON_3_12,
                architecture=lambda_.Architecture.ARM_64,
                handler=handler,
                code=codigo_lambdas,
                environment={
                    "TABELA_FLUXO": tabela_fluxo.table_name,
                    **extras.pop("environment", {}),
                },
                log_group=logs.LogGroup(
                    self,
                    f"Logs{id_}",
                    retention=logs.RetentionDays.ONE_MONTH,
                    removal_policy=RemovalPolicy.DESTROY,
                ),
                **extras,
            )
            tabela_fluxo.grant_read_write_data(fn)
            return fn

        arn_maquina = self.format_arn(
            service="states",
            resource="stateMachine",
```

```
            resource_name=f"rango-{ambiente}-pedido",
            arn_format=ArnFormat.COLON_RESOURCE_NAME,
        )
        guardar = funcao_leve(
            "GuardarToken", "guardar-token", "guardar_token.handler",
            memory_size=256, timeout=Duration.seconds(10),
        )
        guardar.add_to_role_policy(
            iam.PolicyStatement(actions=["states:SendTaskSuccess"], resources=
[arn_maquina])
        )

        confirmar_restaurante = funcao(
            "FnConfirmarRestaurante", "confirmar-restaurante",
"confirmar_restaurante.handler",
            memory_size=256, timeout=Duration.seconds(15),
        )
        cancelar_pedido = funcao(
            "FnCancelarPedido", "cancelar-pedido", "cancelar_pedido.handler",
            memory_size=256, timeout=Duration.seconds(15),
        )
```

O prefixo `Fn` nos IDs evita colisão com os passos da máquina, que vão usar `ConfirmarRestaurante` como ID; dois constructs com o mesmo ID na mesma stack quebram o `cdk synth`. O ARN da máquina foi montado pelo nome, e não com `maquina.grant_task_response(guardar)`, por causa de uma dependência circular: a máquina precisa da função para existir, a função depende da policy da sua role, e a policy precisaria do ARN da máquina. O CloudFormation recusa o template. É o ovo e a galinha, versão infraestrutura como código. Montar o ARN pelo nome quebra o círculo.

Agora os passos e a máquina. Três auxiliares evitam repetição:

```
        def publicar(id_: str, tipo: str) -> tarefas.SnsPublish:
            return tarefas.SnsPublish(
                self,
                id_,
                topic=topico_pedidos,
                message=sfn.TaskInput.from_object(
                    {"pedido_id": sfn.JsonPath.string_at("$.pedido_id")}
                ),
                message_attributes={"tipo": tarefas.MessageAttribute(value=tipo)},
                result_path=sfn.JsonPath.DISCARD,
            )

        def esperar(id_: str, etapa: str, prazo: Duration) -> tarefas.LambdaInvoke:
            return tarefas.LambdaInvoke(
                self,
                id_,
                lambda_function=guardar,
                integration_pattern=sfn.IntegrationPattern.WAIT_FOR_TASK_TOKEN,
                payload=sfn.TaskInput.from_object({
                    "pedido_id": sfn.JsonPath.string_at("$.pedido_id"),
                    "etapa": etapa,
                    "token": sfn.JsonPath.task_token,
                }),
                task_timeout=sfn.Timeout.duration(prazo),
                result_path=f"$.{etapa}",
```

```
            )

        def cancelar(id_: str, estornar: bool) -> sfn.Chain:
            passo = tarefas.LambdaInvoke(
                self,
                id_,
                lambda_function=cancelar_pedido,
                payload=sfn.TaskInput.from_object({
                    "pedido_id": sfn.JsonPath.string_at("$.pedido_id"),
                    "estornar": estornar,
                }),
                result_path=sfn.JsonPath.DISCARD,
            )
            passo.add_retry(
                interval=Duration.seconds(10), max_attempts=6, backoff_rate=2,
                jitter_strategy=sfn.JitterType.FULL,
            )
            return passo.next(publicar(f"Publicar{id_}", "pedido.cancelado")).next(
                sfn.Succeed(self, f"Fim{id_}")
            )
```

O cancelamento termina em `Succeed`, e isso é deliberado. Um pedido cancelado é um desfecho previsto do negócio. `Fail` fica para o que ninguém previu, e é nele que o alerta vai disparar. Acordar alguém por causa de um pedido cancelado é o caminho mais curto para o alerta ser silenciado para sempre.

O fluxo, montado de trás para frente:

```
        estornar = cancelar("Estornar", estornar=True)
        cancelar_sem_pagamento = cancelar("Cancelar", estornar=False)

        entrega = (
            publicar("PublicarConfirmado", "pedido.confirmado")
            .next(esperar("AguardarEntregador", "entregador", Duration.hours(2)))
            .next(publicar("PublicarSaida", "entrega.iniciada"))
            .next(sfn.Succeed(self, "PedidoEmRota"))
        )

        confirmar = tarefas.LambdaInvoke(
            self,
            "ConfirmarRestaurante",
            lambda_function=confirmar_restaurante,
            payload_response_only=True,
            result_path="$.restaurante",
        )
        confirmar.add_retry(
            errors=["ErroTemporario"], interval=Duration.seconds(5), max_attempts=4,
            backoff_rate=2, jitter_strategy=sfn.JitterType.FULL,
        )
        confirmar.add_catch(estornar, errors=["States.ALL"], result_path="$.erro")
        confirmar.next(
            sfn.Choice(self, "RestauranteAceitou")
            .when(sfn.Condition.boolean_equals("$.restaurante.aceito", True), entrega)
            .otherwise(estornar)
        )

        pagamento = esperar("AguardarPagamento", "pagamento", Duration.minutes(15))
        pagamento.add_catch(cancelar_sem_pagamento, errors=["States.Timeout"],
result_path="$.erro")
```

```
        pagamento.next(
            sfn.Choice(self, "PagamentoAprovado")
            .when(sfn.Condition.string_equals("$.pagamento.status", "pago"), confirmar)
            .otherwise(cancelar_sem_pagamento)
        )

        maquina = sfn.StateMachine(
            self,
            "FluxoPedido",
            state_machine_name=f"rango-{ambiente}-pedido",
            definition_body=sfn.DefinitionBody.from_chainable(pagamento),
            timeout=Duration.hours(6),
        )
```

Lido de baixo para cima, o código é o desenho do começo da seção. O estado da execução começa como `{"pedido_id": "..."}`, e cada `result_path` acrescenta um campo (`pagamento`, `restaurante`, `erro`) que o `Choice` lê. O `task_timeout` do pagamento é o PIX nunca pago: em 15 minutos, `States.Timeout` e cancelamento sem estorno. O CDK gera a Amazon States Language e a role da máquina, com permissão para as três Lambdas e o tópico.

Avisos: SNS na frente, SQS atrás

Uma fila com DLQ por canal, cada uma assinando o tópico com o seu filtro, e uma Lambda consumindo cada fila:

```
        for canal, tipos in (
            ("email", ["pedido.confirmado", "pedido.cancelado"]),
            ("sms", ["entrega.iniciada"]),
        ):
            dlq = sqs.Queue(
                self, f"Dlq_{canal}",
                queue_name=f"rango-{ambiente}-{canal}-dlq",
                retention_period=Duration.days(14),
            )
            fila = sqs.Queue(
                self, f"Fila_{canal}",
                queue_name=f"rango-{ambiente}-{canal}",
                visibility_timeout=Duration.minutes(2),
                dead_letter_queue=sqs.DeadLetterQueue(queue=dlq, max_receive_count=5),
            )
            topico_pedidos.add_subscription(
                assinaturas.SqsSubscription(
                    fila,
                    raw_message_delivery=True,
                    filter_policy={"tipo":
sns.SubscriptionFilter.string_filter(allowlist=tipos)},
                )
            )
            avisos = funcao_leve(
                f"Avisos_{canal}", f"avisos-{canal}", "avisos.handler",
                memory_size=256, timeout=Duration.seconds(20),
                environment={"CANAL": canal},
            )
            avisos.add_event_source(
                fontes.SqsEventSource(fila, batch_size=10,
```

```
report_batch_item_failures=True)
            )
```

Função de 20 segundos, fila de dois minutos: a regra das seis vezes. Seis vezes vinte dá cento e vinte, e cento e vinte segundos dão dois minutos; fim da aula de aritmética. O `max_receive_count=5` segue a recomendação da AWS para filas lidas pela Lambda; com menos, uma rajada de throttling manda mensagens boas para a DLQ, que guarda por 14 dias, o máximo. Catorze dias bastam para alguém perceber, desde que alguém olhe. O `raw_message_delivery=True` entrega só o corpo publicado, sem o envelope do SNS, e transforma os atributos em atributos da mensagem SQS, onde o `avisos.py` lê o `tipo`.

De quebra, dá para fechar o furo do capítulo 20. O `DynamoEventSource` da `entrega_concluida` ganha um destino para as falhas:

```
        falhas_entregas = sqs.Queue(
            self, "FalhasEntregas",
            queue_name=f"rango-{ambiente}-entregas-dlq",
            retention_period=Duration.days(14),
        )
```

```
                # dentro do fontes.DynamoEventSource(...) do capítulo 20
                on_failure=fontes.SqsDlq(falhas_entregas),
```

Para streams, a fila recebe a descrição do lote que falhou (shard e números de sequência), sem o registro: o suficiente para reler o Stream em até 24 horas.

A API e o webhook avisam o fluxo

As permissões e variáveis, ainda na stack:

```
        maquina.grant_start_execution(role_app)
        maquina.grant_task_response(role_app)
        tabela_fluxo.grant_read_write_data(role_app)
        container_api = tarefa_api.default_container
        container_api.add_environment("MAQUINA_PEDIDO_ARN", maquina.state_machine_arn)
        container_api.add_environment("TABELA_FLUXO", tabela_fluxo.table_name)

        maquina.grant_task_response(webhook)
        tabela_fluxo.grant_read_write_data(webhook)
        webhook.add_environment("TABELA_FLUXO", tabela_fluxo.table_name)
```

Em `backend/app/main.py`, o `cache.xadd` sai do `criar_pedido`, e entra o início do fluxo:

```
import logging

import boto3

sfn = boto3.client("stepfunctions", region_name=os.environ.get("AWS_REGION", "us-east-
1"))
MAQUINA_PEDIDO = os.environ.get("MAQUINA_PEDIDO_ARN", "")
log = logging.getLogger(__name__)
```

```
def iniciar_fluxo(pedido_id: str) -> None:
    try:
        sfn.start_execution(
            stateMachineArn=MAQUINA_PEDIDO,
            name=pedido_id,
            input=json.dumps({"pedido_id": pedido_id}),
        )
    except Exception:
        log.exception("fluxo do pedido %s não iniciado; a varredura tenta de novo",
pedido_id)

# dentro de criar_pedido, no lugar de cache.xadd(STREAM_PEDIDOS, ...),
# depois do bloco with pool.connection() que faz o INSERT:
    iniciar_fluxo(str(pedido["id"]))
```

O nome da execução é o ID do pedido. Numa máquina Standard, repetir o `StartExecution` com o mesmo nome e a mesma entrada, enquanto a execução roda, devolve a mesma execução; com entrada diferente ou execução encerrada, vem `ExecutionAlreadyExists`. Por isso a entrada é mínima e montada sempre igual, como o avô que conta a mesma história com as mesmas palavras em todo almoço de domingo.

O `except` só registra o erro porque `INSERT` e `StartExecution` são a escrita dupla do capítulo 5, prometida para cá desde o capítulo 12 (nove capítulos de suspense, mais do que muita novela): se a task morrer entre as duas linhas, o pedido existe e o fluxo não. A resposta vem logo adiante.

O `backend/app/worker.py` e a constante `STREAM_PEDIDOS` saem do projeto. O `fluxo.py` ganha uma cópia em `backend/app/fluxo.py` e, em `backend/app/entregas.py`, a rota `iniciar` passa a exigir pedido confirmado e a avisar o fluxo:

```
from app.fluxo import avisar

# em iniciar(), a consulta passa a exigir o novo status
            "SELECT restaurante FROM pedidos WHERE id = %s AND status = 'confirmado'",

# e, antes do return:
    avisar(pedido_id, "entregador", {"entregador": inicio.entregador})
```

Na mesma lógica, o `UPDATE` da `entrega_concluida.py` passa de `AND status = 'pago'` para `AND status = 'confirmado'`. Os status do pedido agora são `recebido`, `pago`, `confirmado` e `entregue`, com as saídas `cancelado` e `estornado`.

No `webhook_pagamento.py` do capítulo 19, depois do `UPDATE` que encontrou o pedido:

```
from fluxo import avisar

# depois do bloco with conexao().cursor(), antes do return 200:
    avisar(pedido_id, "pagamento", {"status": status})
```

Agendas com fuso e a varredura

A cada cinco minutos, a varredura chama o `StartExecution` para os pedidos recentes sem desfecho. Para quem já tem fluxo, nada muda; para o órfão, o fluxo começa. É o bedel passando pelos corredores atrás de aluno perdido. O código fica em `backend/lambdas/rango_lambda/varredura.py`:

```
import json
import os

import boto3

from banco import conexao

sfn = boto3.client("stepfunctions")
MAQUINA = os.environ["MAQUINA_PEDIDO_ARN"]


def handler(event, context):
    pendentes = conexao().execute(
        """
        SELECT id FROM pedidos
        WHERE status IN ('recebido', 'pago')
          AND criado_em BETWEEN now() - interval '1 hour' AND now() - interval '2
minutes'
        """
    ).fetchall()
    for linha in pendentes:
        pedido_id = str(linha["id"])
        try:
            sfn.start_execution(
                stateMachineArn=MAQUINA,
                name=pedido_id,
                input=json.dumps({"pedido_id": pedido_id}),
            )
        except sfn.exceptions.ExecutionAlreadyExists:
            pass  # execução já encerrada
    return {"verificados": len(pendentes)}
```

É o transactional outbox do capítulo 5 sem tabela extra: a própria linha do pedido, com status `recebido`, diz que existe trabalho a fazer. Na stack, a varredura e o resumo de vendas migrado para o Scheduler:

```
        varredura = funcao(
            "Varredura", "varredura-pedidos", "varredura.handler",
            memory_size=256, timeout=Duration.seconds(60), retry_attempts=0,
            environment={"MAQUINA_PEDIDO_ARN": maquina.state_machine_arn},
        )
        maquina.grant_start_execution(varredura)
        scheduler.Schedule(
            self, "VarreduraPedidos",
            schedule=scheduler.ScheduleExpression.rate(Duration.minutes(5)),
            target=agendados.LambdaInvoke(varredura),
        )

        for hora, horas in ((11, 12), (15, 4), (23, 8)):
            scheduler.Schedule(
```

```
                self, f"Resumo{hora}h",
                schedule=scheduler.ScheduleExpression.cron(
                    minute="0", hour=str(hora), time_zone=TimeZone.AMERICA_SAO_PAULO
                ),
                target=agendados.LambdaInvoke(
                    resumo, input=scheduler.ScheduleTargetInput.from_object({"horas":
horas})
                ),
            )
```

As horas agora são as do Bruno, e um eventual horário de verão não desalinha nada. O Brasil aboliu o horário de verão em 2019, mas fuso horário é como nota fiscal: melhor guardar.

Alerta quando um fluxo falha

O último pedaço usa o barramento `default` e os eventos que o próprio Step Functions publica:

```
        topico_alertas = sns.Topic(self, "TopicoAlertas", topic_name=f"rango-{ambiente}-
alertas")
        email_alertas = self.node.try_get_context("email_alertas")
        if email_alertas:

topico_alertas.add_subscription(assinaturas.EmailSubscription(email_alertas))

        events.Rule(
            self,
            "FluxoFalhou",
            event_pattern=events.EventPattern(
                source=["aws.states"],
                detail_type=["Step Functions Execution Status Change"],
                detail={
                    "status": ["FAILED", "TIMED_OUT", "ABORTED"],
                    "stateMachineArn": [maquina.state_machine_arn],
                },
            ),
            targets=[alvos.SnsTopic(topico_alertas)],
        )
```

Estorno que falhou seis vezes, entregador que não saiu em duas horas: tudo vira e-mail. A caixa de alertas vira um grupo de condomínio: chega mensagem o dia inteiro e ninguém lê. Cancelamentos terminam em `Succeed` e não disparam nada.

Deploy e o primeiro pedido acompanhado

O deploy começa reativando os workflows e enviando o commit de `backend/` e `rango-infra/` para a `main`:

```
gh workflow enable ci.yaml
gh workflow enable renovar-tasks.yaml
```

Depois da sua aprovação no environment `producao`, o pipeline publica a imagem nova e o `cdk deploy` cria tudo. O `cdk diff` no log mostra os dois serviços de worker sendo removidos. A assinatura de e-mail do tópico de alertas precisa ser confirmada, como no capítulo 7.

Para os testes, dois atalhos. As variáveis `URL_WEBHOOKS` e `SEGREDO_WEBHOOK_ARN` recebem as saídas da stack, como no capítulo 19, e a chave do webhook vai para a variável `CHAVE_WEBHOOK`. Depois:

```
API=https://api.rango.com.br

criar() {  # $1 = restaurante; imprime "pedido chave"
  local chave; chave=$(uuidgen | tr 'A-Z' 'a-z')
  local id; id=$(curl -s -X POST "$API/pedidos" \
    -H "Content-Type: application/json" -H "Idempotency-Key: $chave" \
    -d "{\"usuario\": \"ana\", \"restaurante\": \"$1\", \"total\": 58.90}" | jq -r .id)
  echo "$id $chave"
}

pagar() {  # $1 = pedido, $2 = chave de idempotência da criação
  local corpo="{\"pedido_id\": \"$1\", \"cobranca\": \"cobranca-${2:0:8}\", \"status\":
\"aprovada\"}"
  local assinatura="sha256=$(printf '%s' "$corpo" \
    | openssl dgst -sha256 -hmac "$CHAVE_WEBHOOK" | awk '{print $NF}')"
  curl -s -X POST "$URL_WEBHOOKS/webhooks/pagamento" -H "Content-Type: application/json"
\
    -H "X-Assinatura: $assinatura" -d "$corpo"
}

MAQUINA=$(aws stepfunctions list-state-machines \
  --query "stateMachines[?name=='rango-producao-pedido'].stateMachineArn" --output text)

passo() {  # em que passo está o pedido $1
  local execucao="${MAQUINA/:stateMachine:/:execution:}:$1"
  aws stepfunctions describe-execution --execution-arn "$execucao" --query status --
output text
  aws stepfunctions get-execution-history --execution-arn "$execucao" --reverse-order \
    --query "events[?ends_with(type, 'StateEntered')] |
[0].stateEnteredEventDetails.name" \
    --output text
}
```

O ARN de uma execução é o da máquina com `execution` no lugar de `stateMachine` e o nome da execução no fim, que é o ID do pedido. O primeiro pedido nasce assim, e o `passo` mostra onde ele está:

```
read PEDIDO CHAVE < <(criar cantina-da-nona)
passo "$PEDIDO"
```

```
RUNNING
AguardarPagamento
```

O fluxo está parado, esperando o PIX. Depois do pagamento, a mesma pergunta:

```
pagar "$PEDIDO" "$CHAVE"
sleep 3
passo "$PEDIDO"
curl -s "$API/pedidos/$PEDIDO" | jq .status
```

```
RUNNING
AguardarEntregador
"confirmado"
```

Em três segundos, o webhook avisou o fluxo, o restaurante confirmou e o evento `pedido.confirmado` passou pelo SNS até a fila de e-mail. Três segundos é menos do que o Bruno levou, na terça, para achar o botão de estorno. O log da Lambda confirma:

```
GRUPO=$(aws lambda get-function-configuration --function-name rango-producao-avisos-
email \
  --query LoggingConfig.LogGroup --output text)
aws logs tail "$GRUPO" --since 5m --format short | grep canal
```

```
2026-10-20T23:41:07 {"canal": "email", "tipo": "pedido.confirmado", "pedido_id":
"8c1f4e2a-..."}
```

Agora o entregador sai:

```
curl -s -X POST "$API/entregas/$PEDIDO/iniciar" \
  -H "Content-Type: application/json" -d '{"entregador": "joao-p"}'
sleep 3
passo "$PEDIDO"
```

```
SUCCEEDED
PedidoEmRota
```

O log da `rango-producao-avisos-sms` mostra o `entrega.iniciada`: o SMS do Bruno é mais um assinante do tópico. É o primeiro pedido do Bruno atendido sem mexer no código de mais ninguém. No console do Step Functions, o diagrama da execução mostra cada passo em verde, com entrada, saída e o tempo de espera do pagamento.

O restaurante recusa

Agora, a simulação do botijão vazio do Sabor do Sertão. A variável `RECUSAR` entra direto na função, somada às que o CDK definiu, porque o `update-function-configuration` substitui o ambiente inteiro:

```
FN=rango-producao-confirmar-restaurante
VARS=$(aws lambda get-function-configuration --function-name $FN \
  --query Environment.Variables | jq -c '. + {"RECUSAR": "sabor-do-sertao"}')
aws lambda update-function-configuration --function-name $FN \
  --environment "{\"Variables\": $VARS}" > /dev/null

read PEDIDO CHAVE < <(criar sabor-do-sertao)
```

```
pagar "$PEDIDO" "$CHAVE"
sleep 5
passo "$PEDIDO"
curl -s "$API/pedidos/$PEDIDO" | jq .status
```

```
SUCCEEDED
FimEstornar
"estornado"
```

O log da `cancelar-pedido` mostra o estorno com a chave `estorno-<pedido>`, e o da `avisos-email` mostra o `pedido.cancelado`. Ninguém abriu o painel do gateway, e o Bruno pode, enfim, jantar em paz. Isso é drift proposital, no sentido do capítulo 15; dá para desfazer na hora com `jq -c 'del(.RECUSAR)'` no mesmo comando, ou deixar o próximo deploy desfazer.

Com `FALHA_RESTAURANTE` em `0.5`, o histórico mostra o `Retry` com intervalos irregulares. E um pedido não pago termina, 15 minutos depois, em `FimCancelar`.

Uma mensagem envenenada

O próximo teste manda à fila de e-mail uma mensagem sem o atributo `tipo`, como faria um bug num publicador futuro:

```
FILA=$(aws sqs get-queue-url --queue-name rango-producao-email --query QueueUrl --output
text)
DLQ=$(aws sqs get-queue-url --queue-name rango-producao-email-dlq --query QueueUrl --
output text)
aws sqs send-message --queue-url "$FILA" --message-body '{"pedido_id": "teste"}'
```

O `avisos.py` lança `KeyError`, devolve a mensagem em `batchItemFailures`, e ela volta à fila a cada dois minutos. No log, cinco linhas `falha em ...: KeyError('tipo')`. Uns dez minutos depois:

```
aws sqs get-queue-attributes --queue-url "$DLQ" \
  --attribute-names ApproximateNumberOfMessages --query Attributes
```

```
{
    "ApproximateNumberOfMessages": "1"
}
```

As mensagens boas continuaram passando. A maçã podre foi para a quarentena, e o resto do cesto seguiu viagem. Ela pode ser inspecionada com `aws sqs receive-message --queue-url "$DLQ" --message-attribute-names All` e apagada com `aws sqs purge-queue --queue-url "$DLQ"`.

O caso oposto é o do provedor de e-mail fora do ar por uma hora: mensagens boas na DLQ. Depois que o provedor volta, um comando devolve todas à fila de origem:

```
DLQ_ARN=$(aws sqs get-queue-attributes --queue-url "$DLQ" \
  --attribute-names QueueArn --query Attributes.QueueArn --output text)
aws sqs start-message-move-task --source-arn "$DLQ_ARN"
```

Sem `--destination-arn`, o SQS devolve cada mensagem à fila de onde ela veio. A idempotência do `avisos.py` garante que quem já recebeu o e-mail não recebe outro.

Quanto isso custa

Com os preços de referência de outubro de 2026 em us-east-1 (aproximados; confira as páginas oficiais), e assumindo mil pedidos por dia:

Item	Conta	Por mês
Step Functions Standard	~9 transições × 30 mil pedidos = 270 mil, menos 4 mil gratuitas, × US$ 0,000025	~US$ 7
Endpoint do Step Functions	2 AZs × 730 h × US$ 0,01	~US$ 15
SNS, SQS, Lambdas, DynamoDB do fluxo	algumas centenas de milhares de requisições	centavos
EventBridge Scheduler	~8.700 invocações, dentro das 14 milhões gratuitas	US$ 0
Workers do ECS removidos	estimativa do capítulo 17	–US$ 14

Uns US$ 8 a mais por mês, o preço de um pastel com caldo de cana na feira para nunca mais estornar pedido à mão. O endpoint é o preço de manter as subnets sem internet; a máquina cresce linearmente, uns US$ 7 a cada mil pedidos diários.

Desmontando

Execuções em andamento seguram a máquina, então o desmonte para as que estiverem esperando antes de destruir a stack:

```
for e in $(aws stepfunctions list-executions --state-machine-arn "$MAQUINA" \
    --status-filter RUNNING --query "executions[].executionArn" --output text); do
  aws stepfunctions stop-execution --execution-arn "$e" > /dev/null
done

cd rango-infra
cdk destroy RangoProducao
gh workflow disable ci.yaml
gh workflow disable renovar-tasks.yaml
unset CHAVE_WEBHOOK
```

Cada execução parada vira `ABORTED` e gera um e-mail de alerta, o teste da regra. É a única ocasião em que uma enxurrada de alertas quer dizer que está tudo certo. Filas, tópicos, máquina, agendas e tabela vão embora com a stack. O banco deixa um snapshot novo, como sempre.

Idempotência, peça por peça

O capítulo 5 disse que, num sistema com entrega pelo menos uma vez, a idempotência precisa estar perto do efeito. Agora você pode ver onde ela ficou em cada ponto do Rangô:

Onde pode repetir	Quem protege
Cliente clica duas vezes em "confirmar"	`Idempotency-Key` + `SET NX` no Valkey (capítulo 5)
API e varredura iniciam o mesmo fluxo	nome da execução = ID do pedido
Webhook reenviado pelo gateway	`UPDATE ... WHERE` + `SendTaskSuccess` repetido ignorado
`Retry` chama o restaurante de novo	checagem de `status = 'confirmado'`
Estorno executado duas vezes	chave `estorno-{pedido_id}` no gateway
SNS ou SQS entregam a mesma mensagem	`put_item` condicional no DynamoDB
Stream reentrega a conclusão da entrega	`UPDATE ... AND status = 'confirmado'` (capítulo 20)

Os serviços tornam a repetição rara e visível; quem garante que ela não causa estrago é o seu código.

Armadilhas e casos reais

> **Armadilha:** Visibility timeout menor que o processamento. Um consumidor leva 45 segundos numa fila com o padrão de 30, a mensagem reaparece no meio do trabalho, e outro consumidor faz a mesma coisa em paralelo. O sintoma é duplicata sem erro nenhum no log. Com Lambda, a própria AWS barra o caso extremo: um timeout de função maior que o visibility timeout da fila faz a criação do event source mapping falhar.
>
> **Armadilha:** DLQ sem alarme. A fila de mensagens mortas do Rangô pode encher em silêncio por 14 dias e então começar a descartar. O alarme sobre `ApproximateNumberOfMessagesVisible` da DLQ chega no capítulo 22; até lá, olhe a

fila depois de cada deploy.

Armadilha: Entrada diferente no `StartExecution`. Se a varredura passar a mandar `{"pedido_id": "...", "origem": "varredura"}`, recebe `ExecutionAlreadyExists` em todo pedido em andamento, e um retry nesse `except` nunca termina. Monte a entrada numa função só.

Armadilha: Laços de espera no Step Functions Standard. Um `Wait` de 10 segundos seguido de um `Choice` que volta para o `Wait`, para "conferir se o pagamento chegou", custa duas transições a cada 10 segundos por pedido. Em 15 minutos, 180 transições, vinte vezes o fluxo inteiro. O callback com token espera de graça.

Armadilha: Mensagens grandes pelo fluxo. A entrada de uma execução do Step Functions aceita até 256 KiB, e o estado acumula o resultado de cada passo. Passe IDs e busque os dados no passo que precisa deles, como o Rangô faz.

Na vida real: Em 25 de novembro de 2020, o Amazon Kinesis Data Streams de us-east-1 passou boa parte do dia com falhas, depois que um aumento de capacidade da frota de front-end fez os servidores passarem do limite de threads do sistema operacional. Serviços da AWS que usam o Kinesis por dentro sofreram junto, entre eles o Cognito, o CloudWatch e o EventBridge, que entregou eventos com atraso. Até o barramento tem um barramento do qual depende; são tartarugas até lá embaixo. O barramento também é uma dependência: um fluxo que tolera atraso sobrevive a um dia assim.

Resumo

- O SQS guarda mensagens até alguém processar. Receber não remove: a mensagem fica invisível pelo visibility timeout e volta se o consumidor não a apagar. A redrive policy manda para a DLQ depois de N recebimentos.
- Fila standard tem vazão quase ilimitada e entrega pelo menos uma vez, sem ordem; FIFO ordena por `MessageGroupId`, com vazão limitada.
- Com Lambda, o event source mapping consulta a fila; `ReportBatchItemFailures` devolve só as mensagens que falharam, e o visibility timeout deve ser seis vezes o timeout da função.
- O SNS copia cada publicação para todas as assinaturas, com filter policy por atributo. SNS na frente e uma fila SQS atrás de cada consumidor é o padrão de fan-out.
- O EventBridge roteia eventos pelo conteúdo, recebe de graça os eventos dos serviços da AWS no barramento `default` e agenda tarefas com fuso horário pelo Scheduler.

- O Step Functions conduz processos de vários passos com `Retry`, `Catch`, `Choice` e callback por token. Standard para processos longos e auditáveis; Express para fluxos curtos e de volume alto.
- Compensação desfaz um passo já feito quando outro adiante falha; o estorno compensa o pagamento.
- O Rangô trocou o stream do Valkey e os workers por uma máquina de estados por pedido, um tópico com filas de e-mail e SMS, uma varredura que fecha a escrita dupla e um alerta para fluxos que falham.
- Idempotência continua sendo trabalho seu, em cada ponto onde algo pode repetir.

Dez lugares para procurar

Um cliente escreve às 21h10: o pedido levou 40 minutos para ser confirmado. Onde ficou parado? A execução do Step Functions mostra que o `ConfirmarRestaurante` demorou, mas não por quê. O log da Lambda está num grupo, o da API em outro, o do webhook num terceiro, e o Rangô já tem dez deles. A DLQ de e-mail tem sete mensagens, e você descobriu por acaso. O resumo de vendas das 11h falhou na quinta, e o Bruno avisou na sexta. Por enquanto, o sistema de monitoramento do Rangô se chama Bruno.

No próximo capítulo, o Rangô ganha olhos: Amazon CloudWatch Logs e Logs Insights para consultar todos os logs de uma vez, métricas e alarmes (a DLQ, finalmente), dashboards, logs estruturados no FastAPI, tracing distribuído com AWS X-Ray e OpenTelemetry seguindo um pedido da API até o SMS, e SLOs para responder com números à pergunta que todo mundo faz primeiro: está lento para quem?

Referências

- Amazon SQS, Developer Guide: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html
- Cotas de mensagens do SQS (tamanho, retenção, visibility timeout, FIFO): https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/quotas-messages.html
- Visibility timeout: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
- Dead letter queues no SQS: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide

/sqs-dead-letter-queues.html

- Preços do SQS: https://aws.amazon.com/sqs/pricing/
- Lambda com SQS (visibility timeout, ReportBatchItemFailures, maxReceiveCount): https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-configure.html
- Amazon SNS, Developer Guide: https://docs.aws.amazon.com/sns/latest/dg/welcome.html
- Filtro de mensagens no SNS: https://docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html
- Preços do SNS: https://aws.amazon.com/sns/pricing/
- Amazon EventBridge, User Guide: https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html
- EventBridge Scheduler, User Guide: https://docs.aws.amazon.com/scheduler/latest/UserGuide/what-is-scheduler.html
- Preços do EventBridge (barramentos, Pipes, Scheduler, archive): https://aws.amazon.com/eventbridge/pricing/
- CDK, módulo `aws_scheduler`: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_scheduler/README.html
- AWS Step Functions, Developer Guide: https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html
- Tratamento de erros no Step Functions (Retry, Catch, jitter): https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html
- API `StartExecution` (idempotência em Standard): https://docs.aws.amazon.com/step-functions/latest/apireference/API_StartExecution.html
- Preços do Step Functions: https://aws.amazon.com/step-functions/pricing/
- Powertools for AWS Lambda (Python), idempotência: https://docs.powertools.aws.dev/lambda/python/latest/utilities/idempotency/
- Resumo do incidente do Kinesis em us-east-1 (25/nov/2020): https://aws.amazon.com/message/11201/

CAPÍTULO 22

Observabilidade com CloudWatch

No capítulo anterior, o pedido do Rangô deixou de ser uma cadeia de chamadas síncronas. Uma máquina de estados do AWS Step Functions conduz cada pedido do pagamento até a saída do entregador, um tópico do SNS espalha os avisos para filas de e-mail e SMS com DLQ, e uma varredura a cada cinco minutos recupera pedidos que ficaram sem fluxo. Uma regra do EventBridge já manda e-mail quando uma execução falha. O sistema ficou mais confiável e, ao mesmo tempo, mais espalhado: são dez lugares diferentes onde alguma coisa pode dar errado sem ninguém perceber. É como ter dez controles remotos na sala: cada um funciona, e nenhum está onde você procura.

Sexta, 21h10, "cadê o meu pedido?"

A mensagem chega pelo Instagram do Rangô: "Fiz o pedido às 20h29, paguei no PIX na hora, e só às 21h07 apareceu confirmado. Quarenta minutos. O que aconteceu?" O cliente descreveu o problema com horário e sintoma, a precisão de um bom relatório de bug, o que já é mais do que muito chamado interno consegue.

Você abre o console do Step Functions e procura a execução pelo ID do pedido. Ela existe, mas começou às 21h05. O pedido passou 36 minutos no limbo, sem existir para ninguém além do cliente e do PIX. O fluxo em si levou dois minutos. O que aconteceu entre 20h29 e 21h05?

A resposta está em algum dos dez grupos de logs da stack. O da API tem um nome como `RangoProducao-LogsApp8F2D41C7-xK2pQ9mZ3aLb`, e os das Lambdas seguem o mesmo padrão. Você abre um, rola a tela, abre outro. O `xK2pQ9mZ3aLb` parece senha de Wi-Fi de hotel, e é tão fácil de lembrar quanto. As linhas da API são JSON, com campos `ts`, `nivel` e `msg`. As das Lambdas são texto solto de `print`, cada uma no dialeto de quem a escreveu, numa sexta diferente. Nenhuma delas diz o ID do pedido de um jeito que dê para procurar nos dez grupos de uma vez.

Enquanto procura, você esbarra em duas coisas que não estava procurando. A DLQ de e-mail tem sete mensagens, todas de quinta de manhã. E o resumo de vendas das 11h de quinta falhou: o Bruno avisou na sexta, porque o dono da Cantina da Nona reclamou que

não recebeu. Nenhum dos dois problemas gerou alerta. A regra do capítulo 21 só observa execuções do Step Functions.

O Rangô tem os dados. Os logs estão lá, e a AWS mede cada fila, cada Lambda e cada requisição no load balancer. Falta o que o capítulo 6 chamou de observabilidade: conseguir responder perguntas que você não previu, como "onde esse pedido ficou parado?", sem abrir dez abas e sem depender de um cliente reclamar. Cliente reclamando é um sistema de alerta com latência de horas e uma taxa de falso negativo constrangedora.

Um serviço, várias perguntas

O Amazon CloudWatch reúne as peças de observabilidade da AWS. Você já usou duas partes dele sem dar esse nome: o alarme de cobrança do capítulo 7 é um alarme do CloudWatch, e os logs das tasks e das Lambdas vão para o CloudWatch Logs desde os capítulos 17 e 19. Este capítulo usa o resto. É como descobrir que o micro-ondas tem outros botões além do "pipoca".

Cada pergunta da sexta tem uma ferramenta:

Pergunta	Ferramenta
O que aconteceu com o pedido 7f3c?	CloudWatch Logs e Logs Insights
Quantos pedidos por minuto? Qual a latência?	Métricas
Quem me avisa quando a DLQ recebe mensagem?	Alarmes
Como está o Rangô agora, numa tela só?	Dashboards
Por qual serviço a requisição passou, e quanto tempo em cada um?	Traces (AWS X-Ray e OpenTelemetry)
O Rangô está bom o suficiente para os clientes?	SLOs

Logs: grupos, streams e retenção

No CloudWatch Logs, cada linha é um *log event*, com horário e texto. Os eventos ficam em *log streams*, uma por origem: cada task do ECS e cada ambiente de execução da Lambda escreve na sua. As streams ficam dentro de um *log group*, a unidade onde você define retenção, permissões e criptografia. A analogia mais próxima é uma tabela de banco: o grupo é a tabela, a stream é o arquivo de onde as linhas vieram.

Você paga pela ingestão e pelo armazenamento. Em `us-east-1`, a ingestão padrão custa cerca de US$ 0,50 por GB, e o armazenamento, cerca de US$ 0,03 por GB por mês. Existe uma classe Infrequent Access, com ingestão pela metade do preço e menos

recursos, boa para logs que você só consulta em incidente. Ou seja, quase todos: log é como seguro de carro, ninguém olha até bater. O nível gratuito mensal inclui 5 GB. Os valores são aproximados, consultados em outubro de 2026; confira a página de preços do CloudWatch antes de planejar.

Logs Insights

O CloudWatch Logs Insights consulta logs com uma linguagem própria, feita de comandos encadeados por `|`, no estilo de um pipe do terminal. Um exemplo:

```
fields @timestamp, message
| filter level = "ERROR"
| sort @timestamp desc
| limit 20
```

O detalhe que muda tudo: quando a linha é JSON, o Logs Insights descobre os campos sozinho. Se a API escreve `{"level": "ERROR", "pedido_id": "7f3c..."}`, você filtra por `pedido_id` sem configurar nada. Ele descobre até 1.000 campos por log group. Se o seu log tem mais de mil campos, o CloudWatch é o menor dos seus problemas. Com texto solto, você precisaria de `parse` e expressões regulares para cada formato. Diz o ditado que quem resolve um problema com expressão regular passa a ter dois problemas.

Uma consulta pode cobrir até 50 log groups de uma vez e rodar por até 60 minutos. A cobrança é pelo volume escaneado, então o intervalo de tempo que você escolhe pesa mais na conta do que a complexidade da consulta. Consultar trinta dias por impulso é o equivalente, em observabilidade, a fazer compras com fome.

Métricas

Uma métrica é uma série temporal: um número por instante. Ela faz pelo sistema o que o termômetro faz pela febre: não cura nada, mas encerra a discussão. Ela mora num *namespace* (`AWS/SQS`, `AWS/Lambda`, `AWS/ApplicationELB`) e é identificada pelo nome mais as *dimensões*, pares chave-valor que dizem de quem é o número. `ApproximateNumberOfMessagesVisible` com a dimensão `QueueName=rango-producao-email-dlq` é uma métrica; a mesma métrica com outra fila é outra.

Os serviços da AWS publicam as próprias métricas de graça. O load balancer publica `RequestCount`, `TargetResponseTime` e as contagens de 5xx; a Lambda, `Invocations`, `Errors`, `Duration` e `Throttles`; o SQS, o tamanho da fila e a idade da mensagem mais antiga, aquela `ApproximateAgeOfOldestMessage` prometida desde o capítulo 5; o RDS, `DatabaseConnections` e `FreeStorageSpace`.

Na hora de ler, você escolhe um *período* (um minuto, cinco minutos) e uma *estatística*: `Sum`, `Average`, `Maximum` ou um percentil como `p99`. O capítulo 6 explicou por que a média esconde os clientes que esperaram demais; aqui o p99 é só uma opção no gráfico.

Na média, quem come um frango inteiro e quem não come nada comeram meio frango cada um, e um deles continua com fome.

Métricas suas, como "segundos entre o pagamento e a confirmação", são *custom metrics*: cerca de US$ 0,30 por métrica por mês nas primeiras 10.000, com dez gratuitas. O jeito mais simples de publicá-las a partir de uma Lambda é o *Embedded Metric Format* (EMF): você escreve no log uma linha JSON num formato combinado, e o CloudWatch extrai a métrica dela. Nenhuma chamada de API extra no caminho do pedido. A métrica viaja escondida dentro do log, como recado escrito em guardanapo.

Alarmes

Um alarme observa uma métrica e tem três estados: `OK`, `ALARM` e `INSUFFICIENT_DATA`. O terceiro é o "sei lá" oficial da AWS, e aparece mais do que se imagina. Você define o limite, o período e quantos períodos olhar. "Dois de três períodos acima do limite" (o *M de N*) evita que um pico isolado acorde alguém às 3h. Quem já foi acordado às 3h por um alarme que se resolveu sozinho às 3h01 conhece o valor dessa regra, e o nome de quem configurou o alarme. A cada mudança de estado, o alarme pode publicar num tópico do SNS, e o Rangô já tem um, o `rango-producao-alertas`.

A configuração que mais pega gente desprevenida é o que fazer com dado ausente. Por padrão (`missing`), período sem dado conta como insuficiente, e o alarme vai para `INSUFFICIENT_DATA`. Há outras três opções: `notBreaching` trata a ausência como normal, `breaching` trata como problema, e `ignore` mantém o estado anterior. O capítulo 18 já usou `notBreaching` no alarme de 5xx, porque minuto sem erro não gera ponto na métrica. A regra prática: se a ausência quer dizer "nada aconteceu", use `notBreaching`; se quer dizer "o que devia rodar não rodou", use `breaching`. Silêncio, em monitoramento, tanto pode ser paz quanto a criança quieta demais no quarto ao lado.

Os dez primeiros alarmes de resolução padrão são gratuitos; depois, cerca de US$ 0,10 por alarme por mês. O alarme de cobrança do capítulo 7 já ocupa um desses dez. Ele foi o primeiro, e com razão: de todos os sustos possíveis na AWS, a fatura é o que a gente quer receber com antecedência.

Traces e o fim do SDK do X-Ray

Logs contam o que cada peça fez. Um *trace* conta a viagem de uma requisição pelas peças, como uma linha do tempo. O log é o depoimento de cada testemunha; o trace é a câmera de segurança. Cada trecho é um *span*: "API recebeu `POST /pedidos`, 180 ms", e dentro dele "INSERT no Postgres, 12 ms" e "StartExecution no Step Functions, 41 ms". Todos os spans de uma viagem compartilham um *trace ID*, que passa de serviço em

serviço num cabeçalho HTTP. O padrão W3C chama esse cabeçalho de `traceparent`; o X-Ray usa o seu, `X-Amzn-Trace-Id`. Dois padrões para a mesma coisa: a indústria mantém viva a tradição dos carregadores de celular.

O AWS X-Ray é o serviço de traces da AWS. Durante anos, instrumentar uma aplicação para ele significava usar o SDK do X-Ray e um processo chamado X-Ray Daemon. Isso mudou: desde 25 de fevereiro de 2026, os dois estão em modo de manutenção, recebendo só correções críticas e de segurança, e o suporte termina em 25 de fevereiro de 2027. O X-Ray continua aceitando dados deles, e o console segue igual, mas a recomendação da AWS para código novo é o OpenTelemetry, seja pela distribuição da AWS (ADOT, AWS Distro for OpenTelemetry), seja pela instrumentação OpenTelemetry padrão. Se você acabou de aprender o SDK do X-Ray num tutorial antigo, sinto muito; a tecnologia tem dessas.

Para o Rangô, isso decide a escolha: nada de SDK do X-Ray. A API usa a distribuição ADOT para Python, que instrumenta FastAPI, boto3 e outras bibliotecas sem você tocar nas rotas. Um aviso da própria AWS: as bibliotecas OpenTelemetry podem não ter todos os recursos que os SDKs do X-Ray tinham, então confira se algo de que você depende ficou para trás.

O destino dos spans também mudou. Com o *Transaction Search*, uma configuração que vale para a conta inteira, o X-Ray recebe spans no formato do OpenTelemetry e grava todos eles num log group chamado `aws/spans`. Você paga pela ingestão como log (cerca de US$ 0,35 por GB), e por padrão 1% dos spans é indexado de graça como resumo de trace, pesquisável no console. Os outros 99% continuam no log group, consultáveis pelo Logs Insights.

SLOs

As métricas dizem o que está acontecendo. Um *SLO* (Service Level Objective) diz se isso é bom o suficiente. É a diferença entre o termômetro e o médico. Ele tem duas partes: um indicador (o SLI), como "p90 do tempo até confirmar o pedido abaixo de 5 minutos", e uma meta, como "em 99% dos períodos de 5 minutos da última semana".

O que sobra da meta é o *error budget*: 1% dos períodos pode ser ruim. É a única meta corporativa que autoriza, por escrito, errar um pouco. Uma semana tem 2.016 períodos de 5 minutos, então o Rangô pode ter cerca de 20 períodos ruins por semana, uns 100 minutos. Esse número muda a conversa com o Bruno. "Está lento" vira "a noite de sexta gastou 40% do orçamento da semana". Orçamento é uma palavra que todo sócio entende, mesmo quando já não entende mais nada da conversa.

A velocidade com que o orçamento some é o *burn rate*. Burn rate 1 consome o orçamento exatamente no ritmo de zerar no fim da janela; acima de 1, ele acaba antes. Na sexta, com todos os períodos ruins durante quase 40 minutos, o burn rate foi 100: o orçamento de uma semana inteira duraria pouco mais de uma hora e meia naquele ritmo, como salário que acaba no dia 5. Alarmes sobre burn rate avisam disso cedo e ignoram o ruído de um período ruim isolado.

O CloudWatch cria SLOs dentro do Application Signals, a parte que descobre serviços e operações e mede latência e disponibilidade. O indicador pode ser qualquer métrica do CloudWatch ou uma expressão matemática sobre métricas, o que basta para o Rangô.

Na prática

O plano: dar nomes aos log groups, fazer todo mundo escrever JSON com o mesmo vocabulário, seguir um pedido pelos dez grupos, publicar uma métrica de negócio, criar alarmes e um painel, ligar os traces e definir um SLO. É uma lista comprida, mas nenhum item exige reunião.

Pré-requisitos: o repositório do capítulo 21, a CLI configurada com `AWS_PROFILE=rango-dev`, `jq` instalado. Como nos capítulos anteriores, o snapshot deixado pelo último `cdk destroy` vai, pelo nome, no `snapshot_db` do `rango-infra/app.py`.

Um vocabulário de log só

A Lambda tem um formato de log JSON embutido. Ligado, cada chamada ao módulo `logging` vira uma linha JSON com `timestamp`, `level`, `message`, `logger` e `requestId`, e os campos que você passa em `extra` entram no mesmo objeto. A API, então, adota as mesmas chaves. Padronizar nome de campo é a briga mais longa e menos glamourosa de qualquer equipe; aqui, a Lambda decidiu por todos. O `backend/app/logs.py` é reescrito:

```
# backend/app/logs.py
import json
import logging
import sys
from datetime import datetime, timezone

from opentelemetry import trace


class FormatoJson(logging.Formatter):
    def format(self, registro: logging.LogRecord) -> str:
        dados = {
            "timestamp": datetime.fromtimestamp(registro.created,
timezone.utc).isoformat(),
            "level": registro.levelname,
            "message": registro.getMessage(),
            "logger": registro.name,
```

```
            **getattr(registro, "campos", {}),
        }
        contexto = trace.get_current_span().get_span_context()
        if contexto.is_valid:
            dados["trace_id"] = format(contexto.trace_id, "032x")
        if registro.exc_info:
            dados["erro"] = self.formatException(registro.exc_info)
        return json.dumps(dados, ensure_ascii=False, default=str)


def configurar() -> None:
    saida = logging.StreamHandler(sys.stdout)
    saida.setFormatter(FormatoJson())
    logging.basicConfig(level=logging.INFO, handlers=[saida], force=True)
```

Duas mudanças além dos nomes. O `trace_id` liga cada linha de log ao trace da requisição, assim que a instrumentação da próxima seção estiver no ar. E o `exc_info`: o formatador do capítulo 6 ignorava a exceção. O `log.exception` do `iniciar_fluxo` gravava "fluxo do pedido não iniciado" e jogava fora o traceback, que era justamente a parte útil. Era guardar a caixa e jogar fora o presente.

No `backend/app/main.py`, o pedido passa a ter o ID num campo próprio, e a mensagem deixa de levar o ID no texto:

```
# em criar_pedido, logo depois do INSERT:
    log.info("pedido criado", extra={"campos": {"pedido_id": str(pedido["id"])}})

# em iniciar_fluxo, no lugar do log.exception anterior:
    except Exception:
        log.exception(
            "fluxo não iniciado; a varredura tenta de novo",
            extra={"campos": {"pedido_id": pedido_id}},
        )
```

Nas Lambdas, os `print` dão lugar ao `logging`. O `confirmar_restaurante.py`, por exemplo, ganha no topo:

```
import logging

log = logging.getLogger()
```

E, no `handler`, depois de perguntar ao restaurante:

```
    log.info("restaurante respondeu", extra={"pedido_id": pedido_id, "aceito": aceito})
```

O mesmo vale para `avisos.py` (`pedido_id`, `canal`, `tipo` em cada envio e em cada falha), `cancelar_pedido.py`, `guardar_token.py`, `varredura.py` (um `log.info` por pedido reenviado ao fluxo), `webhook_pagamento.py`, `entrega_concluida.py` e `resumo_vendas.py`. Com o formato JSON, a Lambda configura o logger raiz com nível `INFO`, sem `basicConfig`.

Na stack, os log groups ganham nomes legíveis, com um prefixo comum. No `rango_infra_stack.py`, logo depois da criação do cluster, onde o capítulo 17 criou o `logs_app`:

```
        grupos_logs: list[logs.LogGroup] = []

        def grupo_de_logs(id_: str, nome: str) -> logs.LogGroup:
            grupo = logs.LogGroup(
                self,
                f"Logs{id_}",
                log_group_name=f"/rango/{ambiente}/{nome}",
                retention=logs.RetentionDays.ONE_MONTH,
                removal_policy=RemovalPolicy.DESTROY,
            )
            grupos_logs.append(grupo)
            return grupo
```

Nas duas auxiliares, `funcao` e `funcao_leve`, o `log_group=logs.LogGroup(...)` dá lugar a estas três linhas:

```
                log_group=grupo_de_logs(id_, f"lambda/{nome}"),
                logging_format=lambda_.LoggingFormat.JSON,
                tracing=lambda_.Tracing.ACTIVE,
```

O `tracing` fica para a seção de traces, mas já entra aqui. E o grupo da API, criado no capítulo 17, passa a usar a mesma auxiliar, logo abaixo dela:

```
        logs_app = grupo_de_logs("App", "api")
```

O ID `LogsApp` continua o mesmo, mas o nome físico mudou, e o CloudFormation substitui o grupo. Como a stack é recriada do zero neste capítulo, isso não custa nada. Numa stack em produção, trocar o nome de um log group significa perder o histórico, que fica no grupo antigo até a retenção vencer (ou até ele ser apagado com a stack). É a versão observável de trocar o número do celular sem avisar ninguém.

Depois do commit na `main` e da aprovação do deploy no environment `producao`, a lista de grupos fica assim:

```
aws logs describe-log-groups --log-group-name-prefix /rango/producao/ \
  --query 'logGroups[].logGroupName' --output text | tr '\t' '\n'
```

```
/rango/producao/api
/rango/producao/lambda/avisos-email
/rango/producao/lambda/avisos-sms
/rango/producao/lambda/cancelar-pedido
/rango/producao/lambda/confirmar-restaurante
/rango/producao/lambda/entrega-concluida
/rango/producao/lambda/guardar-token
/rango/producao/lambda/resumo-vendas
```

```
/rango/producao/lambda/varredura-pedidos
/rango/producao/lambda/webhook-pagamento
```

O rastro de um pedido pelos dez grupos

O console do Logs Insights deixa escolher os grupos numa lista. Para o dia a dia, um script que consulta todos os grupos do prefixo poupa cliques. Ele fica em `scripts/consultar-logs.sh`:

```
#!/usr/bin/env bash
# Uso: scripts/consultar-logs.sh 'consulta do Logs Insights' [minutos]
set -euo pipefail

consulta="$1"
minutos="${2:-60}"

grupos=$(aws logs describe-log-groups --log-group-name-prefix /rango/producao/ \
  --query 'logGroups[].logGroupName' --output text)
fim=$(date +%s)
inicio=$((fim - minutos * 60))

# $grupos sem aspas de propósito: cada nome vira um argumento
id=$(aws logs start-query --log-group-names $grupos \
  --start-time "$inicio" --end-time "$fim" \
  --query-string "$consulta" --query queryId --output text)

while :; do
  resultado=$(aws logs get-query-results --query-id "$id")
  status=$(jq -r .status <<<"$resultado")
  [ "$status" = "Running" ] || [ "$status" = "Scheduled" ] || break
  sleep 1
done

echo "status: $status" >&2
jq -r '.results[] | map(select(.field != "@ptr") | .value) | @tsv' <<<"$resultado"
```

Com um pedido de teste criado pela função `criar` e pago pelo webhook do capítulo 21, e o ID guardado na variável `PEDIDO`, o script segue o pedido:

```
chmod +x scripts/consultar-logs.sh
scripts/consultar-logs.sh "fields @timestamp, @log, level, message
| filter pedido_id = '$PEDIDO'
| sort @timestamp asc" 15
```

```
status: Complete
2026-10-16 14:02:11.482  123456789012:/rango/producao/api                          INFO
pedido criado
2026-10-16 14:02:12.903  123456789012:/rango/producao/lambda/guardar-token        INFO
token guardado
2026-10-16 14:02:19.377  123456789012:/rango/producao/lambda/webhook-pagamento    INFO
pagamento registrado
2026-10-16 14:02:20.115  123456789012:/rango/producao/lambda/confirmar-restaurante  INFO
restaurante respondeu
2026-10-16 14:02:21.640  123456789012:/rango/producao/lambda/avisos-email         INFO
aviso enviado
```

O campo `@log` mostra de qual grupo veio cada linha. É a linha do tempo que você tentou montar na sexta, abrindo abas. São cinco linhas; na sexta, foram quarenta minutos e um café esfriando ao lado do teclado.

Com os logs nesse formato, a investigação da sexta teria levado três minutos. A mesma consulta, para o pedido do cliente, teria mostrado isto:

```
20:29:41.204  /rango/producao/api                            INFO   pedido criado
20:29:43.871  /rango/producao/api                            ERROR  fluxo não iniciado; a
varredura tenta de novo
20:31:02.517  /rango/producao/lambda/webhook-pagamento       INFO   pagamento registrado
21:05:03.090  /rango/producao/lambda/varredura-pedidos       INFO   fluxo iniciado pela
varredura
21:05:03.655  /rango/producao/lambda/guardar-token           INFO   token guardado
21:07:12.048  /rango/producao/lambda/confirmar-restaurante INFO  restaurante respondeu
```

O campo `erro` da segunda linha traria um `ConnectTimeoutError` na chamada ao Step Functions: uma falha isolada de rede, a primeira falácia do capítulo 5. "A rede é confiável", disse alguém um dia, e a rede ri disso até hoje. Para isso a varredura existe. Ela devia ter pegado o pedido às 20h35. A pergunta seguinte é por que ela levou meia hora a mais, e uma segunda consulta responde:

```
scripts/consultar-logs.sh "filter level = 'ERROR'
| stats count(*) as erros by @log, bin(5m)
| sort erros desc" 60
```

Na sexta, essa consulta teria mostrado a varredura falhando a cada cinco minutos, de 20h25 a 21h00, com `OperationalError` e a mensagem do Postgres sobre não haver mais vagas de conexão. Era o pico do jantar: a API em seis tasks, o webhook com seus dez ambientes, dezenas de ambientes da `confirmar-restaurante`, cada um segurando uma conexão aberta. O ambiente da varredura foi reciclado no meio do pico, e o novo nunca conseguiu conexão. A rede de segurança do pedido ficou fora do ar exatamente quando foi necessária, como guarda-chuva que só abre em dia de sol.

A mesma consulta, rodada sobre quinta de manhã, explica as outras duas surpresas. Das 10h50 às 11h25, o provedor de e-mail recusou envios: as sete mensagens da DLQ são desse intervalo, e o resumo das 11h ficou esperando o mesmo provedor até estourar os 60 segundos de timeout. Um defeito externo, dois sintomas, nenhum alerta. Diagnóstico digno de médico de série de TV, só que três dias depois.

Uma terceira consulta, só com os logs da API, dá a latência por rota, a partir do `duracao_ms` que o middleware do capítulo 6 grava:

```
filter message = "requisicao"
| stats count(*) as req, pct(duracao_ms, 50) as p50, pct(duracao_ms, 99) as p99 by rota
| sort p99 desc
```

É a consulta que encerra a discussão "a API está lenta" com o nome da rota culpada.

Uma métrica de negócio

Logs respondem perguntas sobre um pedido. Para a pergunta "os pedidos estão demorando para confirmar?", o que interessa é um número por minuto, com gráfico e alarme. No `confirmar_restaurante.py`, a consulta passa a trazer a idade do pedido:

```
    pedido = conexao().execute(
        "SELECT restaurante, status, extract(epoch FROM now() - criado_em) AS segundos"
        " FROM pedidos WHERE id = %s",
        (pedido_id,),
    ).fetchone()
```

E, quando o restaurante aceita, a função escreve uma linha no Embedded Metric Format:

```
import json
import time


def publicar_metrica(nome: str, valor: float, unidade: str) -> None:
    # Embedded Metric Format: o CloudWatch extrai a métrica desta linha de log.
    print(json.dumps({
        "_aws": {
            "Timestamp": int(time.time() * 1000),
            "CloudWatchMetrics": [{
                "Namespace": "Rango",
                "Dimensions": [["Servico"]],
                "Metrics": [{"Name": nome, "Unit": unidade}],
            }],
        },
        "Servico": "fluxo-pedido",
        nome: valor,
    }))
```

```
    # no handler, dentro do if aceito, depois do UPDATE:
        publicar_metrica("SegundosAteConfirmar", float(pedido["segundos"]), "Seconds")
```

Aqui o `print` é deliberado. O formato JSON da Lambda embrulha as mensagens do `logging` num objeto próprio, e a linha deixaria de estar no formato que o CloudWatch reconhece; o `print` sai como está. É um raro momento em que o `print` é a escolha profissional, e não o resto de uma depuração esquecida. A dimensão `Servico` tem um valor só, e isso também é deliberado, como mostra a seção de armadilhas.

Alarmes que chegam ao seu e-mail

No `rango_infra_stack.py`, entra o import:

```
from aws_cdk import (
    # ... os imports do capítulo 21 ...
```

```
    aws_cloudwatch_actions as cw_actions,
)
```

No laço das filas de aviso do capítulo 21, cada DLQ passa a ser guardada numa lista. Antes do laço, `dlqs: list[sqs.Queue] = []`; dentro dele, depois de criar a `dlq`, `dlqs.append(dlq)`. Então, depois do tópico de alertas:

```
        acao_alerta = cw_actions.SnsAction(topico_alertas)
        alarmes: list[cloudwatch.Alarm] = []

        def avisar(alarme: cloudwatch.Alarm) -> cloudwatch.Alarm:
            alarme.add_alarm_action(acao_alerta)
            alarme.add_ok_action(acao_alerta)
            alarmes.append(alarme)
            return alarme

        for fila_morta in [*dlqs, falhas_entregas]:
            avisar(
                fila_morta.metric_approximate_number_of_messages_visible(
                    period=Duration.minutes(5), statistic="Maximum"
                ).create_alarm(
                    self,
                    f"Alarme{fila_morta.node.id}",
                    alarm_description=f"Mensagens paradas na {fila_morta.node.id}",
                    threshold=0,
                    evaluation_periods=1,

comparison_operator=cloudwatch.ComparisonOperator.GREATER_THAN_THRESHOLD,
                    treat_missing_data=cloudwatch.TreatMissingData.NOT_BREACHING,
                )
            )

        for fn, id_, periodos in ((resumo, "AlarmeResumo", 1), (varredura,
"AlarmeVarredura", 2)):
            avisar(
                fn.metric_errors(period=Duration.minutes(5),
statistic="Sum").create_alarm(
                    self,
                    id_,
                    threshold=1,
                    evaluation_periods=periodos,

comparison_operator=cloudwatch.ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD,
                    treat_missing_data=cloudwatch.TreatMissingData.NOT_BREACHING,
                )
            )

        avisar(erros_5xx)
        avisar(
            db.metric_database_connections(
                period=Duration.minutes(1), statistic="Maximum"
            ).create_alarm(
                self,
                "AlarmeConexoesDb",
                alarm_description="Conexões perto do max_connections (~112 no
db.t4g.micro)",
                threshold=90,
                evaluation_periods=5,
                datapoints_to_alarm=3,
```

```
            )
        )
        avisar(
            db.metric_free_storage_space(
                period=Duration.minutes(5), statistic="Minimum"
            ).create_alarm(
                self,
                "AlarmeDiscoDb",
                alarm_description="Menos de 2 GiB livres no disco do RDS",
                threshold=2 * 1024**3,
                evaluation_periods=3,
                comparison_operator=cloudwatch.ComparisonOperator.LESS_THAN_THRESHOLD,
            )
        )
```

Uma DLQ com qualquer mensagem já é motivo de alerta, e é por isso que o limite é zero. Mensagem na DLQ é como barata na cozinha: quem viu uma sabe que não existe só uma. A métrica `Errors` da Lambda conta exceções e também timeouts, então o resumo de quinta teria disparado o `AlarmeResumo` às 11h01. A varredura pede dois períodos seguidos, porque uma falha isolada se resolve na rodada seguinte; dez minutos sem varredura, não. O alarme de conexões usa o M de N: três minutos de cinco acima de 90 conexões. Na sexta, ele teria disparado por volta de 20h27, antes do pedido do cliente existir. Seria o raro alarme que chega antes da reclamação, e não depois. O `AlarmeDiscoDb` é o alarme de espaço livre que o capítulo 12 deixou prometido: o storage autoscaling aumenta o disco, mas só o alarme conta que algo está enchendo. O autoscaling é o cinto que afrouxa sozinho; o alarme é o amigo que pergunta quantas fatias de pizza foram.

O alarme de 5xx do capítulo 18 contava erros: dez por minuto. Numa noite de pico, dez erros em 3.000 requisições são ruído; numa madrugada, três erros em cinco requisições são um problema. O capítulo 18 prometeu trocar contagens por taxas, e uma expressão matemática faz isso. O novo `erros_5xx` substitui o daquele capítulo, no mesmo lugar, antes do serviço que o usa:

```
        taxa_5xx = cloudwatch.MathExpression(
            expression="100 * FILL(erros, 0) / requisicoes",
            using_metrics={
                "erros": alb.metrics.http_code_target(
                    elbv2.HttpCodeTarget.TARGET_5XX_COUNT, statistic="Sum"
                ),
                "requisicoes": alb.metrics.request_count(statistic="Sum"),
            },
            period=Duration.minutes(1),
            label="% de respostas 5xx",
        )
        erros_5xx = taxa_5xx.create_alarm(
            self,
            "ErrosApi",
            threshold=5,
            evaluation_periods=3,
            datapoints_to_alarm=2,
```

```
comparison_operator=cloudwatch.ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD,
            treat_missing_data=cloudwatch.TreatMissingData.NOT_BREACHING,
        )
```

O `FILL(erros, 0)` preenche com zero os minutos sem 5xx, que não geram ponto. Minuto sem nenhuma requisição deixa a divisão sem dado, e o `NOT_BREACHING` trata isso como normal. O ID continua `ErrosApi`, e o `deployment_alarms` do serviço continua apontando para ele: o rollback automático do blue/green passa a usar a taxa. O tópico de alertas só existe mais adiante na stack, por isso a ação entra depois, com o `avisar(erros_5xx)` do bloco anterior.

São oito alarmes, contando o `ErrosApi`. Com o de cobrança, nove dos dez gratuitos. O Rangô administra a cota gratuita como quem administra o vale-refeição no fim do mês.

Um painel para a sexta à noite

O *dashboard* junta tudo numa tela, a mesma que um dia vai ficar na TV da sala do Rangô, quando o Rangô tiver uma sala e uma TV. Ainda na stack:

```
        def confirmacao(estatistica: str) -> cloudwatch.Metric:
            return cloudwatch.Metric(
                namespace="Rango",
                metric_name="SegundosAteConfirmar",
                dimensions_map={"Servico": "fluxo-pedido"},
                statistic=estatistica,
                period=Duration.minutes(5),
            )

        painel = cloudwatch.Dashboard(
            self, "Painel", dashboard_name=f"rango-{ambiente}",
            default_interval=Duration.hours(6),
        )
        painel.add_widgets(
            cloudwatch.GraphWidget(
                title="Requisições e % de 5xx",
                left=[alb.metrics.request_count(statistic="Sum",
period=Duration.minutes(1))],
                right=[taxa_5xx],
                width=12,
            ),
            cloudwatch.GraphWidget(
                title="Latência da API (segundos)",
                left=[
                    alb.metrics.target_response_time(statistic=s,
period=Duration.minutes(1))
                    for s in ("p50", "p99")
                ],
                width=12,
            ),
        )
        painel.add_widgets(
            cloudwatch.GraphWidget(
                title="Segundos até confirmar o pedido",
                left=[confirmacao("p50"), confirmacao("p90")],
                left_annotations=[cloudwatch.HorizontalAnnotation(value=300, label="5
```

```
min")],
                width=12,
            ),
            cloudwatch.GraphWidget(
                title="Conexões no Postgres e fluxos com falha",
                left=[db.metric_database_connections(statistic="Maximum")],
                right=[maquina.metric_failed()],
                width=12,
            ),
        )
        painel.add_widgets(
            cloudwatch.AlarmStatusWidget(title="Alarmes", alarms=alarmes, width=8),
            cloudwatch.LogQueryWidget(
                title="Erros por grupo de logs",
                log_group_names=[g.log_group_name for g in grupos_logs],
                query_lines=[
                    "filter level = 'ERROR'",
                    "stats count(*) as erros by @log",
                    "sort erros desc",
                ],
                width=16,
            ),
        )
```

O painel fica na stack como qualquer outro recurso: quem recriar o ambiente de homologação ganha o mesmo painel, com as métricas daquele ambiente. Os três primeiros dashboards customizados são gratuitos, com até 50 métricas cada (o quarto costuma ser o painel que alguém montou para uma reunião e nunca mais abriu); o preço a partir do quarto está na página de preços do CloudWatch.

Traces da API até o SNS

Primeiro, a configuração da conta. No console do CloudWatch, em Settings, fica a configuração de traces do X-Ray, onde se ativa o Transaction Search, mantendo a indexação em 1%. O console cria a resource policy que deixa o X-Ray gravar no `aws/spans`. A CLI confirma:

```
aws xray get-trace-segment-destination
```

```
{
    "Destination": "CloudWatchLogs",
    "Status": "ACTIVE"
}
```

A ativação pode levar alguns minutos até aparecer como `ACTIVE`, tempo suficiente para um café e uma dúvida existencial sobre ter clicado no botão certo. O `aws/spans` nasce sem retenção e fora da stack, então a retenção é definida à parte:

```
aws logs put-retention-policy --log-group-name aws/spans --retention-in-days 30
```

Agora a API. O `backend/requirements.txt` ganha o pacote `aws-opentelemetry-distro`, com a versão que o `pip` instalar fixada, como as outras dependências. A AWS exige a versão 0.10.0 ou posterior para mandar spans direto ao X-Ray, sem coletor intermediário. No `backend/Dockerfile`, o comando passa a começar pelo `opentelemetry-instrument`, que carrega a instrumentação antes do Uvicorn:

```
CMD ["opentelemetry-instrument", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--
port", "8000"]
```

A configuração vai em variáveis de ambiente. Na stack, junto das variáveis que o capítulo 21 deu ao `container_api`:

```
        for nome, valor in {
            "OTEL_PYTHON_DISTRO": "aws_distro",
            "OTEL_PYTHON_CONFIGURATOR": "aws_configurator",
            "OTEL_TRACES_EXPORTER": "otlp",
            "OTEL_EXPORTER_OTLP_TRACES_PROTOCOL": "http/protobuf",
            "OTEL_EXPORTER_OTLP_TRACES_ENDPOINT": f"https://xray.
{self.region}.amazonaws.com/v1/traces",
            "OTEL_METRICS_EXPORTER": "none",
            "OTEL_LOGS_EXPORTER": "none",
            "OTEL_RESOURCE_ATTRIBUTES": f"service.name=rango-api,deployment.environment=
{ambiente}",
            "OTEL_PYTHON_FASTAPI_EXCLUDED_URLS": "health",
        }.items():
            container_api.add_environment(nome, valor)

        role_app.add_managed_policy(
            iam.ManagedPolicy.from_aws_managed_policy_name("AWSXrayWriteOnlyPolicy")
        )
        vpc.add_interface_endpoint(
            "XRay", service=ec2.InterfaceVpcEndpointAwsService.XRAY, subnets=privadas
        )
```

Cada variável tem um motivo. As duas primeiras ativam a distribuição da AWS, que assina os envios com as credenciais da task role, sem chave nenhuma. Métricas e logs pelo OpenTelemetry ficam desligados, porque o Rangô já tem os dois pelo CloudWatch. A exclusão do `/health` evita um trace a cada health check do load balancer, que seriam milhares por dia sem informação nenhuma. O `service.name` é o nome com que a API aparece no mapa de serviços. São nove variáveis com `OTEL` no nome: o OpenTelemetry tem muitas qualidades, e a concisão não está entre elas.

O endpoint de interface do X-Ray existe porque as tasks estão em subnets sem saída para a internet, como todo o resto desde o capítulo 15. Sem ele, os spans não saem da VPC, e o exportador registra erros de conexão em silêncio, o tipo mais educado e mais inútil de erro.

A amostragem fica no padrão: sem coletor, o SDK grava 100% dos traces. A AWS sugere 5% para volumes altos, com `OTEL_TRACES_SAMPLER=parentbased_traceidratio` e `OTEL_TRACES_SAMPLER_ARG=0.05`. Com 1.000 pedidos por dia, o Rangô gera pouco volume, e investigar o pedido de um cliente específico exige o trace daquele pedido. Amostrar 5% e torcer para o cliente irritado estar nesses 5% é estratégia de loteria.

As Lambdas já ganharam o `tracing=lambda_.Tracing.ACTIVE` lá atrás: com ele, o próprio serviço Lambda registra cada chamada, sem SDK nenhum no código. Falta o Step Functions. A `sfn.StateMachine` do capítulo 21 ganha:

```
            tracing_enabled=True,
```

O CDK soma as permissões do X-Ray às roles das funções e da máquina. Depois do commit, do deploy aprovado e de um pedido novo, o `trace_id` sai da linha "pedido criado":

```
scripts/consultar-logs.sh "filter message = 'pedido criado' | fields trace_id | sort
@timestamp desc | limit 1" 10
```

O X-Ray mostra o ID no formato `1-` mais os 8 primeiros caracteres, um hífen e os 24 restantes. Um `trace_id` `6710a3c2b7e94f0d8c21a4f3e5d6c7b8` vira `1-6710a3c2-b7e94f0d8c21a4f3e5d6c7b8`. Esse é o valor para a busca de traces do console do CloudWatch. A linha do tempo será parecida com esta (durações ilustrativas):

```
rango-api        POST /pedidos ........................ 186 ms
  ├─ redis       SET idem:...                            2 ms
  ├─ postgres    INSERT pedidos                         11 ms
  └─ StepFunctions StartExecution                       43 ms
rango-producao-pedido (Step Functions)
  ├─ AguardarPagamento → Lambda guardar-token          212 ms
  ├─ ConfirmarRestaurante → Lambda confirmar-restaurante 1.304 ms
  │    └─ Initialization (cold start)                  521 ms
  └─ PublicarConfirmado → SNS Publish                   38 ms
```

O Step Functions continua o trace que chegou no cabeçalho da chamada da API, e cada Lambda aparece com a duração dela, incluindo o cold start. O cold start, enfim, tem rosto e endereço na tela. A passagem do SNS pelas filas até as Lambdas de aviso nem sempre continua o mesmo trace; esse último trecho, o `pedido_id` nos logs já cobre. É a combinação dos dois que leva um pedido da API até o SMS.

Um SLO para o tempo até confirmar

A sintaxe para criar SLOs pela stack muda mais rápido que este livro, então este vai pelo console. No CloudWatch, o caminho é Application Signals, depois Service Level Objectives, e o SLO tem esta configuração:

1. Indicador: métrica do CloudWatch, `Rango` › `SegundosAteConfirmar` com `Servico=fluxo-pedido`, estatística p90.
2. Condição: menor ou igual a 300 (segundos).
3. Cálculo por períodos de 5 minutos.
4. Intervalo móvel de 7 dias, meta de 99%.
5. Nome: `rango-confirmacao-5min`.

Na mesma tela, entra um alarme de burn rate com janela de uma hora e limite 10, com ação no tópico `rango-producao-alertas`. Com burn rate 10, o orçamento da semana acaba em menos de 17 horas: dá tempo de agir antes, e um período ruim isolado não chega perto do limite. Ele é o décimo alarme da conta, o último dentro dos dez gratuitos.

O SLO não vive na stack, então sobrevive ao `cdk destroy`. Como a métrica dele vem da Lambda, ele simplesmente fica sem dados enquanto a stack não existir. Quem não quiser mantê-lo pode apagá-lo no console ao fim do capítulo.

Um segundo SLO natural para o Rangô seria a disponibilidade da API, usando como indicador a mesma expressão do `taxa_5xx`, com meta de 99,5% em períodos de um minuto. O Application Signals já calcula disponibilidade como (1 – taxa de *faults*) × 100, em que fault é resposta 5xx e 4xx conta como sucesso, mas só para serviços que ele descobre, o que exige ligá-lo na ADOT, com custo próprio.

O teste do alarme

O teste simula a quinta-feira. A variável `FALHA_AVISOS` da Lambda de e-mail vai para `1` (um drift proposital, como o `RECUSAR` do capítulo anterior), e dois pedidos são criados. Cada mensagem é recebida cinco vezes, uma a cada dois minutos, e vai para a DLQ. Uns 15 minutos depois, chega o e-mail:

```
ALARM: "RangoProducao-AlarmeDlqemail..." in US East (N. Virginia)
Threshold Crossed: 1 out of the last 1 datapoints [2.0 (16/10/26 14:41:00)] was greater
than the threshold (0.0)
```

Nunca um e-mail com "ALARM" no assunto foi recebido com tanta alegria. Com a variável de volta em `0` e as mensagens devolvidas à fila com o `start-message-move-task` do capítulo 21, chega o e-mail de `OK`. O painel mostra tudo isso no mesmo intervalo: o alarme vermelho, o pico de erros no grupo `avisos-email` e a volta ao normal.

Quanto custa enxergar

Para o Rangô com 1.000 pedidos por dia, em `us-east-1`, valores aproximados de outubro de 2026:

Item	Custo mensal
Logs (cerca de 2 GB)	dentro dos 5 GB gratuitos
1 custom metric (`SegundosAteConfirmar`)	~US$ 0,30
8 alarmes + 1 de burn rate	dentro dos 10 gratuitos
1 dashboard	gratuito (até 3)
Spans no `aws/spans`	centavos a poucos dólares
Endpoint de interface do X-Ray (2 AZs)	~US$ 14,60

O item mais caro é o endpoint, e não a observabilidade em si. Enxergar é barato; caro é o caminho até os olhos. É o preço de manter as tasks sem acesso à internet, a mesma conta dos endpoints do capítulo 17. O capítulo 24 volta a ela.

No fim, a stack é destruída como de costume, e os workflows, desativados:

```
cd rango-infra
cdk destroy RangoProducao
gh workflow disable ci.yaml
gh workflow disable renovar-tasks.yaml
```

O Transaction Search continua ativo na conta, mas só cobra quando chegam spans, como o cartão de crédito sem anuidade que você nunca usa, só que honesto. O `aws/spans` fica, com a retenção de 30 dias.

Armadilhas e casos reais

> **Armadilha:** Dimensão com valor único por pedido. Se o `publicar_metrica` usasse `pedido_id` como dimensão, cada pedido viraria uma métrica diferente. Com 1.000 pedidos por dia, seriam 30.000 métricas por mês, cobradas proporcionalmente às horas em que recebem dados, e a conta do CloudWatch passaria a ter quatro dígitos. Dimensões são para valores de poucos tipos (serviço, ambiente, rota); o ID do pedido vai no log.
>
> **Armadilha:** Alarme de 5xx sobre taxa com pouco tráfego. De madrugada, uma requisição com erro em duas é 50%. O `datapoints_to_alarm=2` ajuda, mas um serviço com tráfego muito baixo pode precisar de uma condição extra: só alertar se houver mais de N requisições no minuto, com uma segunda expressão.
>
> **Armadilha:** O que não rodou não gera erro. Se o agendamento do resumo for apagado por engano, a métrica `Errors` fica zerada para sempre, e o `AlarmeResumo` nunca dispara. Para detectar ausência, alarme sobre `Invocations` com

`treat_missing_data` em `breaching`, numa janela que cubra o intervalo entre execuções.

Armadilha: Média de percentis. O p99 de cada task, somado e dividido por seis, dá um número diferente do p99 do serviço. Calcule o percentil sobre a métrica agregada, como o `TargetResponseTime` do load balancer, ou com `pct()` no Logs Insights sobre todas as linhas.

Armadilha: Dados pessoais no log. Um `log.info("pedido", extra={"pedido": pedido})` grava endereço, telefone e, no pior caso, dados de pagamento num lugar que muita gente da equipe lê e que fica guardado por 30 dias. Logue IDs e busque o resto no banco. O CloudWatch Logs tem políticas de proteção de dados que mascaram padrões conhecidos, mas elas são uma segunda camada, e a LGPD conta o que foi gravado.

Armadilha: Consulta de 30 dias em todos os grupos. O Logs Insights cobra pelo volume escaneado. Comece pelo intervalo mais curto que responde a pergunta e aumente se precisar.

Na vida real: Em 4 de janeiro de 2021, o Slack teve uma falha grave no primeiro dia útil do ano. Segundo o postmortem, no meio do incidente os serviços de dashboard e de alerta da própria empresa ficaram indisponíveis, porque dependiam da mesma rede que estava com problemas. Era o porteiro avisando pelo interfone que o interfone quebrou. O Rangô tem a mesma dependência: se `us-east-1` tiver um dia ruim, o CloudWatch dessa região pode sofrer junto. Para um sistema do tamanho do Rangô, a defesa razoável é um verificador externo simples, de fora da AWS, que acesse `https://api.rango.com.br/health` a cada minuto.

Resumo

- O CloudWatch reúne logs, métricas, alarmes, dashboards, traces (com o X-Ray) e SLOs. Os serviços da AWS publicam as próprias métricas de graça.
- Logs em JSON com o mesmo vocabulário (`level`, `message`, `pedido_id`, `trace_id`) deixam o Logs Insights filtrar sem configuração, em até 50 log groups por consulta. Nomes com prefixo comum (`/rango/producao/...`) tornam a busca em todos os grupos um comando.
- Métricas de negócio saem de uma Lambda pelo Embedded Metric Format, uma linha de log. Dimensões só com valores de poucos tipos.
- Alarmes têm três estados, M de N para ignorar picos isolados e uma escolha consciente para dado ausente. Toda DLQ merece alarme com limite zero.

- Taxa de erro, calculada com expressão matemática, substitui a contagem do capítulo 18, inclusive no rollback automático.
- O SDK e o Daemon do X-Ray estão em manutenção desde fevereiro de 2026, com fim de suporte em fevereiro de 2027. O caminho agora é OpenTelemetry (ADOT) mandando spans ao X-Ray, com Transaction Search gravando tudo no `aws/spans`.
- Um SLO transforma "está lento" em orçamento de erro, e o alarme de burn rate avisa quando o orçamento está acabando rápido demais.

Quem apagou a fila?

Segunda-feira, 8h40, você abre o painel antes do café, o que é sinal de maturidade profissional ou de insônia. O gráfico de requisições tem um pico às 3h17 que não combina com o horário. A taxa de 5xx está normal, mas o Logs Insights mostra 41 mil respostas 401 em duas horas, vindas de poucos endereços IP, na `POST /painel/entrar`, o login do painel criado no capítulo 20. Alguém está testando senhas em massa. Às 3h17 de uma segunda-feira, ninguém com boas intenções tem tanto interesse no painel da Cantina da Nona.

E o gráfico da fila de SMS parou às 7h02. A fila `rango-producao-sms` não existe mais. Ninguém da equipe lembra de ter apagado nada. Toda equipe tem esse fantasma, e ele sempre tem permissão de administrador. Os logs do Rangô contam o que a aplicação fez, mas não quem chamou a API da AWS para apagar uma fila. No próximo capítulo: AWS CloudTrail para saber quem fez o quê, GuardDuty para detectar comportamento estranho na conta, WAF na frente do CloudFront e do load balancer, e a separação em várias contas que impede um engano de virar desastre.

Referências

- Amazon CloudWatch Logs, User Guide: https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html
- Logs Insights, sintaxe de consulta: https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/CWL_QuerySyntax.html
- Cotas do CloudWatch Logs (campos descobertos, anomaly detectors): https://docs.aws.eu/AmazonCloudWatch/latest/logs/cloudwatch_limits_cwl.html
- Aumento de cotas do Logs Insights (50 log groups, 60 minutos), 2023: https://aws.amazon.com/about-aws/whats-new/2023/02/amazon-cloudwatch-

increased-quotas-logs-insights

- Aumento de consultas concorrentes do Logs Insights, 2026: https://aws.amazon.com/about-aws/whats-new/2026/03/amazon-cloudwatch-logs-increased-limits
- Embedded Metric Format, especificação: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Embedded_Metric_Format_Specification.html
- Metric math: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/using-metric-math.html
- CDK, `TreatMissingData`: https://docs.aws.amazon.com/cdk/api/v2/python/aws_cdk.aws_cloudwatch/TreatMissingData.html
- Avaliação de alarmes e dados ausentes (re:Post): https://www.repost.aws/articles/ARtfXuUalKQLycBz2IfuVavQ/understanding-cloudwatch-alarm-evaluation-why-alarms-delay-fire-unexpectedly-or-get-stuck
- Preços do CloudWatch: https://aws.amazon.com/cloudwatch/pricing/
- Fim do suporte aos SDKs e ao Daemon do X-Ray e migração para OpenTelemetry: https://aws.amazon.com/blogs/mt/announcing-aws-x-ray-sdks-daemon-end-of-support-and-opentelemetry-migration
- Transaction Search: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Enable-TransactionSearch.html
- Transaction Search com Lambda: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Enable-Lambda-TransactionSearch.html
- Envio sem coletor com a ADOT: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-OTLP-UsingADOT.html
- SLOs no Application Signals: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-ServiceLevelObjectives.html
- Burn rate para SLOs, anúncio de novembro de 2024: https://aws.amazon.com/about-aws/whats-new/2024/11/application-signals-burn-rate-application-performance-goals
- Google SRE Book, Service Level Objectives: https://sre.google/sre-book/service-level-objectives/

- Google SRE Workbook, alertas sobre SLOs: https://sre.google/workbook/alerting-on-slos/
- Slack, postmortem de 4 de janeiro de 2021: https://slack.engineering/slacks-outage-on-january-4th-2021/

CAPÍTULO 23

Segurança em produção

No capítulo anterior, o Rangô ganhou olhos: logs em JSON com o mesmo vocabulário em todos os grupos, alarmes nas DLQs e na taxa de erro, um dashboard, traces com OpenTelemetry e um SLO para a confirmação do pedido. Foi esse painel que mostrou, numa segunda-feira de manhã, duas coisas que a observabilidade da aplicação sozinha não explica: um robô testando senhas e uma fila que sumiu. Os logs contam o que o Rangô fez. Agora precisamos saber o que fizeram com a conta. E tinha que ser numa segunda-feira, claro: nenhum incidente sério teve até hoje a delicadeza de esperar a quinta à tarde, com o café já tomado.

Segunda-feira, 16 de novembro, 8h40

Vamos retomar a cena com os detalhes. A rota que recebeu as 41 mil respostas 401 é a `POST /painel/entrar`, o login do painel que o capítulo 20 criou para a Cantina da Nona e o Sabor do Sertão acompanharem as entregas. As tentativas começaram às 3h17 e vieram de seis endereços na faixa `203.0.113.0/24`, cada um com um ritmo constante de umas 57 requisições por minuto. Ninguém usa um login assim. É um programa percorrendo uma lista de e-mails e senhas vazados de outros sites, apostando que algum dono de restaurante repetiu a senha. A aposta é boa. Em algum lugar do Brasil, neste exato momento, alguém usa `Cantina123` em seis sites diferentes e se sente muito esperto por ter colocado o número no final.

Nada impediu o robô. O ALB entregou cada requisição, a API consultou a tabela do painel e calculou um hash Argon2 41 mil vezes, com a paciência de um caixa de banco em véspera de feriado, e uma única senha certa abriria o painel de um restaurante, com os pedidos em andamento e a posição de cada entregador na rua.

A fila incomoda mais. Robô testando senha é chato; fila sumindo sozinha é o tipo de coisa que faz a gente olhar desconfiado para o próprio teclado. O alarme da DLQ de SMS não tocou, porque a DLQ continuava vazia: a fila principal deixou de existir às 7h02, com as mensagens dentro. O `cdk drift` acusa o recurso como `DELETED`, e o `cdk deploy` de rotina não traz de volta, como o capítulo 15 avisou.

Três perguntas ficam na mesa. Quem apagou a fila, de onde e com que credencial? Como parar o robô antes que ele chegue à API? E por que uma pessoa da equipe conseguiu apagar um recurso de produção?

A terceira pergunta é a mais desconfortável. Desde o capítulo 8, o Rangô inteiro mora numa conta só, a `rango-estudos`, que também é a conta de gerenciamento da organização. A homologação, a segunda instância da `RangoInfraStack` que o capítulo 15 previu, a equipe sobe ali mesmo quando precisa testar algo; você, leitor, vinha destruindo só a produção no fim de cada capítulo. Homologação e produção dividem a conta, e o grupo `rango-devs` tem `PowerUserAccess` em tudo. O capítulo 16 já tinha avisado que esse permission set lê os segredos de produção. Estava funcionando por sorte. Sorte é uma estratégia de segurança com um histórico impecável, até o dia em que deixa de ter.

CloudTrail: o registro de quem chamou a API

Toda ação na AWS é uma chamada de API. Clicar em "Delete" no console, rodar `aws sqs delete-queue`, um `cdk destroy`, um `boto3` dentro de uma Lambda: tudo vira uma requisição assinada para um endpoint. O **AWS CloudTrail** registra essas chamadas. Cada registro é um **evento** com quem chamou (a identidade, a role, a sessão), quando, de qual IP, com qual programa, quais parâmetros foram enviados e se a AWS aceitou ou recusou.

Pense no `git log` da conta, gravado pela própria AWS: o autor de cada linha é a credencial que assinou a chamada. Ao contrário do `git blame`, ele não aceita a desculpa de que a linha veio de um merge.

Os **eventos de gerenciamento** são as operações sobre os recursos: criar uma fila, apagar uma tabela, assumir uma role. Ficam registrados por padrão e sem custo. Os **eventos de dados** são as operações dentro dos recursos, como um `GetObject` no bucket de fotos ou um `PutItem` no DynamoDB. São muito mais numerosos, ficam desligados por padrão e são cobrados à parte (a AWS tem um talento raro para transformar curiosidade em item de fatura), assim como o **Insights**, que procura picos anormais de chamadas.

Os eventos de gerenciamento aparecem no **Event History** pelos últimos 90 dias, por região. Para guardar por mais tempo, num lugar de onde ninguém consiga apagá-los, você cria uma **trail**, que entrega cópias dos eventos em arquivos num bucket S3. A primeira cópia dos eventos de gerenciamento não tem custo do CloudTrail; você paga só o S3. O CloudTrail Lake, que guarda eventos consultáveis por SQL, existe para quem precisa de consultas frequentes; para o Rangô, a trail basta.

Encontrando quem apagou a fila

O Event History já tem a resposta. Com o profile de administrador, a consulta fica assim:

```
aws cloudtrail lookup-events \
  --profile rango-admin \
  --region us-east-1 \
  --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteQueue \
  --start-time 2026-11-16T00:00:00Z \
  --query 'Events[].CloudTrailEvent' \
  --output text | jq '{eventTime, eventName, sourceIPAddress, userAgent, arn:
.userIdentity.arn, fila: .requestParameters.queueUrl}'
```

O `lookup-events` aceita um atributo de busca por vez (nome do evento, usuário, recurso, origem) e devolve o evento original como uma string JSON dentro do campo `CloudTrailEvent`, por isso o `jq`. A saída:

```
{
  "eventTime": "2026-11-16T10:02:41Z",
  "eventName": "DeleteQueue",
  "sourceIPAddress": "198.51.100.77",
  "userAgent": "aws-cli/2.31.4 md/awscrt#0.27.6 ua/2.1 os/linux#6.8.0-48-generic",
  "arn": "arn:aws:sts::123456789012:assumed-
role/AWSReservedSSO_PowerUserAccess_3f9a1c2b7d4e5f60/diego",
  "fila": "https://sqs.us-east-1.amazonaws.com/123456789012/rango-producao-sms"
}
{
  "eventTime": "2026-11-16T10:02:39Z",
  "eventName": "DeleteQueue",
  "sourceIPAddress": "198.51.100.77",
  "userAgent": "aws-cli/2.31.4 md/awscrt#0.27.6 ua/2.1 os/linux#6.8.0-48-generic",
  "arn": "arn:aws:sts::123456789012:assumed-
role/AWSReservedSSO_PowerUserAccess_3f9a1c2b7d4e5f60/diego",
  "fila": "https://sqs.us-east-1.amazonaws.com/123456789012/rango-homologacao-sms"
}
```

O horário está em UTC: 10h02 é 7h02 em Brasília, o minuto em que o gráfico parou. A identidade é uma sessão da role que o Identity Center cria para o permission set `PowerUserAccess`, e o nome da sessão é o usuário: `diego`. O user agent mostra a CLI rodando num Linux. Duas filas apagadas com dois segundos de diferença. Uma eficiência admirável, apontada para o alvo errado.

Diego entrou na equipe há duas semanas e está testando um provedor novo de SMS em homologação. A conversa durou cinco minutos e não teve nada de dramático. O Diego ficou da cor do molho da Cantina da Nona, pediu desculpas três vezes e mostrou o que tinha rodado. O script de limpeza da manhã era este:

```
aws sqs list-queues --queue-name-prefix rango- --query 'QueueUrls[]' --output text \
  | tr '\t' '\n' | grep -- '-sms$' | xargs -n1 aws sqs delete-queue --queue-url
```

O prefixo `rango-` pega as filas dos dois ambientes. Ninguém agiu de má-fé; a conta permitiu. Um `grep` mal ajustado é a forma mais democrática de desastre que existe: qualquer pessoa, em qualquer nível de senioridade, já escreveu pelo menos um. No postmortem sem culpados, a causa raiz ficou registrada assim: produção e homologação na mesma conta, com a mesma permissão para os dois.

A fila foi recriada com o mesmo nome (o SQS exige esperar 60 segundos para reusar um nome), com os atributos do `cdk synth` e a política de acesso do SNS reaplicada. As mensagens que estavam nela se perderam.

> **Dica:** Para ver tudo o que uma pessoa fez, use `AttributeKey=Username,AttributeValue=diego`. Para tudo o que aconteceu com um recurso, `AttributeKey=ResourceName`.

Um detalhe que confunde: o Event History é por região, e serviços globais como o IAM registram em `us-east-1`. Um invasor que cria máquinas em `ap-southeast-1` não aparece na sua consulta de `us-east-1`. A trail multirregião, mais adiante, junta tudo.

O robô no login: AWS WAF

O CloudTrail explica a fila. O robô pede outra ferramenta, porque as requisições dele nunca passam pela API da AWS: elas chegam ao ALB, que faz o trabalho dele direitinho e entrega cada uma ao Uvicorn. O ALB é um funcionário exemplar e nada desconfiado: entrega com o mesmo capricho a primeira tentativa de senha e a de número 41 mil.

O **AWS WAF** (*Web Application Firewall*) é um filtro de requisições HTTP que roda antes da sua aplicação, no CloudFront, no ALB ou no API Gateway. A analogia com o FastAPI é direta: um middleware que inspeciona a requisição (IP, caminho, cabeçalhos, corpo) e decide se ela segue, é bloqueada ou só é contada. A diferença é que esse middleware roda na infraestrutura da AWS, e a requisição bloqueada nunca consome CPU das suas tasks. É o segurança que barra o penetra na calçada, antes que ele chegue perto do bufê.

Uma **web ACL** é o conjunto de regras associado a um recurso, com uma ação padrão para o que nenhuma regra decidir. Cada **regra** tem uma condição e uma ação (`Allow`, `Block`, `Count`, que só registra, ou um `CAPTCHA`), e as regras são avaliadas por prioridade, da menor para a maior. Os **managed rule groups** são pacotes de regras mantidos pela AWS e atualizados sem você mexer: padrões comuns de ataque, entradas maliciosas conhecidas, IPs de má reputação. Uma web ACL comporta 1.500 WCU (*web ACL capacity units*, a medida do custo de processamento das regras) no preço base. Sim, uma unidade de medida própria. A AWS coleciona siglas como cartório coleciona carimbos.

A peça que resolve o robô é a **regra baseada em taxa** (*rate-based rule*): ela conta requisições por IP numa janela de tempo e bloqueia quem passar do limite. Um dono de restaurante erra a senha duas ou três vezes; ninguém tenta 20 vezes em cinco minutos. Quem faz isso é um robô ou uma pessoa que precisa, com urgência, de uma pausa e de um gerenciador de senhas.

O WAF tem web ACLs regionais (ALB, API Gateway) e web ACLs para CloudFront, que só existem em `us-east-1`. A API do Rangô chega direto ao ALB por `api.rango.com.br`, então o login fica numa web ACL regional. O front ganha uma web ACL própria, mais simples.

Valores de referência da página oficial, outubro de 2026: US$ 5,00 por web ACL por mês, US$ 1,00 por regra ou rule group por mês e US$ 0,60 por milhão de requisições, à parte do ALB e do CloudFront. A web ACL da API, com uma regra de taxa e três rule groups, custa US$ 9 por mês; a do front, US$ 6. O Bot Control (US$ 10,00 por mês por web ACL, mais requisições) e o Fraud Control ficam para quando o ataque vier de milhares de IPs residenciais, cada um tentando poucas vezes, o que passa por baixo de qualquer limite por IP.

Na prática: a web ACL da API

No `rango_infra_stack.py`, os imports ganham `aws_wafv2 as wafv2`. Os rule groups gerenciados repetem a mesma estrutura, então uma função ajuda:

```
def visibilidade(nome: str) -> wafv2.CfnWebACL.VisibilityConfigProperty:
    return wafv2.CfnWebACL.VisibilityConfigProperty(
        cloud_watch_metrics_enabled=True,
        metric_name=nome,
        sampled_requests_enabled=True,
    )


def regra_gerenciada(nome: str, prioridade: int) -> wafv2.CfnWebACL.RuleProperty:
    return wafv2.CfnWebACL.RuleProperty(
        name=nome,
        priority=prioridade,
        statement=wafv2.CfnWebACL.StatementProperty(

managed_rule_group_statement=wafv2.CfnWebACL.ManagedRuleGroupStatementProperty(
                vendor_name="AWS",
                name=nome,
            )
        ),
        # count={} = só registra; trocar por none={} depois de uma semana sem falso
positivo
        override_action=wafv2.CfnWebACL.OverrideActionProperty(count={}),
        visibility_config=visibilidade(nome),
    )
```

O WAF não tem construções L2 no CDK, então usamos as L1 (`Cfn*`), que espelham o CloudFormation campo a campo. É mais verboso, e só isso. Quem sente saudade de Java mata a saudade aqui.

Agora a regra do login e a web ACL, depois da criação do `alb` :

```
        limite_login = wafv2.CfnWebACL.RuleProperty(
            name="LimiteLogin",
            priority=0,
            action=wafv2.CfnWebACL.RuleActionProperty(block={}),
            statement=wafv2.CfnWebACL.StatementProperty(
                rate_based_statement=wafv2.CfnWebACL.RateBasedStatementProperty(
                    limit=20,
                    evaluation_window_sec=300,
                    aggregate_key_type="IP",
                    scope_down_statement=wafv2.CfnWebACL.StatementProperty(
                        byte_match_statement=wafv2.CfnWebACL.ByteMatchStatementProperty(

field_to_match=wafv2.CfnWebACL.FieldToMatchProperty(uri_path={}),
                            positional_constraint="EXACTLY",
                            search_string="/painel/entrar",
                            text_transformations=[
                                wafv2.CfnWebACL.TextTransformationProperty(priority=0,
type="NONE")
                            ],
                        )
                    ),
                )
            ),
            visibility_config=visibilidade("LimiteLogin"),
        )

        firewall_api = wafv2.CfnWebACL(
            self,
            "FirewallApi",
            name=f"rango-{ambiente}-api",
            scope="REGIONAL",
            default_action=wafv2.CfnWebACL.DefaultActionProperty(allow={}),
            rules=[
                limite_login,
                regra_gerenciada("AWSManagedRulesAmazonIpReputationList", 1),
                regra_gerenciada("AWSManagedRulesKnownBadInputsRuleSet", 2),
                regra_gerenciada("AWSManagedRulesCommonRuleSet", 3),
            ],
            visibility_config=visibilidade(f"rango-{ambiente}-api"),
        )
        wafv2.CfnWebACLAssociation(
            self,
            "FirewallApiNoAlb",
            resource_arn=alb.load_balancer_arn,
            web_acl_arn=firewall_api.attr_arn,
        )
```

A regra `LimiteLogin` conta, por IP, só as requisições para `/painel/entrar` (o papel do `scope_down_statement`), numa janela de 300 segundos. Passou de 20, o IP recebe 403 até a taxa cair. O resto da API fica fora da conta.

Os rule groups começam em `Count`: você observa por alguns dias o que cada um teria bloqueado e só então passa a bloquear. É o período de experiência do firewall: ele aponta quem barraria, mas ainda não tem autorização para barrar ninguém. A regra de taxa já nasce bloqueando, porque é estreita e você sabe o que ela pega.

A web ACL do front, criada na mesma stack em `us-east-1`, usa só a lista de reputação:

```
        firewall_front = wafv2.CfnWebACL(
            self,
            "FirewallFront",
            name=f"rango-{ambiente}-front",
            scope="CLOUDFRONT",
            default_action=wafv2.CfnWebACL.DefaultActionProperty(allow={}),
            rules=[regra_gerenciada("AWSManagedRulesAmazonIpReputationList", 0)],
            visibility_config=visibilidade(f"rango-{ambiente}-front"),
        )
```

A `cloudfront.Distribution` existente ganha `web_acl_id=firewall_front.attr_arn`. Depois do commit, o pipeline do capítulo 18 roda o `cdk deploy`.

Para conferir a regra de taxa, o teste é um laço disparado de uma máquina própria (nunca contra o sistema de outra pessoa; isso tem outro nome, e o nome aparece em boletim de ocorrência):

```
for i in $(seq 1 30); do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST
https://api.rango.com.br/painel/entrar \
    -H 'Content-Type: application/json' -d
'{"email":"teste@rango.com.br","senha":"errada"}'
done | sort | uniq -c
```

O WAF agrega as contagens e pode levar até um minuto ou mais para começar a bloquear. Depois de duas ou três rodadas do laço, a saída fica parecida com:

```
     22 401
      8 403
```

Os 403 vêm do WAF, sem passar pela API, e as requisições amostradas no console mostram o IP da sua máquina. Em cinco minutos sem tentativas, o bloqueio cai sozinho. Ser barrado pelo próprio firewall, aliás, é a melhor notícia do dia.

> **Armadilha:** O WAF protege o login de força bruta vinda de poucos IPs, e só. Ele não substitui as defesas da própria aplicação: atraso progressivo depois de erros seguidos na mesma conta, mensagem de erro igual para e-mail inexistente e senha errada, MFA para os donos de restaurante e hash de senha com um algoritmo

> próprio para isso (Argon2 ou bcrypt, via uma biblioteca estabelecida). Código de autenticação deve passar por revisão de alguém de segurança antes de ir para produção.

E o Shield?

CloudFront, Route 53 e load balancers já vêm com o **AWS Shield Standard**, sem custo e sem configuração, contra os ataques de negação de serviço mais comuns nas camadas 3 e 4. O **Shield Advanced** custa US$ 3.000 por mês, com compromisso de um ano, mais uma taxa sobre a transferência de dados (referência, outubro de 2026), e acrescenta proteção de camada 7 via WAF. Compensa quando uma hora fora do ar custa mais que a assinatura; para o Rangô, o Standard e a regra de taxa bastam. Três mil dólares por mês é mais do que a Cantina da Nona fatura em lasanha, e olha que a lasanha é boa.

Organizações com várias contas

Dá para resolver o problema do Diego com IAM, escrevendo policies que distinguem `rango-producao-*` de `rango-homologacao-*`. Funciona até a próxima fila ou o próximo serviço: uma cerca feita de exceções. Policy cheia de exceção envelhece como regex: ninguém lembra por que aquele trecho está ali, e todo mundo tem medo de mexer.

A fronteira mais simples é a **conta**. Recursos de uma conta não enxergam os de outra sem que você abra a porta dos dois lados, como no acesso entre contas do capítulo 8. Um `delete-queue` em homologação nunca encontra a fila de produção. É a ideia de ter bancos separados para teste e produção, em vez de prefixar as tabelas. A prática do mercado, inclusive em empresas pequenas, é uma organização com várias contas:

```
Organização (conta de gerenciamento: rango-estudos)
  │   só faturamento, Organizations, Identity Center
  │
  ├── OU Seguranca
  │     └── rango-seguranca   logs do CloudTrail, administrador do GuardDuty
  │
  └── OU Cargas
        ├── rango-homologacao
        └── rango-producao
```

Uma **OU** (*organizational unit*) é uma pasta de contas, onde você pendura políticas que valem para tudo dentro dela. A conta de gerenciamento fica vazia de propósito, por um motivo que aparece nas SCPs. Contas novas não custam nada. É uma das raras frases deste livro que dispensa asterisco.

Criando as contas

O Organizations está ligado desde o capítulo 8, com `rango-estudos` como conta de gerenciamento. Com o profile `rango-admin`, que aponta para ela:

```
export AWS_PROFILE=rango-admin

RAIZ=$(aws organizations list-roots --query 'Roots[0].Id' --output text)
OU_SEG=$(aws organizations create-organizational-unit --parent-id "$RAIZ" \
  --name Seguranca --query 'OrganizationalUnit.Id' --output text)
OU_CARGAS=$(aws organizations create-organizational-unit --parent-id "$RAIZ" \
  --name Cargas --query 'OrganizationalUnit.Id' --output text)

aws organizations create-account --email aws-seguranca@rango.com.br --account-name
rango-seguranca
aws organizations create-account --email aws-homologacao@rango.com.br --account-name
rango-homologacao
aws organizations create-account --email aws-producao@rango.com.br --account-name rango-
producao
```

A criação é assíncrona: `aws organizations describe-create-account-status --create-account-request-id <id>` mostra quando vira `SUCCEEDED`, com o número da conta. Cada e-mail precisa ser único na AWS inteira. Em pouco tempo, o domínio do Rangô tem mais endereços `aws-` do que funcionários.

As contas nascem na raiz, e cada uma vai para a sua OU:

```
aws organizations move-account --account-id 210987654321 \
  --source-parent-id "$RAIZ" --destination-parent-id "$OU_SEG"
aws organizations move-account --account-id 345678901234 \
  --source-parent-id "$RAIZ" --destination-parent-id "$OU_CARGAS"
aws organizations move-account --account-id 456789012345 \
  --source-parent-id "$RAIZ" --destination-parent-id "$OU_CARGAS"
```

Os números são exemplos (segurança, homologação e produção). No Identity Center, o grupo `rango-devs` recebe `PowerUserAccess` em `rango-homologacao` e um permission set novo, `SomenteLeitura`, em `rango-producao`, baseado na policy gerenciada `ViewOnlyAccess`, que lista e descreve recursos sem ler conteúdo (nada de `GetSecretValue`), mais uma inline para o Logs Insights. O grupo `rango-admins` recebe `AdministratorAccess` nas três contas, e o deploy em produção passa a ser só do pipeline. Os humanos ficam com o papel de plateia em produção, que é o mais seguro para todo mundo, inclusive para os humanos.

Root centralizado

O capítulo 8 prometeu tratar do root das contas novas. Promessa de capítulo, ao contrário de promessa de sprint, é cumprida. Em vez de configurar MFA em três roots, a AWS permite remover as credenciais de root das contas membro e executar as tarefas só-root a partir da conta de gerenciamento:

```
aws organizations enable-aws-service-access --service-principal iam.amazonaws.com
aws iam enable-organizations-root-credentials-management
aws iam enable-organizations-root-sessions
```

A partir daí, contas novas da organização nascem sem credenciais de root. É uma senha a menos para alguém guardar num post-it, o que já conta como avanço da civilização. O root da conta de gerenciamento continua existindo, com o MFA que você configurou no capítulo 7.

A mudança do Rangô para a conta nova

Tirar o Rangô da conta de gerenciamento dá trabalho nos recursos com estado ou fora da stack. É uma mudança de casa em que as caixas são snapshots criptografados e ninguém aparece para ajudar em troca de pizza. Na ordem:

1. `cdk bootstrap` na conta `rango-producao` com `--cloudformation-execution-policies` apontando para uma policy restrita aos serviços do Rangô: a exec role restrita que o capítulo 15 deixou para depois.
2. `RangoRegistro` e `RangoPipeline` na conta nova; a variável `AWS_ACCOUNT_ID` do repositório muda.
3. Hosted zone nova com os registros copiados, NS trocados no Registro.br, certificado do ACM reemitido e ARN novo no `cdk.json`.
4. Snapshots criptografados com a chave `aws/rds` não podem ser compartilhados: o caminho é copiar com uma chave customer managed liberada para a conta de produção, compartilhar a cópia e restaurar.
5. Bucket de fotos novo e `aws s3 sync`.
6. `cdk deploy` da `RangoProducao` pelo pipeline.

Homologação segue o mesmo caminho, com `ambiente="homologacao"`.

SCPs: o teto das permissões

Contas separadas resolvem o engano entre ambientes. Falta proteger cada conta de quem tem permissão demais dentro dela, inclusive um administrador com pressa ou uma credencial roubada.

Uma **SCP** (*service control policy*) é uma policy no formato que você conhece, anexada a uma OU ou a uma conta, que define o máximo que qualquer identidade daquela conta pode fazer. O capítulo 8 já mostrou onde ela entra na avaliação: a permissão efetiva é a interseção. Se a SCP nega, nenhum `Allow` em role ou usuário da conta consegue liberar.

Pense num decorator aplicado acima de todos os outros decorators de permissão, que pode tirar acesso e nunca dar. Ou no síndico do prédio: você tem a chave do apartamento, mas pintar a fachada de roxo depende dele.

As regras que mais confundem, conferidas na documentação:

- SCPs só existem em organizações com *all features* habilitado. Uma organização só com faturamento consolidado não tem.
- SCPs não concedem permissão. A política `FullAWSAccess`, anexada por padrão em tudo, só diz "o teto é tudo". Se você a remover sem colocar outra no lugar, todas as ações das contas abaixo passam a falhar.
- SCPs não afetam a conta de gerenciamento. Nenhum usuário e nenhuma role de lá é limitado por elas. Esse é o motivo técnico para deixar a conta de gerenciamento vazia: tudo o que roda nela fica fora do alcance das suas proteções.
- SCPs afetam o root das contas membro.
- SCPs não afetam service-linked roles, as roles que os próprios serviços da AWS usam (o ECS, por exemplo, para registrar tasks no target group).

O primeiro passo é ligar o tipo de política:

```
aws organizations enable-policy-type --root-id "$RAIZ" --policy-type
SERVICE_CONTROL_POLICY
```

A primeira SCP protege a auditoria e prende as regiões. Ela fica em `organizacao/scp-base.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ProtegerAuditoria",
      "Effect": "Deny",
      "Action": [
        "cloudtrail:StopLogging",
        "cloudtrail:DeleteTrail",
        "cloudtrail:UpdateTrail",
        "guardduty:DeleteDetector",
        "guardduty:DisassociateFromAdministratorAccount",
        "organizations:LeaveOrganization"
      ],
      "Resource": "*"
    },
    {
      "Sid": "SoRegioesDoRango",
      "Effect": "Deny",
      "NotAction": [
        "iam:*",
        "organizations:*",
        "sts:*",
        "route53:*",
        "cloudfront:*",
```

```
        "support:*",
        "budgets:*"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["us-east-1", "sa-east-1"]
        }
      }
    }
  ]
}
```

O segundo bloco lembra a `SoRegioesDoRango` do capítulo 8, agora valendo para a conta inteira e para todos os serviços. O `NotAction` deixa de fora os serviços globais, para que uma chamada a eles nunca seja confundida com uma chamada fora das regiões permitidas. Um invasor com credencial roubada não consegue subir máquinas de mineração em `ap-southeast-1`, que é onde ninguém olharia. Minerar criptomoeda na conta dos outros é o único modelo de negócio cripto com lucro garantido, e por isso tanta gente tenta.

A segunda SCP vale só para produção e responde diretamente à fila apagada. O arquivo é `organizacao/scp-producao.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SoOCloudFormationApagaDados",
      "Effect": "Deny",
      "Action": [
        "sqs:DeleteQueue",
        "dynamodb:DeleteTable",
        "rds:DeleteDBInstance",
        "rds:DeleteDBSnapshot",
        "s3:DeleteBucket",
        "backup:DeleteRecoveryPoint"
      ],
      "Resource": "*",
      "Condition": {
        "ArnNotLike": {
          "aws:PrincipalArn": "arn:aws:iam::*:role/cdk-hnb659fds-cfn-exec-role-*"
        }
      }
    }
  ]
}
```

Em produção, só a role de execução do CloudFormation (a do bootstrap do CDK) apaga filas, tabelas, bancos e buckets. Nem o administrador pelo console. O administrador pode se sentir ofendido; a fila de produção, por outro lado, se sente ótima. Se um recurso precisa sumir, ele sai do código e o pipeline aplica, deixando rastro no git e no CloudTrail.

A documentação recomenda testar SCPs numa OU antes de anexá-las a contas importantes, e nunca começar pela raiz. Por isso a ordem abaixo começa por homologação, que fica numa OU de teste por alguns dias:

```
SCP_BASE=$(aws organizations create-policy --type SERVICE_CONTROL_POLICY \
  --name rango-base --description "Auditoria protegida e regioes do Rango" \
  --content file://organizacao/scp-base.json --query 'Policy.PolicySummary.Id' --output
text)
SCP_PROD=$(aws organizations create-policy --type SERVICE_CONTROL_POLICY \
  --name rango-producao --description "Apagar dados so pelo CloudFormation" \
  --content file://organizacao/scp-producao.json --query 'Policy.PolicySummary.Id' --
output text)

aws organizations attach-policy --policy-id "$SCP_BASE" --target-id "$OU_CARGAS"
aws organizations attach-policy --policy-id "$SCP_PROD" --target-id 456789012345
```

Agora, o equivalente ao script do Diego, com um profile de administrador da conta de produção:

```
aws sqs delete-queue --profile rango-producao-admin \
  --queue-url https://sqs.us-east-1.amazonaws.com/456789012345/rango-producao-sms
```

```
An error occurred (AccessDenied) when calling the DeleteQueue operation: User:
arn:aws:sts::456789012345:assumed-
role/AWSReservedSSO_AdministratorAccess_9b8c7d6e5f4a3b21/voce
is not authorized to perform: sqs:DeleteQueue on resource:
arn:aws:sqs:us-east-1:456789012345:rango-producao-sms with an explicit deny in a service
control policy
```

O final da mensagem diz de onde veio a negação: a resposta está no Organizations, longe das policies da conta. É o tipo de mensagem de erro que dá vontade de emoldurar.

> **Armadilha:** Se você desabilitar o tipo de política SCP na raiz, todas as SCPs são desanexadas. Ao reabilitar, só a `FullAWSAccess` volta; os anexos se perdem e precisam ser refeitos à mão. Mantenha os comandos de `attach-policy` num script versionado, junto com os JSON.

Os logs num lugar que ninguém apaga

Uma trail que grava no bucket da própria conta tem um ponto fraco: quem é administrador ali apaga o bucket e a prova junto. É como pedir ao suspeito que guarde as provas na gaveta dele, por gentileza. Os logs vão para a conta `rango-seguranca`, e a trail é uma **organization trail**: criada na conta de gerenciamento, registra os eventos de todas as contas, em todas as regiões. O bucket nasce numa stack nova, `RangoSeguranca`, em `rango_infra/seguranca_stack.py`:

```python
from aws_cdk import Duration, RemovalPolicy, Stack
from aws_cdk import aws_events as events
from aws_cdk import aws_events_targets as alvos
from aws_cdk import aws_iam as iam
from aws_cdk import aws_s3 as s3
from aws_cdk import aws_sns as sns
from aws_cdk import aws_sns_subscriptions as assinaturas
from constructs import Construct


class SegurancaStack(Stack):
    def __init__(
        self,
        scope: Construct,
        construct_id: str,
        *,
        conta_gerenciamento: str,
        id_organizacao: str,
        email_alertas: str,
        **kwargs,
    ) -> None:
        super().__init__(scope, construct_id, **kwargs)

        trilha_arn = f"arn:aws:cloudtrail:us-east-1:{conta_gerenciamento}:trail/rango-
organizacao"

        bucket = s3.Bucket(
            self,
            "LogsTrilha",
            bucket_name=f"rango-trilha-{self.account}",
            block_public_access=s3.BlockPublicAccess.BLOCK_ALL,
            enforce_ssl=True,
            versioned=True,
            removal_policy=RemovalPolicy.RETAIN,
            lifecycle_rules=[
                s3.LifecycleRule(
                    transitions=[
                        s3.Transition(
                            storage_class=s3.StorageClass.GLACIER_INSTANT_RETRIEVAL,
                            transition_after=Duration.days(90),
                        )
                    ],
                    expiration=Duration.days(730),
                    noncurrent_version_expiration=Duration.days(30),
                )
            ],
        )
        cloudtrail = iam.ServicePrincipal("cloudtrail.amazonaws.com")
        bucket.add_to_resource_policy(
            iam.PolicyStatement(
                actions=["s3:GetBucketAcl"],
                principals=[cloudtrail],
                resources=[bucket.bucket_arn],
                conditions={"StringEquals": {"aws:SourceArn": trilha_arn}},
            )
        )
        bucket.add_to_resource_policy(
            iam.PolicyStatement(
                actions=["s3:PutObject"],
                principals=[cloudtrail],
                resources=[
```

```
                    bucket.arn_for_objects(f"AWSLogs/{conta_gerenciamento}/*"),
                    bucket.arn_for_objects(f"AWSLogs/{id_organizacao}/*"),
                ],
                conditions={
                    "StringEquals": {
                        "aws:SourceArn": trilha_arn,
                        "s3:x-amz-acl": "bucket-owner-full-control",
                    }
                },
            )
        )
```

A condição `aws:SourceArn` impede que outra trail use o seu bucket. O lifecycle segue o capítulo 11: Glacier Instant Retrieval depois de 90 dias, exclusão com dois anos; com exigência legal de retenção, ajuste o prazo e considere o S3 Object Lock. No `app.py`, a stack recebe o `env` da conta de segurança, o ID da organização (`aws organizations describe-organization --query Organization.Id`) e o `email_alertas` do contexto. Com o bucket pronto, a trail nasce na conta de gerenciamento:

```
aws organizations enable-aws-service-access --service-principal cloudtrail.amazonaws.com

aws cloudtrail create-trail \
  --name rango-organizacao \
  --s3-bucket-name rango-trilha-210987654321 \
  --is-multi-region-trail \
  --is-organization-trail \
  --enable-log-file-validation \
  --region us-east-1

aws cloudtrail start-logging --name rango-organizacao --region us-east-1
```

O `--enable-log-file-validation` faz o CloudTrail gravar, a cada hora, um arquivo de resumo (*digest*) assinado com os hashes dos arquivos de log. Se alguém alterar ou apagar um arquivo, `aws cloudtrail validate-logs --trail-arn <arn> --start-time 2026-11-16T00:00:00Z` acusa. É o lacre do pote de geleia: se alguém abriu, dá para ver. Em alguns minutos, os primeiros arquivos `.json.gz` aparecem sob `AWSLogs/<id-da-organização>/<conta>/CloudTrail/<região>/`.

> **Na vida real:** O AWS Control Tower monta esta mesma estrutura (OUs, conta de logs, organization trail, guardrails) com alguns cliques, e compensa para quem vai ter dezenas de contas. Fazer à mão uma vez ensina o que ele cria por você.

GuardDuty: quem olha os logs às 3h

O CloudTrail grava tudo, mas ninguém lê milhões de eventos, do mesmo jeito que ninguém lê os termos de uso. O **Amazon GuardDuty** lê por você. É o colega que lê o contrato inteiro antes de assinar e não reclama de fazer isso às 3h da manhã. Ele analisa

continuamente os eventos de gerenciamento do CloudTrail, os VPC Flow Logs e as consultas DNS das suas VPCs, sem que você precise ligar essas fontes nem pagar por elas separadamente, e compara o que vê com listas de ameaças conhecidas e com o comportamento normal da conta.

O resultado são **findings**, com tipo e severidade. Alguns que fazem sentido para o Rangô:

- `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`: credenciais temporárias de uma instância ou task sendo usadas a partir de um IP fora da AWS. É o padrão do caso Capital One do capítulo 9.
- `Recon:IAMUser/MaliciousIPCaller`: chamadas de API vindas de um IP numa lista de ameaças.
- `CryptoCurrency:EC2/BitcoinTool.B!DNS`: uma máquina consultando domínios de pools de mineração.
- `Policy:IAMUser/RootCredentialUsage`: alguém usando o root.

Os *protection plans* opcionais (eventos de dados do S3, logins no RDS, Runtime Monitoring em containers, varredura de malware) são cobrados à parte. O primeiro uso tem 30 dias grátis por conta e por região, e cada plano novo ganha os seus 30 dias, exceto o Malware Protection for S3, que tem franquia gratuita. O console mostra os dias restantes e uma estimativa de custo diário com base no uso real; use-a no fim do teste para decidir o que manter. Teste grátis de 30 dias, como sabe qualquer pessoa que já "experimentou" um aplicativo de idiomas, é um compromisso firmado com o seu eu do futuro, e o seu eu do futuro esquece.

Ligando para a organização inteira

Na conta de gerenciamento, a administração do GuardDuty é delegada à conta de segurança:

```
aws guardduty enable-organization-admin-account \
  --admin-account-id 210987654321 --region us-east-1
```

Na conta de segurança, o GuardDuty passa a ser ligado automaticamente em todas as contas, atuais e futuras:

```
export AWS_PROFILE=rango-seguranca-admin
DETECTOR=$(aws guardduty list-detectors --region us-east-1 --query 'DetectorIds[0]' --
output text)
aws guardduty update-organization-configuration \
  --detector-id "$DETECTOR" --auto-enable-organization-members ALL --region us-east-1
```

O mesmo se repete em `sa-east-1`: o GuardDuty é regional, e as outras regiões estão bloqueadas pela SCP. Os findings de todas as contas aparecem na conta de segurança. Para não depender de alguém abrir o console, a `SegurancaStack` ganha uma regra do EventBridge:

```
        topico = sns.Topic(self, "TopicoSeguranca", topic_name="rango-seguranca-
alertas")
        topico.add_subscription(assinaturas.EmailSubscription(email_alertas))

        events.Rule(
            self,
            "AchadosGraves",
            event_pattern=events.EventPattern(
                source=["aws.guardduty"],
                detail_type=["GuardDuty Finding"],
                detail={"severity": [{"numeric": [">=", 7]}]},
            ),
            targets=[alvos.SnsTopic(topico)],
        )
```

Severidade 7 ou mais é *high* ou crítica; o resto fica para a revisão semanal, porque mandar tudo por e-mail ensina a equipe a ignorar o GuardDuty com a mesma eficiência com que todo mundo ignora o "responder a todos" da empresa. Para testar, um finding de exemplo resolve:

```
aws guardduty create-sample-findings --detector-id "$DETECTOR" \
  --finding-types "UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS"
\
  --region us-east-1
```

Em alguns minutos, chega no e-mail um JSON com `"severity": 8` e o título do finding, marcado como amostra. É o único e-mail sobre credencial vazada que dá alívio receber. Depois, os exemplos são arquivados no console.

Config e Security Hub: o que está configurado errado agora

GuardDuty olha comportamento. Falta olhar configuração: um Security Group que alguém abriu para `0.0.0.0/0` na porta 5432, um bucket novo sem Block Public Access, um banco com acesso público.

O **AWS Config** grava o histórico de configuração de cada recurso: cada mudança vira um *configuration item*, ligado ao evento do CloudTrail que a causou, e você pergunta como estava o `SgApi` na terça às 15h. É a máquina do tempo que todo mundo quis ter na

hora de responder "quem mexeu nisso?". Por cima rodam as **Config rules**, como `restricted-ssh` e `rds-instance-public-access-check`, que marcam recursos como conformes ou não. Ele cobra por item gravado e por avaliação de regra.

O **AWS Security Hub** junta tudo num painel. A versão nova, em disponibilidade geral desde 2 de dezembro de 2025, correlaciona findings do GuardDuty, do Amazon Inspector (vulnerabilidades em imagens do ECR e em Lambdas) e das checagens de configuração, e prioriza o que é risco real. O serviço anterior, com padrões como o AWS Foundational Security Best Practices, passou a se chamar **Security Hub CSPM** e faz parte do novo. A AWS troca nome de serviço com o entusiasmo de banda de garagem que muda de nome a cada show.

O plano Essentials cobra por *resource unit* por mês: uma instância EC2, ou 18 imagens no ECR, ou 12 Lambdas, ou 125 usuários e roles do IAM; o resto entra sem custo. Inclui o CSPM, o Inspector e um gravador do Config gerenciado pelo serviço. O Rangô tem zero EC2, umas 30 imagens no ECR, 10 Lambdas e menos de 125 roles: umas 4 unidades. O exemplo oficial usa US$ 3,75 por unidade, valor que a AWS pede para confirmar na calculadora. São 30 dias de teste.

A ativação é pelo console da conta de segurança, como administrador delegado. O primeiro relatório assusta como um `npm audit` num projeto de 2019, com dezenas de itens de severidade baixa; no Rangô, vieram primeiro o bucket `rango-trilha` sem Object Lock e o ALB sem logs de acesso. Corrija críticos e altos, e suprima com justificativa escrita o que não se aplica.

AWS Backup: o backup que o administrador não apaga

O capítulo 12 deixou uma fraqueza à vista: os backups automáticos do RDS somem quando a instância é apagada, e um snapshot manual pode ser apagado por qualquer pessoa com permissão na conta. As tabelas do DynamoDB têm o mesmo problema. Se uma credencial de administrador de produção vazar, quem a tiver apaga o banco e os backups juntos. Backup que o atacante consegue apagar é só uma cópia otimista.

O **AWS Backup** centraliza backups de vários serviços (RDS, DynamoDB, EBS, EFS, S3 e outros) em **planos**: regras com horário, retenção e cópias. Os pontos de restauração ficam em **cofres** (*backup vaults*), e um cofre pode estar em outra conta da organização. Com o **Vault Lock** em modo *compliance*, depois de um período de carência de alguns dias, ninguém consegue apagar os pontos de restauração antes do prazo: nem você, nem o root da conta. O root, acostumado a mandar em tudo, descobre como é a vida do resto da equipe.

A `RangoProducao` passa a importar `aws_backup as backup`:

```
        cofre = backup.BackupVault(
            self, "CofreLocal", backup_vault_name=f"rango-{ambiente}"
        )
        cofre_seguranca = backup.BackupVault.from_backup_vault_arn(
            self,
            "CofreSeguranca",
            self.node.try_get_context("cofre_seguranca_arn"),
        )
        plano = backup.BackupPlan(self, "PlanoBackup", backup_plan_name=f"rango-
{ambiente}")
        plano.add_rule(
            backup.BackupPlanRule(
                rule_name="diario",
                backup_vault=cofre,
                schedule_expression=events.Schedule.cron(hour="6", minute="0"),
                delete_after=Duration.days(35),
                copy_actions=[
                    backup.BackupPlanCopyActionProps(
                        destination_backup_vault=cofre_seguranca,
                        delete_after=Duration.days(90),
                    )
                ],
            )
        )
        plano.add_selection(
            "DadosDoRango",
            resources=[
                backup.BackupResource.from_rds_database_instance(db),
                backup.BackupResource.from_dynamo_db_table(tabela_entregas),
                backup.BackupResource.from_dynamo_db_table(tabela_fluxo),
            ],
        )
```

Todo dia às 6h UTC (3h em Brasília, horário em que só backups e robôs de senha estão acordados), o plano copia banco e tabelas para o cofre local, guarda por 35 dias e envia cada ponto de restauração para o cofre da conta de segurança (ARN em `cofre_seguranca_arn` no `cdk.json`), onde fica por 90.

Fora deste código ficam: habilitar a cópia entre contas nas configurações do AWS Backup da conta de gerenciamento, uma política de acesso no cofre de destino aceitando cópias da organização, o Vault Lock aplicado com `aws backup put-backup-vault-lock-configuration` e, para o RDS criptografado com a chave do capítulo 16, a key policy liberando a conta de destino. Antes de confiar no cofre de segurança, o certo é testar uma restauração a partir dele. Backup nunca restaurado é como paraquedas nunca aberto: a fé é enorme, a evidência é nenhuma. O preço depende do serviço e do volume; a conta certa sai da página oficial, com os tamanhos reais.

Quando acontecer de verdade

A segunda-feira do Rangô terminou bem: um engano honesto e um robô sem sorte. Um incidente de verdade começa com um e-mail do GuardDuty às 2h. O roteiro do capítulo 6 vale, com passos próprios de segurança:

1. **Confirmar** o que está acontecendo com o CloudTrail: qual credencial, de qual IP, desde quando, o que ela fez.
2. **Conter** sem destruir. Desativar a access key suspeita (`aws iam update-access-key --status Inactive`). Para uma role, usar "Revoke active sessions" no console do IAM, que acrescenta uma policy negando tudo para sessões emitidas antes daquele instante. No Identity Center, desativar o usuário impede novos logins, mas as sessões já emitidas valem até expirar; se não puder esperar, uma SCP de emergência negando tudo na conta afetada corta todo mundo na hora, inclusive o invasor. Para tráfego, um IP set no WAF bloqueia faixas em segundos.
3. **Preservar evidências.** Antes de apagar uma instância comprometida, tirar um snapshot do disco. Os logs já estão a salvo na conta de segurança.
4. **Erradicar e recuperar**: rotacionar segredos que a credencial podia ler (a rotação do capítulo 16 ajuda), recriar recursos a partir do código, restaurar dados do cofre.
5. **Escrever o postmortem** sem culpados. Sem culpados de verdade, e não daquele jeito em que a sala inteira olha para o estagiário.

A AWS avisa sobre abuso na conta pelo contato de segurança dos *alternate contacts* do capítulo 7. É bom conferir se ele chega a alguém que ainda trabalha no Rangô. Alerta de segurança indo para o e-mail de quem saiu da empresa é o equivalente digital de chamar os bombeiros para o endereço antigo.

> **Dica:** Guarde esse roteiro no repositório, com os comandos prontos e os números das contas.

Armadilhas e casos reais

> **Armadilha:** Trail de uma região só. Uma trail criada pelo console já nasce multirregião, mas pela CLI o padrão é uma região só, sem o `--is-multi-region-trail`. O invasor que cria recursos em outra região fica de fora dos seus arquivos.
>
> **Armadilha:** Web ACL do CloudFront criada fora de `us-east-1`. O CloudFront só aceita web ACLs de escopo `CLOUDFRONT`, e elas só existem em `us-east-1`. A mesma regra do certificado do ACM, do capítulo 14.

> **Armadilha:** Rule group gerenciado esquecido em `Count`. Ligar em modo de contagem é o certo; esquecer lá por meses dá a sensação de proteção sem proteção nenhuma. Ponha uma data no calendário para revisar as métricas e trocar para bloqueio.
>
> **Armadilha:** Limite de taxa e CGNAT. O capítulo 1 mostrou que muitos clientes de operadoras brasileiras compartilham o mesmo IP público. Um limite por IP muito baixo numa rota usada por todos os clientes, como `POST /pedidos`, bloqueia gente honesta. No login do painel, com poucos usuários, o risco é pequeno.
>
> **Armadilha:** O `AWSManagedRulesCommonRuleSet` bloqueia corpos grandes com a regra `SizeRestrictions_BODY`. O Rangô escapa porque as fotos vão direto para o S3, mas uma API que recebe uploads começa a devolver 403 sem motivo aparente.
>
> **Armadilha:** Workload na conta de gerenciamento. Nenhuma SCP vale lá. Se o Rangô continuasse em `rango-estudos`, todas as políticas desta seção seriam decorativas para ele.
>
> **Armadilha:** Custo do Config. Recursos que mudam o tempo todo, como tasks do ECS substituídas a cada deploy, geram muitos itens gravados. Confira a fatura da primeira semana.
>
> **Na vida real:** Os vazamentos de chave do capítulo 8 seguem o mesmo roteiro: em minutos, alguém chama `RunInstances` em várias regiões para minerar criptomoeda. A SCP de regiões barra as máquinas, o GuardDuty avisa, e a trail guarda a prova onde o invasor não alcança.

Resumo

- O CloudTrail registra cada chamada de API: gerenciamento grátis por 90 dias no Event History, por região. Uma organization trail multirregião guarda tudo num bucket de outra conta.
- O WAF filtra requisições no ALB e no CloudFront; uma regra de taxa com `scope_down_statement` protege o login. Rule groups gerenciados começam em `Count`. Shield Standard vem incluso.
- O GuardDuty lê CloudTrail, VPC Flow Logs e DNS sem configuração; o EventBridge manda os findings graves por e-mail.
- Config grava o histórico de configuração; o Security Hub (novo desde dezembro de 2025, com o antigo renomeado para Security Hub CSPM) reúne e prioriza findings.
- Contas separadas são a fronteira mais simples entre ambientes. A conta de gerenciamento fica vazia, porque SCPs não valem nela.

- SCPs definem o teto de permissões das contas membro, nunca concedem nada, valem até para o root e não alcançam service-linked roles. Teste numa OU antes de anexar em produção.
- AWS Backup com cópia para um cofre em outra conta, com Vault Lock, protege os dados até de um administrador comprometido.

A fatura de dezembro

O Rangô está protegido como nunca esteve: três contas novas, trail da organização, GuardDuty, Security Hub, WAF e backups em outra conta. Cada peça custa pouco. A soma é outra conversa.

No dia 4 de janeiro, chega a fatura de dezembro, o primeiro mês inteiro com as contas novas, maior que o faturamento do Rangô no mês. O Bruno manda uma foto do e-mail com uma pergunta só: "isso é normal?". São três palavras, e é a pergunta mais cara do ano. Você não sabe dizer qual linha é de qual coisa, nem quanto custa um pedido. Os endpoints de interface somam mais que a API inteira, os testes grátis do GuardDuty e do Security Hub acabaram no meio do mês, e há um volume EBS de 100 GiB que ninguém lembra de ter criado. Ele está lá, quieto, como a caixa da mudança de três anos atrás que ninguém abriu.

No último capítulo: AWS Cost Explorer, budgets, tags de custo, os vilões clássicos (NAT Gateway, tráfego de saída, recursos esquecidos), Savings Plans e Spot, o Well-Architected Framework, uma revisão final do Rangô e os próximos passos para você.

Referências

- AWS CloudTrail, preços (Event History, primeira cópia grátis, CloudTrail Lake): https://aws.amazon.com/cloudtrail/pricing/
- AWS CloudTrail, User Guide: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html
- CloudTrail, trails para organizações: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/creating-trail-organization.html
- CloudTrail, validação de integridade dos arquivos de log: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html
- AWS WAF, preços: https://aws.amazon.com/waf/pricing

- AWS WAF, regras baseadas em taxa: https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-statement-type-rate-based.html
- AWS WAF, managed rule groups da AWS: https://docs.aws.amazon.com/waf/latest/developerguide/aws-managed-rule-groups-list.html
- AWS Shield, preços: https://aws.amazon.com/shield/pricing
- Amazon GuardDuty, preços e período de teste: https://docs.aws.amazon.com/guardduty/latest/ug/guardduty-pricing.html
- Amazon GuardDuty, página de preços dos protection plans: https://aws.amazon.com/guardduty/pricing
- Amazon GuardDuty, tipos de finding: https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_finding-types-active.html
- AWS Config, preços: https://aws.amazon.com/config/pricing
- AWS Security Hub, anúncio de disponibilidade geral (dezembro de 2025): https://aws.amazon.com/about-aws/whats-new/2025/12/security-hub-near-real-time-risk-analytics/
- AWS Security Hub, perguntas frequentes (Security Hub CSPM): https://aws.amazon.com/security-hub/faqs/
- AWS Security Hub, preços: https://aws.amazon.com/security-hub/pricing/
- AWS Organizations, service control policies: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html
- IAM, gerenciamento centralizado do root das contas membro: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_root-user.html
- AWS Backup, Developer Guide: https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html
- AWS Security Incident Response Guide: https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response-guide/welcome.html

CAPÍTULO 24

Custos e arquitetura

No capítulo anterior, o Rangô ganhou um registro de cada chamada de API no CloudTrail, um WAF na frente do login, o GuardDuty lendo os logs da organização e backups copiados para uma conta que ninguém da produção alcança. Ele também mudou de endereço: agora vive na conta `rango-producao`, ao lado de `rango-homologacao` e `rango-seguranca`, todas debaixo de SCPs, enquanto a `rango-estudos` cuida só do faturamento e do Identity Center. O capítulo terminou com uma fatura chegando, como todo final de temporada que se preze: com um gancho que dá medo.

Segunda-feira, 4 de janeiro

A mensagem do Bruno chega às 9h12. É uma foto da tela do notebook, com o e-mail da AWS aberto: "Your AWS bill for December 2026 is available. Total: USD 687.18". É o tipo de e-mail que faz a pessoa ler o número três vezes, procurando uma vírgula fora do lugar. Embaixo, uma pergunta só: "isso é normal?".

Você faz a conta que ele já fez. O Rangô cobra uma taxa de serviço de R$ 0,50 por pedido, e dezembro fechou com 6.812 pedidos: R$ 3.406. A fatura, convertida no cartão com câmbio e IOF, passou de R$ 3.700. O IOF, como sempre, chegou sem ser convidado. Cada pedido custou mais em infraestrutura do que rendeu. É o modelo de negócio do pasteleiro que perde dinheiro em cada pastel e pretende compensar no volume.

Você entra no console da `rango-estudos`, a conta de gerenciamento da organização. Desde o capítulo 8, quando o AWS Organizations foi ativado, ela paga a conta de todas as outras: isso se chama *consolidated billing*, uma fatura só para a organização inteira. A página de faturas mostra o total por serviço:

Serviço	Dezembro (US$)
Amazon Virtual Private Cloud	227,42
Amazon Simple Storage Service	124,70
Amazon Elastic Container Service	58,00
Elastic Load Balancing	54,00

Serviço	Dezembro (US$)
EC2 - Other	48,98
AWS WAF	36,00
Amazon DynamoDB	34,10
Amazon Relational Database Service	29,40
Amazon CloudWatch	18,20
AWS Config	16,30
Amazon GuardDuty	13,70
AWS Security Hub	8,50
Outros	17,88
Total	**687,18**

Os valores desta história são hipotéticos, montados com os preços de referência que o livro usou (us-east-1, outubro de 2026). As perguntas que eles provocam são reais.

A linha mais cara é a VPC, que o capítulo 10 disse ser gratuita. Antes que você peça o dinheiro do livro de volta: a explicação vem logo adiante, e o capítulo 10 sai inocente. O S3 guarda menos de 5 GB de fotos e custou mais que a API. Há um "EC2 - Other" numa organização sem instâncias desde o capítulo 17. E nada ali diz quanto é produção, quanto é homologação e quanto custa um pedido.

Você responde que não, e pede até o dia seguinte. Prometer explicar uma fatura da AWS em um dia é o tipo de otimismo que só a cafeína explica.

Como a AWS cobra

Antes de abrir qualquer ferramenta, ajuda saber o que a AWS mede. Quase tudo na fatura cai em um de três medidores.

O primeiro é **tempo ligado**: horas de NAT Gateway, endpoint, load balancer, banco, task do Fargate, IPv4 público. Ele gira às 4h da manhã, sem nenhum pedido. Funciona como aluguel: corre igual, esteja você em casa ou viajando. O segundo é **volume guardado**: GB-mês de EBS, S3, snapshot, log, que nunca diminui sozinho. O terceiro é **movimento**: GB que saem para a internet, requisições, invocações, unidades do DynamoDB.

Para um projeto do tamanho do Rangô, o perigo mora no primeiro. Somando as linhas que cobram por hora, o Rangô gastou perto de US$ 0,60 por hora só por existir, nas duas contas. Os 6.812 pedidos mexeram pouco no total. O Rangô gastava quase o mesmo dormindo e trabalhando, o que descreve com precisão muita gente às sextas-feiras.

Cost Explorer: a fatura aberta em fatias

O **AWS Cost Explorer** é a ferramenta da AWS para olhar o gasto por dimensão: serviço, conta, região, tipo de uso, tag. O console dele não tem custo adicional; cada chamada pela API é cobrada. Sim, perguntar pela API quanto você gastou também entra na conta, como um taxímetro que cobra para mostrar o valor da corrida. Os dados são atualizados pelo menos uma vez a cada 24 horas, então o gasto de hoje só aparece amanhã, como notícia de jornal impresso. Ele também projeta o futuro com base no histórico: até 18 meses na visão mensal e 3 meses na diária.

Na primeira vez, é preciso abrir o Cost Explorer no console da conta de gerenciamento para ativá-lo. Ela enxerga o gasto de todas as contas membro, como quem recebe a fatura do cartão principal com os adicionais da família inteira.

Por conta e por serviço

Com o profile de administrador da `rango-estudos`, a primeira pergunta é de onde vem o dinheiro:

```
aws sso login --sso-session rango

aws ce get-cost-and-usage \
  --profile rango-admin \
  --time-period Start=2026-12-01,End=2027-01-01 \
  --granularity MONTHLY \
  --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=LINKED_ACCOUNT Type=DIMENSION,Key=SERVICE \
  --query 'ResultsByTime[0].Groups[].[Keys[0],Keys[1],Metrics.UnblendedCost.Amount]' \
  --output text | sort -t$'\t' -k3 -rn | head -12
```

A data de fim é exclusiva, e o Cost Explorer aceita até dois agrupamentos por consulta. O `sort` separa por tabulação porque os nomes de serviço têm espaços. Nada como um serviço chamado *Simple* com quatro palavras no nome.

```
456789012345    Amazon Simple Storage Service   124.6987
345678901234    Amazon Virtual Private Cloud    115.3196
456789012345    Amazon Virtual Private Cloud    112.1034
345678901234    EC2 - Other 48.9821
456789012345    Amazon DynamoDB 34.0977
456789012345    Amazon Elastic Container Service    29.0113
345678901234    Amazon Elastic Container Service    28.9890
456789012345    Elastic Load Balancing  27.1203
```

```
345678901234    Elastic Load Balancing  26.8811
456789012345    AWS WAF 18.0410
345678901234    AWS WAF 17.9604
456789012345    Amazon Relational Database Service  15.1031
```

A conta `345678901234` é a `rango-homologacao`, que só a equipe usa, e ela custa quase o mesmo que a produção em VPC, load balancer, Fargate e WAF. No total, homologação ficou com US$ 231,28 e produção com US$ 434,70; o resto é de `rango-seguranca` e `rango-estudos`. As contas do capítulo 23, criadas por segurança, viraram de graça a primeira dimensão de custo do Rangô. Homologação é aquele parente que mora de favor e gasta quase tanto quanto o dono da casa.

Descendo ao tipo de uso

"Amazon Virtual Private Cloud" ainda diz pouco, como aquela linha do extrato que diz só "COMPRAS DIVERSAS". O tipo de uso (*usage type*) é o nome do medidor que gerou o valor:

```
aws ce get-cost-and-usage \
  --profile rango-admin \
  --time-period Start=2026-12-01,End=2027-01-01 \
  --granularity MONTHLY \
  --metrics UnblendedCost UsageQuantity \
  --filter '{"Dimensions": {"Key": "SERVICE", "Values": ["Amazon Virtual Private
Cloud"]}}' \
  --group-by Type=DIMENSION,Key=USAGE_TYPE \
  --query 'ResultsByTime[0].Groups[].
[Keys[0],Metrics.UsageQuantity.Amount,Metrics.UnblendedCost.Amount]' \
  --output text
```

```
USE1-PublicIPv4:InUseAddress    3720    18.6000
USE1-VpcEndpoint-Bytes  52.3    0.5230
USE1-VpcEndpoint-Hours  20832   208.3200
```

Os nomes trazem prefixo de região e podem variar; o que importa são as quantidades. São 20.832 horas de endpoint num mês de 744: os 7 interface endpoints do Rangô (Ssm, SecretsManager, EcrApi, EcrDocker, Logs, StepFunctions e XRay) em 2 AZs e 2 contas, a cerca de US$ 0,01 por hora cada, o preço do capítulo 10. E 3.720 horas de IPv4 são 5 endereços: os nós dos dois ALBs e um quinto inesperado. A VPC continua gratuita; o que se paga são as peças dentro dela. É o restaurante que não cobra a mesa, só tudo o que vai em cima dela.

A mesma consulta, filtrando "EC2 - Other", explica a linha fantasma. Esse grupo junta tudo o que o Cost Explorer associa ao EC2 sem ser instância: volumes EBS, snapshots, NAT Gateway, transferência entre AZs. Em homologação, ele mostra `NatGateway-Hours`

com 744 horas, `NatGateway-Bytes` quase zerado e `EBS:VolumeUsage.gp3` com 100 GB-mês. Um NAT Gateway e um volume de 100 GiB numa conta que, pelo código, não tem nenhum dos dois. Na AWS, assombração aparece na fatura, e não no espelho.

Tags de custo: quem gastou o quê

Contas separam ambientes. Dentro da produção, a pergunta seguinte é por componente: quanto custam as fotos, o rastreamento de entregas, o fluxo do pedido, a observabilidade. A resposta vem de *tags*, os pares chave-valor que quase todo recurso da AWS aceita.

Uma tag no recurso só aparece na fatura depois de ativada como **cost allocation tag** na conta de gerenciamento, e vale dali em diante. Até lá, ela é uma etiqueta escrita com tinta invisível.

Tags no CDK

No CDK, `Tags.of(escopo).add(chave, valor)` aplica a tag em tudo abaixo daquele escopo, como um middleware do FastAPI que alcança todas as rotas. Em `app.py`, depois da criação das stacks:

```
cdk.Tags.of(app).add("projeto", "rango")
```

No construtor da `RangoInfraStack`, no começo, a tag do ambiente e uma variável que vai aparecer de novo neste capítulo:

```
        producao = ambiente == "producao"
        Tags.of(self).add("ambiente", ambiente)
```

E no fim do construtor, depois que todos os recursos existem, a tag de componente nos constructs principais, buscados pelo ID:

```
        componentes = {
            "api": ["ServicoApi", "AlbApi"],
            "entregas": ["TabelaEntregas"],
            "fluxo": ["TabelaFluxo"],
        }
        for componente, ids in componentes.items():
            for id_construct in ids:
                Tags.of(self.node.find_child(id_construct)).add("componente",
componente)
```

O `Tags` vem do `aws_cdk`, como o `RemovalPolicy`. O `find_child` falha no `cdk synth` se o ID não existir, o que pega nome digitado errado. Erro de digitação descoberto no synth é o melhor tipo de erro que existe: barato e sem plateia. Depois do deploy, as chaves levam até 24 horas para aparecer como ativáveis. Aí, na conta de gerenciamento:

```
aws ce update-cost-allocation-tags-status \
  --profile rango-admin \
  --cost-allocation-tags-status \
    TagKey=projeto,Status=Active \
    TagKey=ambiente,Status=Active \
    TagKey=componente,Status=Active
```

A partir do dia seguinte, `--group-by Type=TAG,Key=componente` funciona no Cost Explorer. Os grupos vêm como `componente$api`, `componente$entregas` e um `componente$` vazio com o que não tem a tag. Esse grupo nunca zera, porque transferência, IPv4 e algumas taxas não carregam tag; use-o para medir quanto do gasto está sem dono. O `componente$` vazio é o achados e perdidos do escritório: guarda-chuva, carregador e um casaco que ninguém lembra de ter usado.

> **Dica:** A AWS gera sozinha a tag `aws:cloudformation:stack-name` em tudo o que uma stack cria. Ela também pode ser ativada como cost allocation tag, e dá o custo por stack sem uma linha de código.

E dezembro? A AWS permite pedir um preenchimento retroativo (*backfill*) das tags ativadas, de até 12 meses. Só que ele só preenche os períodos em que o recurso já tinha a tag. Os recursos do Rangô nunca tiveram, então dezembro continua explicado só por conta e por serviço. Tags que entram hoje pagam o relatório de fevereiro. Nem a nuvem consegue etiquetar um passado que nunca teve etiqueta.

Avisos que chegam antes da fatura

O capítulo 7 criou o budget `rango-mensal` de US$ 20 e o alarme `cobranca-acima-de-10-usd`. O alarme disparou em setembro, quando o primeiro banco subiu, e nunca mais voltou para OK. O budget avisa todo mês desde outubro. Uma regra do e-mail move tudo da AWS para uma pasta, que tem 31 mensagens não lidas. A pasta se chama `AWS`, mas poderia muito bem se chamar `Depois`.

Um alerta que dispara sempre equivale a nenhum alerta, como o capítulo 22 disse das métricas. É o vizinho que liga o som alto todo dia: na terceira semana, ninguém mais repara. O limite de US$ 20 servia para uma conta de estudos vazia; para o Rangô em produção, virou barulho.

Um budget por ambiente, no código

Um **budget** do AWS Budgets compara o gasto, real ou previsto, com um valor que você define, e avisa ao cruzar um percentual. Os budgets que só monitoram não custam nada, e os dois primeiros com ações também são gratuitos em cada conta. Um budget criado

numa conta membro vê só o gasto daquela conta, o que encaixa no desenho do Rangô: cada stack cria o seu.

No `cdk.json`, um valor por ambiente, ao lado do `email_alertas` do capítulo 21:

```
    "orcamento_producao": 320,
    "orcamento_homologacao": 110
```

E na stack, com `aws_budgets as budgets` nos imports:

```
        orcamento = self.node.try_get_context(f"orcamento_{ambiente}")
        email = self.node.try_get_context("email_alertas")

        def aviso(tipo: str, percentual: int):
            return budgets.CfnBudget.NotificationWithSubscribersProperty(
                notification=budgets.CfnBudget.NotificationProperty(
                    notification_type=tipo,
                    comparison_operator="GREATER_THAN",
                    threshold=percentual,
                    threshold_type="PERCENTAGE",
                ),
                subscribers=[
                    budgets.CfnBudget.SubscriberProperty(
                        subscription_type="EMAIL", address=email
                    )
                ],
            )

        budgets.CfnBudget(
            self,
            "Orcamento",
            budget=budgets.CfnBudget.BudgetDataProperty(
                budget_name=f"rango-{ambiente}-mensal",
                budget_type="COST",
                time_unit="MONTHLY",
                budget_limit=budgets.CfnBudget.SpendProperty(amount=orcamento,
unit="USD"),
            ),
            notifications_with_subscribers=[
                aviso("FORECASTED", 100),
                aviso("ACTUAL", 80),
            ],
        )
```

O aviso que importa é o `FORECASTED`: ele dispara quando a projeção do mês passa do limite. Com ele em dezembro, o NAT esquecido teria aparecido por volta do dia 10, e não no dia 4 do mês seguinte. O `ACTUAL` a 80% é a segunda chance. O `FORECASTED` é o amigo que avisa que você vai se arrepender; o `ACTUAL` é o que confirma que você se arrependeu.

Na `rango-estudos`, o `rango-mensal` passa para o total da organização, uns US$ 450, e o alarme de US$ 10 e o Zero spend são apagados, porque já cumpriram o papel deles. Que descansem em paz, depois de meses gritando para uma pasta que ninguém abria.

Anomalias

Um budget só vê o total do mês. Uma Lambda em laço que gasta US$ 30 numa tarde mal mexe na projeção. É a torneira pingando da versão serverless: ninguém escuta, e a conta chega igual. Para isso existe o **AWS Cost Anomaly Detection**, que aprende o padrão de gasto de cada serviço e avisa quando um dia foge dele. Ele precisa de pelo menos 10 dias de histórico para começar e detecta a anomalia em até 24 horas depois do uso.

No console de Billing and Cost Management, o caminho é um monitor do tipo "AWS services" e uma assinatura diária para o e-mail de alertas, só com anomalias de impacto acima de US$ 20. Sem o filtro, avisos sobre centavos ensinam a equipe a ignorá-lo.

> **Armadilha:** Budget avisa, mas não bloqueia. Passar do limite não desliga nada, e a AWS continua cobrando. As *budget actions* conseguem aplicar uma policy ou uma SCP e parar instâncias EC2 e RDS ao cruzar um limite, mas desligar produção por causa de um número é uma decisão de negócio. Em homologação, pode valer a pena; em produção, quase nunca.

Os vilões clássicos

Com os números abertos, a investigação vira uma lista de suspeitos. Três deles aparecem em quase toda conta da AWS que cresceu sem ninguém olhando para a fatura. Em série policial, o culpado é o ator famoso que aparece pouco; na AWS, o primeiro suspeito costuma ser o NAT Gateway.

O NAT que ninguém pediu

O NAT Gateway de homologação tem um ID, e o capítulo 23 deixou a ferramenta certa para descobrir quem o criou:

```
export AWS_PROFILE=rango-homologacao

aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=CreateNatGateway \
  --query 'Events[].[EventTime,Username]' --output text
```

```
2026-11-30T14:22:41-03:00   diego@rango.com.br
```

O profile `rango-homologacao` segue o modelo dos profiles do capítulo 23. O Diego, de novo; ele está construindo uma reputação. Ele explica no Slack: o gateway de pagamento abriu um ambiente de testes novo, e a API de homologação precisava alcançá-lo. Ele

criou o NAT no console, acrescentou uma rota nas subnets privadas, testou, e o plano era apagar tudo no dia seguinte. Como sabe qualquer pessoa que já escreveu `# TODO: remover amanhã`, o dia seguinte nunca chega.

O preço é o do capítulo 10: US$ 0,045 por hora e por GB processado, mais o Elastic IP a US$ 0,005 por hora (us-east-1, outubro de 2026). Parado, uns US$ 36 por mês. Com tráfego, cada GB que sai para a internet pelo NAT pagaria processamento e saída, perto de US$ 0,135.

Na limpeza, a ordem importa: a rota manual sai primeiro, o NAT depois, e o Elastic IP por último, senão ele continua cobrando como endereço ocioso. O `cdk drift` de homologação mostra a rota a mais, e o capítulo 15 já avisou que um `cdk deploy` não desfaz drift. No postmortem sem culpa, a causa raiz fica registrada: homologação aceita mudança manual, e nada avisava quando um recurso fora do código continuava ligado.

Os endpoints que nós pedimos

No Rangô, pesa mais que o NAT uma escolha que o livro fez de propósito: subnets sem saída para a internet e um interface endpoint por serviço. As tasks não alcançam nenhum endereço fora da AWS, e um pacote malicioso numa dependência não tem para onde mandar dados. Ele pode até tentar, com o sucesso de quem grita dentro de um cofre. O capítulo 17 já tinha comparado cinco endpoints com dois NATs. Com sete, ficou assim (us-east-1, outubro de 2026, aproximado):

Saída das subnets privadas	Custo fixo por mês
7 interface endpoints em 2 AZs	~US$ 100,80
7 interface endpoints em 1 AZ	~US$ 50,40
1 NAT Gateway com Elastic IP	~US$ 36 + GB
2 NAT Gateways com Elastic IP	~US$ 72 + GB

Os gateway endpoints do S3 e do DynamoDB ficam fora da tabela porque não cobram por hora nem por dados, nos dois desenhos. Eles são o motivo de as fotos e as posições dos entregadores nunca terem passado por um NAT. Gratuito e útil ao mesmo tempo é uma combinação tão rara na fatura quanto trevo de quatro folhas.

A decisão muda por ambiente. Em produção, os endpoints ficam nas duas AZs: se uma cair, a outra continua alcançando o ECR, o Secrets Manager e os logs, e os US$ 50 a mais entram no orçamento como preço do isolamento. Em homologação, uma AZ basta. Logo depois da linha `privadas = ...` do capítulo 16, entra a seleção nova:

```
        subnets_endpoints = ec2.SubnetSelection(
            subnet_type=ec2.SubnetType.PRIVATE_ISOLATED,
```

```
            availability_zones=None if producao else [vpc.availability_zones[0]],
        )
```

E, nos sete `add_interface_endpoint`, `subnets=privadas` dá lugar a `subnets=subnets_endpoints`. Tasks da outra AZ continuam funcionando pelo DNS privado do endpoint, pagando US$ 0,01 por GB entre AZs, centavos em homologação.

Tráfego de saída: as fotos dos pratos

A segunda linha da fatura era o S3, com US$ 124,70. Armazenamento quase não pesa: 5 GB custam menos de 12 centavos. O que pesa é a saída. Filtrando o S3 por tipo de uso, aparecem uns 1.450 GB de `DataTransfer-Out-Bytes` em dezembro. Tirando os 100 GB mensais gratuitos de saída que o capítulo 11 citou, sobram 1.350 GB a US$ 0,09: US$ 121,50.

A origem é o capítulo 11. A rota de fotos devolve URLs pré-assinadas do S3, e o React mostra cada foto como chegou do celular do dono do restaurante: 4 MB. É foto de lasanha com resolução de outdoor, cada camada de massa em detalhes que nem a Nona conhece. Um cardápio com 12 pratos baixa 48 MB, e como cada URL pré-assinada é diferente, nem o cache do navegador ajuda.

A maior economia vem de foto menor, e o navegador consegue reduzir a imagem antes do upload. Em `frontend/src/reduzirFoto.ts`:

```
export async function reduzirFoto(arquivo: File, larguraMaxima = 1280): Promise<Blob> {
  const imagem = await createImageBitmap(arquivo);
  const escala = Math.min(1, larguraMaxima / imagem.width);
  const canvas = document.createElement("canvas");
  canvas.width = Math.round(imagem.width * escala);
  canvas.height = Math.round(imagem.height * escala);
  canvas.getContext("2d")!.drawImage(imagem, 0, 0, canvas.width, canvas.height);
  return new Promise((resolve, reject) =>
    canvas.toBlob(
      (blob) => (blob ? resolve(blob) : reject(new Error("Falha ao converter a foto"))),
      "image/jpeg",
      0.82,
    ),
  );
}
```

No `EnviarFoto.tsx`, o `PUT` passa a enviar `await reduzirFoto(arquivo)`, e o pedido da URL de upload declara `image/jpeg`: o capítulo 11 mostrou que o `Content-Type` entra na assinatura, e um tipo diferente termina em `SignatureDoesNotMatch`. A foto de 4 MB vira uns 300 KB. O JPEG está ali porque todo navegador sabe gerá-lo; o Safari, por exemplo, ignora o pedido de WebP no `canvas` e devolve PNG, maior que o original. O Safari mantém viva a nobre tradição de ser o navegador que faz o desenvolvedor front-end suspirar.

A segunda parte é servir as fotos pelo CloudFront que já entrega o React. A transferência do S3 para o CloudFront sai de graça, e o plano pay-as-you-go do capítulo 14 inclui 1 TB de saída por mês. A distribuição ganha um comportamento para `/fotos/*`:

```
        fotos = s3.Bucket.from_bucket_name(
            self, "BucketFotos", f"rango-fotos-pratos-{self.account}"
        )
        distribuicao.add_behavior(
            "/fotos/*",
            origins.S3BucketOrigin.with_origin_access_control(fotos),
            viewer_protocol_policy=cloudfront.ViewerProtocolPolicy.REDIRECT_TO_HTTPS,
            cache_policy=cloudfront.CachePolicy.CACHING_OPTIMIZED,
        )
```

Como o bucket foi criado fora da stack, o CDK não consegue editar a policy dele e avisa isso no `cdk synth`. A permissão do OAC entra à mão em `infra/s3/rango-fotos-policy.json`, com o mesmo formato do bucket do front no capítulo 14: `s3:GetObject` em `fotos/*` para o principal `cloudfront.amazonaws.com`, com a condição `AWS:SourceArn` apontando para a distribuição.

A API para de assinar leituras. O dicionário `configuracao` do capítulo 16 ganha `"url_publica": f"https://{dominio}"`, que chega ao container como `URL_PUBLICA`. E a lista de fotos deixa o Valkey, onde estava desde o capítulo 11: o cache vai embora a cada `cdk destroy` e leva o registro junto, enquanto as fotos continuam no bucket. Guardar a fonte da verdade num cache é como anotar um telefone no vapor do espelho do banheiro. A fonte da verdade passa a ser o próprio bucket, e a `role_app` ganha a listagem, restrita ao prefixo das fotos:

```
        role_app.add_to_policy(
            iam.PolicyStatement(
                actions=["s3:ListBucket"],
                resources=[fotos.bucket_arn],
                conditions={"StringLike": {"s3:prefix": ["fotos/*"]}},
            )
        )
```

```
URL_PUBLICA = os.environ["URL_PUBLICA"]


@router.get("/restaurantes/{restaurante}/fotos")
def listar_fotos(restaurante: str = Restaurante):
    paginas = s3.get_paginator("list_objects_v2").paginate(
        Bucket=BUCKET, Prefix=f"fotos/{restaurante}/"
    )
    chaves = [obj["Key"] for pagina in paginas for obj in pagina.get("Contents", [])]
    return [f"{URL_PUBLICA}/{chave}" for chave in sorted(chaves)]
```

O `registrar_foto` continua conferindo o prefixo e o tamanho, mas o `r.sadd` sai.

As chaves já começam com `fotos/`, então a URL fica `https://rango.com.br/fotos/cantina-da-nona/...jpg`. O upload continua pré-assinado; a leitura de foto de prato é pública por natureza, porque está no cardápio. As fotos antigas podem ser reduzidas uma vez com um script em Pillow.

Com fotos de 300 KB, os acessos de dezembro dariam uns 110 GB, dentro do 1 TB gratuito do CloudFront. A lasanha continua bonita na tela, só que com bem menos calorias em bytes.

> **Na vida real:** Em 2024, o engenheiro Maciej Pocwierz criou um bucket S3 vazio e privado para um teste e recebeu uma conta de cerca de US$ 1.300. Uma ferramenta de código aberto usava o mesmo nome de bucket na configuração padrão, e o S3 cobrava até as requisições negadas com 403. Depois da repercussão, a AWS anunciou em maio de 2024 que o dono deixaria de pagar por requisições não autorizadas vindas de fora da conta ou da organização.

Recursos esquecidos

Sobra o volume EBS de 100 GiB. O CloudTrail conta a história de novo: em 26 de novembro, alguém criou o volume a partir de um snapshot do banco para investigar uma consulta lenta, e o volume nunca foi preso a máquina nenhuma. A gp3 cobra US$ 0,08 por GB-mês com ou sem uso: US$ 8 por mês para guardar uma cópia de banco que ninguém lembra. É a versão na nuvem da esteira ergométrica que virou cabide.

Este laço procura volumes soltos, Elastic IPs sem associação e NATs em todas as regiões da conta do profile ativo:

```
for regiao in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output text);
do
  echo "== $regiao"
  aws ec2 describe-volumes --region "$regiao" \
    --filters Name=status,Values=available \
    --query 'Volumes[].[VolumeId,Size,CreateTime]' --output text
  aws ec2 describe-addresses --region "$regiao" \
    --query 'Addresses[?AssociationId==null].PublicIp' --output text
  aws ec2 describe-nat-gateways --region "$regiao" \
    --filter Name=state,Values=available \
    --query 'NatGateways[].[NatGatewayId,VpcId,CreateTime]' --output text
done
```

O `describe-nat-gateways` usa `--filter`, no singular, uma das esquisitices da CLI. A CLI da AWS tem o charme de casa antiga: cada cômodo foi construído numa época, por um pedreiro diferente. Nas contas da OU `Cargas`, a SCP de regiões do capítulo 23 faz as regiões fora de `us-east-1` e `sa-east-1` responderem com erro de autorização. Ótimo sinal: ninguém consegue criar nada lá. Na `rango-estudos`, que é a conta de gerenciamento e não sofre SCP, o laço varre tudo.

E é na `rango-estudos` que aparece o resto. Cada `cdk destroy` dos capítulos 15 a 22 deixou um snapshot final do banco, e eles continuam lá:

```
aws rds describe-db-snapshots --profile rango-admin --snapshot-type manual \
  --query 'DBSnapshots[].[DBSnapshotIdentifier,AllocatedStorage,SnapshotCreateTime]' \
  --output text
```

Com o banco em `rango-producao` e protegido pelo AWS Backup, esses snapshots viraram arqueologia. O Indiana Jones diria que eles pertencem a um museu; a fatura diz que pertencem à lixeira. Eles saem com `aws rds delete-db-snapshot`, depois de conferidos os nomes. A hosted zone antiga de `rango.com.br` também cobra a taxa mensal; ela sai depois de confirmado que o Registro.br aponta para a zona nova.

Pagar menos pelo que fica ligado

Depois da limpeza, sobra a parte do gasto que tem motivo para existir. A AWS oferece dois descontos para ela, com contratos opostos: você promete ficar, ou aceita ser despejado. Um é casamento; o outro é aluguel por temporada com um proprietário temperamental.

Savings Plans: desconto em troca de compromisso

Um **Savings Plan** é um compromisso de gastar um valor fixo por hora, em dólares, durante 1 ou 3 anos. Em troca, o uso coberto por esse valor sai com desconto. Existem quatro tipos: Compute, EC2 Instance, SageMaker AI e Database. Escolher entre eles lembra plano de celular: todos parecem iguais até a letra miúda.

O **Compute Savings Plan** dá até 66% de desconto e vale para EC2, Fargate e Lambda, em qualquer família, região ou sistema operacional. O **EC2 Instance Savings Plan** chega a 72%, preso a uma família numa região. O **Database Savings Plan** cobre Aurora, RDS, DynamoDB, ElastiCache e outros bancos gerenciados. Os percentuais variam por serviço, prazo e quanto você paga adiantado.

O Cost Explorer recomenda um valor por hora a partir do histórico. A regra é comprometer o piso, nunca o pico: o compromisso é cobrado mesmo que você desligue tudo amanhã, e não pode ser cancelado. Lembra contrato de aluguel de um ano, só que sem cláusula de saída e sem proprietário para comover com uma boa história.

No Rangô, o piso de computação são as 2 tasks da API em produção: uns US$ 29 por mês, perto de US$ 0,04 por hora. Um plano de US$ 0,03 por hora economiza alguns dólares por mês. Para o Rangô de hoje, a limpeza das seções anteriores rende dez vezes mais; o Savings Plan fica para quando o piso crescer. Comprar desconto antes de arrumar a casa é como entrar no atacarejo para economizar e sair com seis quilos de páprica.

Spot: capacidade de sobra, com aviso prévio de 2 minutos

A AWS tem capacidade ociosa em todas as regiões e a vende com desconto de até 90% em relação ao preço normal, o *On-Demand*. É a **Spot**. A condição é que ela pode pedir a máquina de volta quando precisar, com um aviso de interrupção 2 minutos antes. Dois minutos é o tempo de salvar o trabalho e sair da sala com alguma dignidade. Às vezes chega antes uma *rebalance recommendation*, avisando que o risco de interrupção subiu. O EventBridge recebe os dois avisos, e o Auto Scaling Group tem o Capacity Rebalancing para trocar a instância antes da hora.

Spot combina com carga sem estado e tolerante a falha, como a API ficou desde o capítulo 4. No ECS, existe o **Fargate Spot**, que aceita tasks ARM desde setembro de 2024. Na interrupção, a task recebe `SIGTERM` com 2 minutos de prazo, e o `CMD` em forma de lista do capítulo 3 garante que o Uvicorn termine as requisições em andamento.

Homologação é a candidata óbvia. No cluster, os capacity providers do Fargate passam a ficar ativos:

```
        cluster = ecs.Cluster(
            self,
            "Cluster",
            vpc=vpc,
            cluster_name=f"rango-{ambiente}",
            enable_fargate_capacity_providers=True,
        )
```

E no `ecs.FargateService` da API, a estratégia de capacidade por ambiente:

```
            capacity_provider_strategies=None if producao else [
                ecs.CapacityProviderStrategy(capacity_provider="FARGATE_SPOT", weight=1)
            ],
```

Trocar o modo de lançamento de um serviço existente pode fazer o CloudFormation recriá-lo. Em homologação, isso custa alguns minutos fora do ar numa hora combinada. Em produção, a API fica no Fargate comum: o pico do Rangô dura poucas horas por semana, e a economia de colocar as tasks extras em Spot não paga o risco de recriar o serviço que recebe pedidos. Ninguém quer explicar para a Cantina da Nona que o sistema caiu num sábado à noite para economizar uns trocados.

Desligar o que ninguém usa à noite

O pilar de custo do Well-Architected Framework, que aparece na próxima seção, traz um exemplo que serve direto para homologação: um ambiente de desenvolvimento e teste ligado só no horário comercial usa 40 das 168 horas da semana e economiza 75%. A homologação ganha horário comercial, um luxo que muita gente de TI ainda está negociando.

A escala da API em homologação vira zero à noite e nos fins de semana, com ações agendadas no mesmo `escala` do capítulo 18. Nos imports, entram `aws_applicationautoscaling as appscaling` e `TimeZone`:

```
        if not producao:
            for nome, hora, minimo, maximo in (("Noite", "20", 0, 0), ("Manha", "8", 1,
2)):
                escala.scale_on_schedule(
                    nome,
                    schedule=appscaling.Schedule.cron(hour=hora, minute="0",
week_day="MON-FRI"),
                    min_capacity=minimo,
                    max_capacity=maximo,
                    time_zone=TimeZone.AMERICA_SAO_PAULO,
                )
```

A ação "Noite" roda de segunda a sexta às 20h e zera as tasks. A "Manha" volta para 1 a 2 tasks às 8h dos dias úteis. Como nenhuma ação roda no sábado e no domingo, o zero de sexta à noite vale até segunda de manhã. O fuso explícito evita o erro de UTC que o capítulo 19 já mostrou.

O ALB de homologação continua cobrando, porque load balancer não tem pausa, nem para o cafezinho. O banco até pode ser parado, mas o RDS liga sozinho de novo depois de 7 dias, e no `db.t4g.micro` a economia não compensa o trabalho.

O Rangô contra o Well-Architected

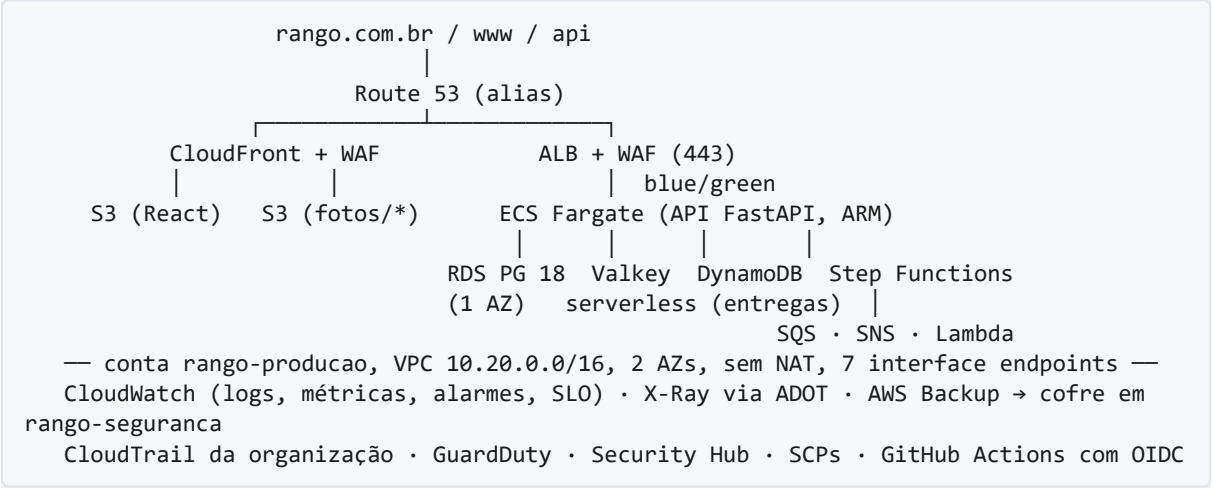
O **AWS Well-Architected Framework** é o conjunto de perguntas que a AWS usa para revisar arquiteturas, organizado em seis pilares: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization e Sustainability. A **AWS Well-Architected Tool**, no console, faz as perguntas uma a uma e guarda as respostas como uma revisão da carga de trabalho. Ela não analisa nada sozinha: quem responde é você, e o valor está nas perguntas que ninguém tinha feito. Funciona como o questionário do médico: dá para mentir, mas aí quem sai enganado é você.

O pilar de custo tem cinco princípios de design, e este capítulo aplicou quatro sem dar nome, como o Monsieur Jourdain de Molière, que falava em prosa sem saber. *Implement cloud financial management*: budgets por ambiente e alguém que lê os avisos. *Adopt a consumption model*: a homologação desligada à noite. *Measure overall efficiency*: o custo por pedido. *Analyze and attribute expenditure*: contas e tags. O quinto, *stop spending money on undifferentiated heavy lifting*, o livro aplica desde o capítulo 12, quando o RDS assumiu backup, patch e failover do Postgres.

Em dezembro, o custo por pedido foi de US$ 687,18 por 6.812 pedidos: perto de US$ 0,10. Com as mudanças deste capítulo, a estimativa para fevereiro fica perto de US$ 430: uns US$ 310 de produção, uns US$ 105 de homologação e o resto nas contas de segurança e gerenciamento. O custo por pedido cai para uns US$ 0,06. A margem segue apertada com uma taxa de R$ 0,50, e essa é a conversa que o Bruno precisa ter com os sócios, agora sabendo quanto custa cada parte. Reunião com números na mesa, pelo menos, costuma terminar no horário.

A revisão final

Este é o Rangô que saiu do `localhost:8000` do capítulo 1. Quem diria que aqueles dois restaurantes guardados em memória iam dar nisto:



E esta é a revisão honesta, pilar por pilar:

Pilar	O que o Rangô tem	O que falta
Operational Excellence	Tudo em CDK, pipeline com OIDC e aprovação, blue/green com rollback por alarme, postmortems sem culpa	Runbooks escritos e um ensaio de incidente por trimestre
Security	Identity Center sem chaves longas, contas separadas, SCPs, segredos rotacionados com KMS, WAF, GuardDuty, trilha imutável	Rotas do entregador ainda sem autenticação (capítulo 20); grupos gerenciados do WAF ainda em `Count`
Reliability	Duas AZs na rede e na API, PITR, backups em outra conta, filas com DLQ e alarme	Banco em uma AZ só; esgotamento de conexões do Postgres sem correção definitiva (capítulo 22); restore testado à mão, não automático

Pilar	O que o Rangô tem	O que falta
Performance Efficiency	CloudFront na borda, Graviton, Valkey como cache, DynamoDB para o rastreamento	Teste de carga com k6 só antes de eventos, nunca no pipeline
Cost Optimization	Contas e tags como dimensão de custo, budgets por ambiente, anomalias, homologação dormindo	Revisar Savings Plans quando o piso de computação crescer
Sustainability	Graviton, fotos menores, escala a zero fora do horário	Medir com a ferramenta de pegada de carbono do console de Billing

Os buracos de confiabilidade são os próximos itens. O Multi-AZ dobra o custo da instância do banco, uma decisão que pede o número de pedidos perdidos por hora de queda. O RDS Proxy, citado no capítulo 19, resolve as conexões e também cobra por hora. As duas mudanças cabem no orçamento que a limpeza abriu. É raro uma faxina pagar a reforma da casa, mas desta vez pagou.

Para onde ir daqui

Você começou este livro sabendo construir a aplicação. Agora conhece o caminho de uma requisição desde o DNS até a linha da tabela, e quanto cada passo custa. Algumas direções para continuar:

1. **Refaça o Rangô sozinho**, sem o livro aberto, numa conta de estudos. Onde você travar está o que ainda falta aprender. Colar do livro não vale; colar do Stack Overflow, com moderação, é tradição da profissão.
2. **Leia postmortems.** Os Post-Event Summaries da AWS e a Amazon Builders' Library contam como quem opera a AWS lida com timeouts, retries e deploys.
3. **Faça uma revisão do Well-Architected** num sistema do seu trabalho.
4. **Aprenda Terraform ou OpenTofu**, muito usados fora do mundo só-AWS, e **Kubernetes** se o seu time precisar; o capítulo 17 deu os critérios.

E leia a fatura todo mês. Ela é o resumo mais honesto da arquitetura que você tem, mais honesto até que o diagrama do wiki, atualizado pela última vez na primeira sprint.

Desligando tudo

Se o Rangô foi um projeto de estudo e você quer parar de pagar, a ordem é:

1. Desabilitar os workflows do GitHub (`gh workflow disable`).

2. `cdk destroy` de `RangoProducao` e `RangoHomologacao`, cada uma na sua conta, e depois os snapshots finais que elas deixarem.
3. `cdk destroy` de `RangoPipeline` e `RangoRegistro`; o repositório do ECR tem `RETAIN` e sai com `aws ecr delete-repository --force`.
4. Buckets restantes, hosted zones e cofres do AWS Backup. Um cofre com Vault Lock em modo compliance só some quando os pontos de recuperação vencerem, como o capítulo 23 escolheu. O cofre cumpre a palavra até contra você.
5. GuardDuty, Security Hub e Config em cada região, o laço de recursos esquecidos em cada conta e, por fim, o fechamento das contas membro.
6. O budget da `rango-estudos` fica até a última fatura chegar zerada.

Armadilhas e casos reais

> **Armadilha:** O Free plan das contas criadas desde 15 de julho de 2025 expira 6 meses depois do cadastro ou quando os créditos acabam. Sem a mudança para o Paid plan, a conta passa por 90 dias de carência e depois é fechada, com os recursos apagados.
>
> **Armadilha:** IPv4 público cobra US$ 0,005 por hora por endereço, em uso ou ocioso: uns US$ 3,65 por mês cada. Tasks em subnet pública com IP público pagam um cada, e com dezenas delas a linha cresce sem ninguém perceber.
>
> **Armadilha:** Testes gratuitos que terminam no meio do mês. GuardDuty, Security Hub e outros serviços têm 30 dias grátis por conta e região. Ligados em várias contas e regiões no mesmo dia, todos começam a cobrar juntos, e a fatura do mês seguinte é a primeira a mostrar o custo real.
>
> **Armadilha:** Cost Explorer pela API em laço. Um script que consulta a cada minuto para alimentar um painel cria uma linha nova na fatura que deveria vigiar.

Resumo

- A AWS cobra tempo ligado, volume guardado e movimento. Em projetos pequenos, o tempo ligado domina, mesmo sem tráfego.
- O Cost Explorer abre a fatura por conta, serviço, tipo de uso e tag; console gratuito, API cobrada, dados a cada 24 horas.
- Contas são a primeira dimensão de custo. Tags de custo precisam de ativação e valem dali em diante.

- Budgets por previsão avisam na primeira semana; o Cost Anomaly Detection pega o dia fora do padrão. Nenhum dos dois bloqueia gasto.
- Vilões clássicos: NAT Gateway, interface endpoints multiplicados por AZ, saída para a internet e recursos esquecidos.
- Savings Plans trocam compromisso por desconto; compre pelo piso. Spot e Fargate Spot dão até 90% para carga sem estado.
- O Well-Architected revisa em seis pilares. O custo por pedido do Rangô cai de uns US$ 0,10 para uns US$ 0,06.

O fim do livro, o começo da operação

O Rangô começou num notebook, com uma sócia em Recife tentando abrir um link que apontava para `localhost`. Hoje ele tem domínio, certificado, rede própria, banco com recuperação até o minuto, deploy sem ninguém digitar comando, alarme que acorda a pessoa certa, trilha de auditoria e uma fatura que alguém sabe ler. No capítulo 1, este livro prometeu uma fatura que não assusta ninguém e confessou que nem o autor acreditava nisso. Agora dá para acreditar: ela continua longe de pequena, mas cada linha tem dono e motivo, e fatura que se entende assusta bem menos.

Nada disso está terminado, e a tabela de pilares tem uma coluna de "o que falta" porque toda arquitetura em produção tem uma. O que mudou foram as perguntas: de "como alguém acessa isso?" para "quanto custa uma hora de banco fora do ar?". A segunda é a pergunta de quem cuida de um sistema em produção, e a partir daqui ela é sua.

Referências

- AWS Cost Explorer e AWS Cost Management (previsões, recursos): https://aws.amazon.com/aws-cost-management/
- AWS Budgets e Cost Explorer, comparação (console gratuito, API cobrada): https://aws.amazon.com/compare/budgets-and-cost-explorer/
- Cotas e limites do AWS Cost Management (Budgets, Cost Anomaly Detection): https://docs.aws.amazon.com/cost-management/latest/userguide/management-limits.html
- Cost allocation tags: https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/cost-alloc-tags.html
- Backfill de cost allocation tags: https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/cost-allocation-

backfill.html

- Anúncio do backfill de tags (março de 2024): https://aws.amazon.com/about-aws/whats-new/2024/03/aws-cost-allocation-tags-retroactive-application/
- Preços da Amazon VPC (NAT Gateway, IPv4 público, endpoints, transferência): https://aws.amazon.com/vpc/pricing/
- Preços do Amazon CloudFront: https://aws.amazon.com/cloudfront/pricing/
- Preços do AWS Fargate: https://aws.amazon.com/fargate/pricing/
- AWS Pricing Calculator: https://calculator.aws/
- Savings Plans, perguntas frequentes: https://aws.amazon.com/savingsplans/faqs
- Tipos de Savings Plans: https://docs.aws.amazon.com/savingsplans/latest/userguide/plan-types.html
- Boas práticas de Spot no EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html
- Fargate Spot com Graviton (setembro de 2024): https://aws.amazon.com/about-aws/whats-new/2024/09/amazon-ecs-graviton-based-spot-compute-fargate/
- Well-Architected, pilar de otimização de custos: https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html
- Princípios de design do pilar de custos: https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/design-principles.html
- AWS Well-Architected Tool: https://aws.amazon.com/well-architected-tool/
- Free Tier com créditos, Free plan e Paid plan (julho de 2025): https://aws.amazon.com/about-aws/whats-new/2025/07/aws-free-tier-credits-month-free-plan/
- Caso do bucket S3 vazio e mudança na cobrança de requisições não autorizadas (InfoQ, maio de 2024): https://www.infoq.com/news/2024/05/aws-empty-s3-bucket-billing/
- Amazon Builders' Library: https://aws.amazon.com/builders-library/
- AWS Post-Event Summaries: https://aws.amazon.com/premiumsupport/technology/pes/